

The Hathor-Core Master-Book

A junior developer's deep-dive into the `hathor-core` full node — the concepts, the stack, and every subsystem, walked end to end.

SUBJECT	hathor-core · complete edition
CHAPTER	Complete · 44 chapters + 3 appendices
BRANCH	study/hathor-core-studies
EDITION	2026 · v1

Python · Twisted · RocksDB · DAG · UTXO · Consensus · P2P · Sync-v2 · Nano-contracts ·
Pydantic · Poetry · Docker

Table of Contents

Chapter 0 — Orientation	26
0.1 What Hathor is	26
What hathor-core is, as a program	27
0.2 What a full node does	27
0.3 The life of a node	28
Act I — Startup	28
Act II — Steady state	30
0.4 The module map	31
Domain model — the data and its rules	32
Storage and lookup — where the ledger lives	33
Networking and clients — talking to the world	33
Mining and monetary — minting and securing coins	33
Higher services — built on the base ledger	34
Infrastructure — making it all run	34
The command-line surface	34
0.5 How to read this book	34
Three reading aids	35
The order, and how to navigate	35
Recap	36
Chapter 1 — Object-Oriented Programming	38
1.1 The problem: programs that grow	38
1.2 Three ways to organize a program	39
1.3 Classes and objects	41
1.4 Encapsulation — the first pillar	42
1.5 Inheritance — the second pillar	43
1.6 Polymorphism — the third pillar	44
1.7 Abstraction — the fourth pillar	45
1.8 Composition over inheritance	46
1.9 The dunder protocol	47
1.10 Bridge — where this lands in hathor-core	48

Recap	49
Chapter 2 — Callbacks & Asynchronous Programming	52
2.1 The problem: waiting is everywhere	52
2.2 Three ways to do many things at once	53
2.3 Callbacks — "call me back when it's ready"	54
2.4 The event loop — the engine that runs the callbacks	56
2.5 Callback hell, and the rescue: futures	57
2.6 async/await — asynchronous code that reads top-to-bottom	58
2.7 The one rule, and the escape hatch for CPU work	59
2.8 Bridge — asynchrony in hathor-core	59
Recap	60
Chapter 3 — Design Patterns	63
3.1 What a design pattern is (and isn't)	63
3.2 Factory — centralize "which class to create"	64
3.3 Builder — construct a complex object step by step	65
3.4 Singleton — one instance, shared everywhere (handle with care)	65
3.5 Adapter — make an incompatible interface fit	66
3.6 Facade — one simple front over a complex subsystem	67
3.7 Proxy — a stand-in that controls access	67
3.8 Observer / Publish-Subscribe — announce events to interested parties	68
3.9 Bridge — patterns in hathor-core	69
Recap	70
Chapter 4 — Wrappers, Decorators, Shims & Dispatch	73
4.1 The unifying idea: behavior around behavior	73
4.2 Wrappers and closures	74
4.3 Decorators — the @ is just sugar	75
4.4 Shims and fallbacks	76
4.5 Dispatch — routing a call to the right code	77
4.6 Bridge — these in hathor-core	78

Recap	79
Chapter 5 — Type Hints & Static Typing	82
5.1 Dynamic typing and its cost at scale	82
5.2 Type hints: annotations that document and check	83
5.3 The static type checker (mypy)	84
5.4 The vocabulary of types	84
5.5 Why a large codebase pays for typing	86
5.6 Bridge — typing in hathor-core	86
Recap	87
Chapter 6 — What a Blockchain Is	90
6.1 The problem: spending the same coin twice	90
6.2 The easy answer, and why it's rejected	91
6.3 The distributed ledger: everyone holds a copy	92
6.4 The primitive: cryptographic hash functions	92
6.5 From records to a tamper-evident chain	93
6.6 Who appends? Sybil attacks and proof-of-work	94
6.7 Finality is probabilistic	95
6.8 Bridge — how Hathor answers these questions	96
Recap	97
Chapter 7 — The UTXO Model	100
7.1 The intuitive model — and why it isn't the only one	100
7.2 The UTXO model: money as discrete coins	101
7.3 How a transaction moves money: consume and create	102
7.4 Why UTXO suits a distributed ledger	103
7.5 Ownership without an owner field: scripts	104
7.6 Hathor's spelling: inputs, outputs, tokens, authorities	104
Recap	105
Chapter 8 — From Chain to DAG: The Vertex Model	107
8.1 Just enough graph theory	107

8.2 The limit of a single chain	108
8.3 The DAG of vertices	109
8.4 The crux: parents versus inputs	109
8.5 What blocks are still for, tips, and confirmation	110
8.6 Bridge — the vertex model in code	111
Recap	112
Chapter 9 — Proof-of-Work, Weight & Accumulated Weight	115
9.1 Proof-of-work, refreshed and made concrete	115
9.2 A two-minute primer on logarithms	116
9.3 Weight: work as a single number	117
9.4 Accumulated weight and score: summing work across the DAG	118
9.5 Keeping the rate steady: the difficulty adjustment algorithm	118
9.6 The heaviest-DAG rule	119
Recap	120
Chapter 10 — Conflicts, Voiding, Finality & Checkpoints	122
10.1 Two kinds of "bad" transaction	122
10.2 Voiding: marking, not deleting	123
10.3 Deciding a conflict, and propagation	124
10.4 Reorgs in a DAG	125
10.5 Finality: probabilistic, then anchored	125
10.6 Bridge — voiding and consensus in code	126
Recap	127
Chapter 11 — Python the Platform	129
11.1 "Python" is three things	130
11.2 The interpreter: what python file.py actually does	130
11.3 Modules: a file is a unit of code	131
11.4 Packages: a directory is a unit too	131
11.5 How import actually works	132
11.6 Why Python for a node?	133
11.7 Bridge — the platform under the codebase	134

Recap	135
Chapter 12 — Virtual Environments	137
12.1 The problem: one shared pile of packages	137
12.2 What a .venv actually is	138
12.3 What "activation" really does	139
12.4 Installing into an environment, and reproducibility	140
12.5 Bridge — environments around hathor-core	141
Recap	141
Chapter 13 — Poetry: Dependency Management	143
13.1 The problem: a list of packages is not enough	143
13.2 Two files: declare in pyproject.toml, pin in poetry.lock	144
13.3 Reading hathor-core's pyproject.toml	145
13.4 The build backend: pyproject.toml is also packaging config	146
13.5 Why Poetry, and not the alternatives?	146
13.6 Bridge — Poetry across the project	147
Recap	148
Chapter 14 — Make: The Task Runner	150
14.1 What Make was built for	150
14.2 Anatomy of a Makefile	151
14.3 .PHONY: targets that aren't files	151
14.4 What the real Makefile does	152
14.5 Why Make, and not the alternatives?	152
14.6 Bridge — Make across the project	153
Recap	153
Chapter 15 — Docker: Containerizing the Node	155
15.1 The problem: "works on my machine," one level deeper	155
15.2 Containers, images, and how they differ from VMs	156
15.3 The Dockerfile: a recipe for an image	157
15.4 Reading hathor-core's multi-stage Dockerfile	157

15.5 Why containerize a node, and why Docker?	158
15.6 Bridge — Docker across the project	159
Recap	159
Chapter 16 — Twisted: Reactor, Deferreds, Protocols	161
16.1 What Twisted is	162
16.2 The problem it solves	163
16.3 The core model and its components	163
16.4 The reactor — the one loop	164
How hathor-core obtains and runs the reactor	165
16.5 The Deferred — Twisted's future	166
A tiny generic Deferred	167
maybeDeferred — "synchronous or asynchronous, I don't care"	167
inlineCallbacks — straight-line asynchronous code	168
Deferreds in hathor-core	168
16.6 Protocols, factories, endpoints, timers	169
16.6.1 The connection lifecycle, in callbacks	169
16.6.2 A tiny generic Protocol and Factory	170
16.6.3 Hathor's Protocol and Factory	171
16.6.4 Endpoints, timers, and the thread-pool escape hatch	172
16.7 Why Twisted and not the alternatives	174
Twisted vs. asyncio	175
The typed-reactor abstraction (why the wrapper exists at all)	176
And the thread question	177
Recap	178
Chapter 17 — structlog: Structured Logging	180
17.1 The problem: print doesn't scale to a daemon	180
17.2 Structured logging: events, not sentences	181
17.3 The structlog model: loggers and the processor pipeline	182
17.4 Why structlog, and not the alternatives?	182
17.5 Bridge — logging across the project	183
Recap	184
Chapter 18 — Pydantic: Validation at the Boundary	186

18.1 The problem: the inside is typed, the outside isn't	187
18.2 The complement to mypy: same syntax, opposite timing	187
18.3 The Pydantic model	188
18.4 Configuring models: encoding invariants	188
18.5 How hathor-core uses it, and why Pydantic	189
18.6 Bridge — Pydantic across the project	190
Recap	190
Chapter 19 — configargparse: CLI as Configuration	192
19.1 argparse: the standard starting point	192
19.2 The problem: one option, three sources	193
19.3 configargparse: three sources, one declaration	193
19.4 How hathor-core uses it	194
19.5 Why configargparse, and not the alternatives?	194
19.6 Bridge — configuration input across the project	195
Recap	195
Chapter 20 — The Quality Toolchain	197
20.1 Four jobs, four tools	197
20.2 mypy — the type checker	198
20.3 flake8 — the linter	198
20.4 isort — the formatter	198
20.5 pytest — the test runner	199
20.6 The gate: making it all mandatory	199
20.7 Why invest in all this?	200
20.8 Bridge — the toolchain across the project	201
Recap	201
Chapter 21 — The Command-Line Surface	203
21.1 Localization — where hathor_cli/ sits	203
21.2 What it does and why it exists	204
How hathor-cli reaches Python — the console-script entry point	205
21.3 The concepts it rests on	205

21.4 The code, walked: dispatch	206
A toy dispatcher first	206
The real dispatch table: CliManager	207
Dispatch and the logging hooks: execute_from_command_line	208
The entry point: main()	209
21.5 The run_node boot path	210
Step 1 — parse arguments	210
Step 2 — select the network and load settings	211
Step 3 — assemble and start: prepare()	211
Step 4 — run the reactor	213
A worked illustration of reuse: quick_test	213
21.6 How it plugs into the lifecycle	213
Recap	214
Chapter 22 — Settings & Configuration	217
22.1 Localization — where hathor/conf/ sits	217
22.2 What it does and why it exists	218
Why one read-only object, and not scattered constants	219
22.3 The concepts it rests on	220
22.4 The code, walked	221
The shape of the package	221
A toy first: a read-only config every function reads	221
Selecting the profile from the environment	222
Loading once, caching forever — the singleton	223
The HathorSettings type — validate at load	224
A representative profile excerpt	226
22.5 How it plugs into the lifecycle	227
Recap	228
Chapter 23 — The Reactor Abstraction	231
23.1 Localization	231
23.2 What it does and why it exists	232
23.3 The concepts it rests on	233
23.4 The code, walked	234
23.4.1 A tiny illustration first	234
23.4.2 ReactorProtocol — three slices joined into one type	234

23.4.3 get_global_reactor — the accessor	236
23.4.4 initialize_global_reactor — the installer, the verify, and the asyncio branch	236
23.4.5 The public surface	238
23.5 How it plugs into the lifecycle	239
The asyncio option, in current reality	239
Recap	240
Chapter 24 — The Composition Root: Builder & CliBuilder	242
24.1 Localization	242
24.2 What it does, and why it exists	243
Why centralize the wiring?	244
Why it makes the node testable	244
24.3 The concepts it rests on	244
24.4 The code, walked	245
24.4.1 A generic builder, first	245
24.4.2 The real Builder — tests and simulator	246
24.4.3 The real CliBuilder — production	249
24.4.4 Two composition roots, one job — and the debt	252
24.4.5 ResourcesBuilder, briefly	252
24.5 How it plugs into the lifecycle	253
Recap	254
Footnotes	255
Chapter 25 — The Vertex Model in Code	256
25.1 Localization	257
25.2 What it does and why it exists	257
25.3 The concepts it rests on	258
25.4 A hand-traced example first	259
25.5 The code, walked	260
25.5.1 GenericVertex: the abstract generic base	260
25.5.2 The shared fields, and what each one models	261
25.5.3 TxInput and TxOutput: the UTXO model, literally	263
25.5.4 Parents versus inputs, in the fields	265
25.5.5 The subclass hierarchy	265
25.5.6 Hashing and serialization (pointer forward)	267
25.6 The metadata split: static versus mutable	267

25.7 Validation state and storage scope	270
25.8 How it plugs into the lifecycle	271
Recap	271
Chapter 26 — The Wire Format: Serialization	274
26.1 The problem, in general terms	275
26.2 The concepts it rests on	276
Bytes, and why everything reduces to them	276
Endianness — the order of a number's bytes	276
Fixed-width integers — spending exactly N bytes	276
Variable-length integers (varint / LEB128) — paying for what you use	277
Length-prefixing — framing variable-length data	277
26.3 Localization	278
26.4 The general framework, walked	280
26.4.1 A toy encoding, by hand	280
26.4.2 Serializer and Deserializer — the two cursors	280
26.4.3 The primitive encoders	282
26.4.4 The compound encoders	283
26.4.5 The adapters	284
26.5 The vertex format, walked	284
26.5.1 The two structs: funds and graph	284
26.5.2 Assembling the whole vertex	286
26.5.3 TxInput and TxOutput byte layouts	287
26.5.4 VertexParser — bytes back into the right subclass	288
26.5.5 The two worlds, reconciled	289
26.6 Why bespoke binary, and not JSON or Protobuf	290
26.7 How it plugs into the lifecycle	291
Recap	292
Chapter 27 — Persistence: Storage & RocksDB	294
27.1 Localization	294
27.2 What it does and why it exists	296
27.3 The concepts it rests on	296
The new idea: an embedded key-value store	297
27.4 The code, walked — a tiny generic store first	299
27.5 The byte layer: RocksDBStorage	299

27.6 The vertex layer: TransactionStorage and its backends	301
Saving a vertex	302
Reading a vertex back	303
Key design and column families, summarized	304
The narrow read interface: VertexStorageProtocol	305
27.7 Walking the DAG, and the allow-scope	305
Traversal helpers	305
Allow-scope: gating by validation state	306
27.8 Crash safety — the flag in the database	307
27.9 The trade-off — why RocksDB and not the alternatives	308
27.10 How it plugs into the lifecycle	309
Recap	310
Chapter 28 — Indexes: Fast Lookups over the Ledger	313
28.1 Localization	314
28.2 What it does and why it exists	314
The access-pattern problem	315
The classic answer: an index	315
The property that makes indexes safe: they are derived	316
28.3 The code, walked	317
28.3.1 A toy first: index a list of records by a field	317
28.3.2 BaseIndex: the shared interface	318
28.3.3 Scope: declaring what a vertex an index wants	319
28.3.4 IndexesManager: one object owns them all	320
28.3.5 Two backends, the same manager	321
28.3.6 The UTXO index — the payoff of Chapter 7	322
28.3.7 The other indexes, in brief	324
28.3.8 Critical vs. non-critical, and why the split	326
28.4 A worked trace: one transaction moves through the indexes	327
28.5 How indexes are rebuilt — and why they can be	327
28.6 How it plugs into the lifecycle	329
Recap	329
Chapter 29 — The Coordinator: HathorManager	332
Localization	333
What it does and why it exists	333

The concepts it rests on	334
A tiny toy first	335
The code, walked	337
The held collaborators	337
The start() sequence	338
_initialize_components: loading the ledger into memory	341
stop: teardown in reverse	343
on_new_tx: the door for incoming vertices	344
The periodic job	344
How it plugs into the lifecycle	345
Recap	346
Chapter 30 — Events & Pub-Sub	348
30.1 Localization	348
30.2 What it does and why it exists	349
30.3 The concepts it rests on	351
30.4 The code, walked — Layer 1: the in-process bus	352
A tiny pub-sub toy first	352
The catalogue: HathorEvents	352
The argument bag: EventArguments	353
PubSubManager: subscribe and publish	353
30.5 The code, walked — Layer 2: the durable event queue	355
It is, itself, a subscriber	355
Translating a bus event into a durable record	356
The persisted record: BaseEvent	357
Receive → persist → broadcast	357
The storage: EventRocksDBStorage	358
Streaming to the world: the WebSocket layer	358
The replayable backlog at startup	359
30.6 How it plugs into the lifecycle	359
Recap	361
Chapter 31 — Verification: Checking the Rules	363
31.1 Localization	364
31.2 What it does and why it exists	365
31.3 The concepts it rests on	365

31.4 The code, walked — the orchestrator and the verifier family	367
31.4.1 One service, many verifiers	367
31.4.2 Dispatch by vertex type	367
31.5 The basic-vs-full split	368
31.6 Proof-of-work: hash below target	370
31.7 Value conservation and token rules	370
31.8 Script evaluation — the heart of signature checking	371
31.8.1 A stack machine, from scratch	372
31.8.2 The locking/unlocking split	372
31.8.3 The real script: P2PKH	373
31.8.4 The opcode dispatch table	374
31.9 Bounding the work: sigops, sizes, counts	375
31.10 Structural rules: parents, timestamps, no double-spend within a tx	375
31.11 Block-only rules	376
31.12 How it plugs into the lifecycle	377
Recap	378
Chapter 32 — Consensus: Which History Wins	381
32.1 Localization	381
32.2 What it does and why it exists	382
Two engines, one interface	383
32.3 The concepts it rests on	384
32.4 The code, walked: the orchestrator	385
The context object	385
32.5 Transaction consensus: resolving conflicts	386
Step 1 — detecting the conflict	387
Step 2 — computing voided_by from the surroundings	387
Step 3 — picking the winner: check_conflicts	388
Step 4 — propagating the void: the BFS	390
A hand-traced example	391
32.6 Block consensus: chains, score, and reorgs	391
Score: accumulated work of a whole sub-DAG	392
Selecting the best chain	392
Reorgs	393

The asymmetry: why block voiding differs from transaction voiding	394
first_block: where a transaction sits in block-time	395
32.7 The other engine: Proof-of-Authority	395
Turns and weight	396
Signatures instead of nonces	397
Producing blocks	397
32.8 How it plugs into the lifecycle	397
Recap	399
Chapter 33 — Ingestion: The Vertex Handler	401
33.1 Localization	401
33.2 What it does, and why it exists	402
The single chokepoint	402
Why the order matters	403
33.3 The concepts it rests on	403
33.4 The code, walked	404
A generic orchestrator first	404
The three entry points	405
Step A — Verify (and the accept / reject / already-known decision)	406
Step B — Save, then run consensus	407
Step C — Re-validate, update indexes, announce	408
Step D — Failure means crash, on purpose	409
The pipeline, in one diagram	409
33.5 How it plugs into the lifecycle	410
Who calls it	410
The convergence point	411
A note on partial vertices	411
Recap	412
Chapter 34 — Peer-to-Peer Networking I: Connections & Protocol	414
34.1 Localization	415
34.2 What it does, and why it exists	416
34.3 The concepts it rests on	417
A state machine, from scratch	417
34.4 The code, walked	418

34.4.1 ConnectionsManager — owner of the peer population	418
34.4.2 HathorProtocol — one object per connection	420
34.4.3 The state machine: HELLO → PEER-ID → READY	422
34.4.4 A worked handshake, end to end	425
34.4.5 The factories — one protocol per connection	425
34.4.6 Peer identity: the four-layer peer	426
34.4.7 Endpoints: where, and optionally who	427
34.4.8 Peer storage: two address books	427
34.4.9 Peer discovery (brief)	427
34.4.10 Netfilter (brief)	428
34.5 How it plugs into the lifecycle	428
Recap	429
Chapter 35 — Peer-to-Peer Networking II: Sync-v2	431
35.1 Localization	431
35.2 What it does and why it exists	433
Why you cannot just ask for everything	433
The generic version of this problem	434
35.3 The concepts it rests on	435
35.4 The code, walked	436
35.4.1 Negotiating a sync version	436
35.4.2 From version to agent: the factory	437
35.4.3 The sync agent: NodeBlockSync	437
35.4.4 Are we even behind? The fast paths	438
35.4.5 Finding the common ancestor	439
35.4.6 Streaming the block backbone	440
35.4.7 Filling in the transaction DAG	441
35.4.8 The mempool sync	443
35.4.9 The message flow, end to end	443
35.5 The design rationale	445
Why backbone-first?	445
Why streaming, not request-per-vertex?	445
35.6 How it plugs into the lifecycle	446
Recap	447
Chapter 36 — The Streaming Surface: WebSockets	449
36.1 The problem: HTTP cannot push	449
Polling, and why it is wasteful	450

Full-duplex: both sides can talk at once	450
36.2 What a WebSocket actually is — the upgrade handshake	451
36.3 Localization	452
36.4 The concepts it rests on	453
36.5 Autobahn: a WebSocket implementation for Twisted	454
What it is and the problem it solves	454
How you use it — a tiny echo server	454
36.6 Surface 1 — the admin WebSocket, walked	455
The factory: one server, many clients	455
Subscribing to the event bus	456
Broadcast vs. targeted send	457
Rate limiting: protecting the client from a firehose	457
The metrics timer	457
The protocol: one client's state machine	457
JSON message framing, typed	458
The history streamer: pushing a large result safely	459
36.7 Surface 2 — the event-queue WebSocket	459
36.8 Why WebSockets, and why Autobahn	460
36.9 How it plugs into the lifecycle	461
Recap	462
Chapter 37 — Mining: Templates, CPU Miner & Stratum	465
37.1 Localization	465
37.2 What it does and why it exists	466
37.3 The concepts it rests on	467
37.4 The code, walked	468
37.4.1 The block template — what goes in, and why	468
37.4.2 How the node builds a template	470
37.4.3 The CPU mining service — solving the puzzle	471
37.4.4 Stratum — the standard miner-to-node protocol	472
37.4.5 Merged mining — Hathor as a free side-effect of Bitcoin	477
37.4.6 The solved block, back into the node	478
37.4.7 The job/share lifecycle, end to end	479
37.5 Why Stratum, and not naive polling	479
37.6 How it plugs into the lifecycle	480

Recap	481
Chapter 38 — Feature Activation: Bit Signalling	484
38.1 Localization	484
38.2 What it does and why it exists	485
The problem: you cannot push an update to a network you don't own	485
The shape of the answer: schedule it, and gate it on demonstrated support	486
38.3 The concepts it rests on	486
Two new concepts unique to this chapter	487
38.4 The code, walked	488
38.4.1 The Feature enum — the catalogue of upgrades	488
38.4.2 Criteria — one feature's schedule	488
38.4.3 The six states	490
38.4.4 The state machine	490
38.4.5 Counting the votes — feature_activation_bit_counts	492
38.4.6 FeatureService — the read side (the brain)	493
38.4.7 Gating behaviour — two real use-sites	494
38.4.8 BitSignalingService — the write side (this node's ballot)	495
38.4.9 A small worked example	496
38.5 How it plugs into the lifecycle	497
38.6 Recap	498
Chapter 39 — Nano-Contracts: The Smart-Contract Runtime	500
39.A — The Runtime	502
39.A.1 What a smart contract is	502
How Hathor differs from Ethereum	503
39.A.2 Localization	503
39.A.3 What a Blueprint is	505
The metaclass: turning annotations into storage	506
39.A.4 How a transaction calls a contract	507
NCActions — moving tokens into and out of a contract	508
The Context	509
39.A.5 The decorators: @public, @view, @fallback	509
39.A.6 The Runner and the syscall surface	510
Dispatching the call	511
The syscall surface: BlueprintEnvironment	512

39.A.7 Metered execution: why, and how	512
Why bound execution at all	513
How Hathor sets it up	513
The honest caveat (a correction to the surface story)	514
39.A.8 Worked trace: deploy and call a Counter	515
39.B — The State	517
39.B.1 Why a trie, and what a trie is	517
Layer one: a trie (prefix tree)	517
Layer two: Merkle hashing → a state root	518
39.B.2 The trie in code	518
39.B.3 Two levels of trie: contract state and block state	519
39.B.4 Typed fields: how self.count becomes bytes	520
Staging: the changes tracker	521
39.B.5 On-chain Blueprints: code stored on the ledger	522
39.B.6 Execution meets consensus	523
Sorting the calls	524
Running the block	524
Applying the effects, and voiding failures	525
39.B.7 How it all plugs into the lifecycle	526
Recap	526
Chapter 40 — Wallets, Keys & Crypto	530
40.1 The problem: proving ownership without a trusted referee	530
40.2 A primer on asymmetric cryptography and digital signatures	531
40.2.1 Two keys instead of one	532
40.2.2 What a digital signature is	532
40.2.3 ECDSA on secp256k1 — the concrete scheme, conceptually	533
40.3 Localization	533
40.4 What it does and why it exists	534
40.5 The concepts it rests on	535
40.6 The code, walked	536
40.6.1 The address pipeline: public key → address	536
40.6.2 KeyPair — one individual, encrypted key	538
40.6.3 Wallet — a file-backed bag of KeyPairs	539
40.6.4 HDWallet — one seed, a whole tree	539

40.6.5 Tracking UTXOs, selecting inputs, and signing — the shared BaseWallet	541
40.6.6 Multisig, briefly	544
40.6.7 The CLI seed-words generator	544
40.7 Why these libraries, not hand-rolled crypto	545
40.8 How it plugs into the node's lifecycle	545
Recap	546
Chapter 41 — Runtime Control: Sysctl	549
41.1 Localization	550
41.2 What it does, and why it exists	550
The problem: a daemon you cannot pause	550
The analogy: Linux sysctl	551
41.3 The concepts it rests on	552
41.4 The code, walked	552
41.4.1 A tiny toy first	552
41.4.2 The Sysctl base class — a node in the tree	553
41.4.3 Dotted-path addressing — walking down the tree	555
41.4.4 Get vs. set	555
41.4.5 The runner — from a text line to a call	556
41.4.6 The transport — a line-based socket protocol	557
41.4.7 Two real parameters, end to end	557
41.4.8 The per-subsystem nodes — a tour	559
41.5 How it plugs into the lifecycle	560
41.5.1 Assembly — the SysctlBuilder	560
41.5.2 Exposure — only if configured	561
41.5.3 The signal path — SIGUSR2	562
41.5.4 Putting it together	563
Recap	563
Chapter 42 — Observability: Metrics, Profiler & Health	566
42.1 Localization	566
42.2 What it does and why it exists	567
The four questions	568
42.3 The concepts it rests on	568
42.4 The code, walked — Metrics and Prometheus	569
42.4.1 What a metric is	569
42.4.2 Pull-based scraping — the Prometheus model	570

42.4.3 Two objects: Metrics and PrometheusMetricsExporter	571
42.4.4 The twist: file-based export, not an HTTP endpoint	572
42.4.5 Why Prometheus, and why pull	573
42.5 The code, walked — the CPU profiler	574
42.5.1 The reactor-stall problem	574
42.5.2 SimpleCUPProfiler — a sampling marker stack	574
42.5.3 Where the marks come from — the @profiler decorator and SiteProfiler	575
42.5.4 The /top API and hathor-cli top	576
42.6 The code, walked — the health-check endpoint	576
42.6.1 Liveness versus readiness	576
42.6.2 What "healthy" means for a Hathor node	577
42.6.3 The two resources	577
42.7 How it plugs into the lifecycle	578
Recap	579
Chapter 43 — Deterministic Testing: Simulator & DAG Builder	581
43.1 Localization	582
43.2 What it does, and why it exists	582
The problem: timing-dependent bugs	582
The cure: determinism	583
43.3 The concepts it rests on	584
43.4 The code, walked — the key trick first	585
Generic warm-up: advance time by hand	585
Hathor's spelling: HeapClock	586
43.5 The code, walked — the Simulator class	587
Building an in-memory node	587
Wiring two nodes together: FakeConnection	588
Running the simulation: stepping time	589
43.6 The code, walked — producing blocks and transactions without mining	590
43.7 The code, walked — writing a DAG in text	591
The DSL	591
Tokenize, then assemble	592
The result: DAGArtifacts	593
43.8 How it plugs into the lifecycle	593
Recap	594

Chapter 44 — The Life of the Node: Everything Working Together	597
Part 1 — The Life of a Connection	598
1.1 Boot and assembly	598
1.2 Finding a peer and shaking hands	598
1.3 Catching up: sync	599
Part 2 — The Life of a Transaction	600
2.1 The wallet builds the transaction	600
2.2 Signing: proving ownership	600
2.3 Proof-of-work: the anti-spam cost	600
2.4 Serialization: becoming bytes	601
2.5 Propagation: onto the network	601
2.6 Ingestion: the vertex handler	601
2.7 Verification: is it valid on its own?	602
2.8 Consensus: which history does it belong to?	602
2.9 Storage: writing it down	602
2.10 Indexes: making it findable	602
2.11 Announcement: telling the interested	603
2.12 The wallet sees its change	603
Part 3 — The Life of a Block	604
3.1 The template: assembling a candidate	604
3.2 Mining and submission	604
3.3 What consensus does differently for a block	604
3.4 Confirming Alice's payment	605
3.5 Two side-effects only blocks have	605
Part 4 — The Life of a Nano-Contract Call	606
4.1 A call arrives, but does not run	606
4.2 Execution at block consensus	606
4.3 State, and the cost of failure	606

Recap	607
Chapter 44 (Extended) — The Life of the Node: Everything Working Together	608
Part 1 — The Life of a Connection	610
1.1 Boot and assembly	610
1.2 Finding a peer	610
1.3 The handshake	611
1.4 Catching up, then relaying	611
Part 2 — The Life of a Transaction	613
2.1 The wallet builds the transaction	613
2.2 Signing — proving ownership	613
2.3 Proof-of-work — the anti-spam toll	614
2.4 Serialization — becoming bytes	614
2.5 Propagation — onto the network	615
2.6 Ingestion — the vertex handler	615
2.7 Verification — is it valid on its own?	615
2.8 Consensus — which history does it belong to?	616
2.9 Storage — writing it down	616
2.10 Indexes — making it findable	617
2.11 Announcement — telling the interested	617
2.12 The loop closes — the wallet sees its change	617
Part 3 — The Life of a Block	619
3.1 The template — a node-built candidate	619
3.2 Mining and submission — work done elsewhere	619
3.3 Consensus by score — and the possibility of a reorg	619
3.4 Confirming Alice's payment	620
3.5 Two machineries only blocks drive	620
Part 4 — The Life of a Nano-Contract Call	621
4.1 Accepted on arrival, but not run	621

4.2 Executed at block consensus	621
4.3 State, verifiability, and the cost of failure	622
Recap	622
Appendix A — Glossary & Jargon Index	623
A	623
B	624
C	626
D	629
E	632
F	632
G	634
H	635
I	636
J	637
K	637
L	638
M	639
N	640
O	641
P	641
R	645
S	646
T	649
U	651
V	651
W	652
X	652
Appendix B — The Annotated Dependency Manifest	653
B.1 The Python platform	653
B.2 Runtime core frameworks	654

B.3 Storage & serialization	655
B.4 Networking & APIs	655
B.5 Cryptography	656
B.6 Observability & logging	657
B.7 CLI, process & utilities	658
B.8 Development & quality toolchain	659
B.9 Build & packaging	660
Recap	661
Appendix C — Cheat Sheets	663
C.1 RocksDB column families	663
C.2 P2P protocol messages	664
C.3 CLI command groups	665
C.4 Sysctl parameter paths	666
C.5 Feature-activation state machine	667
C.6 P2P handshake states	668
C.7 Network constants	668
Recap	669

Chapter 0 — Orientation

What you'll learn in this chapter

- What Hathor is, and what the `hathor-core` software actually is as a program.
- What a full node is responsible for, in plain terms.
- The **life of a node**: the ordered story of what happens between typing `hathor-cli run_node` and a node that is fully synced, verifying transactions, and mining.
- The **module map**: how the codebase divides itself into packages, what each one is for, and where it sits.
- **How to read this book** — the three content tracks, and the footnote / recap / deep-dive convention used throughout.

This chapter is the map you keep folded in your pocket for the rest of the book. It does not explain how anything works in detail — every later chapter does that. Its job is to give you a mental model of the whole system so that when a later chapter says "the manager hands the vertex to the verification pipeline," you already know what the manager is, what a vertex is, and roughly where verification sits. Read it once now; come back to §0.3 and §0.4 whenever you feel lost.

There is almost no code here on purpose. Code starts in Chapter 1.

0.1 What Hathor is

Hathor is a **cryptocurrency network**: a system that lets people hold and transfer a digital token without any bank, company, or central server being in charge. Like Bitcoin, it is a distributed ledger¹ — a shared record of who owns what — that is maintained by many independent computers around the world, none of which has to trust any other.

What makes Hathor distinct from Bitcoin is the shape of that ledger. Bitcoin organizes its history as a single chain of blocks, one after another. Hathor uses a **DAG**² — a directed acyclic graph — in which individual transactions are themselves part of the data structure, not just bundled inside blocks. Blocks still exist (they are how new coins are minted and how the network agrees on an ordering), but ordinary transactions link directly to other transactions, forming a graph rather than a straight

line. We give the full treatment of why this design exists, and what it buys, in **Chapter 8**. For now, hold this one sentence: **in Hathor, both blocks and transactions are nodes in one graph, and the codebase calls any such node a "vertex."**³

On top of that base ledger, Hathor supports a few things worth naming now so the vocabulary isn't a surprise later:

- **The native token, HTR** — the built-in currency, created by mining.
- **Custom tokens** — users can create their own tokens that ride on the same ledger.
- **Nano-contracts** — Hathor's form of smart contracts⁴: small programs, stored on the ledger, that move tokens according to rules. These are the single largest subsystem in the codebase (Chapter 39).

What `hathor-core` is, as a program

`hathor-core` is the **reference implementation** of a Hathor full node, written in Python. "Reference implementation" means it is the canonical, official program that defines how a correct node behaves; if another team wrote a second Hathor node, they would check their behavior against this one.

Concretely, `hathor-core` is a **long-running server process** — a daemon⁵. You start it from a terminal, and it keeps running indefinitely: talking to other nodes over the network, validating new transactions and blocks as they arrive, storing them on disk, and answering questions from wallets and miners. It is not a website, not a phone app, and not a one-shot script that runs and exits. It is the engine that sits underneath all of those.

When you install it, you get one command-line program, conventionally invoked as `hathor-cli`, which can do many jobs (run a node, generate a wallet, run a miner, dump the database, and more). Running an actual node is just one of those jobs — the subcommand `run_node`. The rest of this book is overwhelmingly about what `run_node` sets in motion.

0.2 What a full node does

The word "full" is doing real work in "full node." A full node is one that keeps and independently verifies the **entire** ledger, trusting no one to have done the checking for it. (There are lighter kinds of participant — wallets that ask a full node for answers rather than storing everything themselves — but `hathor-core` is the full kind.)

A full node has five standing responsibilities. Almost every package in the codebase exists to serve one of them, so it is worth fixing them in mind:

1. **Store the ledger.** Keep every block and transaction, plus derived bookkeeping, on disk, and be able to look any of it up quickly. (Chapters 27–28: storage and indexes.)
2. **Validate everything.** When a new transaction or block arrives, check that it obeys every rule — signatures are valid, no coins are spent twice, proof-of-work is sufficient — before accepting it. A full node never takes another node's word for validity. (Chapter 31: verification.)
3. **Reach consensus.** Decide, by the network's agreed rules, which version of history is the real one when there is any ambiguity (for example, two conflicting transactions). (Chapter 32: consensus.)
4. **Talk to peers.** Find other nodes, connect to them, and exchange data so that everyone converges on the same ledger. This is the **peer-to-peer**⁶ layer, and "catching up" to the rest of the network is called **syncing**⁷. (Chapters 34–35: P2P and sync.)
5. **Serve clients.** Expose interfaces — HTTP APIs, WebSockets, a mining protocol — so that wallets can read balances, miners can fetch work, and operators can monitor the node. (Chapters 30, 36, 37: events/WebSocket, mining.)

Everything else — the command-line surface, configuration, the reactor, the builder, metrics — is infrastructure that exists so these five jobs can be done reliably. Keep this list nearby; in §0.4 you will see that the module groups map almost one-to-one onto these responsibilities.

0.3 The life of a node

This is the spine of the entire book. Part II walks the code in roughly the order events happen here, so internalize this narrative now. We tell it in two acts: **startup** (the node assembling itself and getting ready) and **steady state** (the node running and doing its job). There is no code in this section — just the story.

Act I — Startup

Picture an operator typing this into a terminal:

```
hathor-cli run_node --testnet --data ./data
```


1. The command is dispatched. The `hathor-cli` program looks at the first word after it — `run_node` — and routes to the code responsible for that subcommand. (Internally this is a dictionary lookup from command name to a Python module; we cover it in Chapter 21.) There are roughly forty subcommands; `run_node` is the one that boots an actual node.

2. Arguments and logging are set up. The flags (`--testnet`, `--data ./data`, and dozens of optional ones) are parsed into a structured configuration object, and structured logging⁸ is switched on so that everything the node does from here is recorded.

3. Settings are loaded. The node must know which network it is joining — `mainnet` (real money), `testnet` (a practice network), or a private network. This choice selects a **settings profile**: a file of constants (the genesis data, timing parameters, feature schedules) that every later step reads. Get this wrong and the node would be speaking a different dialect than its peers. (Chapter 22.)

4. The reactor is initialized. Hathor is built on a framework called **Twisted**, whose centerpiece is the **reactor**⁹ — the single event loop¹⁰ that will drive the whole program once it is running. At startup the reactor is created but not yet running; think of it as starting the car's electrical system before putting it in gear. Twisted and the reactor are so central that they get a full chapter early (Chapter 16) and an abstraction chapter of their own (Chapter 23).

5. The node is assembled — the "builder" phase. This is the most intricate part of startup. A builder¹¹ constructs, in the right order, every component the node needs and wires them together:

- the **storage** layer opens the on-disk database (RocksDB);
- the **indexes** are prepared (fast lookup tables for balances, addresses, heights, and more);
- the **wallet** is loaded, if one is configured;
- the **pub-sub**¹² system and **event manager** are created (the node's internal announcement system);
- the **verification** and **consensus** services are built;
- the **nano-contract** runtime and the **feature-activation** service are set up.

The result is a single object, the `HathorManager`, holding references to all of these. The manager is the node's central coordinator — the closest thing to a "main object." It is important enough to get its own chapter (Chapter 29), and assembling it is important enough to get another (Chapter 24).

6. The manager starts. Now the assembled node is told to start. Several things happen in a deliberate order:

- **Crash check.** The node inspects a flag in storage. If the last shutdown was not clean, the on-disk bookkeeping may be untrustworthy, and the node refuses to start rather than risk acting on corrupt data. (This is a real safety mechanism, not a formality.)
- **Initialization.** The node loads the **genesis**¹³ — the hard-coded first block and transactions that every node agrees on as the starting point of history — and rebuilds its in-memory view of the ledger from what is already on disk. During this phase the storage is told to allow access to data in any validation state; once initialization finishes, access is narrowed to only fully-valid data. (Why the node temporarily relaxes and then tightens this "scope" is a subtle point we return to in Chapter 25.)
- **Subsystems come online**, again in order: the WebSocket server (if enabled), metrics collection, then the **peer-to-peer connections manager**, then the wallet, then the mining server (if mining is enabled), then the proof-of-authority block producer (only on certain private networks).
- Finally the manager marks itself **READY**.

7. The reactor is put in gear. The reactor's event loop is told to run. Control passes to Twisted, and the startup story ends. From here the node is reactive: it sleeps until something happens — a peer connects, a transaction arrives, a timer fires — and responds.

Act II — Steady state

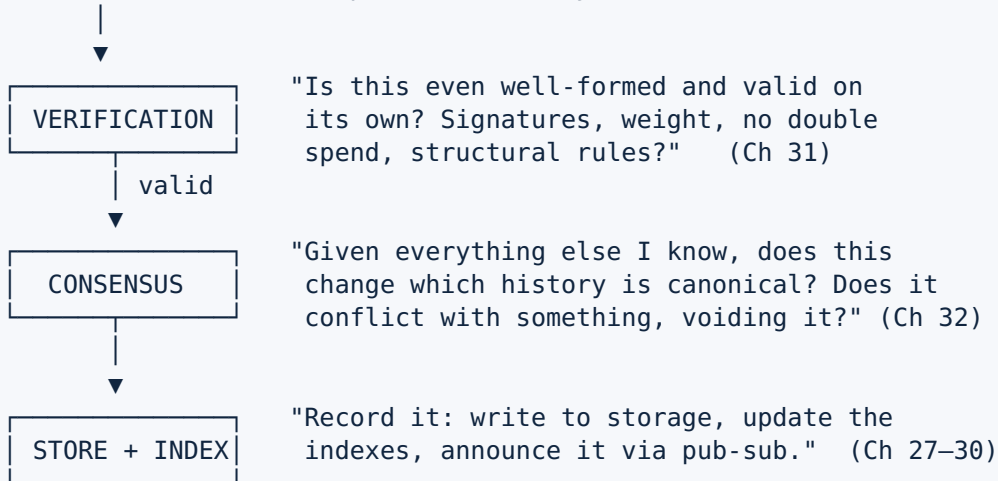
Now the node is alive. Four kinds of activity run concurrently, all driven by the one reactor:

Finding and connecting to peers. The connections manager discovers other nodes (from a bootstrap list and from peers themselves) and opens connections. Each connection performs a handshake in which the two nodes agree on a protocol version and exchange who they are. (Chapter 34.)

Syncing. A freshly started node is usually far behind — it may have a near-empty database while the network has years of history. Over each peer connection, a sync agent negotiates and downloads the blocks and transactions the node is missing, in dependency order, until it has caught up. Hathor's current protocol for this is **sync-v2**¹⁴; an older sync-v1 has been removed from the codebase entirely, so any old comment mentioning it is stale. (Chapter 35.)

Ingesting new vertices. Whether a vertex arrives from syncing or as a brand-new transaction broadcast across the network, it flows through the same pipeline:

vertex arrives (from a peer, or freshly created)



The component that runs this pipeline end-to-end is the **vertex handler** (Chapter 33). It is a small but pivotal piece: it is where "data arrived" becomes "ledger changed."

Serving clients. Concurrently, the node answers wallets over HTTP, streams events over WebSockets, and — if mining is on — hands out work to miners and accepts the blocks they find. A miner repeatedly tries to solve a proof-of-work ¹⁵ puzzle; when it succeeds, the resulting block re-enters the ingestion pipeline above just like any other vertex. (Chapter 37.)

That is the whole life of a node, start to finish. Every chapter in Part II is a zoom-in on one box in this story.

0.4 The module map

`hathor-core` lives in two top-level places: the package `hathor/` (the node itself) and the package `hathor_cli/` (the command-line surface that launches and operates it). Inside `hathor/` are roughly two dozen sub-packages. Rather than list them alphabetically, here they are grouped by the five responsibilities from §0.2 plus the infrastructure that supports them.

hathor-core/	
├─ hathor_cli/	← command-line surface (run_node + ~40 tools)
├─ hathor/	
│ manager.py	← HathorManager: central coordinator ◀ YOU ARE HERE
│ vertex_handler/	← ingestion pipeline (verify→consensus→store)
├─ — DOMAIN MODEL —	
│ transaction/	← vertex model: Block, Transaction, in/outputs, metadata
│ serialization/	← the bespoke binary wire format
│ verification/	← rule-checking for vertices
│ consensus/	← which history is canonical; weights, voiding, PoA
├─ — STORAGE & LOOKUP —	
│ storage/	← low-level RocksDB key-value wrapper
│ transaction/storage/	← vertex-aware storage on top of RocksDB
│ indexes/	← fast lookups: UTX0, address, tokens, height, mempool
├─ — NETWORKING & CLIENTS —	
│ p2p/	← peer-to-peer protocol, connections, sync-v2
│ websocket/	← admin / streaming WebSocket surface
│ event/ pubsub.py	← internal + external event system
├─ — MINING & MONETARY —	
│ mining/	← block templates, CPU miner service
│ stratum/	← the Stratum mining protocol
│ merged_mining/	← mine Hathor as auxiliary work to Bitcoin
│ daa.py difficulty.py	← difficulty adjustment & weight math
│ reward_lock.py	← rules locking freshly-mined rewards
├─ — HIGHER SERVICES —	
│ nanocontracts/	← smart-contract runtime (largest subsystem)
│ feature_activation/	← miner-voted protocol upgrades
│ wallet/ crypto/ pycoin/	← key management & signing
├─ — INFRASTRUCTURE —	
│ builder/	← assembles & wires the node
│ reactor/	← Twisted reactor abstraction
│ conf/	← settings & network profiles
│ sysctl/	← runtime control socket
│ metrics.py profiler/ healthcheck/	← observability
│ simulator/ dag_builder/	← deterministic testing infrastructure
│ execution_manager.py	← task coordination

What follows is one paragraph per major area — enough to know what each is for. The depth comes later.

Domain model — the data and its rules

transaction/ is the heart of the data model. Despite the name, it defines all vertex types: the base class `BaseTransaction`, and its subclasses `Block`, `Transaction`, and `MergeMinedBlock`, along with their inputs, outputs, and the per-vertex **metadata**¹⁶ the

node maintains (height, accumulated weight, whether the vertex has been voided, and more). It is one of the largest and most foundational packages; the data structures it defines are referenced everywhere. (Chapter 25.)

serialization/ turns those vertices into bytes and back. Hathor does **not** use JSON or Protobuf for its wire format; it has a custom, compact binary encoding. Whenever a vertex is sent to a peer or written to disk, this package does the translation. (Chapter 26.)

verification/ holds the rules. For each vertex type there is a verifier that checks the vertex obeys the protocol — structurally and cryptographically — in isolation. (Chapter 31.)

consensus/ decides canonical history. When vertices conflict, or when there are competing chains of blocks, the consensus algorithm uses accumulated **weight**¹⁷ to decide which version wins and marks the losers as **voided**. It also contains the proof-of-authority variant used by certain private networks. (Chapter 32.)

Storage and lookup — where the ledger lives

storage/ is a thin wrapper around **RocksDB**¹⁸, an embedded key-value database. It speaks in raw bytes: put this key, get that key. **transaction/storage/** sits on top and speaks in vertices: store this block, fetch the transaction with this hash, walk the graph from here. **indexes/** maintains the derived lookup tables that make common questions fast — which outputs are unspent (UTXO), what is this address's history, what is the current height, what is waiting in the mempool¹⁹. Indexes can be rebuilt from the stored vertices, which is part of what the initialization phase in §0.3 does. (Chapters 27–28.)

Networking and clients — talking to the world

p2p/ is among the largest packages. It implements the peer-to-peer protocol: discovering peers, establishing and managing connections (each connection is driven by a Twisted protocol object and progresses through a state machine), and running sync-v2 to exchange ledger data. **websocket/** exposes a streaming interface for dashboards and integrations. **event/** and **pubsub.py** form the announcement system: internally, components subscribe to events like "a new block was accepted"; externally, an optional persistent event queue lets downstream systems replay everything the node has seen. (Chapters 30, 34–36.)

Mining and monetary — minting and securing coins

mining/ produces block templates (the puzzle a miner must solve) and includes a simple CPU miner. **stratum/** implements the **Stratum** protocol that real mining software uses to talk to the node. **merged_mining/** lets miners mine Hathor blocks as a

side-effect of mining Bitcoin. `daa.py` and `difficulty.py` hold the difficulty-adjustment math that keeps blocks arriving at a steady rate. `reward_lock.py` enforces that freshly-mined coins can't be spent for a while. (Chapter 37.)

Higher services — built on the base ledger

`nanocontracts/` is the smart-contract engine and the single biggest subsystem (over fifteen thousand lines). It defines blueprints (contract templates), a runner that executes contract code with metered resource limits²⁰, and its own state storage. `feature_activation/` governs protocol upgrades: new features are switched on by miner signalling²¹ over a schedule, so the network can change rules without a flag day. `wallet/`, `crypto/`, and `pycoin/` handle keys, addresses, and digital signatures. (Chapters 38–40.)

Infrastructure — making it all run

`builder/` assembles the node (\$0.3, step 5). `reactor/` wraps Twisted's reactor so the node can optionally run on an asyncio backend instead. `conf/` loads the settings profiles. `sysctl/` is a control socket for poking a running node (query state, tweak parameters). `metrics.py`, `profiler/`, and `healthcheck/` are observability. `simulator/` and `dag_builder/` let tests spin up a deterministic, in-memory network — invaluable for reproducing bugs. (Chapters 23–24, 41–43.)

The command-line surface

`hathor_cli/` is everything you invoke from a terminal: the dispatcher that routes subcommands, `run_node` itself, and a toolbox of operator and developer utilities (generate a wallet, export the database, run a miner, dump nano-contract state, and so on). It is the outermost layer — the front door — and so it is the first package we open in Part II. (Chapter 21.)

0.5 How to read this book

This book is written for a developer who is comfortable writing Python but has not yet met most of the senior-level concepts, surrounding tooling, or blockchain theory that `hathor-core` assumes. So it teaches three kinds of thing, woven together:

- **Programming concepts** — the computer-science scaffolding: object orientation, callbacks, asynchronous programming, design patterns, decorators, and the like. (Part I, Chapters 1–5.)
- **Domain concepts** — blockchain and Hathor theory: the DAG, the UTXO model, weight and consensus, checkpoints. (Part I, Chapters 6–10.)

- **The stack** — the third-party technologies the node is built from: Poetry, Docker, Twisted, RocksDB, and more. (Part I, Chapters 11–20, and at the modules that use them.)

Three reading aids

To keep the main text readable while still defining everything, the book uses three devices consistently. Recognize them and you will know how much detail to expect:

1. **Footnotes** define a single word or piece of jargon on the spot, in a sentence or two. Every footnoted term also appears in the glossary at the back. This page is full of them — scroll to the bottom to see the style.
2. **Recap boxes** look like this:

Recap — vertex (full treatment in Ch. 8 & 25). A vertex is any node in Hathon's ledger graph — either a [Block](#) or a [Transaction](#). The codebase uses "vertex" as the umbrella term when it does not care which kind it is holding. We will define it properly, with the graph theory behind it, in Chapter 8, and meet it in code in Chapter 25.

They re-introduce a concept in the context of the chapter you are reading, just enough to follow along, and point you to the full treatment elsewhere. They exist because meeting an idea again in a new setting is how it sticks — so do not skip them as repetition.

1. **Deep-dives** are the full, once-per-concept treatment — anywhere from two to twenty pages — placed where the concept first truly matters. Each technology and each concept is explained in depth exactly once; everywhere else you get a footnote or a recap box pointing back to it.

The order, and how to navigate

Part I builds your vocabulary before you touch the codebase, concepts first, then the tools. Part II then walks the node in the order of the life-of-a-node story from §0.3, beginning at the command line and ending with the testing infrastructure. You can read straight through, and that is the intended path. But you can also use §0.3 and §0.4 as an index: find the box or package you care about, follow its chapter pointer, and read the recap boxes there to pull in whatever background you are missing.

One convention worth stating plainly: when this book cites code, it gives the file path always, and a line number when it names a specific identifier — like `manager.py:276`. Those are clickable in the source markdown and precise enough to grep. The line numbers are accurate as of the branch named on the cover; if the code has moved since, treat them as close-but-verify.

Recap

Responsibility (§0.2)	Where it lives (§0.4)	Covered in
Store the ledger	<code>storage/</code> , <code>transaction/storage/</code> , <code>indexes/</code>	Ch 27–28
Validate everything	<code>verification/</code>	Ch 31
Reach consensus	<code>consensus/</code> , <code>daa.py</code> , <code>difficulty.py</code>	Ch 32
Talk to peers (sync)	<code>p2p/</code>	Ch 34–35
Serve clients	<code>websocket/</code> , <code>event/</code> , <code>mining/</code> , <code>stratum/</code>	Ch 30, 36–37
Coordinate it all	<code>manager.py</code> , <code>vertex_handler/</code> , <code>builder/</code>	Ch 24, 29, 33
The data itself	<code>transaction/</code> , <code>serialization/</code>	Ch 25–26
Higher services	<code>nanocontracts/</code> , <code>feature_activation/</code> , <code>wallet/</code>	Ch 38–40
Launch & operate	<code>hathor_cli/</code>	Ch 21

You now have the two things this chapter exists to give you: the **life of a node** (§0.3) — the order in which everything happens — and the **module map** (§0.4) — where each piece of that story lives in the codebase. Every remaining chapter is a zoom-in on one box in that story or one tool it depends on. The next part of the book steps back from Hathor entirely to build the programming and blockchain vocabulary the code assumes; if you are impatient to see code, you can jump ahead to Chapter 21 and lean on the recap boxes — but the foundations are there because the code will make far more sense with them in place.

-
1. A ledger is just a record of transactions and balances — historically an accountant's book. A distributed ledger is one copy of which is held and maintained by many computers at once, with no single authoritative master copy. ↩
 2. **DAG** = Directed Acyclic Graph. A graph is a set of items ("nodes") joined by links ("edges"). Directed means each edge points one way. Acyclic means you can never follow the edges in a loop back to where you started. Full treatment in Chapter 8. ↩
 3. Vertex is the standard graph-theory word for a node of a graph. Hathor's code uses it as the common term for "a block or a transaction," since both are nodes of the ledger DAG. ↩
 4. A smart contract is a small program stored on a ledger that automatically moves tokens according to its own rules when called — e.g. "release these funds only if two of three people approve." Hathor's version is called a nano-contract. ↩

5. A daemon is a program that runs continuously in the background rather than finishing and exiting. Web servers and databases are daemons. The word is pronounced "demon" and comes from an old Unix tradition. ↵
6. Peer-to-peer (P2P) means every participant is an equal "peer" that connects directly to other peers, with no central server in the middle. Contrast a client-server model, where everyone talks through one company's servers. ↵
7. Syncing (synchronizing) is the process by which a node that is behind downloads the blocks and transactions it is missing from its peers until its copy of the ledger matches the network's. ↵
8. Structured logging records log entries as machine-readable key-value data (e.g. `event="block accepted" height=42`) rather than as freeform sentences, so the logs can be searched and analyzed programmatically. Hathor uses a library called `structlog`; full treatment in Chapter 17. ↵
9. The reactor is the central object of the Twisted framework: a single loop that waits for events (network data, timers, etc.) and calls the right piece of your code in response. Full treatment in Chapter 16. ↵
10. An event loop is a program structure that repeatedly waits for "something to happen" and then dispatches a handler for it, instead of running top-to-bottom and blocking. It is the core idea behind asynchronous programming — Chapter 2. ↵
11. A builder is a design pattern: an object whose job is to construct another, complicated object step by step. Hathor's builder constructs the fully-wired node. The pattern is explained in Chapter 3; Hathor's builders in Chapter 24. ↵
12. Pub-sub (publish-subscribe) is a messaging pattern where publishers announce events without knowing who is listening, and subscribers register interest in event types. It decouples the announcer from the reactors. Chapter 30. ↵
13. The genesis is the hard-coded starting point of the ledger — the first block and initial transactions — that every node agrees on by definition. Without a shared genesis, two nodes would have no common root of history. ↵
14. Sync-v2 is the current peer-to-peer synchronization protocol. An earlier sync-v1 existed but has been removed; only sync-v2 remains. Chapter 35. ↵
15. Proof-of-work (PoW) is a scheme where producing a valid block requires finding a number that makes the block's hash fall below a target — work that is hard to do but trivial to check. It is what makes rewriting history expensive. Chapter 9. ↵
16. Metadata here means data the node computes and stores about a vertex (its height, accumulated weight, voided status, ...) as opposed to the vertex's own fixed contents. It is recomputed and updated as the ledger evolves. Chapter 25. ↵
17. Weight is a numeric measure of how much proof-of-work a vertex represents; accumulated weight sums the weight reachable through the graph. Consensus prefers the history with the most accumulated weight. Chapter 9 & 32. ↵
18. **RocksDB** is an embedded key-value database: a fast on-disk store of `key → value` byte pairs that runs inside the node's own process rather than as a separate server. Full treatment, and the comparison against alternatives like MongoDB, in Chapter 27. ↵
19. The mempool ("memory pool") is the set of valid transactions a node knows about that have been seen but not yet confirmed/ordered by a block. Chapter 28. ↵
20. Metered execution means the contract runner counts the resources a contract uses (a kind of "gas") and stops it if it exceeds a limit, so a buggy or malicious contract cannot run forever. Chapter 39. ↵
21. Miner signalling is a voting mechanism: miners set specific bits in the blocks they produce to indicate support for a proposed upgrade, and the rule change activates once enough support accumulates over a defined window. Chapter 38. ↵

Chapter 1 — Object-Oriented Programming

What you'll learn in this chapter

- The problem object-oriented programming exists to solve, and how it compares to the other ways of structuring a program (procedural and functional) — and why a system like a node leans on OOP.
- The four pillars in working order: **encapsulation, inheritance, polymorphism, abstraction** — each motivated, defined, and shown in small, runnable examples.
- The Python machinery that expresses them: classes, `self`, `__init__`, methods, properties, abstract base classes, and the "dunder" protocol methods.
- When **composition** beats inheritance.
- A **bridge** to where each idea reappears in `hathor-core`, so the codebase chapters land on prepared ground.

This is a concept chapter, not a code-tour. There is no `hathor-core` source here on purpose — you will meet the real classes in Part II, and they will make far more sense once these ideas are second nature. Everything below is plain Python you could paste into a shell and run. The final section (§1.10) is the only one that looks forward, pointing at exactly where in the codebase each idea is put to work.

If you already know OOP comfortably, skim §1.2 (the paradigm comparison) and §1.10 (the bridge), and move on.

1.1 The problem: programs that grow

Small programs need almost no structure. A fifty-line script that reads a file, transforms some numbers, and prints a result can be one flat sequence of statements, and you will never get lost in it.

The trouble starts when a program gets big and long-lived. A full node is both: tens of thousands of lines, running for months, juggling many things at once — dozens of network connections, a database, a wallet, a mempool, a mining service. Two specific pains show up as a codebase grows, and OOP is a direct response to both.

Pain 1 — data and the rules about it drift apart. Imagine modelling a bank account as a plain number:

```
balance = 100
# ...hundreds of lines later, in some unrelated function...
balance = balance - 9999 # nothing stopped this; the account is now negative
```

The rule "an account may never go negative" lives only in the programmer's head. The data (`balance`) is just a number sitting in the open, and any line anywhere is free to corrupt it. In a large program, "any line anywhere" means thousands of lines, written by several people over years. The rule will eventually be broken by accident.

What we want is to keep the data and the rules that protect it in the same place, and to forbid the rest of the program from touching the data except through those rules. That bundling is the central idea of OOP.

Pain 2 — many things are almost the same, but not quite. A node has several kinds of vertex (blocks, transactions), several kinds of storage backend (in-memory, on-disk), several kinds of peer-connection state. Each family shares most of its behavior but differs in a few specifics. Without a way to say "these are all accounts, and here is the one bit each kind does differently," you end up copy-pasting near-identical code and then fixing the same bug in five places. We want to write the shared part once and let each variant override only what differs.

Hold those two pains — protect invariants¹ by bundling data with behavior, and share behavior across a family of related types — they are exactly what the pillars in §1.3-§1.7 deliver.

1.2 Three ways to organize a program

OOP is not the only way to structure code, and it is not always the best one. It helps to see it next to its two main alternatives, because real codebases — `hathor-core` included — mix them. Python is a **multi-paradigm**² language: it supports all three, and lets you pick per situation.

Procedural (a.k.a. structured) programming. You organize the program as a set of procedures — functions — that operate on data passed to them. The data and the functions are separate things. This is how most people first learn to code, and it is excellent for straightforward, step-by-step tasks. Its weakness is Pain 1: the data floats free of the rules, and as the program grows, it becomes unclear which functions are allowed to change which data.

```
def deposit(account, amount): # data (account) and behavior (deposit) are separate
    account["balance"] += amount
```

Functional programming. You build the program out of pure functions³ — functions whose output depends only on their input and which change nothing outside themselves (no side effects⁴) — and you prefer immutable⁵ data that is never modified in place, only transformed into new values. This style shines for data-transformation pipelines and for safe concurrency (if nothing is ever mutated, two threads can't corrupt each other's data). Its awkward spot is modelling things that have identity and change over time — a network connection that opens, exchanges messages, and closes is naturally a stateful entity, and pretending it is a stream of immutable values can be more contortion than clarity.

```
def with_deposit(account, amount):          # returns a NEW account; original untouched
    return {**account, "balance": account["balance"] + amount}
```

Object-oriented programming. You bundle data together with the behavior that operates on it into **objects**, each with its own identity and its own internal **state**⁶. An object guards its own data and exposes a set of operations (methods) as the only sanctioned way to use or change it.

```
class Account:
    def __init__(self, balance):
        self._balance = balance          # the data...
    def deposit(self, amount):
        self._balance += amount          # ...lives with the behavior that guards it
```

Why OOP for a node? A full node is overwhelmingly a collection of long-lived, stateful entities with identity: the manager, each peer connection, the storage layer, the mempool, every vertex with its evolving metadata. Each has a lifecycle (created, started, used, stopped) and rules about what states are valid. That is precisely the shape OOP models best, which is why `hathor-core` uses OOP as its backbone — while still using functional touches for pure data transformation and procedural code for simple scripts. The lesson is not "OOP is best"; it is "match the paradigm to the shape of the problem," and a node's problems are mostly object-shaped.

A note on dogma. You will meet programmers who treat one paradigm as a religion. Resist it. Paradigms are tools; a senior engineer reaches for whichever makes the current code clearest. This book teaches OOP first because `hathor-core` is built on it, not because the others are lesser.

1.3 Classes and objects

Here is the vocabulary, made precise. A **class** is a blueprint for a kind of object. An **object** (or **instance**⁷) is a concrete thing built from that blueprint. The class `Account` describes what every account is and can do; `my_account = Account(100)` builds one specific account. One class, many instances — just as one architectural blueprint yields many houses, each with its own address and contents.

```
class Account:
    def __init__(self, owner, balance=0):
        self.owner = owner          # an attribute: data belonging to this instance
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount      # a method: behavior belonging to instances
        return self.balance

alice = Account("Alice", 100)      # instantiation: build an object from the class
bob = Account("Bob")              # balance defaults to 0
alice.deposit(50)                 # call a method on the alice instance
print(alice.balance)              # 150
print(bob.balance)               # 0 - bob is a separate object with its own state
```

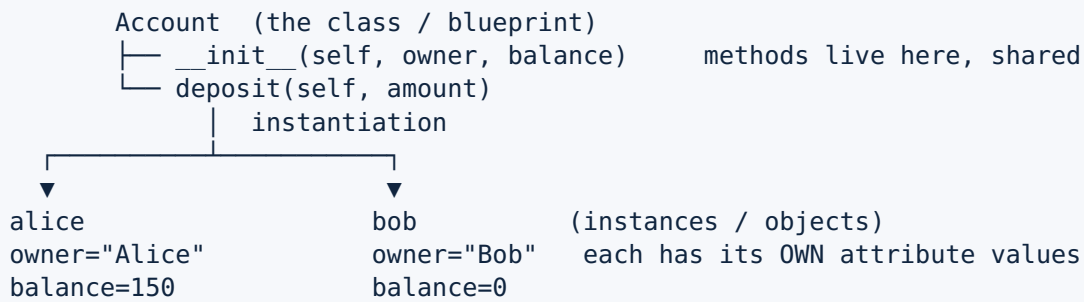
Three pieces of machinery deserve a slow read, because they confuse almost everyone at first:

`__init__` — the initializer. When you write `Account("Alice", 100)`, Python creates a fresh empty object and then immediately calls its `__init__` method to set it up.

`__init__` is where you give the new object its starting **attributes**⁸ (its data). It is not strictly a "constructor"⁹ in the C++/Java sense — the object already exists by the time `__init__` runs — but you can think of it as "the setup routine that runs once, per object, at birth."

`self` — the instance the method is working on. Every method's first parameter is `self`, which is the particular object the method was called on. When you write `alice.deposit(50)`, Python calls `deposit` with `self = alice`. That is how `self.balance += amount` knows to change Alice's balance and not Bob's. `self` is not a keyword — it is just a parameter name (a near-universal convention), and Python passes the instance into it automatically. The single most common beginner stumble is forgetting `self` in a method's signature or when accessing an attribute.

Attributes vs. methods. An attribute is data attached to an object (`alice.owner`, `alice.balance`). A method is a function attached to a class that operates on an instance (`alice.deposit(...)`). Both are reached with the dot. Each instance carries its own attribute values — that is what "its own state" means — while the methods are shared by all instances of the class (they live on the blueprint, not on each house).



1.4 Encapsulation — the first pillar

Encapsulation¹⁰ is the bundling of data with the behavior that operates on it, plus the controlled hiding of that data so the outside world can only touch it through approved operations. It is the direct cure for Pain 1.

Recall the rule "an account may never go negative." With encapsulation, the account enforces its own rule, and no outside code can break it:

```
class Account:
    def __init__(self, balance=0):
        self._balance = balance

    def withdraw(self, amount):
        if amount > self._balance:
            raise ValueError("insufficient funds")  # the rule, enforced in one place
        self._balance -= amount

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("amount must be positive")
        self._balance += amount

    @property
    def balance(self):  # read-only view of the protected data
        return self._balance
```

Two conventions are doing the work here:

The leading underscore. Naming the attribute `_balance` instead of `balance` is Python's way of saying "this is internal; do not touch it from outside." Python does not forcibly prevent access — unlike some languages, there is no hard `private` keyword — it relies on a shared convention: a leading underscore means "hands off."¹¹ The point is that all changes to the balance now funnel through `deposit` and `withdraw`, where the rule lives. The invariant has exactly one guardian.

The `@property`. The `balance` method decorated with `@property` lets outside code read `account.balance` as if it were a plain attribute, while there is no setter, so it cannot be assigned. `account.balance` works; `account.balance = -5` fails. You get a read-only window onto protected internals. (`@property` is itself an example of a decorator, the subject of Chapter 4 — for now, read it as "turns this method into a smart attribute.")

The payoff: the **invariant** " $\text{balance} \geq 0$ " is now true by construction. You can read every other line of the program and never have to worry that one of them quietly corrupted an account, because the only doors into `_balance` are the two methods that check the rule. That is the whole game of encapsulation — shrink the set of code that can break a rule down to one small, auditable place.

1.5 Inheritance — the second pillar

Inheritance¹² lets a new class take on all the attributes and methods of an existing class and then add to or change them. The existing class is the **superclass** (or base/parent); the new one is the **subclass** (or derived/child). The relationship inheritance models is "**is-a**": a savings account is an account.

This is the cure for Pain 2. Write the shared behavior once in the base class; let each variant add only its differences.

```
class Account:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self._balance = balance

    def withdraw(self, amount):
        if amount > self._balance:
            raise ValueError("insufficient funds")
        self._balance -= amount

class SavingsAccount(Account):
    # SavingsAccount IS-A Account
    def __init__(self, owner, balance=0, rate=0.02):
        super().__init__(owner, balance)  # run the base class's setup first
        self.rate = rate                 # then add what's specific to savings

    def add_interest(self):
        # a brand-new method, only on savings
        self._balance += self._balance * self.rate
```

`SavingsAccount` automatically has `withdraw` and `owner` — it did not repeat them. It added `rate` and `add_interest`. Two mechanics matter:

`super().__init__(...)`. A subclass usually wants the base class's setup to run and then its own. `super()` is how a subclass calls up to its parent's version of a method. Here `super().__init__(owner, balance)` runs `Account.__init__` (setting `owner` and `_balance`), after which the subclass sets `rate`. Forgetting the `super()` call is a classic bug: the object skips its parent's initialization and ends up half-built.

Overriding. A subclass can replace a method it inherited by defining one with the same name. This is **method overriding**¹³. Suppose savings accounts forbid overdrawing below a minimum:

```
class SavingsAccount(Account):
    MIN_BALANCE = 100

    def withdraw(self, amount):
        # overrides Account.withdraw
        if self._balance - amount < self.MIN_BALANCE:
            raise ValueError("would drop below minimum balance")
        super().withdraw(amount)
        # reuse the parent's logic for the rest
```

Notice the override can still call `super().withdraw(...)` to reuse the parent's work and only add its extra check. That is the ideal: specialize the difference, inherit the rest.

A caution. Inheritance is tempting and easy to over-use. A deep tower of subclasses ($A \rightarrow B \rightarrow C \rightarrow D$, each changing a little) becomes hard to follow, because to understand class D you must hold all four in your head at once. §1.8 gives the usual remedy. A good rule: use inheritance for genuine "is-a" relationships, and be suspicious of hierarchies more than two or three deep.

1.6 Polymorphism — the third pillar

Polymorphism¹⁴ (Greek: "many shapes") means that code written against a general type works, unchanged, on any of its specific variants — each variant supplying its own behavior behind a shared name. You write `account.withdraw(x)` once, and it does the right thing whether `account` is a plain `Account` or a `SavingsAccount` with its extra rule.

```
def end_of_day(accounts, fee=5):
    for account in accounts:
        account.withdraw(fee)
        # same call; each object runs ITS OWN withdraw

portfolio = [Account("Alice", 200), SavingsAccount("Bob", 500)]
end_of_day(portfolio)
# Alice runs Account.withdraw; Bob runs SavingsAccount.withdraw — automatically.
```

`end_of_day` knows nothing about savings accounts. It will keep working when you add `CheckingAccount`, `BusinessAccount`, and ten more types years later, as long as each has a `withdraw` method. This is why polymorphism matters in a long-lived codebase: it lets you add new variants without editing the code that uses them. New behavior, no rewrites.

Duck typing. Python takes polymorphism a step further than many languages. It does not require the objects to share a common base class at all — it only requires them to have the method you call. The slogan is "if it walks like a duck and quacks like a duck, treat it as a duck." This is **duck typing**¹⁵ :

```
class Report:
    def withdraw(self, amount):
        print(f"deducting {amount} from the report budget")

end_of_day([Account("Alice", 200), Report()]) # works! Report just needs .withdraw
```

`Report` is not an account and inherits nothing from `Account`, but because it has a `withdraw` method, `end_of_day` accepts it. Python checks for the method at the moment of the call, not in advance. This is flexible and concise; the trade-off is that the "contract" (what methods an object must have) is implicit. The next pillar makes that contract explicit when you want it to be.

1.7 Abstraction — the fourth pillar

Abstraction¹⁶ is the practice of describing what something does while hiding how. A car's steering wheel is an abstraction: you turn it to steer, without knowing about the rack-and-pinion underneath. In code, abstraction lets you program against a contract — a set of operations something promises to provide — rather than against any one concrete implementation.

The tool for stating such a contract explicitly is the **abstract base class (ABC)**¹⁷. An ABC declares the methods that every concrete subclass must provide, but does not (necessarily) implement them. It cannot be instantiated directly — it is a contract, not a thing.

A motivating example you will meet for real in the storage chapters: suppose the node needs to store data, and you want to support more than one backend — an in-memory store for tests, a file-backed store for production — interchangeably.

```

from abc import ABC, abstractmethod

class Storage(ABC):
    @abstractmethod
    def save(self, key, value): ...
    # the contract
    # "every Storage MUST provide save"

    @abstractmethod
    def load(self, key): ...
    # "...and load"

class MemoryStorage(Storage):
    def __init__(self):
        self._data = {}
    def save(self, key, value):
        self._data[key] = value
    def load(self, key):
        return self._data.get(key)
    # one concrete implementation

class FileStorage(Storage):
    def save(self, key, value):
        ... # write to disk
    def load(self, key):
        ... # read from disk
    # another, with the same contract

```

Now the rest of the program can be written against `Storage` — "give me something I can `save` to and `load` from" — and neither know nor care which concrete backend it received:

```

def remember(store: Storage, key, value):
    store.save(key, value)
    # works with MemoryStorage, FileStorage, or any future o

```

Two benefits fall out. First, `Storage()` raises an error — Python refuses to instantiate an abstract class, so you cannot accidentally use the empty contract as if it were real storage. Second, if you write a new backend and forget to implement `load`, Python refuses to instantiate that too, telling you the contract is unmet. The implicit duck-typing contract of §1.6 is now written down and enforced.

This pairing — an abstract interface plus interchangeable concrete implementations — is one of the most common shapes in `hathor-core`, and recognizing it is half of reading the codebase.

1.8 Composition over inheritance

Inheritance ("is-a") is not the only way to build a class out of other classes. The alternative is **composition**¹⁸: an object contains other objects and delegates work to them. The relationship is "**has-a**": an account has a transaction log.

```

class TransactionLog:                                # a small, focused object
    def __init__(self):
        self._entries = []
    def record(self, text):
        self._entries.append(text)

class Account:
    def __init__(self, balance=0):
        self._balance = balance
        self._log = TransactionLog()                 # HAS-A log (composition)
    def deposit(self, amount):
        self._balance += amount
        self._log.record(f"deposit {amount}")        # delegate logging to the log object

```

`Account` did not inherit from `TransactionLog` — a log is not a kind of account — it holds one and hands it the logging job. Why prefer this? Composition keeps each object small and single-purpose, and you can swap the contained object (a `FileLog`, a `NullLog` that records nothing) without touching the account. The widely-repeated advice "**favor composition over inheritance**" means: reach for "has-a" by default, and use "is-a" inheritance only when one type is genuinely a specialized kind of another. Composition avoids the deep, brittle towers warned about in §1.5.

A quick test when you are unsure which to use: say it out loud. "A savings account is an account" — inheritance fits. "An account is a transaction log" — nonsense; it has one — composition fits.

1.9 The dunder protocol

Python lets your objects plug into the language's own syntax through specially-named methods wrapped in double underscores — **dunder**¹⁹ methods ("double underscore"), also called magic methods. You have already met `__init__`. A few more you will see constantly:

```

class Money:
    def __init__(self, cents):
        self.cents = cents

    def __repr__(self):
        return f"Money({self.cents}c)"  # how the object prints in logs/debugger

    def __eq__(self, other):
        return self.cents == other.cents  # what == means for two Money objects

    def __add__(self, other):
        return Money(self.cents + other.cents)  # what + means

a = Money(150)
b = Money(150)
print(a)          # Money(150c)          ← __repr__ ran
print(a == b)     # True                  ← __eq__ ran (without it, == compares identity)
print(a + b)      # Money(300c)         ← __add__ ran

```

The pattern: by implementing the protocol method, you teach Python how your object should behave with built-in operations — printing, `==`, `+`, `len()`, iteration, indexing, and more. This is itself a form of polymorphism: `len(x)` works on a list, a string, or your object, because each implements `__len__`. You rarely need many of these, but recognizing them matters — when you see `__repr__` or `__eq__` in `hathor-core`'s classes, now you know they are defining how those objects print and compare, not some obscure ritual.

1.10 Bridge — where this lands in `hathor-core`

You will not need any of these terms in the abstract for long; Part II is built almost entirely from them. So that the codebase chapters land on prepared ground, here is the map from pillar to place. These are forward-pointers, not explanations — each gets its full treatment in the chapter named.

Bridge — OOP in the codebase (full treatment in the chapters named):

- **Inheritance & a base class.** The node's entire data model is one inheritance hierarchy: a `BaseTransaction` superclass with `Block`, `Transaction`, and `MergeMinedBlock` subclasses, each overriding the parts that differ. This is the §1.5 pattern at the heart of the ledger — **Chapter 25**.
- **Abstraction & ABCs.** Storage is defined as an abstract contract with interchangeable backends (in-memory for tests, RocksDB for production) — exactly the `Storage` ABC of §1.7 — **Chapters 27-28**. Hathor also uses a dedicated interface library (`zope.interface`) on top of plain ABCs; that wrinkle is covered where it appears — **Chapter 16 & 34**.
- **Polymorphism.** The verification and consensus code calls shared methods on whatever vertex it is handed, each type supplying its own behavior — §1.6 in action — **Chapters 31-32**.
- **Encapsulation & invariants.** A vertex's `metadata` guards rules like "weight only grows"; the manager guards the node's lifecycle state. The §1.4 "one guardian per invariant" idea is everywhere — **Chapters 25 & 29**.
- **Composition.** The `HathorManager` does not inherit its abilities; it holds a storage object, a connections manager, a wallet, and so on, delegating to each — the §1.8 "has-a" pattern at the largest scale — **Chapter 29**.
- **Dunder methods.** Vertices implement `__eq__` / `__hash__` so they can be compared and used as dictionary keys; many classes implement `__repr__` for logging — **Chapters 25 onward**.

There is also a related concept this chapter deliberately left out: the **design patterns** (factory, builder, observer, ...) that are named, reusable arrangements of the objects we just built. Those are Chapter 3 — read this chapter as the grammar and Chapter 3 as the idioms.

Recap

Pillar / idea	One-line definition	Cures	Python machinery
Class / object	Blueprint vs. concrete instance built from it	—	<code>class</code> , <code>__init__</code> , <code>self</code>
Encapsulation	Bundle data with its rules; hide the data	data corrupted from afar (Pain 1)	<code>_underscore</code> , <code>@property</code>
Inheritance	A subclass reuses & specializes a base class	duplicated near-identical code (Pain 2)	<code>class Sub(Base)</code> , <code>super()</code> , <code>override</code>

Pillar / idea	One-line definition	Cures	Python machinery
Polymorphism	One call, each type runs its own version	editing callers when adding variants	shared method names, duck typing
Abstraction	Program to a contract, hide the how	implicit, unenforced contracts	<code>ABC</code> , <code>@abstractmethod</code>
Composition	An object has others and delegates	brittle deep inheritance towers	hold an instance as an attribute
Dunder protocol	Teach objects to use built-in syntax	—	<code>__repr__</code> , <code>__eq__</code> , <code>__add__</code> , ...

Object-oriented programming earns its place in a node because a node is, at bottom, a population of long-lived stateful things that must each protect their own rules and come in related families. Encapsulation keeps each thing's rules in one auditable place; inheritance and polymorphism let related things share behavior and vary only where they must; abstraction lets the rest of the program depend on contracts rather than concretions; composition keeps the whole assembly from collapsing into a brittle tower. Keep the \$1.10 bridge in mind — when Chapter 25 opens `BaseTransaction` and you see a base class with overriding subclasses, encapsulated metadata, and dunder methods, you will be reading a sentence in a grammar you already know. The next chapter takes on the one concept that turns this static picture into a running node: how a single-threaded program does many things at once, through callbacks and the event loop.

1. An invariant is a condition that must always hold true for an object to be valid — e.g. "an account's balance is never negative." Much of good design is about identifying invariants and ensuring no code can violate them. ↩
2. A multi-paradigm language supports several programming styles (procedural, functional, object-oriented) rather than forcing one. Python, JavaScript, and Scala are multi-paradigm; older languages often were not. ↩
3. A pure function always returns the same output for the same input and has no observable effect beyond that return value. `len(x)` is pure; a function that prints, or edits a global, is not. ↩
4. A side effect is any change a function makes to the world outside itself — modifying a global variable, writing a file, printing, mutating an argument. Pure functional code minimizes side effects. ↩
5. Immutable data cannot be changed after creation; you make a modified copy instead. Python strings and tuples are immutable; lists and dicts are mutable (changeable in place). ↩
6. State is the data an object currently holds — its attribute values at a moment in time. An object with state "remembers" things between method calls; that memory is what makes it stateful. ↩
7. An instance is one concrete object built from a class. "Instantiate" means "create an instance." `Account("Alice")` instantiates the `Account` class. ↩

8. An attribute is a named piece of data attached to an object (e.g. `account.balance`) or to a class. Reached with the dot operator. ↵
9. A constructor is the routine that creates and sets up a new object. In Python the setup step is `__init__`; technically object creation is handled by `__new__`, which you almost never need to touch. ↵
10. Encapsulation = bundling data with the methods that operate on it, and restricting outside access to that data so it can only change through approved methods. ↵
11. Python has no enforced `private`. Convention: one leading underscore (`_x`) means "internal, please don't touch"; two leading underscores (`__x`) trigger name mangling, a stronger but still bypassable form of hiding. The community relies on discipline over locks. ↵
12. Inheritance lets a subclass acquire the attributes and methods of a superclass, then add or override behavior. Models an "is-a" relationship. ↵
13. Overriding is redefining, in a subclass, a method that already exists in the superclass, so instances of the subclass use the new version. Distinct from overloading (multiple methods with the same name but different parameters), which Python does not do in the classic sense. ↵
14. Polymorphism lets a single piece of code operate on objects of different types, each responding to the same method name in its own way. ↵
15. Duck typing is Python's style of polymorphism: an object's suitability is determined by whether it has the needed methods/attributes, not by what class it inherits from. "If it quacks, it's a duck." ↵
16. Abstraction = exposing what an operation does while hiding how it is done, so callers depend on a simple contract rather than complex internals. ↵
17. An abstract base class (ABC) declares methods that subclasses must implement and cannot itself be instantiated. Python provides it via the `abc` module (`ABC`, `@abstractmethod`). It turns an implicit duck-typing contract into an explicit, enforced one. ↵
18. Composition builds an object by giving it other objects as attributes and delegating work to them — a "has-a" relationship, as opposed to inheritance's "is-a." ↵
19. Dunder = "double underscore." Dunder methods like `__init__`, `__repr__`, `__eq__`, `__len__` let your objects integrate with Python's built-in syntax and functions. Also called magic or special methods. ↵

Chapter 2 — Callbacks & Asynchronous Programming

What you'll learn in this chapter

- Why a program spends most of its life waiting, and why naive blocking code wastes that time.
- The three ways to do many things at once — processes, threads, and a single-threaded **event loop** — and the trade-offs that make a node choose the last.
- **Callbacks**: handing over a function to be run later, built on the idea that functions are values.
- How an **event loop** actually works — by building a tiny one from scratch and tracing it.
- The escape from "callback hell": **futures/promises**, and then **async / await**, which lets you write asynchronous code that reads like straight-line code.
- The one unbreakable rule of this model — never block the loop — and a **bridge** to where every piece reappears in `hathor-core` (the Twisted reactor, Deferreds, the thread pool).

This is the most important concept in the book for understanding `hathor-core`. Almost every surprising thing about the codebase — why methods hand each other functions, why you must never call `time.sleep`, why there is a single "reactor" that everything runs on — follows from the ideas here. As with Chapter 1, there is no Hathor source in the body; we build the concepts on plain Python and toy examples, and the closing Bridge (§2.8) maps each one onto the real code.

A note before we start: in Chapter 1 the objects were static — data sitting still, guarded by rules. This chapter is about time: how a program decides what to do **next** when many things could happen and most of them involve waiting.

2.1 The problem: waiting is everywhere

Run a stopwatch on what a full node actually does, and you find it spends the overwhelming majority of its life **waiting** — for bytes to arrive from a peer across the internet, for the disk to return a record, for a timer to fire. Computation — verifying a signature, checking a rule — is a thin sliver between long stretches of waiting.

This split has a name. Work that is dominated by waiting for input/output is **I/O-bound**¹; work dominated by actual calculation is **CPU-bound**². A node is overwhelmingly I/O-bound. That single fact shapes its entire architecture, because how you handle waiting is the difference between a node that serves one peer and a node that serves thousands.

Here is the naive way, and why it fails. Suppose reading from a network connection looks like this:

```
data = connection.recv(1024)  # BLOCKS: the program stops here until bytes arrive
process(data)
```

The call to `recv` is **blocking**³: the line does not return until data is available, and the entire program is frozen on that line in the meantime. If the peer is slow, or silent for thirty seconds, your program does nothing for thirty seconds — it cannot read from a second peer, cannot answer a wallet, cannot fire a timer. One slow conversation stalls everything.

Blocking model, one thread, two peers:

```
recv(peerA) ——— [ waiting 30s for peerA ] ———> got data → now recv(peerB)...
                ▲
                peerB had data ready the whole time, but we never looked.
```

For a node that must hold thousands of simultaneous connections, blocking is a non-starter. The rest of the chapter is about the ways out.

2.2 Three ways to do many things at once

The general problem is **concurrency**⁴: making progress on several tasks over the same span of time. (Note: concurrency is not the same as parallelism⁵ — doing things at literally the same instant on multiple CPU cores. You can be concurrent on a single core by switching between tasks quickly; you are only parallel if two things truly run at once.) There are three classic approaches.

1. Multiple processes. Run several copies of the program, each a separate OS process with its own memory. Robust — a crash in one doesn't touch the others — but heavyweight: processes are expensive to create and can't share data easily. Good for "run N independent workers," poor for "manage 5,000 chatty connections that share state."

2. Multiple threads. A single process can have several **threads**⁶ of execution, sharing the same memory, with the operating system rapidly switching between them (preemptive⁷ multitasking — the OS can interrupt a thread at any moment). Threads

are lighter than processes, but sharing memory is exactly the danger: if two threads touch the same data at once, you get **race conditions**⁸ — bugs that depend on the precise, unpredictable timing of the switches. Defending against them means **locks**⁹, and locks bring their own miseries (forgotten locks corrupt data; over-eager locks cause **deadlocks**¹⁰ where two threads wait on each other forever). Threads are notoriously hard to get right at scale.

Python aside — the GIL. In the standard Python interpreter (CPython) there is a **Global Interpreter Lock**¹¹: only one thread runs Python code at a time, even on a many-core machine. So Python threads give you concurrency but not true parallelism for Python computation. They still help for I/O-bound work (a thread releases the lock while waiting on the network), but they do not let you crunch numbers on eight cores at once. This is a major reason the Python ecosystem leans on the third approach.

3. A single-threaded event loop. Use exactly **one** thread, and have it switch between tasks cooperatively¹² — each task voluntarily yields control whenever it would otherwise wait. While task A waits for the network, the one thread goes and runs task B; when A's data finally arrives, the thread comes back to A. Because there is only one thread, **there is no shared-memory race** — two pieces of your code never run at the same instant, so you almost never need locks. This is how `hathor-core` (via Twisted) handles its thousands of connections, and it is what the rest of this chapter builds.

PARALLELISM (threads on 2 cores) — truly simultaneous

core 1:  task A 
core 2:  task B 

CONCURRENCY (one thread, event loop) — interleaved, never simultaneous

core 1:  switches whenever a task would WAIT

The trade-off of the event-loop model is the one rule we will keep returning to: since everything shares one thread, **any task that refuses to yield — that blocks, or hogs the CPU — freezes the whole program.** Cooperation is mandatory. The whole machinery below exists to make that cooperation natural to write.

2.3 Callbacks — "call me back when it's ready"

The event-loop model needs a way for a task to say "I'm about to wait; here's what to do when the wait is over — go run something else until then." The oldest and most fundamental way to express that is the **callback**¹³: a function you hand to another function, to be called back later.

Callbacks rest on one idea that trips up newcomers from some other languages: in Python, **functions are values**. A function is an ordinary object you can store in a variable, put in a list, pass as an argument, and return from another function. This property is called being **first-class**¹⁴:

```
def shout(text):
    print(text.upper())

action = shout          # store a function in a variable (no parentheses – not calling it!)
action("hi")            # HI – call it through the variable

def do_twice(fn, value): # a function that TAKES a function...
    fn(value)
    fn(value)
do_twice(shout, "echo") # ECHO / ECHO
```

A function like `do_twice` that takes or returns other functions is a **higher-order function**¹⁵. The function you pass in (`shout`, here) is, when its purpose is "run this when something happens," a **callback**.

Now the asynchronous use. Instead of a blocking read that freezes until data is ready, imagine a non-blocking read that returns immediately and promises to call your function once the data arrives:

```
def read_async(name, on_done):
    # Pretend this registers interest and returns RIGHT AWAY.
    # Some time later, when bytes are ready, the event loop will call on_done(data).
    ...

read_async("peer.dat", on_done=lambda data: print("got:", data))
print("this line runs FIRST – the read hasn't finished yet")
```

The shift in shape is the whole point. The blocking version returns the result. The asynchronous version returns nothing useful immediately and instead arranges for your callback to receive the result later. The program flows past the read without waiting, free to do other work, and the callback fires when the data is genuinely ready. The result no longer comes back to where you asked for it — it arrives in a function you left behind. That inversion is the mental adjustment async demands.

Two senses of "callback." Some callbacks are synchronous — `do_twice` calls `shout` immediately, right now. The interesting ones here are asynchronous — handed over now, called much later by the event loop when an event occurs. Same mechanism (a function passed as a value), different timing.

2.4 The event loop — the engine that runs the callbacks

We keep saying "the event loop will call your callback." What is this loop? It is less magical than it sounds — at heart it is a `while` loop over a queue of work. Let's build a miniature one and watch it run, because once you have written an event loop, you never find them mysterious again.

```
from collections import deque

class MiniLoop:
    def __init__(self):
        self._ready = deque()          # a queue of callbacks waiting to run

    def call_soon(self, callback):      # schedule a callback to run later
        self._ready.append(callback)

    def run(self):
        while self._ready:             # while there is work to do...
            callback = self._ready.popleft() # take the next ready callback
            callback()                  # ...run it to completion, then loop
```

That is a working event loop. It does one thing forever: take the next ready callback, run it, repeat. The interesting behavior appears when callbacks schedule more callbacks — that is how a task makes progress in small steps, yielding the thread between each step:

```
loop = MiniLoop()

def greet(n):
    print("hello", n)
    if n < 3:
        loop.call_soon(lambda: greet(n + 1))    # do my next step later, not now

def count(n):
    print("count", n)
    if n < 3:
        loop.call_soon(lambda: count(n + 1))

loop.call_soon(lambda: greet(1))
loop.call_soon(lambda: count(1))
loop.run()
```

Trace the output and the lesson lands:

```
hello 1      ← greet(1) runs, schedules greet(2), returns
count 1      ← count(1) runs, schedules count(2), returns
hello 2      ← greet(2) runs, schedules greet(3), returns
count 2      ← count(2) runs, schedules count(3), returns
hello 3      ← greet(3) runs (n==3, schedules nothing)
count 3      ← count(3) runs, queue empties, loop ends
```

Two independent tasks — greeting and counting — made progress **interleaved, on one thread**, with no threads and no locks. Each callback did a little work, scheduled its continuation, and returned, handing the thread back to the loop so the other task

could run. A real event loop adds two things our toy lacks: **timers** (run this callback in 5 seconds — usually a list kept sorted by due-time) and **I/O readiness** (run this callback when this socket has data — the OS tells the loop which sockets are ready, via a system call like `select/epoll`). But the core — take the next ready callback, run it, repeat — is exactly what you just wrote.

Now the cardinal rule has an obvious cause. Look again at `callback()` in the loop: the loop runs **one callback at a time, to completion**, before it can touch the next. So if any callback blocks (`time.sleep(30)`, a blocking `recv`, a heavy ten-second computation), the loop is stuck inside it — every other task, every timer, every connection is frozen until that one callback returns. **Never block the loop.** Every callback must do a little and return fast.

2.5 Callback hell, and the rescue: futures

Callbacks work, but they have an ergonomics problem. Real tasks are sequences: read A, then read B, then read C, then combine them. Expressed with raw callbacks, each step nests inside the previous one's callback, and the code marches off the right edge of the screen:

```
read_async("a", lambda a:
    read_async("b", lambda b:
        read_async("c", lambda c:
            print(a, b, c))))    # the "pyramid of doom"
```

Add error handling and a loop and this becomes genuinely hard to read and reason about. The affectionate name is **callback hell**, or the pyramid of doom. Two inventions dig us out.

The first: the future (a.k.a. promise). Instead of passing a callback into the async function, have the async function hand you back an object that represents "a result that isn't ready yet." That object is a **future**¹⁶ (Python's `asyncio` calls it `Future`; JavaScript calls it a `Promise`; Twisted, as we'll see, calls it a `Deferred`). You attach callbacks to the future — "when you eventually have the value, call this" — and, more usefully, you can chain them so each step's output feeds the next:

```
future = read_async("a")                # returns a Future immediately
future.add_callback(lambda a: print("a is", a))  # run this once 'a' is ready
```

A future turns "the result arrives in a function I left behind" into "the result is an object I hold," which can be passed around, stored, and chained. It is a real improvement, but chaining `.add_callback(...).then(...)` still has visible plumbing. The second invention hides even that.

2.6 `async / await` — asynchronous code that reads top-to-bottom

The breakthrough is to let you write asynchronous code in the natural straight-line shape — read A, then B, then C — while it runs as cooperative callbacks under the hood. This is what `async / await` does, and it is built on a language feature called a **coroutine**¹⁷: a function that can **pause itself** partway through and be **resumed** later, right where it left off.

You define a coroutine with `async def`, and inside it you use `await` at each point where it would wait:

```
async def load_all():
    a = await read_async("a")    # pause here until 'a' is ready; let the loop run others
    b = await read_async("b")    # then pause here until 'b' is ready
    c = await read_async("c")
    print(a, b, c)              # all three ready, in order, no nesting
```

Read `await X` as: **"pause this coroutine here, hand the one thread back to the event loop so it can run other tasks, and resume me right here with the result once X is ready."** The pyramid of §2.5 is gone; the code looks sequential and reads top-to-bottom, but at every `await` the function quietly steps aside and lets the rest of the node run.

It is essential to understand what `await` does **not** do. It does not start a new thread, and it does not run things in parallel. `load_all` is still cooperative, single-threaded code: `await` is precisely the yield point where this task volunteers to step aside. Between two `await`s, your code runs without interruption (no other task can sneak in — that's why you rarely need locks). The event loop — now usually Python's built-in `asyncio` loop, the grown-up version of our `MiniLoop` — is what drives these coroutines, resuming each one when the thing it awaited is ready.

Same three reads, three notations, identical behavior on one thread:

blocking:	<code>a = recv()</code>	← freezes the whole program while waiting
callback:	<code>recv(on_done=...)</code>	← result arrives in a function you left behind
async/await:	<code>a = await recv()</code>	← pauses just THIS task; loop runs others; resumes here

So `async / await` is not a different model from callbacks — it is the same event-loop-and-callback model with a far nicer syntax. Under the hood, `await` suspends the coroutine and registers its resumption as, in effect, a callback on a future. You get the readability of straight-line code with the efficiency of cooperative concurrency.

2.7 The one rule, and the escape hatch for CPU work

Everything above reduces to a single operational rule, worth stating in bold because violating it is the most common way to break an event-loop program:

Never block the event loop. Inside any callback or coroutine, never do anything that waits without yielding and never do anything that hogs the CPU for long. Concretely: no `time.sleep()` (use the loop's async sleep, which yields); no blocking file or network calls (use the non-blocking/async versions); no ten-second number-crunching in one go. Each of these freezes every task, timer, and connection until it finishes, because — as §2.4 showed — the loop runs one callback at a time to completion.

But what about genuinely CPU-bound work that can't be made to yield — a heavy computation that must run start to finish? You cannot do it on the loop thread without freezing everything. The escape hatch is to push that work **off** the loop, onto a **thread pool**¹⁸ or process pool: a small set of worker threads kept ready for exactly this. The loop hands the heavy job to a worker, gets a future back, and stays free to serve everyone else; when the worker finishes, the loop resumes whoever was waiting. This is the one place the event-loop model deliberately reaches for threads — for the rare CPU-bound task, fenced off from the main loop. (Foreshadowing the Bridge: a node's proof-of-work hashing is exactly such a task, and `hathor-core` runs it in a dedicated pool.)

2.8 Bridge — asynchrony in `hathor-core`

Every concept in this chapter has a direct counterpart in the codebase. `hathor-core` does not use Python's `asyncio` directly as its primary engine; it is built on **Twisted**, a mature asynchronous framework with its own names for these same ideas — but they are the same ideas. Here is the map.

Bridge — async in the codebase (full treatment in the chapters named):

- **The event loop → the Twisted "reactor".** The single object our `MiniLoop` prototyped is, in production, the Twisted **reactor**: the one loop the whole node runs on. Starting the node ends with `reactor.run()` — handing the one thread to the loop forever. The reactor (and the option to swap in an `asyncio`-backed one) is **Chapter 16**, with the node's own wrapper in **Chapter 23**.
- **The future/promise → the Twisted "Deferred".** Twisted's name for "a result that isn't ready yet, with callbacks attached" is the **Deferred**. Everywhere the node does I/O you will see Deferreds returned and callbacks added to them — it is the §2.5 future under a different name — **Chapter 16**.
- **Callbacks everywhere.** A network connection in Twisted is an object whose methods (`dataReceived` , `connectionMade` , `connectionLost`) are callbacks the reactor invokes when those events occur — the §2.3 pattern as the backbone of the whole P2P layer — **Chapters 34-35**.
- **Timers.** The periodic jobs a node runs (checking sync state, refreshing peers) are scheduled on the reactor with Twisted's `LoopingCall` — the §2.4 "run this every N seconds" timer — **Chapters 29 & 34**.
- **The never-block rule, made visible.** Much of the codebase's shape — small methods that return quickly, work split into steps, the absence of `time.sleep` — is the §2.7 rule in practice. When you wonder "why is this broken into so many little callbacks?", the answer is almost always "so it never blocks the reactor."
- **The CPU escape hatch.** Proof-of-work hashing is CPU-bound, so the node runs it off the reactor in a dedicated thread pool (`pow_thread_pool`), exactly the §2.7 pattern — **Chapter 37**.

There is one more idea this chapter sets up without naming: once you accept that the node is a single thread dispatching callbacks, you need a clean way for parts of the system to announce events ("a new block arrived") so other parts can react — without everyone wiring callbacks to everyone else by hand. That is the **publish-subscribe** pattern, a named arrangement of callbacks, and it (with the other design patterns) is **Chapter 3**.

Recap

Concept	One-line definition	Why it matters for a node
I/O-bound	Work dominated by waiting, not computing	A node mostly waits; handling waiting well is everything
Blocking	A call that freezes the program until it returns	One blocking call stalls all peers — forbidden on the loop

Concept	One-line definition	Why it matters for a node
Concurrency vs parallelism	Interleaved progress vs. literally-simultaneous	The node is concurrent on one thread, not parallel
Threads + GIL	OS-switched, shared memory; one Python thread at a time	Races/locks are hard; the GIL blunts CPU parallelism
Event loop	<code>while</code> : take next ready callback, run it, repeat	The single engine the whole node runs on (the reactor)
Callback	A function handed over to be called on an event	How tasks say "do this when the wait ends"
First-class function	A function used as an ordinary value	The mechanism callbacks are built from
Future / Deferred	An object for "a result not ready yet"	Tames nested callbacks; Twisted calls it a Deferred
<code>async / await</code>	Straight-line syntax over cooperative callbacks	Readable <code>async</code> ; <code>await</code> = the yield point
Never block the loop	No sleeping/blocking/CPU-hogging in a callback	Violating it freezes the entire node
Thread pool	Workers for CPU-bound jobs, off the loop	Where proof-of-work hashing runs

Asynchronous programming is the answer to a node's defining condition: it spends its life waiting on a thousand slow conversations at once, and it cannot afford to freeze on any of them. The solution is one thread running an event loop, tasks that cooperate by yielding whenever they would wait, and callbacks (dressed up, at their best, as `async / await`) to resume each task when its wait is over. Hold the single rule above all — never block the loop — because it explains more of `hathor-core`'s design than any other principle. With Chapter 1's objects giving us things and this chapter giving us time, the next chapter assembles both into the recurring, named arrangements professionals reach for by reflex: the **design patterns**.

1. I/O-bound describes work whose speed is limited by input/output — network, disk, waiting on other systems — rather than by the processor. Most server software, including a node, is I/O-bound. ↩
2. CPU-bound describes work whose speed is limited by raw computation — the processor is the bottleneck. Examples: hashing, encryption, large numeric loops. ↩
3. A blocking call does not return control to your program until it has finished. While it waits, the calling thread can do nothing else. Its opposite is non-blocking (returns immediately, result delivered later). ↩

4. Concurrency is making progress on multiple tasks over the same period by interleaving them. It is about structure (dealing with many things at once), not necessarily simultaneity. ↩
5. Parallelism is executing multiple tasks at literally the same instant, which requires multiple processors/cores. All parallel programs are concurrent; not all concurrent programs are parallel. ↩
6. A thread is an independent sequence of execution within a process. Multiple threads in one process share the same memory, which makes communication easy and corruption easy. ↩
7. Preemptive multitasking lets the operating system suspend a thread at any moment to run another. The opposite is cooperative, where a task keeps running until it voluntarily yields. ↩
8. A race condition is a bug where the result depends on the unpredictable timing/order in which concurrent tasks run — e.g. two threads incrementing the same counter and losing an update. ↩
9. A lock (mutex) is a mechanism ensuring only one thread enters a critical section of code at a time, used to prevent race conditions. Misused locks cause deadlocks and slowdowns. ↩
10. A deadlock is a standstill where two or more threads each wait for a resource the other holds, so none can ever proceed. ↩
11. The Global Interpreter Lock is a mutex in standard CPython that allows only one thread to execute Python bytecode at a time. It simplifies the interpreter but prevents threads from running Python computation in parallel. ↩
12. Cooperative multitasking relies on each task voluntarily yielding control (e.g. at an `await`) so others can run. No task is forcibly interrupted, so a task that never yields blocks all others. ↩
13. A callback is a function passed to other code to be called ("called back") when a specific event happens or a result becomes available. ↩
14. A value is first-class if it can be stored in variables, passed as an argument, and returned from functions. In Python, functions are first-class values. ↩
15. A higher-order function is one that takes other functions as arguments and/or returns a function. `map`, `sorted(key=...)`, and decorators are examples. ↩
16. A future (a.k.a. promise) is an object representing a result that will be available later. You attach callbacks to it or `await` it. Twisted's version is the Deferred; JavaScript's is the Promise. ↩
17. A coroutine is a function that can suspend its execution (at an `await`) and later resume from exactly that point, preserving its local state. Defined with `async def` in Python. ↩
18. A thread pool is a managed set of reusable worker threads. Tasks are submitted to the pool rather than creating a new thread each time; used to run blocking or CPU-bound work off the main loop. ↩

Chapter 3 — Design Patterns

What you'll learn in this chapter

- What a **design pattern** is — a named, reusable solution to a recurring design problem — and what it is not.
- The three families: **creational** (how objects are made), **structural** (how objects are composed), **behavioral** (how objects interact).
- Seven patterns that run through `hathor-core`, each motivated and shown in a small example: **Factory, Builder, Singleton, Adapter, Facade, Proxy, Observer/Publish-Subscribe**.
- The trade-offs — including why one of these (Singleton) is considered a smell as often as a solution.
- A **bridge** mapping each pattern to the exact place it shapes the codebase.

Chapter 1 gave us objects; Chapter 2 gave us time. This chapter is about idioms: the handful of object arrangements that recur so often that engineers gave them names, so they can say "use a factory here" and be instantly understood. Knowing the names is half the battle when reading an unfamiliar codebase — the moment you recognize "ah, this is an Observer," a whole file's intent snaps into focus.

As with the previous two chapters, the body is plain-Python and codebase-agnostic; the closing Bridge (§3.9) maps each pattern onto the real `hathor-core` code you'll meet in Part II.

3.1 What a design pattern is (and isn't)

A **design pattern**¹ is a named, reusable solution to a problem that keeps coming up in software design. It is not code you copy and paste — it is a shape, a way of arranging objects, that you re-implement to fit each situation. The term was popularized in 1994 by a book whose four authors are universally nicknamed the "**Gang of Four**"²; the patterns in this chapter are drawn from that catalogue, chosen because they appear repeatedly in `hathor-core`.

Two things a pattern gives you. First, a **vocabulary**: "wrap it in an adapter," "the manager is a facade," "subscribe an observer" are sentences a team understands without further explanation. Second, a **tested shape**: each pattern is a solution others have already debugged, with known trade-offs, so you are not reinventing it from scratch.

And two cautions. A pattern is a means, not a goal — forcing a pattern where a plain function would do is a classic novice mistake ("pattern-itis"). And every pattern has a cost as well as a benefit; the good engineer reaches for one because its trade-off fits, not because it sounds sophisticated.

The Gang of Four sorted patterns into three families, which is how we'll group them:

CREATIONAL	→ how objects are CREATED	Factory, Builder, Singleton
STRUCTURAL	→ how objects are COMPOSED/wrapped	Adapter, Facade, Proxy
BEHAVIORAL	→ how objects INTERACT/communicate	Observer / Publish–Subscribe

3.2 Factory — centralize "which class to create"

The problem. Code that decides which kind of object to create tends to sprinkle that decision everywhere: `if kind == "circle": ... elif kind == "square": ...`, repeated in a dozen places. When you add a new kind, you must hunt down every one of those spots.

The pattern. A **factory**³ puts the creation decision in one place — a function or method whose job is "given some input, return the right object." Callers ask the factory for an object and stop caring which concrete class they got back (an echo of the abstraction idea from §1.7).

```
def make_shape(kind, **kw):                # the factory
    if kind == "circle":
        return Circle(kw["radius"])
    if kind == "square":
        return Square(kw["side"])
    raise ValueError(f"unknown shape: {kind}")

shape = make_shape("circle", radius=5)    # caller doesn't write Circle(...) itself
```

A common, cleaner variant replaces the `if`-ladder with a **registry** — a dictionary from name to class — so adding a type is one new entry, not a new branch:

```
SHAPES = {"circle": Circle, "square": Square}    # the registry

def make_shape(kind, **kw):
    try:
        return SHAPES[kind](**kw)
    except KeyError:
        raise ValueError(f"unknown shape: {kind}")
```

When to use it. Whenever creation logic is non-trivial or the concrete type depends on data (a string from a config file, a byte tag from the network). The payoff is that the rest of the program is insulated from the `if`-ladder: it asks for a shape and the factory decides the kind.

3.3 Builder — construct a complex object step by step

The problem. Some objects need a lot of parts, many of them optional. Cramming everything into one constructor gives you a monster:

`Pizza(size, crust, cheese, sauce, topping1, topping2, ...)` with a parade of arguments, half of them `None`. This is the telescoping constructor anti-pattern.

The pattern. A **builder**⁴ is a separate object that collects the parts one call at a time and assembles the final object only when you ask. It often returns `self` from each step so calls can be chained — a style called a **fluent interface**⁵:

```
class PizzaBuilder:
    def __init__(self):
        self._toppings = []
        self._size = "medium"

    def size(self, s):
        self._size = s
        return self          # return self → calls can be chained

    def add(self, topping):
        self._toppings.append(topping)
        return self

    def build(self):          # assemble the finished object, once
        return Pizza(self._size, self._toppings)

pizza = PizzaBuilder().size("large").add("cheese").add("mushroom").build()
```

When to use it. When constructing an object takes many steps or many optional pieces, especially when those pieces must be gathered from different places before the object can exist. The builder separates how a thing is assembled from what the thing is — and it is the pattern at the very center of how a node wires itself together (see the Bridge).

3.4 Singleton — one instance, shared everywhere (handle with care)

The problem. Sometimes exactly one of something should exist for the whole program — one configuration object, one connection to a shared resource — and every part of the code should reach the same one.

The pattern. A **singleton**⁶ ensures a class has only one instance and gives everyone a single access point to it. In Python the cleanest form is usually a module-level value, or a function that creates the instance on first call and caches it:

```
_settings = None

def get_settings():
    global _settings
    if _settings is None:
        _settings = Settings(load_from_disk())    # created once, on first call
    return _settings                             # every caller gets the SAME object
```

Why "handle with care." Singleton is the most debated pattern in the catalogue, because it is really **global state**⁷ in a smart coat. Globals make code harder to reason about (any function might secretly depend on or mutate the singleton) and harder to **test** (tests can't easily substitute a fake, and one test's changes leak into the next). Use it only for things that are genuinely single by nature — configuration, the event loop — and prefer passing dependencies in (a technique called dependency injection⁸, which the Builder of §3.3 enables) where you can. We flag this because you will meet singletons in `hathor-core` and should recognize both why they're used and why the codebase limits them.

3.5 Adapter — make an incompatible interface fit

The problem. You have an object that does what you need, but its method names or shape don't match what the surrounding code expects. The code wants `.read()`; the object offers `.fetch()`. You can't (or shouldn't) rewrite either side.

The pattern. An **adapter**⁹ is a thin wrapper¹⁰ that translates one interface into another, presenting the interface the caller expects and forwarding the work to the wrapped object:

```
class LegacySensor:                                # the thing you have (wrong interface)
    def fetch(self):
        return 42

class SensorAdapter:                                # presents the .read() interface the code expects
    def __init__(self, sensor):
        self._sensor = sensor
    def read(self):
        return self._sensor.fetch()                # translate read() → fetch()

reading = SensorAdapter(LegacySensor()).read()     # caller only ever sees .read()
```

When to use it. To bolt together two pieces of code that weren't designed to fit — a third-party library, a legacy class, or two of your own subsystems with mismatched conventions. The adapter localizes the mismatch in one small class, so the rest of the

program speaks one consistent interface. It is also how you make two interchangeable backends look identical to their callers — which is exactly how a node hides which event loop it runs on (see the Bridge).

3.6 Facade — one simple front over a complex subsystem

The problem. A subsystem¹² has many moving parts that must be operated in a particular order. Forcing every caller to know that dance — start the CPU, then spin up the disk, then load memory — spreads fragile knowledge everywhere.

The pattern. A facade¹¹ is a single object that offers a simple, high-level interface over a complicated subsystem, doing the intricate coordination behind one or a few easy methods:

```
class Computer:                                # the facade
    def __init__(self):
        self._cpu = CPU()
        self._disk = Disk()
        self._memory = Memory()

    def start(self):                             # one simple call hides the whole sequence
        self._cpu.boot()
        self._disk.spin_up()
        self._memory.load(self._disk.read_boot_sector())
        self._cpu.execute()

Computer().start()                             # callers say "start" and know nothing of the steps
```

When to use it. Whenever a cluster of objects must be coordinated and you want the rest of the program to depend on a simple front rather than the messy internals. The facade doesn't hide the subsystem (advanced callers can still reach the parts) — it provides an easy default path. A node's central coordinator is, at heart, a giant facade (see the Bridge).

3.7 Proxy — a stand-in that controls access

The problem. Sometimes you want to put something between a caller and an object — to delay the object's expensive creation until it's actually needed, to check permissions first, to add logging or caching — without the caller knowing anything changed.

The pattern. A proxy¹³ is a stand-in that implements the same interface as the real object and forwards calls to it, while inserting some control in between. A classic use is **lazy loading**¹⁴ — don't build the costly real object until the first real use:

```

class RealImage:
    def __init__(self, path):
        print(f"loading {path} from disk...") # expensive
        self._path = path
    def display(self):
        print(f"showing {self._path}")

class ImageProxy: # same interface (.display), controls access
    def __init__(self, path):
        self._path = path
        self._real = None
    def display(self):
        if self._real is None: # create the real one only when needed
            self._real = RealImage(self._path)
        self._real.display()

img = ImageProxy("big.png") # nothing loaded yet – cheap
img.display()               # NOW it loads, then displays

```

When to use it. For lazy creation, access control, caching, or remote stand-ins (an object that looks local but forwards to another machine). The proxy and the adapter both wrap — the difference is intent: an adapter changes the interface to fix a mismatch; a proxy keeps the interface and adds control.

3.8 Observer / Publish-Subscribe — announce events to interested parties

The problem. When something notable happens in one part of the program ("a new block was accepted"), several other parts need to react — update an index, notify a websocket client, refresh a metric. Wiring the announcer directly to each reactor (`self.index.update(...); self.websocket.notify(...); self.metrics.bump(...)`) welds them together: the announcer must know about every listener, and adding a listener means editing the announcer.

The pattern. The **observer**¹⁵ pattern — at scale, called **publish-subscribe**¹⁶ — inverts this. The announcer (the subject or publisher) keeps a list of interested parties (observers or subscribers) and notifies all of them when an event occurs, knowing nothing about who they are or what they'll do. Subscribers register a **callback** (Chapter 2!) to be run on each event:


```

class Subject:
    def __init__(self):
        self._subscribers = []

    def subscribe(self, callback):          # listeners register interest
        self._subscribers.append(callback)

    def publish(self, event):              # announce to all, knowing nothing about them
        for callback in self._subscribers:
            callback(event)

bus = Subject()
bus.subscribe(lambda e: print("index updates for", e))
bus.subscribe(lambda e: print("websocket notifies for", e))
bus.publish("new block")
# index updates for new block
# websocket notifies for new block

```

When to use it. Whenever you want to **decouple**¹⁷ event producers from event consumers so each can change independently. Add a new subscriber without touching the publisher; remove one without anyone noticing. This is one of the most consequential patterns in an event-driven system, and it is precisely how a node broadcasts state changes internally and to the outside world. Note how it rests directly on the two prior chapters: it is a list of callbacks (Ch 2) held by an object (Ch 1).

3.9 Bridge — patterns in `hathor-core`

Each pattern above has a home in the codebase. These are forward-pointers; each gets its full treatment in the chapter named.

Bridge — design patterns in the codebase (full treatment in the chapters named):

- **Builder.** The node assembles itself through a builder: `Builder` (tests/simulator) and `CliBuilder` (production) gather storage, indexes, wallet, consensus, and dozens more parts, then produce a wired `HathorManager`. This is the §3.3 pattern at the scale of the whole node — **Chapter 24** (and the `builder` footnote from Ch 1 closes here).
- **Factory.** Twisted builds a fresh protocol object per incoming connection from a `Factory`; the node also uses factories to produce nano-contract runners and to pick a sync agent by negotiated version — §3.2 — **Chapters 34 (protocol factory) & 39 (runner factory)**.
- **Singleton.** The global settings object is fetched through a single accessor (`get_global_settings`) and the reactor is effectively one-per-process — the §3.4 pattern, used sparingly and deliberately — **Chapters 22 (settings) & 16/23 (reactor)**.
- **Adapter.** The reactor wrapper presents one interface over either the Twisted or the asyncio event loop, and the serialization layer uses adapter objects to fit types to the wire format — §3.5 — **Chapters 23 (reactor) & 26 (serialization)**.
- **Facade.** `HathorManager` is a facade: one object exposing `start() / stop()` and high-level operations over storage, consensus, networking, mining, and the wallet — §3.6 at the largest scale — **Chapter 29**.
- **Proxy.** The nano-contract runtime uses proxy/accessor objects to mediate (and control) a contract's access to state; storage scope similarly gates what is reachable — §3.7 — **Chapter 39**.
- **Observer / Publish-Subscribe.** The `PubSubManager` lets internal components subscribe to events (`NEW_BLOCK_ACCEPTED`, ...), and the `EventManager` extends this to a durable, replayable stream for outside consumers — the §3.8 pattern as the node's nervous system, built on Ch 2's callbacks — **Chapter 30**.

Recap

Pattern	Family	One-line intent	Watch for
Factory	Creational	Centralize "which class to create"	the <code>if</code> -ladder or registry dict
Builder	Creational	Assemble a complex object step by step	<code>.build()</code> after chained setup
Singleton	Creational	One shared instance, one access point	hidden global state; test pain

Pattern	Family	One-line intent	Watch for
Adapter	Structural	Translate one interface into another	a wrapper that renames methods
Facade	Structural	One simple front over a complex subsystem	a <code>start()</code> that hides a sequence
Proxy	Structural	Same interface + access control / laziness	a stand-in created before the real one
Observer / Pub-Sub	Behavioral	Notify interested subscribers of events	<code>subscribe()</code> + a list of callbacks

Design patterns are the shared idioms of object-oriented work: named shapes, each a tested answer to a recurring problem, each with a trade-off to respect rather than a badge to collect. You now have the three layers Part II assumes — objects (Ch 1), time and callbacks (Ch 2), and the patterns that arrange them (Ch 3). When Chapter 24 shows a `CliBuilder` gathering parts and calling `build_manager()`, when Chapter 29 presents a `HathorManager` that fronts a dozen subsystems, when Chapter 30 wires a `PubSubManager` full of callbacks — you will not be meeting strangers, only familiar shapes wearing Hathor's names. The remaining concept chapters (wrappers/decorators in Ch 4, typing in Ch 5) round out the programming vocabulary before Track B turns to the blockchain ideas themselves.

1. A design pattern is a named, reusable solution to a commonly occurring problem in software design — a description of a shape to apply, not a finished piece of code. ↩
2. The "Gang of Four" (GoF) are the four authors of *Design Patterns: Elements of Reusable Object-Oriented Software* (1994), the book that catalogued and named 23 classic patterns and made the vocabulary standard. ↩
3. A factory is a function or method whose job is to create and return objects, centralizing the decision of which concrete class to instantiate. Variants include the factory function, factory method, and abstract factory. ↩
4. The builder pattern uses a separate object to construct a complex object step by step, deferring final assembly to a `build()` call. It avoids constructors with long, mostly-optional argument lists. ↩
5. A fluent interface is an API style where methods return the object itself so calls can be chained (`obj.a().b().c()`), reading almost like a sentence. ↩
6. A singleton is a class restricted to a single instance, with one global access point to it. Useful for genuinely-unique resources, but easy to misuse as disguised global state. ↩
7. Global state is data accessible from anywhere in the program. It makes code harder to reason about and test, because any function might read or change it invisibly. ↩
8. Dependency injection is the practice of passing an object its collaborators from outside (e.g. via its constructor) rather than having it create or fetch them itself. It makes dependencies explicit and substitutable (e.g. for testing). ↩

9. An adapter wraps an object to present a different interface than it natively offers, so it can be used where that other interface is expected. Also called a wrapper. ↩
10. A wrapper is any object that encloses another and forwards calls to it, usually adding or altering behavior. Adapters, proxies, and decorators are all kinds of wrapper (decorators are Chapter 4). ↩
11. A facade is an object that provides a simplified, high-level interface to a larger, more complex subsystem, hiding its coordination behind a few easy methods. ↩
12. A subsystem is a cohesive group of classes/modules that together provide some capability (e.g. "storage" or "networking"). A facade fronts a subsystem. ↩
13. A proxy is a stand-in object with the same interface as a real object, forwarding calls to it while inserting control such as lazy creation, access checks, caching, or remote communication. ↩
14. Lazy loading (lazy initialization) means deferring the creation of an expensive object until the first moment it is actually needed, rather than up front. ↩
15. The observer pattern lets an object (the subject) maintain a list of dependents (observers) and notify them automatically when its state changes, typically by calling a registered callback. ↩
16. Publish-subscribe is the observer pattern at system scale: publishers emit events to a channel/bus without knowing the subscribers, and subscribers receive events without knowing the publishers. Maximum decoupling. ↩
17. Decoupling means reducing how much two parts of a system need to know about each other, so each can change independently without breaking the other. ↩

Chapter 4 — Wrappers, Decorators, Shims & Dispatch

What you'll learn in this chapter

- The single idea behind all four words in the title: adding or routing behavior around existing code without editing that code.
- **Wrappers** and the **closures** that make them work.
- **Decorators** — Python's `@` syntax for wrapping a function — built from scratch, including `*args/**kwargs`, `functools.wraps`, and decorators that take arguments.
- **Shims** and **fallbacks** — small compatibility layers that bridge gaps between versions, platforms, or libraries.
- **Dispatch** — how a program routes a call to the right implementation, from polymorphism to dispatch tables to `singledispatch`.
- A **bridge** to where each appears in `hathor-core` (`@property`, `@inlineCallbacks`, the RocksDB/crypto wrappers, the reactor shim, message-type dispatch).

This chapter finishes the programming-vocabulary set. Everything in it is a variation on one move: don't change the thing — surround it, or route to it. That move shows up under four names — wrapper, decorator, shim, dispatch — and once you see them as facets of the same idea, a lot of otherwise-cryptic Python (`@something` lines, `try/except ImportError`, dictionaries of functions) becomes readable.

It builds directly on Chapter 2's "functions are first-class values" and "higher-order functions," and it closes loops opened earlier: the `@property` from §1.4 and the wrapper/adaptor/proxy of Chapter 3 all land here as special cases.

4.1 The unifying idea: behavior around behavior

You constantly need to add something to existing code — timing, logging, access checks, caching, retries — or to bridge code that doesn't quite line up, or to route a request to one of several handlers. The clumsy way is to edit the original code in every spot. The professional way is to leave the original alone and interpose new code around it, or to centralize the routing decision.

The four ideas in this chapter are all that move, at different scales:

WRAPPER	surround one object/function, forward the call, add behavior
DECORATOR	Python's @ syntax for wrapping a function (a wrapper with sugar)
SHIM	a small layer that bridges a gap (version, platform, library)
DISPATCH	route a call to the correct implementation among several

Wrappers and decorators add behavior; shims reconcile behavior; dispatch selects behavior. Keep that three-way split in mind and the rest of the chapter organizes itself.

4.2 Wrappers and closures

We met wrappers in Chapter 3: the adapter (changes an interface) and the proxy (controls access) are both objects that enclose another object and forward to it. The idea generalizes beyond objects — you can wrap a function too. Suppose you want to know how long some function takes, without editing the function:

```
import time

def timed(fn):
    # takes a function...
    def wrapper(*args, **kwargs):
        # ...and returns a NEW function around it
        start = time.perf_counter()
        result = fn(*args, **kwargs) # call the original, unchanged
        elapsed = time.perf_counter() - start
        print(f"{fn.__name__} took {elapsed:.4f}s")
        return result # hand back whatever the original returned
    return wrapper

def slow_add(a, b):
    time.sleep(0.1)
    return a + b

slow_add = timed(slow_add) # replace the name with the wrapped version
slow_add(2, 3)             # prints "slow_add took 0.1003s", returns 5
```

Two pieces of machinery are doing the heavy lifting here, and both deserve a name.

***args and **kwargs.** The wrapper doesn't know — and shouldn't care — what arguments the wrapped function takes. `*args`¹ captures any positional arguments as a tuple, and `**kwargs`² captures any keyword arguments as a dict; passing them on as `fn(*args, **kwargs)` forwards them faithfully. This is how one wrapper works for any function signature.

Closures. Notice that `wrapper` refers to `fn`, a variable from the enclosing `timed` function — and it still works after `timed` has returned. A function that "remembers" variables from the scope where it was defined is called a **closure**³. The inner `wrapper` closes over `fn`, carrying it along for the rest of its life. Closures are what make wrappers possible: the returned function keeps a private handle on the thing it wraps. (Closures are also, quietly, how the lambdas-as-callbacks in Chapter 2 captured their surrounding variables.)

The pattern `x = wrap(x)` — replace a name with a wrapped version of itself — is so common that Python gives it dedicated syntax. That syntax is the decorator.

4.3 Decorators — the `@` is just sugar

A **decorator**⁴ is a higher-order function that takes a function and returns a replacement (usually a wrapper around it), applied with the `@` symbol on the line above a definition. The `@` is pure **syntactic sugar**⁵ — a nicer spelling of something you could already write by hand. These two snippets are identical:

```
@timed                                # the decorator form
def slow_add(a, b):
    time.sleep(0.1)
    return a + b

# ...is exactly the same as:

def slow_add(a, b):
    time.sleep(0.1)
    return a + b
slow_add = timed(slow_add)             # the manual form from §4.2
```

That equivalence is the whole secret of decorators. `@timed` above `def slow_add` means "after defining `slow_add`, pass it through `timed` and rebind the name to the result." Nothing more mysterious than §4.2, with tidier syntax.

One refinement you'll always see: `functools.wraps`. When you wrap a function, the wrapper replaces it, so the original's name and docstring get lost — `slow_add.__name__` would become `"wrapper"`, which wrecks logs and debugging. `functools.wraps` is a tiny decorator you apply to your wrapper to copy that metadata across:

```
import functools

def timed(fn):
    @functools.wraps(fn)                # copy fn's name, docstring, etc. onto wrapper
    def wrapper(*args, **kwargs):
        ...
        return fn(*args, **kwargs)
    return wrapper
```

Treat `@functools.wraps(fn)` as boilerplate that means "make the wrapper still look like the original." You'll see it on nearly every well-written decorator.

Decorators that take arguments. Sometimes you want to configure the decorator itself, e.g. `@retry(times=3)`. That needs one more layer: a function that takes the arguments and returns a decorator. Read it as three nested steps — take the config, take the function, return the wrapper:

```

def retry(times):
    def decorator(fn):
        @functools.wraps(fn)
        def wrapper(*args, **kwargs):
            for attempt in range(times):
                try:
                    return fn(*args, **kwargs)
                except Exception:
                    if attempt == times - 1:
                        raise
            return wrapper
        return decorator

@retry(times=3)
def flaky_call():
    ...

```

1. takes config, returns a decorator
2. takes the function
3. the wrapper
retry() returns the actual decorator

You have already met decorators on classes. `@property` (§1.4) turns a method into a smart attribute; `@staticmethod`⁶ and `@classmethod`⁷ change what a method receives in place of `self`; `@abstractmethod` (§1.7) marks a method as required by an ABC. They are all just decorators — functions that transform the thing defined below them. Recognizing the `@` line as "transform the next definition" is the takeaway.

4.4 Shims and fallbacks

A **shim**⁸ is a small piece of code that sits transparently between two parts of a system to bridge a gap — most often a difference between versions, platforms, or libraries — so that the code on either side can stay unaware of the difference. The word comes from the thin wedge a carpenter slips in to make two mismatched parts fit.

The most common shim you'll see is the **import fallback**: try to use a fast or preferred library; if it isn't available, quietly substitute an alternative that offers the same interface, so the rest of the program never has to know which one it got.

```

try:
    import ujson as json
except ImportError:
    import json

# the rest of the file just uses json.loads(...) — oblivious to which it is

```

prefer the fast JSON library...
...but fall back to the standard one if absent

That `except`-branch is a **fallback**⁹: a safe alternative taken when the preferred path is unavailable or fails. Fallbacks are everywhere in dependable software — a default value when a lookup misses, a backup server when the primary is down, a slower-but-always-present implementation when an optimized one can't load.

Shim vs. adapter. They look similar — both interpose code — but the intent differs (a distinction worth holding). An **adapter** (§3.5) exists to reconcile a genuine interface mismatch by design, and is usually permanent. A **shim** exists to paper over an

environmental difference — an older runtime, a missing feature, a platform quirk — and is often temporary, meant to be removed once the gap closes. A close cousin from the web world is the **polyfill**¹⁰: code that supplies a missing feature so older environments behave like newer ones. All three are wrappers; what separates them is why they exist.

4.5 Dispatch — routing a call to the right code

Dispatch¹¹ is the act of deciding which piece of code runs for a given call. Every program dispatches constantly; the question is whether the decision is made for you or by you.

Dynamic dispatch — the kind you already use. When you write `vertex.verify()` and the right `verify` runs depending on whether `vertex` is a block or a transaction, that selection-by-type is **dynamic dispatch**¹² — and it is exactly the polymorphism of §1.6. Python makes this decision automatically at call time, based on the object's actual class. Most dispatch in object-oriented code is this invisible kind, and it is the cleanest: add a new type, give it the method, and existing calls route to it with no changes.

Manual dispatch — the dispatch table. Sometimes the thing you're routing on isn't an object's type but a value: a message's `"type"` field, a command name typed at a terminal, an event name. The naive form is a long `if/elif` ladder; the clean form is a **dispatch table**¹³ — a dictionary mapping each key to the function that handles it:

```
def handle_ping(msg): ...
def handle_block(msg): ...
def handle_tx(msg): ...

HANDLERS = {                                # the dispatch table
    "ping": handle_ping,
    "block": handle_block,
    "tx": handle_tx,
}

def dispatch(msg):
    handler = HANDLERS.get(msg["type"])
    if handler is None:
        raise ValueError(f"no handler for {msg['type']}")
    handler(msg)                             # route to the right function
```

This is the same registry idea as the factory of §3.2, and the same shape as the event loop of §2.4 (which dispatches each event to its callback). Adding a message type is one new dictionary entry — no editing of `dispatch` itself. A network protocol that receives many message kinds is, at its core, one big dispatch table.

Single dispatch by argument type. Python's `functools singledispatch`¹⁴ is a built-in that picks an implementation based on the type of the first argument — useful when you can't (or don't want to) put the behavior as a method on each type:

```
from functools import singledispatch

@singledispatch
def describe(item):
    return f"something: {item!r}"

@describe.register
def _(item: int):
    return f"the number {item}"

@describe.register
def _(item: list):
    return f"a list of {len(item)} items"
```

It is dispatch (selection of code by type) implemented with decorators (§4.3) and a registry (a table) — a neat convergence of this whole chapter's ideas.

4.6 Bridge — these in `hathor-core`

Every facet above is woven through the codebase. Forward-pointers only; full treatment in the chapters named.

Bridge — wrappers/decorators/shims/dispatch in the codebase:

- **Decorators on the model.** `@property`, `@classmethod`, and `@abstractmethod` appear throughout the vertex classes and the storage/index interfaces — §4.3 — **Chapters 25, 27-28.**
- **`@inlineCallbacks` — the async decorator.** Twisted's `@inlineCallbacks` decorator lets code use `yield` to await Deferreds in straight-line style (the Twisted forerunner of Chapter 2's `async / await`). You'll see it across the networking code — §4.3 + Ch 2 — **Chapters 16 & 34.**
- **Library wrappers.** `hathor/storage/` wraps the `python-rocksdb` library, `hathor/crypto/` wraps `cryptography`, and `hathor/pycoin/` wraps `pycoin` — each a wrapper presenting a node-shaped interface over a third-party tool — §4.2 — **Chapters 27 & 40.**
- **The reactor shim/adapter.** `hathor/reactor/` interposes a layer so the node can run on either the Twisted or an asyncio-backed reactor — a shim over the environment's event loop — §4.4 — **Chapter 23.**
- **Import/feature fallbacks.** Optional dependencies (e.g. Sentry error reporting) are guarded so the node runs with or without them — the §4.4 fallback — **Chapter 42.**
- **Dispatch tables.** The CLI routes subcommands through a name→module dictionary (you saw this in Chapter 0's `CliManager`); the P2P protocol routes incoming messages by type to handler methods; `PubSubManager` dispatches each event to its subscribers — §4.5 — **Chapters 21, 30, 34.**
- **Dynamic dispatch.** Verification and consensus call shared methods that resolve to each vertex type's own implementation — §4.5 / §1.6 — **Chapters 31-32.**

Recap

Idea	What it does	Built from	Tell-tale sign
Wrapper	Surrounds code, forwards, adds behavior	closures, <code>*args/**kwargs</code>	an inner function returning <code>fn(...)</code>
Closure	A function remembering its defining scope	nested functions	inner fn uses an outer variable
Decorator	<code>@</code> sugar for wrapping a function	higher-order fn + closure	a <code>@name</code> line above <code>def</code>

Idea	What it does	Built from	Tell-tale sign
<code>functools.wraps</code>	Keeps a wrapper looking like the original	a decorator	<code>@functools.wraps(fn)</code> boilerplate
Shim / fallback	Bridges a version/platform/library gap	<code>try/except</code> , wrappers	<code>try: import fast /</code> <code>except: import slow</code>
Dispatch table	Routes a value to its handler	a dict of functions	<code>HANDLERS[key](msg)</code>
Dynamic dispatch	Routes a call by object type	polymorphism (§1.6)	<code>obj.method()</code> picking by type
<code>singledispatch</code>	Routes by first-argument type	decorators + registry	<code>@fn.register</code>

These four words name one habit: interpose, don't rewrite. Wrappers and decorators add behavior around a function while leaving it untouched; shims and fallbacks reconcile code with an imperfect environment; dispatch chooses the right implementation, whether automatically (by type) or explicitly (by a table). With objects (Ch 1), time and callbacks (Ch 2), the patterns that arrange them (Ch 3), and now the wrapping-and-routing machinery (Ch 4), you can read the mechanics of almost any line in `hathor-core`. One programming-vocabulary chapter remains — **Chapter 5, type hints and static typing** — after which Track B turns from how the code is built to what it is about: blockchains, UTXOs, and the DAG.

1. `*args` in a function signature collects any extra positional arguments into a tuple. In a call, `*seq` unpacks a sequence into separate positional arguments. The name `args` is convention; the `*` is what matters. ↩
2. `**kwargs` collects any extra keyword arguments into a dict (`kw` = keyword). In a call, `**d` unpacks a dict into keyword arguments. Together `*args`, `**kwargs` forward an arbitrary argument list unchanged. ↩
3. A closure is a function that retains access to variables from the scope in which it was defined, even after that scope has finished executing. The inner function "closes over" those variables. ↩
4. A decorator is a callable that takes a function (or class) and returns a replacement, applied with `@name` above a definition. It is the standard Python way to wrap behavior around a function. ↩
5. Syntactic sugar is syntax that makes something easier to write or read but adds no new capability — it could always be expressed the longer way. `@decorator` is sugar for `f = decorator(f)`. ↩
6. `@staticmethod` marks a method that receives neither `self` nor the class — a plain function namespaced inside a class. Use it for helpers that logically belong to the class but need no instance. ↩

7. `@classmethod` marks a method whose first argument is the class itself (`cls`) rather than an instance. Common for alternative constructors (e.g. `Vertex.from_bytes(...)`). ↵
8. A shim is a small compatibility layer that transparently intercepts calls and bridges a difference (between API versions, platforms, or libraries), letting surrounding code stay unaware of the difference. ↵
9. A fallback is an alternative path taken when the preferred one is unavailable or fails — a default value, a backup service, or a simpler implementation. It makes code degrade gracefully instead of breaking. ↵
10. A polyfill is code (the term comes from web development) that implements a feature the current environment lacks, so older runtimes behave like newer ones. A kind of shim for missing features. ↵
11. Dispatch is the selection of which code to execute for a given call. It can be automatic (by object type) or explicit (by looking a key up in a table). ↵
12. Dynamic dispatch selects the implementation at run time based on an object's actual type — the mechanism behind method calls and polymorphism. Contrast static dispatch, decided at compile time by the declared type. ↵
13. A dispatch table is a data structure (usually a dict) mapping keys — message types, command names, event names — to the functions that handle them, replacing a long `if/elif` chain. ↵
14. `functools.singledispatch` is a Python standard-library decorator that turns a function into one whose implementation is chosen by the type of its first argument, with implementations registered via `@fn.register`. ↵

Chapter 5 — Type Hints & Static Typing

What you'll learn in this chapter

- The difference between **dynamic** and **static** typing, and the specific cost dynamic typing imposes on a large codebase.
- **Type hints**: the annotation syntax (`x: int`, `-> str`, `list[int]`, `int | None`), and the surprising fact that Python mostly ignores them at run time.
- The **static type checker** (mypy): a tool that proves type-correctness without running the code.
- The working vocabulary: `Optional` / `Union`, `Any`, generics and `TypeVar`, and **Protocols** — duck typing made checkable.
- Why a node-sized project pays the annotation cost, and the honest trade-offs.
- A **bridge** to typing in `hathor-core` (`types.py`, the mypy toolchain, the zope plugin, and Pydantic — where types do act at run time).

This is the last of the programming-concept chapters. It is shorter than the others because it adds a discipline on top of code you already understand, rather than a new runtime mechanism. But it is the discipline that makes a 60,000-line codebase navigable: when every function says what it takes and returns, and a tool checks that those promises hold, reading and changing the code becomes far safer. After this, Track B turns from how the code is built to what it is about — blockchains, UTXOs, the DAG.

5.1 Dynamic typing and its cost at scale

Python is **dynamically typed**¹: a variable has no fixed type, and the type of a value is checked only when the code actually runs. This is why you can write `x = 5` and later `x = "hello"` with no complaint, and why a function happily accepts any argument until something inside it fails. It makes Python fast to write and flexible — and it has a precise cost that grows with the size of the program.

The cost is that **type errors stay hidden until the exact line runs**. Consider:

```
def total_fees(transactions):  
    return sum(t.fee for t in transactions)
```

What is `transactions`? A list? Of what? What is `t.fee` — an int, a float, an object? Nothing here says. If a caller passes the wrong thing, you find out only when this line executes — perhaps in production, perhaps months later, on a rare branch. In a small script that is harmless. In a node with thousands of functions calling each other, "what type flows through here?" becomes a question you cannot answer by reading — only by running, and only for the paths you happen to exercise. Whole categories of bug (passing `None` where an object is expected, a string where a number is expected, a misspelled attribute) lurk until run time.

Type hints exist to drag those errors forward — out of "discovered at run time, maybe" and into "caught before the code ever runs."

5.2 Type hints: annotations that document and check

A **type hint** (or **annotation**²) is a label you attach to a variable, parameter, or return value, declaring the type it is expected to hold. The syntax is light:

```
def total_fees(transactions: list[Transaction]) -> int:    # takes a list of Transaction, returns  
    return sum(t.fee for t in transactions)  
  
count: int = 0                                           # a variable annotation  
name: str = "node-1"
```

Read `param: Type` as "this parameter should be a `Type`," and `-> Type` as "this function returns a `Type`." Containers are spelled with brackets: `list[int]` (a list of ints), `dict[str, int]` (a dict from str to int), `tuple[int, int]` (a pair of ints).

Here is the fact that surprises everyone: **Python does not enforce these at run time**. The annotations are, to the interpreter, almost decoration — it will not stop you calling `total_fees("oops")`. So what are they for? Two audiences:

1. **Humans**. The signature now documents itself, and — unlike a comment — the documentation lives in the code where it can be checked.
2. **Tools**. A separate program reads the annotations and verifies the code obeys them. That program is the static checker, and it is where the value comes from.

A note on "optional." Python's typing is gradual³: you can annotate as much or as little as you like, mixing typed and untyped code. You can add hints to one function and leave the rest bare. This lets a codebase adopt typing incrementally rather than all at once.

5.3 The static type checker (mypy)

A **static type checker** reads your annotated code and proves it type-consistent **without executing it** — analysing the source as text, following types from where they're produced to where they're used, and flagging any mismatch. "Static"⁴ means exactly this: analysis done ahead of time, on the code at rest, as opposed to dynamic checks that happen while it runs. The standard checker for Python, and the one `hathor-core` uses, is **mypy**⁵.

Give mypy this:

```
def greet(name: str) -> str:
    return "hi " + name

greet(42)          # we pass an int where a str is required
```

...and without ever running the program, mypy reports:

```
error: Argument 1 to "greet" has incompatible type "int"; expected "str"
```

That error would otherwise have waited for the line to execute. Multiply this across a whole codebase and you have a net that catches a large class of bugs the moment you save the file — typically wired into the editor and the continuous-integration⁶ pipeline so nothing untyped or mistyped slips through. This is **static analysis**⁷: learning facts about a program from its source without running it, of which type-checking is the most common form.

The mental model: hints are claims; mypy is the proof-checker. Writing `-> str` claims "this returns a string"; mypy verifies the claim against the actual `return` statements, and verifies every caller uses the result as a string.

5.4 The vocabulary of types

A handful of constructs cover most annotations you'll read in the codebase.

Optional / **T | None**. Code that may return "a value or nothing" is everywhere (a lookup that might miss). The type is "T or None," written `int | None` (modern syntax) or `Optional[int]` (older, identical meaning). The payoff: mypy then forces callers to handle the `None` case, eliminating the classic "called a method on None" crash.


```
def find_height(block_hash: str) -> int | None:    # an int, or None if not found
    ...

h = find_height(x)
print(h + 1)    # mypy ERROR: h might be None – handle it first
```

Union and Any. `Union[A, B]` (or `A | B`) means "either type." `Any`⁸ is the escape hatch: it means "any type, stop checking here." `Any` is occasionally necessary but is a hole in the net — every value flowing through `Any` is unchecked — so good codebases minimize it.

Generics and `TypeVar`. Some functions work for any type while preserving it: a function returning the first element of a list returns whatever the list holds. A **generic**⁹ expresses that with a **type variable**¹⁰:

```
from typing import TypeVar

T = TypeVar("T")

def first(items: list[T]) -> T:    # returns the SAME type the list contains
    return items[0]

first([1, 2, 3])    # mypy knows this is an int
first(["a", "b"])    # ...and this is a str
```

Protocols — duck typing, made checkable. Recall §1.6: Python doesn't care what class an object is, only whether it has the method you call (duck typing). A **Protocol**¹¹ lets you state that as a type — "anything with a `read()` returning bytes" — and have mypy check it, without the object inheriting anything:

```
from typing import Protocol

class Readable(Protocol):
    def read(self) -> bytes: ...    # the required shape

def consume(source: Readable) -> None:    # accepts ANY object with read() -> bytes
    data = source.read()
```

This is **structural typing**¹²: compatibility is decided by an object's shape (its methods), not its declared class — in contrast to **nominal typing**¹³, where it's decided by name/inheritance (an ABC, §1.7). A Protocol is the typed, checkable form of the duck typing you already met: same flexibility, now with a safety net.

5.5 Why a large codebase pays for typing

Annotations cost effort to write and maintain. A project the size of a node accepts that cost because the returns compound with scale:

- **Documentation that cannot rot.** A comment saying "returns a list of blocks" can drift out of date silently; a `-> list[Block]` annotation is verified every check, so it is always true.
- **Refactoring safety.** Change a function's return type and mypy instantly lists every caller that now breaks — turning a terrifying change into a checklist. This alone justifies typing in long-lived code.
- **Editor intelligence.** With types, an editor can offer accurate autocomplete¹⁴ and catch mistakes as you type, because it knows what each variable is.
- **Bugs caught early.** The whole class of "wrong type" and "forgot the None case" errors is caught before the code runs.

The honest trade-offs: annotations add visual noise, occasionally fight you on genuinely-dynamic code (where `Any` or a cast becomes necessary), and require discipline to keep the checker passing. The consensus in large Python projects — and the choice `hathor-core` makes — is that the safety is worth the friction. Typing is a linter¹⁵-grade tool elevated to a contract.

5.6 Bridge — typing in `hathor-core`

The codebase is thoroughly annotated and checked. Forward-pointers; full treatment in the chapters named.

Bridge — typing in the codebase:

- **mypy in the toolchain.** mypy runs over the whole codebase as part of the quality gate, alongside flake8 and isort — §5.3 — **Chapter 20**.
- **Domain type aliases.** `hathor/types.py` defines named aliases (e.g. for vertex IDs, addresses, amounts) so signatures read in domain terms rather than bare `bytes / int`; you'll see these everywhere in the model — §5.4 — **Chapter 25**.
- **The zope plugin.** Because the node uses `zope.interface` (an interface system predating Protocols), mypy runs with the `mypy-zope` plugin so those interfaces are type-checked too — §5.4 / §1.7 — **Chapters 16 & 20**.
- **Pydantic — where types act at run time.** `hathor-core` uses Pydantic for settings and some models. Pydantic reads the same type hints but, unlike plain Python, enforces them at run time — validating and converting incoming data. It is the one place the §5.2 "hints are ignored at run time" rule is deliberately overridden — §5.2 — **Chapters 18 & 22**.
- **Protocols & ABCs for backends.** The interchangeable storage/index backends (the §1.7 contract) are expressed with abstract classes and checked by mypy, so a backend that misses a method fails the check, not production — §5.4 — **Chapters 27-28**.

Recap

Concept	One-line definition	Why it matters
Dynamic typing	Types checked at run time only	Flexible, but errors hide until executed
Type hint	An annotation declaring expected type	Self-checking documentation
Static typing	Types checked ahead of time, on source	Bugs caught before the code runs
mypy	Python's static type checker	Proves the hints hold across the codebase
<code>T \ None / Optional</code>	"a value, or nothing"	Forces callers to handle the missing case
<code>Any</code>	"stop checking" escape hatch	Necessary sometimes; a hole in the net

Concept	One-line definition	Why it matters
Generic / <code>TypeVar</code>	One definition, type preserved	Reusable code without losing type info
Protocol	Required shape, checked structurally	Duck typing (§1.6) with a safety net
Pydantic	Hints enforced at run time	Validates external data using the same types

Type hints turn Python's run-time flexibility into a contract a tool can verify before the code ever runs. For a small script the discipline barely earns its keep; for a node it is load-bearing — it documents intent that cannot rot, makes large refactors safe, and catches a whole class of bugs at the keyboard instead of in production. You have now finished the programming-concepts track: objects (Ch 1), time and callbacks (Ch 2), the patterns that arrange them (Ch 3), the wrapping-and-routing machinery (Ch 4), and the typing discipline that holds it all together (Ch 5). The next chapters change subject entirely. Track B sets aside how Python works and takes up what a blockchain is — starting, in Chapter 6, with the problem of money without a bank.

1. Dynamic typing means variable types are not fixed and are checked while the program runs. A name can be bound to a value of any type, and type errors surface at execution time. Python, Ruby, and JavaScript are dynamically typed. ↩
2. A type annotation (type hint) is syntax attaching an expected type to a variable, parameter, or return value (`x: int` , `def f() -> str`). In Python it is informational by default — read by tools and humans, not enforced by the interpreter. ↩
3. Gradual typing allows mixing typed and untyped code in the same program, so annotations can be added incrementally rather than all at once. Python's type system is gradual. ↩
4. Static (in "static typing"/"static analysis") means "determined from the source ahead of run time," without executing the program. Its opposite is dynamic — determined while running. ↩
5. mypy is the standard static type checker for Python: a separate tool that reads type hints and reports inconsistencies without running the code. `hathor-core` runs it as part of its checks. ↩
6. Continuous integration (CI) is an automated pipeline that runs tests and checks (type-checking, linting) on every code change, so problems are caught before the change is merged. ↩
7. Static analysis is examining a program's source to learn facts about it (type errors, unused variables, security issues) without running it. Type-checking and linting are forms of static analysis. ↩
8. `Any` is the type that is compatible with everything and disables checking for the values it touches. It is the deliberate escape hatch from the type system; overusing it defeats the purpose of typing. ↩
9. A generic is a function or class parameterized by one or more types, so it works uniformly for many types while preserving them (e.g. a list that "remembers" it holds ints). ↩
10. A type variable (`TypeVar`) is a placeholder standing for "some specific type, the same throughout this signature," used to write generics. `def first(x: list[T]) -> T` returns the list's element type. ↩

11. A Protocol (PEP 544) defines a required set of methods/attributes; any object with that shape satisfies it, no inheritance needed. It is the statically-checkable form of duck typing. ↩
12. Structural typing decides type compatibility by an object's shape (which methods/attributes it has). "If it has the right methods, it fits." Protocols implement this. ↩
13. Nominal typing decides compatibility by declared name/inheritance — an object fits only if its class explicitly is (or subclasses) the required type. ABCs (§1.7) are nominal. ↩
14. Autocomplete is the editor feature that suggests valid attributes/methods as you type. Type information makes these suggestions accurate, because the editor knows each variable's type. ↩
15. A linter is a tool that flags suspicious or non-conforming code (style violations, likely bugs) by static analysis. flake8 is the linter in `hathor-core`'s toolchain; a type checker is a stricter cousin. ↩

Chapter 6 — What a Blockchain Is

What you'll learn in this chapter

- The one hard problem digital money must solve — the **double-spend problem** — and why it is hard only when there is no central authority.
- Why the obvious fix (a trusted company's database) is rejected, and what is gained and lost by removing it.
- The **distributed ledger** idea: everyone keeps a copy, so no one owns the truth.
- The cryptographic primitive the whole edifice rests on — the **hash function** — built up from its properties, not its math.
- How hashes turn a list of records into a **tamper-evident chain**, so altering old history is detectable.
- The membership problem (**Sybil attacks**) and how **proof-of-work** answers "who gets to add the next block."
- Why finality is **probabilistic**, not absolute — and a **bridge** to how Hathor's specific design (a DAG, not a chain) re-answers each of these questions.

Track A taught you how the code is built. Track B turns to what it is about. This chapter is the foundation of the foundation: every later domain chapter — UTXO (Ch 7), the DAG (Ch 8), proof-of-work and weight (Ch 9), voiding and finality (Ch 10) — is a refinement of an idea introduced here. We build it from a single question and follow the consequences.

The body is framework-agnostic: we reason about any system that wants to be digital money without a central authority. Where Hathor's answer differs from the classic blockchain answer — and in several places it differs sharply — the closing Bridge (§6.8) says so and points to the chapter that covers it. Read this chapter for the problem; read the rest of Track B for Hathor's solution.

6.1 The problem: spending the same coin twice

Start with physical cash. When you hand someone a £10 note, two things happen at once: they now have it, and you no longer do. The physical object cannot be in two places. You cannot spend the same note twice, because giving it away is the same act as losing it.

Now try to make digital money. A digital coin is data — a file, a number, a record. And data has a property cash does not: **it copies perfectly and for free**. If a coin is just a file, then "sending" it to Alice means sending her a copy, and you still have the original to send to Bob. You could spend the same coin a thousand times. This is the **double-spend problem**¹, and it is the central problem of digital money. Everything else — the hashing, the chains, the consensus, the mining — exists to solve it.

State it precisely, because the precision matters: the problem is not "can I copy the coin" (of course you can copy data); it is "**can everyone agree on which transfer was the real one**." If the whole world agrees that the coin went to Alice and that the transfer to Bob is invalid, then your copy is worthless — agreement is what makes it money. So the double-spend problem is really a problem of **agreement**: getting many independent parties to share one consistent history of who owns what.

6.2 The easy answer, and why it's rejected

There is a trivial solution, and you use it every day: **put one trusted party in charge of the ledger**. Your bank keeps a database of balances. When you pay Alice, the bank subtracts from your row and adds to hers, in one atomic step, and refuses to do it twice. Double-spending is impossible because there is exactly one authoritative copy of the truth and one party allowed to write to it. This is how essentially all conventional digital payment works.

So why would anyone want anything else? Because that single trusted party is also a single point of control and failure:

- It can **ensor** — refuse your transaction, freeze your account.
- It can **err or be corrupted** — a bug, an insider, or a hack rewrites balances.
- It is a **single target** — one breach, one outage, one coerced operator compromises everyone.
- It demands **trust** — you must believe the operator is honest and competent, forever.

The motivating goal of a cryptocurrency is to get the double-spend guarantee without anointing such a party — to build a ledger that **no single entity owns, controls, or can quietly rewrite**. That is a much harder problem, and the rest of this chapter is the shape of its solution. Note the trade-off up front, because it is the whole bargain: you remove the trusted party and gain censorship-resistance and resilience; you pay for it with enormous complexity and, as we'll see, with speed and with absolute certainty. Whether that trade is worth it is a judgment call, not a technical fact — but understanding the machinery requires taking the goal as given.

6.3 The distributed ledger: everyone holds a copy

If no one party can hold the ledger, the answer is that **everyone holds it**. A **distributed ledger**² is a record of all transactions that is replicated across many independent computers (the nodes³ — the thing `hathor-core` is), each keeping its own full copy, with no master.

This immediately reframes the double-spend problem. There is no longer one database to write to; there are thousands of copies that must somehow **stay in agreement** about the same history. If you could convince half the network the coin went to Alice and the other half it went to Bob, you'd have spent it twice. So the engineering problem becomes:

1. How does a new transaction get **broadcast** to everyone? (The peer-to-peer network — Ch 34.)
2. How can each node **independently verify** a transaction is legitimate, trusting no one? (Verification — Ch 31.)
3. How do all the copies **agree on one ordering** of history, including which of two conflicting transactions is the real one? (Consensus — Ch 32, and §6.6–6.7 here.)
4. How is the agreed history made **tamper-evident**, so no node can secretly rewrite its copy and pass it off as true? (Hashing — §6.4–6.5 here.)

We take these in the order a newcomer needs them: first the tamper-evidence (it's the cryptographic core), then the agreement (it's the subtle part). Both rest on one primitive, so we define that first.

6.4 The primitive: cryptographic hash functions

Almost everything in a blockchain is built from one tool, the **cryptographic hash function**⁴. You do not need its mathematics to understand the system — you need its properties. So we define it by what it does.

A hash function takes an input of any size — a word, a file, an entire transaction — and produces a fixed-size output, called the **hash** or **digest**⁵ (in the systems we care about, 256 bits, written as 64 hexadecimal characters). Think of it as a machine that reads any amount of data and stamps out a short, fixed-length fingerprint of it.

"hello"	→ [hash fn]	→ 2cf24dba5fb0a30e26e83b2ac5b9e29e...
"hello."	→ [hash fn]	→ e6f6a3d6... (utterly different)
<a 4 GB video>	→ [hash fn]	→ 9b2c8f01... (still just 64 hex chars)

Four properties make it useful, and each one earns its keep later:

1. **Deterministic.** The same input always yields the same hash. Hash "hello" on any machine, any day, and you get the identical 64 characters. This is what lets two nodes confirm they hold the same data: compare fingerprints, not the whole file.
2. **One-way (preimage resistance).** Given a hash, you cannot work backwards to recover the input. The fingerprint reveals nothing practical about what produced it. The only way to find an input with a given hash is to guess and check — which is exactly what mining will exploit (§6.6).
3. **Avalanche effect.** Change the input by one bit and the output changes completely and unpredictably — "hello" and "hello." share nothing in their hashes. There is no partial similarity, no gradient. This is what makes tampering detectable: you cannot make a small, hash-preserving edit.
4. **Collision resistance.** It is infeasible to find two different inputs that produce the same hash. So a hash is, in practice, a unique identifier for its input — if two things have the same hash, they are the same thing.

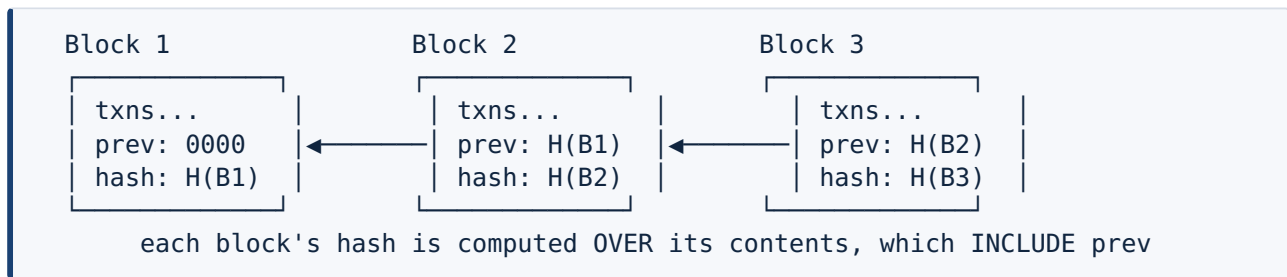
Put properties 3 and 4 together and you get the key consequence: **a hash is a tamper-evident fingerprint.** Publish the hash of a document, and anyone can later re-hash the document and check it matches; if even one character was altered, the hashes diverge and the tampering is exposed. We are about to use that to protect an entire history.

A hash is not encryption. A common confusion: encryption is reversible (with the key, you recover the original); hashing is one-way by design — there is no "unhash." A hash proves integrity ("this data is unchanged"); it does not hide data for later retrieval. Different tools, different jobs.

6.5 From records to a tamper-evident chain

Here is where "block" and "chain" finally enter, and where the classic design gets its name. We want an append-only history of transactions that anyone can verify and no one can secretly rewrite.

First, bundle transactions into **blocks**⁶ — batches of records grouped together, mostly so the network agrees on them a batch at a time rather than one transaction at a time. Now the trick: each block includes, as part of its own data, **the hash of the block before it.** That backward reference is a **hash pointer**⁷ — not just "the previous block is over there," but "the previous block had exactly this fingerprint."



Trace what this buys you. Suppose an attacker wants to alter one transaction in Block 1 — say, to erase a payment. The moment they change Block 1's contents, Block 1's hash changes (avalanche effect, §6.4). But Block 2 contains the old hash of Block 1 as its `prev` pointer — so Block 2 no longer points to the (now altered) Block 1; the link is visibly broken. To repair it, the attacker must update Block 2's `prev`, which changes Block 2's hash, which breaks Block 3's pointer, and so on. **One change anywhere forces re-writing every block after it.** The hash pointers weld the blocks into a structure where the most recent hash certifies the entire history behind it. This is a **hash chain**, and it is the literal "blockchain."

The chain makes tampering detectable and expensive-to-hide, but on its own it does not make tampering hard — re-computing a few hashes is fast. Two more pieces close the gap. First, the history is replicated (§6.3): to convince the network, you must alter not your copy but everyone's. Second — the heart of it — adding or rewriting a block is made deliberately, physically costly, so that out-running the honest network is infeasible. That cost is the consensus mechanism, and it answers the question we have been deferring: who gets to add the next block?

6.6 Who appends? Sybil attacks and proof-of-work

In a network with no central authority and open membership, anyone can join. That openness creates a specific attack. If "the majority decides which history is true" and votes are counted per participant, an attacker just creates a million fake participants and out-votes everyone. Cheaply manufacturing many identities to gain disproportionate influence is a **Sybil attack**⁸, and any open, permissionless system must defend against it. Counting noses doesn't work when noses are free.

The insight that cracked this — the founding idea of Bitcoin — is to make influence depend not on how many identities you have but on how much of some scarce, expensive resource you can prove you spent. If adding a block requires burning something costly, then controlling the history requires controlling a majority of that cost, and fake identities don't help — each fake still has to pay.

Proof-of-work⁹ is the classic choice of scarce resource: computation. To add a block, a participant (a **miner**¹⁰) must find a number — a **nonce**¹¹ — such that hashing the block together with that nonce produces a hash below some agreed **target**¹² (e.g. "the

hash must start with 20 zeros"). Because the hash is one-way and avalanche (§6.4), there is no clever way to find such a nonce — you can only **guess and check**, billions of times per second, until one works. That repeated guessing burns real electricity and time. The result is asymmetric in exactly the useful way:

- **Finding** a valid nonce is hard — it takes the whole network many guesses (tuned, by adjusting the target, to one block every so often).
- **Checking** a claimed nonce is trivial — hash once, compare to the target. Every node verifies a miner's work instantly.

So a miner who finds a valid block has demonstrably spent a large amount of computation, and anyone can confirm it for free. The hash below the target is the proof of the work. Now rewriting history has a price: to alter Block 1 and re-thread the chain (§6.5), an attacker must redo the proof-of-work for Block 1 and every block after it, faster than the honest network extends the chain ahead — which requires out-computing the rest of the world combined. The append-rule becomes "**the history with the most accumulated work wins**," and work cannot be faked, only paid for.

Why "the most work," not "the longest"? Loosely, people say "longest chain wins," but the precise rule is most cumulative work, because the difficulty target changes over time — a shorter chain of harder blocks can represent more total computation than a longer chain of easy ones. Hold this distinction; Hathor takes it much further by measuring work directly as weight (Ch 9).

Proof-of-work is not the only answer to the Sybil problem — **proof-of-stake** makes the scarce resource money at risk rather than computation, and **proof-of-authority** drops openness entirely in favor of a known set of signers (Hathor supports this last one for private networks — Ch 32). But proof-of-work is the original, and it is the one whose vocabulary — nonce, target, mining, difficulty — runs through every full node.

6.7 Finality is probabilistic

One consequence of "most-work-wins" surprises everyone at first: in this design, a transaction is **never absolutely final**. It is only ever increasingly unlikely to be reversed.

Here is why. Suppose two miners find a block at nearly the same moment, each extending the chain differently. The network temporarily splits — some nodes build on one, some on the other. This is a **fork**¹³. The tie breaks when the next block lands on one side, making that branch carry more work; the heavier branch wins and the other is abandoned. Any transaction that was only in the abandoned branch is undone — rolled back — an event called a **reorganization**, or **reorg**¹⁴.

So a transaction one block deep could still be reversed by a short reorg. But each additional block built on top of it is more work an attacker would have to out-pace to reverse it — so the probability of reversal shrinks exponentially with **depth**, the number of blocks piled on after it. This is **probabilistic finality**¹⁵ : you never get a mathematical guarantee that a payment is permanent, only ever-stronger confidence. (This is why exchanges "wait for N confirmations" — they are waiting for enough depth that a reversal is effectively impossible.)

To stop deep reorgs outright — and to protect against certain attacks on young chains — systems often add **checkpoints**¹⁶ : specific (height, hash) anchors, baked into the software, declaring "this block at this height is final by decree; no reorg may go behind it." Checkpoints trade a little of the "no authority" purity for a hard floor under history. Hathor uses them (Ch 10, Ch 32).

That is the complete classic picture: a replicated, hash-chained ledger, extended by whoever proves the most work, with conflicts resolved by most-work-wins and finality that hardens with depth. Every piece answers part of the double-spend problem from §6.1. Hathor keeps the goals and changes several of the mechanisms — which is the bridge out of this chapter.

6.8 Bridge — how Hathor answers these questions

Hathor is a cryptocurrency, so it must solve every problem in this chapter. But its central structural choice is different from the classic blockchain: it does not put transactions inside blocks on a single chain. Each forward-pointer tells you where Hathor's answer is detailed.

Bridge — the classic design vs. Hathor (full treatment in the chapters named):

- **Not a chain — a DAG.** Hathor's biggest divergence: ordinary transactions are not bundled into blocks on one line; they are themselves nodes in a **directed acyclic graph**, each linking to earlier transactions. Blocks still exist (they mint coins and pin down ordering), but the ledger is a graph, not a chain. The codebase calls any node of that graph a vertex. This re-answers §6.5 entirely — **Chapter 8**.
- **Two kinds of link.** Where a classic block had one `prev` pointer, a Hathor vertex has two distinct kinds of edge: **parents** (the DAG/confirmation topology, the analogue of §6.5's hash pointers) and **inputs** (which earlier outputs this transaction spends, the double-spend-relevant edge). Keeping these straight is half of understanding the model — **Chapters 8 & 25**.
- **Ownership by UTXO, not balances.** Hathor tracks coins as discrete **unspent transaction outputs**, not as account balances — there is no "balance" field anywhere; your balance is the sum of outputs you can spend. This is how §6.1's "who owns what" is actually recorded — **Chapter 7**.
- **Work measured as weight.** Hathor expresses proof-of-work not as a target to fall under but as a **weight** — a number that is the base-2 logarithm of the work done. "Most work wins" becomes "most **accumulated weight** wins," computed across the DAG. This is §6.6's append-rule, recast — **Chapter 9**.
- **Conflicts resolved by voiding.** When two transactions spend the same output (the double-spend of §6.1, made concrete), Hathor marks the loser **voided** — a first-class metadata flag — and propagates that voiding to everything built on it. This is §6.7's reorg/conflict resolution, generalized to a graph — **Chapters 10 & 32**.
- **Proof-of-authority too.** For private networks, Hathor can drop proof-of-work entirely and use a fixed set of authorized signers (§6.6's third option) — **Chapter 32**.
- **Checkpoints.** Hathor anchors history with (height, hash) checkpoints from its settings profile (the §6.7 mechanism) — recalled from **Chapter 22**, used in **Chapters 10 & 32**.

Recap

Problem (this chapter)	Classic answer	Where Hathor's answer lives
Double-spend (§6.1)	one agreed history	the DAG + voiding — Ch 8, 10
No trusted party (§6.2)	replicate the ledger	the P2P network — Ch 34

Problem (this chapter)	Classic answer	Where Hathor's answer lives
Keep copies honest (§6.5)	hash-chain, tamper-evident	parents/hash pointers — Ch 8, 25
Who appends? (§6.6)	proof-of-work, most-work-wins	weight & accumulated weight — Ch 9
Sybil resistance (§6.6)	cost a scarce resource	PoW / PoA — Ch 9, 32
Which conflict wins? (§6.7)	heaviest branch	voiding by accumulated weight — Ch 32
Is it final? (§6.7)	probabilistic, by depth	+ checkpoints — Ch 10, 32
Who owns what? (§6.1)	a ledger of transfers	UTXO model — Ch 7

A blockchain is the answer to one question — how do strangers agree on who owns what, with no one in charge — assembled from a few parts: replicate the ledger so no one owns it, hash-chain it so no one can secretly rewrite it, and make appending it cost a scarce resource so no one can cheaply out-vote or out-run the honest majority. The price is complexity, latency, and finality that is only ever probabilistic. Hold the problem firmly, because Hathor keeps the problem and rebuilds the solution: the next chapter replaces "a ledger of balances" with the **UTXO model** — the precise way Hathor records who owns what — and Chapter 8 replaces "a chain of blocks" with the **DAG of vertices** that gives this project its shape.

1. The double-spend problem is the risk that a holder of digital money spends the same unit more than once by sending copies of it to different recipients. Solving it — getting everyone to agree which spend is valid — is the core problem of digital currency. ↩
2. A distributed ledger is a transaction record replicated across many independent computers, each holding a full copy, with no central master. Agreement among the copies replaces trust in a single operator. ↩
3. A node is one computer participating in the network, running the protocol software and (for a full node) keeping and verifying a complete copy of the ledger. **hathor-core** is a full-node implementation. ↩
4. A cryptographic hash function maps input of any size to a fixed-size output (the hash), with properties — determinism, one-wayness, avalanche, collision-resistance — that make it usable as a tamper-evident fingerprint. Examples: SHA-256, SHA-3. ↩
5. A digest (or hash) is the fixed-size output of a hash function — here, 256 bits, usually written as 64 hexadecimal characters. It acts as a compact fingerprint of the input. ↩
6. A block is a batch of transactions grouped and added to the ledger together. In classic blockchains, blocks are the units that form the chain; in Hathor, blocks coexist with free-standing transactions in a graph (Ch 8). ↩

7. A hash pointer is a reference to a piece of data that includes that data's hash, so the reference both locates the data and certifies it is unchanged. Chaining blocks by hash pointer is what makes the history tamper-evident. ↩
8. A Sybil attack is the creation of many fake identities by one actor to gain disproportionate influence in a system that counts participants. Open, permissionless networks must make influence cost a scarce resource to resist it. (Named after a case study of a person with many identities.) ↩
9. Proof-of-work is a scheme where adding a block requires finding a value (the nonce) that makes the block's hash meet a hard target — costly to produce, trivial to verify. The work spent is the proof, and it makes rewriting history expensive. ↩
10. A miner is a participant that performs proof-of-work — repeatedly hashing candidate blocks to find a valid nonce — in exchange for a reward (newly minted coins and/or fees) when it succeeds. ↩
11. A nonce ("number used once") is the variable a miner changes on each attempt so that re-hashing the block yields a different hash, searching for one that meets the target. ↩
12. A target is the threshold a block's hash must fall below to be valid. A lower target means fewer valid hashes, so more guessing — i.e. higher difficulty. Hathor expresses the same idea as a weight instead (Ch 9). ↩
13. A fork is a temporary split in the ledger where different nodes hold different competing versions of recent history, usually because two valid blocks appeared at nearly the same time. It resolves when one branch accumulates more work. ↩
14. A reorganization (reorg) is when a node abandons one branch of history for a competing branch that turned out to carry more work, undoing any transactions that existed only in the abandoned branch. ↩
15. Finality is the assurance that a transaction cannot be reversed. In proof-of-work systems it is probabilistic: never absolute, but exponentially more certain as more blocks are built on top (greater depth). ↩
16. A checkpoint is a hard-coded (height, hash) anchor declaring a particular block final by decree, so no reorganization may rewrite history before it. It bounds how deep a reorg can go, trading a little decentralization for safety. ↩

Chapter 7 — The UTXO Model

What you'll learn in this chapter

- The two ways a ledger can record ownership — the **account model** and the **UTXO model** — and the trade-offs that lead a coin to choose one.
- What an **unspent transaction output (UTXO)** is, and why a balance is something you compute, not something stored.
- How a transaction **consumes** inputs and **creates** outputs, why inputs are spent whole (hence **change**), and the conservation rule that inputs must cover outputs.
- How **locking and unlocking scripts** decide who may spend an output (ownership without a stored "owner" field).
- Hathor's specifics: the TxInput / TxOutput classes, custom **tokens** riding the same model, and **authority outputs**.
- A **bridge** to where the model is implemented and indexed in the codebase.

Chapter 6 left one question open: when the ledger says "who owns what," what is actually written down? There are two answers in wide use, and they lead to different code, different bugs, and different mental models. Hathor uses the one Bitcoin pioneered — the **UTXO model** — and you cannot read the transaction code, the verification rules, or the indexes without it. This chapter builds it from the account model you already intuitively hold, so the contrast does the teaching.

Track B may cite Hathor code, and this chapter does, lightly — but the model is general, so we develop it abstractly first and show Hathor's spelling at the end of each idea. The full code tour of the vertex classes is Chapter 25; here we want the concept solid.

7.1 The intuitive model — and why it isn't the only one

Ask anyone how a bank account works and they'll describe the **account model**¹: there is a row somewhere with your name and a number — your balance — and a payment subtracts from your number and adds to the recipient's. The ledger is a table of (account → balance), and a transaction is an edit to two cells.

ACCOUNT MODEL (e.g. a bank, or Ethereum)

Alice	100
Bob	50

pay Bob 30



Alice 70
Bob 80

(two cells edited, atomically)

This is natural and compact, and some major systems (Ethereum, most banks) use exactly it. But notice what it requires: a single mutable cell per account that everyone must update consistently, and an ordering — you must apply "Alice pays Bob" before "Bob pays Carol," or the balances go wrong. In a distributed setting with no central clock (Chapter 6), shared mutable cells and strict ordering are precisely the hard things. There is another way to record ownership that sidesteps both, and it is the one Hathor uses.

7.2 The UTXO model: money as discrete coins

The **UTXO model**² records ownership not as balances but as a collection of distinct, indivisible chunks of value, each created by some past transaction and not yet spent. UTXO stands for **Unspent Transaction Output**: an output of a transaction that no later transaction has consumed.

The mental shift is from a balance to a wallet full of coins of arbitrary denominations. You don't have "a balance of 70"; you have, say, a 50-coin and a 20-coin — specific objects, each traceable to the transaction that made it. Your balance is just what you get when you add up the coins you can spend:

Balance is derived, not stored. In a UTXO system there is no balance field anywhere. To know how much you have, you find every unspent output payable to you and sum their values. This is a defining property — and in Hathor it is literally true: there is no balance field on any vertex or address; balance is computed from unspent outputs (we cite this in §7.6).

A coin/output, conceptually, holds two things: **a value** (how much) and **a condition that says who may spend it** (the lock — §7.5). That's it. The ledger as a whole is the set of all outputs ever created, partitioned into spent and unspent. The unspent ones — the UTXOs — are the live money.

UTXO MODEL – Alice's "balance" is her unspent outputs

output A: value 50, locked to Alice	●
output B: value 20, locked to Alice	●
output C: value 50, locked to Bob	●
output D: value 30, locked to Alice	x

● = unspent (a UTXO)
x = already spent

Alice's balance = 50 + 20 = 70 (D doesn't count; it's spent)

7.3 How a transaction moves money: consume and create

A transaction in the UTXO model does exactly two things: it **consumes** some existing unspent outputs (as its inputs) and **creates** new outputs. An **input**³ is not a fresh piece of data — it is a pointer to an earlier output, saying "I am spending that specific coin." The outputs are the new coins, with new locks naming new owners.

Walk a concrete payment. Alice has the 50-output and the 20-output from §7.2 and wants to pay Bob 30:

TRANSACTION "Alice pays Bob 30"

INPUTS (coins consumed)

← output A (50)
← output B (20)

in: 70

OUTPUTS (coins created)

value 30 → locked to Bob	(Bob's new coin)
value 40 → locked to Alice	(Alice's CHANGE)

out: 30 + 40 = 70

Three rules fall out of this picture, and each one matters in the verification code (Ch 31):

1. Outputs are spent whole — hence change. An output is indivisible: you cannot spend "30 of a 50-coin." To pay 30, Alice must consume whole outputs summing to at least 30, then send the remainder back to herself as a second output. That returned remainder is **change**⁴ — exactly like getting coins back when you pay for a £7 item with a £10 note. This is why a simple payment usually has two outputs: one to the payee, one of change to the payer.

2. Inputs must cover outputs — conservation of value. The sum of input values must be at least the sum of output values; money cannot be created from nothing. (Any shortfall between inputs and outputs is, in many systems, a fee to the miner.) The one exception is the special transaction that mints new coins — in a blockchain, the block reward — which has outputs with no corresponding inputs, by protocol rule. In Hathor the genesis and block rewards are these minting points (Ch 9, Ch 22).

3. An output can be spent only once. The instant an output is used as some transaction's input, it ceases to be a UTXO. If two transactions both try to consume the same output, that is precisely the **double-spend** of Chapter 6, made concrete and local: two inputs pointing at one output. The whole consensus machinery (Ch 10, Ch 32) exists to ensure exactly one of them wins. This is the deep reason the UTXO model is attractive for a distributed ledger: a double-spend is not a subtle balance inconsistency spread across a table — it is a visible collision, two inputs naming the same prior output, easy to detect.

7.4 Why UTXO suits a distributed ledger

Now the trade-off, stated plainly, against the account model of §7.1.

What UTXO gives up: **simplicity and compactness**. Tracking a pile of discrete coins is more bookkeeping than editing one balance cell; wallets must select which coins to spend, manage change, and stitch a balance together from fragments. The account model is easier to think about for everyday "what's my balance."

What UTXO buys, and why a coin pays that price:

- **Double-spends are local and explicit.** As above: a conflict is two inputs citing one output — detectable by looking at the transactions themselves, with no global balance state to reconcile.
- **No shared mutable cell.** Outputs are created once and never modified — only consumed. There is no balance cell that every transaction races to update. Immutable-once-created data is far friendlier to a system where thousands of nodes apply transactions in slightly different orders (recall the appeal of immutability from Chapter 1 and the functional style from Chapter 2).
- **Parallelism and provability.** Independent transactions touch independent outputs, so they don't contend; and each coin carries its own provenance back through the outputs that made it.

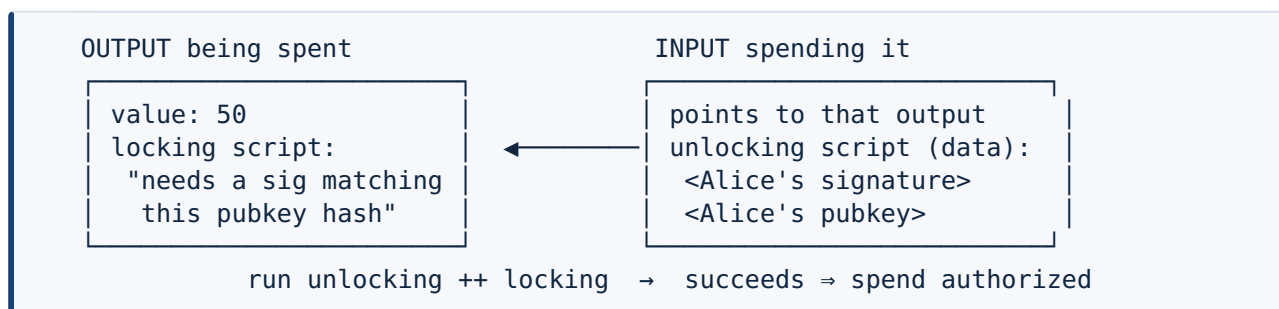
For a DAG-based ledger like Hathor's, where there isn't even a single linear order of transactions to lean on (Chapter 8), the "no shared mutable cell, conflicts are visible collisions" properties are decisive. UTXO and the DAG fit together; that is not a coincidence.

7.5 Ownership without an owner field: scripts

We keep saying an output is "locked to Alice." But there is no `owner = "Alice"` field. Ownership in the UTXO model is enforced by a small program, and this is one of the cleverest ideas in the design — worth meeting now because it reframes what "owning a coin" means.

Each output carries a **locking script**⁵ (also called a `scriptPubKey` in Bitcoin): a condition, expressed in a tiny stack-based scripting language, that must be satisfied to spend the output. The typical condition is "provide a digital signature⁷ from the private key behind this public-key hash." Each input that spends an output carries an **unlocking script**⁶ (the `data` field) supplying the missing pieces — typically the signature and public key.

To validate a spend, the node runs the unlocking script and the locking script together; if the combined program succeeds, the spend is authorized. So "Alice owns this coin" really means **"this output's locking script can only be satisfied by someone holding Alice's private key."** Ownership is the ability to produce a satisfying input, nothing more.



This indirection is what lets the same model express more than simple payments: multi-signature outputs ("needs 2 of these 3 signatures"), time locks ("not spendable until block height H"), and more — all by varying the locking script. The full script language and its evaluation are part of the vertex/verification story (Ch 25, Ch 31); here, hold the principle: **ownership is a satisfiable lock, not a name in a field.**

7.6 Hathor's spelling: inputs, outputs, tokens, authorities

Now the concrete code, briefly — the full tour is Chapter 25. In `hathor/transaction/base_transaction.py`, the two classes are exactly the model:

- **`TxInput`** (`base_transaction.py:936`) holds `tx_id` (the hash of the transaction whose output is being spent), `index` (which output, by position), and `data` (the unlocking script). That `(tx_id, index)` pair is the pointer-to-an-earlier-output from §7.3.

- **TxOutput** (`base_transaction.py:1022`) holds `value` , a `script` (the locking script of \$7.5), and `token_data` (next paragraph). No owner field, exactly as predicted.

Two Hathon-specific extensions are worth naming now so they're not a surprise later:

Custom tokens on the same model. Hathon is not single-currency. Besides the native token **HTR** (whose UID is the single byte `b'\x00'` , `conf/settings.py:29`), users create their own tokens, and they ride the same UTXO machinery. The output's `token_data` field encodes which token this output is denominated in (an index into the transaction's token list). So one output might be 50 HTR and another 1000 of some custom token, both unspent outputs in the same ledger. A token is created by a special transaction whose own hash becomes the token's UID (`TokenCreationTransaction` , Ch 25).

Authority outputs. Some outputs carry no spendable value but instead a permission — the right to mint more of a token, or to melt (destroy) it. These are **authority outputs**, flagged within the same `token_data` byte. They are how a token issuer retains control over supply, expressed in the very same output mechanism — a coin that grants a power rather than holding a value.

Finally, the derived-balance claim from §7.2, made real: the node maintains a **UTXO index** (`hathon/indexes/utxo_index.py`) keyed by (`address` , `token_uid`) , so that "what can this address spend" is answerable quickly. An output is recorded as spent not by deleting it but by noting, in metadata, which transaction spent it (`get_output_spent_by`) — which is what lets a voided transaction (Ch 10) cleanly reverse a spend and return the coin to the unspent set. We meet that index for real in Chapter 28.

Recap

Idea	Account model	UTXO model (Hathon)
Ownership recorded as	a mutable balance cell	a set of unspent outputs
Your balance	stored, read directly	computed by summing your UTXOs
A payment	edit two balance cells	consume inputs, create outputs
Spending part of a coin	adjust the number	impossible — spend whole, return change
A double-spend	a balance inconsistency	a visible collision: two inputs, one output
Who may spend	an <code>owner</code> field	a satisfiable locking script

Idea	Account model	UTXO model (Hathor)
Hathor classes	—	<code>TxInput</code> (:936), <code>TxOutput</code> (:1022)
Multiple currencies	—	<code>token_data</code> on each output; HTR = <code>b'\x00'</code>

The UTXO model records money as a heap of discrete, unspent, individually-locked outputs, where your balance is a sum you compute and a payment is the act of consuming whole coins and minting new ones with new locks. It costs more bookkeeping than account balances and buys, in return, exactly the properties a distributed ledger needs: outputs are immutable once created, there is no shared balance cell to race on, and a double-spend is a plainly visible collision rather than a hidden inconsistency. That last property is the hinge between this chapter and the next two: it is because spends are explicit pointers to prior outputs that Hathor can arrange transactions not in a single chain but in a graph — the **DAG** of Chapter 8 — and resolve the inevitable collisions by **voiding** (Chapter 10). With ownership now precisely defined, the next chapter rebuilds the structure of the ledger itself.

-
1. The account model (or account/balance model) records ownership as a mutable balance per account; a transaction debits one balance and credits another. Used by banks and by Ethereum. Contrast the UTXO model. ↩
 2. The UTXO model (Unspent Transaction Output) records ownership as a set of discrete outputs created by past transactions and not yet spent. A balance is the sum of one's unspent outputs. Used by Bitcoin and Hathor. ↩
 3. An input of a transaction is a reference to a specific earlier output that this transaction consumes (spends). In Hathor it is the `(tx_id, index)` pair plus an unlocking script. An input does not hold value itself; it points at the output that does. ↩
 4. Change is an output a transaction sends back to the payer, equal to the consumed inputs minus the amount paid (minus any fee). It exists because outputs must be spent whole, like receiving change after paying cash with a larger note. ↩
 5. A locking script (Bitcoin: `scriptPubKey`) is the spending condition attached to an output — a small program that must be satisfied to spend it, typically "provide a valid signature for this public key." It encodes ownership without naming an owner. ↩
 6. An unlocking script (Bitcoin: `scriptSig`) is the data an input supplies to satisfy the locking script of the output it spends — usually a signature and public key. In Hathor it is the input's `data` field. ↩
 7. A digital signature is data produced with a private key that anyone can verify with the matching public key, proving the signer authorized a specific message without revealing the private key. It is how a locking script confirms a spender's authority. (Full treatment with the crypto libraries in Ch 40.) ↩

Chapter 8 — From Chain to DAG: The Vertex Model

What you'll learn in this chapter

- Enough **graph theory** to read the rest of the book: nodes, edges, directed, acyclic, and why "acyclic" is non-negotiable for a ledger.
- The limitation of a single **chain of blocks**, and what Hathor gains by letting transactions link directly to each other.
- The **vertex**: Hathor's umbrella term for "a block or a transaction," and the real class hierarchy that implements it.
- **The crux of this whole book**: the two distinct kinds of edge a vertex has — **parents** (DAG/confirmation topology) and **inputs** (which coins it spends) — and why confusing them makes the codebase unreadable.
- What **blocks** are still for in a DAG (ordering and minting), what **tips** are, and how a new vertex **confirms** older ones.
- A **bridge** to the code that defines and walks the graph.

This is the chapter that gives the project its shape, and the one a newcomer most needs to get right. Chapter 6 described the classic design — transactions bundled into blocks, blocks in a single chain. Hathor keeps blocks but dissolves the bundling: ordinary transactions become first-class citizens of the ledger graph, linking directly to one another. The result is a **DAG**, and once you hold its structure — especially the two-kinds-of-edge distinction in §8.4 — the transaction code, the consensus, and the sync protocol all become legible.

We build the graph vocabulary first (§8.1), because the rest depends on it, then motivate the DAG (§8.2–8.3), then the edges (§8.4), then Hathor's classes (§8.5).

8.1 Just enough graph theory

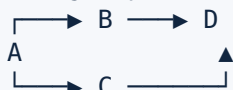
A **graph**¹ is one of the simplest structures in computer science: a set of **nodes**² (also called vertices) connected by **edges** (links). That's all. A social network is a graph (people = nodes, friendships = edges); a road map is a graph (intersections = nodes, roads = edges).

Two refinements turn a plain graph into the kind Hathor uses:

Directed. In a **directed graph**³, each edge has a direction — it points from one node to another, like a one-way street. "A → B" is not the same as "B → A." Hathor's edges are directed: a transaction points back to the earlier ones it builds on.

Acyclic. A **cycle** is a path along the edges that returns to where it started (A → B → C → A). A graph with no cycles is **acyclic**⁴. Put the two together and you get a **DAG** — a Directed Acyclic Graph: edges have direction, and you can never follow them in a loop.

A DAG (edges point one way; no path loops back)



Following arrows from A you reach B, C, D — never back to A.

Why must a ledger be acyclic? Because the edges encode "this came after that." A transaction points to earlier transactions it builds upon. If a cycle existed — X after Y after X — you'd have a contradiction in time: X both precedes and follows itself. Acyclicity is what lets the graph express a consistent history. It also guarantees a **topological order**⁵: you can always lay the nodes out in a line such that every edge points backward, i.e. every vertex comes after everything it depends on. That ordering is what makes it possible to process the graph (verify a vertex only after its dependencies, replay history deterministically) — and it is why a DAG can still serve as a ledger even without a single built-in chain.

8.2 The limit of a single chain

Recall the classic structure (Chapter 6): transactions are collected into blocks, and blocks form one chain, each pointing to its predecessor. This is itself a trivial DAG — a straight line, the simplest acyclic directed graph. It works, but the single-file shape has consequences:

- **Transactions don't exist on the ledger until a block includes them.** Your transaction waits in a holding area (the mempool) until a miner packs it into a block. Until then it is, ledger-wise, in limbo.
- **Throughput is gated by blocks.** History grows one block at a time, at the block interval, so the rate at which transactions become part of the ledger is capped by block production.
- **Everything funnels through miners.** Only miners extend the ledger; ordinary users produce transactions but cannot themselves grow the structure.

Hathor's design question was: what if a transaction could attach to the ledger directly — pointing at earlier transactions — without waiting to be packed into a block? If transactions can reference each other, the single line opens up into a branching, merging graph, and several of the above constraints loosen. That is the move from chain to DAG.

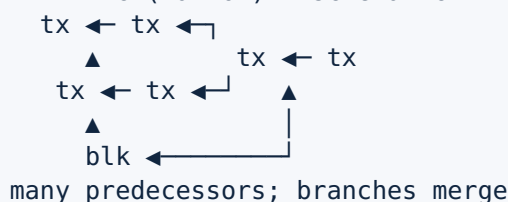
8.3 The DAG of vertices

In Hathor, the ledger is one DAG whose nodes are both blocks and transactions. When a new transaction is created, it points back at a small number of earlier vertices already in the graph — and by doing so, it attaches itself to the ledger immediately, no block required. Because many transactions can attach in parallel, and each can point at more than one predecessor, the structure branches and re-merges into a mesh rather than a line.

A CHAIN (classic)
blk ◀ blk ◀ blk ◀ blk

one predecessor each
(a single line)

A DAG (Hathor) – schematic



Because blocks and transactions live in the same graph and share most of their structure, the codebase needs one word for "a node of this graph, whichever kind it is." That word is **vertex**⁶ — the graph-theory term from §8.1, used throughout `hathor-core`. A vertex is either a block or a transaction; when the code doesn't care which, it says "vertex."

Two questions immediately arise, and the next two sections answer them. First: what exactly are the edges — what does it mean for one vertex to "point at" another? (It turns out there are **two** different kinds, and conflating them is the classic beginner error.) Second: if transactions no longer need blocks, what are blocks still for?

8.4 The crux: parents versus inputs

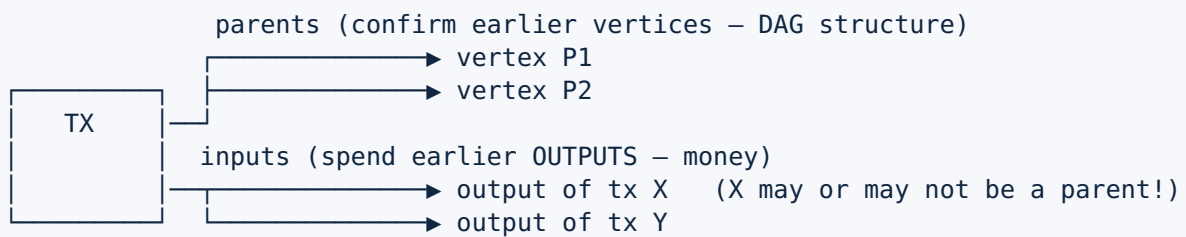
This is the single most consequential distinction in the book. A Hathor vertex has two separate kinds of outgoing edge, which serve completely different purposes. Beginners conflate them and then find the code incomprehensible. Keep them apart and most of the system clicks.

Parents — the DAG/confirmation topology. Every vertex names a small list of **parents**⁷: earlier vertices it attaches to and thereby confirms. This is the structural edge — the analogue of the classic block's `prev` hash pointer (Chapter 6), but

generalized: instead of one predecessor, a vertex has a few. Parents are what make the graph a graph; they encode "this vertex comes after those, and vouches for them." In the code, this is the `parents` field, a list of vertex hashes (`base_transaction.py:179`).

Inputs — the spending edges. Separately, a transaction names **inputs**: pointers to specific earlier outputs it spends (the UTXO mechanism of Chapter 7). This edge is about money — which coins are being consumed — not about graph structure. In the code, this is the `inputs` field, a list of `TxInput` objects, each holding a `(tx_id, index)` pair (`base_transaction.py:936`).

ONE TRANSACTION, TWO KINDS OF EDGE



PARENTS answer "what does this confirm / come after?" (topology)
INPUTS answer "what coins does this spend?" (value)

The two edge sets are independent. A transaction you spend from (an input) need not be one of your parents, and a parent need not be one you spend from. The code treats them as distinct sets — `get_all_dependencies()` deliberately unions both (`base_transaction.py`), precisely because to process a vertex you need everything it depends on either way, but the two roles never merge. Internalize this picture; everything in Chapters 25, 31, and 32 leans on it.

One immediate consequence worth stating: a **double-spend** (Chapter 6–7) is a collision in the input edges — two transactions whose inputs point at the same output — and it is entirely possible for both of those transactions to sit happily in the DAG via their parent edges. The graph structure does not prevent conflicts; it contains them, and consensus (Chapter 10, 32) resolves which conflicting transaction wins by **voiding** the other. The DAG holds both; voiding picks one.

8.5 What blocks are still for, tips, and confirmation

If transactions attach themselves directly, **why keep blocks at all?** Two jobs only blocks do:

1. Minting. New coins must enter the supply somewhere. As in Chapter 6, blocks carry the **reward** — outputs with no inputs — so block production is how HTR is created. Transactions move existing money; blocks make new money.

2. A backbone of ordering. A pure transaction-DAG has no single built-in timeline — many transactions attach in parallel, so "which came first" can be ambiguous. Blocks solve this: they form a chain through the DAG (each block has a block-parent), and that chain acts as a spine that pins down a global ordering and a notion of accumulated work (Chapter 9). Transactions are confirmed by blocks — a transaction becomes increasingly settled as blocks pile up that reach back to it through parent edges.

This is why a **block** in Hathor has a richer parent structure than a transaction. The grounding bears this out: a block has **three parents** — one block parent (continuing the block backbone, always `parents[0]`, `block.py`) plus two transaction parents (attaching the block to the transaction-DAG it confirms). A transaction typically has **two parents**, both other transactions/blocks. The code even exposes this asymmetry: `get_tx_parents()` returns `self.parents[1:]` for a block (dropping the block parent) but all parents for a transaction (`base_transaction.py`).

Two more terms you'll meet constantly:

Tips. The **tips**⁸ of the DAG are the vertices that no other vertex has confirmed yet — the current frontier of the graph. When you create a new transaction, you choose your parents from the tips, thereby confirming them and extending the frontier. Selecting good tips to confirm is part of the protocol (and part of what the mempool tracks — Ch 28).

Confirmation. A vertex is **confirmed**⁹ by every later vertex that can reach it by following parent edges backward. The more vertices (and especially blocks) that reach back to yours, the more work stands behind it, and — exactly as with depth in Chapter 6 — the harder it is to reverse. Confirmation in a DAG is the generalization of "blocks built on top" from a chain.

8.6 Bridge — the vertex model in code

The concepts above are implemented in `hathor/transaction/`, toured fully in Chapter 25. The forward-pointers:

Bridge — the DAG and vertices in the codebase (full treatment in the chapters named):

- **The real base class.** The umbrella type is `GenericVertex[StaticMetadataT]`, an abstract generic base (`base_transaction.py:148`). For historical reasons the names `Vertex` and `BaseTransaction` are aliases of it (`base_transaction.py:933`, `BaseTransaction: TypeAlias = Vertex`) — so when you see `BaseTransaction` in older code, read "any vertex." The generic type parameter ties each vertex kind to its own static metadata (Ch 25). This uses the generics of Chapter 5 and the inheritance/ABC of Chapter 1 — **Chapter 25**.
- **The subclasses.** `Block` (`block.py:45`) and `Transaction` (`transaction.py:55`) are the two main kinds; further specializations exist — `MergeMinedBlock` (`merge_mined_block.py:31`) and `PoaBlock` extend `Block`; `TokenCreationTransaction` and `OnChainBlueprint` (`on_chain_blueprint.py:155`) extend `Transaction`. This is the §1.5 inheritance hierarchy at the heart of the ledger — **Chapters 25, 32, 39**.
- **Parents vs. inputs, in fields.** `parents` (`base_transaction.py:179`) is the topology edge; `inputs` (a list of `TxInput`, `base_transaction.py:936`) is the spending edge. The §8.4 distinction is right there in the class — **Chapter 25**.
- **Walking the graph.** Traversing the DAG (ancestors, descendants, in topological order) is done by dedicated walk utilities in `transaction/storage/` — the §8.1 topological order, made operational — **Chapter 27**.
- **Confirmation, weight, and ordering.** How blocks impose ordering and how confirmation accrues work is the weight story — **Chapter 9** — and how conflicts in the input edges are resolved is voiding — **Chapters 10 & 32**.
- **Genesis as the graph's root.** The DAG bottoms out at the genesis vertices (the first block and two genesis transactions), identified by hash (`transaction/genesis.py`), recalled from **Chapter 22** — every parent chain eventually reaches them.

Recap

Concept	Meaning	In Hathor
Graph / node / edge	nodes joined by links	the ledger itself
Directed	edges point one way	a vertex points back at earlier ones
Acyclic (DAG)	no path loops back	enables a consistent history + topological order

Concept	Meaning	In Hathor
Vertex	a node of the ledger graph	<code>GenericVertex</code> (alias <code>BaseTransaction</code>)
Parents	confirmation/topology edge	<code>parents</code> field; block has 3, tx has ~2
Inputs	spending edge (UTXO)	<code>inputs</code> field, <code>TxInput(tx_id, index)</code>
Block (still needed)	minting + ordering backbone	reward outputs; block-parent chain
Tips	unconfirmed frontier	chosen as parents by new vertices
Confirmation	reachable by later vertices	settles a vertex as work accrues

Hathor's ledger is a single directed acyclic graph in which blocks and transactions are both vertices, transactions attach themselves directly by naming **parents**, and value flows through a separate set of **input** edges — two distinct kinds of link that must never be confused. Blocks survive the move from chain to DAG because they alone mint coins and provide an ordering backbone; transactions gain the ability to join the ledger without waiting on a block. The graph holds conflicting transactions side by side and defers to consensus to pick winners. You now have the ledger's structure (this chapter) and its contents (UTXO, Chapter 7). What remains is the force that orders the backbone and decides contests: **proof-of-work expressed as weight**, the subject of Chapter 9.

1. A graph is a set of nodes connected by edges. It is an abstract structure for representing relationships; many systems (networks, maps, dependencies, ledgers) are naturally graphs. ↩
2. A node (in graph theory, also vertex) is one element of a graph. Hathor uses "vertex" for ledger nodes to avoid confusion with network nodes (the computers), which Chapter 6 defined separately. ↩
3. A directed graph has edges with a direction — each edge goes from one node to another, not symmetrically. An edge $A \rightarrow B$ does not imply $B \rightarrow A$. ↩
4. Acyclic means containing no cycles — no path that follows edges and returns to its starting node. A directed acyclic graph (DAG) combines directedness with acyclicity. ↩
5. A topological order is an arrangement of a DAG's nodes in a line such that every edge points from a later position to an earlier one — i.e. every node appears after all nodes it depends on. Only DAGs have one. It makes dependency-respecting processing possible. ↩
6. A vertex in `hathor-core` is any node of the ledger DAG — a block or a transaction. The umbrella term used when the kind does not matter; implemented as `GenericVertex` (aliased `BaseTransaction`). ↩

7. Parents are the earlier vertices a vertex attaches to and confirms — the DAG's structural (topology) edges. Stored as a list of vertex hashes. The generalization of a blockchain's single `prev` pointer. ↩
8. Tips are the vertices at the frontier of the DAG that no other vertex has yet confirmed (named as a parent). New vertices select their parents from among the tips. ↩
9. A vertex is confirmed by every later vertex that can reach it via parent edges. More confirmations (and accumulated work behind them) make a vertex harder to reverse — the DAG analogue of chain depth. ↩

Chapter 9 — Proof-of-Work, Weight & Accumulated Weight

What you'll learn in this chapter

- How proof-of-work is actually checked: the **nonce** → **hash** → **target** relationship, refreshed from Chapter 6 and made concrete.
- Why Hathor measures work as a **weight** — a single float that is the base-2 logarithm of the number of hash attempts — and why a logarithm is the right unit.
- What the **difficulty adjustment algorithm (DAA)** does: tuning weight so blocks arrive at a steady rate, and giving transactions a minimum weight.
- **Accumulated weight** and **score**: how work sums up across the DAG, and how that sum, not chain length, decides which history wins.
- The **heaviest-DAG rule** — Hathor's generalization of "most-work-wins" — and a **bridge** to the verifier, the DAA, and the consensus code.

Chapter 6 established why proof-of-work exists (Sybil resistance, costly history-rewriting); Chapter 8 established the structure work has to secure (a DAG, not a chain). This chapter joins them: how Hathor quantifies work, and how those quantities, accumulated across the graph, answer "which version of history is real." The central idea — measuring work as a logarithm called weight — is one of Hathor's clean departures from Bitcoin, and once you see why a logarithm, the rest follows.

A short primer on logarithms is included (§9.2) because the whole chapter rests on one, and the book does not assume you carry that comfortably. Track B cites Hathor code; this chapter does, at the points where the concept becomes a concrete function.

9.1 Proof-of-work, refreshed and made concrete

Recall the mechanism from Chapter 6. To produce a valid vertex, a miner must find a **nonce**¹ — a number — such that hashing the vertex together with that nonce yields a hash below a **target**². Because the hash function is one-way and avalanche (Chapter 6 §6.4), there's no shortcut: the miner can only try nonce after nonce, billions per second, until one produces a small-enough hash. Finding it is hard; checking it is one hash.

In code, this is exactly what the verifier does. `verify_pow` (`hathor/verification/vertex_verifier.py:142`) computes the vertex's hash and checks it is numerically below the target. The target itself is derived from the vertex's weight by `get_target` (`base_transaction.py:361`), which computes — conceptually — $\text{target} = 2^{(256 - \text{weight})} - 1$. Read that relationship slowly: a 256-bit hash is a number between 0 and $2^{256}-1$; requiring it to fall below $2^{(256-\text{weight})}$ means only a $1 / 2^{\text{weight}}$ fraction of all possible hashes qualify. So:

The core relationship. A vertex of weight w requires, on average, about 2^w hash attempts to find a valid nonce. Higher weight $\Rightarrow$ smaller target $\Rightarrow$ rarer valid hash $\Rightarrow$ more work. The weight is the difficulty, expressed as an exponent.

That last phrase is the key to the whole chapter. Bitcoin stores difficulty as a compact encoding of the target — a 256-bit threshold. Hathor stores it as the exponent instead: a single floating-point number, the weight. To see why that's the better unit, we need the logarithm.

9.2 A two-minute primer on logarithms

A **logarithm**³ answers the question: "to what power must I raise the base to get this number?" In base 2 — the natural base when we're counting doublings — $\log_2(x)$ asks "2 to the what equals x ?"

$2^{10} = 1024$	so	$\log_2(1024) = 10$
$2^{20} = 1,048,576$	so	$\log_2(1,048,576) = 20$
$2^{21} = 2,097,152$	so	$\log_2(2,097,152) = 21$

Two properties are all you need:

1. **A logarithm turns huge numbers into small, manageable ones.** The number of hash attempts to mine a block is astronomically large — quadrillions and up. Its base-2 logarithm is a tidy number like 60 or 76. Weight lives on the logarithmic scale, so it stays small and readable while the underlying work is gigantic.
2. **Adding logarithms multiplies the underlying numbers.** $\log_2(a) + \log_2(b) = \log_2(a \times b)$. Equivalently, adding one to a weight doubles the work: weight 21 is twice the work of weight 20, and weight 22 is twice again. Each $+1$ of weight is a doubling. This is why weights can be summed to combine work — a property §9.4 relies on completely.

With those in hand, weight has a precise meaning: **weight = $\log_2(\text{number of hash attempts})$** . A block of weight 60 represents about $2^{60} \approx 10^{18}$ attempts; a block of weight 61 represents twice that.

9.3 Weight: work as a single number

So Hathor attaches to every vertex a field `weight`, a `float` (`base_transaction.py:201`), and treats it as the logarithm of the work the vertex embodies. The conversions are explicit in `hathor/utils/weight.py`:

```
def weight_to_work(weight: float) -> int:    # how many attempts a weight represents
    return floor(0.5 + 2 ** weight)          #  $\approx 2^{\text{weight}}$  (utils/weight.py:18)

def work_to_weight(work: int) -> float:      # the inverse
    return log2(work)                        # (utils/weight.py:23)
```

Why is a single float a better representation than Bitcoin's target?

- **It's readable and comparable.** "This block has weight 62" is immediately meaningful, and comparing two weights is comparing two floats. No unpacking a compact threshold.
- **It's additive.** Because adding weights multiplies work (§9.2), the total work behind a region of the DAG is almost just a sum of weights — which is exactly what consensus needs to do constantly (§9.4). Summing thresholds directly would be meaningless; summing logarithms is natural.
- **It's smooth.** Weight is a continuous float, so difficulty can be tuned in fine increments rather than discrete steps — useful for the steady adjustment in §9.5.

The trade-off is small: floating-point weight needs care to compare and store precisely, which is why the code converts to integer work (`weight_to_work`) when it needs exact arithmetic, and keeps weight for the human-facing and per-vertex value. Internally, accumulated work is tracked as an integer (`accumulated_weight: int`) and converted as needed.

Transactions have weight too. In a chain, only blocks are mined. In Hathor's DAG, every vertex — transaction included — carries proof-of-work, because a transaction attaches itself directly to the ledger (Chapter 8) and must pay some cost to do so (an anti-spam measure: making each transaction cost a little work deters flooding the DAG). Transaction weights are much smaller than block weights, but they are real, and they contribute to the accumulated total.

9.4 Accumulated weight and score: summing work across the DAG

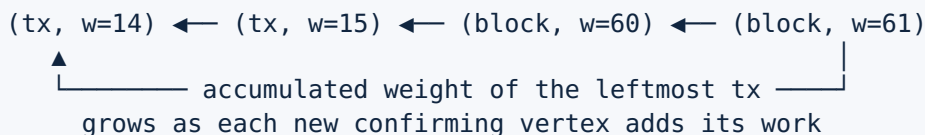
Here is where the DAG (Chapter 8) and weight (this chapter) combine into consensus. A single vertex's weight is the work to make that one vertex. But security comes from all the work piled behind a vertex — every later vertex that confirms it (Chapter 8 §8.5). Hathor captures this with two metadata fields, stored on each vertex's

`TransactionMetadata` (`hathor/transaction/transaction_metadata.py`):

- **accumulated_weight** (`transaction_metadata.py:48`): the total work of a vertex plus all the work that confirms it through the graph. As more vertices attach behind it, its accumulated weight grows. This is the DAG generalization of Chapter 6's "blocks built on top" — except instead of counting blocks, Hathor sums their work.
- **score** (`transaction_metadata.py:49`): the metric used to compare competing block chains. For a block, the score reflects the accumulated work of the best chain up to that block. The chain of blocks with the **highest score** is the canonical one.

The reason weight's additivity (§9.2) matters now becomes concrete: to know how much work stands behind a vertex, the node sums the weights (as work) reachable from it. That sum is only meaningful because each `+1` of weight is a consistent doubling — logarithms are the unit that makes "total work in this region of the graph" a well-defined, computable number.

accumulated weight = this vertex's work + all confirming work behind it



9.5 Keeping the rate steady: the difficulty adjustment algorithm

If weight were fixed, two problems would follow: as more miners join, blocks would come faster and faster (more hashing power finds valid nonces sooner); and the network couldn't hold a predictable rhythm. Every proof-of-work system therefore adjusts difficulty over time. Hathor's lives in `hathor/daa.py` — the **difficulty adjustment algorithm (DAA)** ⁴.

The goal is a steady **average time between blocks** — `AVG_TIME_BETWEEN_BLOCKS = 30` seconds (a settings constant from the hathorlib base, `:156`). The algorithm `calculate_next_weight` (`daa.py:94`) works on the principle every DAA shares:

- Look back over a window of recent blocks and measure how long they actually took to find.
- If they came **faster** than the 30-second target, the network has gained hashing power → **raise** the weight (more work required) to slow back down.
- If they came **slower**, **lower** the weight to speed back up.

It's a feedback loop: weight chases the target block time, nudged up or down by recent reality. (There's also a weight decay safeguard that lowers difficulty if blocks stall badly, so the chain can't freeze if hashing power suddenly drops.)

Transactions get a related treatment: `minimum_tx_weight` (`daa.py:176`) computes a floor on a transaction's weight based on its size and amount — bigger transactions must do a little more work — which is the anti-spam mechanism of §9.3 made precise.

9.6 The heaviest-DAG rule

Now Chapter 6's "most-work-wins" can be stated in Hathor's terms. In a chain, the rule is loosely "longest chain," precisely "most cumulative work." In Hathor's DAG, it is:

The history with the greatest accumulated weight wins. When there are competing versions of history — two block chains, or two conflicting transactions — the node prefers the one with more total work behind it, measured by score / accumulated weight.

This is not "the longest" or "the one with the most vertices" — a few heavy vertices can outweigh many light ones, exactly as Chapter 6 warned. It is total work, summed via weight. Because work cannot be faked (only paid for in hashing), preferring the heaviest history means an attacker wanting to rewrite the past must out-compute everyone confirming the real past — the Chapter 6 security argument, now operating over a graph instead of a line.

What happens when the heaviest history changes — when a competing branch overtakes the current one, or two transactions collide on an input — is the resolution mechanism: the losing side is **voided**. That is Chapter 10, which closes Track B by turning "the heaviest wins" into the concrete rules for conflicts, reorgs, and finality.

Recap

Concept	Meaning	In Hathor
Nonce	the number a miner varies	searched until hash < target
Target	threshold the hash must beat	<code>get_target</code> , $\approx 2^{(256-\text{weight})}$ (:361)
Weight	$\log_2(\text{hash attempts})$ — work as one float	<code>weight</code> field (:201); +1 = double work
weight $\leftrightarrow$ work	convert log $\leftrightarrow$ count	<code>utils/weight.py:18,23</code>
DAA	tune weight for steady block time	<code>daa.py:94</code> ; target 30 s (:156)
min tx weight	per-transaction work floor (anti-spam)	<code>daa.py:176</code>
Accumulated weight	a vertex's work + all confirming work	<code>metadata</code> :48
Score	metric to compare chains	<code>metadata</code> :49
Heaviest-DAG rule	most accumulated weight wins	the consensus preference

Hathor measures proof-of-work as **weight** — the base-2 logarithm of the number of hash attempts — because a logarithm keeps an astronomically large quantity small and readable, and because adding weights corresponds to multiplying work, which lets the node sum work across the DAG. Every vertex, block or transaction, carries weight; the difficulty algorithm tunes block weight toward a 30-second rhythm and sets a per-transaction floor against spam; and the work behind any vertex accrues as **accumulated weight**, with the heaviest history winning. The one question left is operational: when two histories compete — a reorg, or two transactions spending the same coin — exactly how does the loser get marked and removed, and when is anything ever final? That is **voiding, conflicts, and finality**, the close of Track B.

1. A nonce ("number used once") is the field a miner repeatedly changes so each attempt re-hashes the vertex to a different value, searching for one whose hash falls below the target. $\leftrightarrow$
2. A target is the numeric threshold a vertex's hash must fall below to be valid proof-of-work. In Hathor it is derived from the vertex's weight: roughly $2^{(256 - \text{weight})}$, so higher weight means a smaller target and more work. $\leftrightarrow$

3. A logarithm $\log_b(x)$ is the exponent to which the base b must be raised to obtain x (e.g. $\log_2(1024) = 10$). Logarithms compress large numbers and convert multiplication into addition. Hathor's weight is a base-2 logarithm of work. ↩
4. The difficulty adjustment algorithm (DAA) periodically recomputes the required weight so that blocks are found at a roughly constant average rate despite changing total hashing power. Hathor's is in `hathor/daa.py`. ↩

Chapter 10 — Conflicts, Voiding, Finality & Checkpoints

What you'll learn in this chapter

- What a **conflict** is in Hathor — concretely, two transactions spending the same output — and how it differs from an invalid transaction.
- **Voiding**: how the node marks a transaction as "not counted" without deleting it, via the `voided_by` metadata field, and why marking beats deleting.
- How a conflict is decided — the heavier transaction wins — and how voiding **propagates** to everything built on the loser.
- **Reorgs** in a DAG: how a heavier competing block-chain takes over, and what becomes of the displaced history.
- Why finality is **probabilistic**, how confirmations harden it, and how **checkpoints** put a hard floor under it.
- A **bridge** to the consensus code that implements all of this.

This chapter closes Track B by making the abstract promise of Chapters 6–9 operational. We have a ledger shaped as a DAG (Ch 8), money tracked as UTXOs (Ch 7), and work measured as weight with "heaviest wins" as the rule (Ch 9). The missing piece is the mechanism: when two transactions collide, or when a competing branch overtakes the leader, what actually happens in the node? The answer is **voiding** — a single idea that handles double-spends, reorgs, and invalidity uniformly. Once you have it, the consensus chapter (Ch 32) is a detailed reading of machinery you already understand.

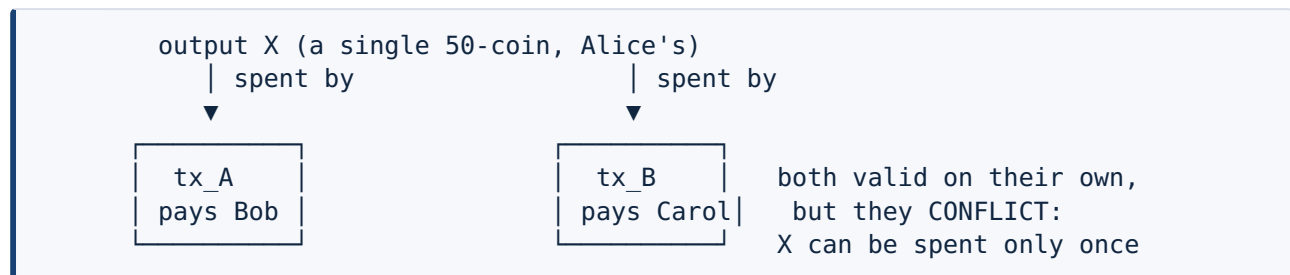
Track B cites Hathor code; this chapter points at the consensus package at the moments the concept becomes a concrete function.

10.1 Two kinds of "bad" transaction

It helps to separate two things newcomers blur together.

An **invalid** transaction breaks a rule on its own — a bad signature, weight below the minimum, malformed structure, inputs that don't cover outputs. Invalidity is detectable by looking at the transaction in isolation, and the node rejects such a transaction at verification (Chapter 31). It never enters the ledger.

A **conflict**¹ is different and subtler: each transaction is individually valid, but they cannot both be true. The canonical case is the **double-spend** (Chapters 6–7): two transactions, each well-formed and correctly signed, that name the same output as an input. Each is fine alone; together they'd spend one coin twice.



Crucially — and this is the DAG difference (Chapter 8 §8.4) — both conflicting transactions can sit in the graph at once, attached via their parent edges. The structure does not forbid the collision; it holds both and waits for consensus to pick a winner. That picking is voiding.

10.2 Voiding: marking, not deleting

When the node detects a conflict (or, as we'll see, when something a transaction depends on goes bad), it does not erase the loser. It **voids**² it: marks it as "present in the graph but not counted as part of the real ledger." A voided transaction's outputs are not spendable, and its effect on balances is undone — but the transaction object stays in storage.

The mark is a single metadata field, `voided_by` (`hathor/transaction/transaction_metadata.py:45`):

```
voided_by: Optional[set[bytes]] # set of hashes causing this vertex to be voided
```

The rule is simple: **if `voided_by` is empty (or `None`), the transaction is executed (counts); if it holds any hashes, the transaction is voided (doesn't count)**. The hashes inside say why — which transactions are responsible for the voiding (a conflicting winner, or a voided ancestor). A transaction that loses a conflict gets its own hash placed in its `voided_by` — it is, in a sense, voiding itself by losing.

Why mark instead of delete? Three reasons, each consequential:

1. **Reversibility.** Consensus can change. A transaction voided today might be un-voided tomorrow if the branch it's on becomes the heaviest (a reorg, §10.4). If you'd deleted it, you couldn't bring it back; a flag, you just clear. Marking makes consensus decisions undoable, which a DAG with shifting weights absolutely needs.

2. **Auditability.** The full graph — including the losing sides of past conflicts — remains inspectable. The `voided_by` set records the reason, so the history of why the ledger looks as it does is preserved.
3. **Referential integrity.** Other vertices may point at a now-voided one through parent edges. Deleting it would tear holes in the graph; marking leaves the structure intact and lets voiding propagate cleanly (§10.3).

This single flag — present-but-not-counted, with a reason and a reverse switch — is the mechanism behind double-spend resolution, reorgs, and dependency-driven invalidation alike. That uniformity is the elegance.

10.3 Deciding a conflict, and propagation

When two transactions conflict, which one wins? The heaviest-DAG rule of Chapter 9 decides it. The consensus code's `check_conflicts` (`hathor/consensus/transaction_consensus.py:302`) compares the conflicting transactions by **accumulated weight** — the total work standing behind each (Chapter 9 §9.4):

- The transaction with the **greater accumulated weight wins**: its hash is removed from its `voided_by`, so it executes.
- The loser keeps (or gains) its hash in `voided_by`, so it stays voided.
- Ties — equal accumulated weight — are broken deterministically (by timestamp, so every node reaches the same decision; consensus must be reproducible).

The winner is "the transaction the rest of the network put more work behind," which is exactly the security property we want: to make your double-spend win, you'd have to out-work everyone confirming the honest transaction.

Voiding propagates. A transaction does not stand alone — others spend its outputs and confirm it. So when a transaction is voided, everything that depends on it must be voided too, or the ledger would be inconsistent (you can't have a valid transaction spending a coin from a voided one). The consensus code walks the descendants — both those that spend the voided transaction's outputs and those that confirm it — and propagates the voiding outward (`transaction_consensus.py`, the voiding-propagation walk around `:374`). The walk uses the DAG traversal machinery (Chapter 8 §8.1's topological order) to reach every affected vertex.

```
tx_A voided → every tx spending tx_A's outputs → and THEIR dependents...  
              (voiding cascades down the DAG until it reaches vertices  
              that don't depend on anything voided)
```


This cascade is why marking-not-deleting matters: propagation is just adding hashes to `voided_by` sets down the graph, and if consensus later reverses, the same walk removes them. One mechanism, both directions.

10.4 Reorgs in a DAG

A **reorg**³ (reorganization, Chapter 6 §6.7) is the block-level version of a conflict. Blocks form the ordering backbone (Chapter 8 §8.5), and sometimes a competing chain of blocks — a fork — accumulates more work than the current best chain. When it does, the node must switch: the heavier chain becomes canonical, and the previously-best chain is displaced.

The block-consensus code (`hathor/consensus/block_consensus.py`) handles this by comparing **score** (Chapter 9 §9.4) between competing chain heads; when a new block makes its branch heavier than the current best, that branch wins (with hash used as a deterministic tiebreaker). The switch is recorded as a reorg when the branches diverged before the old best block. Mechanically, the takeover is the same voiding machinery: the blocks and transactions on the losing branch get marked in their `voided_by`, and any transactions that were only on the losing branch become voided (returning their inputs to the unspent set, Chapter 7). If those transactions are still valid and not conflicting, they can be re-applied on the new branch.

One refinement the grounding makes precise: **block voiding does not propagate to child blocks** the way transaction voiding propagates to dependents. A block being voided affects the transactions that depend on it, but the block backbone is handled by the score comparison directly rather than by cascading voids down the block chain (`consensus/consensus.py`). The distinction matters when you read Chapter 32; here, hold the shape: reorg = the heavier block branch wins, the lighter branch's unique transactions get voided.

10.5 Finality: probabilistic, then anchored

Now the question every user actually cares about: **when is my transaction safe?**

As Chapter 6 §6.7 warned, the honest answer is never with absolute certainty, only with growing confidence. A transaction is settled to the degree that work has accumulated behind it (Chapter 9 §9.4). A freshly-attached transaction with little behind it could still lose a conflict or be displaced by a reorg. But as blocks and transactions pile up confirming it, the accumulated weight an attacker would have to overcome grows, and the probability of reversal falls — exponentially with depth, just as in a chain. This is **probabilistic finality**⁴: confidence asymptotically approaching

certainty, never reaching it. In practice, recipients wait for some number of **confirmations**⁵ — enough confirming work that reversal is economically impossible — before treating a payment as done.

Probabilistic finality has one gap: it says nothing about very deep history being attacked by an adversary willing to spend enormous resources, and it offers no protection while a brand-new node is syncing and hasn't yet seen the work. Hathor closes that gap the way Chapter 6 §6.7 described — with **checkpoints**⁶.

A checkpoint is a hard-coded `(height, hash)` pair — the `Checkpoint` type is literally those two fields (`hathor/checkpoint.py:18`) — declaring "the block at this height is this block, by decree." Checkpoints are shipped in the settings profile (`CHECKPOINTS`, recalled from Chapter 22) and enforced during verification (`verify_checkpoint`, `base_transaction.py`). Their effect:

- **No reorg may rewrite history before a checkpoint.** A competing branch that disagrees with a checkpointed block is rejected outright, however heavy it claims to be. This bounds reorg depth and defeats deep-history attacks.
- **Syncing nodes get trustworthy anchors.** A new node can be sure that the history up to each checkpoint matches everyone else's, without re-deriving all the work from scratch.

The trade-off is honest: checkpoints are a small reintroduction of authority (someone decides what goes in the settings), accepted in exchange for a hard floor under history. They convert the deepest part of the ledger from "probabilistically final" to "final by decree."

10.6 Bridge — voiding and consensus in code

Everything here is implemented in `hathor/consensus/`, toured in Chapter 32. The forward-pointers:

Bridge — conflicts, voiding, finality in the codebase (full treatment in the chapters named):

- **The voided flag.** `voided_by: Optional[set[bytes]]` on `TransactionMetadata` (`transaction_metadata.py:45`) is the present-but-not-counted mark; empty = executed, non-empty = voided. The §1.4 encapsulated-invariant idea applied to consensus state — **Chapters 25 & 32**.
- **Conflict resolution.** `check_conflicts` (`transaction_consensus.py:302`) decides the winner by accumulated weight (Chapter 9), with deterministic tie-breaking — **Chapter 32**.
- **Voiding propagation.** The descendant walk (around `transaction_consensus.py:374`) cascades voiding to dependents and reverses it on un-voiding, using the DAG traversal of Chapter 8 — **Chapters 27 & 32**.
- **Block reorgs.** `block_consensus.py` compares score between chain heads and switches the canonical branch; transaction voiding does the cleanup — **Chapter 32**.
- **The two-tier consensus split.** Hathor separates block consensus (the ordering backbone) from transaction consensus (conflicts among txs) into two modules under `consensus/` ; the top-level `ConsensusAlgorithm` orchestrates them — **Chapter 32**.
- **Checkpoints.** The `Checkpoint(height, hash)` type (`checkpoint.py:18`), sourced from settings (`CHECKPOINTS` , Chapter 22) and enforced by `verify_checkpoint` — **Chapters 22 & 32**.
- **Where ingestion triggers all this.** A newly-accepted vertex runs verification (Chapter 31) and then consensus (this chapter) via the vertex handler — **Chapter 33**.

Recap

Concept	Meaning	In Hathor
Invalid vs. conflict	rule-breaking alone vs. mutually-exclusive	rejected at verify vs. resolved by voiding
Conflict	two valid txs spending one output	the concrete double-spend
Voiding	mark "not counted," don't delete	<code>voided_by</code> set (<code>:45</code>)
Why mark not delete	reversible, auditable, keeps graph intact	un-voiding on reorg

Concept	Meaning	In Hathor
Conflict winner	greater accumulated weight	<code>check_conflicts</code> (:302)
Propagation	voiding cascades to dependents	descendant walk (~:374)
Reorg	heavier block branch takes over	<code>block_consensus.py</code> , by score
Probabilistic finality	confidence grows with confirming work	depth/confirmations
Checkpoint	<code>(height, hash)</code> final-by-decree anchor	<code>checkpoint.py:18</code> , from settings

Voiding is the single mechanism that makes Hathor's consensus work: a present-but-not-counted mark, carrying its own reason and its own reverse switch, that resolves double-spends (the heavier transaction wins, the lighter is voided), cleans up after reorgs (the lighter block branch's unique transactions are voided), and cascades correctly through the DAG because it marks rather than deletes. Finality emerges from accumulated work — probabilistic and depth-hardened — with checkpoints supplying a hard floor where probability alone is not enough. This closes Track B: you now hold the full conceptual ledger — the **problem** (Ch 6), **ownership** as UTXOs (Ch 7), **structure** as a DAG of vertices (Ch 8), **work** measured as weight (Ch 9), and **consensus** as voiding by accumulated weight (Ch 10). Track C turns next to the third foundation — the stack of third-party technologies the node is built from — beginning with the Python platform itself.

1. A conflict is a pair (or set) of individually-valid transactions that cannot all be part of the ledger — classically, two transactions spending the same output (a double-spend). Distinct from an invalid transaction, which breaks a rule on its own. ↩
2. Voiding marks a vertex as not counted toward the real ledger (its outputs unspendable, its effects undone) without deleting it, via the `voided_by` metadata field. Reversible if consensus later changes. ↩
3. A reorg (reorganization) is a switch of the canonical block chain to a competing branch that accumulated more work, displacing the previous branch; transactions unique to the displaced branch are voided. ↩
4. Finality is assurance a transaction cannot be reversed. Hathor's is probabilistic — strengthening with accumulated confirming work — except below a checkpoint, where it is final by decree. ↩
5. Confirmations are the confirming vertices (especially blocks) accumulated on top of a transaction; more confirmations mean more work an attacker must overcome, so recipients wait for a threshold before trusting a payment. ↩
6. A checkpoint is a hard-coded `(height, hash)` anchor declaring a specific block final, so no reorg may rewrite history before it. It bounds reorg depth and gives syncing nodes trustworthy anchors, at the cost of a little reintroduced authority. ↩

Chapter 11 — Python the Platform

What you'll learn in this chapter

- That "Python" is three things, not one: a language, an interpreter that runs it (**CPython**), and an ecosystem of modules and tools — and why a study of `hathor-core` cares about the latter two.
- What the interpreter actually does with your file: source → **bytecode** → execution, and what the **REPL** is.
- How Python organizes code: a file is a **module**, a directory is a **package**, and what `__init__.py` is for.
- How `import` truly works — the search path (`sys.path`), the module cache (`sys.modules`), and the difference between absolute and relative imports — so a line like `from hathor.transaction import Block` stops being magic.
- **Why Python** for a node at all, given it is not the fastest language — the trade-off, and how Hathor mitigates the cost.
- A **bridge** to the rest of Track C and to where the codebase's structure relies on these mechanics.

Track A and Track B taught concepts. Track C turns to the **stack** — the third-party technologies and platform machinery `hathor-core` is built from — and it starts at the bottom, with Python itself. Not the syntax (you have that), but the platform: the interpreter that executes the code, and the module-and-import system that lets one program span hundreds of files across dozens of packages. You cannot reason about how `hathor-core` is assembled, packaged, or run without these, and every later Track C chapter (virtual environments, Poetry, Docker) sits directly on top of them.

This chapter follows the §5 technology-primer shape: what it is, the problem it solves, the core model, how you use it, how Hathor relies on it, and the "why this and not something else" trade-off.

11.1 "Python" is three things

When people say "Python," they conflate three distinct things, and separating them is the first step to understanding the platform:

1. **The language** — the rules of syntax and semantics: what `for`, `def`, `class`, and an f-string mean. This is what you already know and what a language specification defines.
2. **The interpreter** — the actual program, installed on your machine, that reads Python source and executes it. The standard one is **CPython**¹, written in C. When you type `python`, you are running CPython. There are others (PyPy, Jython), but CPython is what `hathor-core` runs on.
3. **The ecosystem** — the standard library that ships with the interpreter, plus the vast collection of third-party packages (Twisted, RocksDB bindings, Pydantic...) installed from outside, plus the tools that manage all of it (pip, Poetry).

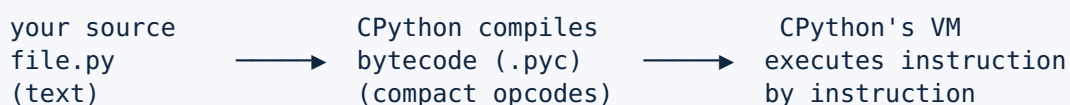
The language is the same everywhere; the interpreter and ecosystem are where the platform lives. A node is a large program leaning heavily on the third item (dozens of external packages) and constrained by the second (CPython's performance characteristics, including the GIL from Chapter 2). So Track C is, in effect, a tour of items 2 and 3.

11.2 The interpreter: what `python file.py` actually does

A common misconception is that Python is "interpreted line by line" like reading a script aloud. The reality is a two-step process, and knowing it explains several things you'll see in the repo.

When you run `python file.py`, CPython:

1. **Compiles** the source to **bytecode**² — a lower-level, compact instruction set that is not machine code for your CPU, but instructions for Python's own virtual machine. This is a compile step, even though Python is called "interpreted."
2. **Executes** that bytecode on the **Python Virtual Machine**³ — a loop inside CPython that reads bytecode instructions one at a time and performs them.



Two visible consequences:

- **__pycache__ and .pyc files.** You'll see directories named `__pycache__` full of `.pyc` files appear throughout a project after it runs. These are CPython caching the compiled bytecode so it needn't recompile unchanged files next time — a speed optimization, nothing you author or commit. (They're in every project's `.gitignore` for that reason.)
- **"Compiled" but not to machine code.** Unlike C or Rust, the bytecode still runs on the VM, not directly on the processor. That indirection is part of why Python is slower than those languages (§11.6) — and why the bytecode is portable across operating systems.

The REPL. Run `python` with no file and you get the **REPL**⁴ — Read-Eval-Print Loop — an interactive prompt that compiles and runs each line as you type it, printing results immediately. It is the fastest way to poke at behavior, and `hathor-core` even ships a subcommand (`hathor-cli shell`, Chapter 21) that drops you into a REPL with the node's objects loaded — invaluable for exploring the running system.

11.3 Modules: a file is a unit of code

A program of any size cannot live in one file. Python's unit of organization is the **module**⁵: a single `.py` file is a module, and its name is the filename without the extension. The file `daa.py` is the module `daa`.

A module is more than a container — it is a **namespace**⁶. Everything defined at the top level of `daa.py` (its functions, classes, constants) becomes an attribute of the module object, reached with a dot: `daa.DifficultyAdjustmentAlgorithm`. This is the same dot-access you met on objects in Chapter 1, applied to files. Importing a module runs its top-level code once and hands you that namespace.

That "once" matters and is worth fixing now: **a module's top-level code executes exactly once per program, the first time it is imported.** Import it again from somewhere else and you get the already-loaded module back, not a fresh run (§11.5 explains the cache that guarantees this). This is why module-level code is the natural home for things that should exist once — a logger, a settings accessor, a registry — a fact that connects directly to the singleton pattern of Chapter 3.

11.4 Packages: a directory is a unit too

Modules group code within a file; **packages**⁷ group modules across directories. A package is a directory of modules treated as one importable unit. So `hathor/` is a package, `hathor/transaction/` is a sub-package, and `hathor/transaction/block.py` is

the `block` module inside it. The dotted path `hathor.transaction.block` mirrors the folder path `hathor/transaction/block.py` exactly — packages are how the filesystem layout becomes the import namespace.

The traditional marker of a package is a file named `__init__.py` in the directory. It does two jobs:

1. **It declares "this directory is a package"** (in classic Python; modern "namespace packages" can omit it, but `hathor-core` uses explicit `__init__.py` files).
2. **It runs when the package is first imported**, so it's a place to set up package-level state or to re-export names. You saw exactly this in Chapter 22: `hathor/conf/__init__.py` re-exports `HathorSettings` so callers can write the short `from hathor.conf import HathorSettings` instead of reaching into the deeper module. An `__init__.py` that gathers a package's public names into one tidy surface is a common, deliberate pattern.

```
hathor/                                ← package (has __init__.py)
├── __init__.py
├── manager.py                         ← module hathor.manager
├── conf/                             ← sub-package
│   ├── __init__.py                  ← re-exports HathorSettings (Ch 22)
│   └── settings.py                  ← module hathor.conf.settings
└── transaction/                     ← sub-package
    ├── __init__.py
    └── block.py                     ← module hathor.transaction.block
```

The dotted name and the directory tree are the same structure seen two ways. Once you internalize that, the whole repository becomes navigable from any import line:

```
from hathor.transaction.block import Block
```

 tells you exactly which file to open.

11.5 How `import` actually works

Import is where beginners hit "magic," so we make it concrete. When Python executes `import hathor.transaction.block`, it does three things, in order:

1. Check the cache first. Python keeps a dictionary of every module already imported, `sys.modules`⁸, keyed by dotted name. If `hathor.transaction.block` is already there, Python returns it instantly and runs nothing further. This is the mechanism behind "top-level code runs once" (§11.3) — the first import populates the cache; every later import is a cache hit. It also means circular imports (A imports B which imports A) are a real hazard, because the second import may find a half-initialized module in the cache.

2. Find the module. On a cache miss, Python searches a list of directories called `sys.path`⁹ — an ordered list of places to look, including the current directory, the standard library, and the installed-packages location (the `site-packages` of your environment, which is Chapter 12's subject). The first match wins. This search path is exactly what a virtual environment manipulates to make a project see its own dependencies and not the system's.

3. Load, execute, cache. Once found, Python compiles the module to bytecode (§11.2), executes its top-level code to build the namespace, stores it in `sys.modules`, and binds the name in your code.

```
import hathor.transaction.block
    |
    v
in sys.modules? —yes—> return cached module (run nothing)
    | no
    v
search sys.path for it —> compile —> run top-level —> store in sys.modules —> bind
```

Absolute vs. relative imports. Two spellings reach a module:

- **Absolute:** `from hathor.transaction.block import Block` — the full path from the project root. Unambiguous, readable, and the dominant style in `hathor-core`.
- **Relative:** `from .block import Block` or `from ..conf import HathorSettings` — the leading dots mean "relative to this package" (one dot = current package, two = parent). Shorter inside a package, but only usable within a package. You'll see both; the absolute form tells you the file outright, which is why this book cites with full paths.

That is the entire import system: a cache, a search path, and a load step. No magic — just a disciplined way to turn a dotted name into a found, compiled, cached namespace.

11.6 Why Python for a node?

A full node is performance-sensitive infrastructure that runs for months and verifies cryptographic data constantly. Python is not the fastest language — CPython's bytecode VM and the GIL (Chapter 2) make pure-Python computation far slower than C, Rust, or Go. So the choice deserves the §5 "why this, not the alternatives" treatment.

What Python costs: raw CPU speed. A tight numeric loop in Python can be one to two orders of magnitude slower than the equivalent in C. For a CPU-bound program, that would be disqualifying.

Why it's chosen anyway:

- **A node is I/O-bound, not CPU-bound (Chapter 2 §2.1).** It spends its life waiting on the network and disk, not crunching numbers. For waiting-dominated work, the language's raw speed barely matters; the concurrency model matters, and Twisted (Chapter 16) gives Python an excellent one.
- **Development speed and clarity.** Python is fast to write, read, and change — valuable for a complex, evolving protocol where correctness and auditability beat microseconds. A node's bugs are expensive; readable code that many people can review is a real asset.
- **A deep ecosystem.** The exact libraries a node needs already exist and are mature: Twisted for async networking, battle-tested crypto bindings, RocksDB bindings, Pydantic. Building on these beats reimplementing them.
- **The slow parts are pushed into C.** The trick that makes the trade work: the genuinely CPU-heavy pieces don't run in Python. RocksDB is C++ (Chapter 27); the cryptography library wraps C (Chapter 40); proof-of-work hashing runs in a thread pool over compiled hash routines (Chapter 2 §2.7, Chapter 37). Python orchestrates; compiled code does the number-crunching. You get Python's clarity at the top and near-C speed where it counts.

The honest summary: Python trades raw speed for development velocity and ecosystem, and a node can afford that trade because it is I/O-bound and because its hot paths delegate to compiled libraries. Pick Go or Rust and you'd gain CPU speed and lose iteration speed and the specific mature libraries Hathor leans on — a different, also-defensible choice that other node implementations make. (The version is pinned, too: `hathor-core` requires a modern Python — the exact constraint lives in `pyproject.toml`, Chapter 13.)

11.7 Bridge — the platform under the codebase

Everything above is the substrate the rest of the book stands on. Forward-pointers:

Bridge — the Python platform in the codebase and the stack:

- **The import tree IS the architecture.** Every `from hathor.x.y import Z` in this book maps to the file `hathor/x/y.py` (§11.4). The module map of Chapter 0 is, literally, the package tree — **Chapter 0**, and every Part II chapter.
- **`__init__.py` re-exports.** The package-surface pattern (§11.4) is used deliberately, e.g. `hathor/conf/__init__.py` — **Chapter 22**.
- **Module-level singletons.** "Top-level code runs once" (§11.3, §11.5) underlies the global settings accessor and the one-per-process reactor — **Chapters 22, 16, 23** (and the singleton pattern, Chapter 3).
- **`sys.path` is what environments manipulate.** The search path of §11.5 is precisely what a **virtual environment** alters so the node sees its own dependencies — **Chapter 12**, next.
- **Versions and dependencies are declared, not assumed.** The Python version constraint and every third-party module the import system resolves are declared in `pyproject.toml` and pinned in `poetry.lock` — **Chapter 13**.
- **The GIL and `async`.** CPython's single-threaded execution of Python code (§11.1) is why the node uses an event loop rather than threads — recalled from **Chapter 2**, realized in **Chapter 16**.
- **The interactive shell.** `hathor-cli shell` drops into the §11.2 REPL with the node loaded — **Chapter 21**.

Recap

Concept	What it is	Why it matters here
CPython	the standard C interpreter	what <code>hathor-core</code> runs on
Bytecode / VM	compiled opcodes run by Python's VM	explains <code>.pyc</code> , portability, some slowness
REPL	interactive read-eval-print loop	<code>hathor-cli shell</code> for live exploration
Module	one <code>.py</code> file = a namespace	top-level code runs once → singletons
Package	a directory of modules (<code>__init__.py</code>)	dotted name mirrors folder path
<code>sys.modules</code>	cache of imported modules	"runs once"; circular-import hazard
<code>sys.path</code>	ordered import search path	what virtual environments manipulate

Concept	What it is	Why it matters here
absolute import	full dotted path from root	the dominant, file-revealing style
Why Python	I/O-bound + ecosystem + C hot paths	speed traded for velocity, mitigated

"Python the platform" is the interpreter that turns your source into bytecode and runs it, plus the module-and-import system that turns a directory tree into one navigable program, plus the ecosystem and tooling layered on top. A node accepts Python's performance cost because it is I/O-bound and pushes its hot paths into compiled libraries, buying clarity and a mature ecosystem in return. The single most useful habit to take from this chapter: read every import as a file path, and the whole repository opens up. The next chapter follows the import search path (`sys.path`) to the place packages actually live — and explains what a **virtual environment** really is, the `.venv` you keep seeing but may never have looked inside.

1. CPython is the reference implementation of the Python interpreter, written in C. It is what you get from python.org and what runs by default when you type `python`. Alternatives (PyPy, Jython, GraalPy) exist but are not used by `hathor-core`. ↵
2. Bytecode is a compact, low-level instruction set that the Python source is compiled to. It is not your CPU's machine code; it runs on Python's own virtual machine. Cached in `.pyc` files under `__pycache__`. ↵
3. The Python Virtual Machine (PVM) is the part of CPython that executes bytecode — a loop that reads and performs bytecode instructions one at a time. "Virtual machine" here means a software CPU, not a whole virtualized computer. ↵
4. The REPL (Read-Eval-Print Loop) is the interactive Python prompt: it reads a line, evaluates it, prints the result, and loops. Started by running `python` with no arguments. ↵
5. A module is a single `.py` file, importable as a namespace whose attributes are the names defined at its top level. The module's name is the filename without `.py`. ↵
6. A namespace is a mapping from names to objects — a "where names live" container. Modules, packages, classes, and function locals are all namespaces; the dot operator reaches into them. ↵
7. A package is a directory of modules (and sub-packages) treated as one importable unit, traditionally marked by an `__init__.py` file. Its dotted import name mirrors its directory path. ↵
8. `sys.modules` is a dictionary CPython keeps of every module already imported in the current process, keyed by dotted name. Imports check it first, which is why a module's top-level code runs only once. ↵
9. `sys.path` is the ordered list of directories Python searches to find a module on import. It includes the standard library and the active environment's installed-packages directory; the first match wins. ↵

Chapter 12 — Virtual Environments

What you'll learn in this chapter

- The problem virtual environments solve — **dependency hell** — and why installing packages "globally" breaks down the moment you have two projects.
- What a `.venv` actually is: not magic, but a directory holding its own Python, its own `site-packages`, and a few scripts.
- Where third-party packages physically live (**site-packages**) and how that connects to the import search path from Chapter 11.
- What "**activating**" an environment really does (it's mostly one change to `PATH`), and why you can ignore activation entirely if you address the environment's Python directly.
- How **pip** installs into an environment, and how this all sets up **Poetry** (Chapter 13), which manages an environment for you.
- A **bridge** to how `hathor-core` is run inside one.

Chapter 11 ended at the import search path, `sys.path`, and noted that one entry on it — where installed packages live — is what a virtual environment manipulates. This chapter opens that box. A virtual environment¹ is one of those tools that feels mysterious until you look inside, at which point it becomes obvious and even a little mundane — which is exactly the understanding you want. You have seen `.venv` directories and `source .../activate` incantations; by the end of this chapter you'll know precisely what they are and why `hathor-core`, like every serious Python project, lives inside one.

This is a short \$5-style primer: the problem, what the thing is, how it works, how Hathor uses it, and why it beats the alternatives.

12.1 The problem: one shared pile of packages

When you install Python, it comes with one global location where third-party packages go — the system **site-packages**² directory. Run `pip install requests` without any environment, and `requests` lands in that one shared pile, available to every Python program on the machine. For a single project, fine. The trouble starts with the second project.

Picture two projects on one machine:

- **Project A** needs `twisted` version 22 and `pydantic` version 1.
- **Project B** needs `twisted` version 24 and `pydantic` version 2.

With one shared site-packages, there is exactly one installed version of `twisted` and one of `pydantic` for both. Install what B needs and you break A; reinstall for A and you break B. They cannot coexist. This is **dependency hell**³: independent projects fighting over a single global namespace of package versions. It gets worse with transitive dependencies (the packages your packages need), where two libraries demand incompatible versions of a third.

Three further pains compound it:

- **Polluting the system Python.** Your operating system itself often uses Python for its own tools. Installing or upgrading packages globally can break those, sometimes badly.
- **No reproducibility.** "It works on my machine" — because your global pile happens to have the right versions. A teammate's differs, and the code behaves differently or won't run.
- **No clean uninstall.** A project's dependencies are tangled into the global pile with everyone else's; you can't cleanly remove just one project's footprint.

The fix is isolation: give each project its own private pile of packages, walled off from the system and from other projects. That private pile is a virtual environment.

12.2 What a `.venv` actually is

Here is the demystifying truth: **a virtual environment is just a directory.**

Conventionally named `.venv` or `venv`, it contains a self-sufficient, isolated Python setup for one project. Look inside and you find three things that matter:

```
.venv/
├── bin/                (Scripts/ on Windows)
│   ├── python         → a symlink (or tiny copy) pointing at a real interpreter
│   ├── pip
│   └── activate        → a script that tweaks your shell (see §12.3)
├── lib/
│   └── python3.x/
│       └── site-packages/ ← THIS project's private package pile
└── pyvenv.cfg          → a small config naming the base interpreter
```

The two load-bearing parts:

1. **Its own `python`**. The `bin/python` inside the environment is a **symlink**⁴ (or thin wrapper) back to a real Python interpreter installed on your system — the environment does not contain a whole copy of Python. But when you run that `python`, it is configured to use the environment's own `site-packages`, not the global one.
2. **Its own `site-packages`**. This is the private pile from §12.1. Packages installed into this environment land here and nowhere else. Project A's environment has `twisted` 22 here; Project B's has `twisted` 24 in its separate `.venv/lib/.../site-packages`. They never collide, because they are different directories.

That is the whole trick, and it connects straight back to Chapter 11: **the environment's `python` puts its own `site-packages` on `sys.path` instead of the global one**. Recall that `import` searches `sys.path` and the first match wins (Chapter 11 §11.5). A virtual environment is, at bottom, a mechanism for swapping which `site-packages` directory is on the import search path. Everything else — activation, the scripts — is convenience around that one idea.

```
running the GLOBAL python:    sys.path includes /usr/lib/.../site-packages
running .venv/bin/python:    sys.path includes .venv/lib/.../site-packages
                             (the global one is excluded)
→ same `import twisted`, different twisted, decided by WHICH python you ran
```

12.3 What "activation" really does

You'll see instructions to activate an environment: `source .venv/bin/activate`. This sounds like flipping a switch deep in Python, but it is almost entirely a shell convenience. Activation mainly does one thing: it **prepends the environment's `bin/` directory to your shell's `PATH`**⁵.

`PATH` is the ordered list of directories your shell searches when you type a command name. By putting `.venv/bin` at the front, activation ensures that when you type `python` or `pip`, the shell finds the environment's `python/pip` first — so commands "just use" the environment without your typing its full path. (It also sets a couple of environment variables and usually changes your prompt to show the active environment's name, so you don't lose track of which one you're in.)

The clarifying consequence: **activation is optional**. Because the real isolation lives in which `python` binary runs (§12.2), you can skip activation entirely and address the environment directly:

```
# with activation:
source .venv/bin/activate
python run_node.py          # "python" now means .venv's python

# without activation – identical effect:
.venv/bin/python run_node.py # address the environment's python directly
```

Both run the same interpreter with the same private `site-packages`. Activation is just so you can type `python` instead of `.venv/bin/python` repeatedly. This matters in practice because **tools that manage environments for you** — Poetry (Chapter 13), Docker (Chapter 15) — often run the environment's Python directly and never "activate" anything in the interactive sense. Understanding that activation is cosmetic keeps those tools from seeming mysterious.

12.4 Installing into an environment, and reproducibility

Inside an environment, **pip**⁶ — Python's package installer — fetches packages (from the Python Package Index, **PyPI**⁷) and places them in this environment's `site-packages`. `pip install twisted` while environment A is active installs Twisted into A alone.

This gives back everything dependency hell took away:

- **Isolation:** each project's versions are independent; A's Twisted 22 and B's Twisted 24 coexist on one machine, in separate environments.
- **A clean system Python:** you never touch the global pile, so OS tools stay safe.
- **Reproducibility:** because an environment is built from an explicit list of packages, a teammate (or a server, or a Docker image) can build an identical environment from the same list — "works on my machine" becomes "works in every environment built from this spec."

That last point is where plain pip shows its limit, and where the next chapter enters. A bare `pip install twisted` installs whatever the latest compatible version is today — which may differ next week, breaking reproducibility. Serious projects therefore record the exact versions of every package (and every transitive dependency) in a **lock file**, and use a tool that installs precisely those. That tool, for `hathor-core`, is **Poetry** — Chapter 13. A virtual environment is the where (an isolated pile); Poetry is the what and which version (a managed, locked, reproducible set of packages installed into that pile).

12.5 Bridge — environments around `hathor-core`

A virtual environment is the container every other Track C tool fills. Forward-pointers:

Bridge — virtual environments in the project and the stack:

- **It manipulates `sys.path`.** The environment's whole job is to put its private `site-packages` on the import search path of Chapter 11 (§11.5) — that is the mechanical link between these two chapters — **Chapter 11**.
- **Poetry manages the environment.** `hathor-core` does not ask you to create or activate a `.venv` by hand; **Poetry** creates one and installs the locked dependency set into it. The environment is the destination; Poetry is the manager — **Chapter 13**, next.
- **The lock file makes it reproducible.** The exact versions filling the environment are pinned in `poetry.lock`, so the same environment can be rebuilt anywhere — **Chapter 13**.
- **Docker is an environment taken to the extreme.** A container isolates not just Python packages but the entire operating environment (system libraries like RocksDB's, the interpreter, the app) — the same isolation instinct, one level down — **Chapter 15**.
- **Why this isolation matters for a node.** Running mainnet infrastructure demands that the software run identically on the developer's laptop and the production server; environments (then Poetry, then Docker) are the layered answer to that demand — recurring through **Chapters 13 & 15**.

Recap

Concept	What it is	Why it matters
Dependency hell	projects fighting over one global package pile	the problem environments solve
site-packages	the directory installed packages live in	each environment has its own
<code>.venv</code>	a directory: own <code>python</code> + own <code>site-packages</code>	the isolation, made concrete
the env's <code>python</code>	a symlink to a real interpreter, own search path	isolation lives here, not in activation
activation	prepends <code>.venv/bin</code> to <code>PATH</code>	convenience; optional
<code>pip</code> / <code>PyPI</code>	installer / package repository	fills an environment

Concept	What it is	Why it matters
reproducibility	rebuild an identical environment elsewhere	needs a lock file → Poetry (Ch 13)

A virtual environment is not magic — it is a directory holding a symlinked Python and a private `site-packages`, whose entire effect is to swap which packages the import system of Chapter 11 can find. It cures dependency hell by giving each project its own isolated pile, keeps the system Python untouched, and lays the groundwork for reproducibility. Activation is a shell convenience you can skip; the real isolation is in which interpreter you run. What a bare environment lacks is a record of exactly which versions belong in it — and that gap is precisely what the next chapter fills: **Poetry**, the tool that declares, locks, and installs `hathor-core`'s dependencies into an environment it manages for you.

1. A virtual environment is an isolated, self-contained Python setup for one project — a directory with its own interpreter link and its own package directory — so each project's dependencies stay separate from other projects' and from the system Python. ↩
2. `site-packages` is the directory where third-party (pip-installed) packages are placed. There is a global one for the system interpreter and a private one inside each virtual environment; which is used depends on which `python` runs. ↩
3. Dependency hell is the situation where multiple projects (or libraries) require conflicting versions of the same package, making it impossible to satisfy them all from a single shared installation. ↩
4. A symbolic link (symlink) is a filesystem entry that points to another file or directory — like a shortcut. A virtual environment's `python` is typically a symlink to a real interpreter, so the environment needn't copy all of Python. ↩
5. `PATH` is an environment variable holding an ordered list of directories the shell searches to resolve a typed command name. Activating a virtual environment prepends its `bin/` so the environment's `python` / `pip` are found first. ↩
6. `pip` is Python's standard package installer. It downloads packages (by default from PyPI) and installs them into the active environment's `site-packages`. Poetry uses the same underlying mechanisms but adds dependency resolution and locking. ↩
7. PyPI (the Python Package Index, at pypi.org) is the central public repository of third-party Python packages that pip downloads from by default. ↩

Chapter 13 — Poetry: Dependency Management

What you'll learn in this chapter

- The gap a dependency manager fills that a bare virtual environment (Chapter 12) leaves open: which exact versions belong in it.
- The difference between **declaring** dependencies (ranges you accept) and **locking** them (the exact versions you got) — and why both files exist.
- How to read `hathor-core`'s `pyproject.toml`: metadata, the Python constraint, runtime vs. **dev dependency groups**, version-constraint syntax (`^`, `~`), and a git-sourced dependency.
- What **dependency resolution** is and why it is harder than it sounds.
- What a **build backend** is, and how `pyproject.toml` doubles as packaging config.
- **Why Poetry** and not plain `pip` / `requirements.txt` / `pipenv` — the trade-off.

Chapter 12 gave us the where: an isolated `.venv` with its own `site-packages`. It left a gap — a bare environment has no record of what should fill it or which versions. A `pip install twisted` grabs whatever is newest today, which breaks reproducibility tomorrow. **Poetry**¹ closes that gap: it is the tool `hathor-core` uses to declare its dependencies, resolve and pin them, manage the virtual environment, and package the project. This chapter reads the project's real `pyproject.toml` to make each idea concrete.

It follows the \$5 technology-primer shape, with the "why this and not the alternatives" discussion at the end.

13.1 The problem: a list of packages is not enough

Imagine managing dependencies by hand. You'd keep a list — "we need Twisted, Pydantic, structlog" — and `pip install` them into your environment (Chapter 12). Two problems surface immediately, and they are the problems every dependency manager exists to solve.

Problem 1 — which versions? "We need Twisted" is underspecified. Twisted 22 and Twisted 24 differ; code written against one may break on the other. So you need to record not just which packages but which versions — and not too rigidly, because you also want to accept compatible bug-fix updates without editing the list constantly. You need a way to say "Twisted, somewhere in the 24.7.x range."

Problem 2 — the dependencies of your dependencies. Twisted itself needs other packages (zope.interface, and more), each with its own version requirements, each with their dependencies — the **transitive**² dependency tree. Two of your direct dependencies might demand conflicting versions of a shared deep dependency. Working out a single set of versions that satisfies everyone is **dependency resolution**³, and doing it by hand is infeasible past a handful of packages.

A dependency manager solves both: you declare your direct needs with flexible constraints, and it resolves the entire transitive tree into one consistent set of exact versions, then records that exact set so it can be reproduced. Poetry splits these two jobs across two files.

13.2 Two files: declare in `pyproject.toml`, pin in `poetry.lock`

This split is the heart of Poetry, and of most modern dependency managers. Keep the two roles distinct:

- `pyproject.toml` — what you declare: your direct dependencies, each with a flexible **version constraint** (a range of acceptable versions). This is human-authored and human-readable. It expresses intent: "I want Twisted compatible with 24.7."
- `poetry.lock` — what Poetry resolved: the exact, pinned version of every package, direct and transitive, that satisfies all the constraints together. This is machine-generated, not hand-edited, and it expresses fact: "we are using Twisted exactly 24.7.0, zope.interface exactly X, ..." `hathor-core`'s lock file is ~234 KB — far larger than `pyproject.toml`, because it records the whole resolved tree.

<code>pyproject.toml</code> (you write)		<code>poetry.lock</code> (Poetry generates)
"twisted ~24.7.0"	—resolve→	twisted == 24.7.0
"pydantic ^2.0"		pydantic == 2.x.y
		zope.interface == ... (transitive)
		... every package, exact version ...
intent (ranges)		fact (pinned, reproducible)

Why two files instead of one? Because the two needs conflict. You want **flexibility** when declaring (accept compatible updates) and **exactness** when installing (everyone gets the identical environment). A range can't give reproducibility (it means different

versions on different days); a pinned list can't give flexible updates. So Poetry keeps the range in `pyproject.toml` and the resolved pins in `poetry.lock`. When you run `poetry install`, it reads the lock file and installs those exact versions — so a teammate, a CI runner, and the production Docker image all build a byte-identical environment. When you deliberately want updates, `poetry update` re-resolves the ranges and rewrites the lock. The lock file is committed to git for exactly this reason: it is the reproducibility guarantee.

13.3 Reading `hathor-core`'s `pyproject.toml`

The real file (`pyproject.toml` at the repo root) is the project's dependency manifest. The parts worth reading:

Project metadata and the Python constraint. The package is named `hathor` (`pyproject.toml:16`), and it declares which Python it runs on: `python = ">=3.11,<4"` (`:56`). That constraint is enforced — Poetry refuses to install into an incompatible interpreter — which is how the platform requirement from Chapter 11 becomes machine-checked rather than a README note.

Runtime dependencies live under `[tool.poetry.dependencies]` (`:55`). A representative few:

```
twisted      = "~24.7.0"           # the async engine (Ch 16)
pydantic     = "^2.0"              # validation (Ch 18)
structlog    = "~22.3.0"          # logging (Ch 17)
cryptography = "~42.0.5"          # crypto primitives (Ch 40)
rocksdb      = {git = "https://github.com/hathornetwork/python-rocksdb.git"}
```

Two constraint operators appear, and they mean different things:

- **~ (tilde, "compatible patch")** — `~24.7.0` accepts `>=24.7.0, <24.8.0`: bug-fix updates within 24.7, but not 24.8. Conservative; used where minor changes might risk breakage.
- **^ (caret, "compatible minor")** — `^2.0` accepts `>=2.0, <3.0`: any 2.x, but not 3.0. Looser; used where the project trusts the library's promise not to break within a major version (this is the **semantic versioning**⁴ contract).

The `rocksdb` line is worth noting: instead of a version, it points at a **git URL** — a custom fork maintained by Hathor (`hathornetwork/python-rocksdb`). Dependency managers can pull from sources other than PyPI; here it's because the node needs a specific build of the RocksDB bindings (Chapter 27).

Dependency groups. Development tools — the test runner, type checker, linters — are declared separately under `[tool.poetry.group.dev.dependencies]` (`:40`): `pytest`, `mypy`, `mypy-zope`, `flake8`, `isort`, `pytest-cov`, `pytest-xdist` (the quality toolchain, Chapter

20). This **grouping** matters: tools needed to develop the node are not needed to run it. When building the production Docker image (Chapter 15), Poetry installs only the main group (`--only=main`), leaving `pytest` and `mypy` out of the shipped image — a smaller, leaner deployment. Separating "what it needs to run" from "what we need to build it" is a real operational win.

Console scripts. The line `hathor-cli = 'hathor_cli.main:main' (:38)` is what creates the `hathor-cli` terminal command, wiring it to the `main()` function (Chapter 21). Poetry installs this as an executable in the environment's `bin/`.

13.4 The build backend: `pyproject.toml` is also packaging config

`pyproject.toml` is not Poetry-specific in origin — it is a Python standard (defined by PEP 518 and PEP 517⁵) for declaring a project's build configuration in one file. The `[build-system]` table (:190) names the **build backend**⁶ — the code that turns the source tree into an installable package:

```
[build-system]
build-backend = "poetry.core.masonry.api"    # pyproject.toml:192
```

A **build backend** is the component that knows how to package a project into the standard distributable formats — a **wheel**⁷ (a pre-built, installable archive) and an **sdist** (a source archive). Here it's Poetry's own backend. This is why one file does double duty: it is both the dependency manifest and the packaging instructions. In the Docker build (Chapter 15), you'll see `poetry build -f wheel` produce a wheel from exactly this config, which is then pip-installed into the final image. The standardization (PEP 517/518) means tools other than Poetry can also read and build the project — the config isn't locked to one tool.

13.5 Why Poetry, and not the alternatives?

Several tools manage Python dependencies. The §5 "why this one" discussion:

vs. plain pip + `requirements.txt`. The oldest approach: a flat text file of packages, installed with `pip install -r`. It works, but it conflates declaration and locking (one file, usually either too loose to reproduce or too pinned to update cleanly), does no real cross-package resolution (pip historically installed greedily and could leave incompatible versions), and doesn't manage the virtual environment or packaging. You end up gluing together pip, `venv`, a separate `setup.py`, and discipline. Poetry unifies all of that.

vs. pipenv. Pipenv pioneered the declare/lock split for applications (`Pipfile` + `Pipfile.lock`) and is the closest predecessor in spirit. Poetry added first-class packaging (the build backend of §13.4) and dependency groups, and converged on the PEP-standard `pyproject.toml` rather than a bespoke `Pipfile`. For a project that is both an application and a publishable package, Poetry's packaging integration is the deciding factor.

vs. newer tools (uv, PDM, Hatch). The ecosystem keeps evolving — faster resolvers and installers have appeared since Poetry. Poetry's advantage for `hathor-core` is maturity and stability: it has been the standard for years, the team's workflow and CI are built around it, and switching a working dependency setup carries risk with little reward. (This is the same "maturity beats novelty" logic that keeps the project on Twisted — Chapter 16.)

The honest summary: Poetry gives one tool for declaring, resolving, locking, environment-managing, and packaging, on top of the PEP-standard `pyproject.toml`. You pay a little — Poetry can be slower than the newest resolvers, and it is one more thing to install — and you get reproducibility and a unified workflow in return. For long-lived infrastructure where "the build must be identical everywhere" is non-negotiable, that trade is firmly worth it.

13.6 Bridge — Poetry across the project

Bridge — Poetry in the project and the stack:

- **It fills the virtual environment of Chapter 12.** Poetry creates/manages the `.venv` and installs the locked set into it — the manager to Chapter 12's container — **Chapter 12**.
- **The Python constraint enforces Chapter 11.** `python = ">=3.11,<4"` (`pyproject.toml:56`) makes the platform requirement machine-checked — **Chapter 11**.
- **Dependency groups shape the Docker image.** `--only=main` excludes dev tools from the production image — **Chapter 15**.
- **Every dependency it pins gets its own chapter.** Twisted (16), structlog (17), Pydantic (18), configargparse (19), cryptography (40), the RocksDB fork (27) — Poetry is the manifest tying them together.
- **The console script is the CLI's front door.** `hathor-cli = 'hathor_cli.main:main' ( :38 )` — **Chapter 21**.
- **The quality tools are declared here.** The dev group lists `pytest/mypy/flake8/isort` — **Chapter 20**.

Recap

Concept	What it is	Where
<code>pyproject.toml</code>	declared deps (ranges) + metadata + build config	repo root
<code>poetry.lock</code>	resolved, pinned exact versions (incl. transitive)	repo root, committed
<code>~ / ^</code> constraints	patch-compatible / minor-compatible ranges	:57 , :80
dependency groups	runtime vs. dev tooling, installable separately	:40 , :55
git dependency	a dep pulled from a repo, not PyPI	rocksdb :74
build backend	code that packages the project (wheel/sdist)	:192
console script	maps a terminal command to a function	:38
why Poetry	one tool: declare+resolve+lock+env+package	vs pip/pipenv/uv

Poetry is the tool that turns "a list of packages we need" into "the exact, reproducible environment the node runs in everywhere." It splits intent from fact — flexible ranges declared in `pyproject.toml`, exact versions pinned in `poetry.lock` — resolves the whole transitive tree into one consistent set, manages the virtual environment of Chapter 12, separates runtime from development dependencies, and doubles as the project's packaging config via the PEP-standard build backend. It is chosen over pip and its successors for maturity and an all-in-one workflow, accepting some speed cost for guaranteed reproducibility. The next chapter looks at how those many Poetry and tool commands are bundled into memorable shortcuts: **Make**, the project's task runner.

1. Poetry is a Python dependency manager and packaging tool. It declares dependencies in `pyproject.toml`, resolves and pins them in `poetry.lock`, manages a project's virtual environment, and builds distributable packages. ↩
2. A transitive dependency is a dependency of a dependency — a package you don't ask for directly but that something you depend on needs. The full set forms the transitive dependency tree. ↩
3. Dependency resolution is computing a single set of package versions that simultaneously satisfies every direct and transitive constraint. It can be computationally hard and is the core job of a dependency manager. ↩
4. Semantic versioning (semver) is a convention where a version `MAJOR.MINOR.PATCH` signals the kind of change: PATCH = bug fixes, MINOR = backward-compatible features, MAJOR = breaking changes. Constraint operators like `^` rely on this promise. ↩

5. A PEP (Python Enhancement Proposal) is a design document defining a Python standard or feature. PEP 518 and PEP 517 standardized `pyproject.toml` and the build-backend interface, so packaging isn't tied to one tool. ↩
6. A build backend is the component that turns a project's source into installable distributions (a wheel and/or sdist). `pyproject.toml`'s `[build-system]` table names it; Poetry provides its own. ↩
7. A wheel (`.whl`) is Python's standard pre-built package format — an archive that installs without a compile/build step, making installation fast and deterministic. An sdist is the source-archive counterpart. ↩

Chapter 14 — Make: The Task Runner

What you'll learn in this chapter

- What **Make** originally was (a build system for C) and how projects repurpose it as a **task runner**.
- The anatomy of a **Makefile**: targets, recipes, prerequisites, and variables.
- What **.PHONY** means and why almost every target in `hathor-core`'s Makefile is phony.
- How the real Makefile bundles long tool invocations into short commands (`make tests`, `make check`, `make docker`).
- **Why Make** for this and not a pile of shell scripts or just typing the commands — the trade-off.

You will see instructions like `make tests` and `make check` throughout a project, and the file behind them — the `Makefile` — can look cryptic. **Make**¹ is one of the oldest tools in software (from 1976), built for a different purpose than the one it usually serves today. Understanding both the original purpose and the modern repurposing makes the file legible. `hathor-core` uses Make as a **task runner**²: a single, discoverable place that names the project's common commands so nobody has to memorize long invocations.

This is a short \$5-style primer; Make is a means, not a deep subsystem, so the chapter is proportionate.

14.1 What Make was built for

Make's original job was **building C programs**, and its core idea is genuinely clever for that. A C project compiles many source files into object files, then links them into a program. Recompiling everything on every change is wasteful, so Make tracks **dependencies** between files and their **timestamps**: it rebuilds a target only if something it depends on changed more recently than the target itself. "Rebuild `program` from `a.o` and `b.o`; rebuild `a.o` from `a.c` only if `a.c` is newer." This dependency-and-timestamp engine is what makes Make a **build system**³.

That machinery earns its keep for compiled languages. Python, though, is not compiled by you (Chapter 11 — CPython compiles bytecode automatically), so a Python project rarely needs Make's timestamp-driven rebuild logic. What it does want is the other half

of Make: a tidy way to define named commands. So Python projects use Make as a **task runner** — keeping the "named commands" feature and ignoring the "rebuild-if-newer" feature. Knowing this split is the key to reading the Makefile without confusion.

14.2 Anatomy of a Makefile

A Makefile is a list of **rules**. Each rule has this shape:

```
target: prerequisites
      recipe           ← a shell command (MUST be indented with a TAB, not spaces)
```

- **Target** — usually a filename to build, but for a task runner, just a name you invoke (`tests` , `check`).
- **Prerequisites** — other targets/files that must be ready first (often empty for task-runner targets).
- **Recipe** — the shell command(s) to run, each line indented by a literal **tab** (a famous Make gotcha: spaces won't work).

You invoke a rule by name: `make tests` runs the `tests` target's recipe. Make also supports **variables** to avoid repetition. The real Makefile opens with one (`Makefile:1`):

```
py_sources = hathor/ hathor_tests/ extras/custom_tests/
```

`$(py_sources)` then expands to that list wherever it's used, so the set of source directories is defined once and reused across the lint, format, and type-check targets — change it in one place, every target follows.

14.3 `.PHONY` : targets that aren't files

Here is the one concept that trips up newcomers reading a task-runner Makefile. Recall Make's origin: a target is normally a file to build, and Make decides whether to run the recipe by checking if that file is out of date. But `tests` is not a file — there is no file named `tests` in the project, and we want `make tests` to run every time, not be skipped because "the `tests` file is up to date."

`.PHONY`⁴ declares a target as not a real file, so Make always runs its recipe regardless of timestamps or any same-named file. In a task-runner Makefile, essentially every target is phony, and `hathor-core`'s Makefile marks them as such (the `.PHONY:` lines throughout). When you see `.PHONY: tests`, read it as "`tests` is a command, not a file — always run it." This single declaration is what converts Make from a build system into a reliable task runner.

14.4 What the real Makefile does

`hathor-core`'s Makefile is a catalogue of the project's routine commands, each wrapping a longer tool invocation. The clusters:

Testing. Several targets run `pytest` with different scopes and coverage settings — `tests-cli`, `tests-lib`, `tests-nano`, `tests-genesis`, `tests-quick` — and an aggregate `tests` (Makefile:63) that runs the suite. For example, `tests-cli` (:33) runs `pytest` scoped to `hathor_cli/` with a coverage floor. The point: a contributor types `make tests` instead of remembering a 100-character `pytest` command with the right flags.

Quality checks. `mypy` (:72) runs the type checker over the packages; `flake8` (:80) lints `$(py_sources)`; `isort-check` (:84) verifies import ordering; `yamllint` checks YAML. The aggregate `check` (:96) runs all of them together — this is the single command a contributor (or CI) runs to validate a change before submitting. One name, the whole quality gate (Chapter 20).

Formatting. `isort` (:107) sorts imports in place; `fmt` (:104) is a friendly alias for it. The difference between `isort` and `isort-check` is fix vs. verify: `make fmt` edits your files, `make check` only reports.

Docker. `docker` (:144) builds the image; related targets push it to registries (Chapter 15). The tag is derived from the git tag or a timestamp — logic you'd never want to retype.

The Makefile, read top to bottom, is therefore a **self-documenting list of how to work on the project**: how to test it, check it, format it, and ship it. That is its real value — not the build engine, but the index of commands.

14.5 Why Make, and not the alternatives?

vs. just typing the commands. You could type the full `pytest --cov=... --cov-fail-under=... -p no:warnings ...` each time. Nobody remembers it correctly, everyone runs it slightly differently, and CI drifts from local. `make tests` makes the canonical command the only command — one source of truth that local development and CI share.

vs. a folder of shell scripts. Shell scripts (`scripts/test.sh`, `scripts/check.sh`) would also centralize commands, and many projects use them. Make's edges: everything lives in one discoverable file (`make` with no clean listing, or a glance at the Makefile, shows all available tasks), variables like `$(py_sources)` are shared across tasks without sourcing, and `make` is preinstalled almost everywhere a developer or CI runner exists. The cost is Make's awkward syntax (tabs-not-spaces, its own variable and escaping rules) and that its real power — the dependency graph — goes unused.

vs. a dedicated Python task runner (invoke, nox, tox, just). These are purpose-built for the task-runner job and avoid Make's archaic syntax. They're reasonable choices; `just` in particular is a modern Make-for-tasks. `hathor-core` stays on Make for the usual reason: it is universal, zero-install, and already works. The team would gain cleaner syntax and lose universality and a working setup — rarely worth it for a tool this peripheral.

The honest summary: Make is a 1970s build system pressed into service as a task runner, kept because it is everywhere and already does the job. You accept its quirky syntax and ignore its build-graph engine; you get a single, shared, zero-install index of every command needed to work on the project.

14.6 Bridge — Make across the project

Bridge — Make in the project and the stack:

- **It wraps the quality toolchain.** `make check` runs flake8 + isort-check + mypy + yamllint together — the Chapter 20 gate behind one command — **Chapter 20**.
- **It wraps Poetry/pytest invocations.** The test targets call pytest with the right coverage flags; contributors don't retype them — **Chapters 13 & 20**.
- **It builds the Docker image.** `make docker` runs the build with the right tag logic — **Chapter 15**.
- **It is the contributor's entry point.** The Makefile is the discoverable list of "how to develop this project," sitting above the individual tools.

Recap

Concept	What it is	In the Makefile
Make	build tool repurposed as task runner	the <code>Makefile</code>
target	a named command (here) or file (originally)	<code>tests:</code> , <code>check:</code> , <code>docker:</code>
recipe	shell command(s) under a target, tab-indented	the <code>pytest/mypy/flake8</code> lines
variable	a reused value	<code>py_sources (:1)</code>
<code>.PHONY</code>	"not a file — always run"	nearly every target
<code>make check</code>	run the whole quality gate	<code>:96</code>

Concept	What it is	In the Makefile
<code>make fmt</code> vs <code>check</code>	fix in place vs. verify only	<code>:104</code> vs <code>:84</code>
why Make	universal, zero-install, one file	vs scripts / just / typing it

Make is an old build system that survives in modern Python projects as a task runner: a single, discoverable file that names every routine command — test, lint, type-check, format, build the image — and wraps each long tool invocation behind a short, shared name. The `.PHONY` declarations are what repurpose it, telling Make these targets are commands to always run, not files to conditionally rebuild. It is kept over shell scripts and newer task runners for being universal and already working, at the cost of archaic syntax. The next chapter takes the isolation theme one level deeper than the virtual environment: **Docker**, which packages not just Python dependencies but the entire operating environment the node runs in.

-
1. Make is a build-automation tool (originally 1976) that runs commands defined in a `Makefile`. Built to compile programs by rebuilding only out-of-date files, it is widely repurposed as a task runner for named commands. ↩
 2. A task runner is a tool that defines and runs a project's common named commands (test, lint, build) from one place, so contributors and CI invoke identical commands without memorizing them. ↩
 3. A build system turns source into artifacts, tracking dependencies and timestamps to rebuild only what changed. Make is one; for compiled languages this incremental rebuild is its main value, mostly unused in Python projects. ↩
 4. A phony target (declared with `.PHONY`) is a Make target that does not correspond to a file, so Make runs its recipe every time rather than skipping it based on file timestamps. Essential when using Make as a task runner. ↩

Chapter 15 — Docker: Containerizing the Node

What you'll learn in this chapter

- The problem Docker solves beyond a virtual environment: isolating not just Python packages but the whole operating environment (system libraries, the interpreter, the OS userland).
- The core vocabulary — **image** vs. **container**, **Dockerfile**, **layers**, **base image** — and how a container differs from a virtual machine.
- How `hathor-core`'s **multi-stage** Dockerfile builds a small, clean production image.
- Why a node ships as a container at all, and **why Docker** versus the alternatives.

Chapter 12 isolated Python packages with a virtual environment. But a node needs more than Python packages to run: it needs specific system libraries (the RocksDB C++ library, OpenSSL, graphviz), a specific Python interpreter version, and a predictable operating-system userland around them. A `.venv` doesn't capture any of that. **Docker**¹ does: it packages the entire environment — OS libraries, interpreter, dependencies, and the app — into one portable artifact that runs identically on a laptop, a CI runner, and a production server. This chapter reads `hathor-core`'s real `Dockerfile` to see how. It follows the \$5 primer shape, ending with the "why Docker, not the alternatives" trade-off.

15.1 The problem: "works on my machine," one level deeper

A virtual environment guarantees the same Python packages everywhere (Chapter 12). It does not guarantee the same everything else. The node's RocksDB bindings need the RocksDB C++ system library installed at a specific version; the crypto stack needs OpenSSL; some tooling needs graphviz. These are installed by the operating system's package manager (`apt` on Debian), not by pip — and they differ between a developer's Ubuntu laptop, a teammate's Mac, and a production server. Even the Python interpreter's patch version can differ.

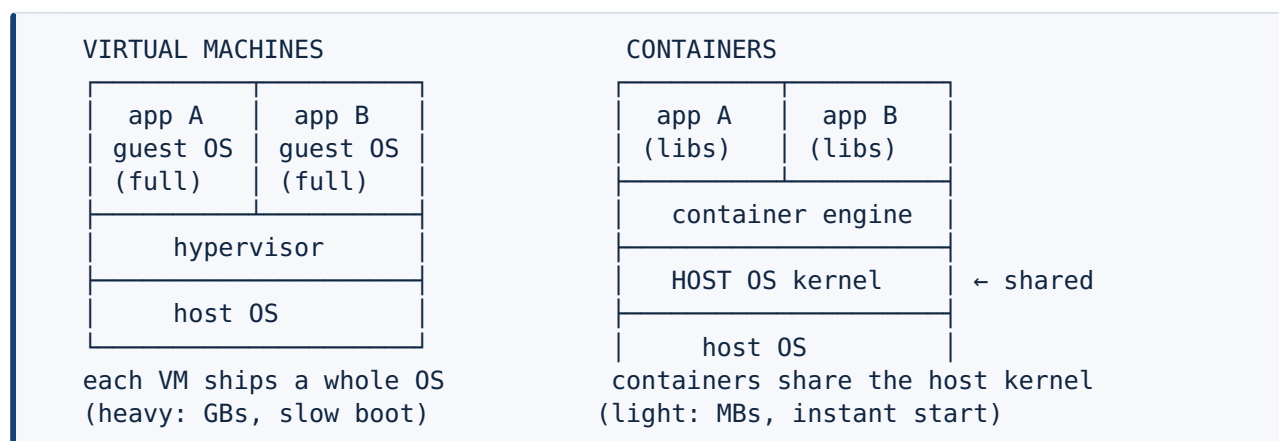
So "works on my machine" can fail one level below the virtual environment: same packages, different system libraries, different behavior — or it won't even start because a `.so` library is missing. The fix is to isolate and ship the whole environment, not just the Python layer. That is **containerization**².

15.2 Containers, images, and how they differ from VMs

Two terms anchor everything, and beginners blur them:

- An **image**³ is a blueprint: a read-only, packaged filesystem containing an OS userland, the interpreter, libraries, and the app — everything needed to run. It is built once and stored.
- A **container**⁴ is a running instance of an image — a live, isolated process tree using that image's filesystem. One image, many containers (like one class, many objects — Chapter 1's distinction, applied to deployments).

The natural question: isn't this just a **virtual machine**⁵? No, and the difference is the reason Docker is practical:



A VM virtualizes hardware and runs a complete guest operating system per app — heavy and slow. A container shares the host's OS **kernel**⁶ and isolates only the userland (filesystem, processes, network) — so it is far lighter (megabytes, not gigabytes) and starts in moments. The container still gets its own copy of system libraries and interpreter, which is exactly the isolation §15.1 needed, without the weight of a whole OS. That lightness is what makes "ship the whole environment" practical for everyday use.

15.3 The Dockerfile: a recipe for an image

An image is built from a **Dockerfile**⁷ — a text recipe of instructions, each producing a **layer**⁸ (a cached filesystem diff). Common instructions: `FROM` (start from a base image), `RUN` (execute a build command, e.g. install packages), `COPY` (bring files in), `EXPOSE` (declare ports), `ENTRYPOINT` (the command to run when a container starts).

The `FROM` instruction names a **base image**⁹ — you never start from bare metal; you start from an existing image (here, an official Python image on a slim Debian) and add to it. Each instruction's layer is cached, so rebuilds skip unchanged steps — the one piece of Make-like incrementalism that is used here.

15.4 Reading `hathor-core`'s multi-stage Dockerfile

The real `Dockerfile` uses a **multi-stage build**¹⁰ — a technique that produces a small final image by doing the heavy building in a throwaway first stage and copying only the results into a clean second stage. Walk it:

Stage 0 — the builder (`Dockerfile:7`). Starts `FROM python:3.12-slim-bullseye` (`:7`; the version comes from `ARG PYTHON=3.12`, `:3`). It then installs a lot: the runtime system libraries (`libssl1.1` `graphviz` `librocksdb6.11`, `:11`) **plus** heavy build-time tooling (`build-essential`, `librocksdb-dev`, `cargo`, `git`, compilers — `:13`) needed to compile the RocksDB bindings and other native code. It installs Poetry, sets `POETRY_VIRTUALENVS_IN_PROJECT=true` (`:16`, so the `.venv` lands in the project dir where it can be copied out later), copies the dependency manifests, runs `poetry install --only=main` (`:20`, production deps only — Chapter 13's group split paying off), copies the source, and builds + installs a wheel (`poetry build -f wheel`, then `pip-installs` it). At the end of this stage, a working environment exists — but so do all the compilers and dev headers, which you do not want in production.

Final stage — the runtime (`Dockerfile:29`). Starts `FROM` the same slim Python base again, fresh. It installs only the runtime system libraries (`libssl1.1` `graphviz` `librocksdb6.11`, `:32`) — none of the build tooling. Then the trick: it **copies the built environment from stage 0** (`:33`), leaving every compiler, header, and build artifact behind in the discarded builder. It exposes the node's ports (`EXPOSE 40403 8080`, `:37` — p2p and API), and sets the entrypoint:

```
ENTRYPOINT ["python", "-m", "hathor"]           # Dockerfile:38
```

So starting a container runs the node directly. The payoff of multi-stage: the final image carries the app and its runtime libraries but not the hundreds of megabytes of build tooling — smaller to store, faster to pull, and a smaller attack surface (no compilers shipped to production).

STAGE 0 (builder) python:3.12-slim + build tools (cargo, gcc, -dev) + poetry install + build wheel (big, messy, discarded)	FINAL STAGE (runtime) python:3.12-slim (fresh) + runtime libs only COPY --from=stage-0 the venv EXPOSE 40403 8080 ENTRYPOINT python -m hathor (small, clean, shipped)
--	---

15.5 Why containerize a node, and why Docker?

Why ship a node as a container at all? A full node is infrastructure operators run on servers they don't control, against a network where behaving identically to other nodes matters. A container guarantees the operator runs the exact environment the developers tested — same interpreter, same RocksDB version, same OpenSSL — eliminating a whole class of "it behaves differently on my server" failures. It also makes deployment one command (`docker run ...`) instead of a page of system-setup instructions, and makes the node easy to run under orchestration (Kubernetes and the like) at scale.

Why Docker, and not the alternatives?

- **vs. a setup script / manual install.** A `setup.sh` that `apt install`s libraries and `pip-install`s the app drifts with the host OS and is hard to reproduce or roll back. An image is immutable and versioned — you run a specific image, and it's identical everywhere.
- **vs. a virtual machine.** A VM gives even stronger isolation but at gigabyte sizes and slow boots (§15.2). For shipping one application, that weight is unjustified; containers give enough isolation far more cheaply.
- **vs. other container runtimes (Podman, etc.).** Docker is the de facto standard with the largest tooling and registry ecosystem; the Dockerfile format it defined is understood by the alternatives anyway. Standardization is the deciding factor.

The honest trade-off: containers add a build step and a runtime engine, and share the host kernel (so they're slightly less isolated than VMs). In return you get reproducible, portable, immutable deployments that capture the entire environment — the property a node operator most needs.

15.6 Bridge — Docker across the project

Bridge — Docker in the project and the stack:

- **It is environment isolation taken to the OS level.** Where Chapter 12's `.venv` isolates Python packages, the container isolates the whole userland — same instinct, one layer down — **Chapter 12**.
- **It consumes Poetry's group split.** `poetry install --only=main` keeps dev tools out of the production image — **Chapter 13**.
- **It bundles the system libraries the node binds to.** `librocksdb` for storage (Chapter 27), `libssl` for crypto (Chapter 40) — the native deps a `.venv` can't supply.
- **make docker builds it.** The Chapter 14 target wraps the build with tag logic — **Chapter 14**.
- **The entrypoint is the node.** `python -m hathor` launches the CLI/run_node path — **Chapter 21**; the exposed ports are p2p (Chapter 34) and the API.

Recap

Concept	What it is	In the Dockerfile
image	read-only blueprint of a full environment	built by the Dockerfile
container	a running instance of an image	<code>python -m hathor</code>
vs. VM	shares host kernel; light, fast	(not a VM)
base image	the image you start FROM	<code>python:3.12-slim (:7)</code>
layer	a cached step in the build	each <code>RUN / COPY</code>
multi-stage	build heavy, ship light	<code>stage-0 (:7) → final (:29)</code>
<code>--only=main</code>	exclude dev deps from the image	<code>:20</code>
ENTRYPOINT	command run on container start	<code>:38</code>

Docker containerizes the node: it packages the interpreter, the system libraries (RocksDB, OpenSSL), the dependencies, and the app into one portable, immutable image that runs identically everywhere — solving the "works on my machine" problem one level below the virtual environment, at the level of the operating-system userland. `hathor-core`'s multi-stage Dockerfile does the heavy compiling in a throwaway builder and copies only the result into a small, clean runtime image. Containers achieve this far more cheaply than virtual machines by sharing the host kernel. This closes the

foundational-tooling cluster (Poetry, Make, Docker); the remaining Track C chapters return to the libraries the node's code calls directly, starting with how it records what it does: **structlog** and structured logging.

1. Docker is a platform for building and running containers — isolated, portable packages of an application together with its entire runtime environment (libraries, interpreter, OS userland). ↵
2. Containerization is packaging an application with everything it needs to run into a container image, so it runs consistently across different machines regardless of their host configuration. ↵
3. An image is a read-only, layered package of a filesystem (OS userland, libraries, app) — the blueprint from which containers are created. Built from a Dockerfile, stored in registries. ↵
4. A container is a running instance of an image: an isolated process (or process tree) with its own filesystem view, using the image's contents. Many containers can run from one image. ↵
5. A virtual machine emulates a complete computer, running a full guest operating system on a hypervisor. Stronger isolation than a container but far heavier, because each VM ships and boots an entire OS. ↵
6. The kernel is the core of an operating system, managing hardware, processes, and memory. Containers share the host's kernel (unlike VMs, which each run their own), which is why they are lightweight. ↵
7. A Dockerfile is a text file of instructions (`FROM` , `RUN` , `COPY` , `ENTRYPOINT` , ...) describing how to build an image. Each instruction produces a cached layer. ↵
8. A layer is the filesystem change produced by one Dockerfile instruction. Layers are cached and reused, so rebuilds skip unchanged steps; images are stacks of layers. ↵
9. A base image is the existing image a Dockerfile starts `FROM` (here an official Python-on-Debian image). You build on top of it rather than from an empty filesystem. ↵
10. A multi-stage build uses multiple `FROM` stages in one Dockerfile, doing the heavy building in early stages and copying only the needed results into a final, smaller image — leaving build tools behind. ↵

Chapter 16 — Twisted: Reactor, Deferreds, Protocols

What you'll learn in this chapter

- What **Twisted** is — the asynchronous-networking framework `hathor-core` is built on — and the concrete problem it solves that plain Python does not.
- The **reactor**: Twisted's name for the single event loop the entire node runs on, and what `reactor.run()` actually does.
- The **Deferred**: Twisted's name for a future, and how `addCallback` / `addErrback` chains, `maybeDeferred`, and `@inlineCallbacks` tame asynchronous code.
- The **Protocol** and **Factory** pattern — the two objects behind every network connection — and why the callbacks `connectionMade`, `dataReceived`, and `connectionLost` are the backbone of the P2P layer.
- The supporting cast: **endpoints**, the **LoopingCall** timer, **callLater**, and **deferToThread** / the thread-pool escape hatch for CPU-bound work.
- **Why Twisted and not asyncio** — the honest trade-off — and how Hathor's own `hathor/reactor/` abstraction keeps the door open to an `asyncio`-backed reactor via `--x-asyncio-reactor`.

Chapter 2 built the ideas of asynchronous programming on plain Python: the event loop, callbacks, futures, `async` / `await`, the never-block rule, and the thread-pool escape hatch. It ended with a deliberate promissory note — its Bridge (\$2.8) mapped each idea onto a Twisted name and pointed here. This chapter pays that note off. Every concept you met in Chapter 2 has a Twisted spelling, and this is where we learn it on the real framework and the real `hathor-core` code.

Twisted is not background detail. It is the most all-touching dependency in the codebase: the node is a Twisted program. Boot ends with one call into Twisted (`reactor.run()`), and from that instant Twisted owns the one thread the whole node lives on. If you internalize this chapter, a great deal of the codebase's shape — small methods that return fast, objects whose methods are named like events, the conspicuous absence of `time.sleep` — stops being mysterious.

We follow the technology-primer template: what it is, the problem it solves, its core components, tiny generic examples, then the real cited Hathor usage, and finally the trade-off against the alternative (`asyncio`).

16.1 What Twisted is

Twisted is an event-driven networking framework for Python. Two words in that sentence carry the weight.

Networking — Twisted's reason to exist is writing programs that talk to many other computers at once: servers, clients, peer-to-peer nodes. It ships mature, tested implementations of the protocols such programs need (TCP, TLS, HTTP, DNS, and more), so you do not re-write socket plumbing for every project.

Event-driven — Twisted does not run your program top-to-bottom. You register handlers for things that might happen — "a connection arrived," "bytes are ready to read," "this timer expired" — and hand control to Twisted's central loop, which waits for those events and calls your handlers when they occur. This is exactly the event-loop model of Chapter 2, packaged as a framework.

Recap — the event loop (full treatment in Ch. 2). An event loop is a `while` loop that does one thing forever: take the next ready piece of work, run it to completion, repeat. It lets a single thread make progress on thousands of tasks by switching between them whenever one would otherwise wait. Twisted's event loop is called the **reactor**. → full treatment in Ch. 2 (§2.4 builds one from scratch).

Twisted is old by software standards — it predates Python's own `asyncio` by more than a decade. That age is mostly an asset (its protocol implementations have been battle-tested in production for years) and partly a liability (it has its own vocabulary, written before `async / await` existed). We name both sides honestly in §16.7.

The category is worth fixing in your mind: Twisted is to asynchronous networking what a web framework is to HTTP request-handling — a body of reusable machinery plus a set of conventions you write your code into. You do not call Twisted so much as Twisted calls you. That inversion — "don't call us, we'll call you" — is the defining feel of the framework, and it follows directly from the event-driven model.

16.2 The problem it solves

Recap — the blocking problem (full treatment in Ch. 2, §2.1). A blocking¹ call (a naive `socket.recv`, a `time.sleep`) freezes the calling thread until it finishes. A node holds thousands of network connections at once; if reading from one slow peer froze the whole program, no other peer could be served. The classic fixes are one-thread-per-connection (memory-heavy, and shared memory invites race conditions and locks) or a single-threaded **event loop** that never blocks. A node is overwhelmingly I/O-bound² — it spends its life waiting on slow conversations — so the event-loop model fits it best. → full treatment in Ch. 2.

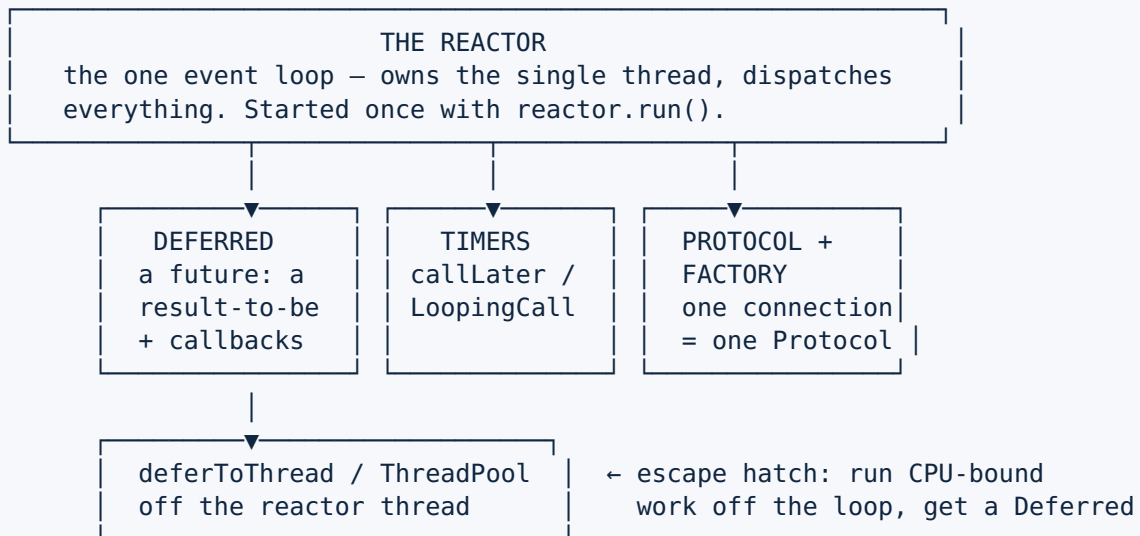
Concretely, without a framework like Twisted you would have to write, by hand:

1. **The loop itself** — the `while` loop that asks the operating system "which of my thousands of sockets have data right now?" (via a system call like `epoll / select`) and dispatches to the right handler.
2. **A timer subsystem** — "run this callback in 30 seconds," kept sorted by due-time and checked every iteration.
3. **Protocol parsers** — turning a raw, arbitrarily-chunked byte stream into whole messages (TCP gives you bytes, not messages; a single `recv` may return half a message or three).
4. **TLS, DNS, connection-retry logic, graceful shutdown** — each a project in itself.

Twisted provides all of this, correct and tested, and asks only that you express your logic as handlers it can call. The problem it solves, in one line: **it lets you write a high-concurrency network program as a set of small callbacks on one thread, without hand-rolling the event loop, the timers, and the protocol plumbing.**

16.3 The core model and its components

Twisted has a small number of central abstractions. Learn these six names and you can read most of the asynchronous code in `hathor-core`:



- **The reactor** — the event loop. There is exactly one per process. It owns the single thread; everything else is something the reactor calls or something that gives the reactor more work. (§16.4)
- **The Deferred** — Twisted's future: an object standing in for a result that is not ready yet, to which you attach callbacks. (§16.5)
- **Timers** — `reactor.callLater(delay, fn)` runs `fn` once after a delay; `LoopingCall` runs a function repeatedly on a fixed interval. (§16.6.4)
- **Protocol** — an object representing one connection; the reactor calls its methods (`dataReceived`, ...) when events happen on that connection. (§16.6.2)
- **Factory** — an object that manufactures a fresh Protocol instance for each new connection, and holds the state shared across connections. (§16.6.3)
- **Endpoints / deferToThread** — endpoints describe "where to listen or connect"; `deferToThread` pushes a blocking or CPU-bound job onto a worker thread and hands you a Deferred for its result. (§16.6.4)

The rest of the chapter takes these one at a time: generic Twisted first, then Hathor's real code.

16.4 The reactor — the one loop

The reactor is the production version of the `MiniLoop` you built in Chapter 2. It is a single object, created once, that runs the event loop for the whole process. You obtain it, schedule some initial work, and then hand it the thread:


```

from twisted.internet import reactor

def say_hello():
    print("the reactor is running now")

reactor.callWhenRunning(say_hello)  # schedule work for once the loop starts
reactor.run()                       # HAND OVER THE THREAD – blocks here forever
print("this line runs only after the reactor stops")

```

`reactor.run()` is the line that matters. Before it, you are in ordinary top-to-bottom Python: importing, configuring, wiring objects together. `reactor.run()` starts the loop and **does not return** until the reactor is stopped — from that call onward, your program is reactive. It sleeps until an event (a connection, readable bytes, an expired timer) wakes it, runs the matching callback to completion, and sleeps again. This is the single most consequential fact about the structure of a Twisted program: there is a clear "before" (synchronous setup) and "after" (the reactive event loop), and the boundary is `reactor.run()`.

A few reactor methods recur:

- `reactor.callWhenRunning(fn)` — run `fn` once, as soon as the loop is up.
- `reactor.callLater(delay, fn)` — run `fn` once, `delay` seconds from now (a one-shot timer).
- `reactor.listenTCP(port, factory)` — start accepting connections; build a Protocol (via the factory) for each.
- `reactor.connectTCP(host, port, factory)` — open an outgoing connection.
- `reactor.addSystemEventTrigger('after', 'shutdown', fn)` — run `fn` during clean shutdown.
- `reactor.stop()` — stop the loop; `reactor.run()` then returns.

That last pair shows up in Hather at once: the manager registers its own `stop` to run during reactor shutdown (`hathor/manager.py:157`), so a clean shutdown of the loop tears the node down in order.

How `hathor-core` obtains and runs the reactor

Hather does not import the global Twisted reactor scattered across the codebase. It wraps reactor access behind its own small package, `hathor/reactor/`, for two reasons we unpack in §16.7: to give the reactor a precise type (Twisted's reactor is typed loosely), and to allow swapping in an `asyncio`-backed reactor. The package exposes two functions (`hathor/reactor/__init__.py:15`):

```

from hathor.reactor.reactor import get_global_reactor, initialize_global_reactor
from hathor.reactor.reactor_protocol import ReactorProtocol

```

`initialize_global_reactor` is called exactly once, early in boot, and `get_global_reactor` hands the same instance to everyone afterward (`hathor/reactor/reactor.py:29`):

```
def get_global_reactor() -> ReactorProtocol:
    global _reactor
    if _reactor is None:
        raise Exception('The reactor is not initialized. Use `initialize_global_reactor()`'.)
    return _reactor
```

Recap — the singleton (full treatment in Ch. 3, §3.4). A singleton is a pattern guaranteeing one shared instance of something, reached through a global accessor. The module-level `_reactor` plus `get_global_reactor()` is exactly that — and Twisted's reactor is inherently a singleton (there can be only one event loop per process). Chapter 3 flagged the global-state smell; here it is unavoidable, because the loop is singular. → full treatment in Ch. 3.

The actual boot path lives in the `run_node` command. It initializes the reactor with the `asyncio` flag drawn from the command line (`hathor_cli/run_node.py:202`):

```
from hathor.reactor import initialize_global_reactor
reactor = initialize_global_reactor(use_asyncio_reactor=self._args.x_asyncio_reactor)
```

and, at the very end of startup — after the whole node has been assembled and the manager started — hands over the thread (`hathor_cli/run_node.py:592`):

```
self.reactor.run()
```

That single line is the hinge of the entire node lifecycle. Everything in Part II up to Chapter 33 is setup that happens before it; everything the node does in steady state happens inside the loop it starts. When Chapter 0 said "the reactor is put in gear," this is the gear.

16.5 The Deferred — Twisted's future

Recap — the future (full treatment in Ch. 2, §2.5). A future is an object representing "a result that isn't ready yet." Instead of blocking until a value arrives, an asynchronous function returns a future immediately; you attach a callback that fires when the value is ready, and you can chain callbacks so each step feeds the next. This tames the nested "pyramid of doom" of raw callbacks. Twisted's name for a future is the **Deferred**. → full treatment in Ch. 2.

A `Deferred` is an object that says: "I don't have the result yet. Here is where you attach the code to run when I do — and here is where you attach the code to run if it fails instead." Those two halves are the heart of the design. A `Deferred` carries **two chains of callbacks**:

- the **callback** chain — run on success, each receiving the previous one's return value;
- the **errback** chain — run on failure, each receiving a `Failure`³ (Twisted's wrapper around an exception).

A tiny generic `Deferred`

```
from twisted.internet.defer import Deferred

def got_result(value):
    print("success:", value)
    return value * 2          # whatever I return feeds the NEXT callback

def got_error(failure):
    print("something failed:", failure.getErrorMessage())

d = Deferred()
d.addCallback(got_result)    # attach a success handler
d.addErrback(got_error)      # attach a failure handler

# ... later, when the result is ready, somebody "fires" the Deferred:
d.callback(21)               # prints "success: 21", and got_result returns 42
```

Two things to notice. First, you attach handlers before the result exists; firing the `Deferred` (`d.callback(21)`) is what triggers them. Second, callbacks **chain**: `got_result` returns `42`, and that value would be passed to the next `addCallback` if one were attached. This is how you express "read A, then transform A, then store the result" as a pipeline rather than a pyramid:

```
d.addCallback(parse)         # step 1: parse the raw bytes
d.addCallback(validate)      # step 2: validate the parsed object (gets parse's result)
d.addCallback(store)         # step 3: store it (gets validate's result)
d.addErrback(log_failure)    # if ANY step above raised, jump here
```

If any callback raises, control skips the remaining callbacks and jumps to the next errback — the same way a `try / except` skips to its handler. This unified success-and-error chaining is the `Deferred`'s real contribution over a bare callback.

`maybeDeferred` — "synchronous or asynchronous, I don't care"

A recurring problem: a handler you call might finish immediately (return a plain value) or might kick off asynchronous work (return a `Deferred`). You would rather not branch on which. `maybeDeferred(fn, *args)` solves this: it calls `fn`, and whatever happens, you get back a `Deferred` — already-fired if `fn` returned a value or raised, still-pending if `fn` returned a `Deferred`. The caller treats every case uniformly.

`inlineCallbacks` — straight-line asynchronous code

Chaining `.addCallback(...)` still has visible plumbing. Twisted's answer — predating `async / await` — is the `@inlineCallbacks` decorator, which lets you `yield` a `Deferred` and write asynchronous code that reads top-to-bottom:

```
from twisted.internet.defer import inlineCallbacks

@inlineCallbacks
def load_all():
    a = yield read_async("a")    # pause here until the Deferred fires; loop runs others
    b = yield read_async("b")    # then pause here
    print(a, b)                 # both ready, in order, no nesting
```

Read `yield someDeferred` as exactly the `await` of Chapter 2: "pause this function here, hand the one thread back to the reactor so it can run other work, and resume me right here with the result once the `Deferred` fires." `@inlineCallbacks` is Twisted's coroutine mechanism — the same idea as `async / await`, spelled with `yield` because it was built before Python had the newer keywords. (Inside an `@inlineCallbacks` function on older Python you "returned" a value with `returnValue(x)`; on modern Python a plain `return x` works, so `returnValue` is legacy you may still encounter.)

Recap — decorators (full treatment in Ch. 4). A decorator wraps a function to give it new behavior without rewriting it: `@inlineCallbacks` transforms an ordinary generator function into one the reactor can drive step-by-step, pausing at each `yield`. You met the "interpose, don't rewrite" idea, and `@inlineCallbacks` specifically, in Chapter 4's Bridge. → full treatment in Ch. 4.

Deferreds in `hathor-core`

The manager imports the `Deferred` machinery directly (`hathor/manager.py:23`):

```
from twisted.internet import defer
from twisted.internet.defer import Deferred
```

The cleanest real example is the P2P protocol's message dispatcher. When a message arrives, `recv_message` looks up a handler and runs it — but a handler may finish synchronously or asynchronously, so it wraps the call in `maybeDeferred` and chains a success callback and an errback (`hathor/p2p/protocol.py:322`):

```
def recv_message(self, cmd: ProtocolMessages, payload: str) -> None:
    """ Executed when a new message arrives. """
    assert self.state is not None
    now = self.reactor.seconds()
    self.last_message = now
    ...
    cmd_handler = self.state.cmd_map.get(cmd)
    if cmd_handler is None:
        ...
        self.send_error_and_close_connection('Invalid Command: {} {}'.format(cmd, payload))
        return

    deferred_result: Deferred[None] = defer.maybeDeferred(cmd_handler, payload)
    deferred_result \
        .addCallback(lambda _: self.reset_idle_timeout()) \
        .addErrback(self._on_cmd_handler_error, cmd)
```

This one method shows the Deferred pattern at work. `maybeDeferred(cmd_handler, payload)` (:342) turns "synchronous or asynchronous handler" into one shape — always a Deferred. Then a callback/errback pair is chained onto it (:343 – :345): on success, reset the connection's idle timer; on failure, jump to `_on_cmd_handler_error` — a single place that catches whatever any command handler raised, instead of a `try/except` around every command. The errback reads the original exception back out of the boxed `Failure` (`hathor/p2p/protocol.py:347`):

```
def _on_cmd_handler_error(self, failure: Failure, cmd: ProtocolMessages) -> None:
    self.log.error(f'recv_message processing error:\n{failure.getTraceback()}',
                  reason=failure.getErrorMessage())
    self.send_error_and_close_connection(f'Error processing "{cmd.value}" command')
```

`failure.getTraceback()` and `failure.getErrorMessage()` are how you recover the traceback and message from the `Failure` box that travelled down the errback chain. Note the extra argument: `addErrback(self._on_cmd_handler_error, cmd)` passes `cmd` along, so the failure handler knows which command failed — extra positional args to `addCallback / addErrback` are forwarded to the handler after the result/failure.

16.6 Protocols, factories, endpoints, timers

This section covers the four pieces that turn the reactor into an actual network node. Each gets a generic Twisted sketch, then Hathor's real usage.

16.6.1 The connection lifecycle, in callbacks

Recall the central inversion: Twisted calls you. A network connection in Twisted is represented by a **Protocol** object whose methods are callbacks the reactor invokes as the connection's life unfolds:

```

reactor accepts/opens a connection
    |
    ▼
connectionMade()      ← "you're connected; set things up"
    |
    ▼ (bytes arrive, possibly many times)
dataReceived(bytes)   ← "here are some bytes that just arrived"
dataReceived(bytes)
    |
    ▼
connectionLost(reason) ← "the connection ended; clean up"

```

These three method names — `connectionMade`, `dataReceived`, `connectionLost` — are the §2.3 callback pattern made the backbone of every connection. You do not call them; you implement them, and the reactor calls them at the right moments.

16.6.2 A tiny generic Protocol and Factory

```

from twisted.internet.protocol import Protocol, Factory

class Echo(Protocol):                                # ONE connection
    def connectionMade(self):
        print("a peer connected")
    def dataReceived(self, data):                    # called each time bytes arrive
        self.transport.write(data)                  # echo them straight back
    def connectionLost(self, reason):
        print("peer gone:", reason)

class EchoFactory(Factory):                          # MAKES one Echo per connection
    def buildProtocol(self, addr):
        return Echo()

```

Two objects, two responsibilities — and this division is the point:

- A **Protocol** instance handles **one** connection. Its per-connection state (buffers, who the peer is, how far a handshake has progressed) lives on `self`. A new one is created for every connection and discarded when that connection closes. `self.transport` is the object you call to send bytes back out.
- A **Factory** is created **once** and makes a fresh Protocol for each incoming connection via `buildProtocol(addr)`. Shared, cross-connection state (a registry of all peers, configuration, references to the rest of the application) lives on the factory, so every Protocol it builds can reach it.

Recap — the factory pattern (full treatment in Ch. 3, §3.1). A factory is an object whose job is to create other objects, centralizing the "how to build one" decision. Twisted's `Factory.buildProtocol` is a textbook example: the reactor says "a connection arrived, give me a handler," and the factory manufactures one — wiring in whatever shared dependencies the new Protocol needs. → full treatment in Ch. 3.

You start a server by handing the factory to the reactor:

```
from twisted.internet import reactor
reactor.listenTCP(8000, EchoFactory())    # accept connections on port 8000
reactor.run()
```

`listenTCP` tells the reactor to accept TCP connections on the port; for each one, it calls `EchoFactory().buildProtocol(addr)`, attaches a transport, and starts delivering events to that Protocol's callbacks.

16.6.3 Hathor's Protocol and Factory

Hathor's per-connection object is `HathorProtocol`, and its docstring states the one-per-connection rule outright (`hathor/p2p/protocol.py:51`):

```
class HathorProtocol:
    """ Implements Hathor Peer-to-Peer Protocol. An instance of this class is
        created for each connection.
    """
    ...
```

A note on layering. Raw TCP delivers an arbitrarily chunked byte stream — a single `dataReceived` call might hand you half a message, or two-and-a-half messages. Hathor does not parse that by hand; it builds on Twisted's `LineReceiver`, which buffers the stream and calls back once per complete line. The concrete class combines Hathor's logic with that Twisted base (`hathor/p2p/protocol.py:430`):

```
class HathorLineReceiver(LineReceiver, HathorProtocol):
    """ Implements HathorProtocol in a LineReceiver protocol.
        It is a TCP connection which sends one message per line. """
    def connectionMade(self) -> None:
        super(HathorLineReceiver, self).connectionMade()
        self.setLineMode()
        self.on_connect()                                # Hathor's per-connection setup

    def connectionLost(self, reason: Failure = connectionDone) -> None:
        super(HathorLineReceiver, self).connectionLost()
        self.on_disconnect(reason)                      # Hathor's per-connection cleanup

    def lineReceived(self, line: bytes) -> None:         # one whole message at a time
        ...
        self.recv_message(cmd, msgdata)                 # → the dispatcher of §16.5
```

This is multiple inheritance used as composition of behaviors: `LineReceiver` supplies the buffer-the-stream-into-whole-lines machinery; `HathorProtocol` supplies the Hathor handshake and message logic. The lifecycle hooks are overridden to do both — `connectionMade` (:436) calls up to `LineReceiver` and then runs Hathor's `on_connect`, and `connectionLost` (:441) does the same with `on_disconnect`. Crucially, Hathor does not implement raw `dataReceived` at all; `LineReceiver` does, re-assembling the chunked byte stream into whole lines and calling `lineReceived` (:450) once per complete

message — which parses the command and routes to the `recv_message` dispatcher of §16.5 (:477). The reactor sees a Protocol; Hathor sees its own peer object; Twisted's base class bridges the two.

The matching factory lives in `hathor/p2p/factory.py`. A shared base implements `buildProtocol` to manufacture a fresh `HathorLineReceiver` per connection, wiring in the dependencies every connection needs (`hathor/p2p/factory.py:43`):

```
class _HathorLineReceiverFactory(ABC, protocol.Factory):
    inbound: bool

    def buildProtocol(self, addr: IAddress) -> HathorLineReceiver:
        p = HathorLineReceiver(
            my_peer=self.my_peer,          # this node's own identity    (shared)
            p2p_manager=self.p2p_manager,  # the connections manager      (shared)
            use_ssl=self.use_ssl,
            inbound=self.inbound,
            settings=self._settings,
        )
        p.factory = self                  # give the Protocol a handle back to its factory
        return p
```

Two concrete factories subclass it — one for connections we accept, one for connections we open (`hathor/p2p/factory.py:55` , `:61`):

```
class HathorServerFactory(_HathorLineReceiverFactory, protocol.ServerFactory):
    inbound = True          # a peer connected to us

class HathorClientFactory(_HathorLineReceiverFactory, protocol.ClientFactory):
    inbound = False         # we connected to a peer
```

The split is the textbook one: shared, cross-connection state (`my_peer` , the `ConnectionsManager` , TLS settings) lives on the factory, created once; per-connection state lives on each `HathorProtocol` , created fresh by `buildProtocol` . The single `inbound` flag is all that distinguishes "they called us" from "we called them" — useful because the handshake differs slightly by direction. We meet this whole machine properly — the handshake, the peer state machine, sync-v2 — in **Chapters 34-35**; here it is the canonical illustration of the Protocol/Factory split.

16.6.4 Endpoints, timers, and the thread-pool escape hatch

Endpoints. `reactor.listenTCP` / `connectTCP` are the low-level way to start listening or connecting. Twisted also offers endpoints — small objects that describe "where" (a TCP server on port 8000, a TLS client to `host:port`) independently of the act of using them. They are a thin convenience layer over the same reactor calls; the typed reactor abstraction Hathor uses exposes `listenTCP` and `connectTCP` directly (`hathor/reactor/reactor_tcp_protocol.py:33` , `:43`), and the P2P layer builds its connections on top.

Timers — `callLater` and `LoopingCall`. A node needs both one-shot deadlines and repeating background work. Twisted offers two timer forms:

- `reactor.callLater(delay, fn)` — fire `fn` once, after `delay` seconds.
- `LoopingCall(fn)` — fire `fn` repeatedly, every N seconds, until stopped.

Hathor uses `callLater` for one-shot deadlines, such as the per-connection idle timeout — if a peer goes quiet, fire `on_idle_timeout` later (`hathor/p2p/protocol.py:247`):

```
self._idle_timeout_call_later = self.reactor.callLater(self.idle_timeout, self.on_idle_timeout)
```

Generic `LoopingCall`:

```
from twisted.internet.task import LoopingCall

def heartbeat():
    print("still alive")

lc = LoopingCall(heartbeat)
lc.start(5.0, now=False)      # run heartbeat() every 5 seconds; don't fire immediately
```

Hathor uses exactly this to poll its sync state. The manager creates the `LoopingCall` in its constructor (`hathor/manager.py:251`):

```
self.lc_check_sync_state = LoopingCall(self.check_sync_state)
self.lc_check_sync_state.clock = self.reactor
self.lc_check_sync_state_interval = self.CHECK_SYNC_STATE_INTERVAL
```

and starts it when the manager starts (`hathor/manager.py:331`):

```
self.lc_check_sync_state.start(self.lc_check_sync_state_interval, now=False)
```

Two details worth pausing on. The `clock = self.reactor` assignment (:252) makes the timer read its sense of "now" from the reactor rather than the wall clock — which is what lets the **simulator** (Chapter 43) advance time deterministically in tests by stepping a fake reactor's clock. And `now=False` means "wait one full interval before the first run," not "fire immediately." The polled method itself, `check_sync_state` (`hathor/manager.py:933`), reads the current time from `self.reactor.seconds()` (:935) — the node asks the reactor for the time, never the OS, so everything stays consistent under simulation.

Recap — never block the loop (full treatment in Ch. 2, §2.7). The reactor runs one callback at a time, to completion. So any callback that blocks — `time.sleep`, a synchronous network or disk call, a long CPU crunch — freezes every connection, timer, and task until it returns. The rule is absolute: never block the reactor. Each callback must do a little and return fast. → full treatment in Ch. 2.

The thread-pool escape hatch — CPU-bound work off the loop. The never-block rule has one awkward exception: genuinely CPU-bound work that cannot be made to yield. Proof-of-work⁴ hashing is the canonical case — it must grind a hash function as fast as possible, with no I/O to yield on. Running it on the reactor thread would freeze the node. The escape hatch (Chapter 2, §2.7) is to push that work onto a **thread pool**⁵ of worker threads and get a Deferred for its result, leaving the reactor free.

The manager creates a dedicated pool for exactly this (`hathor/manager.py:234`):

```
# Thread pool used to resolve pow when sending tokens
self.pow_thread_pool = ThreadPool(minthreads=0, maxthreads=settings.MAX_POW_THREADS, name='Pow th
```

It is started when the node starts (`hathor/manager.py:310`) and stopped on shutdown (`hathor/manager.py:356`):

```
self.pow_thread_pool.start()
...
if self.pow_thread_pool.started:
    self.pow_thread_pool.stop()
```

The generic Twisted way to send one job to a pool and get a Deferred back is `deferToThread`:

```
from twisted.internet.threads import deferToThread

d = deferToThread(expensive_hash, block_header) # runs on a worker thread
d.addCallback(on_hash_found)                   # fires back on the reactor thread
```

The crucial property: `deferToThread` runs the function on a worker thread, but the Deferred's callbacks fire back **on the reactor thread** — so your success handler is back in the safe single-threaded world, with no locks needed. This is the one place the node deliberately reaches for threads, fenced off behind a Deferred. (The comment on `pow_thread_pool` names the use: resolving proof-of-work when sending a transaction, so the API call does not block the reactor while it hashes.) The proof-of-work machinery that uses this pool is detailed in **Chapter 37**.

16.7 Why Twisted and not the alternatives

This is the mandatory trade-off discussion. The honest version requires acknowledging that the obvious modern alternative — Python's own `asyncio` — did not exist when Twisted was chosen, and that Hathor has built a bridge toward it.

Twisted vs. `asyncio`

Python's standard library now ships `asyncio`: an event loop, futures, and the `async / await` syntax, built into the language. It covers much of the same ground as Twisted. So why is `hathor-core` a Twisted program?

The case for Twisted:

- **Maturity and battle-testing.** Twisted is over two decades old. Its TCP, TLS, and protocol implementations have run in production at scale for years. `asyncio` is younger and, especially in its early years, had rougher edges and fewer ready-made protocol implementations.
- **It predates `asyncio` by a decade-plus.** When the architecture was laid down, Twisted was the answer for asynchronous networking in Python; `asyncio` was not yet a credible option. Much foundational code, and the developers' fluency, grew up Twisted-shaped. Rewriting a working node's entire `async` core onto a different loop is a large, risky change with little user-visible benefit.
- **A complete framework, not just a loop.** Twisted bundles protocol abstractions (Protocol/Factory), `LoopingCall`, thread-pool integration, endpoints, and more as one coherent toolkit. `asyncio` gives you the loop and the primitives; you assemble more of the rest yourself.

The case for `asyncio`:

- **It is standard.** No third-party dependency; `async / await` is built into the language and is what most Python programmers now learn first. New contributors are more likely to know `asyncio` than Twisted's `Deferred / @inlineCallbacks` vocabulary.
- **Cleaner native syntax.** `async / await` is more ergonomic than `@inlineCallbacks + yield`, and the wider ecosystem increasingly targets `asyncio`.

Hathor's actual position — and a verified current reality. Hathor has not picked one and burned the bridge. It keeps the reactor behind the `hathor/reactor/` abstraction precisely so the kind of reactor can be chosen at boot. `initialize_global_reactor` takes a `use_asyncio_reactor` flag, and when set, it installs Twisted's **`asyncio reactor`** — a Twisted reactor whose event loop is, underneath, Python's `asyncio` loop (`hathor/reactor/reactor.py:44`):

```
def initialize_global_reactor(*, use_asyncio_reactor: bool = False) -> ReactorProtocol:
    ...
    if use_asyncio_reactor:
        import asyncio
        from twisted.internet import asyncioreactor
        from twisted.internet.error import ReactorAlreadyInstalledError
        try:
            loop = asyncio.get_running_loop()
        except RuntimeError:
            loop = asyncio.new_event_loop()
            asyncio.set_event_loop(loop)
        try:
            asyncioreactor.install(loop)
        except ReactorAlreadyInstalledError as e:
            ...
    from twisted.internet import reactor as twisted_reactor
    ...
```

This is a best-of-both arrangement: the code stays written against Twisted's APIs (Deferreds, Protocols, `LoopingCall`), but the underlying loop can be Python's `asyncio`. The option is exposed on the command line as `--x-asyncio-reactor` (`hathor_cli/run_node.py:161`), and the boot path threads the flag straight through to `initialize_global_reactor` (`run_node.py:202`).

Two honest caveats about the current reality, both visible in the code:

- **The `x_` prefix means experimental.** The flag is `--x-asyncio-reactor`. In this codebase the `x- / --x-` convention marks an unstable, opt-in feature. The default path is the standard Twisted reactor; the `asyncio` backend is a deliberate, off-by-default option, not the recommended mode. So the production answer remains "Twisted (default reactor)," with `asyncio` as an experimental alternative.
- **The default is the plain Twisted reactor.** When `use_asyncio_reactor` is false, the function skips the whole `asyncio` block and imports and returns Twisted's already-installed global reactor (`reactor.py:80`), casting it to Hathor's `ReactorProtocol` type. There is even a defensive comment in the code: if an `asyncio` reactor cannot be installed because some import already installed the default reactor indirectly, it raises with a pointed explanation (`reactor.py:70`). (This is a real footgun: importing from `hathor` at module level in a non-`RunNode` CLI tool can install the default reactor before `initialize_global_reactor` runs — which is also why both reactor functions carry the warning "must NOT be called at module level.")

The typed-reactor abstraction (why the wrapper exists at all)

There is a second motive for `hathor/reactor/` beyond the `asyncio` swap, and it ties back to Chapter 5. Twisted's reactor is typed loosely — its capabilities are spread across several `zope.interface`⁶ interfaces (`IReactorCore`, `IReactorTime`, `IReactorTCP`),

which static type-checkers like `mypy` do not follow well. Hathor defines a Python `Protocol`⁷ type that stitches those interfaces into one named type the checker does understand (`hathor/reactor/reactor_protocol.py:22`):

```
class ReactorProtocol(
    ReactorCoreProtocol,
    ReactorTimeProtocol,
    ReactorTCPProtocol,
    Protocol,
):
    """A Python protocol that represents the intersection of Twisted's
    IReactorCore+IReactorTime+IReactorTCP interfaces."""
    pass
```

Each piece stubs one Twisted interface — the time slice declares `seconds()`, `callLater()`, and `getDelayedCalls()` (`hathor/reactor/reactor_time_protocol.py:31`); the core slice declares `run()`, `stop()`, `callWhenRunning()`, and `addSystemEventTrigger()` (`hathor/reactor/reactor_core_protocol.py:38`); the TCP slice declares `listenTCP()` and `connectTCP()` (`hathor/reactor/reactor_tcp_protocol.py:33`). At initialization the code asserts the real reactor genuinely satisfies all three Twisted interfaces with `verifyObject`, then casts it to `ReactorProtocol` (`hathor/reactor/reactor.py:82`):

```
assert verifyObject(IReactorTime, twisted_reactor) is True
assert verifyObject(IReactorCore, twisted_reactor) is True
assert verifyObject(IReactorTCP, twisted_reactor) is True
_reactor = cast(ReactorProtocol, twisted_reactor)
```

So the wrapper buys two things at once: a single, mypy-legible type for "the reactor," and the seam where an alternative (asyncio-backed) reactor can be slotted in. The reactor abstraction itself gets a short dedicated chapter — **Chapter 23** — which builds on this section rather than repeating it.

Recap — Protocol the typing tool vs. Protocol the Twisted class (full treatment in Ch. 5 and §16.6). Confusingly, "Protocol" names two unrelated things here. Twisted's `Protocol` is the connection-handler class of §16.6. Python's `typing.Protocol` is a static-typing tool — "structural typing," i.e. any object with the right methods qualifies, no inheritance required. `ReactorProtocol` uses the typing one to describe the reactor's shape. → full treatment in Ch. 5 (typing) and §16.6 (Twisted Protocol).

And the thread question

The other "alternative" is the one Chapter 2 already settled: why a single-threaded event loop at all, instead of one OS thread per connection? Because a node holds thousands of mostly-idle connections; a thread each would burn memory and invite the race conditions and locking bugs that shared-memory threading is infamous for, and CPython's GIL⁸ would blunt any CPU parallelism anyway. The event-loop model gives

high I/O concurrency on one thread with almost no locking — and reserves real threads for the rare CPU-bound job, via the pool of §16.6.4. Twisted is the framework that makes that model practical in Python.

Recap

Twisted concept	Chapter-2 idea it realizes	Where in <code>hathor-core</code>
Reactor	the event loop	<code>hathor/reactor/</code> ; <code>run_node.py:202</code> , <code>:592</code>
<code>reactor.run()</code>	starting the loop (hand over the thread)	<code>run_node.py:592</code>
Deferred	a future / promise	<code>manager.py:23</code> ; <code>protocol.py:322</code>
<code>maybeDeferred</code> / <code>addErrback</code>	uniform result + error chaining	<code>protocol.py:342</code> , <code>:345</code> , <code>:347</code>
<code>@inlineCallbacks</code> (<code>yield</code>)	<code>async / await</code> (the yield point)	§16.5; cross-ref Ch. 4 Bridge
Protocol	per-event callbacks (<code>dataReceived</code> , ...)	<code>HathorProtocol</code> <code>protocol.py:51</code> , <code>:430</code>
Factory	the factory pattern (Ch. 3)	<code>factory.py:43</code> (<code>buildProtocol</code>) , <code>:55</code> , <code>:61</code>
<code>callLater</code>	one-shot timer	<code>protocol.py:247</code> (idle timeout)
LoopingCall	the periodic timer	<code>manager.py:251</code> , started <code>:331</code>
ThreadPool / <code>deferToThread</code>	the CPU escape hatch	<code>pow_thread_pool</code> <code>manager.py:234</code> , <code>:310</code>
asyncio reactor	swap the loop's backend	<code>reactor.py:44</code> (<code>--x-asyncio-reactor</code>)
<code>ReactorProtocol</code>	a typed name for the reactor (Ch. 5)	<code>reactor_protocol.py:22</code> ; <code>reactor.py:82</code>

Twisted is the asynchronous engine the entire node runs on, and every one of its abstractions is a Chapter-2 idea wearing a Twisted name: the **reactor** is the event loop, the **Deferred** is the future, `@inlineCallbacks` is `async / await` , the **Protocol**'s methods are the event callbacks, and the **thread pool** is the never-block rule's escape

hatch. Hathor wraps reactor access in `hathor/reactor/` for two reasons — a type the checker can follow, and a seam where an experimental `asyncio`-backed reactor can be installed via `--x-asyncio-reactor` (default off; standard Twisted reactor in production). From here, three threads of the book pick these names back up: the **reactor abstraction** gets its short dedicated treatment in **Chapter 23**; the **Protocol/Factory** pattern returns in full, with the handshake and peer state machine, in the **P2P chapters 34-35**; and **Deferreds** and `@inlineCallbacks` recur quietly throughout Part II wherever the node does something that finishes later. With Twisted understood, you can now read the node not as a script that runs but as a loop that reacts.

1. A blocking call does not return control to your program until it has finished; while it waits, the calling thread can do nothing else. Its opposite is non-blocking (returns immediately, result delivered later). Full treatment in Ch. 2. ↩
2. I/O-bound describes work whose speed is limited by input/output — network, disk, waiting on other systems — rather than by the processor. A node is overwhelmingly I/O-bound. Full treatment in Ch. 2. ↩
3. A Twisted Failure is an object that captures an exception together with its traceback, so an error can be carried along a Deferred's errback chain (and inspected later) rather than being raised and lost. Think of it as "an exception, boxed for asynchronous travel." ↩
4. Proof-of-work (PoW) is a scheme where producing a valid block (or transaction) requires finding a number that makes its hash fall below a target — hard to do, trivial to check. The hashing is CPU-bound, which is why Hathor runs it on a thread pool. Full treatment in Ch. 9. ↩
5. A thread pool is a managed set of reusable worker threads. Jobs are submitted to the pool rather than spawning a new thread each time; used to run blocking or CPU-bound work off the main event loop. Full treatment in Ch. 2. ↩
6. `zope.interface` is a library that lets Python code declare interfaces — named sets of methods a class promises to provide — and verify at runtime that an object implements one. Twisted uses it to describe the reactor's capabilities (`IReactorCore` , `IReactorTime` , `IReactorTCP`). It is an older, runtime-checked cousin of `typing.Protocol` . ↩
7. Python's `typing.Protocol` enables structural typing: any object that has the required methods/attributes satisfies the protocol, with no need to inherit from it. It is a static-typing tool, checked by `mypy` , and is unrelated to Twisted's connection-handler `Protocol` class. Full treatment in Ch. 5. ↩
8. The Global Interpreter Lock is a mutex in standard CPython that allows only one thread to execute Python bytecode at a time. It simplifies the interpreter but prevents threads from running Python computation in parallel — a reason the ecosystem leans on the single-threaded event-loop model. Full treatment in Ch. 2. ↩

Chapter 17 — structlog: Structured Logging

What you'll learn in this chapter

- Why `print()` is not how production software records what it does, and the problem **logging** solves.
- What makes logging structured — **key-value events** instead of formatted sentences — and why that matters for a daemon you can't watch live.
- The core model of **structlog**: loggers, **log levels**, bound **context**, and the **processor** pipeline that formats output (human-readable in dev, JSON in production).
- How `hathor-core` configures and uses it (`get_logger`, `logger.info(event, **fields)`).
- **Why structlog** over `print` and Python's standard `logging` — the trade-off.

A node runs unattended for months on a remote server. When something goes wrong — a peer misbehaves, a transaction is rejected, sync stalls — there is no one watching the screen. The only record of what happened is what the node wrote to its **logs**¹. So how a node logs is not a detail; it is the difference between a diagnosable incident and a mystery. `hathor-core` uses **structlog**², a library for **structured logging**, and this chapter explains both the practice and the tool.

It follows the §5 primer shape, ending with the "why this, not the alternatives" discussion.

17.1 The problem: `print` doesn't scale to a daemon

When learning, you debug with `print()`. It fails as production infrastructure for concrete reasons:

- **No severity.** Everything is the same "level." You can't say "this is routine info" vs. "this is a warning" vs. "this is a fatal error," and you can't filter to just the serious ones when the log is millions of lines.

- **No destination control.** `print` goes to standard output. Real software needs to route logs to files, to the system journal, to an aggregation service — and to switch verbosity without editing code.
- **No structure.** A printed sentence like `peer 0xabc disconnected after 30s` is for a human reading one line. When you have millions of lines and need to ask "how many disconnections happened, and which peers?", prose is nearly unsearchable — you're reduced to fragile text-matching.

Logging addresses the first two: a logging framework gives **levels** (DEBUG/INFO/WARNING/ERROR/CRITICAL³) and configurable routing. Structured logging addresses the third, and it is the idea worth dwelling on.

17.2 Structured logging: events, not sentences

The shift is from logging formatted prose to logging data. Instead of building a sentence:

```
logger.info(f"peer {peer_id} disconnected after {seconds}s") # a string
```

you log an **event name plus key-value fields**:

```
logger.info("peer disconnected", peer_id=peer_id, seconds=seconds) # structured
```

The difference looks small and is profound. The structured call produces a record with fields — `event="peer disconnected" peer_id="c17_0xabc" seconds=30` — which can be rendered as JSON⁴ and then **queried like a database**: "show every `peer disconnected` event where `seconds > 60`," "count events grouped by `peer_id`." The prose version can only be grepped with brittle regexes. For a system whose health is understood after the fact from its logs — the definition of **observability**⁵ — this queryability is the whole game.

A second benefit: **context binding**. structlog lets you bind fields to a logger once and have them attached to every subsequent event automatically. Bind `peer_id` when a connection opens, and every log line for that connection carries it without your repeating it — so you can later filter the entire life of one connection out of the interleaved firehose of thousands.

PROSE LOG (grep-only)	STRUCTURED LOG (queryable)
"peer 0xabc disconnected after 30s"	{event: "peer disconnected",
"block accepted height 42"	peer_id: "0xabc", seconds: 30,
→ text matching, fragile	conn_id: 7}
	→ filter/group/count by field

17.3 The structlog model: loggers and the processor pipeline

structlog's design has two parts you'll see in the code.

Getting a logger. A module obtains a logger with `get_logger()` and logs by calling a level method with an event string and fields:

```
from structlog import get_logger
logger = get_logger()
logger.info("removing all events and related data...") # event, optionally + fields
```

This pattern recurs across `hathor-core` (e.g. `hathor_cli/reset_event_queue.py`). The first positional argument is the **event** (a short, stable name); keyword arguments are the structured fields.

The processor pipeline. The mechanism that makes structlog flexible is a chain of **processors**⁶: each log event passes through an ordered list of small functions, each transforming the event dict, before it is finally rendered. Processors add a timestamp, add the level, add bound context, and — at the end — a **renderer** turns the event dict into output. This is the decorator/pipeline idea from Chapter 4 applied to log records: behavior composed as a sequence of transformations.

The renderer at the end of the chain is what makes one log style for humans and another for machines from the same logging calls:

- In **development**, a console renderer produces colorized, aligned, human-friendly lines. `hathor-core` even subclasses structlog's `ConsoleRenderer` (a custom `ConsoleRenderer` in `hathor_cli/util.py`) to tune that output.
- In **production**, a JSON renderer emits one JSON object per line — exactly the queryable form of §17.2, ready for a log-aggregation system.

The configuration that assembles this pipeline lives in `hathor_cli/util.py` (the `setup_logging` function calls `structlog.configure(...)`, wiring the processor chain and renderer; it also optionally adds a Sentry⁷ processor for error reporting). The key idea: the same `logger.info(...)` calls throughout the codebase produce friendly output locally and machine-parseable JSON in production, because only the final renderer changed.

17.4 Why structlog, and not the alternatives?

vs. `print`. Covered in §17.1: no levels, no routing, no structure. Fine for a script, unfit for a daemon.

vs. Python's standard logging. The standard library does have levels and routing (handlers, formatters), and it's the baseline. What it lacks natively is structure and context binding: standard logging is oriented around formatting a message string, and attaching arbitrary key-value fields (and especially binding them across many calls) is awkward. structlog is built around the event-dict-and-fields model from the ground up, and crucially it **integrates with**, rather than replaces, standard logging — it can route its output through the stdlib's handlers, so you keep the mature routing and gain the structure. That "best of both" is the deciding factor.

vs. other structured loggers (loguru, etc.). loguru is a popular ergonomic alternative; the choice between it and structlog is largely taste and ecosystem. structlog's processor-pipeline model and its clean stdlib integration suit a large application that wants explicit control over how events are built and rendered.

The honest summary: structlog costs a little setup (the processor pipeline must be configured once) and a small mental shift (log events with fields, not sentences). In return a long-running node produces logs you can query, filter, and aggregate — the foundation of diagnosing anything in production. For unattended infrastructure, that is not optional polish; it's how the system is operated.

17.5 Bridge — logging across the project

Bridge — structlog in the project and the stack:

- **Configured once at startup.** `setup_logging` in `hathor_cli/util.py` builds the processor pipeline and picks the renderer (console vs. JSON) — called during the boot sequence of **Chapter 21**.
- **Used everywhere.** `get_logger()` + `logger.info(event, **fields)` appears across `hathor/` — you'll see it in nearly every Part II chapter.
- **The processor pipeline is Chapter 4's idea.** A chain of transforming functions ending in a renderer — decorators/dispatch applied to log records — **Chapter 4**.
- **It feeds observability alongside metrics.** Logs answer "what happened"; Prometheus metrics answer "how much/how often" — **Chapter 42**.
- **Optional Sentry integration.** Errors can be forwarded to Sentry via a processor — an example of the optional-dependency fallback pattern of **Chapter 4**.

Recap

Concept	What it is	Why it matters
logging	recording what software does, with levels + routing	the daemon's only witness
log level	severity (DEBUG...CRITICAL)	filter signal from noise
structured logging	events with key-value fields, not prose	logs become queryable
context binding	fields attached once, on every later event	trace one connection's whole life
processor pipeline	chain of transforms ending in a renderer	same calls → console or JSON
renderer	final formatter of the event	human-readable dev / JSON prod
<code>get_logger</code> / <code>.info(event, **f)</code>	the usage pattern	throughout <code>hathor-core</code>
why structlog	structure + binding + stdlib integration	vs print / bare logging

structlog gives `hathor-core` structured logging: every log call records a named event with key-value fields rather than a prose sentence, so the logs a long-running node leaves behind can be filtered, grouped, and counted like data instead of grepped like text. Its processor pipeline lets the same logging calls render as friendly console output in development and machine-parseable JSON in production, and context binding lets a single connection or transaction be traced out of the interleaved stream. It is chosen over `print` (no levels/structure) and bare stdlib logging (awkward structure) while integrating with the latter's routing. The next chapter turns from recording what the node does to validating what enters it: **Pydantic**, which checks external data at the boundary using the very type-hint syntax of Chapter 5.

-
1. Logging is the practice of emitting timestamped records of a program's activity to a destination (console, file, service), typically with severity levels and configurable routing, so behavior can be understood after the fact. ↩
 2. structlog is a Python library for structured logging. It models each log entry as an event name plus a dictionary of key-value fields, passes it through a pipeline of processors, and renders it (human-readable or JSON). ↩

3. A log level is a severity tag on a log entry — commonly DEBUG, INFO, WARNING, ERROR, CRITICAL — used to filter which entries are recorded or shown, so routine detail can be separated from serious problems. ↩
4. JSON (JavaScript Object Notation) is a text format for structured data as key-value objects and arrays. Emitting logs as one JSON object per line makes them directly ingestible by log-analysis tools. ↩
5. Observability is the degree to which a system's internal state can be understood from its external outputs (logs, metrics, traces). Structured logs are a pillar of it, especially for systems no one watches live. ↩
6. A processor in structlog is a function in the pipeline that receives and transforms a log event dict (adding a timestamp, level, bound context, etc.) before the final renderer turns it into output. ↩
7. Sentry is an error-tracking service that aggregates and alerts on exceptions from running software. structlog can forward error events to it via a processor; it is an optional dependency in `hathor-core`. ↩

Chapter 18 — Pydantic: Validation at the Boundary

What you'll learn in this chapter

- The boundary problem: external data (config, CLI args, API requests) arrives untyped and untrusted, and must be checked before the node acts on it.
- The complementary roles of **mypy** (Chapter 5, static, before run) and **Pydantic** (run time, on real data) — same type-hint syntax, opposite timing.
- The core model of **Pydantic**: a **model** is a class whose fields are type-annotated, and constructing it **validates and coerces** the input or raises.
- Model configuration that encodes invariants — **extra='forbid'** , **frozen=True** — and custom **validators**.
- How `hathor-core` uses it (`RunNodeArgs` , `HathorSettings` , a custom `BaseModel`), and **why Pydantic** over hand-written checks.

Chapter 5 taught type hints and `mypy` — a static safety net that checks the code's internal consistency before it runs. But `mypy` cannot help with data the node receives while running: a settings file someone edited, command-line flags, a JSON body from an API client. That data is untyped text and entirely untrusted, and a wrong value can corrupt the node deep in its operation. **Pydantic**¹ is the tool that guards that boundary: it validates external data at run time, turning it into typed, trustworthy objects — and it does so using the same type-hint syntax `mypy` reads. This chapter completes the typing story Chapter 5 began.

It follows the §5 primer shape; the trade-off discussion closes it.

18.1 The problem: the inside is typed, the outside isn't

Inside the node, Chapter 5's hints and mypy keep types consistent. But every program has a **boundary**² where data crosses in from the outside world, and at that boundary all guarantees vanish:

- A **settings** YAML file is just text until parsed — a number might be a string, a required field might be missing, a typo'd key might be silently present (Chapter 22).
- **CLI arguments** arrive as strings — `--listen 8080` gives the string `"8080"`, not the integer `8080`.
- An **API request body** is whatever a client sent — possibly malformed, possibly malicious.

Two dangers follow. First, wrong types: code expecting an `int` gets a `str` and fails far from the source (the dynamic-typing pain of Chapter 5 §5.1, now with attacker-influenced data). Second, invalid values: a number out of range, an unknown field, a missing required key. You want to catch all of this **at the boundary, immediately**, with a clear error — not deep in the call stack hours later. This is the **fail-fast**³ principle, and it is what a validation library provides.

18.2 The complement to mypy: same syntax, opposite timing

Here is the relationship that makes Pydantic click, and it directly extends Chapter 5. Recall that type hints are **ignored at run time** (§5.2) — mypy reads them before the program runs, then they do nothing. Pydantic uses the identical hint syntax but gives it run-time teeth:

mypy (Chapter 5)	Pydantic (this chapter)
reads hints BEFORE running	enforces hints WHILE running
checks code's consistency	checks real data's validity
no effect at run time	raises on bad data at run time
guards: the inside	guards: the boundary

They are complementary, not competing. mypy ensures the code is internally consistent before it ships; Pydantic ensures the data crossing in conforms before the code touches it. A field annotated `port: int` means, to mypy, "treat this as int"; to Pydantic, "at run time, make this an int or raise." Same `: int`, two enforcement moments. Once you see this, the two halves of the typing story fit together.

18.3 The Pydantic model

The central concept is the **model**⁴: a class inheriting from Pydantic's `BaseModel`, whose fields are declared with type hints. Constructing the model from raw data triggers validation.

```
from pydantic import BaseModel

class ServerConfig(BaseModel):
    host: str
    port: int
    debug: bool = False          # a default makes the field optional

cfg = ServerConfig(host="localhost", port="8080")  # note: port is a STRING
print(cfg.port, type(cfg.port))                  # 8080 <class 'int'> ← coerced!

ServerConfig(host="x", port="not-a-number")      # raises ValidationError
```

Two behaviors to notice:

- **Coercion.** `port="8080"` (a string) became the integer `8080`. Pydantic coerces compatible types where it safely can — directly solving the "CLI args arrive as strings" problem of §18.1.
- **Validation with clear errors.** `port="not-a-number"` raises a `ValidationError` describing exactly which field failed and why — at construction, the boundary, not later. This is fail-fast made concrete.

This embodies a principle often summarized as **"parse, don't validate"**⁵: rather than scattering `if`-checks throughout the code, you parse the untrusted input once into a typed object at the boundary; thereafter the rest of the program works with a value that is guaranteed well-formed by its type. The validation happens in exactly one place, and downstream code can trust what it holds. (`hathor-core` uses **Pydantic v2**, the current major version, with APIs like `field_validator`, `model_validator`, and `ConfigDict`.)

18.4 Configuring models: encoding invariants

Beyond field types, a Pydantic model can be configured to enforce stronger invariants — and this is where it connects to the design ideas of earlier chapters. `hathor-core` defines a custom base model (`hathor/utils/pydantic.py:79`) that sets two config options:

```
class BaseModel(pydantic.BaseModel):
    model_config = ConfigDict(extra='forbid', frozen=True)  # utils/pydantic.py:83
```

- `extra='forbid'` — reject any input field the model does not declare. A typo'd or unexpected key raises instead of being silently ignored. This is the safety

property you met in Chapter 22 for settings: a malformed profile fails at load, not later.

- **frozen=True** — make instances **immutable** after construction (assigning to a field raises). This is the read-only invariant of Chapter 1 (§1.4) and Chapter 22, enforced by the library rather than by convention: once parsed, the object cannot be accidentally mutated.

(Note a deliberate distinction the codebase makes: this custom `BaseModel` is `frozen=True`, but `HathorSettings` — Chapter 22 — sets only `extra='forbid'`, not `frozen`. The settings object's read-only-ness rests on `extra='forbid'` plus convention, while this general-purpose base is fully frozen. Both are honest about what they guarantee.)

Custom validators go further than types allow. A field validator runs your own logic on a field's value (e.g. "this weight must be positive"); a model validator checks relationships between fields (e.g. Chapter 22's rule that a proof-of-authority network must not also configure a block reward). You met both in the settings chapter; they are how domain rules — not just type correctness — get enforced at the boundary.

18.5 How `hathor-core` uses it, and why Pydantic

Two representative models, both at the boundary:

- **RunNodeArgs** (`hathor_cli/run_node_args.py:24`) — the parsed, validated form of the node's command-line arguments (Chapter 21). Raw CLI strings become a typed object the rest of the boot path can trust.
- **HathorSettings** (`hathor/conf/settings.py:32`) — the validated network configuration loaded from a YAML profile (Chapter 22), with field and model validators enforcing network rules.

The project also wraps Pydantic with helpers — a custom `BaseModel` (§18.4) and a `Hex` generic for hex-encoded byte fields in JSON — so the same validation discipline applies consistently across the codebase.

Why Pydantic, and not the alternatives?

- **vs. hand-written validation.** You could write `if not isinstance(port, int): raise ...` everywhere. It's verbose, error-prone, easy to forget a case, and scatters the rules across the code. Pydantic declares the rules once in the model's types and config, generates the checks, and produces uniform, descriptive errors. Less code, fewer gaps.
- **vs. dataclasses / TypedDict.** Standard `@dataclass` (and `TypedDict`) give you typed structure but do **no run-time validation** — a `dataclass` happily accepts a string for an `int` field, because (Chapter 5) hints are ignored at run

time. They organize data; they don't guard the boundary. Pydantic adds the enforcement they lack.

- **vs. other validation libraries (marshmallow, cerberus).** These predate Pydantic's type-hint-driven style and use separate schema definitions. Pydantic's advantage is that the type hints are the schema — one declaration serves mypy, the reader, and run-time validation — which is exactly the unification this chapter is about.

The honest trade-off: Pydantic adds a dependency and a small run-time cost (validation isn't free), and its coercion rules occasionally surprise (knowing when it coerces vs. rejects takes learning). In return you get boundary safety with almost no boilerplate, expressed in the same type language as the rest of the code. For a node ingesting untrusted config and requests, that safety is essential.

18.6 Bridge — Pydantic across the project

Bridge — Pydantic in the project and the stack:

- **It completes Chapter 5's typing story.** Same hint syntax, enforced at run time instead of statically — the static/dynamic halves of typing — **Chapter 5**.
- **It guards settings.** `HathorSettings` validates the network profile at load, fail-fast — **Chapter 22**.
- **It guards CLI args.** `RunNodeArgs` turns raw flags into a trusted typed object — **Chapter 21**.
- **It enforces invariants from Chapter 1.** `frozen=True` / `extra='forbid'` make read-only and no-unknown-fields library-enforced, not convention — **Chapters 1 & 22**.
- **mypy understands it.** The `pydantic.mypy` plugin (Chapter 20) lets the type checker reason about models — **Chapter 20**.
- **It is declared by Poetry.** `pydantic = "^2.0"` — **Chapter 13**.

Recap

Concept	What it is	Why it matters
boundary	where external data crosses in	where guarantees vanish, must re-check
mypy vs Pydantic	static (before run) vs run-time	same hints, complementary timing

Concept	What it is	Why it matters
model	<code>BaseModel</code> subclass with typed fields	the unit of validation
coercion	safe type conversion of input	"8080" → 8080
<code>ValidationError</code>	clear failure at construction	fail-fast at the boundary
<code>extra='forbid'</code>	reject unknown fields	typos fail loudly
<code>frozen=True</code>	immutable after construction	read-only invariant, enforced
validators	custom field/model rules	domain rules, not just types
why Pydantic	hints are the schema	vs hand-checks / dataclasses

Pydantic guards the node's boundary: it turns untrusted external data — settings, CLI arguments, API bodies — into typed, validated, optionally-immutable objects at run time, raising a clear error the moment something is wrong rather than failing deep in the system later. It uses the exact type-hint syntax of Chapter 5 but enforces it while running, making it the run-time complement to mypy's static checks, and its model configuration turns design invariants (read-only, no unknown fields, domain rules) into library-enforced guarantees. It is chosen over hand-written checks and plain dataclasses because the type hints are the schema. The next chapter covers the tool that produces some of that boundary data in the first place — `configargparse`, which unifies command-line flags, environment variables, and config files.

1. Pydantic is a Python library for data validation using type hints. A model class declares typed fields; constructing it validates and coerces input data or raises a descriptive error. `hathor-core` uses Pydantic v2. ↩
2. A boundary is any point where data enters a program from outside (files, network, user input, other systems). Data at the boundary is untyped and untrusted and must be validated before the program relies on it. ↩
3. Fail-fast is the principle of detecting and reporting an error as early and as close to its source as possible — here, validating input at the boundary so a bad value raises immediately with a clear message rather than causing an obscure failure later. ↩
4. A Pydantic model is a class subclassing `BaseModel` whose attributes are declared with type hints. It validates and coerces data on construction and provides serialization to/from dicts and JSON. ↩
5. "Parse, don't validate" is a design maxim: instead of repeatedly checking untrusted data throughout the code, parse it once at the boundary into a type that cannot hold invalid data, so downstream code can trust it by construction. ↩
6. Pydantic v2 is the current major version, with a rewritten core and APIs including `ConfigDict`, `field_validator`, and `model_validator`. It is faster than v1 and is what `hathor-core` uses. ↩

Chapter 19 — configargparse: CLI as Configuration

What you'll learn in this chapter

- What Python's standard **argparse** does — turning command-line strings into a structured options object — and where it stops.
- Why production software needs the same option settable three ways: **CLI flag, environment variable, and config file** — and the **precedence** rules between them.
- How **configargparse** extends argparse to provide all three from a single declaration.
- How `hathor-core` uses it (`create_parser`, `auto_env_var_prefix='hathor_'`).
- **Why configargparse**, and how it hands off to Pydantic (Chapter 18) for validation.

A node has dozens of options — which network, which data directory, which ports, which features. How an operator supplies those options matters: a developer wants quick command-line flags; a containerized deployment (Chapter 15) wants environment variables; a long-lived server wants a config file checked into its ops repo. Forcing one mechanism annoys someone. **configargparse**¹ lets a single option be set by any of the three, and `hathor-core` uses it as the front end of its configuration. This is a short §5-style primer — the tool is narrow but the pattern it embodies is worth understanding.

19.1 argparse: the standard starting point

Python's standard library includes **argparse**², the conventional way to handle command-line arguments. You declare the options a program accepts; argparse parses the raw `sys.argv` strings into a structured object, handles types and defaults, and generates `--help` text automatically.

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("--hostname", default="localhost")
parser.add_argument("--port", type=int, default=8080)
args = parser.parse_args()      # reads sys.argv
print(args.hostname, args.port) # e.g. localhost 8080
```

argparse is solid and ubiquitous. Its limit is in the name: it parses arguments — the command line, and nothing else. It has no notion of reading the same option from an environment variable or a file. For a script, that's fine. For deployable infrastructure, it's the start of a familiar problem.

19.2 The problem: one option, three sources

Real software is configured in different ways in different contexts, and the same option needs to be reachable from each:

- **Command-line flags** — immediate and explicit, ideal for development and one-off overrides: `--hostname node1`.
- **Environment variables**³ — the standard way to configure software in containers (Chapter 15) and the cloud, where injecting a flag into a running container is awkward but setting `HATHOR_HOSTNAME=node1` in the environment is natural. This is a core tenet of **twelve-factor**⁴ app design: configuration comes from the environment.
- **Config files** — a persistent, version-controllable record of a deployment's settings, better than a long, fragile command line for a server that runs for months.

If you support these with argparse alone, you write glue: read the file yourself, check environment variables yourself, merge them with the parsed flags yourself, and define the **precedence**⁵ — when the same option is set two ways, which wins? That glue is boilerplate every project reinvents. `configargparse` packages it.

19.3 configargparse: three sources, one declaration

configargparse is a drop-in extension of argparse: you declare each option once, exactly as in argparse, and it can then be supplied by a command-line flag, a matching environment variable, or a config file entry — automatically, with a sensible precedence. Because it subclasses argparse, existing argparse code keeps working; you gain the extra sources for free.

The standard precedence (highest wins) is: **command-line flag → environment variable → config file → the option's default**. An explicit `--hostname` on the command line overrides everything; absent that, the environment variable is used; absent that, the config file; absent all, the default. This ordering matches intuition — the more explicit and immediate the source, the higher its priority — and it means a deployment can set a baseline in a config file or environment and still allow a quick command-line override for debugging.

highest priority → lowest

--hostname node1 (explicit, now)	>	HATHOR_HOSTNAME=node1 (container/cloud)	>	config file (persistent)	>	default (fallback)
-------------------------------------	---	--	---	-----------------------------	---	-----------------------

19.4 How `hathor-core` uses it

The project funnels argument parsing through one helper, `create_parser` in `hathor_cli/util.py:30`:

```
def create_parser(*, prefix=None, add_help=True):  
    return configargparse.ArgumentParser(auto_env_var_prefix=prefix or 'hathor_', ...)
```

The key piece is `auto_env_var_prefix='hathor_'`. This tells `configargparse` to automatically derive an environment-variable name for every declared option by prefixing it — so the `--hostname` flag is also settable via the `HATHOR_HOSTNAME` environment variable, the `--data` flag via `HATHOR_DATA`, and so on, with no per-option wiring. One line grants the whole CLI an environment-variable interface, which is exactly what a containerized node deployment (Chapter 15) wants. The parser this produces is what `run_node` uses to read the operator's options (Chapter 21).

Note the division of labor with the previous chapter. `configargparse` handles where the values come from (flag, env, file) and produces a flat set of raw values. **Pydantic** (Chapter 18) then handles whether those values are valid — the raw parsed arguments are fed into the `RunNodeArgs` model, which validates and coerces them into a trusted, typed object. Source-gathering and validation are two distinct jobs, handled by two tools in sequence: `configargparse` gathers, `Pydantic` validates.

19.5 Why `configargparse`, and not the alternatives?

vs. plain `argparse` + manual glue. You can read env vars and config files yourself and merge them, but you'll reimplement precedence and naming for every project, with bugs. `configargparse` does it once, correctly, as an `argparse` superset — minimal change for a complete feature.

vs. a heavier config framework (e.g. `Dynaconf`, `Hydra`). These offer richer layering, profiles, and dynamic reloading. For a node whose network profile is already handled separately by the settings system (Chapter 22), that power would overlap and overcomplicate. `configargparse`'s narrow job — "the same options, from three sources" — fits the actual need without a large new abstraction.

vs. putting everything in the settings YAML. The network constants live in the settings profile (Chapter 22); the operational options (which data dir, which ports, debug flags) are per-deployment and belong on the CLI/env interface. The two systems are complementary: settings define the network, configargparse+Pydantic define how this operator runs a node on it.

The honest summary: configargparse is a small, focused extension that buys the three-source configuration pattern for almost nothing, staying compatible with the argparse everyone already knows. You don't get the advanced features of a full config framework — which this project doesn't need, because the settings system covers the rest.

19.6 Bridge — configuration input across the project

Bridge — configargparse in the project and the stack:

- **It is the front of the boot sequence.** `create_parser ( util.py:30 )` builds the parser `run_node` uses to read operator options — **Chapter 21**.
- **It hands raw values to Pydantic.** Parsed args feed `RunNodeArgs` for validation — **Chapter 18**.
- **Environment variables suit containers.** `auto_env_var_prefix='hathor_'` gives every flag an `HATHOR_*` env var — the container-friendly config of **Chapter 15**.
- **It is distinct from the settings system.** Operational options (CLI/env) vs. network constants (the YAML profile) — **Chapter 22**.
- **Declared by Poetry.** `configargparse = "~1.7.1"` — **Chapter 13**.

Recap

Concept	What it is	In <code>hathor-core</code>
argparse	stdlib CLI argument parser	the baseline configargparse extends
configargparse	argparse + env vars + config files	<code>create_parser (util.py:30)</code>
env var config	options from the environment	<code>auto_env_var_prefix='hathor_'</code>
precedence	flag > env > file > default	which source wins

Concept	What it is	In <code>hathor-core</code>
12-factor config	configuration from the environment	container-friendly deployment
gather vs validate	source-collection vs correctness	configargparse → Pydantic
why configargparse	three sources, one declaration	vs glue / heavy frameworks

configargparse extends Python's standard argparse so a single declared option can be supplied as a command-line flag, an environment variable, or a config-file entry, with a clear precedence among them — exactly the flexibility deployable infrastructure needs, since a developer, a container, and a long-running server each prefer a different source. `hathor-core` enables this for its whole CLI with one setting, `auto_env_var_prefix='hathor_'`, then hands the gathered values to Pydantic (Chapter 18) for validation: gather, then validate. The last Track C chapter steps back from individual libraries to the discipline that keeps the whole codebase healthy — the **quality toolchain**: the type checker, linter, formatter, and test runner that every change must pass.

1. configargparse is a Python library that subclasses the standard `argparse`, letting each declared option be set from a command-line flag, an environment variable, or a config file, with a defined precedence — a drop-in upgrade for argparse-based programs. ↩
2. argparse is Python's standard-library module for parsing command-line arguments: you declare the options a program accepts, and it parses `sys.argv` into a structured object and generates help text. It handles only the command line. ↩
3. An environment variable is a named value held in a process's environment, set outside the program (in the shell, the container config, or the OS). Software commonly reads configuration from environment variables, especially in containers. ↩
4. Twelve-factor refers to a set of widely-cited principles for building deployable web/network software. One factor is that configuration should come from the environment, keeping config separate from code. ↩
5. Precedence is the rule deciding which source wins when the same option is set in more than one place. configargparse's order, highest first, is: command-line flag, environment variable, config file, default. ↩

Chapter 20 — The Quality Toolchain

What you'll learn in this chapter

- The four distinct quality jobs and the tool for each: **type checking** (mypy), **linting** (flake8), **formatting** (isort), and **testing** (pytest) — and why they are separate roles, not one tool.
- How `hathor-core` configures each, and the one setting they must agree on.
- What a **CI gate** is and how `make check` plus the test targets form it.
- Test scale tools: **pytest-xdist** for parallelism and **coverage** for measuring what the tests exercise.
- **Why** a project invests in this machinery — the trade-off of effort for safety in long-lived, multi-author code.

The final Track C chapter is about the tools that keep the codebase correct and consistent over time, not the ones that make it run. A node is long-lived and worked on by many people; without automated enforcement, type errors creep in, styles diverge, imports tangle, and regressions slip through. `hathor-core` runs four kinds of automated check on every change, each a distinct job. This chapter names them, shows how the project wires them, and explains the **gate** that makes passing them mandatory. It draws together threads from Chapter 5 (typing), Chapter 13 (the dev dependency group), and Chapter 14 (Make).

20.1 Four jobs, four tools

Newcomers often lump these together as "the linting stuff." They are genuinely different jobs, and seeing the distinction is the point of this chapter:

TYPE CHECKER	(mypy)	"do the types line up?"	– correctness of types
LINTER	(flake8)	"is this code suspicious/ugly?"	– style + likely bugs
FORMATTER	(isort)	"are imports ordered right?"	– mechanical layout
TEST RUNNER	(pytest)	"does it actually work?"	– behavior is correct

A **type checker** verifies type consistency without running the code (Chapter 5). A **linter**¹ flags stylistic problems and likely-bug patterns (unused variables, undefined names, lines too long) by static inspection. A **formatter**² rewrites code into a canonical layout (here, import ordering) so style is never debated or hand-fixed. A **test**

runner³ executes the test suite to check behavior. Type checker and linter report; formatter fixes; test runner verifies behavior. Four questions, four answers, four tools — no single one substitutes for another.

20.2 mypy — the type checker

mypy (Chapter 5) reads the type hints across the codebase and reports inconsistencies before the code runs. `hathor-core` configures it under `[tool.mypy]` in `pyproject.toml`: 101, and the configuration is notably strict:

- Global flags like `disallow_incomplete_defs` (:103) and `no_implicit_optional` (:104) reject half-typed functions and the old implicit-Optional behavior.
- **Per-module overrides** (:145 – :170) crank strictness even higher for the core packages — `disallow_untyped_defs`, `warn_return_any`, `no_implicit_reexport`, and more — so the heart of the node is held to a tighter standard than peripheral code. This is gradual typing (Chapter 5 §5.7) used deliberately: maximal strictness where correctness matters most.
- Two **plugins** extend mypy's understanding: `pydantic.mypy` (:114) so it reasons about Pydantic models (Chapter 18), and `mypy_zope:plugin` (:115) so it understands the `zope.interface` contracts Twisted relies on (Chapter 16). The plugins are why mypy can type-check code built on those libraries at all.

Run via `make mypy` (Makefile:72), which invokes `mypy -p hathor -p hathor_tests ...` across the packages.

20.3 flake8 — the linter

flake8⁴ inspects the source for style violations and likely bugs without running it — unused imports, undefined names, overlong lines, and the like. `hathor-core` configures it in `setup.cfg`:34, with `max-line-length = 119` (:35) and a small set of deliberate ignores (e.g. E731, allowing lambda assignment, :37). One targeted exception is worth noting: `hathor/__init__.py` ignores F401 ("imported but unused"), because that file intentionally re-exports names (the package-surface pattern of Chapter 11 §11.4) — an "unused" import there is the whole point. Run via `make flake8` (Makefile:80) over `$ (py_sources)`.

20.4 isort — the formatter

isort⁵ sorts and groups import statements into a canonical order (standard library, third-party, first-party), automatically. Import ordering is exactly the kind of mechanical detail humans get wrong and waste review time on; a formatter ends the

debate by making one ordering the ordering. It's configured under `[tool.isort]` in `pyproject.toml:93`, knowing which packages are "first-party" via `known_first_party = "hathor,hathor_tests" (:97)`.

Two usages, and the difference matters (Chapter 14): `make isort` / `make fmt` rewrites your imports in place; `make isort-check` only verifies ordering without changing files — the form CI uses, so a contributor with unsorted imports gets a failure, not a silent edit.

The one setting they must agree on. isort's `line_length = 119 (:98)` deliberately matches flake8's `max-line-length = 119 ($20.3)`. If a formatter wrapped lines at one width and the linter complained at another, they'd fight forever — the formatter producing code the linter rejects. Keeping the two numbers equal is a small but essential bit of toolchain hygiene: the tools must share the rules they jointly enforce.

20.5 pytest — the test runner

pytest⁶ is the framework that finds and runs the project's automated tests and reports pass/fail. It is configured under `[tool.pytest.ini_options]` in `pyproject.toml:180`: tests live in `hathor_tests (testpaths, :182)`, and `addopts = "-n auto" (:183)` is significant — the `-n auto` enables **pytest-xdist**⁷, which runs tests across multiple CPU cores in parallel, with `auto` choosing the worker count. A large test suite that ran serially would be painfully slow; parallelism keeps the feedback loop short. A `slow` marker (`:185`) lets long tests be selected or skipped (e.g. `make tests-quick` excludes them for a fast pass).

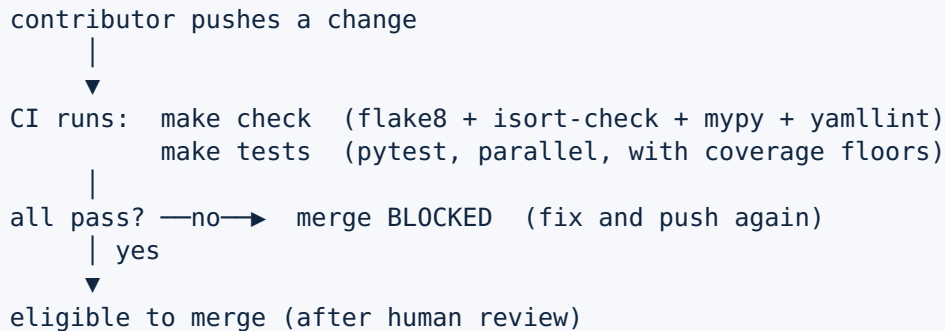
Coverage. Alongside pytest, **coverage**⁸ measures which lines of the code the tests actually execute — reported as a percentage and an annotated HTML report. Coverage doesn't prove the tests are good, only that code was reached; but a low number reliably flags code with no tests at all. Some targets enforce a floor (e.g. `tests-cli` requires a minimum coverage percentage), so a change that drops coverage below the bar fails. The config lives in `setup.cfg` and `.coveragerc_full`.

20.6 The gate: making it all mandatory

Having the tools is not enough; passing them must be required, or standards erode. Two layers make that so:

The aggregate command. `make check (Makefile:96)` runs flake8, isort-check, mypy, and yamllint together — one command that answers "is this change clean?" A contributor runs it before submitting; if it passes locally, it will pass everywhere.

The CI gate. Recall **continuous integration**⁹ from Chapter 5: an automated system runs `make check` and the test targets on every proposed change (every pull request), and the change cannot be merged unless they pass. This is the **gate** — the mechanism that turns "we should type-check and test" into "code that fails type-checking or tests does not enter the codebase, ever." The gate is what makes the whole toolchain effective: it removes the option of skipping the checks. mypy's strictness, flake8's rules, isort's ordering, and pytest's suite are only as good as the enforcement that no change bypasses them.



20.7 Why invest in all this?

The toolchain costs real effort: writing type hints, fixing lint warnings, maintaining tests, waiting for CI. The \$5-style trade-off:

What it costs: developer time and friction. Every change carries the overhead of satisfying four tools, and the checks add minutes to every iteration.

Why it's worth it for a node:

- **Multi-author, long-lived code.** Many people change the codebase over years. Without automated enforcement, each person's assumptions and styles diverge until the code is inconsistent and fragile. The tools enforce a shared standard no one has to police by hand.
- **The cost of a bug is high.** A type error or untested edge case in a node can mean a corrupted ledger or a consensus split — failures far more expensive than the time the toolchain takes. Catching them in CI, before merge, is vastly cheaper than catching them in production.
- **Reviewers focus on what matters.** When formatting and obvious errors are caught automatically, human review can concentrate on logic and design rather than import order and missing type hints. The tools do the mechanical work so people do the thoughtful work.

The honest summary: a quality toolchain trades up-front, per-change effort for a sustained reduction in bugs, inconsistency, and review friction — a trade that grows more favorable the longer the project lives and the more people touch it. For critical, long-lived infrastructure, it is not optional discipline; it is how the code stays trustworthy.

20.8 Bridge — the toolchain across the project

Bridge — the quality toolchain in the project and the stack:

- **mypy enforces Chapter 5.** Static typing becomes a merge requirement, with plugins for Pydantic and zope.interface — **Chapters 5, 16, 18.**
- **The tools are declared by Poetry.** All four live in the dev dependency group, kept out of the production image — **Chapters 13 & 15.**
- **Make is the interface.** `make check` and the `make tests-*` targets wrap the tools — **Chapter 14.**
- **flake8's `__init__` exception** serves the re-export pattern of **Chapter 11.**
- **Verification of behavior** complements this book's own quality discipline (the §7 style rules) — and pytest's suite is what lets Part II's code be changed safely.

Recap

Job	Tool	Config	Make target
type checking	mypy (+ pydantic, zope plugins)	<code>pyproject.toml:101</code>	<code>make mypy (:72)</code>
linting	flake8	<code>setup.cfg:34</code>	<code>make flake8 (:80)</code>
formatting	isort	<code>pyproject.toml:93</code>	<code>make fmt / isort-check</code>
testing	pytest (+ xdist, coverage)	<code>pyproject.toml:180</code>	<code>make tests</code>
the gate	<code>make check</code> + CI	<code>Makefile:96</code>	required to merge
shared rule	line length 119	<code>isort :98 = flake8 :35</code>	—

The quality toolchain is four distinct jobs done by four tools: mypy checks that types line up, flake8 flags suspicious or ugly code, isort keeps imports in a canonical order, and pytest verifies that the code actually works — with pytest-xdist parallelizing the

suite and coverage measuring what it reaches. They share the rules they jointly enforce (line length agreed across formatter and linter) and are made mandatory by a CI gate that blocks any change failing `make check` or the tests. The investment buys consistency and caught bugs in exchange for per-change effort — a trade that pays off increasingly as a long-lived, multi-author node evolves. This completes Track C and Part I: you now hold the programming concepts (Track A), the blockchain domain (Track B), and the full stack the node is built from (Track C). Part II turns to the node itself, walking the codebase in the order it comes alive — beginning at the command line that you can now read top to bottom.

-
1. A linter is a tool that statically analyzes source code to flag stylistic issues and likely bugs (unused imports, undefined names, overly long lines) without running it. `flake8` is the linter here. ↩
 2. A formatter automatically rewrites code into a canonical style (spacing, import order) so formatting is consistent and never argued over. `isort` formats imports; it changes files rather than only reporting. ↩
 3. A test runner discovers and executes a project's automated tests and reports results. `pytest` is the runner here; it finds test functions, runs them, and summarizes passes and failures. ↩
 4. `flake8` is a Python linter that combines several checkers (`pyflakes` for likely bugs, `pycodestyle` for style) to report problems by static inspection. Configurable line length and per-file ignores. ↩
 5. `isort` is a tool that automatically sorts and groups Python import statements into a consistent order (standard library, third-party, first-party). Run in "fix" mode it edits files; in "check" mode it only verifies. ↩
 6. `pytest` is the most widely used Python testing framework. It discovers test functions, runs them, provides rich assertions and fixtures, and reports results. Extensible via plugins like `pytest-xdist` and `pytest-cov`. ↩
 7. `pytest-xdist` is a `pytest` plugin that distributes tests across multiple CPU cores (or machines), running them in parallel to shorten total test time. Enabled here with `-n auto`. ↩
 8. Code coverage measures which lines (or branches) of the code are executed by the test suite, reported as a percentage. High coverage doesn't guarantee good tests, but low coverage reliably reveals untested code. ↩
 9. Continuous integration (CI) is an automated system that runs checks (linters, type checker, tests) on every proposed change and blocks merging unless they pass — the gate that makes the toolchain mandatory. ↩

Chapter 21 — The Command-Line Surface

What you'll learn in this chapter

- How typing `hathor-cli` in a terminal actually reaches Python code — the **console-script entry point** that the packaging system installs.
- How a single program with ~40 sub-tools routes the first word you type (`run_node`, `gen_peer_id`, `shell`, ...) to the right module, using a **dispatch table** instead of a chain of `if/elif`.
- The role of the two logging hooks (`PRE_SETUP_LOGGING`, `LOGGING_CAPTURE_STDOUT`) that let each subcommand opt into or out of the shared logging setup.
- The full `run_node` boot path in code: parse arguments → pick the network/settings → assemble the node with a builder → start it → hand control to the reactor.
- Where this chapter **hands off**: to settings (Ch 22), the reactor (Ch 23), and the composition root (Ch 24).

This is the first chapter of Part II, and so the first time the book opens real `hathor-core` source. We start here for one reason: this is the front door. Every other subsystem in the node is reached, ultimately, because someone typed a command and a dispatcher routed it. Chapter 0 told the story of "the life of a node" in prose; this chapter shows you the whole of Act I in code — the command is dispatched, arguments and logging are set up, settings are chosen, the node is assembled, started, and the reactor is run — and points you at the chapters that go deep on each later step.

We will lean on three ideas you have already met in Part I: the **dispatch table** (Ch 4), **CLI-as-configuration** with `argparse/configargparse` (Ch 19), and the **Twisted reactor** (Ch 16). This chapter does not re-teach them; it re-establishes each in context and points back.

21.1 Localization — where `hathor_cli/` sits

`hathor-core` is two top-level Python packages: `hathor/`, which is the node itself, and `hathor_cli/`, which is the thin command-line surface that launches and operates it. `hathor_cli/` is the outermost layer — it imports from `hathor/`, never the other way around.

```

hathor-core/
├── pyproject.toml      ← declares the `hathor-cli` console script
├── hathor_cli/        ← THE COMMAND-LINE SURFACE ◀ YOU ARE HERE
│   ├── main.py        ← CliManager: the dispatch table + entry point main()
│   ├── run_node.py    ← RunNode: the boot path for an actual node
│   ├── run_node_args.py ← RunNodeArgs: the typed (Pydantic) model of run_node's flags
│   ├── util.py        ← create_parser(), setup_logging(), process_logging_*()
│   ├── quick_test.py  ← a RunNode subclass that exits after one tx
│   ├── side_dag.py    shell.py  mining.py  peer_id.py  wallet.py  ... (~40 tools)
│   └── ...
└── hathor/           ← the node (everything this CLI launches)
    ├── manager.py     ← HathorManager (Ch 29)
    ├── builder/       ← composition root (Ch 24)
    ├── reactor/       ← reactor abstraction (Ch 23)
    └── conf/          ← settings profiles (Ch 22)

```

The dependency arrow points one way: `hathor_cli/` reaches into `hathor/` (you will see it import `hathor.reactor`, `hathor.builder`, `hathor.conf` in §21.5), and `hathor/` never reaches back. That is deliberate. The node's logic should not know or care whether it was started from a terminal, a test harness, or a simulator. Keeping the entry point as a separate, dependent layer is what makes the same node assemblable three different ways.

Context — the front door. `hathor_cli/` is where a human (an operator, a developer, a miner) meets the node. It owns nothing about the ledger, the network, or consensus. Its single job is to turn a line of text typed in a terminal into the right call into `hathor/`. Because it is the entry point, it is also where two cross-cutting concerns are set up first, before anything else runs: command-line **arguments** and **logging**. Everything downstream assumes those are already in place.

21.2 What it does and why it exists

A full node is not the only program a Hathor operator runs. They also need to generate a peer identity, create a wallet, run a miner, export the database, dump nano-contract state, open a debugging shell, and a couple dozen other small tasks. You could ship each of those as its own separate executable — `hathor-run-node`, `hathor-gen-peer-id`, `hathor-shell`, and so on — but that is forty commands for a user to remember and forty entries to install.

`hathor_cli/` takes the other approach, the same one `git` takes: **one program, many subcommands**. You type the program name, then the subcommand as the first argument:


```
hathor-cli run_node --testnet --data ./data
hathor-cli gen_peer_id
hathor-cli shell
```

The program reads that first word, finds the code responsible for it, and runs it. This is the pattern Chapter 0 §0.3 called "the command is dispatched" — Act I, step 1.

How `hathor-cli` reaches Python — the console-script entry point

When you `pip install` (or `poetry install`) `hathor-core`, how does the shell command `hathor-cli` come to exist? It is declared in `pyproject.toml`¹ (`pyproject.toml:37-38`):

```
[tool.poetry.scripts]
hathor-cli = 'hathor_cli.main:main'
```

This is a **console-script entry point**². At install time, the packaging tool reads that line and generates a tiny executable named `hathor-cli` on your `PATH`. That generated stub does essentially one thing: import the object named on the right — the function `main` in the module `hathor_cli.main` — and call it. The format is always `package.module:object`. So typing `hathor-cli` in a terminal is, after one layer of generated glue, exactly the same as running `from hathor_cli.main import main; main()`.

Note that the packages list (`pyproject.toml:32-35`) ships both `hathor` and `hathor_cli` as installable packages. You will sometimes see the codebase import a CLI module as `hathor.cli.*` and sometimes as `hathor_cli.*`; both spellings appear in this very file's source. The canonical entry point, the one packaged into the executable, is `hathor_cli.main:main`. When you `grep` for CLI code, search for both spellings to be safe.

21.3 The concepts it rests on

This chapter sits on three Part I ideas. Each gets a one-paragraph re-establishment here and a pointer to its full treatment — do not skip these; meeting an idea again in context is how it sticks.

Recap — the dispatch table (full treatment in Ch. 4). A dispatch table is a dictionary that maps a key (here, a command name like `"run_node"`) to the thing that handles it (here, a Python module). To "dispatch" is to look up the handler by key and invoke it. The alternative is an `if cmd == "run_node": ... elif cmd == "shell": ...` ladder. The dictionary wins on three counts: adding a command is one line, not a new branch; the set of commands becomes data you can iterate (to print a help screen); and lookup is $O(1)$ regardless of how many commands there are. The cost is one layer of indirection — you can't read the control flow top-to-bottom. For ~40 commands, that trade is clearly worth it. → Ch 4.

Recap — CLI as configuration: argparse and configargparse (full treatment in Ch. 19). `argparse` is Python's standard library for declaring command-line flags (`--testnet`, `--data PATH`) and parsing them into a structured object. `configargparse` is a drop-in superset that adds one capability: a flag can be supplied either on the command line or from an environment variable or from a config file. A node is run by operators and by automated deployments, so being able to set the same option three ways matters. `hathor_cli` builds every parser through one helper, `create_parser()` in `hathor_cli/util.py:30`, which returns a `configargparse.ArgumentParser` with the env-var prefix `hathor_`. → Ch 19.

Recap — the Twisted reactor (full treatment in Ch. 16, recap in Ch. 23). Twisted is the asynchronous framework `hathor-core` is built on. Its centerpiece is the **reactor**: a single event loop that, once `running`, waits for events (a peer connects, a timer fires, data arrives) and calls the right handler for each. The crucial fact for this chapter: the reactor is created early but only runs at the very end of boot. `reactor.run()` is a blocking call — it does not return until the node shuts down. So the last line of the `run_node` boot path is, in effect, "hand the program over to Twisted forever." → Ch 16 & 23.

There is also one component this chapter points at but does not open: the **composition root**, Hathor's two builders (`Builder` and `CliBuilder`). The `run_node` path uses `CliBuilder` to assemble the fully-wired node before starting it. That assembly is intricate enough to get its own chapter. → Ch 24.

21.4 The code, walked: dispatch

A toy dispatcher first

Before the real `CliManager`, here is the whole idea in five lines of plain Python:

```
def run_node(): print("booting a node...")
def gen_peer_id(): print("new peer id...")

COMMANDS = {"run_node": run_node, "gen_peer_id": gen_peer_id} # the table

def main(argv):
    cmd = argv[1] # the first word after the program name
    COMMANDS[cmd]() # look it up, call it – that's dispatch
```

`COMMANDS` is the dispatch table. `main` reads the first argument, looks up the handler, and calls it. Adding a third command is one new dictionary entry. The real Hathor code is this idea, hardened: the values are modules (not functions), there is grouping for the help screen, there is error handling for unknown commands, and there is a logging-setup step wrapped around the call.

The real dispatch table: `CliManager`

The dispatcher is the class `CliManager` in `hathor_cli/main.py:26`. Its constructor builds the table. First it imports every subcommand module (`hathor_cli/main.py:34-68`), then registers each one with `add_cmd` (starting at `hathor_cli/main.py:70`):

```
self.add_cmd('hathor', 'run_node', run_node, 'Run a node')
self.add_cmd('hathor', 'gen_peer_id', peer_id, 'Generate a new random peer-id')
self.add_cmd('mining', 'run_miner', mining, 'Run a mining process (running node required)')
self.add_cmd('dev', 'shell', shell, 'Run a Python shell')
self.add_cmd('multisig', 'gen_multisig_address', multisig_address, 'Generate a new multisig address')
# ... ~40 of these, grouped: hathor / mining / side-dag / docs / multisig /
# tests / wallet / oracle / events / features / dev
```

`add_cmd` (`hathor_cli/main.py:110`) does the bookkeeping behind the table:

```
def add_cmd(self, group, cmd, module, short_description=None):
    self.command_list[cmd] = module # the dispatch table itself
    self.groups[group].append(cmd) # grouping, for the help screen
    if short_description:
        self.cmd_description[cmd] = short_description
    self.longest_cmd = max(self.longest_cmd, len(cmd)) # for aligned help output
```

Two things to notice. First, `command_list` (`hathor_cli/main.py:29`) is the dispatch table: command-name → module. Second, the value is a whole module (`ModuleType`), not a function. The convention across `hathor_cli` is that every subcommand module exposes a top-level `main()` function; dispatching a command means "call `module.main()`." Storing the module rather than `module.main` lets the dispatcher also read other top-level attributes off it — which is exactly how the logging hooks work (below).

The grouping (`self.groups`, a `defaultdict(list)` at `hathor_cli/main.py:31`) exists only to produce a readable help screen. `help()` (`hathor_cli/main.py:117`) sorts the groups and prints each command under its group heading, padded to `longest_cmd` so the

descriptions line up. This is the payoff of making the command set data: the help screen is generated by iterating the same structure that drives dispatch, so the two can never drift out of sync.

Dispatch and the logging hooks: `execute_from_command_line`

The actual routing happens in `execute_from_command_line` (`hathor_cli/main.py:134`).

Walk it top to bottom:

```
def execute_from_command_line(self):
    from hathor_cli.util import process_logging_options, process_logging_output, setup_logging

    if len(sys.argv) < 2:          # no subcommand given → print help, exit 0
        self.help()
        return 0

    cmd = sys.argv.pop(1)          # take the first word; REMOVE it from argv
    if cmd == 'help':
        self.help()
        return 0

    if cmd not in self.command_list: # unknown command → friendly error, exit -1
        print('Unknown command: "{}"'.format(cmd))
        print('Type "{} help" for usage.'.format(self.basename))
        return -1

    sys.argv[0] = '{} {}'.format(sys.argv[0], cmd) # so the subcommand sees a sane progname
    module = self.command_list[cmd]               # ← the dispatch lookup
```

The detail that trips people up is `sys.argv.pop(1)` (`hathor_cli/main.py:141`). The dispatcher consumes the subcommand word and removes it from `sys.argv`. After that line, `sys.argv` looks as if the subcommand module had been invoked directly — its own argument parser (which knows nothing about subcommands) sees only its flags. This is the standard `git`-style trick: the outer layer eats the first word, the inner layer parses the rest.

Now the two logging hooks. Each subcommand module may declare two optional top-level attributes; the dispatcher reads them off the module with `getattr` and a default:

```
if '--help' in sys.argv:
    capture_stdout = False
else:
    capture_stdout = getattr(module, 'LOGGING_CAPTURE_STDOUT', False) # main.py:157
# ... (a --pdb debugger hook omitted) ...
pre_setup_logging = getattr(module, 'PRE_SETUP_LOGGING', True)        # main.py:167
if pre_setup_logging:
    output = process_logging_output(sys.argv)
    options = process_logging_options(sys.argv)
    setup_logging(logging_output=output, logging_options=options, capture_stdout=capture_stdout)
    module.main()
else:
    module.main(capture_stdout=capture_stdout)
```

This is the wrapper pattern from Chapter 4 applied to logging: the dispatcher wraps each subcommand call with logging setup, and the subcommand customizes that wrapper by setting two flags.

- **PRE_SETUP_LOGGING** (default `True`) decides who sets up structured logging³. If `True`, the dispatcher does it — it calls `setup_logging` before calling the command. If a module sets it to `False`, the dispatcher does **not** set up logging itself; instead it passes `capture_stdout` into the command's own `main()`, trusting the command to configure logging on its own schedule. A command that needs to read some of its own arguments before configuring logging would set this `False`.
- **LOGGING_CAPTURE_STDOUT** (default `False`) decides whether plain `print()` output should be redirected into the structured logging stream. It is forced `False` whenever `--help` is present (`hathor_cli/main.py:154`), because help text must go straight to the terminal, un-captured.

Both are read with `getattr(module, NAME, default)`, so a subcommand module that declares neither attribute gets the defaults (`PRE_SETUP_LOGGING=True`, `LOGGING_CAPTURE_STDOUT=False`): logging is set up for it, stdout is not captured. The opt-in is per-command, expressed as data on the module, not as special cases inside the dispatcher. The dispatcher stays generic.

`process_logging_output` (`hathor_cli/util.py:138`), `process_logging_options` (`hathor_cli/util.py:159`), and `setup_logging` (`hathor_cli/util.py:172`) all live in `hathor_cli/util.py`. The first two peel the logging flags (`--json-logs`, `--disable-logs`, `--debug`) out of `sys.argv` before the main parser runs and return small typed records (`LoggingOutput`, `LoggingOptions`); `setup_logging` then configures structlog from them — including an observer that funnels Twisted's own log messages into the same stream (`hathor_cli/util.py:355`). The canonical treatment of structured logging is Chapter 17.

The entry point: `main()`

The function the console script actually calls is `main` at the bottom of the file (`hathor_cli/main.py:177`):

```
def main():
    try:
        sys.exit(CliManager().execute_from_command_line())
    except KeyboardInterrupt:
        logger.warn('Aborting and exiting...')
        sys.exit(1)
    except Exception:
        logger.exception('Uncaught exception:')
        sys.exit(2)
```

Three things happen here. It constructs a fresh `CliManager` (building the dispatch table), runs `execute_from_command_line`, and passes that return value to `sys.exit` — so the integer a subcommand returns becomes the process's **exit code**⁴. The `try/except` is the program's last line of defence: a `Ctrl-C` becomes a clean "Aborting..." and exit code 1; any other uncaught exception is logged with a full traceback (`logger.exception`) and exits with code 2. This is why an operator who kills a node sees a tidy message rather than a Python traceback, and why a crash always produces a logged stack trace and a non-zero exit code a deployment system can detect.

21.5 The `run_node` boot path

Dispatch ends with `module.main()`. For the `run_node` subcommand, that module is `hathor_cli/run_node.py`, whose `main()` is two lines (`hathor_cli/run_node.py:595`):

```
def main():
    RunNode().run()
```

All the work is in the class `RunNode` (`hathor_cli/run_node.py:49`). Crucially, almost everything happens in its **constructor** — `RunNode()` boots the entire node — and `run()` only starts the reactor at the very end. We walk the constructor (`__init__`, `hathor_cli/run_node.py:501`) in order.

Step 1 — parse arguments

The constructor builds the parser and parses `argv` (`hathor_cli/run_node.py:514-515`):

```
self.parser = self.create_parser()
raw_args = self.parse_args(argv)
self._args = self._parse_args_obj(vars(raw_args))
```

`create_parser` (a classmethod, `hathor_cli/run_node.py:64`) starts from `create_parser()` in `util.py` (the configargparse parser) and adds every `run_node` flag — `--testnet`, `--data`, `--listen`, `--stratum`, `--wallet`, and dozens more (`hathor_cli/run_node.py:75-177`). Network flags are grouped as mutually exclusive (`hathor_cli/run_node.py:80`) so you cannot pass `--testnet` and `--localnet` together.

`_parse_args_obj` (`hathor_cli/run_node.py:587`) then validates the raw dict into a typed model:

```
def _parse_args_obj(self, args):
    from hathor_cli.run_node_args import RunNodeArgs
    return RunNodeArgs.model_validate(args)
```

Recap — RunNodeArgs , the typed flag model (Pydantic full treatment in Ch. 18, settings in Ch. 22). `RunNodeArgs` (`hathor_cli/run_node_args.py:20`) is a Pydantic model: a class whose fields are typed (`hostname: str | None` , `auto_hostname: bool` , ...) and validated at construction. It is configured `extra='forbid'` , `frozen=True` (`run_node_args.py:21`), meaning an unexpected key is an error and the object is immutable once built. Parsing the argv dict through this model converts a loose bag of strings into a checked, read-only configuration object the rest of boot can trust. The class doc-comment in `create_parser` even reminds maintainers that any new CLI flag must be added here too (`run_node.py:67-68`). → Ch 18, Ch 22.

Step 2 — select the network and load settings

Still in the constructor, the parsed args choose which network the node joins. The mechanism is an environment variable that points at a settings YAML file (`hathor_cli/run_node.py:519-532`):

```
if self._args.config_yaml:
    os.environ['HATHOR_CONFIG_YAML'] = self._args.config_yaml
elif self._args.testnet:
    os.environ['HATHOR_CONFIG_YAML'] = TESTNET_INDIA_SETTINGS_FILEPATH
elif self._args.nano_testnet:
    os.environ['HATHOR_CONFIG_YAML'] = NANO_TESTNET_SETTINGS_FILEPATH
elif self._args.localnet:
    os.environ['HATHOR_CONFIG_YAML'] = LOCALNET_SETTINGS_FILEPATH
```

This is the single most consequential choice at boot: get the network wrong and the node speaks a different dialect than its peers and will never sync. (Note that some retired networks — `testnet-hotel` , `testnet-golf` — are now hard rejections, `run_node.py:523-528` .) Immediately after, the constructor forces the settings to load and validates them early (`hathor_cli/run_node.py:534-540`):

```
try:
    get_global_settings()
except (TypeError, ValidationError) as e:
    raise PreInitializationError('An error was found while trying to initialize HathorSettings...')
```

Loading settings here, before any heavy assembly, means a malformed profile fails fast with a clear error rather than halfway through building the node. The settings machinery — how `HATHOR_CONFIG_YAML` becomes a frozen `HathorSettings` object — is Chapter 22.

Step 3 — assemble and start: `prepare()`

The constructor's last real act is to call `self.prepare()` (`hathor_cli/run_node.py:542`). `prepare()` (`hathor_cli/run_node.py:180`) is where the node comes into being. In order:

Process hygiene and safety gates. It sets the process title (`setproctitle` , `run_node.py:185`), the recursion limit (`run_node.py:187-190`), and checks the open-file-descriptor limit is at least 256, exiting if not (`run_node.py:192-195`). Then three guards run: `validate_args` , `check_unsafe_arguments` , and `check_python_version` (`run_node.py:197-199`). The unsafe-arguments guard (`run_node.py:387`) is worth knowing: certain flags — listed in `UNSAFE_ARGUMENTS` (`run_node.py:50-60`), e.g. `--test-mode-tx-weight` , `--x-asyncio-reactor` — are refused unless `--unsafe-mode` is explicitly given, so a dangerous testing flag cannot be left on against mainnet by accident.

Create the reactor (but do not run it). (`hathor_cli/run_node.py:201-203`):

```
from hathor.reactor import initialize_global_reactor
reactor = initialize_global_reactor(use_asyncio_reactor=self._args.x_asyncio_reactor)
self.reactor = reactor
```

The reactor is created now and stored, but its event loop is not started until step 4. By default this is the Twisted reactor; `--x-asyncio-reactor` swaps in an asyncio-backed one (the abstraction that makes that swap possible is Chapter 23).

Assemble the node with `CliBuilder`. (`hathor_cli/run_node.py:205-213`):

```
from hathor_cli.builder import CliBuilder
builder = CliBuilder(self._args)
try:
    self.manager = builder.create_manager(reactor)
except BuilderError as err:
    self.log.error(str(err))
    sys.exit(2)
```

This is Chapter 0 §0.3 Act I step 5. `CliBuilder` constructs, in dependency order, every component the node needs — storage, indexes, wallet, pub-sub and the event manager, verification, consensus, the nano-contract runtime, feature activation — and wires them into a single `HathorManager` (`hathor_cli/run_node.py:210`). A failure during assembly is caught and turned into a clean log line plus exit code 2, rather than a raw traceback. The builder is Chapter 24; the manager it produces is Chapter 29.

Wire up listeners and resources. With the manager built, `prepare()` attaches the optional network-facing servers to the reactor: the Stratum mining server (`run_node.py:218-228`) and, via a `ResourcesBuilder` , the HTTP status/API server (`run_node.py:233-248`). Each is bound with `reactor.listenTCP(...)` — registered with the reactor, but nothing flows until the loop runs.

Start the manager. (`hathor_cli/run_node.py:250`) calls `self.start_manager()` , which (`run_node.py:292-294`) starts Sentry if configured and then calls `self.manager.start()` . This is Chapter 0 §0.3 Act I step 6: the crash check, the genesis load and in-memory rebuild, and the bringing-online of subsystems (WebSocket, metrics, the P2P connections manager, wallet). The mechanics live in Chapter 29.

After `prepare()` returns, the constructor registers OS signal handlers (`run_node.py:543`; `SIGUSR1` reloads peers, `SIGUSR2` opens a `sysctl` pipe) and, if `--sysctl` was given, starts the runtime-control socket (`run_node.py:544-545`). The node is now fully built and started — but still frozen, because the reactor has not run.

Step 4 — run the reactor

Back in `main()`, after the constructor returns, `RunNode.run()` is called (`hathor_cli/run_node.py:591`):

```
def run(self):
    self.reactor.run()
```

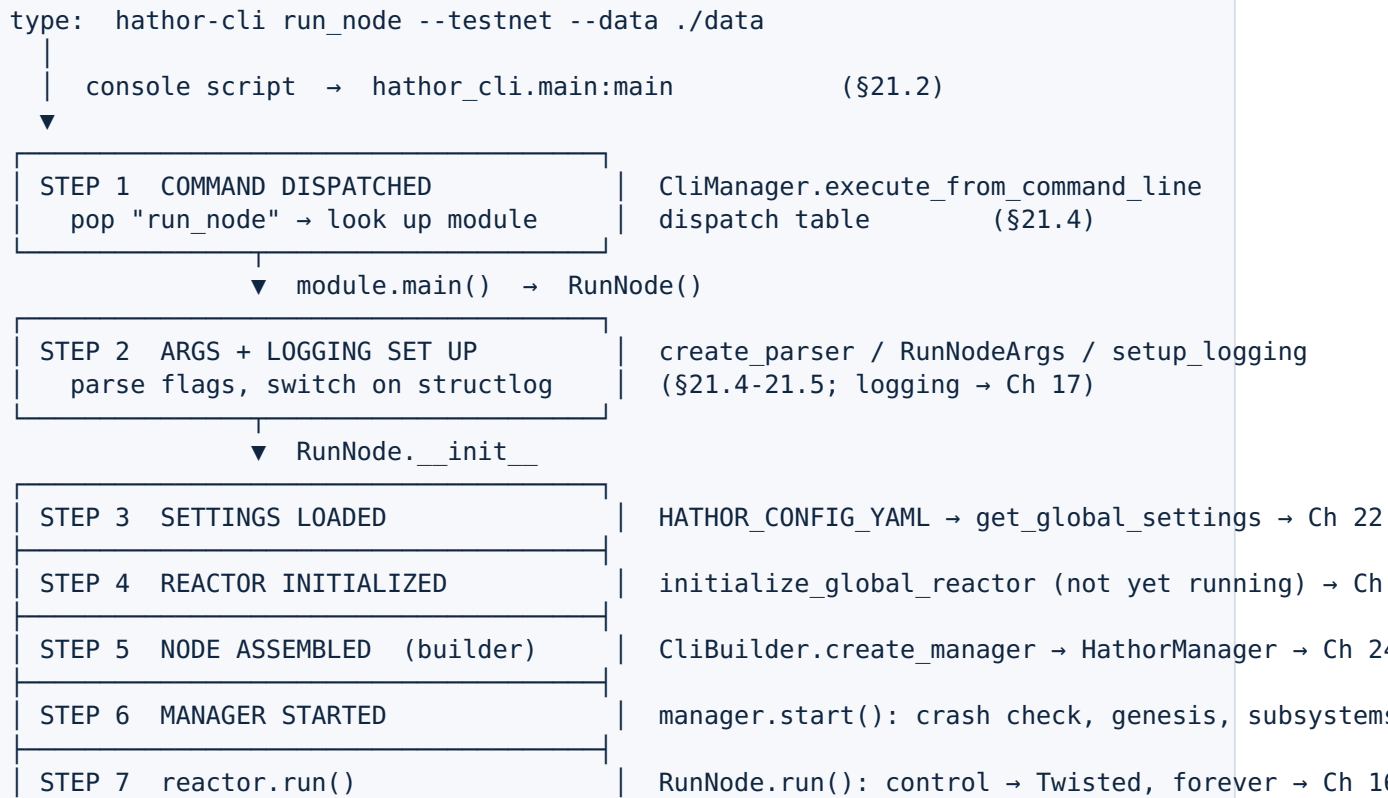
This is Chapter 0 §0.3 Act I step 7 — "the reactor is put in gear." `reactor.run()` blocks; control passes to Twisted's event loop and does not return until shutdown. From here the node is reactive: it sleeps until an event arrives (a peer connects, a vertex is received, a timer fires) and dispatches the matching handler. → Ch 16 & 23.

A worked illustration of reuse: `quick_test`

`quick_test` (`hathor_cli/quick_test.py`) is a subclass of `RunNode` that boots a node exactly as above but stops after it receives a single transaction — a smoke test. That it is a subclass (inheritance, Ch 1) rather than a copy is the reason a one-off test command stays in lock-step with the real boot path: change `RunNode`, and `quick_test` inherits the change. This is the same payoff as the dispatch table — write the boot sequence once, vary only what differs.

21.6 How it plugs into the lifecycle

Put against the life-of-a-node story (Chapter 0 §0.3, Act I), this chapter owns the whole startup sequence — but implements only its first two steps, delegating the rest:



Steps 1-2 are `hathor_cli`'s own job, done in code you have now read in full (`main.py`) and walked line by line (`run_node.py`). Steps 3 onward are invoked from `run_node.py` but implemented elsewhere, and each is a chapter of its own. The CLI's contribution is to be the reliable, generic front door: route the command, parse and validate the configuration, choose the network, turn on logging, then drive a builder and start the manager — and finally get out of the way by handing the process to the reactor.

Recap

Phase	File (<code>hathor_cli/...</code>)	Responsibility
Reach Python	<code>pyproject.toml:37-38</code>	<code>hathor-cli = 'hathor_cli.main:main' — console script → <code>main()</code></code>
Build the table	<code>main.py:26</code> <code>CliManager.__init__</code> , <code>add_cmd (:110)</code>	register ~40 subcommands → modules, grouped
Dispatch	<code>main.py:134</code> <code>execute_from_command_line</code>	pop first word, look up module, call <code>module.main()</code>

Phase	File (<code>hathor cli/...</code>)	Responsibility
Logging hooks	<code>main.py:157,167</code> <code>LOGGING_CAPTURE_STDOUT</code> , <code>PRE_SETUP_LOGGING</code>	per-command opt-in to shared logging setup
Entry + safety	<code>main.py:177</code> <code>main()</code>	construct manager, run, <code>sys.exit</code> , catch <code>Ctrl-C</code> / crashes
Boot a node	<code>run_node.py:49</code> <code>RunNode</code> , :595 <code>main()</code>	the whole <code>run_node</code> sequence
Parse + type args	<code>run_node.py:514-517</code> , <code>run_node_args.py:20</code>	<code>configargparse</code> parser → frozen Pydantic <code>RunNodeArgs</code>
Pick network	<code>run_node.py:519-540</code>	set <code>HATHOR_CONFIG_YAML</code> , load + validate settings early
Assemble + start	<code>run_node.py:180</code> <code>prepare()</code> (<code>reactor</code> :202 , <code>CliBuilder</code> :208 , <code>start_manager</code> :250)	build → wire → start the manager
Run reactor	<code>run_node.py:591</code> <code>run()</code> → <code>reactor.run()</code>	control → Twisted, until shutdown
Parser + logging	<code>util.py:30</code> <code>create_parser</code> , :172 <code>setup_logging</code>	<code>configargparse</code> parser; structlog configuration

The command-line surface is small and deliberately dumb: it holds no ledger logic, only the routing and configuration plumbing that every subcommand needs before its real work begins. The two ideas to carry forward are the **dispatch table** (the command set is data, so adding a command and generating the help screen are both one-liners) and the **wrapped call** (the dispatcher sets up logging around each subcommand, customized by two flags the module declares). For `run_node` , the chapter's parting picture is the seven-step boot path — and the next two chapters fill in its first hand-offs: **Chapter 22** explains how step 3 turns `HATHOR_CONFIG_YAML` into a frozen settings object, and **Chapter 24** explains how step 5's `CliBuilder` assembles the wired node that step 6 then starts.

-
1. `pyproject.toml` is the standard configuration file for a modern Python project — dependencies, build settings, and tool config in one place. Hathor uses Poetry to read it. Full treatment in Chapter 13. ↩

2. A console-script entry point is a line in packaging metadata of the form `name = "package.module:function"`. At install time the packaging tool generates an executable called `name` on your `PATH` that imports and calls that function. It is how a `pip install` makes a shell command appear. ↵
3. Structured logging records each log entry as machine-readable key-value data (e.g. `event="node started" network="testnet"`) rather than a freeform sentence, so logs can be searched and analyzed programmatically. Hathor uses the `structlog` library; full treatment in Chapter 17. ↵
4. An exit code (or status code) is the small integer a process returns to the shell when it ends. `0` means success; any non-zero value signals an error. Shells and deployment tools branch on it (`if program; then ...`). Hathor uses `0` for success, `1` for a `Ctrl-C` abort, `2` for an uncaught crash. ↵

Chapter 22 — Settings & Configuration

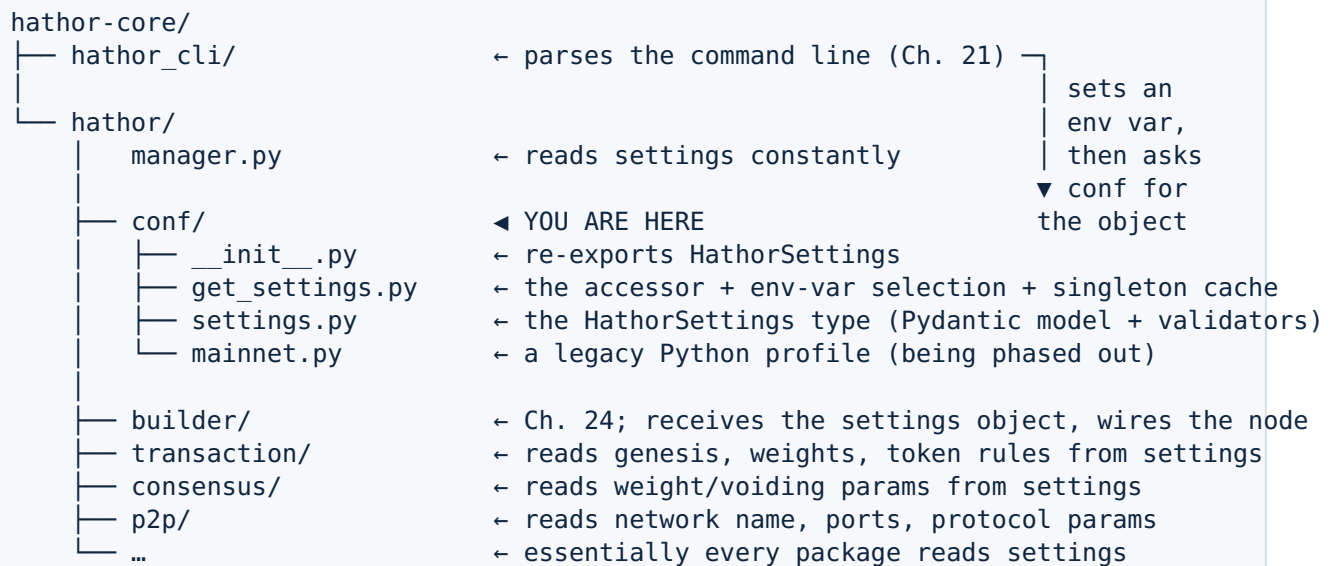
What you'll learn in this chapter

- Why a full node needs hundreds of agreed-upon constants, and why getting even one of them wrong puts the node on a different network — not a broken one, a different one.
- What a **network profile** is (`mainnet` , `testnet` , a private net), and why Hathor stores profiles as **YAML** files rather than hard-coded Python.
- How `get_global_settings()` selects a profile from an environment variable, validates it into a `HathorSettings` object with **Pydantic**, and caches it as a process-wide **singleton**.
- Why the settings object is built **once, validated at load, and treated as read-only** — the fail-fast vs. crash-deep trade-off.
- Where settings sits in the boot sequence: right after the command line is parsed (Ch. 21), feeding the builder (Ch. 24) and then every component below it.

In Chapter 21 the command line handed control to `run_node` . The very next thing the node must establish, before it builds a single component, is which network it is joining and what the rules of that network are. That is this chapter. The package is small — four files in `hathor/conf/` — but it is load-bearing: nearly every other package reads from the object this one produces.

22.1 Localization — where `hathor/conf/` sits

`hathor/conf/` is **infrastructure** (the bottom band of the module map in Chapter 0, §0.4). It has almost no dependencies of its own and is depended on by almost everything else. It is one of the first things touched at boot and is then read for the lifetime of the process.



The YAML profiles themselves (`mainnet.yml` , `testnet.yml`) do **not** live in this folder. They ship inside an installed dependency, `hathorlib` , and `hathor/conf/` points at them by default. We will come back to why in §22.5; for now, hold the shape: `hathor/conf/` is the machinery that loads a profile, and a profile is a file of constants.

Where this fits. `hathor/conf/` answers one question for the rest of the node: "what are the rules of the network I am on?" It is read at boot by the builder and then continuously at runtime by verification, consensus, p2p, mining, and more. Because every component reads the same object, the package is also the node's single source of truth — there is no second place a constant can hide.

22.2 What it does and why it exists

A full node does not get to invent the rules. To be on the same network as everybody else, it must agree, to the bit, on a long list of constants:

- the **genesis**¹ — the exact first block and initial transactions, byte-for-byte;
- the **network name** (`mainnet` , `testnet-...`) used in the peer handshake;
- monetary rules — how many tokens genesis mints, the decimal places, the block reward schedule, the halving interval;
- timing and difficulty parameters — target seconds between blocks, the weight² math;
- the **feature-activation**³ schedule — which protocol upgrades switch on, and when;
- structural limits — maximum transaction size, maximum number of inputs/outputs, script size caps;

- the network ports and bootstrap peers used to find the rest of the network.

There are hundreds of these. Here is the uncomfortable property that makes configuration a correctness concern and not a convenience: **a node with the wrong constants is not a broken node — it is a node on a different network.** If your node believes genesis is one set of bytes and the rest of the network believes another, the two will never share a common root of history (§0.3), so they can never agree on anything. Your node will reject every block the network sends and the network will reject every block yours sends. Nothing crashes. It just silently fails to be part of the network. The same is true, more subtly, of a single mismatched weight constant or a wrong reward schedule: your node will compute different validity and quietly fork off on its own.

So the job of `hathor/conf/` is narrow and strict:

1. **Decide which profile to load** — `mainnet`, `testnet`, or a private network — from the operator's choice.
2. **Load and validate** that profile into a single typed object, failing immediately if anything is malformed.
3. **Hand that one object to everyone**, and make sure everyone gets the same one.

The rest of this chapter is those three steps in code.

Why one read-only object, and not scattered constants

Imagine the naïve alternative: sprinkle the constants through the code as module-level globals — `MAX_TX_SIZE = 100_000` here, `DECIMAL_PLACES = 2` there. Three problems appear immediately.

You cannot switch networks. The constants are baked in at import time. Running the same binary against `testnet` would mean a different build. Hathor needs one binary that can join any network depending on a flag.

You cannot validate the set as a whole. Some constants constrain each other — for example, a proof-of-authority⁴ network must not define block rewards (rewards are a proof-of-work concept). Scattered globals give you nowhere to check that rule. A single object validated at load gives you exactly one place.

You lose a single source of truth. With globals, a constant can be shadowed, re-assigned, or read inconsistently from two places. Bundling them into one object that is built once and never mutated means every component is reading the identical value. This is the **single source of truth** principle, and for network constants it is not a nicety — it is the difference between being on the network and not.

So Hathor uses one object, the `HathorSettings`, built once and treated as read-only. The next two sections give you the two background ideas the implementation rests on — the singleton and validated models — then §22.4 walks the code.

22.3 The concepts it rests on

This package is small precisely because it leans on two patterns you have already met. Here they are, re-established in the configuration context.

Recap — singleton / single global accessor (full treatment in Ch. 3). A singleton is a value of which there is exactly one instance for the whole program, reached through a shared accessor rather than constructed by each caller. Configuration is the textbook case: every component wants "the settings," and they must all get the same settings, so you build the object once, stash it in a module-level variable, and have every caller go through one function — `get_global_settings()`. Chapter 3 flagged the singleton as a global-state smell to use sparingly; configuration is one of the few cases where it earns its place, because the alternative (threading a settings argument through every constructor in the codebase) is worse, and because the value is read-only after load.

Recap — Pydantic validation models (full treatment in Ch. 18). Pydantic is a library for declaring a data shape as a Python class with type-annotated fields, and having it **validate and coerce** raw input (a dict parsed from YAML, say) into a typed object — rejecting anything that does not fit. You write `class HathorSettings(...)` with fields like `DECIMAL_PLACES: int`, and Pydantic guarantees that what you get back has an `int` there or raises an error trying. It also runs custom validators — small methods that enforce cross-field rules. Chapter 18 is the full treatment; here it is the mechanism that turns a YAML file into a trustworthy `HathorSettings`.

Read-only config, and why. A read-only⁵ object is one nothing mutates after it is created. Settings are loaded once at boot and then read — never written — for the rest of the node's life. Treating them as read-only buys two things. First, safety: no stray line, anywhere in tens of thousands of lines, should quietly mutate a network constant mid-run and fork the node. This is the same invariant-protection idea from Chapter 1 — bundle the data and forbid casual mutation — applied to the most safety-critical data the node holds. Second, shareability: because nobody changes it, the one shared instance can be handed to every component without fear that one component's edit leaks into another's view. (The model also forbids unknown fields at load, which is the validation half of that safety story; we see exactly how in §22.4.)

Recap — genesis (from the Ch. 0 footnote; full treatment in Ch. 25 & 32). The genesis is the hard-coded starting point of the ledger: the first block and the initial transactions, identical for every node on a network by definition. It is the shared root every node measures history from. It lives in the settings profile because it is a per-network constant — `mainnet` and `testnet` have different genesis data, and that difference is exactly what makes them different networks. We meet genesis as code in the vertex model (Ch. 25) and see how consensus anchors to it (Ch. 32); here, it is one (very consequential) entry in the profile.

22.4 The code, walked

The shape of the package

Three files do the work, and one re-exports.

<code>hathor/conf/__init__.py</code>	→ re-exports <code>HathorSettings</code> so callers can write <code>`from hathor.conf import HathorSettings`</code>
<code>hathor/conf/get_settings.py</code>	→ the accessor: choose a profile, load it, cache it
<code>hathor/conf/settings.py</code>	→ the <code>HathorSettings</code> type: a Pydantic model + validators
<code>hathor/conf/mainnet.py</code>	→ a legacy Python-module profile (deprecated path)

`__init__.py` is one line of substance — it re-exports the accessor so the rest of the codebase has a short, stable import:

```
# hathor/conf/__init__.py:15
from hathor.conf.get_settings import HathorSettings
```

A toy first: a read-only config every function reads

Before the real code, here is the whole idea in a handful of lines of plain Python — a config built once, frozen against mutation, and reached through one accessor:

```
from dataclasses import dataclass

@dataclass(frozen=True)                                # frozen=True → assigning to a field raises
class Config:
    network: str
    max_tx_size: int

_config = None                                         # the single cached instance (the singleton)

def get_config() -> Config:
    global _config
    if _config is None:                                # build it the first time...
        _config = Config(network="mainnet", max_tx_size=100_000)
    return _config                                    # ...and hand back the same object every time after
```

Every function anywhere calls `get_config()` and gets the identical object. Try `get_config().max_tx_size = 5` and Python raises, because the dataclass is `frozen`. Hold this picture — Hathor's version is the same skeleton with two upgrades: the values come from a YAML file chosen at runtime, and validation is done by Pydantic instead of a bare dataclass.

Selecting the profile from the environment

The accessor lives in `get_settings.py`. The public entry point is `get_global_settings()`, and it does nothing but delegate:

```
# hathor/conf/get_settings.py:39
def get_global_settings() -> 'Settings':
    return HathorSettings()
```

`HathorSettings()` here is **a function, not the class** — a naming choice worth pausing on, because it is confusing the first time. The type `HathorSettings` lives in `settings.py`; this function in `get_settings.py` has the same name and returns an instance of that type. Most callers across the codebase use `get_global_settings()`; the same-named function is the historical spelling kept for compatibility. Both reach the same cache.

The function's job is to decide which profile to load, and it does so from environment variables⁷:

```
# hathor/conf/get_settings.py:43
def HathorSettings() -> 'Settings':
    settings_module_filepath = os.environ.get('HATHOR_CONFIG_FILE')
    if settings_module_filepath is not None:
        return _load_settings_singleton(settings_module_filepath, is_yaml=False)

    from hathorlib import conf
    settings_yaml_filepath = os.environ.get('HATHOR_CONFIG_YAML', conf.MAINNET_SETTINGS_FILEPATH)
    return _load_settings_singleton(settings_yaml_filepath, is_yaml=True)
```

Read this as a two-tier choice with a default:

1. If `HATHOR_CONFIG_FILE` is set, load a **Python-module** profile from that path (`is_yaml=False`). This is the legacy path — the docstring and a runtime warning (below) say it is being deprecated.
2. Otherwise, load a **YAML** profile from `HATHOR_CONFIG_YAML` (`is_yaml=True`).
3. If that env var is unset, fall back to `conf.MAINNET_SETTINGS_FILEPATH` — the `mainnet.yml` that ships inside the `hathorlib` package. **The default, with no configuration at all, is mainnet** (`get_settings.py:58`).

How does the env var get set? The operator does not normally set it by hand. Recall from Chapter 21 that `run_node` parses flags like `--testnet` or `--config-yaml <path>`; the CLI translates that choice into the `HATHOR_CONFIG_YAML` environment variable before anything calls `get_global_settings()`. So the data flow is: **flag → env var (set by the**

CLI) → profile path → loaded object. The environment variable is the hand-off channel between the command-line layer and the configuration layer; it keeps `hathor/conf/` from having to know anything about argument parsing.

Why an environment variable and not a direct function argument? Because `get_global_settings()` is called from hundreds of places, most of which have no access to the parsed CLI object. An env var is a process-global channel that any of those call sites can read without being handed anything. It is the same reasoning as the singleton itself: the value is needed everywhere, so it travels through a global channel rather than down every call path.

Loading once, caching forever — the singleton

The actual load-and-cache lives in `_load_settings_singleton`. The module holds one cache slot:

```
# hathor/conf/get_settings.py:30
class _SettingsMetadata(NamedTuple):
    source: str          # which file we loaded from
    is_yaml: bool        # YAML profile or legacy Python module?
    settings: 'Settings' # the validated HathorSettings instance

_settings_singleton: Optional[_SettingsMetadata] = None
```

`_settings_singleton` starts as `None` and is filled on first load. Note what is cached: not just the settings object, but also where it came from (`source`) and how (`is_yaml`). That bookkeeping powers a guard against a dangerous mistake — loading two different configurations into one process:

```
# hathor/conf/get_settings.py:72
def _load_settings_singleton(source: str, *, is_yaml: bool) -> 'Settings':
    global _settings_singleton

    if _settings_singleton is not None:
        if _settings_singleton.is_yaml != is_yaml:
            raise Exception('loading config twice with a different file type')
        if _settings_singleton.source != source:
            raise Exception('loading config twice with a different file')
        return _settings_singleton.settings # already loaded → hand back the same object
    # ...first-load path below...
```

The logic: if the cache is already populated, return it — unless the caller is asking for a **different** file or file-type than the one already loaded, in which case raise. This enforces the single-source-of-truth invariant at runtime. A program that tried to be on two networks at once (load `mainnet` then `testnet`) is a bug so severe the package refuses to let it happen quietly; it crashes instead. This is the singleton pattern doing its second job: not just "build once" but "guarantee everyone agrees."

On the first load, the function warns if the legacy Python-module path was used, then builds and stores the instance:

```
# hathor/conf/get_settings.py:83
    if not is_yaml:
        log = logger.new()
        log.warn(
            "Setting a config module via the 'HATHOR_CONFIG_FILE' env var will be deprecated soon"
            "Use the '--config-yaml' CLI option or the 'HATHOR_CONFIG_YAML' env var to set a yaml"
        )
    from hathor.conf.settings import HathorSettings as Settings

    settings_loader = load_yaml_settings if is_yaml else load_module_settings
    _settings_singleton = _SettingsMetadata(
        source=source,
        is_yaml=is_yaml,
        settings=settings_loader(Settings, source)
    )
    return _settings_singleton.settings
```

`settings_loader` is chosen by the `is_yaml` flag — either `load_yaml_settings` or `load_module_settings`, both imported from `hathorlib.conf.utils` (`get_settings.py:22`). This is the **dispatch** idea from Chapter 4: pick the function from a flag, then call it uniformly. The YAML loader parses the file and feeds the resulting dict into the Pydantic model; the module loader reads constants off a Python module. Either way, the output is a validated `HathorSettings`, stored in the cache, and returned. Every subsequent call short-circuits at the `if _settings_singleton is not None` check above.

A companion accessor, `get_settings_source()` (`get_settings.py:62`), returns the path that was loaded — useful for logging "which network am I on" — and asserts that settings were loaded first.

The `HathorSettings` type — validate at load

Now the type itself, in `settings.py`. It is a Pydantic model — and this is the recent migration the chapter set out to verify: it **is** Pydantic (Pydantic v2's `ConfigDict`, `field_validator`, `model_validator`, imported at `settings.py:16`), built as a subclass of a base model that ships in the shared `hathorlib` library:

```
# hathor/conf/settings.py:32
class HathorSettings(LibSettings):
    model_config = ConfigDict(extra='forbid')
```

Two facts are packed into those two lines.

It inherits from `LibSettings`. `LibSettings` is `hathorlib.conf.settings.HathorSettings` (imported at `settings.py:22`), itself a Pydantic model. The bulk of the field declarations — the network name (`P2P_NETWORK_NAME: str`), the size limits, the timing constants, the genesis fields — live in that shared library base class, and `hathor-core`'s subclass adds the fields specific to a full node (checkpoints, feature activation, the consensus-algorithm choice). The reason for the split: `hathorlib` is reused by lighter Hathor tools

(wallets, libraries) that need the same constants without the full node; keeping the common settings in the library avoids two copies drifting apart — the single-source-of-truth principle applied across packages, not just within one.

`extra='forbid'`. This is a Pydantic configuration that makes the model **reject any field the YAML contains that the model does not declare**. A typo in a profile —

`DECIMAL_PLACE` instead of `DECIMAL_PLACES` — does not silently fall through as an ignored extra key; it raises at load. This is the validation half of read-only's safety story: not only should nothing change the object after load, you cannot even load an object with unexpected contents. Combined with type validation, it means a malformed profile is caught at boot, not at the moment some component first reads a missing constant deep in the call stack. (This is the fail-fast vs. crash-deep trade-off: pay the cost of a clear error at startup, instead of an obscure one hours later.)

The subclass adds three full-node fields, each with a validator that turns raw YAML into the right Python type:

```
# hathor/conf/settings.py:36
CHECKPOINTS: list[Checkpoint] = []

@field_validator('CHECKPOINTS', mode='before')
@classmethod
def _parse_checkpoints(cls, checkpoints):
    # YAML gives {height: "hexhash"}; turn it into [Checkpoint(height, bytes), ...]
    if isinstance(checkpoints, dict):
        return [Checkpoint(h, bytes.fromhex(_hash)) for h, _hash in checkpoints.items()]
    ...
```

A validator⁶ is a method Pydantic runs while building the object. `mode='before'` means it runs before type-coercion, so it sees the raw YAML shape — here, a `{height: hexstring}` dict — and converts it into the typed shape the field declares — a `list[Checkpoint]`. This is how a human-friendly YAML representation (a map of heights to hex strings) becomes a strongly-typed list the rest of the code can rely on. The `FEATURE_ACTIVATION` field (`settings.py:54`) has the same shape: a sub-model built from a nested dict.

The third field carries a cross-field rule — the kind no scattered-globals design could enforce:

```
# hathor/conf/settings.py:65
    CONSENSUS_ALGORITHM: ConsensusSettings = PowSettings()

    @model_validator(mode='after')
    def _validate_consensus_algorithm(self) -> Self:
        """Validate that if Proof-of-Authority is enabled, block rewards must not be set."""
        if self.CONSENSUS_ALGORITHM.is_pow():
            return self
        if (self.BLOCKS_PER_HALVING is not None or
            self.INITIAL_TOKEN_UNITS_PER_BLOCK != 0 or
            self.MINIMUM_TOKEN_UNITS_PER_BLOCK != 0):
            raise ValueError('PoA networks do not support block rewards')
        return self
```

A `model_validator(mode='after')` runs once, after all individual fields are set, so it can see the whole object and check relationships between fields. The rule: a proof-of-authority⁴ network mints no block reward (in PoA, blocks are signed by authorities, not mined, so there is nothing to reward). If a profile declares PoA and a reward schedule, the two are contradictory, and the model refuses to build. This is the payoff of the single-validated-object design from §22.2: cross-field invariants live in exactly one place and are checked exactly once, at load.

The module also defines a few monetary constants used to compute genesis amounts (`settings.py:24`):

```
# hathor/conf/settings.py:24
DECIMAL_PLACES = 2                                # HTR has 2 decimal places
GENESIS_TOKEN_UNITS = 1 * (10 ** 9)                # 1B units
GENESIS_TOKENS = GENESIS_TOKEN_UNITS * (10 ** DECIMAL_PLACES) # → 100B (with decimals)
HATHOR_TOKEN_UID = b'\x00'                         # the native token's id is a single null byte
```

`HATHOR_TOKEN_UID = b'\x00'` is worth a glance: the native token HTR is identified everywhere in the code by a single null byte, distinguishing it from custom tokens (which get longer ids). That one constant ripples through the entire monetary subsystem.

A representative profile excerpt

The default profile is a YAML file. A profile reads as a flat list of the constants the model declares; below is a representative excerpt (field names are real `HathorSettings` fields; treat the exact values as illustrative of shape, not as line-verified mainnet numbers):

```
# (representative excerpt of a YAML profile)
P2P_NETWORK_NAME: mainnet           # announced in the peer handshake
DECIMAL_PLACES: 2                   # HTR has two decimal places

# genesis – the shared root of history; differs per network
GENESIS_BLOCK_HASH: "00000033...e9c"
GENESIS_BLOCK_NONCE: 3526202
GENESIS_OUTPUT_SCRIPT: "76a914...88ac"

# monetary schedule
INITIAL_TOKEN_UNITS_PER_BLOCK: 64    # block reward when the chain starts
MINIMUM_TOKEN_UNITS_PER_BLOCK: 1     # reward floor after halvings
BLOCKS_PER_HALVING: 2102400         # how often the reward halves

# timing / difficulty
AVG_TIME_BETWEEN_BLOCKS: 30         # target seconds per block

# structural limits
MAX_NUM_INPUTS: 255
MAX_NUM_OUTPUTS: 255
```

Walk three of these to feel why a mismatch is fatal:

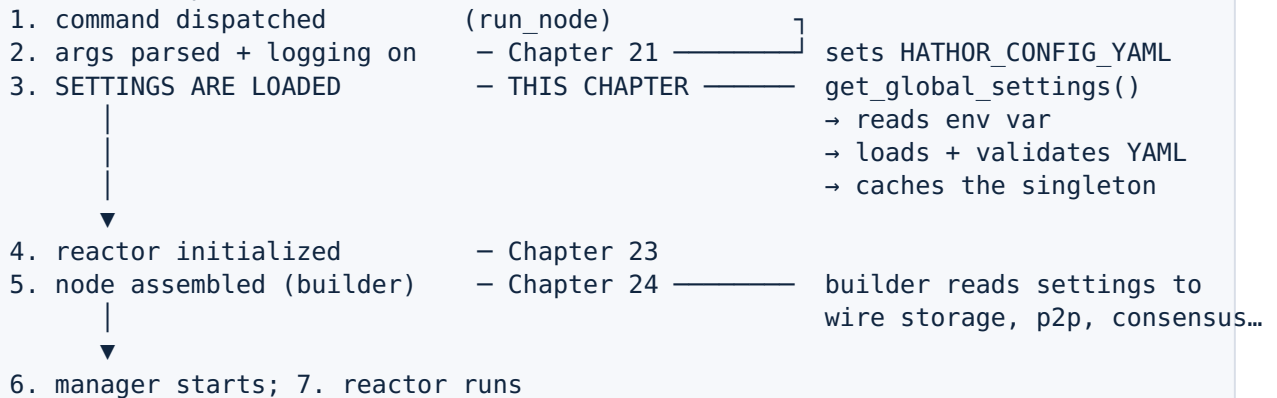
- **P2P_NETWORK_NAME** . Two nodes compare this string during the handshake (Ch. 34). If yours says `mainnet` and the peer's says `testnet-golf`, the handshake fails and you never connect. A one-word difference is the boundary between two networks.
- **GENESIS_BLOCK_HASH (and the rest of the genesis data)**. This is the root of history. If your genesis bytes differ from the network's by a single bit, your hash is different, your chain has a different bottom, and no block the network produces will ever link to yours. You will reject everything and be rejected by everything — silently.
- **AVG_TIME_BETWEEN_BLOCKS** . This feeds the difficulty-adjustment math (Ch. 32). Get it wrong and your node computes a different expected weight for blocks than the network does, so it disagrees about which blocks are valid. Again: not a crash — a fork.

That is the whole reason this package validates so strictly and treats its output as read-only. The cost of a wrong constant is not an exception you can catch; it is a node that looks healthy and is on its own private island.

22.5 How it plugs into the lifecycle

Place this chapter precisely in the boot story from Chapter 0, §0.3, **Act I**:

Act I – Startup



The sequence matters. Settings load **after** the command line (Ch. 21), because the chosen network comes from a flag the CLI translates into the env var. Settings load **before** the builder (Ch. 24), because the builder needs the constants to construct components correctly — it reads the network name to set up p2p, the genesis to seed storage, the consensus algorithm to pick a consensus engine, and so on. From step 5 onward, every component that needs a constant calls `get_global_settings()` and receives the one cached, validated, read-only object. By the time the reactor is running (step 7), the settings have been fixed and shared for the entire rest of the process's life — they are never reloaded.

One design consequence to carry forward: because the value is a process-wide singleton selected by an env var, **tests and the simulator** (Ch. 43) must set that env var (or otherwise install a settings object) before anything reads it, and cannot freely switch networks mid-process — the duplicate-load guard in §22.4 will stop them. That constraint is the price of the single-source-of-truth guarantee, and the test infrastructure is built around it.

Recap

Question	Answer	Where in code
Which network am I on?	Chosen by an env var the CLI sets (<code>HATHOR_CONFIG_YAML</code>), default <code>mainnet</code>	<code>get_settings.py:43 , :58</code>
How is a profile stored?	A YAML file of constants (legacy Python-module path is deprecated)	<code>get_settings.py:53–59</code>
What turns YAML into an object?	A Pydantic model with field/model validators	<code>settings.py:32 , :36 , :65</code>

Question	Answer	Where in code
Why won't a typo slip through?	<code>model_config = ConfigDict(extra='forbid')</code> rejects unknown fields	<code>settings.py:33</code>
Why only one settings object?	A module-level singleton cache, with a guard against loading two	<code>get_settings.py:36, :72</code>
How does everyone read it?	One accessor, <code>get_global_settings()</code> , returns the cached instance	<code>get_settings.py:39</code>
Why does a wrong constant matter?	A node with different constants is on a different network, silently	\$22.2, \$22.4

Settings is the node's single source of truth for "what network am I on and what are its rules." It is loaded once, validated strictly at load (fail fast, not crash deep), cached as a read-only singleton, and read by everything below it. You now know how `get_global_settings()` chooses a profile from the environment, parses it into a `HathorSettings` with Pydantic, and refuses to load two networks at once. The next chapter (23) covers the reactor abstraction the node spins up immediately after settings are fixed; from there, Chapter 24 takes this settings object into the builder, where it is read to wire the entire node together.

1. The genesis is the hard-coded first block and initial transactions that every node on a network agrees on by definition — the shared root of history. Different networks have different genesis data; that difference is part of what makes them distinct networks. Full treatment in Ch. 25 & 32. ↩
2. Weight is a numeric measure of how much proof-of-work a vertex represents; consensus prefers the history with the most accumulated weight. The constants that drive the weight/difficulty math live in the settings profile. Full treatment in Ch. 9 & 32. ↩
3. Feature activation is Hathor's mechanism for switching protocol upgrades on over a schedule, by miner signalling. The schedule for a network is part of its settings profile (the `FEATURE_ACTIVATION` field). Full treatment in Ch. 38. ↩
4. Proof-of-authority (PoA) is a consensus variant used by certain private networks, where blocks are signed by designated authorities rather than mined via proof-of-work. Because nothing is "mined," PoA networks define no block reward — a rule the settings model enforces. Full treatment in Ch. 32. ↩↩
5. A read-only object is one that nothing is supposed to mutate after it is created; configuration is treated this way so no stray code can change a network constant mid-run. (Pydantic models are mutable by default unless explicitly frozen; in Hathor the read-only discipline is enforced partly by convention and partly by the load-time `extra='forbid'` validation that rejects malformed input in the first place.) ↩

6. A validator (in Pydantic) is a method that runs while the model is being built, to coerce raw input into the declared type (`mode='before'`) or to check relationships between fields once they are all set (`mode='after'`). It is how a human-friendly YAML shape becomes a strongly-typed, internally-consistent object — or is rejected. ↩
7. An environment variable is a named value the operating system makes available to a running process (read in Python via `os.environ`). It is a process-global channel: any part of the program can read it without being passed it explicitly, which is why it suits a setting that must be reachable from hundreds of call sites. ↩

Chapter 23 — The Reactor Abstraction

What you'll learn in this chapter

- Why Hathor wraps Twisted's reactor in its own tiny package instead of importing the global reactor everywhere.
- What `ReactorProtocol` is — a typed name for the slice of the reactor Hathor actually uses — and how it is stitched from three smaller protocols.
- How `initialize_global_reactor()` installs the reactor once at boot, and `get_global_reactor()` hands the same one to everyone afterward.
- What `verifyObject` checks before the reactor is handed out, and why that check sits next to a `cast`.
- What the experimental `--x-asyncio-reactor` flag does, why it is opt-in rather than the default, and the module-level-import footgun the code warns about.

This is the short companion to Chapter 16. There we taught the Twisted reactor, `Deferred`, `Protocol`, and `Factory` from the ground up. Here we look at one thin package built on top of that machinery: `hathor/reactor/`. The reactor itself is Twisted's; this chapter is about the small wrapper Hathor puts around it, and the two problems that wrapper solves. We recap Chapter 16's concepts only enough to follow along — the full treatment stays there.

23.1 Localization

`hathor/reactor/` is one of the smallest packages in the tree. It is an `__init__.py` plus two groups of files: one module for the type and one for the access functions.

```

hathor/
├── builder/           ← Ch 24: assembles the node
├── conf/             ← Ch 22: settings & profiles
├── manager.py        ← HathorManager: the node's brain (Ch 29)
├── p2p/              ← the peer-to-peer stack (Ch 34-35)
├── reactor/          ← YOU ARE HERE
│   ├── __init__.py   (re-exports the 3 public names)
│   ├── reactor.py    (the accessor + the initializer)
│   ├── reactor_protocol.py (ReactorProtocol = the 3 below, joined)
│   ├── reactor_core_protocol.py (run/stop/callWhenRunning slice)
│   ├── reactor_time_protocol.py (callLater/seconds slice)
│   └── reactor_tcp_protocol.py (listenTCP/connectTCP slice)
├── transaction/      ← the vertex model (Ch 25)
└── ...

```

Six files, well under two hundred lines of real code between them. Yet almost every long-lived object in a running node holds a reference to whatever this package returns: the manager, the P2P connections, every timer, every scheduled retry. The reactor is the single thread on which the whole node lives (Ch 16), and `hathor/reactor/` is the one door through which the rest of the code reaches it.

Context. `hathor/reactor/` does not implement an event loop. It is a naming and access layer: it gives Twisted's loosely-typed reactor one clear type (`ReactorProtocol`), and it gives the whole program one function to install it (`initialize_global_reactor`) and one function to fetch it afterward (`get_global_reactor`). Everything downstream depends on this package instead of on `twisted.internet.reactor` directly.

23.2 What it does and why it exists

The package exists to solve two separate problems. Keep them apart in your head; they are why the code is shaped the way it is.

Problem 1 — the reactor has no useful type. When you write `from twisted.internet import reactor`, the object you get back is, as far as a type checker is concerned, barely typed.¹ Twisted's reactor is a single object that happens to provide more than a dozen interfaces² — one for core lifecycle (`run`, `stop`, `callWhenRunning`), one for timed calls (`callLater`), one for TCP (`listenTCP`, `connectTCP`), and so on. Code that calls `reactor.callLater(...)` has no compile-time guarantee the object even has that method. For a codebase that runs `mypy`,³ that is a real gap: bugs a type checker should catch slip through. Hathor closes it by declaring, in one place, exactly which capabilities it relies on, as one type a checker can follow.

Problem 2 — the node needs one shared reactor, reachable from anywhere. A

reactor is, by nature, a singleton⁴ — a program has exactly one event loop. Dozens of objects, built at different times by the builder (Ch 24), all need to schedule work on that loop. Passing the reactor by hand through every constructor would be noisy and error-prone. So `hathor/reactor/` keeps the reactor in one module-level variable and exposes functions to install and reach it. This is the **global accessor** pattern (Ch 3): one well-known function, `get_global_reactor()`, that always returns the same object.

A useful way to see the whole package: it is a typed front door to a shared resource. The type is `ReactorProtocol`; the door is the accessor pair.

23.3 The concepts it rests on

This chapter leans on four ideas taught canonically elsewhere. Here are the reminders; follow the pointers if any feels unfamiliar.

Recap — the reactor / event loop (full treatment in Ch. 16 and Ch. 2). An event loop is a single thread that waits for events (a socket becomes readable, a timer fires) and runs the callback registered for each, one at a time, forever. Twisted's **reactor** is that loop. A network node uses one so it can juggle thousands of connections without a thread per connection. → full treatment in Ch. 16 (the framework) and Ch. 2 (the idea).

Recap — `zope.interface` and structural contracts (full treatment in Ch. 16 and Ch. 5). Twisted describes what the reactor can do with `zope.interface`⁵ interfaces — named bundles of methods, like `IReactorTime` ("can schedule timed calls"). An object that provides an interface promises those methods exist, and you can check that at runtime. This is the runtime-checked cousin of Python's `typing.Protocol` (Ch 5), the static-typing tool Hathor uses to mirror those interfaces. → full treatment in Ch. 16 (zope) and Ch. 5 (`Protocol`).

Recap — adapter / shim (full treatment in Ch. 3 and Ch. 4). An adapter is a thin layer that presents an existing thing through a different, more convenient interface, without changing the thing itself. `hathor/reactor/` is a shim: it neither replaces Twisted's reactor nor wraps its behavior — it only re-presents it under a clearer type and a single accessor. → full treatment in Ch. 3 (adapter) and Ch. 4 (shim).

Recap — singleton / global accessor (full treatment in Ch. 3). A singleton is a resource of which there is exactly one instance for the whole process. A global accessor is the function everyone calls to reach it. Together they avoid threading one shared object through every constructor by hand. Chapter 3 flagged this as a global-state smell; here it is unavoidable, because the event loop is singular. → full treatment in Ch. 3.

23.4 The code, walked

23.4.1 A tiny illustration first

Before the real code, a generic sketch of the move at the heart of this package: take a loosely-typed object and re-present it under a precise type.

Suppose a third-party library hands you a `connection` object whose type is `Any` — the type checker knows nothing about it. You know, from the docs, that it has `send()` and `close()`. You can declare a `Protocol` that names exactly those, then tell the type checker "treat this object as that":

```
from typing import Any, Protocol, cast

class Sendable(Protocol):
    def send(self, data: bytes) -> None: ...
    def close(self) -> None: ...

raw: Any = third_party.connect()      # type checker knows nothing
conn = cast(Sendable, raw)            # promise: it behaves like Sendable
conn.send(b'hello')                  # now type-checked
```

Nothing about `raw` changed at runtime. `cast`⁷ is a promise to the type checker, not a conversion. From here on, `conn.send` is checked and `conn.frobnicate()` is flagged as an error. That is precisely what `hathor/reactor/` does to Twisted's reactor — only the "promise" type is `ReactorProtocol`, and before the promise is made the code first verifies it is true (§23.4.4).

23.4.2 `ReactorProtocol` — three slices joined into one type

The type lives in `hathor/reactor/reactor_protocol.py`. It is built from three smaller protocols, one per Twisted interface Hathor uses (`reactor_protocol.py:22`):

```

from typing import Protocol

from hathor.reactor.reactor_core_protocol import ReactorCoreProtocol
from hathor.reactor.reactor_tcp_protocol import ReactorTCPProtocol
from hathor.reactor.reactor_time_protocol import ReactorTimeProtocol

class ReactorProtocol(
    ReactorCoreProtocol,
    ReactorTimeProtocol,
    ReactorTCPProtocol,
    Protocol,
):
    """
    A Python protocol that represents the intersection of Twisted's
    IReactorCore+IReactorTime+IReactorTCP interfaces.
    """
    pass

```

`ReactorProtocol` is a `typing.Protocol`⁶ — a structural type. Its body is `pass` because it adds nothing of its own; it exists purely to name the union of three capability slices. Each slice is its own small protocol that re-declares one Twisted interface so `mypy` can follow it:

- `ReactorCoreProtocol` — lifecycle: `run()`, `stop()`, `callWhenRunning()`, `addSystemEventTrigger()` (`reactor_core_protocol.py:38`).
- `ReactorTimeProtocol` — timers: `seconds()`, `callLater()`, `getDelayedCalls()` (`reactor_time_protocol.py:32`).
- `ReactorTCPProtocol` — TCP: `listenTCP()` (`reactor_tcp_protocol.py:33`), `connectTCP()` (`:43`).

Joining several contracts into one type like this gives an **intersection type**¹⁰: an object is a `ReactorProtocol` only if it provides all three slices at once. The class docstring says exactly that — it is "the intersection of Twisted's `IReactorCore` + `IReactorTime` + `IReactorTCP` interfaces."

Why split into three sub-protocols rather than list every method in one class? Because each slice mirrors one Twisted interface, so the code that verifies the real reactor (§23.4.4) can check one interface against one slice. The full Twisted reactor provides much more (UDP, SSL, threads, subprocesses); Hathor names only this slice. If a future need arises — say, UDP — the fix is to add a fourth sub-protocol and a fourth base, in this one place, and the checker will then permit `reactor.listenUDP(...)` everywhere.

Why not just type things as "the whole reactor"? Because Twisted exposes no single class that means "the reactor"; it ships a bag of interfaces and one concrete object that happens to satisfy many of them. `ReactorProtocol` is Hathor's answer to the missing name.

23.4.3 `get_global_reactor` — the accessor

The accessor and initializer both live in `hathor/reactor/reactor.py`. The module holds one piece of state (`reactor.py:26`):

```
# Internal variable that should NOT be accessed directly.
_reactor: ReactorProtocol | None = None
```

That module-level `_reactor` is the single slot the whole program shares.

`get_global_reactor()` (`reactor.py:29`) reads it:

```
def get_global_reactor() -> ReactorProtocol:
    global _reactor

    if _reactor is None:
        raise Exception('The reactor is not initialized. Use `initialize_global_reactor()`.')

    return _reactor
```

Read it top to bottom. If a reactor has been installed, return it — the common case, hit thousands of times. If the slot is empty, it does **not** silently invent one; it raises. That strictness is deliberate. There is exactly one correct moment to choose and install the reactor — early in boot, where the asyncio flag is known (§23.5). If

`get_global_reactor()` quietly installed a default reactor on first use, an accidental early call would lock in the wrong reactor and the `--x-asyncio-reactor` choice could never take effect. Raising turns that ordering mistake into a loud, named failure instead of a silent wrong default.

The docstring carries a second rule (`reactor.py:34`): this function "must NOT be called in the module-level, only inside other functions." The reason is the same import-time footgun we meet fully in §23.4.4 — touching the reactor while a module is being imported can install it before the initializer runs.

23.4.4 `initialize_global_reactor` — the installer, the verify, and the asyncio branch

The accessor reads the slot. The initializer fills it, and is the only place that gets to choose which reactor goes in. It is defined at `reactor.py:44`. Take it in three parts.

Part 1 — the already-installed guard.

```
def initialize_global_reactor(*, use_asyncio_reactor: bool = False) -> ReactorProtocol:
    global _reactor

    if _reactor is not None:
        log = logger.new()
        log.warn('The reactor has already been initialized. Use `get_global_reactor()`.')
        return _reactor
```


If the slot is already full (`reactor.py:51`), the function does not raise — it logs a warning and returns the existing reactor. The expectation is that this function is called exactly once; a second call is treated as a caller mistake worth a log line, not a crash. (Note the asymmetry with `get_global_reactor` , which raises when the slot is empty. Empty-on-read is a real bug; full-on-init is a harmless redundancy.)

Part 2 — the asyncio branch. When `use_asyncio_reactor` is true (`reactor.py:56`):

```
if use_asyncio_reactor:
    import asyncio

    from twisted.internet import asyncioreactor
    from twisted.internet.error import ReactorAlreadyInstalledError

    try:
        loop = asyncio.get_running_loop()
    except RuntimeError:
        loop = asyncio.new_event_loop()
        asyncio.set_event_loop(loop)

    try:
        asyncioreactor.install(loop)
    except ReactorAlreadyInstalledError as e:
        msg = (
            "There's a Twisted reactor installed already. It's probably the default one, installed
            "one of our imports. This can happen, for example, if we import from the hathon m
            "entrypoint-level, like in CLI tools other than `RunNode`. ..."
        )
        raise Exception(msg) from e
```

The branch reuses the current asyncio event loop if one is already running, otherwise creates and registers a fresh one (`reactor.py:62–66`). It then asks Twisted to install its `asyncioreactor` backed by that loop (`reactor.py:69`). The effect: Twisted's reactor and Python's `asyncio` share a single underlying loop, so code written against either can run together.

The `try/except` around `asyncioreactor.install` (`reactor.py:70`) catches a Twisted error, `ReactorAlreadyInstalledError`,⁹ and re-raises it with a pointed explanation. That message names the real footgun: importing anything from the `hathon` module at entrypoint level — as a CLI tool other than `RunNode` might — can install the **default** Twisted reactor indirectly, before this function runs. Once the default reactor is in, the asyncio install fails. The verbose message exists because this is a confusing failure to debug from the stack trace alone.

The ordering rule, stated plainly. Both `initialize_global_reactor` and `get_global_reactor` carry the docstring warning "must NOT be called in the module-level, only inside other functions" (`reactor.py:34` , `:47`). Twisted installs a reactor the first time `twisted.internet.reactor` is imported. If any module-level import triggers that before the initializer chooses, the asyncio branch can no longer take effect. This is why `run_node` calls the initializer from inside `prepare()` (`run_node.py:202`), not at the top of the file.

Part 3 — verify, then cast. Whether or not the asyncio branch ran, the function finishes the same way (`reactor.py:80`):

```
from twisted.internet import reactor as twisted_reactor

assert verifyObject(IReactorTime, twisted_reactor) is True
assert verifyObject(IReactorCore, twisted_reactor) is True
assert verifyObject(IReactorTCP, twisted_reactor) is True

# We cast to ReactorProtocol, our own type that stubs the necessary
# Twisted zope interfaces, to aid typing.
_reactor = cast(ReactorProtocol, twisted_reactor)
return _reactor
```

It imports whichever reactor is now current, then runs `verifyObject`⁸ three times (`reactor.py:82–84`) — once per Twisted interface that `ReactorProtocol` mirrors. `verifyObject` is a runtime check: it confirms the real reactor genuinely provides every method of `IReactorTime` , `IReactorCore` , and `IReactorTCP` . Only after those pass does the code `cast` the reactor to `ReactorProtocol` (`reactor.py:87`) and store it.

This pairing is the whole point of the abstraction in miniature. The `cast` is a promise to the static type checker (§23.4.1); the `verifyObject` asserts the runtime truth behind that promise. Recall from §23.4.1 that a bare `cast` is unchecked — it would happily lie. The three asserts make sure the lie is actually true before it is told, so a reactor that silently lacked, say, `connectTCP` would fail loudly here at boot rather than mysteriously much later.

23.4.5 The public surface

`hathor/reactor/__init__.py` re-exports the three public names — `initialize_global_reactor` , `get_global_reactor` , and `ReactorProtocol` (`__init__.py:18–22`). The rest of the codebase imports from `hathor.reactor` , never from `twisted.internet` . That one rule is what keeps the seam intact: change the wrapper once, and every consumer follows.

23.5 How it plugs into the lifecycle

Recall the boot skeleton from the Orientation chapter (Ch 0, §0.3, Act I). Step 4 is "**The reactor is initialized** — the event loop is created and installed as the global reactor." That step is a single call into this package.

Following the order in `run_node.py`'s `prepare()` method:

1. Settings are loaded and the network profile is selected (Ch 22).
2. Runtime preconditions are validated — `validate_args()`, `check_unsafe_arguments()`, `check_python_version()` (`run_node.py:197–199`).
3. **The reactor is installed:**
`initialize_global_reactor(use_asyncio_reactor=self._args.x_asyncio_reactor)` (`run_node.py:202`). The chosen reactor is stored on `self.reactor` (`:203`) and in the package's module slot.
4. Hand off to the builder (Ch 24):
`CliBuilder(self._args).create_manager(reactor)` (`run_node.py:208–210`). Every object that needs to schedule work is handed `self.reactor` or reaches it through `get_global_reactor()`.
5. At the very end of boot, `self.reactor.run()` (`run_node.py:592`) starts the loop. That call does not return until the node shuts down — it is Act II.

This placement is why the chapter sits where it does. Settings (Ch 22) decide what to build; the reactor (this chapter) provides the loop everything will run on; the builder (Ch 24) does the building. The reactor must exist before the builder runs, because the objects the builder creates capture a reference to it.

The asyncio option, in current reality

The `--x-asyncio-reactor` flag is defined at `run_node.py:161` with the help text "Use asyncio reactor instead of Twisted's default." Two facts about its current state, both visible in the code:

- **The `x-` prefix marks it experimental.** In this codebase the `x-` / `--x-` convention flags unstable, opt-in features — `--x-asyncio-reactor` sits in the same `UNSAFE_ARGUMENTS` list as `--x-ipython-kernel` (`run_node.py:58–59`). The flag uses `action='store_true'`, so it defaults to `False`.
- **The default is the standard Twisted reactor.** When the flag is off, `initialize_global_reactor` skips the asyncio branch entirely and installs Twisted's ordinary global reactor. You opt into the asyncio-backed reactor only when you need Twisted and `asyncio` code to interoperate on one loop.

Treating asyncio as opt-in is deliberate. The standard reactor is the well-tested production path; swapping the event-loop backend is the kind of change you do not want every node to take by default. The seam exists so the option can be exercised — not so it is the norm. The deeper "why Twisted, not asyncio" trade-off is argued in Ch 16, §16.7.

Recap

Fact	Where
The package is a typed seam over Twisted's reactor, not an event loop of its own	<code>hathor/reactor/</code>
<code>ReactorProtocol</code> = intersection of three slice-protocols mirroring <code>IReactorCore</code> / <code>IReactorTime</code> / <code>IReactorTCP</code>	<code>reactor_protocol.py:22</code>
The three slices declare the methods Hathor uses	<code>reactor_core_protocol.py:38</code> , <code>reactor_time_protocol.py:32</code> , <code>reactor_tcp_protocol.py:33</code>
One shared module-level slot holds the reactor	<code>reactor.py:26</code>
<code>get_global_reactor()</code> returns it; raises if not yet initialized	<code>reactor.py:29</code> , <code>:38</code>
<code>initialize_global_reactor(...)</code> installs it; a second call warns and returns the existing one	<code>reactor.py:44</code> , <code>:51</code>
The asyncio backend is installed before the reactor import; an indirect default-reactor install is caught and re-explained	<code>reactor.py:56–78</code>
<code>verifyObject</code> checks the three interfaces at runtime, then <code>cast</code> makes the static promise	<code>reactor.py:82–87</code>
Both accessor and initializer must not be called at module level	<code>reactor.py:34</code> , <code>:47</code>
Boot installs the reactor in <code>prepare()</code> , before the builder	<code>run_node.py:202</code>
<code>--x-asyncio-reactor</code> is experimental and off by default	<code>run_node.py:161</code> , <code>:58</code>
<code>self.reactor.run()</code> is the last line of boot	<code>run_node.py:592</code>

The reactor abstraction earns its keep with very little code: one type to make the reactor legible to `mypy`, one accessor so the whole node shares a single loop, and one initializer that picks the backend, verifies it, and installs it once. Hold onto two facts as you move on — every long-lived object you meet from here holds a reactor reference, and that reference is always handed out through this package. Next, **Chapter 24** turns to the builder: the composition root that constructs those objects and threads the freshly-installed reactor through all of them.

1. Twisted predates Python's modern type system, so its reactor is exported as a plain object satisfying many interfaces rather than as one statically-typed class. Without help, a checker treats most accesses on it as untyped. ↵
2. An interface is a named list of methods an object promises to provide — a contract. It says nothing about implementation, only about the shape of the object. ↵
3. `mypy` is a static type checker for Python: it reads type annotations and flags mismatches before the program runs. `hathor-core` runs it in CI (Ch 20), so untyped objects weaken its coverage. ↵
4. A singleton is a resource of which exactly one instance exists per process. An event loop is inherently one-per-process, which makes the reactor a natural singleton. Full treatment in Ch. 3. ↵
5. `zope.interface` is the library Twisted uses to declare and check interfaces at runtime. Its interface names conventionally start with a capital `I` (`IReactorTime`). It is the runtime-checked cousin of `typing.Protocol`. ↵
6. Python's `typing.Protocol` enables structural typing: any object with the required methods satisfies the protocol, with no need to inherit from it. It is a static-typing tool, checked by `mypy`, and is unrelated to Twisted's connection-handler `Protocol` class. Full treatment in Ch. 5. ↵
7. `typing.cast(T, x)` tells the type checker "treat `x` as type `T`." It performs no runtime check or conversion; it only changes how the checker reasons about the value — which is why Hathor pairs it with a runtime `verifyObject` check. ↵
8. `zope.interface.verify.verifyObject(Interface, obj)` checks at runtime that `obj` actually provides every method the interface declares, raising if not. Hathor uses it to confirm the real reactor matches `ReactorProtocol` before casting to that type. ↵
9. Twisted's own `ReactorAlreadyInstalledError` is raised by `asyncioreactor.install` when a reactor is already in place. Hathor catches it only inside the `asyncio` branch, to replace the bare error with a message explaining the likely cause (an indirect default-reactor install at import time). ↵
10. An intersection type describes a value that satisfies several contracts at once. Python has no dedicated syntax for it, so the idiom is to define a type that inherits from each contract, as `ReactorProtocol` does with its three slice-protocols. ↵

Chapter 24 — The Composition Root: Builder & CliBuilder

What you'll learn in this chapter

- What a composition root is, and why a program should have exactly one place where its objects are constructed and connected.
- How `hathor-core` builds a full node: storage, indexes, pub-sub, the event manager, consensus, verification, P2P, and the `HathorManager` that ties them together.
- The two composition roots that coexist in the codebase — `Builder` (for tests and the simulator) and `CliBuilder` (for production) — and why there are two.
- Where this "builder phase" sits in the life of a node: between the reactor being readied (Ch. 23) and the manager being started (Ch. 29).
- The honest trade-off of having two builders that can drift apart, and the `TODO` in the code that admits it.

This chapter is step 5 of the life-of-a-node story from Chapter 0 (§0.3, "the node is assembled — the builder phase"). Everything Part II has covered so far — the command line (Ch. 21), the settings (Ch. 22), the reactor (Ch. 23) — has been preparation. This is where those ingredients are turned into a real, wired-together node object, ready to be switched on.

24.1 Localization

The package `hathor/builder/` is small in file count but central in role: nothing else can produce a working node without it. There is also a second builder, for the command line, that lives next door under `hathor_cli/`.

```

hathor-core/
├── hathor/
│   ├── builder/
│   │   ├── __init__.py
│   │   ├── builder.py
│   │   ├── resources_builder.py
│   │   └── sysctl_builder.py
│   ├── manager.py
│   ├── consensus/
│   ├── transaction/storage/
│   ├── indexes/
│   ├── p2p/
│   └── verification/
└── hathor_cli/
    ├── builder.py
    └── run_node.py

```

- ← YOU ARE HERE (part 1)
- ← exports Builder, BuildArtifacts, ResourcesBuilder, SyncSupportLevel
- ← Builder (tests + simulator path)
- ← ResourcesBuilder (the HTTP/REST API tree)
- ← builds the sysctl control surface (Ch. 41)
- ← HathorManager – the object being built (Ch. 29)
- ← ConsensusAlgorithm (Ch. 32)
- ← tx storage backends (Ch. 27)
- ← IndexesManager (Ch. 28)
- ← ConnectionsManager (Ch. 34)
- ← VerificationService (Ch. 31)
- ← YOU ARE HERE (part 2): CliBuilder
- ← thin shim; imports RunNode from hathor_cli

Two things to notice. First, the two builders live in different top-level packages:

`Builder` is in `hathor/builder/`, but `CliBuilder` is in `hathor_cli/builder.py` (`hathor_cli/builder.py:60`). Second, the class that actually drives the production boot — `RunNode` — lives in `hathor_cli/run_node.py`, and it is the one that imports and uses `CliBuilder` (`hathor_cli/run_node.py:207`). This is the same run-node body Chapter 21 walked as "the CLI surface": there, the focus was argument parsing and the boot sequence; here, the focus is the one step inside it that assembles the node.

Context. This package is the assembly line of the node. Every other chapter in Part II describes a part — the vertex model, consensus, the P2P manager, the indexes. This chapter describes the one place where all those parts are instantiated, handed their dependencies, and bolted onto a single `HathorManager`. If you want to know "what gets built, in what order, with what wired to what," these two files are where you look.

24.2 What it does, and why it exists

A running node is not one object. It is roughly a dozen long-lived objects that hold references to each other: a transaction store, a set of indexes over that store, a pub-sub bus, an event manager that listens on the bus, a consensus engine, a verification service, a P2P connection manager, a wallet, a mining service. Each has its own constructor arguments — and many of those arguments are other objects from the same list. Consensus needs the pub-sub bus. The event manager needs the bus and the reactor. The verification service needs the settings, the difficulty algorithm, and the feature service.

Somewhere, someone has to call all those constructors **in the right order**, passing each object the collaborators it depends on. That "somewhere" is the **composition root**¹.

Why centralize the wiring?

Consider the alternative: each entry point — the test suite, the simulator, the production CLI — constructs the node inline, where it happens to need one. You would then have the same dozen-line wiring incantation copied in three places. Change one dependency — say, consensus now needs a new collaborator — and you must find and fix every copy. Miss one, and that code path silently builds a subtly different, broken node.

Centralizing the wiring means the construction order and the dependency edges live in **one** place. Every caller gets the same node, built the same way. When a component's constructor changes, there is exactly one site to update.

Why it makes the node testable

A composition root also makes dependencies explicit and injectable — the heart of the dependency-injection idea from Chapter 3. Because the manager receives its collaborators as constructor arguments instead of creating them itself, a test can hand it a fake. The manager does not know, and does not care, whether its storage writes to RocksDB or to a Python dictionary in memory. It talks to the abstract `TransactionStorage` interface. That substitutability is what makes the whole node testable, and it is decided here, in the builder.

So the builder package exists to answer one question, once: given a configuration, produce a fully wired `HathorManager`.

24.3 The concepts it rests on

This chapter pays off ideas the foundations already taught. Recap, don't re-teach.

Recap — the Builder pattern (full treatment in Ch. 3 §3.3). A Builder separates how an object is assembled from what it finally is. You push configuration in, one call at a time, then ask for the finished product. The builder holds the half-built state; you hold only the handle. Hathor's `Builder` is the textbook shape: fluent `set_*` methods that each return the builder, and a terminal `build()` that produces the result. → full treatment in Ch. 3.

Recap — dependency injection (full treatment in Ch. 3). Objects receive their collaborators from outside rather than constructing them internally. The builder is where the injection happens: it creates the pub-sub bus once, then passes that one bus into consensus, into the event manager, into the P2P manager. None of those build their own bus. → full treatment in Ch. 3.

Recap — Facade (full treatment in Ch. 3 §3.6 and Ch. 29). `HathorManager` is the Facade: one object presenting a small interface (`start()` , `stop()` , `on_new_tx()`) over a complex subsystem. The builder's job is to construct that subsystem and place the facade on top of it. The builder builds the thing; Ch. 29 describes what the thing does. → full treatment in Ch. 29.

Recap — interchangeable backends (full treatment in Ch. 1 and Ch. 27). The same node can keep its ledger in memory (for tests) or in RocksDB (for production). That choice is made here, in the builder, by instantiating one storage class or the other. The rest of the node speaks only to the abstract `TransactionStorage` interface and never learns which backend it got. → full treatment in Ch. 27.

24.4 The code, walked

24.4.1 A generic builder, first

Before the real node, a toy. Strip the Builder pattern down to its bones: a thing assembled from optional parts, produced by one terminal call.

```
class SandwichBuilder:
    def __init__(self):
        self._bread = None
        self._filling = None

    def set_bread(self, bread):
        self._bread = bread
        return self          # fluent: return self so the calls chain

    def set_filling(self, filling):
        self._filling = filling
        return self

    def build(self):
        bread = self._bread or "white"          # a default, if unset
        return Sandwich(bread, self._filling)

# usage
sandwich = SandwichBuilder().set_bread("rye").set_filling("cheese").build()
```

Three ideas to carry forward:

1. **Setters return `self`**. That is the fluent interface — it lets calls chain in one line. Each setter only stashes a value.
2. **`build()` is terminal**. Construction happens at the end, once, when all the configuration is in.
3. **Defaults live in the builder**. If you never call `set_bread`, the builder picks one. The caller need not specify everything.

Hathor's `Builder` is this exact shape, scaled up to a full node, with one addition: instead of computing defaults inline inside `build()`, it uses lazy, memoized getters. We'll see why that matters.

24.4.2 The real `Builder` — tests and simulator

`Builder` lives at `hathor/builder/builder.py:118`. Its `__init__` does almost nothing but declare a long list of `Optional` slots, all initialized to `None` (`builder.py:127 – 201`). Each slot is a place a `set_*` call can drop a value, or a place a getter can later cache one:

```
# hathor/builder/builder.py (abridged from __init__, lines 127–166)
class Builder:
    def __init__(self) -> None:
        self.artifacts: Optional[BuildArtifacts] = None # the finished bundle
        self._settings: Optional[HathorSettingsType] = None
        self._reactor: Optional[Reactor] = None
        self._pubsub: Optional[PubSubManager] = None
        self._tx_storage: Optional[TransactionStorage] = None
        self._indexes_manager: Optional[IndexesManager] = None
        self._rocksdb_storage: Optional[RocksDBStorage] = None
        # ... a dozen more slots: wallet, consensus, feature_service, daa, etc.
```

The fluent setters. Each `set_*` checks that the node has not already been built, stashes a value, and returns `self`. The guard is a one-liner, `check_if_can_modify` (`builder.py:313`):

```
# hathor/builder/builder.py:313–320
def check_if_can_modify(self) -> None:
    if self.artifacts is not None:
        raise ValueError('cannot modify after build() is called')

def set_event_manager(self, event_manager: EventManager) -> 'Builder':
    self.check_if_can_modify()
    self._event_manager = event_manager
    return self
```

That guard enforces an invariant: once you call `build()`, the builder freezes. You cannot retroactively swap a dependency on a node that already exists. (How does it know it has built? `build()` sets `self.artifacts`; the guard checks for it.)

The lazy, memoized getters. This is where Hathor's `Builder` departs from the sandwich toy. Instead of computing defaults inside `build()`, each component has a private `_get_or_create_*` method that returns the slot if it is set, otherwise constructs a default and caches it in the slot:

```
# hathor/builder/builder.py:444-447
def _get_or_create_pubsub(self) -> PubSubManager:
    if self._pubsub is None:
        self._pubsub = PubSubManager(self._get_reactor())
    return self._pubsub
```

Why cache? Because these objects are shared. The pub-sub bus that consensus receives must be the **same** bus the event manager listens on. If each getter built a fresh object on every call, consensus and the event manager would end up talking past each other on two different buses. Caching in the slot guarantees a single instance, no matter how many components ask for it. The pattern is create-once, then return-the-same-one. (This is the runtime cousin of the Singleton idea from Chapter 3, but scoped to one builder rather than the whole program — much safer.)

You can see the sharing in `_get_or_create_p2p_manager`, which pulls several already-memoized collaborators (`builder.py:463 - 481`):

```
# hathor/builder/builder.py:471-481 (abridged)
self._p2p_manager = ConnectionsManager(
    settings=self._get_or_create_settings(),
    reactor=reactor,
    my_peer=my_peer,
    pubsub=self._get_or_create_pubsub(),  # ← the one shared bus
    ssl=enable_ssl,
    whitelist_only=False,
    rng=self._rng,
    ...
)
```

The `pubsub` here is the very same object every other getter receives. That is dependency injection (Ch. 3) made concrete.

The backend choice. The memory-versus-RocksDB decision (Ch. 27) is made by which getter path runs. In `Builder`, the tx storage getter always builds a RocksDB-backed store, but the RocksDB storage itself can be a temporary directory when no path was set — which is how the simulator and tests get a throwaway store (`builder.py:455 - 461`):

```
# hathor/builder/builder.py:455-461
def _get_or_create_rocksdb_storage(self) -> RocksDBStorage:
    if self._rocksdb_storage is None:
        self._rocksdb_storage = RocksDBStorage(
            path=self._rocksdb_path,
            cache_capacity=self._rocksdb_cache_capacity,
        ) if self._rocksdb_path else RocksDBStorage.create_temp(self._rocksdb_cache_capacity)
    return self._rocksdb_storage
```

The rest of the node sees only the abstract `TransactionStorage` type (`builder.py:500`) and is none the wiser about where the bytes land.

build() — the orchestration. `build()` (`builder.py:203`) is the terminal step. It first asserts you have not built twice and that a sync version is enabled (`builder.py:204 – 208`), then calls the getters in dependency order and hands every result to the `HathorManager` constructor:

```
# hathor/builder/builder.py:210–235 (abridged)
settings = self._get_or_create_settings()
reactor = self._get_reactor()
pubsub = self._get_or_create_pubsub()
peer = self._get_peer()
consensus_algorithm = self._get_or_create_consensus()
p2p_manager = self._get_or_create_p2p_manager()
wallet = self._get_or_create_wallet()
event_manager = self._get_or_create_event_manager()
indexes = self._get_or_create_indexes_manager()
tx_storage = self._get_or_create_tx_storage()
feature_service = self._get_or_create_feature_service()
verification_service = self._get_or_create_verification_service()
daa = self._get_or_create_daa()
# ... and several more
```

The single line where the facade is born and every dependency is injected is the `HathorManager(...)` call (`builder.py:257`):

```
# hathor/builder/builder.py:257–283 (abridged)
manager = HathorManager(
    reactor,
    settings=settings,
    pubsub=pubsub,
    consensus_algorithm=consensus_algorithm,
    daa=daa,
    peer=peer,
    tx_storage=tx_storage,
    p2p_manager=p2p_manager,
    event_manager=event_manager,
    wallet=wallet,
    bit_signaling_service=bit_signaling_service,
    verification_service=verification_service,
    cpu_mining_service=cpu_mining_service,
    feature_service=feature_service,
    # ... ~20 collaborators in total
)
```

BuildArtifacts — handing back the parts, not only the whole. `build()` does not return the bare manager. It returns a `BuildArtifacts`, a `NamedTuple` (`builder.py:93`) that bundles the manager together with its collaborators:

```
# hathor/builder/builder.py:93-109 (abridged)
class BuildArtifacts(NamedTuple):
    """Artifacts created by a builder."""
    peer: PrivatePeer
    settings: HathorSettingsType
    reactor: Reactor
    manager: HathorManager
    p2p_manager: ConnectionsManager
    pubsub: PubSubManager
    consensus: ConsensusAlgorithm
    tx_storage: TransactionStorage
    indexes: IndexesManager
    wallet: Optional[BaseWallet]
    # ...
```

Why hand back the parts? Tests and the simulator routinely need to reach past the facade — to inspect the storage directly, push a transaction onto the pub-sub bus, or assert on the indexes. `BuildArtifacts` gives them a handle to each piece without forcing them to re-derive it from the manager. The `build()` method stores the bundle in `self.artifacts` and returns it (`builder.py:293 – 311`); storing it is also what arms the `check_if_can_modify` freeze.

24.4.3 The real `CliBuilder` — production

`CliBuilder` lives at `hathor_cli/builder.py:60`. It is the path the production command line drives. It does the same job as `Builder` — produce a wired `HathorManager` — but its shape is different. There are no fluent `set_*` setters and no `BuildArtifacts` bundle. Instead, one large method, `create_manager(reactor)`, reads everything it needs from the parsed CLI arguments (`self._args`) and constructs the node top to bottom.

The class opens with an honest admission (`hathor_cli/builder.py:60 – 64`):

```
# hathor_cli/builder.py:60-64
class CliBuilder:
    """CliBuilder builds the core objects from args.

    TODO Refactor to use Builder. It could even be ported to a Builder.from_args classmethod.
    """
```

`create_manager` (`hathor_cli/builder.py:79`) is a long, linear procedure. Walked in order, it builds:

1. **Settings** — `get_global_settings()` (`hathor_cli/builder.py:93`). The settings object (Ch. 22) is read once and threaded into nearly every component below.
2. **Peer identity** — `PrivatePeer.create_from_json_path(...)` if a peer file was given, else `PrivatePeer.auto_generated()` (`hathor_cli/builder.py:102 – 105`). This is the node's cryptographic identity on the P2P network (Ch. 34).
3. **Transaction storage and indexes** — production is **always RocksDB**. The old `--memory-storage` and `--memory-indexes` flags are now deprecated and

rejected (hathor_cli/builder.py:127 – 128); the builder then opens the RocksDB store and an index manager over it (builder.py:130 – 160):

```
# hathor_cli/builder.py:127–158 (abridged)
memory_msg = 'is deprecated. use --temp-data instead'
self.check_or_raise(not self._args.memory_storage, f'--memory-storage {memory_msg}')
self.check_or_raise(not self._args.memory_indexes, f'--memory-indexes {memory_msg}')

self.check_or_raise(bool(self._args.data) or self._args.temp_data,
                    'either --data or --temp-data is expected')
self.rocksdb_storage = (
    RocksDBStorage(path=self._args.data, cache_capacity=self._args.rocksdb_cache)
    if self._args.data else RocksDBStorage.create_temp(self._args.rocksdb_cache)
)
indexes = RocksDBIndexesManager(self.rocksdb_storage, settings=settings)
tx_storage = TransactionRocksDBStorage(
    reactor=reactor, rocksdb_storage=self.rocksdb_storage,
    settings=settings, indexes=indexes, ...)
```

A throwaway store is still possible — `--temp-data` routes to `RocksDBStorage.create_temp(...)` (builder.py:134) — but it is still RocksDB, just in a temporary directory. `check_or_raise` (hathor_cli/builder.py:69) is `CliBuilder`'s way of rejecting a misconfigured launch with a clean message (a `BuilderError`) rather than a raw traceback.

4. **Event storage, feature storage** — `EventRocksDBStorage(...)` and `FeatureActivationStorage(...)` (hathor_cli/builder.py:161 – 162).

5. **Wallet and hostname** — `self.create_wallet()` if a wallet was configured (hathor_cli/builder.py:181 – 182) and `self.get_hostname()` (:185).

6. **Pub-sub bus** — `PubSubManager(reactor)` (hathor_cli/builder.py:195). Created once, injected everywhere below.

7. **Event manager** — `EventManager(...)` (hathor_cli/builder.py:210), given the event storage, the optional WebSocket factory, the reactor, and the same pub-sub bus.

8. **Feature service** — `FeatureService(...)` (hathor_cli/builder.py:246), then later the **bit-signaling service** `BitSignalingService(...)` (:265). Together these drive feature activation, the node's on-chain upgrade mechanism (Ch. 38).

9. **Consensus** — `ConsensusAlgorithm(...)` (hathor_cli/builder.py:249), given the soft-voided tx ids, settings, and feature service (Ch. 32).

10. **Difficulty algorithm (DAA)** — `DifficultyAdjustmentAlgorithm(...)` (hathor_cli/builder.py:280), Ch. 32.

11. **Vertex verifiers and verification service** — `VertexVerifiers.create_defaults(...)` then `VerificationService(...)` (hathor_cli/builder.py:282 – 294). These enforce every protocol rule on incoming vertices (Ch. 31).

12. **CPU mining service** — `CpuMiningService()` (hathor_cli/builder.py:296).

13. **P2P manager** — `ConnectionsManager(...)` (hathor_cli/builder.py:298), given the settings, reactor, peer, and the same pub-sub bus:

```
# hathor_cli/builder.py:298–308 (abridged)
p2p_manager = ConnectionsManager(
    settings=settings,
    reactor=reactor,
    my_peer=peer,
    pubsub=pubsub,          # ← the one shared bus, again
    ssl=True,
    whitelist_only=False,
    rng=Random(),
    ...
)
```

1. **Vertex handler** — `VertexHandler(...)` (`hathor_cli/builder.py:310`), the ingestion pipeline that ties verification, consensus, and execution together (Ch. 33).
2. **The manager** — `HathorManager(...)` (`hathor_cli/builder.py:338`), the facade, receiving every object built above:

```
# hathor_cli/builder.py:338–363 (abridged)
self.manager = HathorManager(
    reactor,
    settings=settings,
    hostname=hostname,
    pubsub=pubsub,
    consensus_algorithm=consensus_algorithm,
    daa=daa,
    peer=peer,
    tx_storage=tx_storage,
    p2p_manager=p2p_manager,
    event_manager=event_manager,
    wallet=self.wallet,
    checkpoints=settings.CHECKPOINTS,
    bit_signaling_service=bit_signaling_service,
    verification_service=verification_service,
    cpu_mining_service=cpu_mining_service,
    vertex_handler=vertex_handler,
    feature_service=self.feature_service,
    # ... ~20 collaborators in total
)
```

16. Post-construction wiring, then return. After the manager exists, the builder wires it back into its collaborators (`p2p_manager.set_manager(self.manager)`, `hathor_cli/builder.py:370`), optionally attaches a Stratum mining factory (`:374 – 377`), registers peer discovery and `--listen` addresses (`:388 – 403`), and finally returns the bare manager (`hathor_cli/builder.py:409`) — no `BuildArtifacts` bundle. Production code does not need to reach past the facade the way tests do.

Compare the two `HathorManager(...)` calls — `builder.py:257` and `hathor_cli/builder.py:338`. They pass nearly the same arguments in nearly the same shape. That near-duplication is the whole point of the next section.

24.4.4 Two composition roots, one job — and the debt

The codebase has **two** composition roots that build the same product:

	Builder	CliBuilder
File	<code>hathor/builder/builder.py</code>	<code>hathor_cli/builder.py</code>
Configured by	fluent <code>set_*</code> calls	parsed CLI args (<code>self._args</code>)
Returns	<code>BuildArtifacts</code> (manager + parts)	the bare <code>HathorManager</code>
Internal style	lazy, memoized <code>_get_or_create_*</code>	one linear <code>create_manager</code>
Used by	tests, simulator	<code>run_node</code> (production)

Why two? Historically, the test/simulator `Builder` grew up as a clean, composable API; the production path accreted its own construction code, tied to argument parsing, in `CliBuilder`. They were never unified. The `TODO` at `hathor_cli/builder.py:63` is the project acknowledging it in its own words: "Refactor to use Builder. It could even be ported to a `Builder.from_args` classmethod." — i.e. `CliBuilder` should eventually delegate to `Builder` instead of duplicating its wiring.

The cost is real and worth stating plainly. The same wiring — which collaborator goes into which constructor — is written twice. When consensus gains a new dependency, both `builder.py:257` and `hathor_cli/builder.py:338` must change in lockstep. Miss one and the two paths build different nodes: tests pass against a node that production never actually runs. This is precisely the drift a single composition root exists to prevent — and here the project has two of them. The duplication in the two `HathorManager(...)` calls above is the visible symptom.

So, a practical rule for a junior contributor: **when you change how a component is constructed, search both files**. Treat `CliBuilder` as authoritative for what production runs, and `Builder` as authoritative for what tests and the simulator run. They are supposed to agree, but the language does not enforce it — only discipline does.

24.4.5 ResourcesBuilder, briefly

The third export in `hathor/builder/__init__.py` (`__init__.py:16`), `ResourcesBuilder` (`hathor/builder/resources_builder.py:66`), is not a composition root for the node. It is a smaller builder that takes an already-built manager plus a couple of its pieces (the wallet, the event WebSocket factory, the feature service) — see its `__init__` (`resources_builder.py:67 – 83`) — and assembles the node's HTTP/REST resource tree,

the API surface the node exposes over the wire (`build()` at `resources_builder.py:85`, `create_resources()` at `:114`). It runs after the manager is built. It is mentioned here only so you know what the fourth name in the builder package is; its detail belongs with the API chapters.

24.5 How it plugs into the lifecycle

This chapter is step 5 of Act I's boot sequence (Ch. 0 §0.3). It sits between two neighbors:

- **Before it:** the reactor is selected and readied (Ch. 23), the settings are loaded (Ch. 22), and the CLI has parsed its arguments (Ch. 21). The builder consumes all three: it takes the reactor as the argument to `create_manager`, reads the settings via `get_global_settings()`, and reads the CLI flags from `self._args`.
- **After it:** the manager goes live (Ch. 29). The builder returns a fully wired but not-yet-started `HathorManager`. Nothing is listening, no peers are connected, no consensus is running. That happens only when the caller calls `manager.start()`.

You can see the hand-off in the production run-node body, inside `RunNode.prepare` (`hathor_cli/run_node.py:207 – 216`):

```
# hathor_cli/run_node.py:205–216 (abridged)
from hathor.builder import ResourcesBuilder
from hathor.exception import BuilderError
from hathor_cli.builder import CliBuilder
builder = CliBuilder(self._args)
try:
    self.manager = builder.create_manager(reactor)  # ← the builder phase
except BuilderError as err:
    self.log.error(str(err))
    sys.exit(2)

self.tx_storage = self.manager.tx_storage
self.wallet = self.manager.wallet
```

`RunNode` constructs the `CliBuilder` with the parsed args, asks it for a manager, and then pulls a couple of references off the manager (the storage, the wallet) for its own convenience (`hathor_cli/run_node.py:215 – 216`). Note the `try/except`: a `BuilderError` raised by `check_or_raise` becomes a clean `sys.exit(2)` with a logged message, not a crash dump — that is the whole reason `check_or_raise` exists. The later call to `manager.start()` — the moment the node actually goes live — is the subject of Chapter 29.

Recap

Concern	Where	Central type
What a composition root is	this chapter, §24.2	— (a role, not a class)
Build a node for tests/simulator	<code>hathor/builder/builder.py:118</code>	<code>Builder</code>
Bundle the manager with its parts	<code>hathor/builder/builder.py:93</code>	<code>BuildArtifacts</code>
Freeze the builder after build	<code>hathor/builder/builder.py:313</code>	<code>check_if_can_modify</code>
Build a node for production	<code>hathor_cli/builder.py:79</code>	<code>CliBuilder.create_manager</code>
Reject a bad launch cleanly	<code>hathor_cli/builder.py:69</code>	<code>check_or_raise</code>
Drive the production boot	<code>hathor_cli/run_node.py:208</code>	<code>RunNode.prepare</code>
Build the HTTP/REST API tree	<code>hathor/builder/resources_builder.py:66</code>	<code>ResourcesBuilder</code>
The duplication debt	<code>hathor_cli/builder.py:63</code>	the <code>TODO</code>

A composition root is the one place an application builds its object graph and injects dependencies. `hathor-core` has two of them: `Builder` (fluent setters, memoized getters, returns a `BuildArtifacts` bundle) for tests and the simulator, and `CliBuilder` (one linear `create_manager` reading CLI args, returns the bare manager) for production. Both converge on a near-identical `HathorManager(...)` call that injects every collaborator — and that near-duplication is honest technical debt the code's own `TODO` admits. The builder phase consumes the reactor (Ch. 23), the settings (Ch. 22), and the CLI args (Ch. 21), and produces a wired-but-not-started manager that Chapter 29 will bring to life.

The next chapter steps into the data the node spends its life managing: the **vertex model** (Ch. 25) — transactions and blocks as nodes in a DAG. Once you know what a vertex is, the storage, verification, and consensus services this chapter wired together will finally have something concrete to act on.

Footnotes

1. A **composition root** is the single place in an application where the object graph is assembled — where you call the constructors and decide who depends on whom. The term comes from the dependency-injection literature. The rule of thumb: construct objects in one place, as close to the program's entry point as you can, and pass dependencies inward. Keep the rest of the code free of constructor calls for its own collaborators. ↩

Chapter 25 — The Vertex Model in Code

What you'll learn in this chapter

- How the abstract ideas of Chapters 7 and 8 — the UTXO model and the DAG of vertices — become concrete Python classes in `hathor/transaction/`.
- The shape of the **GenericVertex** base class: the fields every block and every transaction share, what each field models in the real world, and why the base is generic and abstract.
- The two concrete subclasses — **Block** and **Transaction** — and the further specializations built on them (`MergeMinedBlock`, `PoaBlock`, `TokenCreationTransaction`, `OnChainBlueprint`).
- The **TxInput** and **TxOutput** classes in code, mapped one-to-one onto the UTXO concepts of Chapter 7, including how a single byte encodes which token an output holds and whether it is an authority.
- The crucial **parents-versus-inputs** distinction (Chapter 8) seen directly in the fields, and how the code keeps the two edge sets separate.
- The **metadata split** — immutable static metadata versus the mutable `TransactionMetadata` that consensus rewrites — and why a node needs both.
- Where **validation state** and storage **scope** fit, and how a vertex travels through the node's lifecycle.

Two earlier chapters built the theory this chapter pays off. Chapter 7 taught the **UTXO model**: money as a heap of discrete, individually-locked outputs, where a transaction consumes some and creates others. Chapter 8 taught the **DAG**: a ledger arranged as a directed acyclic graph of vertices, where each vertex has two distinct kinds of outgoing edge — **parents** (the confirmation/topology edge) and **inputs** (the spending edge). This chapter does not re-teach either; it shows you the real classes that are those ideas. By the end you should be able to open `base_transaction.py` and read it as fluently as the diagrams in Chapters 7 and 8.

Everything here lives in one package, `hathor/transaction/`, and overwhelmingly in one file, `base_transaction.py` (about 1150 lines). We start by locating the package, then state what it is for, recap the concepts it rests on, and finally walk the code in detail.

25.1 Localization

The package sits in the **domain-model** group of the module map (Chapter 0, §0.4). It is foundational: storage, verification, consensus, indexes, p2p, and the wallet all import these classes. Nothing in the node manipulates a block or a transaction without going through the types defined here.

```
hathor/
├─ transaction/
│   │ base_transaction.py    ◀ YOU ARE HERE – GenericVertex, TxInput, TxOutput, TxVersion
│   │ block.py              ◀ Block (subclass of GenericVertex[BlockStaticMetadata])
│   │ transaction.py        ◀ Transaction (subclass of GenericVertex[TransactionStaticMetadata])
│   │ merge_mined_block.py  ◀ MergeMinedBlock (subclass of Block)
│   │ token_creation_tx.py  ◀ TokenCreationTransaction (subclass of Transaction)
│   │ static_metadata.py    ◀ VertexStaticMetadata / BlockStaticMetadata / TransactionStaticMetadata
│   │ transaction_metadata.py ◀ TransactionMetadata (the mutable, computed state)
│   │ validation_state.py   ◀ ValidationState enum (INITIAL → BASIC → FULL ...)
│   │ genesis.py            ◀ is_genesis(), the graph's root
│   │ types.py              ◀ TokenInfo for token accounting
│   │ aux_pow.py            ◀ Bitcoin auxiliary proof-of-work (for merge mining)
├─ scripts/                ◀ the locking/unlocking script language (P2PKH, opcodes) → Ch 28
├─ headers/                ◀ optional vertex headers (nano-contract, fee)
├─ poa/                    ◀ PoaBlock (proof-of-authority block) → Ch 32
├─ storage/                ◀ vertex-aware storage + graph-walk utilities → Ch 27
```

Context. This package is the node's **data model** — the in-memory and on-disk shape of every block and transaction in the ledger. It defines what a vertex is (its fields), what kinds of vertex exist (the class hierarchy), and what the node remembers about each one (the two metadata objects). It does **not** decide whether a vertex is valid — that is verification, Chapter 31 — nor which vertices win a conflict — that is consensus, Chapter 32. It is the vocabulary those chapters speak.

25.2 What it does and why it exists

A full node's entire job is to hold a ledger and reason about it. Before it can store a transaction, verify it, or sync it to a peer, it needs an answer to a flat-footed question: what is a transaction, as a thing in memory? This package answers that. It gives the node:

1. **A representation for every ledger object.** A class whose instances hold the data a block or transaction is made of — its outputs, its inputs (if any), the vertices it confirms, its proof-of-work nonce and weight.
2. **One umbrella type plus specific kinds.** A common base (`GenericVertex`) so the rest of the node can pass around "a vertex" without caring whether it holds a block or a transaction, and concrete subclasses (`Block` , `Transaction` , and four more) for the cases where the kind does matter.

3. **A precise map onto the domain models.** The `TxInput` / `TxOutput` classes are the UTXO model of Chapter 7 made literal; the `parents` field is the DAG edge of Chapter 8 made literal.
4. **A place to keep what the node computes about each vertex.** A vertex's own contents are fixed once it is created (you cannot un-sign a transaction). But the node learns facts about a vertex over time — how much accumulated work stands behind it, whether it has been voided, which block first confirmed it. That derived knowledge lives in **metadata** objects attached to the vertex, kept separate from the vertex's own immutable data. The separation is the most conceptually interesting thing in the package, and §25.6 is devoted to it.

Why does this deserve its own large package rather than, say, a couple of dataclasses? Because a vertex is not a passive bag of fields. It knows how to hash itself (the proof-of-work identity), serialize itself to the wire format, compute its dependencies, and answer questions like "what addresses are involved here?" The class gathers the data and the operations that belong with it — the object-oriented principle of Chapter 1 (encapsulation: keep the data and the code that maintains its invariants together).

25.3 The concepts it rests on

Four ideas from earlier chapters meet here. Each gets a recap box; the full treatment is in the cited chapter.

Recap — vertex, DAG, parents (full treatment in Ch. 8). Hathor's ledger is one directed acyclic graph. Its nodes are called **vertices**; a vertex is either a **block** or a **transaction**. Each vertex names a short list of **parents** — earlier vertices it attaches to and thereby confirms. Parents are the structural (topology) edge; they are the generalization of a classic blockchain's single "previous block" pointer. A block has three parents (one block-parent continuing the block backbone, plus two transaction-parents); a transaction typically has two. → Ch 8.

Recap — UTXO, inputs, outputs, scripts (full treatment in Ch. 7). Money is recorded as a set of discrete **unspent transaction outputs** (UTXOs), each holding a value and a locking script that says who may spend it. A transaction **consumes** existing outputs (its **inputs**, each a pointer `(tx_id, index)` to a prior output) and **creates** new ones. There is no stored balance and no owner field; ownership is "can you satisfy the locking script?" Inputs and outputs are the spending edge — entirely separate from parents. → Ch 7.

Recap — inheritance, abstract base classes, generics (full treatment in Ch. 1 and Ch. 5). A **subclass** inherits fields and methods from a base class and may add or override its own. An **abstract base class** (ABC) defines a shared interface but cannot itself be instantiated; it marks some methods `@abstractmethod`, forcing every concrete subclass to supply them. A **generic** class is parameterized by a type variable (written `Class[T]`), so the same class definition can be specialized to different companion types while keeping them straight for the type-checker. `GenericVertex[StaticMetadataT]` uses all three at once. → Ch 1, Ch 5.

Recap — encapsulated invariants (full treatment in Ch. 1). A well-designed class does not merely store fields; it guards rules about them ("a vertex's hash, once set, must match its contents", "outputs may only be added, never mutated"). The class keeps the data and the code that maintains those rules together, so no outside code can put an object into an inconsistent state. Watch for these invariants as we walk the code; they are the why behind otherwise puzzling design choices. → Ch 1.

25.4 A hand-traced example first

Before the real classes, build the picture with plain objects. Suppose Alice holds two unspent outputs — a 50-HTR output and a 20-HTR output, both created by some earlier transaction `X` — and wants to pay Bob 30 HTR. From Chapter 7 we know this is one transaction that consumes both inputs and creates two outputs (one to Bob, one of change back to Alice). From Chapter 8 we know the same transaction must also name **parents** to attach itself to the DAG.

Written as Python-shaped pseudocode (not yet the real API), the transaction is two independent lists plus a parents list:

```
tx = Transaction(
    # --- the SPENDING edge (UTXO model, Ch 7) ---
    inputs=[
        TxInput(tx_id=X_hash, index=0, data=<Alice's signature + pubkey>), # the 50-HTR output
        TxInput(tx_id=X_hash, index=1, data=<Alice's signature + pubkey>), # the 20-HTR output
    ],
    outputs=[
        TxOutput(value=30, script=<lock to Bob>), # Bob's coin
        TxOutput(value=40, script=<lock to Alice>), # Alice's change
    ],
    # --- the TOPOLOGY edge (DAG, Ch 8) — UNRELATED to the above ---
    parents=[parent_hash_1, parent_hash_2], # two tips this tx confirms
)
```

Three things to notice, because each becomes a real design point:

- **inputs** and **parents** are different lists with different meanings. The inputs say "I spend these coins"; the parents say "I confirm these vertices." A

tx Alice spends from (X) need not be among her parents, and her parents need not be txs she spends from. Conflating them is the classic beginner error Chapter 8 warned about.

- **Conservation is not enforced by the constructor.** Inputs sum to 70, outputs to 70 — but the class does not check that. Constructing the object and validating it are separate steps (validation is Chapter 31). The class's job is to hold the data faithfully; the rules live elsewhere. (This is itself a design decision we will see honored throughout.)
- **There is no from field and no balance.** Ownership of the consumed coins is proved by the `data` (unlocking script) inside each input, exactly as Chapter 7 described.

Now the real classes.

25.5 The code, walked

25.5.1 `GenericVertex` : the abstract generic base

The umbrella type is `GenericVertex`, declared at `base_transaction.py:148`:

```
StaticMetadataT = TypeVar('StaticMetadataT', bound=VertexStaticMetadata, covariant=True)

class GenericVertex(ABC, Generic[StaticMetadataT]):
    """Hathor generic vertex"""
```

Three things are happening in that one line, each from a recap box above:

- **ABC** — it is an abstract base class. You cannot write `GenericVertex(...)`; you must instantiate a concrete subclass (`Block` or `Transaction`). The base marks several methods `@abstractmethod` — for example `is_block` (`base_transaction.py:255`), `is_transaction` (`:260`), `get_funds_struct` (`:438`), and `get_token_uid` (`:895`) — so every concrete vertex must answer them. The base provides everything common; the subclasses fill in what differs.
- **`Generic[StaticMetadataT]`** — it is generic over a type variable. The companion "static metadata" type differs per vertex kind (a block's static metadata records its height; a transaction's does not). Rather than have one untyped metadata field, the base is parameterized so that `Block` is `GenericVertex[BlockStaticMetadata]` and `Transaction` is `GenericVertex[TransactionStaticMetadata]`. The type-checker then knows that `some_block.static_metadata.height` is valid but `some_tx.static_metadata.height` is not. (`bound=VertexStaticMetadata` constrains the variable: whatever you plug in must be a subclass of `VertexStaticMetadata`. This is the generics machinery of Chapter 5.)

- `StaticMetadataT` is the name of that type variable. We meet the static-metadata classes in §25.6.

Aside — why "GenericVertex" and not just "Vertex"? The names `Vertex` and `BaseTransaction` exist, but as aliases, defined just after the class (`base_transaction.py:932`):

```
> Vertex: TypeAlias = GenericVertex[VertexStaticMetadata]
> BaseTransaction: TypeAlias = Vertex
>
```

`Vertex` is `GenericVertex` with its type parameter pinned to the common base metadata — the type you use when you hold "some vertex, kind unknown."
`BaseTransaction` is kept only for backwards compatibility; the docstring at `:927` says it "can be removed in the future." So when you see `BaseTransaction` in older code, read it as "any vertex." Throughout the rest of the codebase, function signatures say `Vertex` or `BaseTransaction` and mean exactly the same thing.

25.5.2 The shared fields, and what each one models

Every vertex is built by the base `__init__` (`base_transaction.py:170`). The fields are listed in `__slots__` at `:151`; here is what each one models in the world and why it must exist. (`__slots__` is a memory optimization: it tells Python the exact, fixed set of attributes an instance may have, so each object can skip its per-instance dictionary. With millions of vertices in memory this matters; it also doubles as a guard against typo-ing a new attribute into existence.)

Field	Set at	Models	Why it must exist
<code>version</code>	<code>:200</code>	which kind of vertex this is (a <code>TxVersion</code> enum)	the wire format and the dispatcher use it to pick the right class to parse the bytes (<code>TxVersion.get_cls</code> , <code>:115</code>)
<code>signal_bits</code>	<code>:199</code>	a byte of extra bits a block uses for feature-activation miner signalling (Ch 38)	lets a block carry "I vote for upgrade X" without a new field; ignored by transactions
<code>weight</code>	<code>:201</code>	the proof-of-work effort of this vertex, as <code>log2(work)</code> (Ch 9)	the unit consensus sums into accumulated weight; anti-spam for transactions

Field	Set at	Models	Why it must exist
timestamp	:198	the Unix time the vertex was created	ordering, tie-breaking conflicts (Ch 10), and validity windows
nonce	:197	the number a miner varied to make the hash meet the target (Ch 9)	the proof-of-work solution; part of what is hashed
inputs	:202	the UTXOs this vertex spends — a <code>list[TxInput]</code> (Ch 7)	the spending edge; empty for blocks
outputs	:203	the new coins this vertex creates — a <code>list[TxOutput]</code> (Ch 7)	where value (and block reward) comes from
parents	:204	the vertices this one confirms — a <code>list[VertexId]</code> (Ch 8)	the DAG topology edge; the graph itself
_hash	:206	this vertex's identity: the double-SHA256 of its contents	the key it is stored under; what parents and inputs point at
storage	:205	a back-reference to the <code>TransactionStorage</code> (or <code>None</code>)	lets a vertex look up its parents and spent outputs on demand (Ch 27)
_metadata	(lazy)	the mutable, computed <code>TransactionMetadata</code> (§25.6)	the node's evolving knowledge about the vertex
_static_metadata	:207	the immutable computed <code>VertexStaticMetadata</code> (§25.6)	derived facts that never change (e.g. a block's height)
headers	:209	optional trailing sections (nano-contract header, fee header)	extensibility without changing the core format (Ch 39)
name	:212	a debug-only label	convenience in tests and logs; never serialized

A few of these reward a closer look.

`_hash` and the identity invariant. A vertex is identified by its hash, and the hash is computed from its contents. This is the central invariant of the class: **the stored `_hash` must equal `calculate_hash()` of the current fields**. The hash is the double-SHA256 of the serialized vertex (`calculate_hash` , :628 , built from `calculate_hash1` / `calculate_hash2` at :603 / :613 , which fold in the nonce at :624). Because the hash depends on every field, changing any field would change the identity — which is exactly why the outputs and inputs must be treated as immutable once the vertex exists. The leading underscore on `_hash` and the property guard at :340 (which asserts the hash has been set before anyone reads it) enforce that you never hand out an uninitialized identity.

`parents` is a list of `VertexId` , not of vertices. `VertexId` is just an alias for `bytes` (`types.py:26`) — a 32-byte hash. The vertex stores the hashes of its parents, not the parent objects. To get the actual parent objects you call `get_parents()` (:382), which asks the storage to fetch each one by hash. This keeps a vertex small and avoids loading the whole graph into memory at once; it is why `storage` is a field. The same is true of `TxInput.tx_id` — inputs point at prior transactions by hash, not by reference.

`inputs` is empty for blocks. Blocks mint coins; they do not spend them (Chapter 7, §7.3). The `Block.__init__` (`block.py:48`) does not even accept an `inputs` argument — it calls `super().__init__` without one, leaving the list empty. A transaction's `__init__` (`transaction.py:60`) accepts inputs and, additionally, a `tokens` list (next section).

25.5.3 `TxInput` and `TxOutput` : the UTXO model, literally

These two small classes are Chapter 7 turned into code. `TxInput` (`base_transaction.py:936`) is the spending pointer:

```
class TxInput:
    def __init__(self, tx_id: VertexId, index: int, data: bytes) -> None:
        # tx_id: hash of the transaction that contains the output of this input
        # index: index of the output you are spending from transaction tx_id
        # data: data to solve output script (the unlocking script)
        self.tx_id = tx_id
        self.index = index
        self.data = data
```

The `(tx_id, index)` pair is the "pointer to a specific earlier output" from §7.3: "spend output number `index` of transaction `tx_id`." The `data` field is the **unlocking script** of §7.5 — the bytes (typically a signature and public key) that satisfy the spent output's locking script. Note `get_sighash_bytes` (:971): when a transaction is hashed for signing, each input's `data` is cleared to zero-length, because you cannot sign over the very signature you are about to produce. That is a subtle correctness detail the class hides from its callers.

`TxOutput` (`base_transaction.py:1022`) is a coin: a value plus a lock plus a token tag.

```

class TxOutput:
    TOKEN_INDEX_MASK      = 0b01111111  # low 7 bits: which token
    TOKEN_AUTHORITY_MASK = 0b10000000  # high bit:  is this an authority output?

    def __init__(self, value: int, script: TxOutputScript, token_data: int = 0) -> None:
        if value <= 0 or value > MAX_OUTPUT_VALUE:
            raise InvalidOutputValue
        self.value = value          # amount (or, for authorities, an authority bitmask)
        self.script = script        # the locking script (Ch 7 §7.5)
        self.token_data = token_data

```

`value` and `script` are exactly the "value" and "locking script" of Chapter 7. The constructor does enforce one invariant — `value` must be positive and within range (: 1044) — but, as promised in §25.4, it does **not** check conservation against any inputs; that is verification's job.

The interesting field is `token_data`, a single byte that does two jobs through bit-masking. Recall from Chapter 7 that Hathor is multi-token: one output might hold HTR, another a custom token. Rather than spend a whole separate field, the byte packs both "which token" and "is this special" together:

- **The low 7 bits — which token.** `get_token_index()` (: 1099) returns `token_data & TOKEN_INDEX_MASK`. Index 0 always means HTR, the native token (whose UID is the single byte `b'\x00'`, `conf/settings.py:29`); higher indices point into the transaction's `tokens` list. This is why a transaction carries a `tokens` list and a block does not — a block only ever holds HTR (`Block.get_token_uid` asserts `index == 0`, `block.py:256`).
- **The high bit — authority or value.** `is_token_authority()` (: 1103) returns true when `token_data & TOKEN_AUTHORITY_MASK` is set. An **authority output** carries no spendable money; instead its `value` field is reinterpreted as a permissions bitmask — bit 0 is the right to **mint** more of the token, bit 1 the right to **melt** (destroy) it (`TOKEN_MINT_MASK / TOKEN_MELT_MASK`, : 1029 – : 1031 ; checked by `can_mint_token / can_melt_token`, : 1115 / : 1119). This is the "a coin that grants a power rather than holding a value" idea from §7.6, made concrete: the same `TxOutput` class expresses both money and authority, distinguished by one bit.

A hand-trace of the byte: `token_data = 0b10000001` means "high bit set → authority output" and "low bits 0000001 → token index 1." Combined with a `value` of `0b01` (`TOKEN_MINT_MASK`), it is "a mint-authority output for the transaction's first custom token." The bookkeeping that consumes these — checking that an output's mint authority was also present on some input, so authorities cannot be conjured — lives in `Transaction._update_token_info_from_outputs` (`transaction.py:432`), pointing forward to verification (Ch 31).

25.5.4 Parents versus inputs, in the fields

Chapter 8 §8.4 called the parents/inputs distinction "the single most consequential distinction in the book." Here it is, sitting in the class as two separate fields with two separate types:

```
self.inputs = inputs or [] # list[TxInput] - the SPENDING edge (Ch 7)
self.parents = parents or [] # list[VertexId] - the TOPOLOGY edge (Ch 8)
```

The code keeps them rigorously apart, and two methods make the separation visible:

- `get_all_dependencies()` (`base_transaction.py:487`) unions both edge sets:

```
return set(chain(self.parents, (i.tx_id for i in self.inputs)))
```

It exists because to process a vertex you need everything it points at, either way — you cannot verify a transaction without both its parents and the transactions it spends from. But notice it builds one set only to walk dependencies; it never merges the roles.

- `get_tx_parents_ids()` (`:491`) deals with the block's asymmetry:

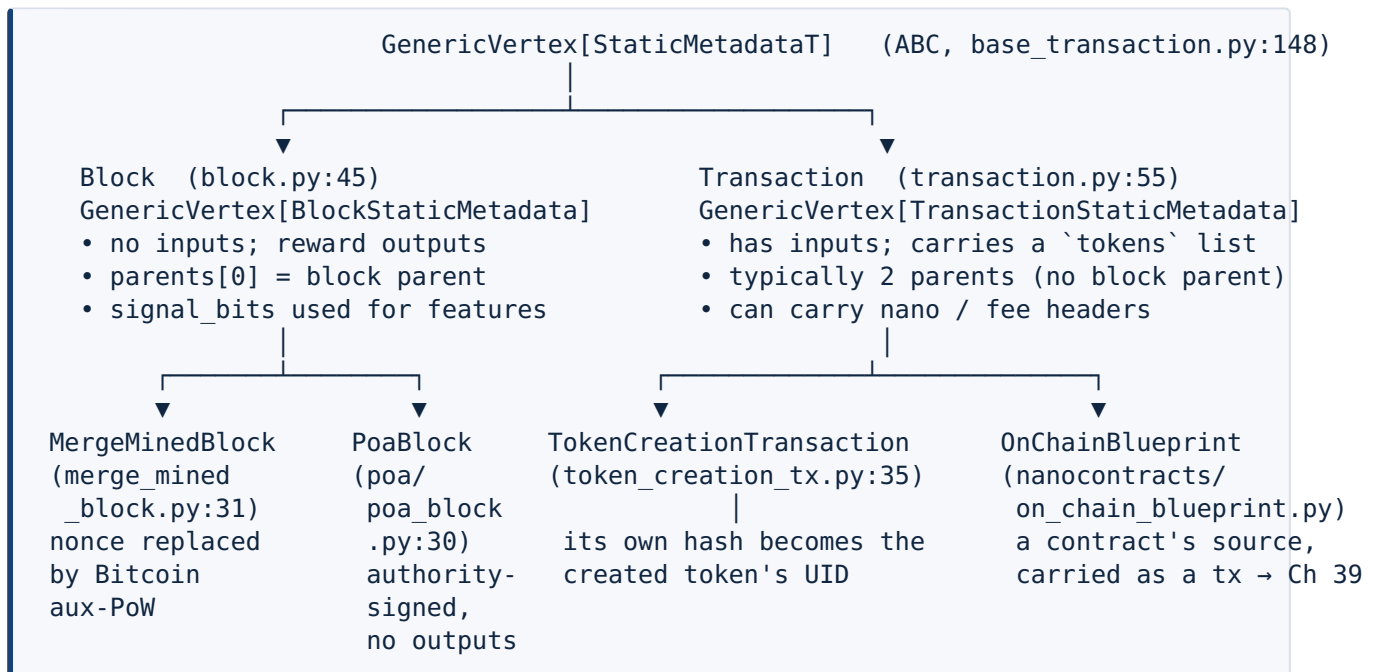
```
return set(self.parents[1:] if self.is_block else self.parents)
```

For a block, `parents[0]` is the block-parent (the block-backbone edge); the rest are transaction-parents. So this method drops `parents[0]` for blocks and keeps all parents for transactions. The `Block` class exposes the dropped one directly via `get_block_parent_hash()` → `return self.parents[0]` (`block.py:169`). This is exactly the "a block has three parents, `parents[0]` is the block parent" claim from Chapter 8, now visible in code.

The payoff: a **double-spend** is a collision in the input edges (two transactions whose `inputs` cite the same `(tx_id, index)`), and the class even has a method to detect it on the fly — `Transaction.is_double_spending()` (`transaction.py:463`) walks its inputs and asks each spent output's metadata whether it is already spent by someone else. The graph (parents) happily holds both conflicting transactions; resolving the conflict is consensus (Ch 32). The structure contains conflicts; it does not prevent them — precisely as Chapter 8 promised.

25.5.5 The subclass hierarchy

`GenericVertex` is abstract; the real objects are its descendants. There are two concrete branches and four leaf specializations.



What distinguishes each subclass, and why:

- **Block** (block.py:45) declares its kind via `is_block → True` / `is_transaction → False` (:82 / :87). It overrides the funds serialization to have no inputs and no token list (`get_funds_struct` , :222), adds a small `data` field (:74), and exposes the block-parent helpers (`get_block_parent_hash` , :166). Its `get_token_uid` asserts index 0 and returns HTR (:245) — blocks are HTR-only.
- **Transaction** (transaction.py:55) is the mirror: `is_transaction → True` , has inputs, and adds a `tokens` list in its own `__slots__` (:56) — the list of custom-token UIDs that an output's `token_data` index points into (`get_token_uid` , :277). It also caches the sighash (:93) because signing a multi-input transaction re-serializes the same body many times.
- **MergeMinedBlock** (merge_mined_block.py:31) extends `Block` for merged mining (Chapter 0; full treatment Ch 37). A miner mines Hathor as a side-effect of mining Bitcoin, so the proof-of-work is a Bitcoin "auxiliary PoW" object rather than a plain nonce. The subclass swaps the nonce out (`get_struct_nonce` returns the aux-PoW bytes, :84) and computes its hash through that aux-PoW (`calculate_hash` , :80). Everything else it inherits.
- **PoaBlock** (poa/poa_block.py:30) extends `Block` for proof-of-authority networks (private chains; Ch 32). It **asserts it has no outputs** (:47) — PoA blocks carry no reward — and replaces proof-of-work with a `signer_id` and a `signature` (:61), serialized into the graph fields (`get_graph_struct` , :86). Authority, not work, is what makes the block valid.
- **TokenCreationTransaction** (token_creation_tx.py:35) extends `Transaction` to create a custom token. Its defining trick: **its own hash becomes the new token's UID**. `update_hash()` is overridden so that after the hash is computed,

`self.tokens = [self.hash] (:86 - :90)` — the transaction declares the token it just minted. It also carries the token's name, symbol, and version.

- **OnChainBlueprint** extends `Transaction` to carry a nano-contract blueprint (contract source) on the ledger. It is only registered when nano-contracts are enabled (`TxVersion.get_cls`, `base_transaction.py:131`), and its internals belong to the nano-contracts subsystem — Chapter 39.

How does a stream of bytes off the wire become the right one of these classes? The `version` field. `TxVersion` (`base_transaction.py:99`) is an integer enum (`REGULAR_BLOCK = 0`, `REGULAR_TRANSACTION = 1`, `TOKEN_CREATION_TRANSACTION = 2`, ...), and `TxVersion.get_cls()` (:115) is a small dispatch table mapping each version to its class. The parser reads the version byte, looks up the class, and asks that class to build itself from the bytes. This is the factory-by-dispatch pattern of Chapter 3, and it is why every vertex carries its own version.

25.5.6 Hashing and serialization (pointer forward)

A vertex's bytes are produced in layers — `get_funds_struct()` (the value part: inputs, outputs, tokens), `get_graph_struct()` (the topology part: weight, timestamp, parents), then the nonce and any headers — assembled by `get_struct()` (:477). The hash is the double-SHA256 over a fixed 64-byte mining header (`get_mining_header_without_nonce`, :593) plus the nonce. You have seen the methods named here; the format itself — why the bytes are laid out as they are, how variable-length values are encoded — is the subject of Chapter 26 (serialization), which treats this package's `get_*_struct/create_from_struct` methods as its raw material. For this chapter it is enough to know that a vertex can turn itself into bytes and back, and that its identity is the hash of those bytes.

25.6 The metadata split: static versus mutable

This is the conceptual heart of the package, and the part most worth slowing down for.

A vertex's own data — its inputs, outputs, parents, nonce — is **fixed forever** once the vertex exists, because changing it would change the hash and thus the identity (§25.5.2). But the node learns things about a vertex as the ledger grows around it. Some of that learned knowledge is itself permanent; some of it changes as the graph changes. Hathor splits these into **two separate objects**, and understanding why is understanding the package.

The problem. Consider two facts the node computes about a block:

1. Its **height** — how many blocks lie between it and genesis.
2. Whether it is currently **voided** — whether consensus has decided it lost a conflict and should not count.

These two facts have opposite natures. A block's height is determined the moment the block is created (it is one more than its block-parent's height) and can never change — a block does not move in the chain. But a block's voided status can change: a later, heavier competing chain can void it in a reorg (Chapter 10), and a still-later development could even un-void it. Storing both in one undifferentiated "metadata" blob would lose this distinction and invite bugs (code that caches height is safe; code that caches voided-status is not).

The solution: two classes.

Static metadata — computed once, never changes

`VertexStaticMetadata` (`static_metadata.py:36`) is, in its own docstring, for "vertex attributes that are not intrinsic to the vertex data, but can be calculated from only the vertex itself and its dependencies, and whose values never change." It is an abstract base (a Pydantic model — Ch 18) with one common field, `min_height` (`:48`), and two concrete subclasses:

- `BlockStaticMetadata` (`:66`) adds `height` (`:67`) and the feature-activation bit counts (`:72`, for Ch 38). `height` is computed from the block-parent's height plus one (`_calculate_height`, `:107`).
- `TransactionStaticMetadata` (`:178`) adds `closest_ancestor_block` (`:181`) — the highest-up block the transaction depends on, used by feature activation for transactions.

The genericity from §25.5.1 now pays off: because `Block` is `GenericVertex[BlockStaticMetadata]`, the expression `block.static_metadata.height` type-checks, while the same expression on a transaction is a type error — a transaction's static metadata has no `height` at all. The type variable `StaticMetadataT` is what threads each vertex kind to the right static-metadata class.

Static metadata is set once and then frozen: `set_static_metadata` (`base_transaction.py:911`) refuses to change a value already set (it asserts the new value equals the old, `:914`). Each subclass knows how to build its static metadata from storage — `init_static_metadata_from_storage` (`Block` at `block.py:378`, `Transaction` at `transaction.py:492`) — and the build is required to be "fast, ideally $O(1)$ " (`static_metadata.py:90`), because it runs for every vertex on load.

Mutable metadata — rewritten as the ledger evolves

`TransactionMetadata` (`transaction_metadata.py:40`) holds everything the node computes about a vertex that can change. Its central fields, and what they model:


```

class TransactionMetadata:
    voided_by: Optional[set[bytes]] # :45 who voids this vertex (None/empty = executed)
    accumulated_weight: int # :48 total work of this vertex + all that confirm it
    score: int # :49 the consensus metric for blocks (Ch 32)
    first_block: Optional[bytes] # :50 the first block that confirmed this tx
    spent_outputs: dict[int, list[bytes]] # :42 which tx spent each of my outputs
    conflict_with: Optional[list[bytes]] # :44 txs that conflict with this one
    validation: ValidationState # :51 how far validation has progressed (§25.7)

```

- **voided_by** (:45) is the voiding mechanism of Chapter 10. An empty or `None` set means the vertex is executed (it counts); a non-empty set means it is voided, and the set's contents say why (its own hash if it lost a conflict, a conflicting tx's hash, a special `PARTIALLY_VALIDATED_ID` marker while it is still being validated). Voiding is a mark, not a delete — that is what lets it be reversed. The class gives careful helpers (`add_voided_by` / `del_voided_by` , : 379 / :386) that maintain the "empty means `None` " memory optimization.
- **accumulated_weight** (:48) is the sum of this vertex's work and the work of everything that confirms it (Chapter 9). It grows as more vertices pile on, so it is recomputed by a graph walk (`update_accumulated_weight` , `base_transaction.py:703`).
- **score** (:49) is the metric consensus uses to choose the canonical block chain (Chapter 32). For a non-genesis transaction it defaults to 0 (`get_metadata` , `base_transaction.py:668`).
- **spent_outputs** (:42) is how the node records, without deleting anything, that output `index` of this vertex has been spent by some later transaction. `get_output_spent_by(index)` (:142) returns the spender — but only if that spender is not itself voided, which is what makes a voided spend cleanly "give the coin back" (Chapter 7 §7.6, Chapter 10).

The invariant to carry away: a vertex's own fields are immutable; its `TransactionMetadata` is the node's mutable, evolving opinion about it. Who maintains the mutable side? Consensus (Chapter 32) writes `voided_by` and `score` ; ingestion (Chapter 33) sets `first_block` and `spent_outputs` ; the accumulated-weight walk updates `accumulated_weight` . The vertex class itself only holds the metadata and offers the accessor `get_metadata()` (`base_transaction.py:646`), which lazily loads it from storage or builds a fresh one (:667) on first access. That laziness is itself an invariant worth noting: metadata is built on demand and cached on the vertex, so the same in-memory vertex always hands back the same metadata object.

	Static metadata	TransactionMetadata
Class	<code>VertexStaticMetadata</code> (+ <code>Block/Transaction</code> subclasses)	<code>TransactionMetadata</code>
File	<code>static_metadata.py</code>	<code>transaction_metadata.py</code>

	Static metadata	TransactionMetadata
Mutability	computed once, never changes	rewritten as the graph evolves
Example fields	<code>height</code> , <code>min_height</code> , <code>closest_ancestor_block</code>	<code>voided_by</code> , <code>accumulated_weight</code> , <code>score</code> , <code>first_block</code>
Who writes it	the vertex on load (<code>init_static_metadata_from_storage</code>)	consensus, ingestion, the accumulated-weight walk
Why separate	facts that can be cached and trusted forever	facts that depend on the ever-changing rest of the DAG

Keeping these apart is not pedantry. It tells every other part of the node which facts it may cache and forget about (static) and which it must re-read after the graph changes (mutable). Mistaking one for the other is a category of bug the type system and the two-class split are there to prevent.

25.7 Validation state and storage scope

Two smaller mechanisms round out the model; both are covered in depth elsewhere, so here is just enough to recognize them.

Validation state. A vertex is not validated all at once. `ValidationState` (`validation_state.py:19`) is an enum tracking how far a vertex has progressed: `INITIAL` (not validated) → `BASIC` (its own structure and graph info check out, but not its dependencies) → `FULL` (all parents and inputs reached `FULL` and full validation ran), with `CHECKPOINT` / `CHECKPOINT_FULL` for vertices whose validity is assured by a known checkpoint, and `INVALID` as the terminal failure (`:45 - :50`). The reason for a partial state is dependency order: you can check a transaction's weight before you have its parents, but you cannot check that it does not double-spend until every input transaction is itself fully validated (the docstring at `:37` explains this). The state lives on the mutable metadata (`validation`, `transaction_metadata.py:51`), and `set_validation` (`base_transaction.py:524`) keeps it in sync with the `PARTIALLY_VALIDATED_ID` marker in `voided_by` — a partially-validated vertex is "voided by" its own incompleteness until it is fully connected. Full treatment in Chapters 31 and 33.

Storage scope. Chapter 0 §0.3 mentioned that during initialization the node temporarily lets storage return data in any validation state, then narrows it. That mechanism is `TxAllowScope` (in `transaction/storage/`), and it exists precisely because of the partial-validation states above: while rebuilding its view of the ledger, the node

must be able to load not-yet-fully-valid vertices; in normal operation it should only ever hand out fully-valid ones. The scope is the switch that widens and narrows what storage will return. Full treatment with storage, Chapter 27.

25.8 How it plugs into the lifecycle

These classes are the currency the rest of Part II trades in. Tracing one vertex's life:

1. **Born or received.** A vertex is created locally (the wallet builds a transaction; a miner produces a block) or arrives from a peer as bytes, parsed by `create_from_struct` into the right subclass via `TxVersion` (§25.5.5). Its own fields are now fixed; its hash is its identity.
2. **Loaded from storage.** When the node reads a vertex back from disk, storage reconstructs the object and calls `init_static_metadata_from_storage` to attach its (immutable) static metadata, and `get_metadata` to attach its (mutable) `TransactionMetadata`. Storage and the graph-walk utilities are **Chapter 27**.
3. **Verified.** The verification pipeline checks the rules this package deliberately did not enforce — conservation of value, valid signatures (running the locking/unlocking scripts in `transaction/scripts/`), sufficient weight, structural correctness — and advances the `ValidationState`. **Chapter 31**.
4. **Consensus updates its metadata.** As the vertex is woven into the DAG, consensus computes `accumulated_weight` and `score`, decides conflicts, and writes `voided_by`. This is the mutable metadata of §25.6 being rewritten. **Chapter 32**.
5. **Ingested.** The vertex handler runs steps 3 and 4 end-to-end, sets `first_block` and `spent_outputs`, updates the indexes, and announces the change. "Data arrived" becomes "ledger changed." **Chapter 33**.

At every step the object is the same `Block` or `Transaction`; what changes is its mutable metadata, never its identity.

Recap

Concept (earlier chapter)	In code	Where
Vertex / DAG node (Ch 8)	<code>GenericVertex</code> (ABC, generic); alias <code>Vertex / BaseTransaction</code>	<code>base_transaction.py:</code> 148 , :932

Concept (earlier chapter)	In code	Where
Block (Ch 8)	<code>Block</code> → <code>MergeMinedBlock</code> , <code>PoaBlock</code>	<code>block.py:45</code> ; <code>merge_mined_block.py:31</code> ; <code>poa/poa_block.py:30</code>
Transaction (Ch 8)	<code>Transaction</code> → <code>TokenCreationTransaction</code> , <code>OnChainBlueprint</code>	<code>transaction.py:55</code> ; <code>token_creation_tx.py:35</code>
Parents — topology edge (Ch 8 §8.4)	<code>parents: list[VertexId]</code> ; <code>parents[0] = block.parent</code>	<code>base_transaction.py:204</code> ; <code>block.py:169</code>
Inputs — spending edge (Ch 7)	<code>inputs: list[TxInput]</code> , <code>(tx_id, index, data)</code>	<code>base_transaction.py:202</code> , :936
Output / coin (Ch 7)	<code>TxOutput(value, script, token_data)</code>	<code>base_transaction.py:1022</code>
Multi-token + authorities (Ch 7 §7.6)	<code>token_data</code> byte: index mask + authority bit	<code>base_transaction.py:1025</code> , :1099 , :1103
Voiding (Ch 10)	<code>TransactionMetadata.voided_by</code>	<code>transaction_metadata.py:45</code>
Accumulated weight / score (Ch 9)	<code>TransactionMetadata.accumulated_weight</code> , <code>.score</code>	<code>transaction_metadata.py:48</code> , :49
Immutable derived facts	<code>VertexStaticMetadata</code> (Block adds height)	<code>static_metadata.py:36</code> , :66
Partial validation	<code>ValidationState</code> enum	<code>validation_state.py:19</code>

The `hathor/transaction/` package is where the two great abstractions of Part I's domain track stop being diagrams and become Python you can instantiate. A vertex is a `GenericVertex` — abstract, generic over its static-metadata companion, realized as a `Block` or a `Transaction` (and four further leaves). Its `inputs` and `outputs` are the UTXO model of Chapter 7 down to the bit-packed `token_data` byte; its `parents` are the DAG topology of Chapter 8, kept rigorously separate from the inputs. Around each

vertex the node keeps two kinds of memory: static metadata it computes once and trusts forever, and mutable `TransactionMetadata` — `voided_by`, `accumulated_weight`, `score` — that consensus rewrites as the graph grows. The vertex's own fields are frozen by the hash that names it; the node's opinion about the vertex is what changes. Hold that one sentence and the rest of Part II reads cleanly. Next, Chapter 26 answers a question this chapter kept deferring: how exactly do these objects become the compact bytes that travel between nodes and onto disk?

1. `VertexId` is a type alias for `bytes` (`hathor/types.py:26`) — specifically the 32-byte double-SHA256 hash that identifies a vertex. Storing parents and inputs as `VertexId` rather than as object references keeps a vertex small and lets the storage layer load the graph lazily. ↩
2. An abstract base class (ABC) is a class you cannot instantiate directly; it defines a shared interface and marks some methods `@abstractmethod`, forcing every concrete subclass to implement them. `GenericVertex` is an ABC, so you only ever create a `Block` or a `Transaction`. Full treatment in Chapter 1. ↩
3. A generic class is parameterized by a type variable, written `Class[T]`. `GenericVertex[StaticMetadataT]` is generic over its static-metadata type, so `Block = GenericVertex[BlockStaticMetadata]` and `Transaction = GenericVertex[TransactionStaticMetadata]` stay distinct to the type-checker. Full treatment in Chapter 5. ↩
4. `__slots__` declares the fixed set of attributes instances may have, replacing each object's per-instance dictionary with a compact fixed layout. It saves memory (decisive when millions of vertices are in RAM) and prevents accidental attribute typos. It is a Python optimization, not a domain concept. ↩
5. An authority output carries no spendable value; instead its `value` field is reinterpreted as a permissions bitmask (mint and/or melt rights for a token). It is flagged by the high bit of the output's `token_data` byte. Authorities are how a token issuer retains control over a token's supply — see Chapter 7 §7.6. ↩
6. The sighash ("signature hash") is the hash of a transaction's body that a signer actually signs. Each input's `data` (unlocking script) is blanked while computing it, because you cannot sign over the signature you are in the middle of producing. `Transaction` caches it (`transaction.py:93`) because signing many inputs re-serializes the same body repeatedly. ↩
7. A reorg (reorganization) is when the node switches its canonical block chain to a different, heavier one, voiding the blocks of the chain it abandons. Because voiding is a reversible mark on mutable metadata (`voided_by`), not a deletion, a reorg can flip a block from executed to voided without destroying its data. Full treatment in Chapter 32. ↩

Chapter 26 — The Wire Format: Serialization

What you'll learn in this chapter

- What **serialization** and **deserialization** are, framed as a general problem: how an object that lives in memory becomes a flat stream of bytes for the network or disk, and how it is reconstructed.
- The byte-level vocabulary a junior reader needs first — **bytes**, **endianness**, **fixed-width integers**, **variable-length integers (varint / LEB128)**, and **length-prefixing** — each defined and motivated before any Hathor code.
- The two serialization worlds inside `hathor-core`: the hand-rolled `struct`-based codec that turns a **vertex** into bytes, and the newer general-purpose `hathor/serialization/` framework (`Serializer/Deserializer` plus per-type encoders) that recent subsystems build on.
- Exactly how a vertex is laid out on the wire: the **funds struct**, the **graph struct**, the **nonce**, and the precise byte layouts of `TxInput` and `TxOutput`.
- How `VertexParser` reads raw bytes back into the correct vertex subclass using the version byte.
- **Why Hathor uses a bespoke binary format and not JSON or Protobuf** — the determinism that consensus and hashing demand, compactness, byte-exact cross-node reproducibility — and the honest cost of that choice.

Chapter 25 built the vertex classes — `Block`, `Transaction`, their inputs, outputs, and metadata — as objects living in the node's memory. It ended on a promise: those objects must eventually leave memory. A transaction you create has to travel across the network to your peers; a block the node accepts has to be written to disk so it survives a restart. Both directions require turning a Python object into a sequence of bytes, and turning those bytes back into the same object on the other side. That translation is **serialization**, and it is this chapter's whole subject.

Serialization is not a footnote in a blockchain. It is load-bearing in a way it never is in an ordinary web application, and the reason is the hash. A vertex's identity is the hash of its serialized bytes (Chapter 25, §25.5.2; the proof-of-work of Chapter 9 is computed over those same bytes). If two nodes serialized the same transaction into even slightly different byte strings, they would compute different hashes, disagree about the

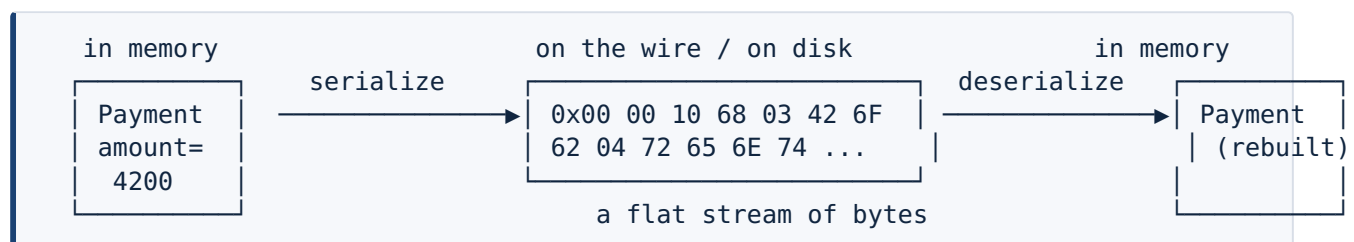
transaction's identity, and fall out of consensus. So the format cannot merely be a way to write the object down — it must be the one, exact, reproducible way, identical on every node, forever. That requirement shapes every decision in this chapter.

26.1 The problem, in general terms

Start away from Hathor entirely. You have an object in memory — say, in a hypothetical accounting program, a `Payment` with an amount, a recipient name, and a note:

```
payment = Payment(amount=4200, recipient="Bob", note="rent")
```

Inside the running program this is a tidy bundle of fields the language manages for you. But "in memory" is a fiction the moment you want to do anything outside this one process. A network socket does not carry Python objects; it carries bytes. A file on disk does not store fields; it stores bytes. To send the payment to another machine, or to save it, you must answer a flat question: **which exact sequence of bytes represents this object?** Producing that sequence is **serialization** (also called marshalling or encoding). The reverse — taking a byte sequence and rebuilding the object — is **deserialization** (unmarshalling, decoding, parsing).



The contract between the two sides is the **format**: a precise agreement about what each byte means and in what order the bytes appear. Get the format right on both ends and the rebuilt object equals the original. Get it wrong by a single byte and you get garbage, a crash, or — worse for a blockchain — a different but plausible-looking object.

Any serialization format has to make four decisions, and the rest of this chapter is really just Hathor's answers to them:

1. **How are numbers written as bytes?** A Python `int` has no fixed size; bytes do. You must choose how many bytes a number occupies, and in which order those bytes go.
2. **How are variable-length things framed?** A script or a note can be any length. The reader, scanning left to right, must know where one field ends and the next begins.
3. **How are the fields ordered, and which is which?** The reader rebuilds fields in a fixed sequence; the writer must emit them in that same sequence.

4. **How does the reader know which kind of object it is reading?** A stream of bytes is just bytes; something in it must say "this is a `Block`" versus "this is a `Transaction`."

Before Hathor's answers, the vocabulary those answers use.

26.2 The concepts it rests on

Bytes, and why everything reduces to them

A **byte**¹ is eight bits — a number from 0 to 255. Every file, every network packet, every disk block is, at bottom, a sequence of bytes. Higher-level things (text, numbers, objects) are interpretations layered on top of bytes by agreement. Python gives you a `bytes` object (immutable) and a `bytearray` (mutable) to hold such sequences; serialization is the craft of building one of these from an object, and reading one back.

Endianness — the order of a number's bytes

A single byte holds 0–255, which is rarely enough; a timestamp or a coin value needs several bytes. The moment a number spans more than one byte, a question appears that has no "natural" answer: **in what order do the bytes go?** Consider the number 4200, which in hexadecimal is `0x1068`. It needs two bytes: `0x10` and `0x68`. There are two ways to lay them down:

```
the number 4200 = 0x1068, written in two bytes:
```

big-endian	:	<code>0x10 0x68</code>	← most-significant byte first ("big end" first)
little-endian	:	<code>0x68 0x10</code>	← least-significant byte first ("little end" first)

This choice is called **endianness**². Neither order is more correct; what matters is that the writer and reader agree. Big-endian is often called "network byte order" because most internet protocols use it, and it reads naturally left-to-right like we write decimal numbers. **Hathor's vertex format is big-endian throughout** — its low-level `int_to_bytes` helper hard-codes `byteorder='big'` (`hathor/transaction/util.py:32`), and the `struct` format strings start with `!` (network/big-endian order, e.g. `_GRAPH_FORMAT_STRING = '!dIB'`, `base_transaction.py:72`). The general serialization framework's integer encoder also hard-codes big-endian (`encoding/int.py:48`). Whenever you see a multi-byte number in this chapter, read it big end first.

Fixed-width integers — spending exactly N bytes

The simplest way to write a number is to decide up front how many bytes it gets and always use that many. A timestamp gets 4 bytes; a length counter gets 1 byte; a hash gets 32 bytes. This is a **fixed-width** encoding. Its virtue is that the reader always

knows exactly how far to advance. Its cost is waste: the number 5 stored in 4 bytes still occupies 4 bytes (`00 00 00 05`), three of them zero. The vertex format uses fixed-width integers for fields whose range it knows in advance — which is most of them.

Variable-length integers (varint / LEB128) — paying for what you use

When a number's range is wide but its typical value is small, fixed width wastes space. The fix is a **variable-length integer**, or **varint**³: an encoding that uses one byte for small numbers and grows only as needed. The standard scheme — the one Hathor's general framework uses — is **LEB128**⁴ ("Little Endian Base 128"). The idea is straightforward once you see it:

- Chop the number into 7-bit groups.
- Store each group in the low 7 bits of a byte.
- Use the high bit of each byte as a **continuation flag**: `1` means "another byte follows," `0` means "this is the last byte."

So a number under 128 fits in a single byte (high bit `0`); a number up to 16,383 takes two bytes; and so on. The reader keeps consuming bytes until it meets one whose high bit is `0`. Hathor's implementation is `encode_leb128 / decode_leb128` (`hathor/serialization/encoding/leb128.py:54` and `:76`); the continuation bit is the `| 0b1000_0000` on every non-final byte (`:73`) and the `& 0b1000_0000` test that stops the reader (`:90`). (The "signed" variant additionally checks the sign bit `0b0100_0000`, so it can encode negative numbers too.)

Length-prefixing — framing variable-length data

The last primitive idea is **length-prefixing**. When a field can be any length — a script, a note, a list of inputs — the reader scanning left-to-right needs to know where it ends before it reads it. The universal trick: write the length first, then the data. The reader reads the length, then reads exactly that many bytes.

a length-prefixed blob "rent" (4 bytes of data):

0x04	0x72 0x65 0x6E 0x74
length	'r' 'e' 'n' 't'

"read 4" then read 4 bytes

The alternative — a delimiter, like the null byte that ends a C string — fails for binary data, because the data itself might contain the delimiter byte. Length-prefixing has no such ambush: the length is separate metadata, and the data that follows is opaque. Hathor length-prefixes every variable field. The only thing that varies is how the length itself is written — a fixed-width count in the vertex codec, a LEB128 count in the general framework — as we will see.

Recap — the vertex (full treatment in Ch. 25). A vertex is any node of Hathor's ledger DAG — a `Block` or a `Transaction`. Each carries a `version`, `signal_bits`, `weight`, `timestamp`, a `nonce`, a list of `inputs` (`TxInput`, the UTXO spending pointers), a list of `outputs` (`TxOutput`, the coins it creates), and a list of `parents` (the DAG topology edge). A transaction additionally carries a `tokens` list. Those are the fields this chapter turns into bytes. → full treatment in Ch. 25.

Recap — the hash and determinism (full treatment in Ch. 9 & 25). A vertex's identity is the double-SHA256 of its serialized bytes; the proof-of-work nonce is the number a miner varies so that this hash falls under a target. Because the hash is computed over the serialized bytes, the serialization must be byte-for-byte identical on every node. Serialization is therefore not a convenience layer — it is part of the consensus rules. → full treatment in Ch. 9 (proof-of-work) and Ch. 25 (the hash invariant).

26.3 Localization

Two parts of the tree do serialization. The first is the general-purpose framework, the package this chapter is named for:

hathor/	
└─ serialization/	◀ YOU ARE HERE – the general encoding framework
serializer.py	← Serializer (ABC): write_byte / write_bytes / write_struct
deserializer.py	← Deserializer (ABC): read_byte / read_bytes / peek_* / read_*
bytes_serializer.py	← BytesSerializer (concrete: accumulate into memory)
bytes_deserializer.py	← BytesDeserializer (concrete: read from a memoryview)
types.py	← Buffer = bytes memoryview
consts.py	← DEFAULT_LEB128_MAX_BYTES, DEFAULT_BYTES_MAX_LENGTH
exceptions.py	← SerializationError + OutOfDataError / BadDataError / ToolError
└─ encoding/	← per-primitive encoders (each a pair of free functions)
int.py	← fixed-width signed/unsigned ints (big-endian)
leb128.py	← variable-length ints (the canonical varint)
bytes.py	← length-prefixed byte blobs (length is LEB128)
bool.py	← a bool as one byte (0x00 / 0x01)
utf8.py	← length-prefixed UTF-8 text
output_value.py	← the clever coin-value codec (4-or-8 bytes)
ecv.py	← elliptic-curve value encoding
└─ compound_encoding/	← encoders for shapes built from primitives
optional.py	← "maybe a value" (a presence flag + the value)
collection.py	← length-prefixed lists/sets
mapping.py	← length-prefixed dicts
tuple.py	← fixed-arity tuples
signed_data.py	← a payload plus its signature
└─ adapters/	← wrappers that constrain an encoder
max_bytes.py	← caps how many bytes a field may consume
generic_adapter.py	← base for such wrapping adapters

The second is older, and lives inside the transaction package — the hand-rolled codec the vertex classes use:

hathor/	
└─ transaction/	
base_transaction.py	← get_funds_struct / get_graph_struct / get_struct, __bytes__, TxInput.__bytes__, TxOutput.__bytes__
transaction.py	← Transaction.get_funds_struct + parse-back methods
block.py	← Block.get_funds_struct + parse-back methods
vertex_parser.py	← VertexParser.deserialize: bytes → right subclass
util.py	← int_to_bytes, unpack, output_value_to_bytes

Context. Two serialization worlds coexist, and that is the central fact to hold while reading this chapter. The **vertex wire format** — the bytes that get hashed, sent to peers, and stored — is produced by hand-written code in `hathor/transaction/` using Python's built-in `struct` module and a few helpers in `transaction/util.py`. The newer **hathor/serialization/ framework** is a general, reusable encoding toolkit (a `Serializer/Deserializer` pair plus per-type encoders) that more recent subsystems — nano-contracts, vertex headers — build on, and into which the vertex codec's value encoding has already been migrated (`transaction/util.py` calls `decode_output_value` from the new package). Read §26.4 for the framework's design and §26.5 for the vertex format itself; they meet at the coin-value codec.

26.4 The general framework, walked

Build the intuition with a hand-encoded toy first, then meet the real classes.

26.4.1 A toy encoding, by hand

Suppose you must serialize a tiny record: one integer `id` and one variable-length blob `name`. You decide the format yourself:

- `id` → a 2-byte big-endian unsigned integer.
- `name` → a length-prefixed blob: one byte of length, then that many bytes.

Encoding `id=4200`, `name=b"Bob"` by hand:

```
id = 4200 = 0x1068, big-endian, 2 bytes    → 0x10 0x68
len(name) = 3, one byte                    → 0x03
name = b"Bob"                             → 0x42 0x6F 0x62

full stream: 10 68 03 42 6F 62            (6 bytes)
```

Decoding reverses it with the format as the only guide: read 2 bytes as a big-endian int → 4200; read 1 byte → 3; read 3 bytes → `b"Bob"`. Notice that the decoder is a cursor moving left to right, consuming exactly as many bytes as the format dictates at each step. Every serializer in the world, Hathor's included, is this cursor idea dressed up. The `hathor/serialization/` framework's only ambition is to give you reusable, named pieces for "read 2 bytes as an int" and "read a length-prefixed blob" so you do not hand-roll the cursor every time.

26.4.2 `Serializer` and `Deserializer` — the two cursors

The framework is built around a matched pair, both declared as **abstract base classes**⁵ (so the byte-level mechanics can have more than one concrete implementation). The **`Serializer`** (`hathor/serialization/serializer.py:32`) is the writing cursor: its concrete subclass `BytesSerializer` accumulates the output as a list

of byte parts (`bytes_serializer.py:21`). The **Deserializer** (`hathor/serialization/deserializer.py:32`) is the reading cursor: its concrete subclass **BytesDeserializer** holds the input as a `memoryview` and shortens it as bytes are consumed (`bytes_deserializer.py:24`). They are deliberate mirror images — whatever you write, you can read back.

The two abstract methods every serializer must provide are tiny:

```
@abstractmethod
def write_byte(self, data: int) -> None:
    """Write a single byte."""

@abstractmethod
def write_bytes(self, data: Buffer) -> None:
    ...
```

(`serializer.py:41`, `:46`.) The concrete **BytesSerializer** implements them by appending to its internal list of `memoryview` parts and joining them only at the end, in `finalize()` (`bytes_serializer.py:33`). This "defer the join until the end" detail is a performance choice — concatenating bytes repeatedly is costly, so it keeps the pieces apart and joins once.

The deserializer's core methods come in two flavours, **peek** and **read**. A read consumes bytes and advances the cursor; a peek looks without consuming:

```
@abstractmethod
def peek_byte(self) -> int:
    """Read a single byte but don't consume from buffer."""

@abstractmethod
def read_byte(self) -> int:
    """Read a single byte as unsigned int."""

@abstractmethod
def read_bytes(self, n: int, *, exact: bool = True) -> Buffer:
    """Read n bytes, when exact=True it errors if there isn't enough data"""
```

(`deserializer.py:47`, `:62`, `:67`.) The `peek` capability matters more than it looks: the coin-value codec in §26.5.3 peeks at the first byte to decide whether a value is 4 or 8 bytes long before it commits to reading. The `exact=True` default means "if there are fewer than `n` bytes left, raise" — a malformed stream is rejected, not silently truncated. The concrete reader raises `OutOfDataError` (`bytes_deserializer.py:47`) in that case.

There are also two `struct`-aware convenience methods that bridge to Python's built-in `struct` module: `write_struct(data, format)` packs a tuple per a format string, and `read_struct(format)` / `peek_struct(format)` unpack one (`serializer.py:52`, `deserializer.py:87`, `:56`). These are the seam where the new framework reuses the same `struct` machinery the old vertex codec uses directly.

26.4.3 The primitive encoders

The `encoding/` modules are the framework's real content. Each is a pair of free functions, `encode_*` and `decode_*`, that take the serializer (or deserializer) as their first argument and call its `write_*` / `read_*` methods. This is a deliberate design: the cursor classes know only how to move bytes; the meaning of each type lives in a small, independently-testable function. Each module's docstring even contains runnable doctest examples — a good place to see the exact bytes each encoder produces.

Fixed-width integers (`encoding/int.py`). A thin, safe wrapper over Python's built-in `int.to_bytes`:

```
def encode_int(serializer: Serializer, number: int, *, length: int, signed: bool) -> None:
    try:
        data = int.to_bytes(number, length, byteorder='big', signed=signed)
    except OverflowError:
        raise ValueError('too big to encode')
    serializer.write_bytes(data)
```

(`int.py:42`.) Two things to note. First, the byte order is hard-coded to `'big'` — there is one endianness for the whole format, so it can never accidentally mix orders. Second, the `OverflowError` that `to_bytes` raises when a value does not fit in `length` bytes is translated into a `ValueError` (`:50`), so the caller sees a clean "won't fit" failure rather than a leaked low-level exception. The decoder mirrors it with `int.from_bytes(..., byteorder='big', signed=signed)` (`int.py:60`).

Variable-length integers (`encoding/leb128.py`). This is the canonical varint from §26.2. The encoder loops, peeling off 7 bits at a time and setting the continuation bit on every byte but the last:

```
def encode_leb128(serializer: Serializer, value: int, *, signed: bool) -> None:
    if not signed and value < 0:
        raise ValueError('cannot encode value <0 as unsigned')
    while True:
        byte = value & 0b0111_1111          # low 7 bits of the remaining value
        value >>= 7                          # shift them off
        if signed:
            cont = (value == 0 and (byte & 0b0100_0000) == 0) or \
                   (value == -1 and (byte & 0b0100_0000) != 0)
        else:
            cont = (value == 0 and (byte & 0b1000_0000) == 0)
        if cont:
            serializer.write_byte(byte)      # last byte: high bit stays 0
            break
        serializer.write_byte(byte | 0b1000_0000) # more to come: set continuation bit
```

(`leb128.py:54`.) The decoder mirrors it, reassembling the value 7 bits at a time and stopping when it meets a byte with the high bit clear — the `if (byte & 0b1000_0000) == 0` test at `leb128.py:90`, after which it reads any remaining bytes via `read_byte` in a

loop. (Note: the current `encode_leb128` takes no length cap; the repository keeps the recommended cap as a constant, `DEFAULT_LEB128_MAX_BYTES = 4` in `consts.py:15`, applied by callers that want it — see §26.6.)

Length-prefixed byte blobs (`encoding/bytes.py`). This is §26.2's length-prefixing made literal, and it composes the two integer encoders cleanly:

```
def encode_bytes(serializer: Serializer, data: bytes) -> None:
    encode_leb128(serializer, len(data), signed=False)
    serializer.write_bytes(data)

def decode_bytes(deserializer: Deserializer) -> bytes:
    size = decode_leb128(deserializer, signed=False)
    return bytes(deserializer.read_bytes(size))
```

(`bytes.py:67` , `:77` .) The length goes first as an unsigned LEB128, then the raw bytes. The reader does the reverse: read the length, then read exactly that many bytes. (The module docstring works a nice example: a 4-byte blob gets a 1-byte length prefix `\x04`, while a 128-byte blob gets a 2-byte length prefix `\x80\x01` — the varint growing only when it must.)

Booleans (`encoding/bool.py`) are one byte: `0x01` for true, `0x00` for false, and any other byte is rejected as invalid (`encode_bool` / `decode_bool` , `bool.py:61` , `:68`). **Text** (`encoding/utf8.py`) is encoded to UTF-8 and then length-prefixed exactly like `bytes`.

26.4.4 The compound encoders

The `compound_encoding/` package builds shapes out of those primitives, and these are where serialization stops being about single values and starts being about whole data structures.

- **Optional** (`compound_encoding/optional.py`) handles "a value that might be absent." The trick from §26.1: write a one-byte presence flag, and only if it is set, write the value — composed from whatever encoder handles the inner value.
- **Collection** (`compound_encoding/collection.py`) handles lists and sets: length-prefix the count, then encode each item.
- **Mapping** (`compound_encoding/mapping.py`) does the same for dictionaries (count, then key/value pairs); **tuple** does it for fixed-arity tuples; **signed_data** pairs a payload with its signature.

The pattern throughout is composition: a compound encoder knows the shape (a count followed by items) and is given a function for the element. This is how the framework stays general without a giant table of every concrete type — the higher-order-function style of Chapter 2 applied to bytes.

26.4.5 The adapters

The `adapters/` package holds wrappers that constrain an existing cursor rather than define a new one. The clearest is `max_bytes` (`adapters/max_bytes.py`):

`Serializer.with_max_bytes(n)` returns a wrapper that enforces no more than `n` bytes are written, and `Deserializer.with_max_bytes(n)` enforces no more than `n` are read (`serializer.py:56`, `deserializer.py:92`). Conceptually this is the **adapter / decorator** idea from Chapter 3 and Chapter 4 — interpose a wrapper that adds a check without rewriting the thing it wraps. Such caps exist because a deserializer reads attacker-supplied bytes; a field allowed to be "any length" is a denial-of-service waiting to happen (§26.6).

26.5 The vertex format, walked

Now the format that actually matters for consensus: how a `Block` or `Transaction` becomes the exact bytes that get hashed, sent, and stored. This path predates the framework of §26.4 and is hand-written in `hathor/transaction/` using Python's `struct` module and the helpers in `transaction/util.py`. It is the canonical example of a bespoke binary format, so we walk it in full.

26.5.1 The two structs: funds and graph

A serialized vertex is built in two halves, mirroring the two kinds of edge from Chapter 25. The split is not cosmetic; it reflects that a vertex has a value aspect (what coins it moves) and a topology aspect (where it sits in the DAG).

The **funds struct** carries the value side — the header bytes, the counts, and the inputs, outputs, and tokens themselves. For a `Transaction` (`transaction.py:205`):

```
def get_funds_struct(self) -> bytes:
    struct_bytes = pack(
        FUNDS_FORMAT_STRING,      # '!BBBBB' – five unsigned bytes, big-endian
        self.signal_bits,
        self.version,
        len(self.tokens),
        len(self.inputs),
        len(self.outputs),
    )
    for token_uid in self.tokens:
        struct_bytes += token_uid      # 32 bytes each
    for tx_input in self.inputs:
        struct_bytes += bytes(tx_input) # §26.5.3
    for tx_output in self.outputs:
        struct_bytes += bytes(tx_output) # §26.5.3
    return struct_bytes
```

Read the layout off the code directly. Five single-byte header fields come first, in this exact order: `signal_bits`, `version`, the **token** count, the **input** count, and the **output** count (`FUNDS_FORMAT_STRING = '!BBBBB'`, `transaction.py:44`). Then the bodies, in the

same order the counts appeared: the token UIDs (32 bytes each), then the inputs, then the outputs. The matching reader, `get_funds_fields_from_struct` (`transaction.py:163`), unpacks the same five header bytes and then loops the recorded number of times to rebuild each list:

```
(self.signal_bits, self.version, tokens_len, inputs_len, outputs_len), buf = \
    unpack(_FUNDS_FORMAT_STRING, buf)
for _ in range(tokens_len):
    token_uid, buf = unpack_len(TX_HASH_SIZE, buf)    # 32 bytes
    self.tokens.append(token_uid)
for _ in range(inputs_len):
    txin, buf = TxInput.create_from_bytes(buf, verbose=verbose)
    self.inputs.append(txin)
for _ in range(outputs_len):
    txout, buf = TxOutput.create_from_bytes(buf, verbose=verbose)
    self.outputs.append(txout)
```

(`transaction.py:177 - :201`.) The counts are why this works: because the writer prefixed each list with its length (the length-prefixing idea of §26.2, applied to lists of structures rather than blobs), the reader knows exactly how many times to loop. A `Block` overrides `get_funds_struct` to a shorter three-field header — `signal_bits`, `version`, output count — then just the outputs, with no inputs and no tokens (`block.py:222`; `block.py`'s `_FUNDS_FORMAT_STRING = '!BBB'` at `:39`) — because a block mints coins, never spends them, and only ever holds HTR (Chapter 25, §25.5.3). The version byte tells the reader which of these two layouts to expect.

A subtlety worth pausing on — field order versus the base class. The base `GenericVertex.get_funds_struct` is abstract (`base_transaction.py:438`, just raise `NotImplementedError`): the base class deliberately refuses to define a funds layout, because the two concrete kinds differ. The on-wire truth is therefore always a concrete subclass's method — `Transaction` writes `signal_bits`, `version`, `#tokens`, `#inputs`, `#outputs`; `Block` writes `signal_bits`, `version`, `#outputs`. Both put `signal_bits` first and `version` second, which is exactly what the parser relies on: `VertexParser.deserialize` reads the version from `data[1]`, the second byte (§26.5.4). When in doubt, trust the subclass that runs and the parser that reads — they are the authority on what the bytes mean.

The **graph struct** carries the topology side — the proof-of-work weight, the timestamp, and the parents (`base_transaction.py:442`):

```
def get_graph_struct(self) -> bytes:
    struct_bytes = pack(_GRAPH_FORMAT_STRING, self.weight, self.timestamp, len(self.parents))
    for parent in self.parents:
        struct_bytes += parent                # 32 bytes each
    return struct_bytes
```

The format string is `_GRAPH_FORMAT_STRING = '!dIB'` (`base_transaction.py:72`): a `d` (8-byte double — the weight), an `I` (4-byte unsigned int — the timestamp), and a `B` (1-byte parent count), all big-endian. Then each parent hash follows verbatim, 32 bytes apiece — fixed, so no per-parent length prefix is needed (a hash is always exactly 32 bytes). The reader is `get_graph_fields_from_struct` (`base_transaction.py:413`), which unpacks `'!dIB'` and then reads `parents_len` hashes of `TX_HASH_SIZE` (32) bytes each (`:424 – :432`). That the weight is serialized as a `double` is notable: weight is logically `log2(work)`, a real number by nature (Chapter 9), so a floating-point field is the honest representation.

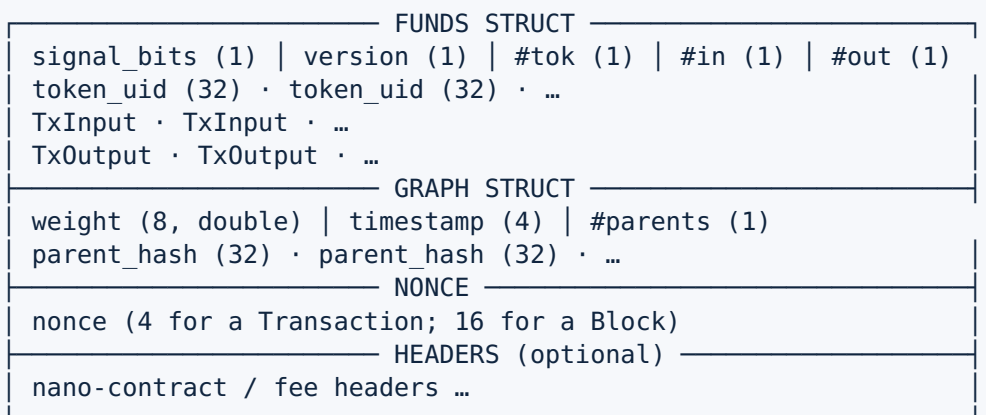
26.5.2 Assembling the whole vertex

The two structs plus the proof-of-work nonce plus any headers make the full serialization (`base_transaction.py:477`):

```
def get_struct(self) -> bytes:
    struct_bytes = self.get_struct_without_nonce() # funds struct + graph struct
    struct_bytes += self.get_struct_nonce()         # the nonce
    struct_bytes += self.get_headers_struct()       # optional trailing headers
    return struct_bytes

def __bytes__(self) -> bytes:
    return self.get_struct()
```

`get_struct_without_nonce` (`:457`) concatenates the funds struct then the graph struct. `get_struct_nonce` (`:467`) writes the nonce in `SERIALIZATION_NONCE_SIZE` bytes — and that size **differs by vertex kind**: a `Transaction` uses **4 bytes** (`transaction.py:58`), a `Block` uses **16 bytes** (`block.py:46`), because a block's proof-of-work search space is much larger. `get_headers_struct` (`:453`) appends any optional trailing headers (nano-contract, fee). The whole thing is reachable through Python's `bytes(vertex)` because `__bytes__` is defined (`:329`) — so anywhere the codebase needs a vertex's bytes, it writes `bytes(tx)`, and that is the on-wire form. The full byte layout of a transaction:



This same byte string is what the node hashes for the vertex's identity: the hash is the double-SHA256 built from a mining header without the nonce (`get_mining_header_without_nonce`, `base_transaction.py:593`) folded together with the nonce (`calculate_hash` at `:628`, via `calculate_hash1` / `calculate_hash2` at `:603` / `:613`). Because the hash is taken over exactly these bytes, the layout is part of the protocol — change one field's width or order and every hash in the ledger would change.

26.5.3 TxInput and TxOutput byte layouts

The inputs and outputs each serialize themselves through their own `__bytes__`. A `TxInput` (`base_transaction.py:959`) is the UTXO spending pointer of Chapter 7:

```
def __bytes__(self) -> bytes:
    ret = self.tx_id                # 32 bytes – which prior tx
    ret += int_to_bytes(self.index, 1) # 1 byte – which output of it
    ret += int_to_bytes(len(self.data), 2) # 2 bytes – unlocking-script length
    ret += self.data                # the unlocking script itself
    return ret
```

Layout: a 32-byte transaction hash, a 1-byte output index, a 2-byte length, then the unlocking-script bytes. The `(tx_id, index)` pair is the pointer "spend output `index` of transaction `tx_id`"; `data` is the signature-and-public-key that satisfies that output's lock. Note the length-prefixing again — the script is variable, so its length comes first, here as a fixed 2-byte field (max 65,535 bytes). The reader `TxInput.create_from_bytes` reverses it: 32 bytes for the hash, then `unpack('!BH', buf)` for the 1-byte index and 2-byte length, then that many script bytes (`:983` – `:998`).

A nearby method earns a mention because Chapter 25 referenced it:

`get_sighash_bytes` (`:971`) serializes an input with its `data` blanked to zero length (`int_to_bytes(0, 2)`, `:980`). This is the signing trick — you cannot sign over the very signature you are about to produce, so the body that gets signed uses empty unlocking scripts.

A `TxOutput` (`base_transaction.py:1022`) is a coin (`__bytes__` at `:1069`):

```
def __bytes__(self) -> bytes:
    ret = output_value_to_bytes(self.value) # 4 OR 8 bytes (see below)
    ret += int_to_bytes(self.token_data, 1) # 1 byte – token index + authority bit
    ret += int_to_bytes(len(self.script), 2) # 2 bytes – locking-script length
    ret += self.script                     # the locking script itself
    return ret
```

Layout: the value (variable, 4 or 8 bytes — next paragraph), the one `token_data` byte (Chapter 25, §25.5.3: low 7 bits pick the token, high bit flags an authority output — `TOKEN_INDEX_MASK` / `TOKEN_AUTHORITY_MASK` at `:1025` / `:1026`), a 2-byte script length, then the locking-script bytes. The reader is symmetric (`create_from_bytes`, `:1081`): decode the value, `unpack('!BH', buf)` for the `token_data` byte and the script length, then the script.

The value field is the most interesting micro-encoding in the whole vertex format, and it is the bridge into the §26.4 framework. A coin value is usually small enough for 4 bytes, but Hathor must allow values up to 2^{63} , which needs 8. Rather than always spending 8 bytes, the codec writes small values in 4 bytes and large values in 8, and steals the **sign bit** to tell them apart. `transaction/util.py`'s `bytes_to_output_value` now delegates to the framework's `decode_output_value`, whose encoder partner is `encode_output_value` (`hathor/serialization/encoding/output_value.py:89`):

```
MAX_OUTPUT_VALUE_32 = 2 ** 31 - 1    # fits in 4 signed bytes
MAX_OUTPUT_VALUE_64 = 2 ** 63        # the absolute maximum

def encode_output_value(serializer: Serializer, number: int, *, strict: bool = True) -> None:
    if number < 0:
        raise ValueError('Number must not be negative')
    if strict and number == 0:
        raise ValueError('Number must be strictly positive')
    if number > MAX_OUTPUT_VALUE_64:
        raise ValueError(...)
    if number > MAX_OUTPUT_VALUE_32:
        serializer.write_bytes((-number).to_bytes(8, byteorder='big', signed=True)) # 8-byte for
    else:
        serializer.write_bytes(number.to_bytes(4, byteorder='big', signed=True)) # 4-byte for
```

The mechanism: a positive value that fits in 4 bytes is written as a positive 4-byte signed integer — its top bit is `0`. A value too big for that is written as its negation in 8 bytes — a negative number, whose top bit is `1`. The decoder peeks at the first byte (`decode_output_value`, :108): if it is negative (`value_high_byte < 0`), the value was the 8-byte negated form, so read 8 bytes (`'!q'`) and negate back; otherwise read 4 bytes (`'!i'`) as-is. One bit, no separate length field, and the common case stays compact. The decoder even rejects a value that was written in 8 bytes but would have fit in 4 (:125) — enforcing the one canonical encoding rule that §26.6 is about. This is the kind of byte-thrift a bespoke format buys you and a generic one rarely does.

26.5.4 VertexParser — bytes back into the right subclass

Serialization's hard half is deserialization, because the reader is handed raw, possibly-hostile bytes and must decide what they are before it can parse them. Hathor answers the §26.1 question "which kind of object is this?" with the **version byte**. The entry point is `VertexParser.deserialize` (`hathor/transaction/vertex_parser.py:53`):

```
def deserialize(self, data: bytes, storage: TransactionStorage | None = None) -> BaseTransaction:
    """ Creates the correct tx subclass from a sequence of bytes """
    from hathor.transaction import TxVersion
    version = data[1] # the version field is the SECOND byte
    try:
        tx_version = TxVersion(version)
        is_valid = self._settings.CONSENSUS_ALGORITHM.is_vertex_version_valid(
            tx_version, include_genesis=True, settings=self._settings,
        )
        if not is_valid:
            raise StructError(f"invalid vertex version: {tx_version}")
        cls = tx_version.get_cls() # dispatch: version -> concrete class
        return cls.create_from_struct(data, storage=storage)
    except ValueError as e:
        raise StructError('Invalid bytes to create transaction subclass.') from e
```

The flow is: read the version from `data[1]` (the second byte — recall the wire order is `signal_bits` then `version`, §26.5.1), confirm the running network's consensus algorithm actually permits that version (a PoA network, for instance, rejects regular blocks), look up the matching class through `TxVersion.get_cls()` (`base_transaction.py:115` — the dispatch table mapping `REGULAR_BLOCK` → `Block`, `REGULAR_TRANSACTION` → `Transaction`, `TOKEN_CREATION_TRANSACTION` → `TokenCreationTransaction`, `MERGE_MINED_BLOCK` → `MergeMinedBlock`, `POA_BLOCK` → `PoaBlock`, and — only when nano-contracts are enabled — `ON_CHAIN_BLUEPRINT` → `OnChainBlueprint`), and hand the whole byte string to that class's `create_from_struct`. That method (`transaction.py:143`) constructs an empty instance, calls the `get_*_fields_from_struct` readers in funds-then-graph order, reads the nonce (`unpack('!I', buf)` — 4 bytes for a transaction, `:152`), then loops `get_header_from_bytes` for any trailing headers, and finally `update_hash()`. An unknown or disallowed version raises `StructError`, so a malformed or future-versioned vertex is rejected cleanly rather than misinterpreted. This single dispatch⁶ point is why every vertex must carry its own version: it is the one piece of self-description in an otherwise positional, schema-less format.

26.5.5 The two worlds, reconciled

You have now seen both serialization paths and the seam between them. The vertex codec (§26.5) is hand-written, positional, and `struct`-based; it is frozen because the bytes it produces are hashed and therefore part of consensus — you cannot change it without changing every vertex's identity. The general framework (§26.4) is newer, composable, and the home of reusable encoders; newer subsystems (nano-contract data, vertex headers) build on it, and the vertex codec's value encoding has already moved into it (`output_value.py`, called back from `transaction/util.py`). The honest summary: Hathor is mid-migration. The framework is where new serialization should live; the vertex format is a stable island that the framework is slowly absorbing one codec at a time. For a reader, the rule is simple — for the bytes that get hashed, read `transaction/`; for everything newer, read `serialization/`.

26.6 Why bespoke binary, and not JSON or Protobuf

This is the question the whole chapter exists to answer, and it is a real engineering decision with real trade-offs, not a foregone conclusion. A node could, in principle, represent a transaction as JSON (`{"version": 1, "inputs": [...], ...}`) or use a schema tool like Protobuf⁷. Hathor uses neither. Here is the case, and the cost.

1. Determinism — the decisive reason. A vertex's identity is the hash of its bytes (§26.2, §26.5.2), and consensus requires every node to compute the same hash for the same vertex. That demands a **canonical** encoding: exactly one byte string per object, reproducible on every machine, in every language, forever. JSON fails this outright. JSON does not fix the order of object keys, the whitespace, the number formatting (`4200` vs `4.2e3`), or the handling of large integers — two correct JSON encoders can emit different bytes for the same data, and then their hashes differ and the nodes disagree. Protobuf is closer but still does not guarantee a canonical byte form across implementations (field ordering and default-value handling can vary; its spec explicitly does not promise deterministic output). A hand-specified positional binary format has no such freedom: there is one order, one width, one endianness for every field — and the codec even rejects a value encoded in 8 bytes that would have fit in 4 (`output_value.py:125`), closing the last door to two encodings of one number. The bytes are the spec. This is why blockchains across the board use bespoke binary formats for the data that gets hashed. **Determinism is not a nice-to-have here; it is the requirement that rules out the alternatives.**

2. Compactness. A node stores and ships millions of vertices. The binary form is dense: a 4-byte timestamp instead of a 20-character ISO string, a 1-byte count instead of `"inputs":[`, a small coin value in 4 bytes instead of a decimal string. JSON would inflate every field with field names, quotes, brackets, and — because binary like a 32-byte hash is not representable in JSON at all without an encoding such as hex or base64 — would roughly double the size of every hash. Over the whole ledger and over every byte sent to every peer, that overhead is large. Compactness directly lowers disk usage, bandwidth, and sync time.

3. Byte-exact reproducibility and bounded parsing. Because the format is positional and self-contained, parsing is a straight left-to-right cursor walk with no reflection and no schema registry — and re-serializing a parsed vertex yields the identical bytes, which is exactly what the hash check relies on. The deserializer also stays defensive cheaply: `read_bytes(..., exact=True)` raises `OutOfDataError` rather than returning short data (`bytes_deserializer.py:65`), the `with_max_bytes` adapters cap how much a field may consume (`deserializer.py:92`), and the recommended LEB128 length cap (`consts.py:15`) keeps a length field from claiming an absurd size. A hostile peer cannot send a tiny message that claims a multi-gigabyte field and exhaust the node's memory. A general JSON/Protobuf parser is harder to bound this tightly.

Now the honest cost, because every choice has one:

- **You maintain the format by hand.** There is no `.proto` schema generating reader and writer for you; `get_funds_struct` and `get_funds_fields_from_struct` are two hand-written functions that must stay in lockstep. If they drift, vertices serialize one way and parse another, and the bug is silent until a hash mismatches.
- **No human-readability.** You cannot open a stored vertex in a text editor and read it; you need the format in your head or a decoder in your hand. (This is partly why `TxInput` / `TxOutput` keep `to_human_readable` methods that render to a JSON-friendly dict — for APIs and debugging, not for the wire.)
- **Versioning is manual.** Adding a field means bumping the version byte and teaching the parser the new layout — there is no schema-evolution machinery doing it for you. Hathor's `version` field and the optional trailing `headers` (§26.5.2) are the deliberate seams that make this manageable, but it is still hand-work.

The trade is the classic systems trade: a bespoke binary format costs developer effort and readability to buy determinism, density, and bounded speed. For data whose hash is its identity and whose encoding is part of the consensus rules, that trade is not close — the things you give up are conveniences, and the thing you gain is correctness.

26.7 How it plugs into the lifecycle

Serialization sits at every boundary where a vertex crosses out of, or into, the node's memory.

- **Creating and signing.** When the wallet builds a transaction, it serializes the body with each input's `data` blanked (`get_sighash_bytes`, §26.5.3) to produce the sighash the owner signs (Chapter 25) — serialization is already in play before the vertex is even complete.
- **Hashing and mining.** The proof-of-work nonce is searched against the hash of the serialized bytes; mining a block is, mechanically, re-serializing with different nonces until the hash meets the target (Chapter 9, Chapter 37).
- **Sending to peers.** When the node ships a vertex over a P2P connection, it sends `bytes(vertex)` and the peer reconstructs it with `VertexParser.deserialize` (Chapter 34). The two nodes agree on the object precisely because they agree on the bytes.
- **Writing to and reading from storage.** When a vertex is persisted, it is stored as these bytes and read back through the same parser. That is the subject of the next chapter — the format you just learned is exactly what lands in RocksDB.

- **APIs.** Read-only surfaces (the JSON/HTTP and WebSocket feeds) expose a vertex both as a hex string of its serialized bytes and, via `to_human_readable`, as a friendly dict — decoded through the same paths.

At every one of these boundaries, the invariant from the chapter's opening holds: the bytes are canonical, so the hash is canonical, so every node agrees.

Recap

Decision (§26.1)	Hathor's answer	Where
How numbers become bytes	big-endian throughout; fixed-width for known ranges, LEB128 varint in the framework	<code>transaction/util.py:32</code> ; <code>base_transaction.py:72</code> ; <code>encoding/int.py:42</code> ; <code>encoding/leb128.py:54</code>
Framing variable-length data	length-prefix everything (2-byte fixed length for scripts; 1-byte counts for lists; LEB128 in the framework)	<code>TxInput.__bytes__</code> <code>base_transaction.py:959</code> ; <code>TxOutput.__bytes__</code> :1069 ; <code>bytes.py:67</code>
Vertex field order	funds struct (signal_bits, version, counts, tokens/in/out) then graph struct (weight, timestamp, parents) then nonce then headers	<code>transaction.py:205</code> ; <code>base_transaction.py:442</code> , :477
Which kind of object	the version byte at <code>data[1]</code> , dispatched through <code>TxVersion.get_cls()</code>	<code>vertex_parser.py:53</code> ; <code>base_transaction.py:115</code>
The two cursors (framework)	<code>Serializer</code> (write_) / <code>Deserializer</code> (read_/peek_*), with free <code>encode_*</code> / <code>decode_*</code> functions	<code>serializer.py:32</code> ; <code>deserializer.py:32</code>
Coin value codec	4 bytes if $\leq 2^{31}-1$, else 8 bytes negated; the sign bit is the flag	<code>serialization/encoding/output_value.py:89</code>
Nonce size	4 bytes for a <code>Transaction</code> , 16 bytes for a <code>Block</code>	<code>transaction.py:58</code> ; <code>block.py:46</code>
Why bespoke binary	determinism for hashing/consensus, compactness, bounded parsing — at the cost of hand-maintenance and no readability	§26.6

Serialization is where Chapter 25's objects stop being Python and become the protocol. A vertex's bytes are produced by a hand-written, positional, big-endian format in `hathor/transaction/` — a funds struct, a graph struct, a nonce, and optional headers — with `TxInput` and `TxOutput` laying out their own bytes and a one-bit trick keeping coin values compact. `VertexParser` reads those bytes back into the right subclass by dispatching on the version byte. Alongside it, the newer `hathor/serialization/` framework offers a reusable `Serializer / Deserializer` toolkit, built from small `encode_*` / `decode_*` functions, that the rest of the node is migrating onto. The format is bespoke and binary, not JSON or Protobuf, for one reason above all the others: the hash is taken over these exact bytes, so the encoding must be canonical and reproducible on every node — determinism is the whole game. With the bytes of a vertex now in hand, the next chapter follows them to their resting place: Chapter 27, persistence, where these byte strings are written to and read from RocksDB.

1. A byte is eight bits, holding an integer from 0 to 255. All files and network data are sequences of bytes; everything else — text, numbers, objects — is an interpretation layered on top of bytes by agreement. ↩
2. Endianness is the order in which the bytes of a multi-byte number are laid down. Big-endian puts the most-significant byte first (the way we write decimal); little-endian puts the least-significant byte first. Neither is more correct; writer and reader need only agree. Hathor uses big-endian everywhere. ↩
3. A varint (variable-length integer) is an integer encoding that uses fewer bytes for smaller numbers — one byte for small values, more only as needed — instead of a fixed width. It saves space when most values are small. ↩
4. LEB128 ("Little Endian Base 128") is the standard varint scheme Hathor's framework uses: the number is split into 7-bit groups, each stored in the low 7 bits of a byte, with the high bit of each byte acting as a "more bytes follow" flag. The reader stops at the first byte whose high bit is 0. It is the same scheme used by WebAssembly and the DWARF debug format. ↩
5. An abstract base class (ABC) is a class you cannot instantiate directly; it declares methods (often `@abstractmethod`) that concrete subclasses must implement. `Serializer` and `Deserializer` are ABCs, with `BytesSerializer / BytesDeserializer` as the concrete in-memory implementations. Full treatment in Chapter 1. ↩
6. Dispatch means selecting which piece of code to run based on a value at runtime — here, choosing a vertex class based on the version byte. Full treatment of dispatch as a pattern is in Chapter 4. ↩
7. Protobuf (Protocol Buffers) is Google's schema-driven binary serialization tool: you write a `.proto` schema and it generates reader/writer code. It is compact and convenient, but its output is not guaranteed to be byte-for-byte canonical across implementations, which is why it is unsuitable for data whose hash must match on every node. ↩

Chapter 27 — Persistence: Storage & RocksDB

What you'll learn in this chapter

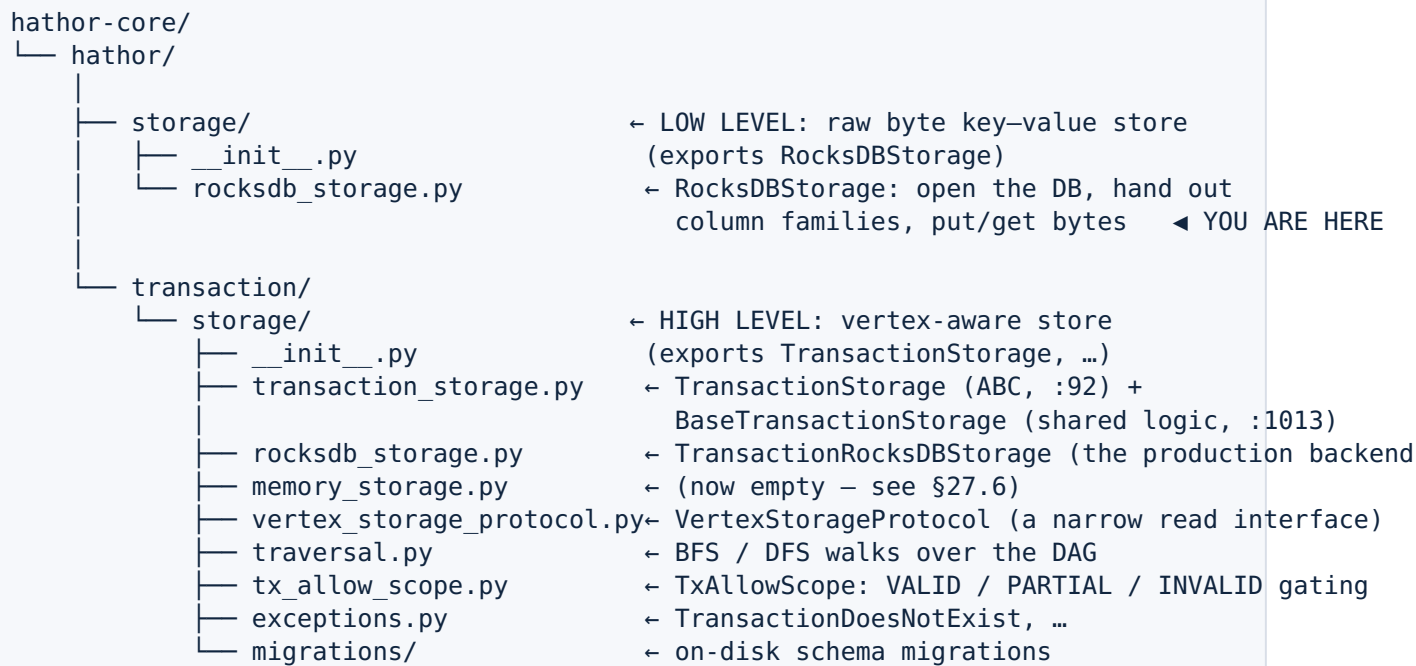
- What an **embedded key-value store** is, and what makes **RocksDB** one — the LSM-tree, column families, and ordered keys — at a level you can reason about without reading C++.
- How `hathor-core` layers a low-level byte store (`hathor/storage/`) under a **vertex-aware** store (`hathor/transaction/storage/`), and why that two-layer split exists.
- How a single vertex is written and read back: which bytes go where (the body, the mutable metadata, the static metadata — three slots), and how the bytes become a live `Block` or `Transaction` again.
- How the storage **walks the DAG** (BFS/DFS helpers), what an **allow-scope** is, and how a stored flag lets the node detect that it crashed last time.
- The decision itself: **why RocksDB and not MongoDB, not a SQL database, not plain files** — with the trade-offs stated honestly.

By now you have met the data and its shape. Chapter 25 defined the **vertex model** in code — `BaseTransaction` and its subclasses `Block` and `Transaction`, their inputs and outputs, and the mutable per-vertex **metadata**. Chapter 26 showed how a vertex is turned into a compact stream of bytes and back (the bespoke serialization). This chapter is the next link in that chain: once you have the bytes, **where do they go, and how do you get them back fast?** That is persistence, and in `hathor-core` it rests on an embedded database called RocksDB.

This is the book's canonical treatment of RocksDB. Earlier chapters dropped a one-line footnote ("an embedded key-value store"); here we open it up.

27.1 Localization

Persistence lives in **two** packages, deliberately stacked. The lower one knows nothing about Hathor; it speaks raw bytes. The upper one knows everything about vertices.



The naming collision is worth flagging once so it never trips you: there are **two** files called `rocksdb_storage.py`. `hathor/storage/rocksdb_storage.py` is the byte layer (class `RocksDBStorage`). `hathor/transaction/storage/rocksdb_storage.py` is the vertex layer (class `TransactionRocksDBStorage`). The first does not import or know about transactions; the second owns an instance of the first.

Recap — the module map (full treatment in Ch. 0 §0.4). Chapter 0 grouped the packages by the five jobs of a full node. The very first job was store the ledger: " `storage/` is a thin wrapper around RocksDB ... `transaction/storage/` sits on top and speaks in vertices." This chapter is the zoom-in on that one sentence. The derived lookups — "what does this address own?" — are a separate job, handled by `indexes/` in Chapter 28.

Context. Everything else in the node is downstream of storage. Verification reads parents from it; consensus reads and writes metadata to it; the sync agent fills it; the indexes are rebuilt from it; the API serves answers out of it. If storage is wrong, the whole node is wrong — which is exactly why the layer also carries the crash-safety flag (§27.8). It is the one place where "the ledger" stops being an idea and becomes files on a disk.

27.2 What it does and why it exists

Strip away the vocabulary and the storage layer answers four blunt questions:

1. **Where do I put a vertex so it survives a restart?** A node may run for months and then be killed, rebooted, or crash. Everything it learned — every block and transaction it validated — must still be there when it comes back. RAM is not enough; the data must hit a disk.
2. **How do I find one vertex again, fast, out of millions?** The node is constantly asked "do you have the vertex with hash `0xabc...`?" and "give me its parents." With tens of millions of vertices, scanning is out of the question. Lookup by hash must be effectively instant.
3. **How do I store the bookkeeping that changes over time?** A vertex's own contents (its inputs, outputs, parents) never change after creation — they are fixed by the hash. But the node's opinion about that vertex — its accumulated weight, whether it has been voided, its height — changes as the ledger grows. That mutable part is the **metadata**, and it must be stored and updated separately from the immutable body. (Chapter 25 split the metadata in two: a static part computed once at creation and a mutable part rewritten as the ledger evolves; storage keeps both, in their own places — §27.6.)
4. **How do I walk the graph?** Many questions ("what confirms this transaction?", "trace funds back to their source") are answered by following edges through the DAG. The storage layer must let callers traverse from a vertex to its parents, children, and the vertices that spend it.

A full node cannot function without all four. The storage package exists to provide them, and to do so with one more property the others depend on: **integrity**. A half-written record after a power cut must not silently corrupt the ledger. That requirement is most of why the node uses a real database engine instead of writing files by hand.

27.3 The concepts it rests on

Four ideas from earlier chapters carry directly into this one. We recap them briefly, then build the one genuinely new concept — what RocksDB actually is.

Recap — abstraction & interchangeable backends (full treatment in Ch. 1 & Ch. 5). An **abstract base class**¹ declares what operations exist (`save_transaction` , `get_transaction`) without committing to how. Concrete subclasses fill in the how. Because callers depend only on the abstract type, you can swap one implementation for another and nothing upstream notices. Hathor uses this directly: `TransactionStorage` is the abstract contract; the production code stores to RocksDB while tests can use a lighter, in-process variant — and the manager, verification, and consensus are written against the abstract type, never against RocksDB by name.

Recap — the vertex model (full treatment in Ch. 25). A `vertex` is any node of Hathor's ledger graph — a `Block` or a `Transaction` , both subclasses of `BaseTransaction` . Each vertex has an **immutable body** (version, inputs, outputs, parents, nonce) fixed at creation, and a **mutable metadata** object (`TransactionMetadata`) the node maintains as the ledger evolves — height, accumulated weight, `voided_by` , validation state. Hold this split: storage persists the two parts in two different places.

Recap — serialization (full treatment in Ch. 26). Serialization turns a live object into a flat sequence of bytes; deserialization reverses it. Hathor uses a custom compact binary format for vertex bodies. A database stores bytes, so serialization is the step that happens just before a write and just after a read. In this chapter you will see exactly where: `bytes(tx)` for the body on the way in, `VertexParser.deserialize(...)` on the way out, and `TransactionMetadata.to_bytes()` / `.from_bytes()` for the metadata.

Recap — DAG traversal (full treatment in Ch. 8). The ledger is a directed acyclic graph. Two standard ways to visit every reachable node are **breadth-first search** (BFS — explore neighbours level by level, using a queue) and **depth-first search** (DFS — plunge down one path before backtracking, using a stack). Hathor's storage ships both, plus a BFS variant that visits in timestamp order. We see them in §27.7.

The new idea: an embedded key-value store

Two words need unpacking before RocksDB makes sense: **embedded** and **key-value**.

Key-value store. The simplest possible database. You hand it a key (some bytes) and a value (some bytes), and it remembers the pairing. Later you hand it the same key and it returns the value. That is the entire interface: `put(key, value)` , `get(key)` , `delete(key)` . There are no tables, no columns, no `SELECT ... WHERE` , no joins. If a Python `dict` could survive a reboot and hold more data than fits in RAM, it would be a key-value store. The mental model is exactly that: **a dictionary that lives on disk**.

Embedded. The database runs inside your own program's process, as a library you import — not as a separate server you connect to over a network. When `hathor-core` calls `put`, it is a function call into linked C++ code, not a request sent to another machine (or even another process). Contrast a server database like PostgreSQL or MongoDB: there, the database is its own long-running program; your code opens a socket to it and speaks a protocol. Embedded means no socket, no protocol, no second program to install, start, secure, or keep alive. SQLite is the famous embedded SQL database; RocksDB is an embedded key-value one.

RocksDB, then, is an embedded, ordered key-value store written in C++, originally built at Facebook from Google's LevelDB. "Ordered" is the one extra property beyond the dictionary model, and it earns its keep below. Three of its mechanisms matter to us.

The LSM-tree. Most of the reason a database is hard to write from scratch is making writes both fast and durable. A naïve approach — open the file, seek to the right spot, overwrite a few bytes — is slow, because disks (especially spinning ones) hate random seeks, and dangerous, because a crash mid-write leaves a torn record. RocksDB uses a **log-structured merge-tree**². The idea: never overwrite in place. New writes are appended to an in-memory table (and a write-ahead log on disk for durability). When that table fills, it is flushed to disk as a new, immutable, sorted file. Reads check the newest files first. Periodically a background process called **compaction**³ merges the sorted files together, discarding superseded values. The payoff: writes are sequential appends (fast and crash-safe), and reads stay fast because the files are sorted and can be binary-searched. The cost: a value may live in several files until compaction tidies up, so reads occasionally do more work, and compaction consumes background CPU and disk. For a node that writes constantly (every new vertex, every metadata update) this trade is favourable.

Column families. A single RocksDB database can be sliced into named sub-stores called **column families**⁴. Think of them as separate dictionaries that share one database file, one transaction log, and one open handle — but keep their own key-spaces and can be tuned and compacted independently. They are how Hathor avoids key collisions and keeps unlike data apart: the vertex bodies live in one column family, their metadata in another, even though both are keyed by the same 32-byte hash. Without column families you would have to smear a prefix onto every key to keep the two namespaces from clashing.

Ordered keys. Unlike a hash map, a RocksDB column family keeps its keys in sorted byte order, and lets you iterate them in that order (and seek to a position). For lookup-by-hash this ordering is irrelevant — a hash is random, so its sort order is meaningless. But it is what makes "give me every vertex" (a full scan, used at startup to rebuild indexes) a clean sequential read, and it is the property the indexes of Chapter 28 exploit heavily.

27.4 The code, walked — a tiny generic store first

Before the real classes, here is the whole idea in miniature: a persistent dictionary with a put/get interface and a serialization step on each side. No Hathor, no RocksDB — just the shape.

```
import json

class TinyStore:
    """A dict-on-disk: keys are bytes, values are bytes."""
    def __init__(self):
        self._db = {}  # pretend this survives a reboot

    def put(self, key: bytes, value: bytes) -> None:
        self._db[key] = value

    def get(self, key: bytes) -> bytes | None:
        return self._db.get(key)

class AccountStore:
    """A 'vertex-aware' layer: speaks Accounts, not bytes."""
    def __init__(self, backend: TinyStore):
        self._backend = backend

    def save(self, account) -> None:
        key = account.id.encode()
        value = json.dumps(account.to_dict()).encode()  # serialize
        self._backend.put(key, value)

    def load(self, account_id: str):
        value = self._backend.get(account_id.encode())
        if value is None:
            raise KeyError(account_id)
        return Account.from_dict(json.loads(value))  # deserialize
```

Every concept of the real layer is already here. `TinyStore` is the byte layer: it knows keys and values, nothing else. `AccountStore` is the domain layer: it knows how to turn an `Account` into bytes (`to_dict` → `json.dumps`) and back (`json.loads` → `from_dict`), and it picks the key (`account.id`). Swap `TinyStore` for a `RedisStore` or a `RocksDBStore` and `AccountStore` does not change a line — that is the interchangeable-backend pattern. Hold this picture; the production code is the same skeleton with more muscle.

27.5 The byte layer: `RocksDBStorage`

The low layer is small — about a hundred lines in `hathor/storage/rocksdb_storage.py`. Its whole job is: open one RocksDB database and hand out column families.


```
# hathor/storage/rocksdb_storage.py
class RocksDBStorage:
    """ Creates a RocksDB database
        Give clients the option to create column families
    """
    def __init__(self, path, cache_capacity=None) -> None:
        self.log = logger.new()
        # keep a reference to the TemporaryDirectory; it is cleaned up when GC'd
        self.path, self.temp_dir = self._get_path_and_temp_dir(path)
        db_path = os.path.join(self.path, _DB_NAME) # _DB_NAME = 'data_v2.db'
        ...
        self._db = rocksdb.DB(db_path, options, column_families=column_families)
```

A few details deserve a junior-level gloss.

Opening the database (`rocksdb_storage.py:69`). The constructor builds an `Options` object (`rocksdb_storage.py:44`) and opens the database at `<path>/data_v2.db`. The options it sets are operational tuning, not correctness: an 80 MB write buffer instead of the 4 MB default (`rocksdb_storage.py:46`), memory-mapped reads and writes (`:48 - :49`), **no compression** (`:47` — Hathor trades disk space for CPU), and a 3 GB cap on the write-ahead log so the `.log` files cannot grow without bound (`:53`). You do not need to memorize these; the point is that the byte layer is also where disk-engine policy is decided, once, for the whole node.

Discovering existing column families (`rocksdb_storage.py:57 - :67`). RocksDB will not open a database without being told the names of every column family it already contains. So the code first lists them (`list_column_families`, `:59`); if that throws — meaning the database does not exist yet — it calls `repair_db` to create a fresh one and starts with an empty list (`:60 - :63`). Then it opens all of them.

Creating a column family on demand (`rocksdb_storage.py:93`):

```
def get_or_create_column_family(self, cf_name: bytes) -> 'rocksdb.ColumnFamilyHandle':
    cf = self._db.get_column_family(cf_name)
    if cf is None:
        cf = self._db.create_column_family(cf_name, rocksdb.ColumnFamilyOptions())
    return cf
```

This is the seam between the two layers. The vertex layer does not open the database; it asks the byte layer for the named column families it wants, creating them the first time. `get_db()` (`:90`) hands back the raw handle so the upper layer can issue the actual `put / get / delete` calls.

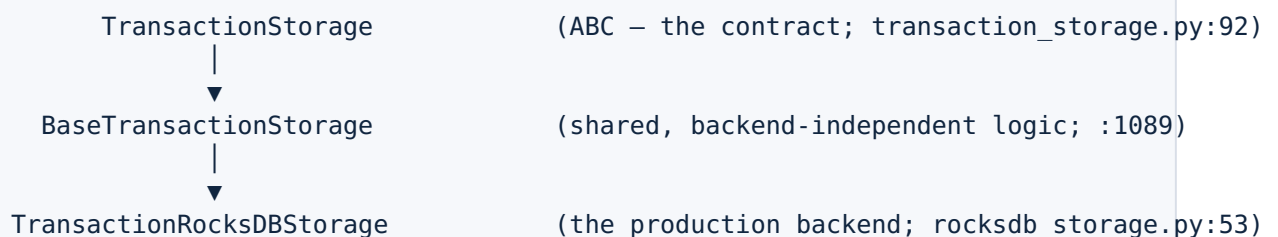
The temp path. `create_temp()` (`rocksdb_storage.py:73`) builds a `RocksDBStorage` backed by a `tempfile.TemporaryDirectory`. The comment at `:38` explains the one subtlety: the object must keep a reference to that `TemporaryDirectory`, because Python deletes the directory when the object is garbage-collected. This is what powers a real-

but-throwaway database — used in tests and by the `--temp-data` CLI flag (Chapter 24) — without spinning up anything different from production. It is the production code path, pointed at scratch space.

That is the entire byte layer. It speaks `(column_family, key_bytes) → value_bytes` and has no idea what a transaction is.

27.6 The vertex layer: `TransactionStorage` and its backends

On top of the byte layer sits the vertex-aware store. Here the class structure matters, so let us name it precisely.



`TransactionStorage` (`transaction_storage.py:92`) is the **abstract base class** — the contract every backend must honour: `save_transaction` (`:425`), `get_transaction` (`:541`), `transaction_exists` (`:490`), `_get_transaction` (`:510`), traversal, allow-scope, and many more. Its own docstring is a candid bit of legacy: "Legacy sync interface, please copy @deprecated decorator when implementing methods" (`:93`).

`BaseTransactionStorage` (`transaction_storage.py:1089`, declared `class BaseTransactionStorage(TransactionStorage)`) sits between the contract and the backend. It implements the parts that are the same regardless of where bytes land — the allow-scope validation (`pre_save_validation:442`, `post_get_validation:456`, both calling `_validate_transaction_in_scope:474`), the weak-reference identity map, the genesis handling (`_save_or_verify_genesis:333`), and the index bookkeeping — and leaves the genuinely storage-specific holes (`_get_transaction`, `transaction_exists`, `_save_static_metadata`, `save_transaction`, the migration accessors) as `abstractmethod`s for the backend to fill.

`TransactionRocksDBStorage` (`rocksdb_storage.py:53`) is the concrete backend that fills those holes against RocksDB. In production this is always the one in use.

One detail worth fixing now, because it shapes the read/write code below: the storage keeps an in-memory **identity map** — a `WeakValueDictionary` keyed by hash (`transaction_storage.py:138`). Its purpose is correctness, not speed: it guarantees that at any moment there is at most one live Python object for a given vertex (`_save_to_weakref:352` even asserts this), so two parts of the node that both "load" the

same transaction get the same object and see each other's metadata updates. (A weak reference lets the object be garbage-collected once nobody else holds it.) Separately, the RocksDB backend keeps a real **write-back cache** for throughput, covered next.

About the "memory backend." The book's chapter brief — and the orientation in Chapter 0 — describe an in-memory storage used in tests as the classic second backend proving the abstraction. That pattern is real and the abstraction is genuine, but note the current state of the tree: `hathor/transaction/storage/memory_storage.py` exists but is **empty (0 bytes)**, and the package's `__init__.py` exports only `TransactionStorage`, `TransactionRocksDBStorage`, and `VertexStorageProtocol`. So on this branch the interchangeable-backend design is expressed mainly through (a) the `TransactionStorage` ABC plus `BaseTransactionStorage`, and (b) the throwaway RocksDB created by `create_temp()` / `--temp-data`, which gives tests a real backend on scratch disk rather than a separate in-RAM class. The seam to add another backend is still there — implement the abstract methods — but a populated memory backend is not present here. (Stated plainly because the brief expected one; the code is the authority.)

Saving a vertex

The public `save_transaction` is abstract on the base class, but the base supplies the shared front-half — `pre_save_validation` (`transaction_storage.py:442`, which runs the allow-scope check `_validate_transaction_in_scope:474`) and the static-metadata write (`_save_static_metadata:435`). The RocksDB subclass calls that base method, then writes the body and mutable metadata:

```
# hathor/transaction/storage/rocksdb_storage.py (TransactionRocksDBStorage)
def save_transaction(self, tx, *, only_metadata=False) -> None:           # :202
    super().save_transaction(tx, only_metadata=only_metadata)             # :203 base checks + st
    self._save_transaction(tx, only_metadata=only_metadata)               # :204 into the write-b
    self._save_to_weakref(tx)                                              # :205 into the identit

def _save_transaction(self, tx, *, only_metadata=False) -> None:         # :207
    self._update_cache(tx)                                                 # keep the live object
    self.cache_data.dirty_txs.add(tx.hash)                                # mark it "needs writin

def _save_transaction_to_db(self, tx) -> None:                             # :211 (runs on flush)
    key = tx.hash
    self._db.put((self._cf_tx, key), self._tx_to_bytes(tx))              # :214 body → cf_tx (bi
    meta = tx.get_metadata(use_storage=False).to_bytes()                  # :215 mutable metadata
    self._db.put((self._cf_meta, key), meta)                              # :216 metadata → cf_met

def _save_static_metadata(self, tx) -> None:                               # :218 (called by the
    self._db.put((self._cf_static_meta, tx.hash), tx.static_metadata.json_dumpb()) # :220 stati
```

This is the heart of the chapter, so trace it slowly.

- **The key is the hash.** The body, the mutable metadata, and the static metadata are all stored under `tx.hash` — the 32-byte identifier of the vertex (Chapter 25) — each in its own column family. The hash is the primary key; there is no separate id, no auto-increment, nothing to look up first. This is the

access pattern that makes a key-value store the right tool: the node almost always already knows the hash of what it wants.

- **Three values, three slots.** The immutable body is encoded with `bytes(tx)` (via `_tx_to_bytes`, :178 / :213) — Hathor's compact binary serialization from Chapter 26 — into `_cf_tx`. The mutable metadata is encoded with `TransactionMetadata.to_bytes()` (:215) into `_cf_meta`. The static metadata is encoded with `static_metadata.json_dumps()` (:220) into `_cf_static_meta`. The three are kept apart on purpose, mirroring Chapter 25's split: the body is fixed forever, the static metadata is computed once at creation and never changes, and the mutable metadata is rewritten constantly by consensus.
- **only_metadata.** Consensus updates metadata constantly (a new vertex changes the accumulated weight and voided status of old ones) without touching their bodies. The `only_metadata=True` flag is threaded through so a metadata-only update can avoid re-encoding the body. Since the body never changes after creation, rewriting it would be wasted work.

The write-back cache. Notice that `save_transaction` does not immediately hit the disk. `_save_transaction` (:207) only puts the vertex in an in-memory LRU cache (`cache_data.cache`, an `OrderedDict`, configured by `CacheConfig` at `transaction_storage.py:75`) and adds its hash to a `dirty_txs` set. The real disk write (`_save_transaction_to_db`, :211) happens later, in a background flush: a timer (`_start_flush_thread`, :111) periodically uses Twisted's `deferToThread` (Chapter 16) to write all dirty vertices off the reactor thread (`_flush_to_storage`, :127); an evicted-but-dirty vertex is also flushed immediately on eviction (`_cache_popitem`, :138). This is a classic write-back cache: batch writes for throughput, but never lose an update. (It is also why `is_empty` (:305) and `_get_all_transactions` (:287) call `_flush_to_storage` first — they must see the latest writes.)

Reading a vertex back

The base class's `get_transaction` (`transaction_storage.py:541`) applies the allow-scope check (`post_get_validation:456`), then delegates to the abstract `_get_transaction`, which the backend implements with a three-tier lookup:

```

# hathor/transaction/storage/rocksdb_storage.py:241 (TransactionRocksDBStorage)
def _get_transaction(self, hash_bytes) -> 'BaseTransaction':
    if tx := self.cache_data.cache.get(hash_bytes): # :242 1) write-back cache?
        ...; return tx
    if tx := self.get_transaction_from_weakref(hash_bytes): # :248 2) live identity map?
        ...; return tx
    tx = self.get_transaction_from_db(hash_bytes) # :253 3) finally, the disk
    if not tx:
        raise TransactionDoesNotExist(hash_bytes.hex()) # :255
    ...; return tx

def _get_transaction_from_db(self, hash_bytes) -> Optional['BaseTransaction']: # :266
    tx_data = self._db.get((self._cf_tx, hash_bytes)) # :268 fetch body bytes
    meta_data = self._db.get((self._cf_meta, hash_bytes)) # :269 fetch metadata bytes
    if tx_data is None:
        return None # :271
    tx = self._load_from_bytes(tx_data, meta_data) # :273 reassemble body + mutable meta
    self._load_static_metadata(tx) # :274 attach static meta (cf_static_...)
    return tx

def _load_from_bytes(self, tx_data, meta_data) -> 'BaseTransaction': # :170
    tx = self.vertex_parser.deserialize(tx_data) # :173 bytes → vertex
    tx._metadata = TransactionMetadata.from_bytes(meta_data) # :174 bytes → mutable met
    tx.storage = self # :175 back-pointer
    return tx

```

Reading is the mirror of writing, and it is where Chapter 26 pays off. A lookup checks the **write-back cache** first (:242), then the **weakref identity map** (:248 — so a vertex already live elsewhere is reused, not duplicated), and only then the **disk** (:253). On a disk read, the body bytes go through `vertex_parser.deserialize(...)` (:173) — the `VertexParser` reads the version byte, dispatches to the right class, and rebuilds a live `Block` or `Transaction` (Chapter 25/26). The mutable metadata bytes are parsed with `TransactionMetadata.from_bytes(...)` (:174); the static metadata is loaded separately from its own column family by `_load_static_metadata` (:222 , using `VertexStaticMetadata.from_bytes`). Finally the reconstructed vertex is given a back-pointer to the storage (`tx.storage = self` , :175) so that it can later fetch its own parents and children on demand — which is how traversal works without the walker holding the whole graph in memory.

If the body key is missing, `_get_transaction` raises `TransactionDoesNotExist` (:255). `transaction_exists` (:232) answers the cheaper question directly: it checks the cache, then RocksDB's `key_may_exist` bloom-filter probe ⁵, then confirms with a real `get`.

Key design and column families, summarized

one RocksDB database (data_v2.db)

cf	b'tx'	key = vertex hash (32 B)	value = bytes(tx)
cf	b'meta'	key = vertex hash (32 B)	value = TransactionMetadata
cf	b'static-meta'	key = vertex hash (32 B)	value = VertexStaticMetadata
cf	b'attr'	key = attribute name	value = small string
cf	b'migrations'	key = migration name	value = state byte

[binary b...]
[mutable M...]
[fixed-at...]
[node-level...]
[schema ve...]

There are **five** column families; their names are constants at `rocksdb_storage.py:46 - :50` (`_CF_NAME_TX`, `_CF_NAME_META`, `_CF_NAME_STATIC_META`, `_CF_NAME_ATTR`, `_CF_NAME_MIGRATIONS`), resolved to handles in the constructor (`:72 - :76`). The split of metadata into two families — `meta` (mutable) and `static-meta` (immutable) — is the on-disk expression of Chapter 25's static-vs-mutable metadata distinction. The `attr` family is a small general-purpose key→string store the upper layer exposes through `add_value` / `get_value` / `remove_value` (`rocksdb_storage.py:338 / :344 / :341`) — this is where the node-level flags of §27.8 live. The `migrations` family records, per named migration, whether it has run (`get_migration_state` / `set_migration_state`, `:181 / :188`), so an upgraded node can fix up an old on-disk layout once and remember it did — the migration runner itself lives on the base class (`_check_and_apply_migrations`, `transaction_storage.py:214`).

The narrow read interface: `VertexStorageProtocol`

Not every caller needs the full storage. Verification, for instance, only ever reads a handful of things. `vertex_storage_protocol.py:22` defines a **Protocol**⁶ — a structural interface — exposing just four read methods: `get_vertex`, `get_block`, `get_parent_block`, `get_best_block_hash`. Its docstring (`:23`) explains the intent: a verification method can accept "a RocksDB storage or an ephemeral simple memory storage" — anything shaped like these four methods. This is the minimal-surface idea from Chapter 5: depend on the smallest interface that does the job, so more kinds of object can satisfy it.

27.7 Walking the DAG, and the allow-scope

Traversal helpers

`traversal.py` provides three ready-made walkers over the graph, all subclasses of one `GenericWalk` base (`traversal.py:54`): `BFSTimestampWalk` (`:182`, BFS ordered by timestamp via a heap), `BFSOrderWalk` (`:218`, plain BFS via a queue), and `DFSWalk` (`:249`, DFS via a stack). The three differ only in their to-visit container — a heap, a deque, or a list — which is the textbook distinction between BFS and DFS made concrete: **same algorithm, different bag for the pending work**.

Two design choices make these walkers usable across the whole node.

They walk over a `VertexStorageProtocol`, not a concrete store (`traversal.py:61`). A walker fetches each neighbour with `self.storage.get_vertex(_hash)` (`:129`). Because it depends only on the narrow protocol, the same traversal code runs over the real RocksDB store or any stand-in.

They distinguish the two kinds of edge Chapter 8 insisted on. A vertex has verification edges (parents/children — the confirmation DAG) and funds edges (inputs/spent-outputs — the money DAG). The walker's `is_dag_verifications` and `is_dag_funds` flags (`traversal.py:107`, `:113`) pick which edges to follow, and `is_left_to_right` picks the direction (toward the tips, or back toward genesis). So one mechanism answers both "what confirms this?" and "where did these funds come from?".

The control flow is cooperative: `run()` (`:148`) yields each vertex to the caller, and the caller decides per-vertex whether to keep going via `add_neighbors()` or stop via `skip_neighbors()` (`:136`, `:142`). That lets a caller prune branches it does not care about instead of walking the entire reachable graph.

Allow-scope: gating by validation state

Recap — validation state (full treatment in Ch. 25). Every vertex carries a validation state in its metadata. The three that matter here: **VALID** (fully verified and connected), **PARTIAL** (known but not yet fully verified — common mid-sync), and **INVALID**. Sync can store a vertex before all its dependencies have arrived, so the storage must sometimes hold not-yet-valid data.

That creates a hazard: most of the node should only ever see fully-valid vertices, but during startup and sync the storage legitimately contains partial ones. The **allow-scope** is the guard. `TxAllowScope` (`tx_allow_scope.py:25`) is a bit-flag enum — `VALID`, `PARTIAL`, `INVALID`, and `ALL` (`:31` – `:34`) — and the storage holds a "current scope," defaulting to `VALID` only (`transaction_storage.py:152`). The enum's `is_allowed(tx)` method (`tx_allow_scope.py:36`) checks a vertex's validation state against the scope; the storage's `save_transaction` and `get_transaction` both run it (via `_validate_transaction_in_scope`, `transaction_storage.py:474`), and a vertex outside the scope is treated as if it does not exist — the storage raises `TransactionNotInAllowedScopeError` (`exceptions.py:50`), a subclass of `TransactionDoesNotExist`.

The scope is switched with a context manager so it always reverts:

```
# hathor/transaction/storage/tx_allow_scope.py:52
@contextmanager
def tx_allow_context(tx_storage, *, allow_scope):
    previous_allow_scope = tx_storage.get_allow_scope()
    try:
        tx_storage.set_allow_scope(allow_scope)
        yield
    finally:
        tx_storage.set_allow_scope(previous_allow_scope)
```


This is the "scope" Chapter 0 §0.3 mentioned in the startup story. You can see it explicitly in the manager: just before initializing components it widens the scope (`manager.py:315`, `set_allow_scope(VALID | PARTIAL | INVALID)`) so it can load and connect partial data, and immediately after it narrows back to valid-only (`manager.py:317`). The context-manager form guarantees the relaxed window always closes, because `finally` restores the previous value even on error. The base class also exposes convenience wrappers — `allow_partially_validated_context()` / `allow_invalid_context()` (`transaction_storage.py:386` / `:394`) and the query `is_only_valid_allowed()` (`:402`) — so callers can widen the scope locally or ask which regime they are in.

27.8 Crash safety — the flag in the database

A full node must not act on a half-built database. If the previous run died mid-write — power cut, `kill -9`, an unhandled exception in consensus — the on-disk metadata may be inconsistent, and trusting it could corrupt the ledger going forward. Hathor guards against this with a flag stored in the database itself.

The mechanism is the small `attr` column family from §27.6. Two attribute keys carry the state, declared on the abstract base: `_manager_running_attribute = 'manager_running'` (`transaction_storage.py:106`) and `_full_node_crashed_attribute = 'full_node_crashed'` (`:109`). The storage sets, clears, and reads these flags through the generic attribute interface (`add_value` / `get_value` / `remove_value`, `rocksdb_storage.py:338` / `:344` / `:341`).

The manager wires them into startup. The first lines of `HathorManager.start` check both flags before doing anything else:

```
# hathor/manager.py:285
if self.tx_storage.is_full_node_crashed():
    self.log.error("... it wasn't stopped correctly. The storage is not reliable anymore ...")
    sys.exit(-1)                                     # :291 refuse to start
if self.tx_storage.is_running_manager():             # :296 still marked "running"?
    self.log.error("... it wasn't stopped correctly ...")
    sys.exit(-1)                                     # :302 refuse to start
...
self.tx_storage.start_running_manager(self._execution_manager) # :343 set the "running" flag
```

If a previous run set the "running" flag and never cleared it (because `stop` never ran — a crash), the next start sees it still set and exits, telling the operator to discard the storage and re-sync. The comment at `manager.py:293 - :295` spells out the subtlety: only the metadata may be wrong after a crash, not the blocks and transactions themselves — but that is enough to make the bookkeeping untrustworthy. The split is deliberate: the policy ("crashed → refuse to start") lives in the manager (Chapter 29), but the durable bit it reads lives in storage, because storage is the only thing that survives a crash.

The intuition is the same as a journaling filesystem's "dirty" flag, or the lock file a long-running program leaves behind: a single persisted boolean that says "I did not finish cleanly last time, so check yourself before you proceed." It costs one tiny key and saves the node from compounding a crash into corruption.

27.9 The trade-off — why RocksDB and not the alternatives

This is the decision the chapter exists to justify. The node has to persist millions of `hash → bytes` records and look them up by hash, fast, with crash safety, inside one Python process. Three families of alternative were available. Each was a worse fit, for concrete reasons.

Versus MongoDB (a document/NoSQL server). MongoDB is a separate server program: you install it, run it as its own process (often on its own machine), secure it, and your node talks to it over a network socket. For a node whose access pattern is "fetch the value at this hash," every one of those is pure overhead. You pay a network hop (serialize a request, send it, wait, receive, deserialize) for what is, with an embedded store, a local function call. You add a second daemon every operator must deploy, monitor, and keep alive — and a second thing that can be down, misconfigured, or breached. And you gain document features (ad-hoc queries, secondary indexes, aggregation) the node does not use, because it looks vertices up by hash, not by querying their contents. More moving parts, more failure modes, for capabilities the access pattern never exercises.

Versus a SQL database (PostgreSQL, MySQL). The same server objection applies — a separate process and a network hop — but the deeper mismatch is the data model. SQL's strength is the relational model: tables with typed columns, joins across them, declarative `WHERE` queries the engine optimizes. Hathor's storage needs none of that. There is no join to do; the primary access is "value at this key." Mapping a vertex onto columns would mean either decomposing it into a wide, awkward table (and reassembling it on every read) or storing the binary blob in one column — at which point you are using a relational engine as a key-value store and paying for the SQL machinery you never touch. The relational model solves a problem the node does not have, while charging for it in deployment weight and per-row overhead. (SQLite would remove the server objection — it is embedded — but not the model mismatch; you would still be using SQL to emulate `get(key)`.)

Versus plain files (one file per vertex, or a hand-rolled format). This is the tempting "just write bytes to disk myself" option, and it fails on the unglamorous parts a database already solves. You would have to implement: **atomic writes** (a crash mid-write must not leave a torn record — RocksDB's write-ahead log and LSM design give this); **ordered iteration** (the startup index rebuild and the indexes of Chapter 28 need

to scan keys in order — a directory of files does not); **fast lookup at scale** (millions of files in one directory cripples most filesystems); and **compaction** (reclaiming space from superseded metadata without rewriting everything). Each of these is a small database in disguise. RocksDB is that database, written by people who specialize in it.

What it costs, honestly. RocksDB is not free of downsides, and naming them is part of understanding the choice.

- **Key-value only.** There are no ad-hoc queries. You cannot ask the store "which outputs does this address own?" — the data is keyed by hash, and that question is about contents. The answer is the entire reason **indexes** (Chapter 28) exist: they are extra key-value tables, derived from the vertices, that pre-compute the lookups the base store cannot do. The flip side of a key-value store's speed is that every non-hash question must be indexed in advance.
- **A C++ dependency with build friction.** RocksDB is native code, reached through Python bindings. Hathor depends on a **custom git fork** of those bindings (covered in Chapter 13, Poetry) rather than a vanilla PyPI package, which means a heavier, less portable build than pure Python — the price of an embedded native engine.
- **Background compaction.** The LSM design spends CPU and disk I/O compacting in the background. For a write-heavy node this is the right trade, but it is real work happening even when the node looks idle.

The verdict the codebase reaches: the node's access pattern is lookup by hash, write constantly, scan occasionally, survive crashes, deploy as one process. An embedded ordered key-value store is the precise shape of that need, and RocksDB is a mature one. The features the alternatives add — documents, relations, a server — are exactly the features a full node does not use, and every one of them carries a cost it would rather not pay.

27.10 How it plugs into the lifecycle

Storage is among the first things to come alive and among the last to matter, because everything else reads through it.

- **Built in the composition root (Chapter 24).** The `Builder` / `CliBuilder` constructs the `RocksDBStorage` (pointed at `--data <dir>`, or at scratch space for `--temp-data`), then wraps it in a `TransactionRocksDBStorage`, passing in the `VertexParser` (for deserialization), the `IndexesManager`, the reactor, and the settings. Production is always RocksDB; the in-memory storage flag was removed (Chapter 24). The fully-wired storage is handed to the `HathorManager`.

- **Initialized in `manager.start` (Chapter 29).** On start the manager runs the crash check against the storage flag (§27.8), then loads genesis and rebuilds its in-memory view by scanning the store — during which the allow-scope is widened to `ALL` and then narrowed back (§27.7). Any pending on-disk **migrations** run here.
- **Read and written by everything after.** Verification reads parents through the narrow `VertexStorageProtocol`; the vertex handler and consensus write bodies and metadata (`save_transaction`, often `only_metadata=True`); the indexes are populated from the store and consulted by the API; the sync agent fills it as it downloads. The storage layer is the shared ground the rest of the node stands on.

Recap

Concern	Where it lives	Central type / call
Open the DB, hand out column families	<code>hathor/storage/rocksdb_storage.py</code>	<code>RocksDBStorage (:28)</code> , <code>get_or_create_column_family (:93)</code> , <code>create_temp (:73)</code>
Vertex-aware contract (the ABC)	<code>transaction/storage/transaction_storage.py:92</code>	<code>TransactionStorage</code> ; <code>BaseTransactionStorage (:1089)</code>
Production backend	<code>transaction/storage/rocksdb_storage.py:53</code>	<code>TransactionRocksDBStorage</code>
Save a vertex	same	<code>save_transaction (:202) → cache → flush _save_transaction_to_db (:211): bytes(tx) → cf_tx , to_bytes() → cf_meta</code>
Read a vertex	same	<code>_get_transaction (:241, cache→weakref→db) → _get_transaction_from_db (:266) → _load_from_bytes (:170): deserialize + from_bytes</code>
Static metadata (separate)	same	<code>_save_static_metadata (:218)</code> , <code>_load_static_metadata (:222)</code> , <code>cf_static_meta</code>

Concern	Where it lives	Central type / call
Column families (×5)	<code>rocksdb_storage.py:46 – :50</code>	<code>b'tx'</code> , <code>b'meta'</code> , <code>b'static-meta'</code> , <code>b'attr'</code> , <code>b'migrations'</code>
Narrow read interface	<code>vertex_storage_protocol.py:22</code>	<code>VertexStorageProtocol</code>
Walk the DAG	<code>traversal.py</code>	<code>BFSTimestampWalk (:182)</code> , <code>BFSOrderWalk (:218)</code> , <code>DFSWalk (:249)</code>
Gate by validation state	<code>tx_allow_scope.py:25</code>	<code>TxAllowScope</code> , <code>is_allowed (:36)</code> , <code>tx_allow_context (:52)</code>
Crash safety	<code>manager.py</code> + <code>attr</code> family	<code>is_full_node_crashed (manager.py:285)</code> , <code>is_running_manager (:296)</code> , <code>start_running_manager (:343)</code>

The storage layer is where the ledger stops being a graph in memory and becomes bytes on a disk that survive a reboot. Two stacked packages do it: a thin byte store that opens one RocksDB database and slices it into column families, and a vertex-aware store that keys every block and transaction by its hash — the immutable body in one family, the mutable metadata in another, the static metadata in a third — reassembles them on the way back out (through a write-back cache and a weakref identity map), and adds traversal helpers, validation-state gating, and a crash flag riding along. The one thing this layer deliberately cannot do is answer any question that is not "value at this hash." That limitation is the entire premise of the next chapter: **Chapter 28, Indexes** — the derived key-value tables that pre-compute the lookups RocksDB-by-hash can never answer on its own, starting with the one the storage layer cannot: what does this address own?

1. An **abstract base class** (ABC) declares methods without implementing them (marked `@abstractmethod`) so subclasses must supply the bodies. You cannot instantiate an ABC directly; it exists to define a contract. Full treatment in Chapter 1. ↩
2. An **LSM-tree** (log-structured merge-tree) is a storage design that never overwrites data in place. Writes are appended to an in-memory buffer (plus an on-disk log for safety); when the buffer fills it is flushed as a new immutable sorted file; background compaction later merges those files and discards superseded values. It makes writes fast and crash-safe at the cost of occasional extra read work and background CPU. ↩
3. **Compaction** is the LSM-tree's background housekeeping: it merges several sorted on-disk files into fewer, dropping values that have been overwritten or deleted, so reads stay fast and disk space is reclaimed. It runs even when the node is otherwise idle. ↩

4. A **column family** is a named, independent key-space inside a single RocksDB database — like a separate dictionary that shares the same database file and write-ahead log but keeps its own keys and can be tuned and compacted on its own. Hathor uses five: `tx`, `meta`, `static-meta`, `attr`, `migrations`. ↩
5. A **Bloom filter** is a compact, probabilistic membership test: it can answer "definitely not present" with certainty, or "possibly present" (with a small false-positive rate). RocksDB keeps one per data file so a lookup can skip files that certainly do not hold the key. `key_may_exist` exposes this — a fast "no" without touching the data, which is why `transaction_exists` probes it before doing a real read. ↩
6. A **Protocol** (Python `typing.Protocol`) is a structural interface: any object that happens to have the right methods satisfies it, with no explicit inheritance required ("if it walks like a duck..."). It lets a function accept any object shaped correctly, not just one named subclass. Full treatment in Chapter 5. ↩

Chapter 28 — Indexes: Fast Lookups over the Ledger

What you'll learn in this chapter

- What an **index**¹ is, in the general database sense, and the access-pattern problem it solves: storage can fetch a vertex by its hash, but cannot answer "which outputs can this address spend?" or "what block is at height 800,000?" without reading the whole ledger.
- The trade-off every index makes: it spends **disk space and update work** at write time to buy **fast answers** at read time.
- Why an index holds **no truth of its own** — it is derived state, always recomputable from the vertices in storage, which is exactly what makes it safe to rebuild after a crash.
- The real indexes Hathor keeps — **UTXO, address, tokens, height, timestamp/sorted, mempool tips**, and the info/counts and nano-contract indexes — what each one maps and who asks for it.
- How the **IndexesManager** owns them all behind one object, offered in two interchangeable backends (in-memory and RocksDB), the same abstraction pattern you met at storage.
- How indexes are **kept in sync** as vertices are executed and voided, and how they are **rebuilt from scratch** at boot when their on-disk state is stale.

The previous chapter (27) gave the node a place to put vertices and get them back: a key-value store, keyed by each vertex's hash. That answers exactly one kind of question — "give me the vertex whose id is X" — and answers it fast. But almost nothing a real user wants to know is phrased that way. A wallet asks "what is my balance?"; a block explorer asks "what is the latest block?"; the sync protocol asks "what is the tip of the chain?"; the mempool asks "which transactions are not yet confirmed?" None of these is a lookup by hash, and none of them can be answered without machinery that the bare key-value store does not have. That machinery is `hathor/indexes/`. This chapter is about the derived lookup tables the node maintains on top of storage so those questions become cheap.

28.1 Localization

`hathor/indexes/` sits in the **storage-and-lookup** group of the module map (Chapter 0, §0.4), directly above the storage layer of Chapter 27 and directly below the manager (Chapter 29) and the read-facing APIs that consume it. It depends on the vertex model (Chapter 25) and on storage; almost nothing depends on it except the manager and the APIs.

```
hathor/
├── transaction/
│   └── storage/                ← KV-by-hash: fetch a vertex by its id (Ch 27)
├── indexes/                  ◀ YOU ARE HERE – derived lookup tables over storage
│   ├── manager.py            ← IndexesManager (ABC) + RocksDBIndexesManager
│   ├── base_index.py         ← BaseIndex (ABC): the shared index interface
│   ├── scope.py              ← Scope: which vertices an index cares about (rebuild engine)
│
│   ── the real indexes (each = abstract interface + 2 backends) ──
│   ├── utxo_index.py         ← unspent outputs, keyed by (address, token)      → "what can I spend?"
│   ├── address_index.py      ← address → its transaction history                → "my history?"
│   ├── tokens_index.py       ← token uid → info + supply + holders              → "tell me about token X"
│   ├── height_index.py       ← block height → block hash (+ the height tip)     → "the chain has height N"
│   ├── timestamp_index.py    ← vertices sorted by timestamp (all/blocks/txs)    → "give me a vertex at time T"
│   ├── mempool_tips_index.py ← the tips of the unconfirmed mempool              → "what's pending?"
│   ├── info_index.py         ← global counts + first/latest timestamps          → node statistics
│   ├── nc_creation_index.py   ← nano-contract creations
│   ├── nc_history_index.py    ← per-contract call history                      ] → Ch 39
│   └── blueprint_*_index.py   ← blueprint listing + history
│
│   ── per-index backends ──
│   ├── memory_*_index.py     ← in-RAM implementations (tests, simulator)
│   └── rocksdb_*_index.py    ← persisted implementations (production)
```

Context. `hathor/indexes/` answers one question for the rest of the node: "given the ledger, look this up fast." Storage (Ch 27) can return a vertex by its hash and nothing else; this package precomputes every other way the node and its clients want to slice the ledger — by address, by token, by height, by time, by mempool membership. It is read constantly by the APIs (Ch 36/40), the wallet, the sync protocol, and mining, and it is kept current by consensus (Ch 32) and ingestion (Ch 33). Because every entry in it is derived from vertices that storage already holds, the whole package can be thrown away and rebuilt — and that is exactly what the node does at boot when an index is stale.

28.2 What it does and why it exists

Start with the problem in its most general form, before any Hathor code.

The access-pattern problem

A key-value store is a dictionary: you hand it a key, it hands you back the value. Hathor's storage is exactly that, with the key being a vertex's 32-byte hash and the value being the serialized vertex (Chapter 27). This is the right primitive — it is the one lookup that the data structure makes naturally, because a vertex is named by its hash.

But consider the questions a node is actually asked all day:

1. "How much HTR does address `H...abc` own?" — There is no field anywhere that stores a balance (Chapter 7: the UTXO model has no balances). The answer is the sum of the values of every unspent output locked to that address. To compute it from raw storage you would have to read **every transaction ever made**, check each output's script, and track which were later spent. For a ledger with millions of vertices, that is seconds-to-minutes of work for one balance query.
2. "What is the block at height 800,000?" — A block does not store "I am at height 800,000" in a way storage can search on; the hash tells you nothing about the height. From raw storage you would walk the block backbone from a known point counting blocks.
3. "Give me transactions between 9:00 and 9:05 this morning." — Storage has no notion of time-order. You would scan everything and filter.
4. "What transactions has address `H...abc` ever been part of?" — Again, a full scan.

The common shape: **the question keys on something other than the hash** (an address, a height, a timestamp, a token), and **storage is keyed only on the hash**. Answering by brute-force scan is correct but unusably slow.

The classic answer: an index

This is not a blockchain problem; it is a database problem, and databases have solved it for fifty years with **indexes**. The idea is the same as the index at the back of a book. The book's pages are stored in page order (the equivalent of "by hash" — the natural physical order). If you want to find every page that mentions "Pydantic," reading the whole book is the brute-force scan. Instead the publisher precomputes, once, a sorted list — `Pydantic → pages 18, 192, 269` — and prints it at the back. Finding the topic is now instant.

A database index is the same trade made in software: pick a question you will ask often, precompute a lookup table keyed the way that question is phrased, and keep that table updated as the data changes. You pay two ongoing costs — **extra space** (the table) and **extra work on every write** (you must update the table when the data

changes) — in exchange for turning a full scan into a direct lookup. The node's indexes are exactly this: an address-keyed table, a height-keyed table, a time-sorted table, and so on, each precomputing the answer to one family of questions.

The property that makes indexes safe: they are derived

Here is the conceptual point to hold above all others, because it explains the whole design of the package.

An index holds no truth of its own. Every entry in every index is computed from vertices that already live in storage. The height index says "height 800,000 → hash 0000ab..." — but that fact is already implied by the blocks in storage; the index merely caches it in a convenient shape. The UTXO index says "address A has these three unspent outputs" — but that, too, is fully determined by the transactions in storage. Nothing is ever recorded only in an index.

This makes an index **derived** (sometimes called secondary) state, as opposed to the **primary** state in storage. And derived state has a defining property: **it can always be recomputed from the primary state**. If an index is lost, corrupted, or never built, the node can reconstruct it by replaying the vertices in storage through the index's update logic — exactly as you could regenerate a book's back-index by re-reading the book.

Why does this matter so much? Two reasons:

- **Crash recovery.** If the node dies mid-write, an index on disk may be half-updated and therefore wrong. Because the index is derived, the safe response is not delicate repair but blunt rebuild: clear it and replay. The node does precisely this at boot (§28.5). The primary state in storage is the only thing that must survive a crash intact; the indexes are disposable.
- **It bounds the danger.** A bug in index code can give a wrong answer to a query, but it can never corrupt the ledger, because the ledger does not live in the index. The worst case is a stale lookup table, fixable by a rebuild. This is why some indexes are even allowed to be skipped entirely unless a feature needs them (§28.4).

Recap — storage / KV-by-hash (full treatment in Ch. 27). Hathor's persistence layer is a key-value store: each vertex is written under its 32-byte hash and read back by that hash. It is backed by **RocksDB** (an embedded on-disk key-value database). It is the **primary** store of the ledger — the source of truth. Its one fast operation is "fetch the vertex with this id." Everything in this chapter is built on top of it to support the other ways the node wants to look data up. → full treatment in Ch. 27.

Recap — the UTXO model (full treatment in Ch. 7). Money in Hathor is a set of discrete **unspent transaction outputs** (UTXOs). There is no stored balance and no account; an address's balance is the sum of the unspent outputs locked to it. A transaction consumes some outputs (its inputs) and creates new ones; an output that has been consumed is "spent" and no longer counts. Computing a balance therefore means finding the currently unspent outputs for an address — exactly the job of the UTXO index. → full treatment in Ch. 7.

Recap — DAG tips and the mempool (full treatment in Ch. 8 & Ch. 0). Hathor's ledger is a directed acyclic graph of vertices. The **tips** are the vertices nothing else confirms yet — the frontier of the graph. The **mempool** is the set of valid transactions the node knows about that have been seen but not yet confirmed by a block. The mempool tips are the tips that are still unconfirmed; tracking them lets the node hand out parents for new transactions and report what is pending. → Ch. 8 (DAG/tips), Ch. 0 (mempool).

Recap — interchangeable backends (full treatment in Ch. 1 & Ch. 5). When two implementations expose the same methods, code written against the shared interface works with either, and you can swap them at construction time. Hathor uses this for storage (memory vs. RocksDB) and again here: every index is an abstract interface with a `Memory...` and a `RocksDB...` implementation, and an `IndexesManager` that bundles one full set. The rest of the node never knows which it holds. This is the Liskov-substitutable interface idea from object orientation, applied to lookup tables. → Ch. 1 (ABCs/interfaces), Ch. 5 (typing).

28.3 The code, walked

28.3.1 A toy first: index a list of records by a field

Strip the idea to its bones. Suppose you have a list of payment records and you keep being asked "give me all payments from a given person." The records are stored in arrival order (the "by hash" of our toy). The naive answer scans the whole list every time:

```
records = [
    {"id": 1, "frm": "alice", "to": "bob", "amt": 30},
    {"id": 2, "frm": "alice", "to": "carol", "amt": 10},
    {"id": 3, "frm": "bob", "to": "alice", "amt": 5},
]

def payments_from(name):
    # 0(n) every call – the brute-force scan
    return [r for r in records if r["frm"] == name]
```

An **index** precomputes the answer once, keyed the way the question is asked — here, a dict from sender to the list of their record ids:

```
from collections import defaultdict

by_sender: dict[str, list[int]] = defaultdict(list)  # the index

def add_record(r):                                # update the index on every write...
    records.append(r)
    by_sender[r["frm"].append(r["id"])

def payments_from(name):                          # ...so the read is now O(1)
    return by_sender.get(name, [])
```

Three observations carry straight into Hathor:

- **The index costs space and write-work.** `by_sender` is extra memory, and `add_record` now does extra work on every insert. You paid at write time to make reads cheap.
- **The index is derived.** `by_sender` contains nothing that is not already in `records`. Delete it and you can rebuild it by replaying `records` through `add_record`. (Hold this — it is \$28.5.)
- **Removal must be handled too.** If a record can be retracted, you must also remove its id from `by_sender`, or the index will report stale answers. Every real index has an add path and a remove path — and in Hathor, an executed path and a voided path, which are add and remove under different names.

Hathor's indexes are this skeleton, scaled up: the keys are addresses, heights, timestamps and tokens; the values are vertex hashes; the backing store can be RAM or RocksDB; and "add/remove" become "a vertex was executed / a vertex was voided."

28.3.2 `BaseIndex` : the shared interface

Every index inherits from one abstract base, `BaseIndex` (`base_index.py:32`). Its own docstring states the purpose: "so we can interact with indexes without knowing anything specific to [their] implementation... to generalize how we initialize indexes and keep track of which ones are up-to-date." It is small:

```

# hathor/indexes/base_index.py:32
class BaseIndex(ABC):
    def __init__(self, *, settings: HathorSettings) -> None:
        self.settings = settings
        self.log = logger.new()

    def init_start(self, indexes_manager: 'IndexesManager') -> None:
        pass
        # optional hook; most indexes don't need it

    @abstractmethod
    def get_scope(self) -> Scope: ...
        # which vertices do I care about? (:49)

    @abstractmethod
    def get_db_name(self) -> Optional[str]: ... # my on-disk state key, or None (:54)

    @abstractmethod
    def init_loop_step(self, tx: BaseTransaction) -> None: ... # rebuild: feed me a tx (:63)

    @abstractmethod
    def force_clear(self) -> None: ...
        # wipe me before a rebuild (:69)

```

These four abstract methods are the rebuild contract, and they are worth reading as a set, because together they are the "derived, rebuildable" property made into code:

- `get_scope()` returns a `Scope` — a small declaration of which kinds of vertex this index cares about (blocks? transactions? voided ones?). The rebuild engine uses it to feed each index only the vertices relevant to it (§28.3.3, §28.5).
- `get_db_name()` returns the string under which the index records, in storage, when it was last fully built. If it returns `None`, the index keeps **no** persisted state and is therefore rebuilt on **every** startup (the in-memory mempool tips index is like this). This single return value is how the node decides, per index, whether a rebuild is needed.
- `init_loop_step(tx)` is the rebuild step: "here is one more vertex from storage; fold it into yourself." Replaying every vertex through this method reconstructs the index from scratch.
- `force_clear()` empties the index so a rebuild starts from a clean slate.

Notice what is not here: there is no `get_balance` or `get_height` on the base. Those live on the specific index interfaces (`UtxoIndex`, `HeightIndex`, ...). The base captures only what is common to all indexes — how to clear, scope, and rebuild them — so the manager can treat them uniformly.

28.3.3 Scope : declaring what a vertex an index wants

`Scope` (`scope.py:23`) is a tiny `NamedTuple` that makes "which vertices does this index care about?" into data rather than code:

```
# hathor/indexes/scope.py:23
class Scope(NamedTuple):
    include_blocks: bool
    include_txs: bool
    include_voided: bool
    topological_order: bool = False

    def matches(self, tx: BaseTransaction) -> bool: # :30
        if tx.get_metadata().voided_by and not self.include_voided:
            return False
        if tx.is_block and not self.include_blocks:
            return False
        if not tx.is_block and not self.include_txs:
            return False
        return True
```

Each index declares its scope as a module-level constant. Compare three real ones:

- The **UTXO** index uses `include_blocks=True, include_txs=True, include_voided=False` (`utxo_index.py:36`) — it cares about both kinds of vertex, but only executed ones, because a voided transaction's outputs are not spendable.
- The **address** index uses `include_voided=True` (`address_index.py:30`) — it indexes all history, voided included, because a wallet wants to see attempts that failed too.
- The **height** index uses `include_blocks=True, include_txs=False, include_voided=False` (`height_index.py:30`) — only executed blocks have a height; transactions and voided blocks are irrelevant to it.

The `topological_order` flag asks the rebuild iterator to deliver vertices in dependency order (parents before children) rather than arbitrary order, which some indexes need to compute their entries correctly. `matches()` and `get_iterator()` (`scope.py:30, :40`) are what the manager calls during a rebuild to route the right vertices to the right index. The payoff of expressing scope as data is in §28.5: the manager can union the scopes of all indexes that need rebuilding and make a single pass over storage feeding each one only what it asked for.

28.3.4 IndexesManager : one object owns them all

The rest of the node does not talk to fifteen separate indexes. It talks to one `IndexesManager` (`manager.py:53`), an abstract base that holds every index as a typed attribute and offers methods that fan a single event out to all of them. Its own docstring: "IndexesManager manages all the indexes that we will have in the system... so it will know which index is better to use in each moment." The attributes (`manager.py:62 - :76`) are the full roster:

```
# hathor/indexes/manager.py:62
info: InfoIndex                # global counts + first/latest timestamps
sorted_all: TimestampIndex     # every vertex, by timestamp
sorted_blocks: TimestampIndex  # blocks, by timestamp
sorted_txs: TimestampIndex     # transactions, by timestamp
height: HeightIndex            # block height → block hash
mempool_tips: MempoolTipsIndex # tips of the unconfirmed mempool
addresses: Optional[AddressIndex] # address → history (optional)
tokens: Optional[TokensIndex]    # token uid → info + supply (optional)
utxo: Optional[UtxoIndex]        # unspent outputs (optional)
nc_creation: Optional[NCCreationIndex] # nano-contracts → Ch 39 (optional)
nc_history: Optional[NCHistoryIndex] # (optional)
blueprints: Optional[BlueprintTimestampIndex] # (optional)
blueprint_history: Optional[BlueprintHistoryIndex] # (optional)
```

Two things to read off this list immediately.

Some indexes are always present; some are `Optional`. `info`, the three `sorted_*` timestamp indexes, `height`, and `mempool_tips` are non-optional — the node cannot function without them (sync needs the height tip and the mempool tips; the node needs counts and time ordering). The rest are `Optional[...]` and default to `None`; they are switched on only if a feature or flag requires them. This is the "indexes cost space and write-work" trade-off made into a runtime choice: an operator who does not run a wallet-facing API need not pay for the UTXO or address indexes at all. The `enable_address_index`, `enable_tokens_index`, `enable_utxo_index`, and `enable_nc_indexes` methods (`manager.py:108 – :126`, abstract; implemented at `:451 – :477`) build the optional index lazily, the first time it is asked for, and do nothing if it already exists.

`iter_all_indexes()` (`manager.py:90`) returns every non-`None` index as a flat iterator (`filter(None, [...])` drops the disabled ones). This is the seam that lets the manager do anything "to every index" without naming them — clear them, rebuild them, check their freshness. The first such use is a safety check, `__init_checks__` (`manager.py:78`), which asserts no two indexes claim the same `get_db_name()`, so two indexes can never overwrite each other's on-disk state.

28.3.5 Two backends, the same manager

The abstraction pattern of Chapter 27 reappears exactly. `IndexesManager` is abstract; the concrete one used in production is `RocksDBIndexesManager` (`manager.py:421`), constructed from the same `RocksDBStorage` the node already opened:

```
# hathor/indexes/manager.py:421
class RocksDBIndexesManager(IndexesManager):
    def __init__(self, rocksdb_storage, *, settings):
        self.db = rocksdb_storage.get_db()
        self.info = RocksDBInfoIndex(self.db, settings=settings)
        self.height = RocksDBHeightIndex(self.db, settings=settings)
        self.mempool_tips = MemoryMempoolTipsIndex(settings=settings) # see note below
        self.sorted_all = RocksDBTimestampIndex(self.db, scope_type=ALL, settings=settings)
        self.sorted_blocks = RocksDBTimestampIndex(self.db, scope_type=BLOCKS, settings=settings)
        self.sorted_txs = RocksDBTimestampIndex(self.db, scope_type=TXS, settings=settings)
        self.addresses = self.tokens = self.utxo = None # optional → off by default
        ...
        self.__init_checks__() # must be last
```

Two details reward attention.

The mempool tips index is in-memory even in the RocksDB manager

(`manager.py:434`). The comment explains why: "use of RocksDBMempoolTipsIndex is very slow and was suspended." So the production manager mixes backends — most indexes persist to RocksDB, but the mempool tips live in RAM and are rebuilt every boot. This is legitimate precisely because the index is derived: a RAM-only index has `get_db_name() → None` and is reconstructed from storage at startup, no different from any other rebuild. It is a clean example of the trade-off being tuned per index: the mempool is small and the rebuild is cheap, so paying the rebuild cost beats paying a slow persisted-write cost on every transaction.

The same three `TimestampIndex` objects, different scopes. `sorted_all`, `sorted_blocks`, and `sorted_txs` are the same class parameterized by a `scope_type` (`ALL` / `BLOCKS` / `TXS`). One implementation, three instances, each maintaining a time-sorted list of a different slice of the ledger. This is why the package has more index attributes than index files.

A memory-only counterpart exists for tests and the simulator (the `memory*_index.py` files), assembled the same way. The node code above the manager — the APIs, the wallet, sync — calls the same methods regardless of which backend is underneath.

28.3.6 The UTXO index — the payoff of Chapter 7

This is the index most worth walking in full, because it makes the UTXO model concrete and shows the executed/voided pattern in its clearest form. Its job: answer "which unspent outputs does this address own (optionally of this token), and what do they sum to?" — i.e. compute a spendable balance without a full scan.

The unit it stores is `UtxoIndexItem` (`utxo_index.py:38`), a frozen dataclass — one entry per indexed unspent output:

```
# hathor/indexes/utxo_index.py:38
@dataclass(frozen=True)
class UtxoIndexItem:
    token_uid: bytes      # which token this output holds (HTR = b'\x00')
    tx_id: bytes          # the transaction (or block) that created the output
    index: int            # which output of that tx
    address: str          # the address the output is locked to (the lookup key)
    amount: int           # the value, in the smallest unit
    timelock: Optional[int] # unspendable until this Unix time, if set
    heightlock: Optional[int] # unspendable until this block height (block rewards)
```

Each field models a real thing a spender needs to know. `(tx_id, index)` is the UTXO pointer of Chapter 7 — it names the exact output. `address` is the field the index is keyed on, because "find my coins" is keyed on the address. `amount` lets the index sum to a target without re-reading the output. `timelock / heightlock` are there because an output can be present-but-not-yet-spendable: a freshly-mined block reward is locked for `REWARD_SPEND_MIN_BLOCKS` blocks (`from_tx_output`, `utxo_index.py:71`), and a wallet must know that before offering the coin as change. The `from_tx_output` constructor (`:54`) builds an item from a vertex output, refusing **authority** outputs (`:57`) and outputs whose script is not a recognizable address (`:61`) — those carry no spendable, address-locked value, so they have no place in a UTXO-by-address table.

Now the heart of it: keeping the table correct as the ledger changes. The manager calls `update(tx)` (`utxo_index.py:123`) whenever a vertex's status settles, and `update` dispatches on whether the vertex is **executed** or **voided**:

```
# hathor/indexes/utxo_index.py:123
def update(self, tx: BaseTransaction) -> None:
    tx_meta = tx.get_metadata()
    if tx_meta.voided_by:
        self._update_voided(tx)      # this tx lost / is invalid → undo its effect
    else:
        self._update_executed(tx)    # this tx counts → apply its effect
```

The **executed** path (`_update_executed`, `:133`) is the index analogue of "a transaction happened": the outputs it spent are no longer unspent, and the outputs it created now are.

```
# hathor/indexes/utxo_index.py:133
def _update_executed(self, tx):
    assert not tx.get_metadata().voided_by
    for tx_input in tx.inputs:          # the outputs this tx consumed...
        spent_tx = tx.get_spent_tx(tx_input)
        spent_txout = spent_tx.outputs[tx_input.index]
        if self._should_index_output(spent_txout):
            self._remove_utxo(UtxoIndexItem.from_tx_output(spent_tx, tx_input.index, spent_txout))
    for index, tx_output in enumerate(tx.outputs): # ...the new outputs it created
        self._add_output_to_index(tx, index, tx_output)
```


Read it against the toy in §28.3.1: this is `add_record` plus a matching remove. Spending an output **removes** its item (it is no longer unspent); creating an output **adds** one. Balance queries are then just "sum the `amount` of the items for this address," with no scan of the ledger.

The **voided** path (`_update_voided`, :166) is the exact mirror, and it is why the index can be trusted across reorgs (Chapter 10). When a transaction is voided, everything it did must be undone: the outputs it created are no longer real, so they are **removed**; and the inputs it spent are un-spent again, so those prior outputs are **re-added**.

```
# hathor/indexes/utxo_index.py:166
def _update_voided(self, tx):
    for tx_input in tx.inputs:                                # outputs this tx had spent...
        spent_tx = tx.get_spent_tx(tx_input)
        spent_txout = spent_tx.outputs[tx_input.index]
        if self._should_index_output(spent_txout):
            self._add_utxo(UtxoIndexItem.from_tx_output(spent_tx, tx_input.index, spent_txout))
    for index in range(len(tx.outputs)):                      # outputs this tx created...
        self._remove_output_from_index(tx, index)             # ...disappear
```

This add/remove symmetry is the whole reason voiding can be a reversible mark (Chapter 10) rather than a deletion: when consensus flips a transaction from executed to voided or back, the index has a precise undo for every change it ever applied. Reorgs become "replay the executed/voided transitions through `update`," and the lookup table stays consistent with the canonical ledger.

One last honest note from the code: the UTXO index is documented as **not critical** and used by optional, wallet-facing APIs (`utxo_index.py:88 - :93`); it is `Optional` on the manager and `None` unless `enable_utxo_index()` is called. A node that serves no such API never builds it — the trade-off declined.

28.3.7 The other indexes, in brief

Each remaining index is the same idea — precompute a question's answer, keyed the question's way, maintain it on add/remove (executed/voided), rebuild from storage when stale — so the descriptions can be short. What differs is what each maps and who asks.

Address index (`address_index.py:37`, `AddressIndex`). Maps an **address** → **the list of transaction hashes that involve it** (as input or output), so a wallet can show "my history." Its scope includes voided vertices (`:30`), because a wallet wants to see failed attempts too — a deliberate contrast with the UTXO index, which wants only spendable (executed) outputs. It also holds an optional `pubsub` reference (`:40`) so that, in some configurations, new history for an address can be announced to subscribers (pub-sub is Chapter 30). The wallet (Ch 40) and the address-history API (Ch 36) are its main callers.

Tokens index (`tokens_index.py` , `TokensIndex`). Maps a **token uid** → **that token's metadata and running supply** — its name and symbol, how much is in circulation (minted minus melted), and which outputs/authorities exist for it. Custom tokens (Chapter 7's multi-token model) are created by `TokenCreationTransaction` , and the supply changes whenever the token is minted or melted; the index keeps a running total so "tell me about token T" and "how much of T exists?" are direct lookups instead of a scan. The manager also routes nano-contract token events here — `create_token_info_from_contract` , `add_to_total` , `destroy_token` (`manager.py`:254 , :266 , :328) — so contract-minted tokens show up in the same place (Chapter 39).

Height index (`height_index.py`:50 , `HeightIndex`). Maps a **block height** → **the block hash at that height** on the canonical chain, and tracks the **height tip** (the highest block) via `get_height_tip` (:67). This is the chain backbone made queryable: "what is the block at height N?" and "how tall is the chain?" become O(1). It has an `add_new` path for extending the chain and an `add_reorg` path (:60) for replacing an existing height during a reorg — the executed/voided pattern in height-index clothing. Sync (Chapter 35) leans on this constantly to negotiate what each peer is missing; mining (Chapter 37) reads the tip to build the next block template.

Timestamp / sorted indexes (`timestamp_index.py` , `TimestampIndex` ; instances `sorted_all` , `sorted_blocks` , `sorted_txs`). Maintain vertices **sorted by timestamp**, one instance per slice (all / blocks-only / txs-only). They answer "give me vertices in this time window" and "walk the ledger in time order" — used by block explorers, time-range APIs, and any traversal that wants chronological order. Adding a vertex inserts it into the sorted structure (`add_tx`); voiding/removal deletes it (`del_tx`), called from the manager's `add_to_non_critical_indexes` / `del_from_non_critical_indexes` (`manager.py`:348 , :397).

Mempool tips index (`mempool_tips_index.py` , `MempoolTipsIndex` ; in production the in-memory `MemoryMempoolTipsIndex`). Tracks the **tips of the mempool** — the unconfirmed transactions at the frontier of the DAG that nothing else yet confirms. From these tips the full mempool can be derived by walking down. This is the index that answers "what is pending?" and supplies parents for a newly-created transaction (a new tx must confirm two existing tips, Chapter 8). It is **critical** but **not persisted**: the manager updates it via `update_critical_indexes` (`manager.py`:194) on every vertex and via `del_from_critical_indexes` (:379) on removal, and rebuilds it from storage at every boot. Its scope makes it the one index the node updates eagerly and separately from the rest, because the mempool must always be current for mining and for accepting new transactions.

Info index (`info_index.py` , `InfoIndex`). Holds **global bookkeeping** — total vertex/transaction/block counts and the first/latest timestamps the node has seen — updated by `update_counts` / `update_timestamps` (`manager.py:346` , `:375`). It backs node-statistics and dashboard queries. Cheap to maintain (a handful of counters) and so always present.

Nano-contract indexes (`nc_creation_index.py` , `nc_history_index.py` , `blueprint_timestamp_index.py` , `blueprint_history_index.py`). Track contract creations, per-contract call history, and blueprint listings/history. They are enabled together via `enable_nc_indexes` (`manager.py:466`) only when nano-contracts are active, and the manager feeds them from contract-execution records (`non_critical_handle_contract_execution` , `:203` , with a matching unexecution path for reorgs at `:275`). They exist for the same reason as the others — "what contracts exist?", "what has this contract done?" must be cheap — and are covered with the nano-contract subsystem in **Chapter 39**.

28.3.8 Critical vs. non-critical, and why the split

You will have noticed the manager's update methods come in two flavours:

`update_critical_indexes` / `del_from_critical_indexes` (the mempool tips) and `update_non_critical_indexes` / `add_to_non_critical_indexes` / `del_from_non_critical_indexes` (everything else). The distinction is operational, not conceptual:

- **Critical** indexes (the mempool tips) must be updated immediately and in lock-step with the ledger, because the node's ability to accept and create transactions depends on them being correct right now.
- **Non-critical** indexes (UTXO, address, tokens, timestamp, info, nano) can lag slightly or be rebuilt without stopping the node, because only queries depend on them — and a query getting a momentarily-stale answer is recoverable, whereas mining on a stale mempool is not.

This split is the "derived and disposable" property turned into a performance policy: the node spends its synchronous, can't-get-it-wrong effort only on the one index that gates correctness of new transactions, and treats the rest as catch-up work.

28.4 A worked trace: one transaction moves through the indexes

Tie it together with a concrete vertex. Suppose address **A** holds one unspent 50-HTR output (created by an earlier transaction **X**, output 0), and A sends 30 HTR to address **B**, with 20 HTR change back to A. The new transaction **T** has one input — **(X, 0)** — and two outputs: 30 to B, 20 to A. Trace what each index does when **T** is **executed**:

1. **UTXO index.** `_update_executed(T)` runs (`utxo_index.py:133`): the input **(X, 0)** is removed (A's 50-HTR coin is now spent), and two items are added — 30 HTR to B, 20 HTR to A. A's spendable balance, previously `{50}`, is now `{20}`; B's gains `{30}`. No ledger scan; just three table edits.
2. **Address index.** **T**'s hash is appended to A's history (it was both spender and change-receiver) and to B's history. Both addresses can now list **T**.
3. **Timestamp indexes.** **T** is inserted into `sorted_all` and `sorted_txs` at its timestamp; a time-window query spanning that moment will now include it.
4. **Mempool tips index.** Before a block confirms it, **T** is a mempool tip — it is added so the node can offer it as a parent and report it as pending. When a later block confirms **T**, it leaves the mempool tips (`del_from_critical_indexes`).
5. **Info index.** The transaction count increments; the latest-timestamp may advance.

Now suppose a reorg later **voids** **T** (a heavier chain confirmed a conflicting spend of **(X, 0)**). `update(T)` now takes the voided branch (`_update_voided`, `:166`): the 30-to-B and 20-to-A items are removed, and the original **(X, 0)** 50-HTR item is re-added — A's spendable balance returns to `{50}`. Every other index applies its own undo (`del_from_non_critical_indexes` with `remove_all`, `manager.py:386`). The lookup tables snap back to exactly what they would have been had **T** never executed — which is the point: the indexes track the canonical ledger, and the canonical ledger just changed.

28.5 How indexes are rebuilt — and why they can be

Everything so far assumed the indexes were already correct and only needed incremental upkeep. But at boot — especially after an unclean shutdown — they may be stale or missing. Because indexes are derived, the recovery is not repair but **rebuild from storage**, and the engine for it is `_manually_initialize` (`manager.py:128`). It is the clearest demonstration in the codebase of the "derived, rebuildable" property, so it is worth reading.

The logic, step by step:

```
# hathor/indexes/manager.py:128 (condensed)
def _manually_initialize(self, tx_storage):
    db_last_started_at = tx_storage.get_last_started_at()

    indexes_to_init = []
    for index in self.iter_all_indexes():
        name = index.get_db_name()
        if name is None:
            indexes_to_init.append(index)
            continue
        if db_last_started_at != tx_storage.get_index_last_started_at(name):
            indexes_to_init.append(index)

    for index in indexes_to_init:
        index.force_clear()

    ...
    overall_scope = reduce(operator.__or__, (i.get_scope() for i in indexes_to_init))
    for tx in overall_scope.get_iterator(tx_storage):
        for index in indexes_to_init:
            if index.get_scope().matches(tx):
                index.init_loop_step(tx)
```

The mechanism rests on a **timestamp handshake**. Storage records, once, when the database was last started cleanly (`get_last_started_at`). Each persisted index records when it was last fully built (`get_index_last_started_at(name)`). If those two stamps match, the index is known-current and skipped. If they differ — or if the index keeps no persisted state at all (`get_db_name()` is `None`) — the index is **stale** and goes on the rebuild list. (Indexes that need rebuilding first have their stamp reset to `NULL_INDEX_LAST_STARTED_AT`, `manager.py:155`, so that a crash during the rebuild leaves them correctly marked stale and they are rebuilt again next time — the rebuild is itself crash-safe.)

The rebuild then does exactly what the toy in §28.3.1 promised: **clear, then replay**. `force_clear()` empties each stale index; then the manager takes the union of all their scopes (`reduce(operator.__or__, ...)`, `manager.py:170`) and makes **a single pass** over the vertices in storage, handing each vertex to every stale index whose scope `matches` it, via `init_loop_step`. One read of the ledger reconstructs every stale index at once. (Notice the care taken for performance: the storage cache is shrunk during the load and restored afterward, `manager.py:161 / :191`, because the rebuild touches every vertex once and a large cache would only waste memory.)

This is the whole argument for "indexes hold no truth" paying off operationally. The node never has to trust an index across a crash. It checks the stamp; if anything is uncertain, it clears and replays from the one thing that is trusted — the vertices in storage. An index can be wrong, deleted, or brand-new, and the node converges to a correct index the same way every time. This is also why a fresh node, or one that just enabled the UTXO index for the first time, builds that index the same way: there is no special "first build" path — building and rebuilding are one mechanism.

28.6 How it plugs into the lifecycle

Place the package in the life-of-a-node story (Chapter 0, §0.3):

```
Act I – Startup
  5. node assembled (builder)      – Ch 24 — the IndexesManager is created here, wrapping
      |                               the same RocksDB the storage layer opened
      ▼
  6. manager starts → INITIALIZATION
      |   _manually_initialize(): check each index's stamp;
      |   clear + replay storage into any stale index  ◀ THIS CHAPTER (§28.5)
      |   (this is "indexes are prepared", Ch 0 §0.3 step 6)
      ▼
  7. reactor runs → STEADY STATE
      |   ingestion (Ch 33) executes/voids a vertex →
      |       consensus (Ch 32) sets voided_by / first_block / spent_outputs →
      |       manager.update_critical_indexes()      (mempool tips, eager)
      |       manager.update_non_critical_indexes()  (UTXO, address, tokens, ...)
      |   APIs / wallet / sync / mining READ the indexes – Ch 36, 40, 35, 37
```

The sequence: the **builder** (Chapter 24) constructs the `IndexesManager` over the already-open RocksDB and hands it to the `HathorManager` (Chapter 29). During the manager's **initialization**, `_manually_initialize` runs — this is the "indexes are prepared" line in the orientation chapter's boot narrative — bringing every index into agreement with storage before the node serves anyone. In **steady state**, the **ingestion pipeline** (Chapter 33) and **consensus** (Chapter 32) are the writers: when a vertex is executed or voided, they update the vertex's metadata and then call the manager's update methods to keep the indexes in step. Everyone else is a reader: the wallet (Chapter 40) reads the address and UTXO indexes for balances and history; the HTTP/WebSocket APIs (Chapter 36) read all of them; sync (Chapter 35) reads the height index and mempool tips; mining (Chapter 37) reads the height tip and mempool tips to build templates.

The clean division to carry forward: **storage is written first and is the source of truth; the indexes are written second and are derived from it**. A reader who wants the fact about a vertex goes to storage by hash; a reader who wants to find vertices by any other key goes to an index. And if the indexes and storage ever disagree, storage wins, because the indexes can always be rebuilt from it.

Recap

Index	Maps (key → value)	Who asks
UTXO (<code>utxo_index.py</code>)	(address, token) → unspent <code>UtxoIndexItems</code>	wallets / spend-building APIs — "what can I spend?"

Index	Maps (key → value)	Who asks
Address (<code>address_index.py</code>)	address → its transaction hashes (incl. voided)	wallet, address-history API — "my history?"
Tokens (<code>tokens_index.py</code>)	token uid → info + running supply	token info/supply APIs (incl. nano-contract tokens)
Height (<code>height_index.py</code>)	block height → block hash; the height tip	sync, mining, explorers — "the chain backbone"
Timestamp (<code>timestamp_index.py</code>)	timestamp → vertices (all / blocks / txs)	time-window queries, ordered traversal
Mempool tips (<code>mempool_tips_index.py</code>)	the unconfirmed DAG frontier	mining + new-tx parents — "what's pending?" (critical, RAM-only)
Info (<code>info_index.py</code>)	global counts + first/latest timestamps	node statistics / dashboards
Nano-contract (<code>nc_*</code> , <code>blueprint_*</code>)	contract/blueprint creations + histories	Chapter 39 (enabled with nano-contracts)

Indexes are the node's derived lookup tables: precomputed answers to every question that is not "give me the vertex with this hash." Each spends disk space and per-write effort to turn a full-ledger scan into a direct lookup, keyed the way the question is asked — by address, token, height, time, or mempool membership. The `IndexesManager` owns them all behind one interface, offered in interchangeable memory and RocksDB backends, with optional indexes switched on only when a feature needs them. The defining property is that an index holds **no truth of its own**: every entry is recomputable from the vertices in storage, which is why the node updates them with a precise executed/voided symmetry, why reorgs are just replayed transitions, and why boot recovery is a blunt clear-and-replay rather than delicate repair. Storage is the source of truth; the indexes are how the node finds things in it. The next chapter steps up to the object that owns both — and the whole node besides: **Chapter 29, the `HathorManager`**, the coordinator that brings storage, indexes, and every other subsystem to life and drives them through the lifecycle this part of the book has been tracing.

-
1. An **index**, in databases, is a derived (secondary) data structure that precomputes the answer to a class of lookup queries, keyed the way the query is phrased, so the answer can be found without scanning all the primary data. It costs extra storage and extra work on every write, in exchange for

fast reads. The index at the back of a book — topic → page numbers — is the same idea on paper. Because it is computed from the primary data, it can always be rebuilt and contains no information that is not already implied by that data. ↵

2. Derived state (also secondary state) is data computed from some primary source of truth, holding nothing the primary does not already imply. Its defining property is that it can be discarded and recomputed. Hathor's indexes are derived from the vertices in storage; storage is primary. This is what makes index corruption recoverable by rebuild rather than fatal. ↵
3. A **tip** of the ledger DAG is a vertex that no other vertex confirms yet — a leaf at the growing frontier of the graph. A **mempool tip** is such a tip that is also still unconfirmed by any block. New transactions attach to (confirm) existing tips, so the node must always know what its current tips are. Full treatment of the DAG and tips in Chapter 8. ↵

Chapter 29 — The Coordinator: HathorManager

What you'll learn in this chapter

- What `HathorManager` is: the single object that holds references to every subsystem of a running node, and the one place that owns the node's lifecycle.
- Why it is a **facade** that holds its parts rather than inheriting from them — the payoff of the builder you met in Chapter 24.
- The exact ordered sequence of `start()` : why a **crash check** comes first, why the storage "allow-scope" is widened during initialization and narrowed afterwards, and why the node only declares itself **READY** at the very end.
- What `_initialize_components()` does — loading genesis, rebuilding indexes and metadata from disk — and why this rebuild is necessary at all.
- How `stop()` tears the node down in reverse, and how `on_new_tx` hands an incoming vertex to the ingestion pipeline.
- Where this object sits in the life of a node: built by Chapter 24, run on the reactor of Chapters 16/23, feeding vertices to Chapter 33.

Every program of any size has a question: where is the centre? In a small script the centre is `main()`. In a long-running server with two dozen cooperating subsystems, the centre is a single object that holds all of them and knows the order in which they must come alive and the order in which they must die. In `hathor-core` that object is `HathorManager`, defined in `hathor/manager.py`. This chapter is about what that object holds, and what it does once the builder hands it over.

You have already met it twice, in passing. Chapter 0 §0.3 told the story of the node booting; step 6 of that story — "the manager starts" — is exactly the `start()` method we walk here. Chapter 24 built the manager: it constructed every part and passed them all to the `HathorManager(...)` constructor, then returned a wired-but-not-yet-started object. This chapter picks up the instant after that hand-off. The builder builds the thing; this chapter is what the thing does.

Localization

`hathor/core/manager.py` is a single module of roughly a thousand lines (995 on this branch). It defines one main class, `HathorManager` (`manager.py:78`), plus a small helper dataclass `ParentTx` (`manager.py:964`) used when choosing transaction parents. It sits at the root of the `hathor/` package — not inside any sub-package — because it is not a part of the node; it is the node's coordinator, the thing that owns the parts.

```
hathor-core/
└─ hathor/
    │   manager.py           ← HathorManager: the coordinator   ◀ YOU ARE HERE
    │   execution_manager.py ← crash-callback registry (used by start/stop)
    │   vertex_handler/      ← the ingestion pipeline start() feeds (Ch 33)
    │
    └─ builder/              ← constructs the HathorManager (Ch 24)
        │   reactor/        ← the event loop start() runs on (Ch 16/23)
        │
        └─ transaction/storage/ ← tx_storage: crash flags, genesis, allow-scope (Ch 27)
            │   indexes/      ← rebuilt during _initialize_components (Ch 28)
            │   consensus/    ← consensus_algorithm: voiding, soft-voided ids (Ch 32)
            │   verification/  ← verification_service (Ch 31)
            │   p2p/           ← connections: peer manager, started in start() (Ch 34)
            │   pubsub.py      ← pubsub: the announcement bus (Ch 30)
            │   event/         ← event_manager: persistent event queue (Ch 30)
            │   mining/ stratum/ ← cpu_mining_service, stratum_factory (Ch 37)
            │   feature_activation/ ← bit_signaling_service, feature_service (Ch 38)
            │   wallet/       ← optional wallet (Ch 40)
            └─ websocket/     ← optional admin websocket (Ch 36)
```

Context. The manager is the convergence point of the whole codebase. Almost every package in the tree above is represented by exactly one attribute on `HathorManager`. That is not an accident of size — it is the design. There is one object that knows about everything, and everything else mostly does not know about each other. When you want to find "the node," you have found it: it is this object.

What it does and why it exists

`HathorManager` does two jobs, and it is worth separating them clearly because they are different kinds of responsibility.

Job one: it holds everything. The constructor (`manager.py:98`) takes more than twenty collaborators as arguments and stores each on `self`: the storage, the consensus algorithm, the P2P connections manager, the verification service, the pub-sub bus, the event manager, the mining service, the feature-activation services, the optional wallet and websocket, and more. The manager does not build any of these — they arrive fully formed from the builder. It just keeps the references. This makes the manager the node's facade: a single front object through which the rest of the program

reaches any subsystem. A web API handler that needs to look up a transaction calls `manager.tx_storage.get_transaction(...)`; a sync agent that wants to announce a new block calls back through the manager. The manager is the directory.

Job two: it owns the lifecycle. Holding references is static; the manager's active work is sequencing. Subsystems cannot come alive in any order. The storage must be initialized before the indexes can be rebuilt from it. The indexes must be ready before checkpoints can be verified against the database. Peers must not be allowed to connect before the node has finished loading its own ledger, or the node would start answering questions with half-loaded data. The manager encodes all of these ordering constraints in two methods — `start()` (`manager.py:276`) and `stop()` (`manager.py:345`) — plus the private `_initialize_components()` (`manager.py:437`) that `start()` calls in the middle. Most of this chapter is a walk through those three methods, because the order is the design.

Why does this job belong to one object rather than being scattered? Because ordering constraints are global. Only something that can see every subsystem at once can sequence them correctly. If each subsystem started itself, no single piece would know that "indexes after storage, peers after both" — the knowledge would be smeared across the codebase and impossible to reason about. Centralizing it in one `start()` method makes the boot order readable: you can read `start()` top to bottom and know exactly what happens when.

The concepts it rests on

Four ideas from earlier chapters meet here. The manager is where they pay off, so this chapter recaps rather than re-teaches them.

Recap — Facade (full treatment in Ch. 3 §3.6). A facade is a single object that presents a simple, unified front over a set of more complicated subsystems. Callers talk to the facade instead of learning the internals. `HathorManager` is the book's central example: dozens of subsystems, one front object. The facade does not have to do the subsystems' work itself — it mostly delegates to the part that does. → full treatment in Ch. 3 §3.6.

Recap — composition over inheritance (full treatment in Ch. 1). The manager has-a storage, has-a consensus algorithm, has-a connections manager. It does **not** inherit from any of them. This is composition: building a capable object by holding other objects, rather than by extending a base class. The advantage here is that the manager can hold many unrelated subsystems at once — you cannot inherit from twenty parents, but you can hold twenty fields. → full treatment in Ch. 1.

Recap — the builder injects the parts (full treatment in Ch. 24). The manager does not construct its collaborators; they are passed in. Chapter 24's `Builder` / `CliBuilder` is the composition root that creates storage, indexes, services, and then calls `HathorManager(...)` with all of them. This is dependency injection: a class receives its dependencies instead of making them, which is what lets tests substitute fakes and lets production and the simulator share one manager class. → full treatment in Ch. 24.

Recap — the reactor and LoopingCall (full treatment in Ch. 16, recap Ch. 23). Hathor runs on Twisted's reactor, a single event loop that waits for events and calls your code in response. The manager is handed the reactor in its constructor and uses it two ways: it registers `stop` to run on reactor shutdown (`manager.py:159`), and it schedules a repeating timer — a `LoopingCall` (`manager.py:251`) — to periodically check sync state. A `LoopingCall` is Twisted's way of saying "call this function every N seconds." → full treatment in Ch. 16.

There is one more concept the manager leans on heavily — the storage allow-scope — which is local enough that we explain it inline when `start()` reaches it, below.

A tiny toy first

Before the real `start()`, here is the shape of the idea in miniature. Imagine a coordinator that holds three parts and must start them in a fixed order, do some one-time loading in the middle, and tear down in reverse:

```

class NodeState(Enum):
    INITIALIZING = "INITIALIZING"
    READY = "READY"

class Coordinator:
    def __init__(self, storage, network, clients):  # parts are injected
        self.storage = storage
        self.network = network
        self.clients = clients
        self.state = None
        self.started = False

    def start(self):
        if self.started:
            raise Exception("already started")
        self.started = True

        if self.storage.crashed_last_time():  # safety check FIRST
            raise SystemExit("storage unreliable; refusing to start")

        self.state = NodeState.INITIALIZING  # not ready yet
        self.storage.open()                  # 1. storage before anything
        self._load_from_storage()             # 2. one-time rebuild
        self.network.listen()                 # 3. peers only after loading
        self.clients.serve()                  # 4. clients last
        self.state = NodeState.READY         # only now: READY

    def _load_from_storage(self):
        self.storage.load_genesis()
        self.storage.rebuild_indexes()

    def stop(self):
        self.clients.stop()                  # reverse order
        self.network.stop()
        self.storage.close()
        self.started = False

```

Three lessons live in this toy, and all three carry into the real code unchanged:

1. **The safety check runs before any subsystem is touched.** If the data on disk is untrustworthy, you want to bail before you have opened sockets or started timers.
2. **The state is `INITIALIZING` during the whole boot and only becomes `READY` at the last line.** Anything that asks "are you ready?" during boot must get "no."
3. **`stop()` is `start()` reversed.** Clients came up last, so they go down first; storage came up first, so it closes last. You stop depending on a thing before you tear that thing down.

The real `HathorManager.start()` is this toy with twenty parts instead of three, and with two extra wrinkles — the allow-scope widen/narrow and the crash flags written to storage. We turn to it now.

The code, walked

The held collaborators

The constructor signature (`manager.py:98`) is the inventory of the node. Required collaborators are passed as keyword-only arguments; optional ones default to `None` . A representative sample, with what each is and which chapter covers it:

Attribute (<code>self...</code>)	Set at	Subsystem	Chapter
<code>tx_storage</code>	<code>manager.py:190</code>	vertex storage (RocksDB)	27
<code>pubsub</code>	<code>manager.py:189</code>	in-process event bus	30
<code>_event_manager</code>	<code>manager.py:193</code>	persistent event queue	30
<code>consensus_algorithm</code>	<code>manager.py:201</code>	voiding, scores, PoA	32
<code>verification_service</code>	<code>manager.py:198</code>	per-vertex rule checks	31
<code>connections</code>	<code>manager.py:203</code>	P2P connections manager	34
<code>vertex_handler</code>	<code>manager.py:204</code>	ingestion pipeline	33
<code>_bit_signaling_service</code>	<code>manager.py:197</code>	feature-activation signalling	38
<code>feature_service</code>	<code>manager.py:207</code>	feature gating queries	38
<code>cpu_mining_service</code>	<code>manager.py:199</code>	CPU proof-of-work helper	37
<code>metrics</code>	<code>manager.py:212</code>	observability counters	42
<code>wallet</code>	<code>manager.py:221</code>	optional wallet	40
<code>websocket_factory</code>	<code>manager.py:210</code>	optional admin websocket	36
<code>stratum_factory</code>	<code>manager.py:229</code>	optional Stratum server	37
<code>poa_block_producer</code>	<code>manager.py:248</code>	optional PoA producer	32

Two details in the constructor are worth pausing on, because they are not mere assignment.

First, the very first thing the constructor does is refuse to start in one specific situation (`manager.py:139`):

```
if event_manager.get_event_queue_state() is True and not enable_event_queue:
    raise InitializationError(
        'Cannot start manager without event queue feature, as it was enabled in the '
        'previous startup. Either enable it, or use the reset-event-queue CLI command ...'
    )
```

This is a consistency guard. The persistent event queue (Chapter 30) is an append-only log of everything the node has emitted. If it was on last time and you now start with it off, the log would develop a silent gap. Rather than allow that, the constructor fails loudly and tells the operator how to fix it. Note where this lives: it is a precondition of even building a usable manager, so it is checked in `__init__`, not in `start()`.

Second, the constructor wires the manager's own `stop` into reactor shutdown (`manager.py:157`):

```
add_system_event_trigger = getattr(self.reactor, 'addSystemEventTrigger', None)
if add_system_event_trigger is not None:
    add_system_event_trigger('after', 'shutdown', self.stop)
```

This is how `stop()` gets called when the operator presses Ctrl-C: Twisted fires a "shutdown" event, and the manager has asked to be told. The `getattr(...)` guard is defensive — some reactor implementations (the asyncio backend, certain test reactors) may not expose `addSystemEventTrigger`, so the manager only registers if the method exists.

The `state` attribute itself is declared `None` initially (`manager.py:161`). The `NodeState` enum (`manager.py:84`) has exactly two members:

```
class NodeState(Enum):
    INITIALIZING = 'INITIALIZING'
    READY = 'READY'
```

There is no `STOPPED` or `SHUTDOWN` member. The lifecycle of `state` is `None` → `INITIALIZING` → `READY`, and it is never set back. Shutdown is tracked by a separate boolean, `is_started` (`manager.py:177`). This is worth flagging because it would be reasonable to expect a third state and there isn't one; the code reads "ready or not."

The start() sequence

`start()` (`manager.py:276`) is the heart of the chapter. Here is the full ordered sequence, with the why of the order spelled out at each step.

Step 0 — the once-only guard. (`manager.py:279`)

```
if self.is_started:
    raise Exception('HathorManager is already started')
self.is_started = True
```

A node is started exactly once. Calling `start()` twice is a programming error, so it raises immediately.

Step 1 — the crash checks, before anything else. (`manager.py:285` and `:296`) This is the most consequential design decision in the whole method, and it must come first:

```

if self.tx_storage.is_full_node_crashed():
    self.log.error('... The storage is not reliable anymore ... you must remove your '
                  'storage and do a full sync ...')
    sys.exit(-1)

if self.tx_storage.is_running_manager():
    self.log.error('... it wasn\'t stopped correctly. The storage is not reliable anymore ...')
    sys.exit(-1)

```

To understand why this matters, you need to know what these two flags mean and why a crash makes the database untrustworthy. The storage keeps two boolean attributes on disk (Chapter 27):

- `is_running_manager()` (`transaction_storage.py:859`) reads a flag that is **set** at the very end of `start()` and **cleared** at the start of `stop()` . So if the node is reading "manager is running" when it has only just begun to start, the previous run must have crashed before it reached `stop()` — an unclean shutdown.
- `is_full_node_crashed()` (`transaction_storage.py:868`) reads a flag set by a crash callback (`on_full_node_crash` , `transaction_storage.py:864`) that the execution manager invokes when an unrecoverable error tears the node down mid-flight.

Why does an unclean shutdown poison the data? The comment in the code (`manager.py:295`) is precise: "The metadata is the only piece of the storage that may be wrong, not the blocks and transactions." The blocks and transactions themselves are content-addressed¹ and verified — they cannot be silently corrupt. But the metadata (Chapter 25) — accumulated weight, score, which vertices are voided, the best-block pointer — is computed incrementally as vertices arrive, and updating it is a multi-step operation. A crash partway through a metadata update leaves the bookkeeping in a half-written state: an output might be marked spent while the spending transaction is also marked voided, or a score might reflect a chain tip that the best-block pointer no longer agrees with. There is no cheap way to know exactly which records are half-written, so the node takes the safe path: it refuses to start and tells the operator to re-sync (from scratch or from a trusted snapshot), which rebuilds the metadata cleanly. `sys.exit(-1)` is blunt on purpose — continuing on suspect bookkeeping could mean accepting an invalid ledger.

Doing this check first is the whole point. Before a single socket is opened or a single timer is scheduled, the node verifies it is even safe to run. The toy above made the same choice.

Step 2 — the event manager and the state transition. (`manager.py:304`)

```

if self._enable_event_queue:
    self._event_manager.start(str(self.my_peer.id))

self.state = self.NodeState.INITIALIZING
self.pubsub.publish(HathorEvents.MANAGER_ON_START)
self._event_manager.load_started()
self.pow_thread_pool.start()

```

The state becomes `INITIALIZING` (`manager.py:307`) — the node is now officially booting but not ready. It publishes a `MANAGER_ON_START` event on the pub-sub bus (`manager.py:308`) so any in-process subscriber can react. It signals the event manager that the load phase has begun (`load_started`, `manager.py:309`). And it starts the **pow thread pool** (`manager.py:310`): a small pool of worker threads used to compute proof-of-work off the reactor thread when the node itself needs to produce a vertex, so that the CPU-bound hashing never blocks the event loop. (Recall from Chapter 2 / 16 the rule: never block the reactor; offload CPU-bound work to a thread pool.)

Step 3 — widen the allow-scope, initialize, then narrow it. (`manager.py:313`) This is the second design wrinkle the toy did not have:

```

self.tx_storage.disable_lock()
self.tx_storage.set_allow_scope(TxAllowScope.VALID | TxAllowScope.PARTIAL | TxAllowScope.INVALID)
self._initialize_components()
self.tx_storage.set_allow_scope(TxAllowScope.VALID)
self.tx_storage.enable_lock()

```

The allow-scope (`TxAllowScope`, Chapter 27) is a filter on the storage: it controls which validation states of vertices the storage will hand back. Normally a running node only wants `VALID` vertices — fully verified, safe to act on. But during initialization the node is rebuilding its view of the ledger by walking everything on disk, and on disk there may be vertices in `PARTIAL` (partially validated, mid-sync) or even `INVALID` states left over from the last run. To rebuild correctly, the initialization code must be able to see all of them. So the manager temporarily widens the scope to `VALID | PARTIAL | INVALID`, runs `_initialize_components()`, and then narrows it back to `VALID` only. After this point, any code that asks the storage for a vertex transparently gets only valid ones — the relaxed window is closed.

The surrounding `disable_lock()` / `enable_lock()` pair (`transaction_storage.py:518` / `:523`) turns off the storage's internal access lock during single-threaded initialization (nothing else is running yet, so the lock would be pure overhead) and turns it back on once the node is about to go concurrent. Initialization itself is covered below.

Step 4 — bring subsystems online, in dependency order. (`manager.py:321` – `:340`)


```

if self.websocket_factory:
    self.websocket_factory.start()      # before metrics: metrics may push to it
self.metrics.start()
self.connections.start()              # peers can now connect
self.start_time = time.time()
self.lc_check_sync_state.start(self.lc_check_sync_state_interval, now=False)
if self.wallet:
    self.wallet.start()
if self.stratum_factory:
    self.stratum_factory.start()
if self.poa_block_producer:
    self.poa_block_producer.start()

```

The order is not alphabetical or arbitrary; it is dependency order. The websocket factory starts before metrics (`manager.py:321`) because metrics may push data into it. `connections.start()` (`manager.py:327`) — the P2P layer — comes only after `_initialize_components()` has finished, because allowing peers to connect before the node has loaded its own ledger would mean answering sync requests with incomplete data. The `lc_check_sync_state` `LoopingCall` (`manager.py:331`) is started with `now=False`, meaning the first run is delayed one interval rather than firing immediately. The optional subsystems — wallet, stratum mining server, PoA block producer — each start only if they were configured (the `if self.x:` guards).

Step 5 — mark the manager running, last. (`manager.py:343`)

```

self.tx_storage.start_running_manager(self._execution_manager)

```

This is the symmetric counterpart of the Step 1 crash check. `start_running_manager` (`transaction_storage.py:848`) does two things: it registers the crash callback with the execution manager (so that if the node dies unrecoverably, the `full_node_crashed` flag gets set), and it writes the `manager_running` flag to disk. From this instant on, an unclean shutdown will be detectable next time, because the flag will still be set when the next `start()` reads it. Setting this flag last — after everything else has come up successfully — means the node only declares "I am running" once it actually is.

Notice what is not in `start()`: a call to `reactor.run()`. The manager starts its subsystems, but the event loop is run by the CLI layer (`run_node`, Chapter 21) after `start()` returns. The manager readies the node; the caller puts it in gear.

_initialize_components: loading the ledger into memory

`_initialize_components()` (`manager.py:437`) is the one-time load phase. Its docstring is honest about its scope: "This method runs through all transactions, verifying them and updating our wallet." The ordered steps:

```

if self.wallet:
    self.wallet._manually_initialize()          # rebuild wallet's view
self.tx_storage.pre_init()                     # network check + migrations + genesis
self._bit_signaling_service.start()
...
self._verify_soft_voided_txs()
self.tx_storage.indexes._manually_initialize(self.tx_storage)  # rebuild indexes
self._verify_checkpoints()
self.tx_storage.update_last_started_at(started_at)
...
self._event_manager.load_finished()
self.state = self.NodeState.READY             # the transition

```

Walking the load:

- `tx_storage.pre_init()` (`transaction_storage.py:192`) checks that the on-disk network matches the configured one (you must not point a mainnet database at testnet) and applies any pending schema migrations. The **genesis**² itself — the hard-coded first block and transactions every node agrees on — is saved-or-verified in the storage's own constructor via `_save_or_verify_genesis()` (`transaction_storage.py:333` , called at `:1110`): on a fresh database it writes genesis; on an existing one it verifies the stored genesis matches the configured one. Either way, after `pre_init()` the genesis is present and trusted, anchoring the rest of history.
- `_verify_soft_voided_txs()` (`manager.py:495`) guards a subtle compatibility issue. Soft-voided transactions (Chapter 32) are ones the network has agreed to treat as void by policy rather than because of a conflict. The list of their ids comes from settings. This method checks that every soft-voided id which exists in the database is in fact marked soft-voided in its metadata; if one is present but unmarked, the database predates the soft-voiding rule and is incompatible, so the node exits and asks for a re-sync. This is the same "trust the content, distrust stale metadata" philosophy as the crash check.
- `indexes._manually_initialize(tx_storage)` (`manager.py:462`) rebuilds the in-memory indexes from the stored vertices (Chapter 28). This answers a question a junior reader should be asking: why rebuild at all — aren't the indexes saved? Some are persisted, but several derived indexes (and the mempool-tips index) live only in memory and must be reconstructed on each boot by walking the database once. This single pass — done while the allow-scope is wide — is the bulk of boot time on a large database, and is exactly the "rebuild its in-memory view of the ledger" step from the life-of-a-node story in Chapter 0 §0.3.

- `_verify_checkpoints()` (`manager.py:522`) checks that every checkpoint³ expected to exist at the current best height is present in the database with the correct hash. Checkpoints are hard-coded "this block at this height is final" markers (Chapter 10); verifying them on boot detects a database that has diverged from the canonical history. A failure here exits the node.
- `update_last_started_at(started_at)` (`manager.py:472`) records the boot timestamp — the comment calls it the last step before actually starting.

After the optional event-queue load-phase handling, the method calls

`self._event_manager.load_finished()` and sets `self.state = self.NodeState.READY` (`manager.py:482`). This single line is the boundary between "booting" and "alive." It logs 'ready' with the vertex count and total load time (`manager.py:492`) — the line you see in a node's logs that tells you boot finished. Everything before it was preparation; everything after it is steady-state operation.

stop: teardown in reverse

`stop()` (`manager.py:345`) undoes `start()` . It begins with its own once-only guard and flips `is_started` to `False` (`manager.py:346`), then tears down roughly in the reverse of the start order:

```
self.tx_storage.stop_running_manager()    # clear the "running" flag FIRST
self.connections.stop()                  # stop accepting peers
self.pubsub.publish(HathorEvents.MANAGER_ON_STOP)
if self.pow_thread_pool.started:
    self.pow_thread_pool.stop()
self.metrics.stop()
if self.websocket_factory:
    self.websocket_factory.stop()
if self.lc_check_sync_state.running:
    self.lc_check_sync_state.stop()
if self.wallet:
    self.wallet.stop()
if self.stratum_factory:
    wait_stratum = self.stratum_factory.stop()    # may return a Deferred
    if wait_stratum:
        waits.append(wait_stratum)
if self._enable_event_queue:
    self._event_manager.stop()
if self.poa_block_producer:
    self.poa_block_producer.stop()
self.tx_storage.flush()                  # persist any buffered writes
return defer.DeferredList(waits)
```

The first call, `stop_running_manager()` (`manager.py:353` , → `transaction_storage.py:854`), clears the `manager_running` flag on disk. This is the moment that makes the shutdown "clean": because the flag is now cleared, the next `start()` will not see "manager was running" and will boot normally. If the process dies before reaching this line, the flag stays set and the next boot triggers the crash check — which is exactly the intended behaviour.

The method then stops each subsystem, guarding the optional ones with `if`. Some teardowns are asynchronous: `stratum_factory.stop()` may return a `Deferred`⁴, which is collected into `waits`. The final `self.tx_storage.flush()` (`manager.py:382`) forces any buffered writes to disk, and `stop()` returns a `DeferredList` so the caller can wait for all the asynchronous teardowns to complete before the process exits. Returning a `Deferred` is what lets Twisted's "after shutdown" trigger (registered in the constructor) hold the process open until cleanup is done.

on_new_tx: the door for incoming vertices

The manager is the facade for ingestion too, but it does almost no ingestion work itself — it delegates. The relevant methods form a short funnel:

- `submit_block(blk)` (`manager.py:796`) — used by the mining APIs; checks the block builds on the current tip, then calls `propagate_tx`.
- `push_tx(tx, ...)` (`manager.py:820`) — used by the public "push transaction" API; runs a few cheap pre-checks (already exists? double-spend? spending a voided tx? reward still locked? non-standard script?) and raises a specific exception for each, then calls `propagate_tx`.
- `propagate_tx(tx)` (`manager.py:851`) — attaches the storage to the vertex if needed, then calls `on_new_tx` with `propagate_to_peers=True`.
- `on_new_tx(vertex, ...)` (`manager.py:864`) — the single funnel point:

```
def on_new_tx(self, vertex, *, quiet=False, propagate_to_peers=True, reject_locked_reward=True):
    success = self.vertex_handler.on_new_relayed_vertex(
        vertex, quiet=quiet, reject_locked_reward=reject_locked_reward
    )
    if propagate_to_peers and success:
        self.connections.send_tx_to_peers(vertex)
    return success
```

The actual verify → consensus → store → announce pipeline lives in the **vertex handler** (`self.vertex_handler.on_new_relayed_vertex`), which is Chapter 33's subject. The manager's role is to be the door: every path by which a new vertex enters the node — from a peer, from the mining server, from the push-tx API — converges on `on_new_tx`, which hands it to one pipeline and, if accepted, relays it onward to peers. This is the facade doing what a facade does: presenting one entry point, delegating the work to the part that owns it.

The periodic job

The constructor created one `LoopingCall` (`manager.py:251`):

```
self.lc_check_sync_state = LoopingCall(self.check_sync_state)
self.lc_check_sync_state.clock = self.reactor
```

`start()` runs it every `CHECK_SYNC_STATE_INTERVAL = 30` seconds (`manager.py:96`). The callback, `check_sync_state` (`manager.py:933`), checks whether the node has recent blockchain activity (`has_recent_activity`, `manager.py:908`); the first time it does, it logs how long the initial sync took and then stops itself (`self.lc_check_sync_state.stop()`, `manager.py:946`). It is a one-shot "are we caught up yet?" poller dressed as a repeating timer — it repeats only until the node has caught up for the first time, then retires. Setting `.clock = self.reactor` ties the timer to the node's reactor so it is deterministic under the simulator (Chapter 43), which substitutes a fake clock.

How it plugs into the lifecycle

This chapter is one box in the life-of-a-node story from Chapter 0 §0.3 — specifically Act I step 6, "the manager starts," and the entry into Act II, steady state. The handoffs are:

- **Before it:** Chapter 24's builder constructed the manager and every subsystem it holds, returning a wired-but-not-started object. The constructor we walked is the destination of that build.
- **During it:** `start()` runs on no event loop yet — it is straight-line code. After `start()` returns, the CLI layer (Chapter 21) calls `reactor.run()` (Chapters 16/23), and the node becomes reactive: it sleeps until a peer connects, a vertex arrives, or the 30-second timer fires.
- **After it, in steady state:** vertices arriving from peers (Chapters 34–35) or created by mining (Chapter 37) enter through `on_new_tx` and flow into the vertex handler (Chapter 33). The pub-sub events the manager publishes (`MANAGER_ON_START`, `MANAGER_ON_STOP`, and every event the subsystems emit) flow through the announcement system of Chapter 30.
- **At the end:** Twisted's shutdown trigger fires `stop()` (registered in the constructor), which tears the node down in reverse and clears the crash flag so the next boot is clean.

The manager is, in one sentence, the object that turns a pile of constructed parts into a living node and back into a stopped one.

Recap

Phase	Method / line	What happens	Why the order
Construct	<code>__init__</code> <code>manager.py:98</code>	hold every injected subsystem; guard event-queue consistency; register <code>stop</code> on shutdown	facade: one object knows all parts
Guard	<code>start()</code> <code>manager.py:285</code> , <code>:296</code>	crash checks via <code>is_full_node_crashed</code> / <code>is_running_manager</code> ; exit if unclean	metadata may be half-written after a crash → don't run on suspect data
Begin	<code>manager.py:307</code> – <code>:310</code>	<code>state=INITIALIZING</code> ; publish <code>MANAGER_ON_START</code> ; start pow thread pool	not READY yet; off-reactor hashing ready
Load	<code>_initialize_components</code> <code>manager.py:437</code>	widen allow-scope; genesis via <code>pre_init</code> / <code>_save_or_verify_genesis</code> ; rebuild indexes; verify checkpoints; narrow scope	rebuild in-memory ledger view before peers connect
Become ready	<code>manager.py:482</code>	<code>state=READY</code> ; log <code>'ready'</code>	the boot/ steady-state boundary
Subsystems up	<code>manager.py:321</code> – <code>:340</code>	websocket → metrics → connections → wallet → stratum → PoA	dependency order; peers only after load
Mark running	<code>manager.py:343</code>	<code>start_running_manager</code> : register crash callback, set on-disk flag	last, so "running" means truly running
Ingest	<code>on_new_tx</code> <code>manager.py:864</code>	delegate to <code>vertex_handler</code> ; relay to peers if accepted	facade door → pipeline (Ch 33)

Phase	Method / line	What happens	Why the order
Teardown	<code>stop()</code> <code>manager.py:345</code>	clear running flag first; stop subsystems in reverse; flush; return <code>DeferredList</code>	clean shutdown → next boot is clean

The `NodeState` enum has only two members — `INITIALIZING` and `READY` (`manager.py:84`) — and the node's running status is tracked separately by the `is_started` boolean. The whole chapter reduces to one idea: there is one object that holds the node, and the order in which it brings its parts to life — crash check first, ledger load before peers, `READY` last, running-flag last of all — is the design.

The manager publishes events as it starts and as the node runs, but it does not implement the announcement system itself; it holds a `pubsub` and an `event_manager` and leans on them. That announcement system — how an in-process event becomes something a subscriber, or an external WebSocket client, can react to — is the subject of Chapter 30.

-
1. Content-addressed means a piece of data is identified by the hash of its own bytes. Change one byte and the hash changes, so a vertex cannot be silently altered without its id changing — which is why the blocks and transactions themselves cannot be quietly corrupted, only the separately-stored metadata about them. ↩
 2. The genesis is the hard-coded starting point of the ledger — the first block and initial transactions every node agrees on by definition. Hathor stores or verifies it during storage initialization, anchoring the rest of history. Full treatment: Ch. 0 §0.3 and the storage chapter (Ch. 27). ↩
 3. A checkpoint is a hard-coded "the block at this height has this hash and is final" marker shipped in settings. Verifying checkpoints on boot detects a database that has diverged from canonical history. Full treatment in Ch. 10. ↩
 4. A Deferred is Twisted's placeholder for a result that is not ready yet — a "future." Returning one from `stop()` lets the caller wait for asynchronous teardown to finish before the process exits. Full treatment in Ch. 16. ↩

Chapter 30 — Events & Pub-Sub

What you'll learn in this chapter

- Why a node needs an announcement system at all — and the cost of wiring components together without one.
- The **two layers** Hathor builds, and why they are separate: the in-process **PubSubManager** (ephemeral, observer-pattern, internal) versus the durable **EventManager** and its **event queue** (persisted, replayable, external-facing).
- How **PubSubManager** works in code: the **HathorEvents** catalogue, **subscribe / publish**, and the subtle detail of when a published callback actually runs (it usually waits for the reactor).
- How **EventManager** subscribes to the bus, turns ephemeral events into numbered, persisted **BaseEvent** records in RocksDB, and streams them to external clients over WebSocket — with acknowledgement and flow control.
- Where both layers are created and started in the node's lifecycle, and who fires the events you'll most often care about.

Chapter 29 introduced the **HathorManager** — the coordinator that holds every subsystem and starts them in order. This chapter is about how those subsystems talk to each other after startup, without being wired together by hand. It is the payoff for two earlier chapters: the **observer / publish-subscribe** pattern from Chapter 3, and the **callback** mechanics from Chapter 2. If those are hazy, the recap boxes below will refresh them in context; the full treatments live where named.

30.1 Localization

The event machinery lives in two places. The first is a single file at the top of the **hathor/** package; the second is a sub-package next to it.

hathor-core/	
└─ hathor/	
pubsub.py	← the in-process bus ◀ YOU ARE HERE (Layer 1)
manager.py	← creates pubsub, starts event_manager (Ch 29)
vertex_handler/	← fires NEW_TX_ACCEPTED & friends (Ch 33)
└─ event/	← the durable, external feed ◀ (Layer 2)
__init__.py	
event_manager.py	← EventManager: subscribes to pubsub, persists, broadcasts; the heart of Layer 2
└─ model/	
base_event.py	← BaseEvent: the persisted, numbered record
event_type.py	← EventType enum + map from HathorEvents
event_data.py	← typed payloads (TxData, ReorgData, ...)
└─ node_state.py	← LOAD / SYNC node-state marker
└─ storage/	
event_storage.py	← EventStorage ABC (the interface)
└─ rocksdb_storage.py	← EventRocksDBStorage (the real backend)
└─ websocket/	
factory.py	← EventWebSocketFactory: holds connections, broadcasts events
protocol.py	← EventWebSocketProtocol: per-client state machine
request.py	← client → node messages (START_STREAM, ACK, ...)
└─ response.py	← node → client messages (EventResponse, ...)
└─ resources/	
└─ event.py	← HTTP endpoint to read events (not streaming)

Context. pubsub.py is the node's internal nervous system: any component can announce "something happened" and any other component can listen, with neither knowing about the other. The event/ package builds a second system on top of that bus — a durable, ordered, replayable feed that the outside world (dashboards, exchanges, indexers) can subscribe to over a WebSocket and never miss a beat, even across restarts. The first is for the node talking to itself; the second is for the node talking to the world.

30.2 What it does and why it exists

Start with the problem, because the design only makes sense against it.

A running node is full of moments worth announcing. A new transaction gets accepted. A peer connects, then becomes ready, then disconnects. The consensus algorithm reorganizes the chain. A nano-contract finishes executing. Each of these moments matters to several parts of the program at once. When a transaction is accepted, the address index wants to record which addresses it touched, the metrics counter wants to tick up, the wallet may want to update a balance, and any connected WebSocket dashboard wants to be told.

The naive way to handle this is direct wiring. Inside the code that accepts a transaction, you would write:

```
self.address_index.add_tx(tx)
self.metrics.tx_accepted += 1
self.wallet.on_new_tx(tx)
self.websocket.notify(tx)
```

This works, and for a two-component program it is fine. But it has a structural flaw that grows with the codebase. The transaction-acceptance code now knows about the address index, the metrics object, the wallet, and the WebSocket server. It imports them, holds references to them, and must be edited every time a new listener appears or an old one is removed. The announcer is welded to its audience. In a node with dozens of subsystems, this welding becomes a web of cross-references that is hard to reason about and harder to change.

Recap — observer / publish-subscribe (full treatment in Ch. 3, §3.8). The pattern inverts the wiring. The announcer — the publisher — keeps a list of interested parties (subscribers) and notifies all of them when an event occurs, knowing nothing about who they are or what they do. Subscribers register a callback to be run on each event. The result is **decoupling**¹: you can add a subscriber without touching the publisher, and remove one without anyone noticing. The transaction-acceptance code shrinks to one line — `publish("new tx accepted", tx=tx)` — and stops caring who listens.

So the first thing the event system does is **decouple producers of events from consumers of events**. That is the job of `PubSubManager` (Layer 1). It is in-process, in-memory, and ephemeral: an event is published, every current subscriber's callback runs, and then the event is gone. Nobody who was not subscribed at that instant will ever see it. This is exactly right for components inside the same process that are all alive at the same time.

But there is a second, different need. Systems outside the node — a block explorer, a cryptocurrency exchange, an analytics pipeline — want a complete, ordered, gap-free record of everything the node has seen, and they want to be able to disconnect, reconnect, and pick up exactly where they left off. An ephemeral in-memory bus cannot serve them: if the consumer is offline when an event fires, the event is lost forever, and there is no notion of "event number 4,217, please resume from there."

That is why Layer 2 exists. The `EventManager` subscribes to the in-process bus, and for the events that matter to outsiders it does three things the bus does not: it assigns each event a **sequential id**, it **persists** the event to RocksDB, and it **broadcasts** it to connected WebSocket clients with acknowledgement and flow control. The persisted, numbered log is the **event queue**²: a durable, replayable feed. A consumer can ask "send me everything from id 4,217 onward," go offline, come back, and resume — because the events are on disk, not in volatile memory.

The two layers, then, answer two different questions:

	Layer 1 — <code>PubSubManager</code>	Layer 2 — <code>EventManager</code> + queue
Audience	the node's own components	external systems
Lifetime	one notification, then gone	persisted forever (until pruned)
Ordering	none guaranteed	strict, gap-free <code>id</code> sequence
Replay	impossible	the whole point
Built on	nothing (it is the base)	the Layer-1 bus

Hold that distinction; the rest of the chapter fills in each layer.

30.3 The concepts it rests on

Before the code, three quick recaps. None is re-taught from scratch here — each points to its canonical chapter.

Recap — callbacks (full treatment in Ch. 2). A callback is a function you hand to someone else to be called later, when some condition is met. Pub-sub is callbacks at scale: subscribing is "here is a function, call it when this event happens." Everything in this chapter is ultimately a list of callbacks being invoked.

Recap — the reactor and `callLater` (full treatment in Ch. 16, recapped in Ch. 23). Hathor runs on Twisted's **reactor**, a single event loop that waits for things to happen and dispatches handlers. `reactor.callLater(0, fn)` schedules `fn` to run "as soon as the loop is free" rather than right now. The pub-sub bus uses this to avoid running subscriber callbacks in the middle of whatever code just published — a detail we examine in §30.4.

Recap — RocksDB persistence (full treatment in Ch. 27). RocksDB is the embedded key-value store the node uses for everything on disk. It organizes data into named **column families** — independent key spaces inside one database. The durable event queue is just two more column families (one for events, one for metadata) in the same RocksDB instance that holds the ledger. → full treatment in Ch. 27.

With those in hand, we walk the code.

30.4 The code, walked — Layer 1: the in-process bus

A tiny pub-sub toy first

The whole of `PubSubManager` is an elaboration of one twelve-line idea. Here it is in neutral Python, the same shape you met in Chapter 3:

```
class Bus:
    def __init__(self):
        self._subscribers = {}          # event name → list of callbacks

    def subscribe(self, event, callback):
        self._subscribers.setdefault(event, []).append(callback)

    def publish(self, event, **data):
        for callback in self._subscribers.get(event, []):
            callback(event, data)

bus = Bus()
bus.subscribe("new_tx", lambda ev, d: print("index sees", d["tx"]))
bus.publish("new_tx", tx="abc123")      # → index sees abc123
```

A dictionary from event name to a list of callbacks; `subscribe` appends; `publish` loops and calls. Hold this picture — the real class is this plus three refinements: a typed catalogue of event names, a typed argument bag, and a reactor-aware delivery mechanism.

The catalogue: `HathorEvents`

The real bus does not key on free-form strings. It keys on an enum so that a typo cannot silently create a new, unlistened-to event. The catalogue lives at `hathor/pubsub.py:37`:

```
class HathorEvents(Enum):
    MANAGER_ON_START = 'manager:on_start'
    MANAGER_ON_STOP = 'manager:on_stop'
    NETWORK_PEER_CONNECTED = 'network:peer_connected'
    NETWORK_NEW_TX_PROCESSING = 'network:new_tx_processing'
    NETWORK_NEW_TX_ACCEPTED = 'network:new_tx_accepted'
    CONSENSUS_TX_UPDATE = 'consensus:tx_update'
    CONSENSUS_TX_REMOVED = 'consensus:tx_removed'
    REORG_STARTED = 'reorg:started'
    REORG_FINISHED = 'reorg:finished'
    # ... wallet, peer, nano-contract events ...
```

The enum members are grouped by namespace in their string values (`manager:`, `network:`, `consensus:`, `reorg:`, `wallet:`, `nc:`), which is documentation for the human reader; the code only ever compares enum members. The class's docstring (`pubsub.py:38`) describes, per event, when it fires and what it publishes — read it as the authoritative index of the node's internal life. The two you will see most are `NETWORK_NEW_TX_ACCEPTED` (`pubsub.py:127`) — fired whenever a vertex is accepted into the ledger — and `MANAGER_ON_START` (`pubsub.py:112`) — fired once when the node boots.

The argument bag: `EventArguments`

Different events carry different data. A transaction-accepted event carries a `tx`; a reorg event carries the old and new best blocks and the reorg size. Rather than force every callback to accept a fixed signature, the bus wraps the published keyword arguments in a small object, `EventArguments` (`pubsub.py:160`):

```
class EventArguments:
    def __init__(self, **kwargs: Any) -> None:
        for key, value in kwargs.items():
            setattr(self, key, value)

    def __contains__(self, key: str) -> bool:
        return hasattr(self, key)
```

It does nothing but turn `EventArguments(tx=vertex)` into an object with a `.tx` attribute, and let `'tx' in args` test for presence. The class body lists the attributes that may appear, with their types (`pubsub.py:164`), as a hint to readers and type-checkers — but any given instance only has the attributes that were actually passed. A subscriber reads the fields it knows its event carries.

The callback signature is fixed by a type alias at `pubsub.py:185`:

```
PubSubCallable = Callable[[HathorEvents, EventArguments], None]
```

Every subscriber is a function taking the event key and the argument bag, returning nothing.

PubSubManager : subscribe and publish

The bus itself is `PubSubManager` (`pubsub.py:188`). Its state (`pubsub.py:194`) is the dictionary you'd expect, plus a delivery queue and a reactor reference:

```
def __init__(self, reactor: Reactor) -> None:
    self._subscribers = defaultdict(list)
    self.queue: deque[tuple[PubSubCallable, HathorEvents, EventArguments]] = deque()
    self.reactor = reactor
    self.log = logger.new()
    self._call_later_id: IDelayedCall | None = None
```

`subscribe` (`pubsub.py:204`) is the toy's `subscribe` with one guard — a callback is not added twice:

```
def subscribe(self, key: HathorEvents, fn: PubSubCallable) -> None:
    if fn not in self._subscribers[key]:
        self._subscribers[key].append(fn)
```

`unsubscribe` (`pubsub.py:216`) removes it. So far, the toy.

The interesting part is `publish` (`pubsub.py:257`), because of when it runs the callbacks:

```
def publish(self, key: HathorEvents, **kwargs: Any) -> None:
    args = EventArguments(**kwargs)
    for fn in self._subscribers[key]:
        if not self.reactor.running:
            fn(key, args)
        else:
            self.queue.append((fn, key, args))
            self._schedule_call_next()
```

Read this carefully — it is the one genuinely non-obvious thing in the whole bus. There are two delivery modes:

1. **Reactor not running** (startup, or tests). Subscribers are called **synchronously, right now**, in the publishing call's own stack. There is no event loop yet to defer to.
2. **Reactor running** (normal operation). Each `(callback, key, args)` triple is **appended to a queue**, and a single deferred drain is scheduled via the reactor.

Why defer during normal operation? Because publishing often happens deep inside critical code — for instance, in the middle of accepting a vertex, while storage locks may be held and metadata is mid-update. Running an arbitrary subscriber's callback synchronously at that moment risks re-entrancy: the callback might publish another event, or call back into the half-updated subsystem. By queueing the callbacks and running them on the next reactor tick, the bus guarantees that subscribers run after the publishing operation has fully returned and the stack has unwound — a clean, well-defined point.

The drain is `_call_next` (`pubsub.py:222`):

```
def _call_next(self) -> None:
    if not self.queue:
        return
    try:
        while self.queue:
            fn, key, args = self.queue.popleft()
            fn(key, args)
    except Exception:
        self.log.error('event processing failed', key=key, args=args)
        raise
    finally:
        self._schedule_call_next()
```

It pops and calls every queued callback in FIFO order, then re-schedules itself in case new events were queued while it ran. Scheduling is `_schedule_call_next` (`pubsub.py:239`); its core is the reactor call at `pubsub.py:255`:

```
self._call_later_id = self.reactor.callLater(0, self._call_next)
```

`callLater(0, ...)` means "run this as soon as the loop is free." The method also guards against scheduling twice (`pubsub.py:252`) and, if called from a non-reactor thread, hops back onto the reactor thread via `callFromThread` (`pubsub.py:246`) — because Twisted requires that reactor-facing work happen on the reactor's own thread. That thread-safety hop is the reason `publish` can be called from a worker thread (e.g. the proof-of-work pool from Ch. 16) and still deliver safely.

The Layer-1 invariant. When the reactor is running, a published event is not delivered inline; it is delivered on a subsequent reactor tick, in `publish` order. Code that publishes must not assume subscribers have run by the time `publish` returns.

That is the entire in-process bus: a typed catalogue, a typed argument bag, a dictionary of callback lists, and a reactor-deferred drain. Every internal "X happened" announcement in the node flows through this one object. Among its many publishers, the most central is the **vertex handler** (Ch. 33), which on accepting a vertex publishes — in order — `NETWORK_NEW_TX_PROCESSING`, then any consensus-generated events, then `NETWORK_NEW_TX_ACCEPTED` (`hathor/vertex_handler/vertex_handler.py:259 – 262`).

30.5 The code, walked — Layer 2: the durable event queue

Everything so far is ephemeral. Now we build the persistent, external-facing feed on top of it. The central class is `EventManager` (`hathor/event/event_manager.py:59`), whose own docstring states the pipeline plainly (`event_manager.py:60`): "Events are received from PubSub, persisted on the storage and sent to WebSocket clients." Three stages — **receive, persist, broadcast** — map onto three collaborators: the `PubSubManager` it subscribes to, the `EventStorage` it writes to, and the `EventWebSocketFactory` it broadcasts through.

It is, itself, a subscriber

`EventManager` does not invent its own way of hearing about node activity. It is a Layer-1 subscriber — the bridge between the two layers. At startup it registers one handler for a fixed list of bus events (`event_manager.py:48`):

```
_SUBSCRIBE_EVENTS = [  
    HathorEvents.NETWORK_NEW_TX_ACCEPTED,  
    HathorEvents.REORG_STARTED,  
    HathorEvents.REORG_FINISHED,  
    HathorEvents.CONSENSUS_TX_UPDATE,  
    HathorEvents.CONSENSUS_TX_REMOVED,  
    HathorEvents.NC_EVENT,  
    HathorEvents.NC_EXEC_SUCCESS,  
]
```


and `_subscribe_events` (`event_manager.py:141`) wires them all to a single method:

```
def _subscribe_events(self) -> None:
    for event in _SUBSCRIBE_EVENTS:
        self._pubsub.subscribe(event, self._handle_hathor_event)
```

Note which bus events are not in the list: the peer-connection and wallet events stay purely internal. Only a curated subset of node activity is promoted to the durable, external feed — the events an outside indexer would actually want.

Translating a bus event into a durable record

When the bus delivers one of those events, `_handle_hathor_event` (`event_manager.py:176`) translates it from the internal vocabulary (`HathorEvents`) into the external one (`EventType`):

```
def _handle_hathor_event(self, hathor_event: HathorEvents, event_args: EventArguments) -> None:
    event_type = EventType.from_hathor_event(hathor_event)
    if event_type is not None:
        self._handle_event(event_type, event_args)
    ...
```

`EventType` (`hathor/event/model/event_type.py:31`) is a separate enum from `HathorEvents`, and the separation is deliberate: it is the **public** event vocabulary, and it does not match the internal one one-for-one. The mapping lives in a dictionary at `event_type.py:52`:

```
_HATHOR_EVENT_TO_EVENT_TYPE = {
    HathorEvents.NETWORK_NEW_TX_ACCEPTED: EventType.NEW_VERTEX_ACCEPTED,
    HathorEvents.REORG_STARTED:           EventType.REORG_STARTED,
    HathorEvents.REORG_FINISHED:          EventType.REORG_FINISHED,
    HathorEvents.CONSENSUS_TX_UPDATE:     EventType.VERTEX_METADATA_CHANGED,
    HathorEvents.CONSENSUS_TX_REMOVED:    EventType.VERTEX_REMOVED,
    HathorEvents.NC_EVENT:                 EventType.NC_EVENT,
}
```

Notice the renaming: the internal `NETWORK_NEW_TX_ACCEPTED` becomes the public `NEW_VERTEX_ACCEPTED`; `CONSENSUS_TX_UPDATE` becomes `VERTEX_METADATA_CHANGED`. The public names are chosen for the external reader, who thinks in terms of "a vertex was accepted," not the node's internal plumbing. Keeping the two enums separate means the internal events can be renamed or reorganized without breaking the contract external consumers depend on. The `EventType` enum also includes types that have no `HathorEvents` source — `LOAD_STARTED`, `LOAD_FINISHED`, `FULL_NODE_CRASHED`, `TOKEN_CREATED` (`event_type.py:32 - 41`) — which the `EventManager` emits directly, not via the bus.

The persisted record: `BaseEvent`

The durable record is `BaseEvent` (`hathor/event/model/base_event.py:25`), a Pydantic model³. Its fields are the contract with the outside world:

```
class BaseEvent(BaseModel):
    id: NonNegativeInt          # unique, sequential – determines event order
    timestamp: float            # informative only; may not be monotonic
    type: EventType             # one of the public event types
    data: EventData             # typed payload, varies by type
    group_id: Optional[NonNegativeInt] = None # links related events (e.g. a reorg)
```

Each field models something the consumer needs. The `id` (`base_event.py:29`) is the spine of the whole queue: it is a gap-free, monotonically increasing integer, and it is what makes replay possible — "resume from id N" is a meaningful request precisely because ids are sequential and stored. The comment at `base_event.py:30` is candid that `timestamp` is not reliable for ordering (the system clock can move backwards); ordering is `id`'s job, not the timestamp's. The `data` field is a typed payload whose concrete type depends on `type` — a mapping declared at `event_type.py:61` (`NEW_VERTEX_ACCEPTED` → `TxData` , `REORG_STARTED` → `ReorgData` , and so on) and enforced by a model validator (`base_event.py:62`) that rejects any event whose data shape does not match its type. The `group_id` (`base_event.py:38`) ties together a burst of related events — every `VERTEX_METADATA_CHANGED` emitted during one reorg shares the reorg's group id, so a consumer can treat the burst as one logical operation.

Receive → persist → broadcast

The translation method funnels into `_handle_event` (`event_manager.py:188`), which is the three-stage pipeline in four lines:

```
def _handle_event(self, event_type: EventType, event_args: EventArguments) -> None:
    assert self.is_running, 'Cannot handle event, EventManager is not started.'
    event = self._handle_event_creation(event_type, event_args) # number & build it
    self._event_storage.save_event(event)                       # persist (Layer 2 durability)
    self._event_ws_factory.broadcast_event(event)               # push to live clients
    self._last_event = event
```

`_handle_event_creation` (`event_manager.py:200`) assigns the next id and the right `group_id`, then builds the `BaseEvent`. The id assignment is at `event_manager.py:315` — 0 for the very first event, otherwise `self._last_event.id + 1` — which is what guarantees the gap-free sequence. The `group_id` is handled by three small helpers depending on whether the event opens a group (`REORG_STARTED` , `_GROUP_START_EVENTS` at `event_manager.py:40`), closes one (`REORG_FINISHED` , `_GROUP_END_EVENTS` at `event_manager.py:44`), or falls inside one.

Then the two effects that distinguish Layer 2 from Layer 1: **save to storage** (durable, replayable) and **broadcast to WebSocket clients** (live delivery). A Layer-1 publish does neither; it just calls callbacks. This is the concrete difference between the two layers, in two method calls.

The storage: `EventRocksDBStorage`

Persistence is an interface, `EventStorage` (`hathor/event/storage/event_storage.py`), with one real implementation, `EventRocksDBStorage` (`hathor/event/storage/rocksdb_storage.py:36`). It reuses the same low-level RocksDB wrapper as the ledger (`rocksdb_storage.py:20`) and carves out two column families (`rocksdb_storage.py:28`):

```
_CF_NAME_EVENT = b'event'          # id (8-byte big-endian) → serialized BaseEvent
_CF_NAME_META  = b'event-metadata' # bookkeeping: last group id, node state, ...
```

The key for an event is its id encoded as 8 big-endian bytes (`rocksdb_storage.py:85`), which means RocksDB stores events in id order on disk — so "iterate from id N" is a cheap seek-and-scan, not a full table walk. That is exactly what `iter_from_event` does (`rocksdb_storage.py:47`): seek to the requested id, then yield each subsequent record, deserializing it from JSON. `save_event` (`rocksdb_storage.py:77`) enforces the no-gaps invariant at write time — it raises `ValueError` if the incoming id is not exactly `last + 1` (`rocksdb_storage.py:81`), so a bug that skipped an id would fail loudly rather than corrupt the feed silently.

The Layer-2 invariant. Event ids are contiguous from 0, enforced both when building an event (`event_manager.py:315`) and when storing it (`rocksdb_storage.py:81`). A consumer that has seen id N has, by construction, seen every id $\leq N$. This is what makes "resume from N" correct.

Streaming to the world: the WebSocket layer

The live feed reaches external clients over a WebSocket⁴, handled by the `event/websocket/` sub-package. We cover it lightly here and defer the general WebSocket machinery to Chapter 36; what matters now is how it consumes the queue.

`EventWebsocketFactory` (`hathor/event/websocket/factory.py:31`) is a Twisted/Autobahn factory (Ch. 16's factory pattern): it builds one `EventWebsocketProtocol` per connecting client and keeps the set of live connections (`factory.py:59`). When `EventManager` calls `broadcast_event` (`factory.py:84`), the factory pushes the new event to every connection that is ready to receive it.

The "ready" qualifier is the interesting part, and it is what makes the feed reliable rather than fire-and-forget. Each client connection is a small state machine (`EventWebsocketProtocol`, `protocol.py:34`) implementing **acknowledged, windowed delivery**:

- The client opens the stream with a `START_STREAM` request (`request.py:22`) that says "I last acknowledged event id X; my window size is W." — i.e. "resume after X, and don't send me more than W unacknowledged events at a time."
- The node sends events strictly in id order, but only up to W ahead of the last id the client has acknowledged. The check is `can_receive_event` (`protocol.py:58`): the stream must be active, the event must be exactly the next one expected, and the number of in-flight (sent-but-unacked) events must be below the window.
- The client periodically sends `ACK` (`request.py:35`) to confirm it processed up to some id and to adjust its window. Each ack lets the node send more (`protocol.py:127`).

This back-pressure⁵ keeps a slow consumer from being flooded, and the resume-from-last-ack semantics, combined with the durable queue, are what let a consumer disconnect and reconnect without losing or duplicating events. If the client falls behind or reconnects, `send_next_event_to_connection` (`factory.py:105`) reads the missing events straight out of `EventStorage` and replays them — the same persisted records, no matter how long the client was away. That is the durable queue earning its keep.

The replayable backlog at startup

One more piece ties durability to the node's own history. When the event queue is first enabled on a node that already has a populated ledger, there is a backlog: thousands of vertices were accepted before the feed existed, and a faithful feed must include them. `handle_load_phase_vertices` (`event_manager.py:334`) handles exactly this. During the node's load phase it walks the existing vertices in topological order and manufactures a `NEW_VERTEX_ACCEPTED` event for each, batching them into storage in chunks of `N_LOAD_EVENTS_PER_BATCH` (`= 10,000`, `event_manager.py:38`). The decision to do this hinges on `_should_reload_events` (`event_manager.py:322`): if the previous node state was `None` or `LOAD`, the queue is (re)built from scratch; otherwise the node resumes from where it left off. This is why a consumer can trust the feed to be a complete history of the ledger, not merely "everything since the feature was switched on in this session."

30.6 How it plugs into the lifecycle

Both layers are created and started by machinery you've already met. The thread, in order:

Creation (the builder, Ch. 24). The `Builder` creates the single `PubSubManager` lazily via `_get_or_create_pubsub` (`hathor/builder/builder.py:444`), passing it the reactor; every subsystem that needs the bus is handed this same instance. The durable layer is assembled in `_get_or_create_event_manager` (`builder.py:538`): it builds the `EventRocksDBStorage` (`builder.py:535`), the `EventWebsocketFactory` (`builder.py:544`), and wires them — together with the shared `pubsub` — into the `EventManager` (`builder.py:550`). All of this lands on the `HathorManager` the builder produces.

Wiring (the manager, Ch. 29). The manager stores the bus as `self.pubsub` (`hathor/manager.py:189`) and hands it to subsystems that publish or subscribe — for example the wallet (`manager.py:223`) and, via `register_wallet`, a `NETWORK_NEW_TX_PROCESSING` subscription (`manager.py:261`).

Startup (the manager, Ch. 29). Inside `_initialize_components` the order is exact and worth noting (`manager.py:304`):

```
if self._enable_event_queue:
    self._event_manager.start(str(self.my_peer.id))    # Layer 2 subscribes to the bus

self.state = self.NodeState.INITIALIZING
self.pubsub.publish(HathorEvents.MANAGER_ON_START)  # first Layer-1 event
self._event_manager.load_started()                  # first Layer-2 event
```

The `EventManager` subscribes to the bus before the manager publishes `MANAGER_ON_START`, so no early event is missed. `EventManager.start` (`event_manager.py:92`) is where the durable layer comes alive: it decides whether to reload the queue or resume (`event_manager.py:101`), refuses to start if the previous run left an event group unclosed (`event_manager.py:110`, the crash-detection guard), subscribes to the bus (`event_manager.py:111`), and starts the WebSocket factory (`event_manager.py:114`). The load-phase backlog is generated later in initialization (`manager.py:476`), and `load_finished` is called when initialization completes (`manager.py:481`).

Steady state. From here, every accepted vertex (Ch. 32–33) flows through `vertex_handler` → `pubsub.publish(NETWORK_NEW_TX_ACCEPTED)` → bus drains on the next reactor tick → `EventManager._handle_hathor_event` → persisted as `NEW_VERTEX_ACCEPTED` → broadcast to WebSocket clients. The whole nervous system, end to end, on one event.

Shutdown. On stop, the manager publishes `MANAGER_ON_STOP` (`manager.py:355`) and stops the event manager (`manager.py:377`), which closes the WebSocket factory.

Recap

Layer	Purpose	Persistence	Audience	Built on
<code>PubSubManager</code> (<code>pubsub.py:188</code>)	decouple producers from consumers inside the process	none — ephemeral	the node's own components	callbacks (Ch 2) + observer pattern (Ch 3)
<code>EventManager</code> + event queue (<code>event_manager.py:59</code>)	a durable, ordered, replayable feed	RocksDB, gap-free <code>id</code> (<code>rocksdb_storage.py:36</code>)	external systems	the Layer-1 bus
<code>EventRocksDBStorage</code> (<code>rocksdb_storage.py:36</code>)	persist events as <code>id</code> → <code>BaseEvent</code>	RocksDB, two column families	(internal to Layer 2)	RocksDB (Ch 27)
<code>EventWebsocketFactory</code> / <code>Protocol</code> (<code>factory.py:31</code> , <code>protocol.py:34</code>)	stream the queue with ack + back-pressure	reads from storage on replay	external WebSocket clients	Autobahn/ Twisted (Ch 16, 36)

The two systems are one idea applied at two scales. `PubSubManager` is the observer pattern — a dictionary of callback lists — used so the node's components never have to know about each other; it is ephemeral by design, because every component that cares is alive at the same instant. `EventManager` is a subscriber to that bus that adds the three things outsiders need and insiders don't: a sequential `id`, durable RocksDB storage, and acknowledged WebSocket streaming. Keep the dividing line sharp — in-process and gone versus persisted and replayable — and the rest of the package is detail.

Events are how the node announces that something happened. The next chapter, **Chapter 31 (Verification)**, is about the very first stage a new vertex passes through before any such announcement is possible — the rule-checking that decides whether a vertex is even allowed to enter the ledger and, eventually, fire the `NEW_VERTEX_ACCEPTED` event you now know the full life of.

-
1. Decoupling means reducing how much two pieces of code need to know about each other. Tightly coupled code shares references and assumptions, so a change to one forces changes to the other. Loosely coupled code communicates through a narrow, stable interface — here, "publish an event" — so each side can change freely. ↩
 2. The event queue is Hathor's name for the durable, numbered log of events on disk. "Queue" here means an ordered sequence consumers read in order, not a transient in-memory buffer. It is the persistent half of the event system, off by default and enabled per node. ↩
 3. Pydantic is a Python library for declaring data models as classes with typed fields; it validates and coerces data at runtime and can serialize to/from JSON. Full treatment in Ch. 18. Here it gives `BaseEvent` its field types and the validator that keeps each event's data shape matching its type. ↩
 4. A WebSocket is a persistent, two-way connection between a client and a server over a single long-lived TCP link, unlike HTTP's request-then-close model. It lets the node push events to a client the moment they happen, rather than the client polling. Full treatment in Ch. 36. ↩
 5. Back-pressure is a flow-control technique where a fast producer is held back so a slower consumer is not overwhelmed. Here the client declares a window (how many unacknowledged events it can hold), and the node refuses to send beyond it until the client acknowledges, keeping memory bounded on both sides. ↩

Chapter 31 — Verification: Checking the Rules

What you'll learn in this chapter

- The single question this package answers: is this vertex valid on its own? — and why that is a different question from the one consensus (Ch 32) asks.
- The shape of `hathor/verification/`: one orchestrator (`VerificationService`) and a family of per-type verifiers, wired together by `VertexVerifiers`.
- The **basic-vs-full** split — why some checks can run with no other data on hand, and others need the rest of the ledger — and how that split maps onto a vertex's validation state.
- The concrete rules, each tied to code: proof-of-work, minimum weight, value conservation and token rules, structural parent/timestamp rules, sigops bounds, and the reward lock.
- **Script evaluation as a stack machine** — built up from a neutral toy first, then the real P2PKH script and the opcodes behind it. This is where Chapter 7's "ownership is a satisfiable lock" finally gets paid off in code.
- Where verification sits in the lifecycle: invoked by the vertex handler (Ch 33) on every new vertex, before consensus — and what "it failed verification" means for the ledger.

A full node never takes another node's word for anything. When a block or transaction arrives — downloaded during sync, broadcast by a peer, or pushed in through the API — the node's first job is to ask: does this thing obey the protocol's rules at all? Not "is it the version of history we'll keep" — that is consensus, the next chapter. The narrower, prior question is: is this vertex even **well-formed and rule-abiding**, considered by itself? That is what `hathor/verification/` decides.

This is a gate, and it is unforgiving. A vertex that fails verification is rejected outright. It is never written to storage as valid, never indexed, never shown to wallets, never given to consensus to weigh. The rest of the node gets to assume that anything it handles has already passed through here.

31.1 Localization

The package is small and flat. Each file holds one verifier class; one file holds the orchestrator that calls them in the right order.

```
hathor-core/
└─ hathor/
    └─ verification/
        └─ verification_service.py      ◀ YOU ARE HERE
        └─ vertex_verifiers.py          ◀ VertexVerifiers: the bundle of all verifiers
        └─ verification_params.py       ◀ VerificationParams: per-run settings/flags
        └─ vertex_verifier.py           ◀ VertexVerifier: checks shared by ALL vertices
        └─ block_verifier.py            ◀ BlockVerifier: block-only rules
        └─ transaction_verifier.py      ◀ TransactionVerifier: tx-only rules
        └─ merge_mined_block_verifier.py ◀ aux-pow (merged mining)
        └─ poa_block_verifier.py        ◀ proof-of-authority block rules
        └─ token_creation_transaction_verifier.py
        └─ nano_header_verifier.py      ◀ nano-contract header rules (→ Ch 39)
        └─ fee_header_verifier.py       ◀ fee-header rules
        └─ on_chain_blueprint_verifier.py ◀ on-chain blueprint rules (→ Ch 39)
    └─ transaction/
        └─ scripts/
            └─ execute.py               ◀ the script "virtual machine" lives here, not in veri
            └─ opcode.py                 ◀ script_eval / execute_eval: the evaluator
            └─ p2pkh.py                  ◀ the Opcode enum + one function per opcode
            └─ multi_sig.py               ◀ the standard pay-to-public-key-hash script
            └─ construct.py              ◀ the m-of-n multisignature script
            └─ helpers to build/parse scripts
```

One thing to fix in mind early: the **script evaluator does not live in `verification/`**. It lives in `hathor/transaction/scripts/`, alongside the vertex model. Verification calls into it — when the transaction verifier needs to check that an input is allowed to spend an output, it hands the two scripts to `script_eval` (`transaction/scripts/execute.py:103`). We treat the script machine here, in this chapter, because checking signatures is a verification rule; but the code that implements it is shared with wallets and miners, so it sits in the model package, not the rule package.

Context. Verification is the "is this even valid?" stage of the ingestion pipeline you met in §0.3. Every new vertex flows: **verify** (this chapter) → **consensus** (Ch 32, which decides canonical history) → **store + index** (Ch 27–30). The component that runs this pipeline is the vertex handler (Ch 33). Verification's contract with the rest of the node is simple and absolute: if it passes here, it broke no protocol rule; if it fails here, it is discarded and goes no further.

31.2 What it does and why it exists

Imagine the node had no verification stage and trusted whatever arrived. A malicious peer could send a "transaction" that spends coins it never owned (no valid signature), or that prints money out of thin air (outputs worth more than inputs), or a "block" whose proof-of-work was never actually done. The node would store it, the indexes would record fake balances, and — worse — it would relay the garbage on to its peers. The whole network's promise ("trust no one, check everything") would collapse.

Verification exists so that **each vertex carries its own proof of legitimacy, and the node checks that proof in isolation**. "In isolation" is the key phrase. A verifier may need to read the inputs a transaction references (to see how much they were worth and what lock they carry), and it may need the parents (to check timestamps), but it does **not** ask "is this the best chain?" or "does this conflict with some other transaction the network prefers?" Those are global, comparative questions about which history wins. Verification only asks the local, absolute question: taken on its own terms, does this vertex break any rule?

That distinction maps onto a vocabulary you met in Chapter 10.

Recap — invalid vs. conflict (full treatment in Ch. 10 & 32). A vertex is **invalid** when it breaks a protocol rule: bad signature, bad proof-of-work, money created from nothing, malformed structure. An invalid vertex is **wrong** — it can never be part of any valid history, so it is rejected and forgotten. A vertex is in **conflict** when it is perfectly valid on its own but competes with another valid vertex (two transactions spend the same coin). Conflicts are not errors; they are resolved by consensus, which marks the loser **voided** (kept, but flagged as not-counting) rather than deleting it. **Verification handles invalidity. Consensus handles conflict.** This chapter is entirely about the first; → Ch 32 for the second.

So the package's job, stated tightly: **given one vertex, run every applicable protocol rule against it, raising an exception the moment any rule is broken; raise nothing and the vertex is valid.** The verifiers do not return `True / False` per rule — they either return quietly or raise a specific exception (`PowError`, `InputOutputMismatch`, `InvalidInputData`, ...). Verification is, in effect, a long list of assertions about correctness.

31.3 The concepts it rests on

Verification is almost pure consumer of ideas defined earlier in the book. It is where those ideas get enforced. Four recap boxes set the stage; the rest of the chapter shows the enforcement code.

Recap — the vertex and its scripts (full treatment in Ch. 25). Every block and transaction is a vertex. A `Transaction` has **inputs** (each naming an earlier output it wants to spend, `TxInput.tx_id + TxInput.index`, plus an `data` field holding an unlocking script) and **outputs** (each carrying a `value`, a `token_data` byte, and a `script` — the locking script). A `Block` has outputs (its reward) but no inputs. Verification reads these fields and checks the rules over them. → Ch 25 for the full data model.

Recap — the UTXO model and value conservation (full treatment in Ch. 7). Hathor tracks unspent transaction outputs (UTXOs), not account balances. A transaction consumes whole outputs as inputs and creates new outputs. The core monetary rule: for each token, the sum of inputs must equal the sum of outputs — coins are neither created nor destroyed — **unless** the transaction holds explicit mint or melt authority for that token. Ownership of an output is "whoever can satisfy its locking script." Verification is where that rule and that ownership check are actually performed. → Ch 7.

Recap — proof-of-work, weight, target (full treatment in Ch. 9). Each vertex carries a `weight` (a float; `weight = log2` of the expected number of hashing attempts). From the weight you derive a numeric **target**: `target = 2** (256 - weight) - 1` (`base_transaction.py:361`). Proof-of-work is valid when the vertex's own hash, read as a 256-bit integer, is **less than** that target — i.e. the miner found a hash small enough, which on average takes `2**weight` tries. Verification also checks that the weight is at least the network-required minimum, so a vertex can't claim a trivially-easy target. → Ch 9.

Recap — validation state (full treatment in Ch. 25). Each vertex's metadata records a `ValidationState`: `INITIAL` → `BASIC` → `FULL` (or the checkpoint variants), or `INVALID` (`transaction/validation_state.py:46–50`). It records how far verification has progressed. `BASIC` means "the checks that need no other data have passed." `FULL` means "every check, including those that read the inputs and parents, has passed." This state is what lets the node verify a vertex in two stages — central to §31.5.

31.4 The code, walked — the orchestrator and the verifier family

31.4.1 One service, many verifiers

The package follows a clean division of labour. There is **one orchestrator** — `VerificationService` (`verification_service.py:34`) — whose job is to call the right checks, in the right order, for whatever vertex type it is handed. The actual checks live in a **family of small verifier classes**, one per vertex type, each holding only the rules specific to that type:

- `VertexVerifier` (`vertex_verifier.py:52`) — rules shared by every vertex: proof-of-work, parents, outputs, sigops on outputs, headers, version.
- `BlockVerifier` (`block_verifier.py:35`) — block-only rules: reward amount, height, no inputs, checkpoints, mandatory feature signalling.
- `TransactionVerifier` (`transaction_verifier.py:67`) — tx-only rules: input scripts (signatures), value conservation, conflicts within the tx, the reward lock.
- `MergeMinedBlockVerifier`, `PoaBlockVerifier`, `TokenCreationTransactionVerifier`, plus the nano/fee/blueprint header verifiers — the niche types.

These eight are bundled into a single `NamedTuple`, `VertexVerifiers` (`vertex_verifiers.py:32`), built once at boot by `create_defaults` (`vertex_verifiers.py:43`) and handed to the service. The service reaches them as `self.verifiers.vertex`, `self.verifiers.block`, `self.verifiers.tx`, and so on. Why a `NamedTuple` and not a loose pile of constructor arguments? It is one object to pass around, every field is named and typed, and it is immutable — you cannot accidentally swap a verifier at runtime.

Recap — the facade pattern (full treatment in Ch. 3). `VerificationService` is a facade: callers (the vertex handler) say "validate this vertex" and the service hides the messy choreography — which type it is, which verifiers to call, in which order, with which checks skipped. The caller never touches an individual verifier. This keeps the rule-ordering logic in exactly one place. → Ch 3 for the pattern.

31.4.2 Dispatch by vertex type

How does the service know which type-specific checks to run? It reads the vertex's `version` field and branches with a `match` statement. Here is the pattern, from `verify` (`verification_service.py:182`):

```

match vertex.version:
    case TxVersion.REGULAR_BLOCK:
        assert type(vertex) is Block
        self._verify_block(vertex, params)
    case TxVersion.REGULAR_TRANSACTION:
        assert type(vertex) is Transaction
        self._verify_tx(vertex, params)
    case TxVersion.TOKEN_CREATION_TRANSACTION:
        ...
    case _: # pragma: no cover
        assert_never(vertex.version)

```

Two details worth noticing, because they are deliberate. First, the `assert type(vertex) is Block` uses `type(...) is` rather than `isinstance(...)`; the comment at `verification_service.py:114` explains it — each subclass gets its own branch, so an exact-type check (not a subclass-accepting one) catches a vertex that lied about its version. Second, the final `case _: assert_never(...)` is a typing trick: `assert_never`¹ makes the type-checker prove that every `TxVersion` has a branch. If someone adds a new vertex version and forgets to handle it here, mypy fails the build. The dispatch is exhaustive by construction.

31.5 The basic-vs-full split

This is the single most structural idea in the package, so it earns its own section.

Some rules can be checked with **nothing but the vertex itself in hand**. Is the proof-of-work valid? Just hash the bytes and compare to the target — no other data needed. Are the outputs all positive and not too numerous? Just read the outputs. Is the weight a finite number? Just read the field.

Other rules **require the rest of the ledger**. To check a transaction's signatures, you must fetch the outputs it is spending — and those live in other transactions, which must already be in storage. To check value conservation, you must know how much each spent output was worth. To check parents' timestamps, you must fetch the parents.

Hathor names these two tiers **basic** and **full**, and `VerificationService` exposes them as two entry points:

```

def validate_basic(self, vertex, params) -> bool:      # verification_service.py:50
    ...
    self.verify_basic(vertex, params)
    vertex.set_validation(ValidationState.BASIC)

def validate_full(self, vertex, params, ...) -> bool:  # verification_service.py:64
    ...
    self.verify(vertex, params)
    vertex.set_validation(ValidationState.FULL)

```

Note the naming convention, because it is easy to trip on: the `validate_*` methods (with the `d`) are the public entry points — they run the checks and update the vertex's validation state. The `verify_*` methods are the workers — they run the checks and raise on failure but **do not** touch the validation state (`verify_basic`'s docstring says so explicitly, `verification_service.py:107`). So `validate_basic` = `verify_basic` + record the new state.

The internal split runs all the way down. The pure-no-dependency checks are gathered under `verify_without_storage` (`verification_service.py:291`) — its very name advertises the contract: these checks read no other transaction. It is what `verify_basic` leans on, and it is also re-run as the first step of full verification. The dependency-needing checks live in `verify` (`verification_service.py:172`), which for a transaction calls `verify_inputs` (signatures, by reading the spent outputs), `verify_sum` (conservation), `verify_parents`, and the reward lock.

Why split at all? Because of sync order. When a node is catching up, it may receive a transaction before it has the transactions that transaction spends. It cannot run full verification yet — the dependencies are not on disk. But it can run basic verification immediately (the proof-of-work is checkable in isolation), mark the vertex `BASIC`, and store it. Later, once every dependency has reached `FULL`, the node circles back and promotes the vertex to `FULL`. The basic tier is, in effect, a cheap early filter: it rejects obvious garbage (failed PoW) the instant it arrives, without waiting for data that may be hours away.

Here is the basic tier for a regular transaction, so you can see exactly which checks are deemed "no dependencies needed" (`verification_service.py:156`):

```
def _verify_basic_tx(self, tx, params) -> None:
    if tx.is_genesis:
        return
    self.verifiers.tx.verify_parents_basic(tx)           # count parents, no fetch
    if self._settings.CONSENSUS_ALGORITHM.is_pow():
        self.verifiers.tx.verify_weight(tx)             # weight >= minimum
    self.verify_without_storage(tx, params)             # PoW, output rules, sigops...
```

`verify_parents_basic` (`transaction_verifier.py:81`) only counts parents and checks for duplicates — it does not fetch them, so it qualifies as basic. The deeper parent check, `verify_parents` (`vertex_verifier.py:65`), does fetch every parent (to compare timestamps and confirm the right kinds of parent), so it belongs to full verification.

Genesis is exempt. Throughout the verifiers you will see early returns like `if tx.is_genesis: return` (e.g. `verification_service.py:158`, `:220`, `:259`). The genesis block and transactions are hard-coded into the settings and agreed by every node by definition — there is no earlier history to validate them against, so they are accepted axiomatically. → Ch 0 / Ch 22 for genesis.

31.6 Proof-of-work: hash below target

The first rule any vertex must pass — and the cheapest to check — is proof-of-work. It is in the no-storage tier (`verification_service.py:335` for blocks, `:348` for txs), and the check itself is six lines (`vertex_verifier.py:142`):

```
def verify_pow(self, vertex, *, override_weight=None) -> None:
    assert self._settings.CONSENSUS_ALGORITHM.is_pow()
    numeric_hash = int(vertex.hash_hex, vertex.HEX_BASE)      # hash as a 256-bit int
    minimum_target = vertex.get_target(override_weight)        # 2**(256-weight) - 1
    if numeric_hash >= minimum_target:
        raise PowError(...)
```

The logic is exactly the Chapter 9 rule. The vertex's hash is a 256-bit number (read from hex, `HEX_BASE = 16`, `base_transaction.py:159`). The target is derived from the vertex's claimed weight (`get_target`, `base_transaction.py:361`). A higher weight means a smaller target, which means fewer acceptable hashes, which means more work to find one. If the hash is **not** below the target, the miner did not do the work they claimed, and `PowError` is raised.

But `verify_pow` only checks the hash against the weight the vertex claims. What stops a cheater from claiming `weight = 1` (trivial to mine) on a real transaction? A separate check: **minimum weight**. For transactions, `verify_weight` (`transaction_verifier.py:93`) recomputes the minimum weight the difficulty algorithm requires for a tx of this size and amount, and rejects anything meaningfully below it:

```
def verify_weight(self, tx) -> None:
    min_tx_weight = self._daa.minimum_tx_weight(tx)
    ...
    if tx.weight < min_tx_weight - self._settings.WEIGHT_TOL:
        raise WeightError(...)
```

For blocks the analogue is `BlockVerifier.verify_weight` (`block_verifier.py:60`), which calls the difficulty-adjustment algorithm² to compute the required block weight from the recent block history. So PoW is two rules working together: you must do the work for the weight you claim (`verify_pow`) **and** the weight you claim must be at least what the network demands (`verify_weight`). One without the other would be a hole.

31.7 Value conservation and token rules

The monetary heart of transaction verification is `verify_sum` (`transaction_verifier.py:257`). It enforces the conservation rule from Chapter 7: for each token, money in must equal money out, unless authority says otherwise.

The work begins before `verify_sum`, with a `token_dict` — a summary, per token, of how much that token is being minted or melted by the transaction. The convention (documented at `transaction_verifier.py:271`) is `amount = outputs - inputs`, so a

positive amount means the tx is trying to create that token (mint) and a negative amount means destroy it (melt). `verify_sum` then walks each token and applies the rules:

- For any non-native token whose amount is nonzero, `_check_token_permissions` (`transaction_verifier.py:337`) demands the matching authority: melting needs melt authority (`ForbiddenMelt`), minting needs mint authority (`ForbiddenMint`). No authority, no creation or destruction.
- For the **native token HTR**, the rule is strict and absolute. HTR cannot be minted or melted by an ordinary transaction; the only legitimate sources of new HTR are block rewards and the deposit/withdraw mechanics of custom tokens. The code computes an `htr_expected_amount` from those mechanics and then asserts the tx's HTR delta matches exactly: a surplus raises `InputOutputMismatch` ("invalid surplus of HTR", `:310`), a deficit raises the same ("invalid deficit", `:329`), and the function ends on `assert htr_info.amount == htr_expected_amount` (`:335`). HTR is conserved to the satoshi.

Two supporting checks sit nearby. `verify_outputs` (`vertex_verifier.py:153`) rejects any output with a non-positive value (`InvalidOutputValue`) and forbids an authority output on the native token (`InvalidToken` — you cannot hold mint/melt authority over HTR, because HTR has no mint/melt). And `verify_output_token_indexes` (`transaction_verifier.py:247`) makes sure every output's `token_data` index points at a token actually listed in the transaction.

A small worked check. Suppose a transaction has two inputs worth 5 HTR and 3 HTR (8 in) and two outputs worth 6 HTR and 2 HTR (8 out). The HTR delta is $8 - 8 = 0$; `htr_expected_amount` is 0 (no token deposit/withdraw); `0 == 0`, conservation holds, accepted. Now suppose the outputs were 6 and 3 (9 out): delta $9 - 8 = +1$, a surplus of 1 HTR — money from nothing — and `verify_sum` raises `InputOutputMismatch`. That single comparison is what makes the currency real.

For **token-creation** transactions there is an extra layer:

`TokenCreationTransactionVerifier` (`token_creation_transaction_verifier.py:24`) checks the new token's name and symbol are well-formed and that the version is permitted (`verify_token_info`, `:43`), and that the tx genuinely mints a positive amount of the new token (`verify_minted_tokens`, `:30`).

31.8 Script evaluation — the heart of signature checking

This is the deepest idea in the chapter, and the one that pays off Chapter 7's slogan: ownership is a satisfiable lock. We build it up carefully, because the mechanism — a tiny **stack machine** — may be new to you.

31.8.1 A stack machine, from scratch

A **stack**³ is a list you only touch at one end: you push a value on top, and you pop the top value off. Last in, first out. A **stack machine** is a way of evaluating a program where every operation works by pushing and popping this single shared stack. There are no variables and no addresses — just the stack and a list of instructions.

Here is a neutral toy. Suppose our instructions are: numbers (which mean "push me") and the word `ADD` (which means "pop two numbers, push their sum"). Evaluate the program `3 4 ADD`:

instruction	action	stack after
-----	-----	-----
3	push 3	[3]
4	push 4	[3, 4]
ADD	pop 4, pop 3, push 3+4=7	[7]

The program ends with `[7]` on the stack — that is its result. Now add a verb `EQ` ("pop two, push 1 if equal else 0") and a rule: the program is valid only if it ends with exactly one value, and that value is 1 (true). Evaluate `5 5 EQ`:

```
5    push 5    [5]
5    push 5    [5, 5]
EQ   5==5 → 1  [1]
```

Ends with `[1]` — valid. The program `5 6 EQ` would end with `[0]` — invalid. That is the entire conceptual model: instructions push data or transform the top of the stack, and "success" is a single `1` left at the end. Hathor's script language is exactly this idea with cryptographic verbs added.

31.8.2 The locking/unlocking split

Recall from Chapter 7 that ownership is encoded as two scripts. The **output** carries a locking script (the puzzle: "to spend me, satisfy this"). The **input** that wants to spend it carries an unlocking script (the solution: "here is the proof"). To check that the spend is legitimate, the node **concatenates the unlocking script followed by the locking script and runs the whole thing on one stack**. If it ends with a single `1`, the spend is authorized.

In code this is `script_eval ( transaction/scripts/execute.py:103 )`, which the transaction verifier calls once per input via `verify_script ( transaction_verifier.py: 172 )`:

```
script_eval(tx, input_tx, spent_tx, params.features.opcodes_version)
```


`script_eval` reaches into the spent transaction, grabs the locking script of the exact output being spent (`spent_tx.outputs[txin.index].script`), and hands it plus the input's unlocking data to `raw_script_eval` (`execute.py:125`), which does the concatenation:

```
full_data = input_data + output_script    # unlock, then lock
execute_eval(full_data, log, extras)
```

`execute_eval` (`execute.py:54`) is the loop. It walks the bytes; pushdata bytes go on the stack, opcodes call their handler (`execute_op_code`); at the end it calls `evaluate_final_stack` (`execute.py:86`), which enforces our toy's exact rule — **the stack must hold exactly one value, and that value must be 1** — raising `FinalStackInvalid` otherwise. The toy from §31.8.1 is the real success condition.

31.8.3 The real script: P2PKH

The standard Hathor output script is **P2PKH** — pay to public key hash (`scripts/p2pkh.py:25`). Its locking script is a fixed sequence of opcodes (`create_output_script` , : 70):

```
OP_DUP  OP_HASH160  <pubKeyHash>  OP_EQUALVERIFY  OP_CHECKSIG
```

And the matching unlocking data the spender supplies (`create_input_data` , : 94) is just two pushed values:

```
<signature>  <pubKey>
```

In English the locking script says: "to spend this coin, present a public key that hashes to this stored hash, and a signature that key validates." That is the lock; only the holder of the matching private key can produce both halves. Let us trace the concatenated program `<sig> <pubKey> OP_DUP OP_HASH160 <pubKeyHash> OP_EQUALVERIFY OP_CHECKSIG` step by step on the stack:

instruction	action	stack after
-----	-----	-----
<code><sig></code>	push the signature	<code>[sig]</code>
<code><pubKey></code>	push the public key	<code>[sig, pubKey]</code>
<code>OP_DUP</code>	duplicate top	<code>[sig, pubKey, pubKey]</code>
<code>OP_HASH160</code>	pop pubKey, push SHA256→RIPEMD160 of it	<code>[sig, pubKey, hash(pubKey)]</code>
<code><pubKeyHash></code>	push the hash stored in the output	<code>[sig, pubKey, hash(pubKey), storedHash]</code>
<code>OP_EQUALVERIFY</code>	pop two, fail unless equal	<code>[sig, pubKey]</code>
<code>OP_CHECKSIG</code>	pop pubKey & sig, verify; push 1 if valid	<code>[1]</code>

It ends with `[1]` — valid. Each opcode is one small function in `opcode.py` :

- `op_dup` (`opcode.py:166`) pushes a copy of the top item. (This is how the public key gets used twice — once to hash, once to verify.)

- `op_hash160` (`opcode.py:275`) pops the top, hashes it SHA-256 then RIPEMD-160, pushes the digest. This is how an address is derived from a public key.
- `op_equalverify` (`opcode.py:200`) pops two items, and raises `EqualVerifyFailed` unless they are equal. This is the check "the key you gave hashes to the address this coin was sent to" — i.e. you are spending the right person's coin.
- `op_checksigs` (`opcode.py:238`) pops a public key and a signature, and verifies the signature against the transaction's signing data (`tx.get_sighash_all_data()` , `opcode.py:266`) using that key. Valid → push `1`; invalid → push `0`. This is the proof that the spender holds the private key.

So the two cryptographic facts — you control the right address (`OP_EQUALVERIFY`) and you can sign for it (`OP_CHECKSIG`) — are what "ownership" reduces to, exactly as Chapter 7 promised. There is no account, no balance, no list of who-owns-what; there is only a lock and the question of whether you can open it.

31.8.4 The opcode dispatch table

How does the evaluator turn an opcode byte into the right function? A dictionary, in `execute_op_code` (`opcode.py:635`):

```
opcode_fns = {
    Opcode.OP_DUP: op_dup,
    Opcode.OP_EQUAL: op_equal,
    Opcode.OP_EQUALVERIFY: op_equalverify,
    Opcode.OP_CHECKSIG: op_checksigs,
    Opcode.OP_HASH160: op_hash160,
    ...
}
opcode_fn = opcode_fns.get(opcode)
if opcode_fn is None:
    raise ScriptError(f'unknown opcode: {opcode}')
opcode_fn(context)
```

Recap — dispatch table (full treatment in Ch. 4). Instead of a long `if opcode == ... elif ...` chain, the evaluator maps each opcode to its handler in a dictionary and calls whatever it finds. Adding an opcode is adding a row. An unknown opcode falls through to `None` and is rejected — the script language is closed, only the listed verbs exist. → Ch 4 for dispatch.

Notice the `version` parameter: some opcodes (the `OP_DATA_*` family, `OP_CHECKDATASIG`) are only added to the table under `OpcodesVersion.V1` (`opcode.py:654`). This is how the protocol can retire or gate opcodes over time via feature activation (Ch 38) without breaking old vertices.

Two design choices to absorb. First, the language is **deliberately limited** — there are no loops and no jumps. A script is a straight-line list of instructions that runs once, top to bottom. That is on purpose: a script that cannot loop cannot run forever, so

verification has a bounded cost. (Hathor's general-purpose programmable money is nano-contracts, Ch 39, which run under an explicit resource meter precisely because they are expressive.) Second, the multisig case (`raw_script_eval` , `execute.py:128`) runs the evaluator twice — once to check the redeem script matches its hash, once to check the signatures satisfy it — because a single concatenated pass would leave leftover signatures on the stack and fail the "exactly one value" rule.

31.9 Bounding the work: sigops, sizes, counts

Verification must be cheap, because the node runs it on every vertex from every peer; an attacker who could make one vertex expensive to check could grind the node to a halt. So several rules cap the size of the work, independent of correctness:

- **Sigops.** Signature verification is the most expensive operation in a script. The node counts the signature operations across all outputs (`verify_sigops_output` , `vertex_verifier.py:182`) and all inputs (`verify_sigops_input` , `transaction_verifier.py:105`), and rejects a vertex whose totals exceed `MAX_TX_SIGOPS_OUTPUT` / `MAX_TX_SIGOPS_INPUT` with `TooManySigOps` . A `SigopCounter` walks the script bytes counting `OP_CHECKSIG` -class opcodes without actually executing them.
- **Counts.** At most `MAX_NUM_INPUTS` inputs and `MAX_NUM_OUTPUTS` outputs (`verify_number_of_inputs` , `transaction_verifier.py:237` ; `verify_number_of_outputs` , `vertex_verifier.py:177`), raising `TooManyInputs` / `TooManyOutputs` .
- **Sizes.** Each input's unlocking data is capped at `MAX_INPUT_DATA_SIZE` (`transaction_verifier.py:146`) and each output script at `MAX_OUTPUT_SCRIPT_SIZE` (`vertex_verifier.py:172`).

None of these are correctness rules in the "is this fraud?" sense. They are denial-of-service defenses: a bound on how much computation one vertex can demand.

31.10 Structural rules: parents, timestamps, no double-spend within a tx

Beyond money and signatures, a vertex must be structurally sound.

Parents. `verify_parents` (`vertex_verifier.py:65`) is the structural workhorse of full verification. It fetches every parent from storage (raising `ParentDoesNotExist` if one is missing), and checks: no duplicate parents (`DuplicatedParents`); each parent's timestamp is strictly before the vertex's (`TimestampError` — a vertex cannot confirm something from its own future); and the right shape of parents — a transaction must

have exactly 2 transaction parents and 0 block parents, a block exactly 2 transaction parents and 1 block parent (the constants at `vertex_verifier.py:42–47`), with blocks ordered before transactions, else `IncorrectParents`.

Recap — parents vs. inputs (full treatment in Ch. 8 & 25). A vertex has two kinds of edge. **Parents** are the DAG confirmation links ("I build on these vertices"); **inputs** are the spending links ("I consume these outputs"). `verify_parents` checks the parent edges; `verify_inputs` checks the input edges. They are entirely separate checks over entirely separate fields. → Ch 8.

No double-spend within a single transaction. A transaction must not list the same output twice in its own inputs. `_verify_inputs` (`transaction_verifier.py:135`) tracks a set of `(tx_id, index)` keys and raises `ConflictingInputs` on a repeat. Note the scope: this catches a tx spending its own coin twice. The cross-transaction double-spend — two different txs spending the same coin — is a **conflict**, not an invalidity, and is left to consensus (Ch 32). `verify_conflict` (`transaction_verifier.py:389`) does reject one narrow case here: spending an output that is already spent by a confirmed (block-included, non-voided) transaction (`ConflictWithConfirmedTxError`) — because that competitor has already won, so the newcomer cannot be valid.

Reward lock. Freshly-mined block rewards cannot be spent immediately; they must "mature" for a number of blocks. `verify_reward_locked` (`transaction_verifier.py:191`) checks that any block reward this transaction spends is old enough relative to the best block height, raising `RewardLocked` otherwise. This prevents a miner from spending a reward that a reorg might later erase. (For blocks, the analogous maturity is checked via `verify_height`, `block_verifier.py:53`.)

31.11 Block-only rules

A block is not a transaction, and `BlockVerifier` (`block_verifier.py:35`) carries the rules that only make sense for blocks:

- **Reward amount** (`verify_reward`, :69): the sum of a block's outputs must equal exactly the protocol-defined issuance for its height (`get_tokens_issued_per_block`), else `InvalidBlockReward`. A miner cannot pay themselves more than the schedule allows. This is the one place HTR is legitimately created.
- **No inputs** (`verify_no_inputs`, :79): a block has no inputs; any input raises `BlockWithInputs`.
- **No custom tokens in outputs** (`verify_output_token_indexes`, :84): block reward outputs must be HTR only; `BlockWithTokensError` otherwise.

- **Data size** (`verify_data` , :89): the block's free-form data field is capped at `BLOCK_DATA_MAX_SIZE` .
- **Checkpoints** (`verify_checkpoints` , :107): a block must not fork the chain below a configured checkpoint height — `CheckpointError` . This is the hard backstop against deep reorgs (Ch 10).
- **Mandatory signalling** (`verify_mandatory_signaling` , :93): during a feature's `MUST_SIGNAL` phase, blocks are required to signal support; a silent block raises `BlockMustSignalError` (Ch 38).

The two specialized block types add one rule each:

`MergeMinedBlockVerifier.verify_aux_pow` (`merge_mined_block_verifier.py:28`) checks the Bitcoin auxiliary proof-of-work and its Merkle path⁴ , and

`PoaBlockVerifier.verify_poa` (`poa_block_verifier.py:28`) checks the authority signature and turn-based weight on proof-of-authority⁵ networks — where there is no mining, so the block is signed by an authorized producer rather than mined.

31.12 How it plugs into the lifecycle

Verification does not run itself. It is invoked by the **vertex handler** (Ch 33), the component that owns the ingestion pipeline. On every new vertex — whether it arrived from a peer, was relayed, or was pushed via the API — the handler builds a `VerificationParams` (the per-run flags: which features are active, whether to reject stale or conflicting vertices, the opcode version) and calls `validate_full` (`vertex_handler/vertex_handler.py:213`):

```
if not metadata.validation.is_fully_connected():
    try:
        self._verification_service.validate_full(vertex, params)
    except HatherError as e:
        raise InvalidNewTransaction(f'full validation failed: {str(e)}') from e
```

The shape of the contract is right there. Verification raises on any broken rule; the handler catches it and turns it into `InvalidNewTransaction` — the vertex is rejected, never stored as valid, never passed to consensus, never relayed. If `validate_full` returns without raising, the vertex is individually valid, its state is now `FULL` , and the handler proceeds to the next stage — consensus — which decides where this now-trusted vertex sits in canonical history.

`VerificationParams` (`verification_params.py:24`) is worth one sentence: it is a frozen dataclass of per-run knobs, with a `default_for_mempool` constructor (:38) that sets the strict combination used for real-time vertices (reject too-old vertices, harden token restrictions, reject conflicts with confirmed txs). Different ingestion paths — syncing old history vs. accepting a fresh mempool tx — pass different params, which is how the same verifier code enforces slightly different policies depending on context.

Recap

Rule (what is checked)	Verifier method	File:line
Orchestration / dispatch by type	<code>VerificationService.verify / verify_basic</code>	<code>verification_service.py:172 / :100</code>
Basic vs full split (public)	<code>validate_basic / validate_full</code>	<code>verification_service.py:50 / :64</code>
No-dependency checks bundle	<code>verify_without_storage</code>	<code>verification_service.py:291</code>
Proof-of-work (hash < target)	<code>VertexVerifier.verify_pow</code>	<code>vertex_verifier.py:142</code>
Target from weight	<code>BaseTransaction.get_target</code>	<code>base_transaction.py:361</code>
Minimum weight (tx / block)	<code>verify_weight</code>	<code>transaction_verifier.py:93 / block_verifier.py:60</code>
Value conservation + token authority	<code>TransactionVerifier.verify_sum</code>	<code>transaction_verifier.py:257</code>
Outputs positive, no HTR authority	<code>VertexVerifier.verify_outputs</code>	<code>vertex_verifier.py:153</code>
Input scripts (signatures)	<code>verify_inputs → verify_script → script_eval</code>	<code>transaction_verifier.py:131 / :172 / execute.py:103</code>
Script evaluation loop	<code>execute_eval / evaluate_final_stack</code>	<code>execute.py:54 / :86</code>
Opcode dispatch	<code>execute_op_code</code>	<code>opcode.py:635</code>
P2PKH lock	<code>P2PKH.create_output_script</code>	<code>p2pkh.py:70</code>
Sigops bound	<code>verify_sigops_output / verify_sigops_input</code>	<code>vertex_verifier.py:182 / transaction_verifier.py:105</code>

Rule (what is checked)	Verifier method	File:line
Parents (count, type, timestamp)	<code>VertexVerifier.verify_parents</code>	<code>vertex_verifier.py:65</code>
No double-spend within a tx	<code>TransactionVerifier._verify_inputs</code>	<code>transaction_verifier.py:135</code>
Reward lock (maturity)	<code>verify_reward_locked</code>	<code>transaction_verifier.py:191</code>
Block reward amount	<code>BlockVerifier.verify_reward</code>	<code>block_verifier.py:69</code>
Block has no inputs / no tokens	<code>verify_no_inputs</code> / <code>verify_output_token_indexes</code>	<code>block_verifier.py:79</code> / <code>:</code> <code>84</code>
Checkpoint forbids deep fork	<code>BlockVerifier.verify_checkpoints</code>	<code>block_verifier.py:107</code>
Mandatory feature signalling	<code>verify_mandatory_signaling</code>	<code>block_verifier.py:93</code>
Invoked by the vertex handler	<code>validate_full</code> call site	<code>vertex_handler.py:213</code>

Verification is the node's lie detector. Every rule in this chapter is an absolute statement about one vertex — its work was done, its money balances, its signatures hold, its structure is legal — checked without reference to any competing history. That is precisely its boundary. A vertex can pass every check here and still not make it into the ledger that the network keeps, because two perfectly-valid transactions might spend the same coin. Deciding which of two valid-but-conflicting histories wins — comparing accumulated weight, voiding the loser, handling reorgs — is the next stage. Once a vertex is individually valid, the question becomes: which history does it belong to? That is Chapter 32, consensus.

-
1. `assert_never(x)` is a typing helper that the static type-checker treats as "this line must be unreachable." If the checker can prove some case reaches it (e.g. an unhandled enum value), the build fails. At runtime it raises if ever actually hit. It is how a `match` over a fixed set of cases is made provably exhaustive. ↩
 2. **DAA** = Difficulty Adjustment Algorithm. The code that decides how hard mining should be right now, so blocks keep arriving at a steady average rate as total network hashing power rises and falls. Full treatment in Ch 32 / see Ch 9. ↩
 3. A stack is a last-in-first-out collection: you can only add (push) to the top and remove (pop) from the top. Think of a stack of plates. A stack machine evaluates a program using only one such stack as its working memory. ↩
 4. A Merkle path is a short list of hashes that proves one item belongs to a larger hashed set without revealing the whole set. Merged mining uses it to prove a Hathor block was committed inside a Bitcoin block. → Ch 37. ↩
 5. Proof-of-authority (PoA) is a consensus variant for private networks where blocks are not mined but signed by a fixed set of authorized producers taking turns. There is no proof-of-work and no mining reward. → Ch 32. ↩

Chapter 32 — Consensus: Which History Wins

What you'll learn in this chapter

- The difference between **verification** (Ch 31, "is this vertex valid on its own?") and **consensus** ("given everything else I know, which history is the real one?").
- How Hathor records a "this is not part of the real ledger" decision without deleting anything, using the `voided_by` set on a vertex's metadata.
- How two conflicting transactions are resolved: the one with the **higher accumulated weight** wins, the loser is **voided**, and that voiding **propagates** to everything downstream of the loser.
- How competing chains of blocks are resolved by **score**, and what a **reorg**¹ (reorganization) is — when the node abandons one chain of blocks for a heavier one.
- Why **block voiding behaves differently from transaction voiding** — an asymmetry that trips up most readers.
- How Hathor's alternative consensus engine, **Proof-of-Authority**² (PoA), replaces mining with a fixed set of authorized block signers — used on private networks.

This is one of the load-bearing chapters of the book. Chapter 10 gave you the vocabulary — conflict, voiding, reorg, finality — at the conceptual level. This chapter pays that vocabulary off against the real code in `hathor/consensus/`. By the end you should be able to point at the exact method that decides a conflict, trace by hand what happens when a heavier transaction arrives, and explain why the algorithm marks losers instead of erasing them.

32.1 Localization

The consensus package is small in file count and dense in logic. It sits in the domain model group of the codebase, right next to verification, because together they answer the two questions a node asks of every incoming vertex.

```

hathor-core/
└─ hathor/
    └─ verification/                ← "is this vertex valid in isolation?" (Ch 31)
    └─ consensus/                  ◀ YOU ARE HERE – "which history is canonical?"
        ├── __init__.py            ← exports ConsensusAlgorithm
        ├── consensus.py           ← ConsensusAlgorithm: the orchestrator
        ├── context.py             ← ConsensusAlgorithmContext + ReorgInfo
        ├── block_consensus.py     ← BlockConsensusAlgorithm: chain selection, score, reorg
        ├── transaction_consensus.py ← TransactionConsensusAlgorithm: conflict resolution
        ├── consensus_settings.py  ← PowSettings / PoaSettings (which engine?)
        └─ poa/                   ← the Proof-of-Authority variant
            ├── poa.py             ← signer set, weight rule, signature verification
            ├── poa_signer.py      ← PoaSigner: signs PoA blocks
            └─ poa_block_producer.py ← the PoA equivalent of a miner
    └─ vertex_handler/            ← the pipeline that runs verification THEN consensus (Ch 33)

```

Recap — verification vs. consensus (full treatment in Ch. 10 & Ch. 31).

Verification asks a local question about one vertex: are its signatures valid, is its proof-of-work sufficient, does it spend coins that exist, is it well-formed? A vertex either passes or it is rejected outright — and a rejected vertex never enters the ledger. Consensus asks a global question: given a vertex that is already valid, does accepting it change which version of history the node considers real? A vertex can be perfectly valid and still end up voided by consensus — for example, because a competing valid transaction spends the same coin and happens to carry more weight. Verification produces a yes/no; consensus produces a ranking. → full treatment of the conceptual split in Ch. 10 §10.1.

Context. Consensus is where "data arrived and checked out" becomes "the ledger now says X." It is invoked once per accepted vertex, immediately after verification, by the vertex handler (Ch 33). It never validates — it assumes the vertex is already valid — and it writes its decisions into the metadata the storage layer keeps for each vertex (Ch 25): `voided_by`, `accumulated_weight`, `score`, `first_block`. Every downstream consumer — the indexes (Ch 28), the wallet, the APIs — reads those fields to know what is "really" in the ledger.

32.2 What it does and why it exists

A full node receives vertices from many peers, in unpredictable order, with no global clock. Two honest users on opposite sides of the world can each spend the same coin at nearly the same instant; both transactions are individually valid; both reach the node. Verification cannot reject either — each one, looked at alone, is legitimate. Something

has to decide which of the two the ledger will honor, and that decision must be **the same on every node in the network**, or the network forks into disagreeing copies of reality.

That "something" is the consensus algorithm. Its job, stated precisely, is to assign every vertex one of two states:

- **executed** — this vertex is part of the canonical ledger; its outputs are spendable, its effects count.
- **voided** — this vertex is not part of the canonical ledger right now; its effects are suppressed.

Hathor encodes this with a single piece of metadata, `voided_by`, a set of vertex hashes (`transaction_metadata.py:45`). The rule is stated at the top of the orchestrator class (`consensus.py:58-82`):

```
class ConsensusAlgorithm:
    """Execute the consensus algorithm marking blocks and transactions as either executed or voided.

    The consensus algorithm uses the metadata voided_by to set whether a block or transaction is
    If voided_by is empty, then the block or transaction is executed. Otherwise, it is voided.
    """
```

So: `voided_by` empty (or `None`) → **executed**. `voided_by` non-empty → **voided**. That is the whole state model.

Recap — voiding is marking, not deleting (full treatment in Ch. 10 §10.3). When a vertex loses a conflict, the node does **not** erase it. It stays in storage, fully present, with its `voided_by` set populated to record why it lost (which hashes are causing its voidance). The reason is reversibility: consensus decisions are provisional. A transaction that loses today can win tomorrow if a heavier transaction confirming it arrives, and a chain of blocks abandoned now can be re-adopted if it later overtakes the current best chain. You can only "un-void" something you kept. → full treatment in Ch. 10.

Two engines, one interface

There is a second reason this package exists as a unit: Hathor supports **two consensus mechanisms**, and the node must be able to run either one.

- **Proof-of-Work** (the default, public mainnet/testnet): blocks are produced by mining, and the "real" history is the one backed by the most computational work. This is what almost all of `block_consensus.py` and `transaction_consensus.py` implement.
- **Proof-of-Authority** (private networks): there is no mining. A fixed, configured set of signers take turns producing blocks, each block carrying a cryptographic signature instead of a proof-of-work nonce. The `poa/` subpackage implements this.

Which engine a node uses is a settings choice, `CONSENSUS_ALGORITHM`, defaulting to `PowSettings()` (`conf/settings.py:65`). We cover PoW first, in depth, then PoA as a focused variant in §32.7.

32.3 The concepts it rests on

Before walking the code, three ideas from earlier chapters need to be fresh. Each gets a recap box, not a re-teaching.

Recap — accumulated weight and the heaviest-DAG rule (full treatment in Ch. 9).

Every vertex carries a **weight**³: a number measuring how much proof-of-work it represents ($\text{weight} = \log_2$ of the expected number of hash attempts). The **accumulated weight** of a vertex is the total work of the vertex plus all the work that has piled up behind it — every later vertex that confirms it adds its own weight to the pile. Hathor's consensus rule is therefore the heaviest rule, not the longest rule: when two histories compete, the one with more accumulated work wins, because reproducing that work is what an attacker would have to redo to rewrite it. The metadata fields are `accumulated_weight` (`transaction_metadata.py:48`) and, for blocks, `score` (`:49`). → full treatment in Ch. 9.

Recap — the vertex graph and its two edge types (full treatment in Ch. 8 & Ch. 25).

A vertex is any node of Hathor's ledger graph — a `Block` or a `Transaction`. Vertices are linked by **two distinct kinds of edge**: `parents` (the confirmation/verification DAG — "I confirm these vertices") and `inputs` (the funds DAG — "I spend this output"). Consensus walks **both** graphs: voiding has to flow to everything that confirms a voided vertex and everything that spends a voided vertex's outputs. → full treatment in Ch. 8 (the model) and Ch. 25 (the code: `parents`, `TxInput`, `TxOutput`).

Recap — graph traversal: BFS over the DAG (full treatment in Ch. 8 & Ch. 27).

To propagate a decision to "everything downstream," consensus repeatedly performs a **breadth-first walk**⁴ over the DAG, starting from a vertex and visiting its descendants in timestamp order. The codebase's tool for this is `BFSTimestampWalk` (`hathor/transaction/storage/traversal.py`), which can be told to follow the funds DAG, the verification DAG, or both, and to walk left-to-right (toward descendants) or right-to-left (toward ancestors). You will see it used in almost every propagation method below. → full treatment of DAG traversal in Ch. 27.

With those in hand, the code becomes readable.

32.4 The code, walked: the orchestrator

The public entry point is one method: `ConsensusAlgorithm.unsafe_update` (`consensus.py:132`). The vertex handler calls it exactly once per accepted vertex.

```
def unsafe_update(self, base: BaseTransaction) -> list[ConsensusEvent]:
    assert self.tx_storage.is_only_valid_allowed()
    meta = base.get_metadata()
    assert meta.validation.is_valid()          # consensus assumes verification already passed
    ...
    context = self.create_context()
    ...
    if isinstance(base, Transaction):
        context.transaction_algorithm.update_consensus(base)
    elif isinstance(base, Block):
        context.block_algorithm.update_consensus(base)
```

Three things to read off this:

1. **The "unsafe" in the name is a contract, not a warning about bugs.** The docstring (`consensus.py:133-138`) says the caller is responsible for crashing the full node if this method throws. Consensus mutates a lot of metadata across many vertices; a half-applied update would leave the ledger inconsistent. So if anything goes wrong mid-update, the node must not limp on — it must crash and restart from the last consistent on-disk state. The vertex handler honors this: on any exception it marks the vertex with `CONSENSUS_FAIL_ID` and calls `crash_and_exit` (`vertex_handler.py:178-183`).
2. **The block/transaction split is dispatched right here.** A `Block` and a `Transaction` need genuinely different consensus logic — blocks compete as chains ranked by score; transactions compete as conflicts ranked by accumulated weight. So there are two algorithm objects, `block_algorithm` and `transaction_algorithm`, and `unsafe_update` routes to the right one by type. Everything in §32.5 is the transaction side; §32.6 is the block side.
3. **A fresh context object is created per call.** `create_context()` (`consensus.py:127`) returns a `ConsensusAlgorithmContext` that lives only for the duration of this one update.

The context object

`ConsensusAlgorithmContext` (`context.py:42`) is the scratchpad for a single consensus run. It holds the two algorithm instances, the running set of affected transactions, and the optional reorg record:

```

class ConsensusAlgorithmContext:
    consensus: 'ConsensusAlgorithm'
    block_algorithm: 'BlockConsensusAlgorithm'
    transaction_algorithm: 'TransactionConsensusAlgorithm'
    txs_affected: set[BaseTransaction]
    reorg_info: ReorgInfo | None
    ...
    def save(self, tx: BaseTransaction) -> None:
        """Only metadata is ever saved in a consensus update."""
        assert tx.storage is not None
        self.txs_affected.add(tx)
        tx.storage.save_transaction(tx, only_metadata=True)

```

Two design points worth pausing on:

- **save() only ever writes metadata** (`context.py:72-76`). Consensus never changes a vertex's contents — the inputs, outputs, signatures are immutable facts. It only changes the node's opinion about the vertex (its `voided_by`, `score`, `first_block`). Keeping that invariant in one method makes it impossible to accidentally rewrite history while reasoning about it.
- **txs_affected accumulates every vertex touched** so the orchestrator can, after the core logic finishes, update the indexes and publish events for exactly those vertices (`consensus.py:166-167`, `222-227`). Consensus does the thinking; the orchestrator does the bookkeeping afterward.

`ReorgInfo` (`context.py:35-39`) is a small frozen record naming the three blocks that define a reorg — the `common_block` where the chains diverged, the `old_best_block`, and the `new_best_block`. It is set at most once per run via `mark_as_reorg` (`context.py:78-81`), which asserts it was not already set.

32.5 Transaction consensus: resolving conflicts

This is the heart of the chapter. The driver is

`TransactionConsensusAlgorithm.update_consensus` (`transaction_consensus.py:49`):

```

def update_consensus(self, tx: Transaction) -> None:
    self.mark_inputs_as_used(tx)
    self.update_voided_info(tx)
    self.set_conflict_twins(tx)
    self.execute_nano_contracts(tx)

```

We will take the first three in order. (`execute_nano_contracts` is a no-op for plain transactions — `:55-61` — because nano-contract execution happens when a block confirms the transaction, not when the transaction itself arrives. That belongs to Ch 39.)

Step 1 — detecting the conflict

When a transaction spends an output, consensus records that the output is now spent by this transaction. If the output was **already** spent by some other transaction, the two transactions are in conflict — a **double-spend**⁵.

`mark_input_as_used` (`transaction_consensus.py:69`) does the detection:

```
def mark_input_as_used(self, tx: Transaction, txin: TxInput) -> None:
    spent_tx = tx.storage.get_transaction(txin.tx_id)
    spent_meta = spent_tx.get_metadata()
    spent_by = spent_meta.spent_outputs[txin.index]      # who else spends this output?
    assert tx.hash not in spent_by

    meta = tx.get_metadata()
    if spent_by:                                       # someone already does → conflict!
        # We initially void ourselves. This conflict will be resolved later.
        if not meta.voided_by:
            meta.voided_by = {tx.hash}
        else:
            meta.voided_by.add(tx.hash)
        if meta.conflict_with:
            meta.conflict_with.extend(set(spent_by) - set(meta.conflict_with))
        else:
            meta.conflict_with = spent_by.copy()
    ...
    spent_by.append(tx.hash)                          # record ourselves as a spender
```

Read the key move carefully: **a newly-arrived transaction that conflicts voids itself first** (`meta.voided_by = {tx.hash}`). It does not assume it is the winner. It also records the hashes it conflicts with in `conflict_with` (`transaction_metadata.py:44`), and tells each conflicting transaction about itself in turn (`:93-103`). The actual winner is decided later, in `check_conflicts`. Voiding-yourself-then-competing keeps the algorithm symmetric: whether you arrived first or second, the same comparison decides the outcome.

Recap — within-tx vs. cross-tx double spends (full treatment in Ch. 31 §double-spend). Two outputs spent by the same transaction at once is a structural error — verification rejects it as **invalid**, and it never reaches consensus. Two different transactions spending the same output is a **conflict** — both are valid in isolation, both reach consensus, and this method is where the node notices. Invalid is verification's job; conflict is consensus's job. → full treatment in Ch. 31.

Step 2 — computing voided_by from the surroundings

`update_voided_info` (`transaction_consensus.py:177`) is the longest method on the transaction side. Its job: compute this transaction's `voided_by` from its context, then trigger conflict resolution. It works in layers.

First it **inherits voiding from parents and inputs**. If any vertex this transaction confirms (a parent) or spends from (an input) is voided, this transaction must inherit that voidance — you cannot be part of the canonical ledger if you build on something that is not:

```
voided_by: set[bytes] = set()

# Union of voided_by of parents
for parent in tx.get_parents():
    parent_meta = parent.get_metadata()
    if parent_meta.voided_by:
        voided_by.update(
            self.context.consensus.filter_out_voided_by_entries_from_parents(parent, parent_meta.
        )
...
# Union of voided_by of inputs
for txin in tx.inputs:
    spent_tx = tx.storage.get_transaction(txin.tx_id)
    spent_meta = spent_tx.get_metadata()
    if spent_meta.voided_by:
        voided_by.update(spent_meta.voided_by)
```

This is the **propagation invariant** stated in the orchestrator docstring (`consensus.py:71-73`): the `voided_by` of a vertex is always a subset of the `voided_by` of everything that confirms it or spends from it. Voidance flows strictly downstream.

Then it **adds its own hash if it is losing a conflict**. A transaction's own hash appears in its `voided_by` only when it has a conflict and is not (yet) the winner (`:224-225`):

```
if meta.conflict_with:
    voided_by.add(tx.hash)
```

Finally, having settled its own `voided_by`, it **kicks off conflict resolution** for the transactions involved (`:259-262`):

```
meta = tx.get_metadata()
if meta.voided_by == {tx.hash}:          # voided ONLY by itself → it is a live conflict candidate
    self.check_conflicts(tx)
```

The guard `voided_by == {tx.hash}` is precise: a transaction is a candidate to win a conflict only if the sole reason it is voided is its own conflict — not because a parent or input is also voided. If it is voided for some other reason too, it cannot win regardless, so there is nothing to resolve.

Step 3 — picking the winner: `check_conflicts`

`check_conflicts` (`transaction_consensus.py:302`) is the method Chapter 10 promised. It implements the **heavier-wins** rule:


```

def check_conflicts(self, tx: Transaction) -> None:
    """Check which transaction is the winner of a conflict, the remaining are voided."""
    meta = tx.get_metadata()
    if meta.voided_by != {tx.hash}:
        return

    # gather conflicting transactions that are still candidates
    candidates: list[Transaction] = []
    conflict_list: list[Transaction] = []
    for h in meta.conflict_with or []:
        conflict_tx = cast(Transaction, tx.storage.get_transaction(h))
        conflict_list.append(conflict_tx)
        conflict_tx_meta = conflict_tx.get_metadata()
        if not conflict_tx_meta.voided_by or conflict_tx_meta.voided_by == {conflict_tx.hash}:
            candidates.append(conflict_tx)

```

It then checks whether `tx` has the highest accumulated weight among the candidates, in two passes — first against already-voided candidates (:327-335), then against executed ones (:337-351):

```

# Compare against executed candidates
tie_list = []
for candidate in candidates:
    tx_meta = candidate.get_metadata()
    if not tx_meta.voided_by:
        candidate.update_accumulated_weight(stop_value=meta.accumulated_weight)
        tx_meta = candidate.get_metadata()
        d = tx_meta.accumulated_weight - meta.accumulated_weight
        if d == 0:
            tie_list.append(candidate)
        elif d > 0:
            is_highest = False
            break
if not is_highest:
    return

# We won or tied: void every conflicting tx...
for conflict_tx in sorted(conflict_list, key=lambda x: x.timestamp, reverse=True):
    self.mark_as_voided(conflict_tx)

if not tie_list:
    # ...and if it wasn't a tie, declare ourselves the winner.
    self.mark_as_winner(tx)

```

The decision rule, plainly:

- **Strictly heavier than every rival → win.** `mark_as_winner` removes the transaction's own hash from `voided_by`, un-voiding it (:362-372).
- **Strictly lighter than some rival → lose.** Stay voided; do nothing more.
- **Exact tie on accumulated weight → both stay voided.** Note the subtlety: in a tie, the candidate is added to `tie_list`, the rivals are voided, but `mark_as_winner` is not called — so nobody wins. A tie resolves to "all conflicting transactions remain voided" until some later transaction breaks the tie by piling more weight behind one of them. The system refuses to guess.

The invariant the algorithm maintains is asserted in `assert_valid_consensus` (: 291-300): **two transactions that conflict can never both be executed**. That is the whole point — at most one spender of an output is ever live.

On the tie-break by timestamp. The grounding notes (and Ch 10) mention a timestamp tie-break. Read the code precisely: timestamp is used to order the voiding of the conflict list — `sorted(conflict_list, key=lambda x: x.timestamp, reverse=True)` at :355 — not to crown a winner. An exact accumulated-weight tie does **not** elect the older transaction; it leaves both voided. Timestamp only determines the order in which the losers are processed. (Block consensus does use a hash tie-break to pick a chain — §32.6 — but transaction consensus deliberately does not pick a winner on a weight tie.)

Step 4 — propagating the void: the BFS

When a transaction loses, its voidance must reach **everything downstream of it** — every transaction that confirms it and every transaction that spends one of its outputs. That is what `add_voided_by` does (`transaction_consensus.py:462`), via a breadth-first walk over both DAGs:

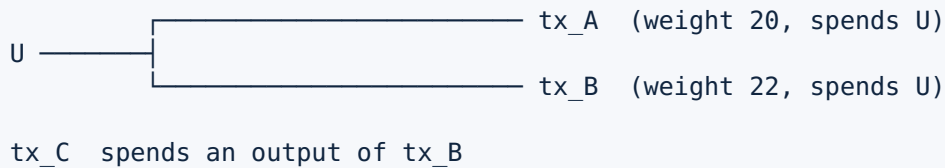
```
def add_voided_by(self, tx: Transaction, voided_hash: bytes, *, is_dag_verifications: bool = True):
    """
    bfs = BFSTimestampWalk(tx.storage, is_dag_funds=True, is_dag_verifications=is_dag_verifications,
                            is_left_to_right=True)
    check_list: list[Transaction] = []
    for tx2 in bfs.run(tx, skip_root=False):
        meta2 = tx2.get_metadata()
        if tx2.is_block:
            assert isinstance(tx2, Block)
            self.context.block_algorithm.mark_as_voided(tx2)
        ...
        if meta2.voided_by:
            meta2.voided_by.add(voided_hash)
        else:
            meta2.voided_by = {voided_hash}
        ...
        self.context.save(tx2)
        bfs.add_neighbors()
```

The walk starts at the loser and visits every descendant in timestamp order (`is_left_to_right=True` — toward newer vertices), stamping `voided_hash` into each one's `voided_by`. The mirror-image method `remove_voided_by` (: 374) does the reverse walk when a transaction is un-voided (e.g. because it just won a conflict), peeling the hash back out of every descendant and re-checking their conflicts in case un-voiding revives a downstream winner.

This BFS is the mechanism behind the docstring's promise: if tx1 is voided and tx2 verifies tx1, then tx2 must be voided as well (`consensus.py:67-69`).

A hand-traced example

Take a concrete double-spend. Alice has one unspent output, `U`, worth 10 HTR.



Two transactions, `tx_A` and `tx_B`, both spend `U`. Both are individually valid (Alice signed both). `tx_C` later spends an output of `tx_B`.

Trace what consensus does, assuming `tx_A` arrives first, then `tx_B`, then `tx_C`:

1. **`tx_A` arrives.** `mark_input_as_used` sees `U` has no spender yet, so no conflict. `tx_A` is executed (`voided_by` stays empty). `U` now records `spent_by = [tx_A]`.
2. **`tx_B` arrives.** `mark_input_as_used` sees `U.spent_by = [tx_A]` — conflict. `tx_B` voids itself (`voided_by = {tx_B}`) and records `conflict_with = [tx_A]`; `tx_A` learns `conflict_with = [tx_B]` too. Then `update_voided_info` calls `check_conflicts(tx_B)`. `tx_B`'s accumulated weight (22) is compared against the executed candidate `tx_A` (20). `tx_B` is strictly heavier, so: `mark_as_voided(tx_A)` voids `tx_A`, and `mark_as_winner(tx_B)` un-voids `tx_B`.
The ledger flips: `tx_B` executed, `tx_A` voided.
3. **`tx_C` arrives** spending `tx_B`'s output. Since `tx_B` is executed (not voided), `tx_C` inherits no voidance and is itself executed.

Now imagine the reverse order — `tx_B` first, then `tx_A`. `tx_B` executes. `tx_A` arrives, conflicts, voids itself, and `check_conflicts(tx_A)` finds `tx_A` (20) is lighter than the executed `tx_B` (22). `tx_A` stays voided; nothing flips. **Same final state, regardless of arrival order** — which is exactly the determinism the network needs. Every node, however it received the two transactions, ends with `tx_B` and `tx_C` executed and `tx_A` voided.

And had `tx_A` already been confirmed and `tx_C`-like descendants built on it before `tx_B` arrived? The `add_voided_by` BFS would walk from `tx_A` forward, voiding every descendant that hangs off it. That cascade is why voiding has to be a graph walk and not a single flag flip.

32.6 Block consensus: chains, score, and reorgs

Blocks play a different game. Where transactions compete pairwise over a shared input, blocks compete as **chains** — and the winning chain is the one with the highest **score**.

Score: accumulated work of a whole sub-DAG

The block-side weight metric is `score`, computed by `calculate_score` (`block_consensus.py:577`) and its recursive helper `_score_block_dfs` (`:512`). The docstring (`:120-123`) defines it:

The score of a block is calculated as the sum of the weights of all transactions and blocks both directly and indirectly verified by the block.

So a block's score is the total work of everything behind it in the DAG — every ancestor block, and every transaction those blocks confirm. The score of a chain is the score of its head block. Crucially, score is **monotonic**: each new block on top adds its own work plus the work of the transactions it newly confirms, so a longer-standing, more-confirmed chain naturally accrues a higher score. The code even relies on score being **immutable once computed** for a given block — the sub-DAG behind a block never changes, so the score can be cached and only re-checked for consistency (`:562-573`).

`score` is stored in metadata (`transaction_metadata.py:49`), distinct from a transaction's `accumulated_weight`.

Selecting the best chain

`BlockConsensusAlgorithm.update_voided_info` (`block_consensus.py:109`) is the block-side counterpart to the transaction method of the same name. Its enormous docstring (`:109-159`) enumerates eight cases for how an arriving block can attach to the existing block tree — to the head or tail of the best chain or a side chain, with single or multiple best chains. The common ones collapse to a simple comparison.

When the new block attaches to the head of the current best chain (the usual case), it extends the winner and there is nothing to decide (`:196-203`). When it attaches anywhere else — building on a side chain⁶ — the node must ask whether that side chain has now overtaken the best chain:

```

# First, void this block (otherwise it would always look like a head).
self.mark_as_voided(block, skip_remove_first_block_markers=True)

head = storage.get_best_block()
head_meta = head.get_metadata(force_reload=True)
best_score = head_meta.score

score = self.calculate_score(block)

winner = False
if score > best_score:
    winner = True
elif score == best_score:
    # Use block hashes as a tie breaker.
    if block.hash < head.hash:
        winner = True

```

Read off the **score comparison** (`block_consensus.py:227-232`):

- **New chain's score strictly greater than the current best → the new chain wins.**
- **Exact tie → break by block hash** (the lexicographically smaller hash wins). Unlike transaction conflicts, block ties are broken deterministically, because the chain has to make progress — leaving two competing heads live indefinitely would stall block production. The hash is an arbitrary-but-deterministic tiebreak that every node computes identically.
- **Less → the new block stays voided** on its side chain.

When the new block wins, the algorithm voids the old head's chain down to the fork point and promotes the new chain (`:244-263`):

```

# Winner, winner, chicken dinner!
common_block = self._find_first_parent_in_best_chain(block)
self.add_voided_by_to_multiple_chains([head], common_block)
self.update_score_and_mark_as_the_best_chain_if_possible(block)
...
if common_block != head:
    self.mark_as_reorg_if_needed(common_block, block)
storage.indexes.height.update_new_chain(height, block)

```

Reorgs

A **reorg** is what just happened in that last block: the node switched its best chain from one branch to another, abandoning blocks it previously considered canonical.

`mark_as_reorg_if_needed` (`block_consensus.py:265`) records it:

```
def mark_as_reorg_if_needed(self, common_block: Block, new_best_block: Block) -> None:
    """Mark as reorg only if reorg size > 0."""
    _, old_best_block_hash = storage.indexes.height.get_height_tip()
    old_best_block = storage.get_transaction(old_best_block_hash)
    reorg_size = old_best_block.get_height() - common_block.get_height()
    if reorg_size == 0:
        assert old_best_block.hash == common_block.hash
        return
    self.context.mark_as_reorg(ReorgInfo(
        common_block=common_block,
        old_best_block=old_best_block,
        new_best_block=new_best_block,
    ))
```

The **reorg size** is how many blocks deep the old chain is abandoned — the height difference between the old best block and the common ancestor (`common_block`) where the two chains diverged. A reorg of size 0 (the chains share their tip) is not a reorg at all and is skipped. When a real reorg is recorded, the orchestrator later emits `REORG_STARTED` and `REORG_FINISHED` events (`consensus.py:210-220`, `250-251`) so downstream systems can react, and it re-checks the mempool: transactions that were valid on the old chain but became invalid on the new one (for example, a reward that is no longer unlocked at the new height) are found and removed by `_compute_vertices_that_became_invalid` (`consensus.py:357`).

Recap — finality is probabilistic (full treatment in Ch. 10 §finality). Because a heavier competing chain can always, in principle, arrive and trigger a reorg, no block is ever absolutely final — only increasingly improbable to be reversed as more work piles on top of it. This is why wallets wait for a number of confirmations before treating a payment as settled. The hard floor on reorgs is the **checkpoint**⁷ mechanism (verified during verification, Ch 31): the node refuses any reorg that would rewrite history before a hard-coded checkpoint height. → full treatment in Ch. 10.

The asymmetry: why block voiding differs from transaction voiding

This is the single most misunderstood part of the package, and the orchestrator docstring calls it out explicitly (`consensus.py:75-81`):

```
# When a block is not in the bestchain, its voided_by contains its hash. This hash is also propagated
# through the transactions that spend one of its outputs.
#
# Differently from transactions, the hash of the blocks are not propagated through the voided_by
# other blocks. For example, if b0 <- b1 <- b2 <- b3 is a side chain, i.e., not the best blockchain,
# then b0's voided_by contains b0's hash, b1's voided_by contains b1's hash, and so on. The hash
# b0 will not be propagated to the voided_by of b1, b2, and b3.
```

Spell out the contrast:

- **Transaction voiding propagates the loser's hash downstream.** If `tx1` loses, every descendant gets `tx1`'s hash stamped into its `voided_by`. A voided transaction's hash spreads through the graph.

- **Block voiding does not propagate a block's hash to later blocks.** Each block on a side chain carries only its own hash in `voided_by`. `b1` does not inherit `b0`'s hash.

Why the difference? Because the two kinds of vertex answer different questions. A transaction's `voided_by` records the full set of reasons it is not canonical, and those reasons must travel downstream so any descendant can see exactly what is wrong. A block on a side chain is voided for one self-contained reason — it is on a side chain — and that reason is fully expressed by the block's own hash being present. Each side-chain block is independently "not on the best chain"; there is no useful information in copying `b0`'s hash into `b1`, because `b1` already says "I am off-chain" via its own hash. What does propagate from a voided block is the voidance of any **transaction** that spends one of its outputs — `add_voided_by` (`block_consensus.py:414`) walks the block's spent outputs and voids the spending transactions:

```
spent_by: Iterable[bytes] = chain(*meta.spent_outputs.values())
for tx_hash in spent_by:
    tx = storage.get_transaction(tx_hash)
    self.context.transaction_algorithm.add_voided_by(tx, voided_hash)
```

So when a chain is abandoned, the blocks themselves are quietly marked off-chain, but any transaction that depended on those blocks' rewards or confirmations gets the full downstream voiding treatment from §32.5. The block tree is a tree; the transaction graph is a graph; their voiding rules differ because their topologies differ.

first_block: where a transaction sits in block-time

One more block-side concept connects the two halves. When a block is marked as the best chain, `_score_block_dfs` sets each newly-confirmed transaction's `first_block` field (`block_consensus.py:554-557`) to that block's hash. `first_block` (`transaction_metadata.py:50`) records which block first confirmed this transaction on the best chain — effectively, the transaction's position in block-ordered time. During a reorg, `remove_first_block_markers` (`block_consensus.py:475`) clears these for the abandoned chain, because those transactions are no longer confirmed by any best-chain block until they are re-confirmed by the new one. This is how a transaction knows whether it is still in the mempool (`first_block is None`) or settled into a block.

32.7 The other engine: Proof-of-Authority

Everything above assumes **Proof-of-Work** — blocks earn their place by burning computation, and the heaviest chain wins. Hather's `poa/` subpackage replaces that with **Proof-of-Authority**, used on private networks where you do not want (or cannot afford) real mining.

Recap — Proof-of-Authority (full treatment: this section is the canonical one).

In PoA there is no mining and no open competition to produce blocks. Instead a **fixed, configured set of signers** — identified by their public keys — are the only parties permitted to produce blocks, and they take turns. A block is "earned" not by proof-of-work but by a valid **digital signature** from an authorized signer. It is the consensus model of a permissioned network: you trust a known set of authorities rather than anonymous hash power.

The configuration lives in `consensus_settings.py`. A network is either PoW or PoA, chosen by a discriminated union (`ConsensusSettings`, :155-161) on the `type` field. `PoaSettings` (:116) carries the signer list:

```
class PoaSettings(_BaseConsensusSettings):
    type: Literal[ConsensusType.PROOF_OF_AUTHORITY] = ConsensusType.PROOF_OF_AUTHORITY
    # A list of Proof-of-Authority signer public keys that have permission to produce blocks.
    signers: tuple[PoaSignerSettings, ...]
```

Each `PoaSignerSettings` (:99) is a public key with an optional `start_height / end_height` window, so the authorized set can change over time (a signer can be added or retired at a chosen height).

Turns and weight

The clever part is how PoA reuses the same heaviest-chain machinery without any mining. Blocks still have a **weight**, and the block consensus from §32.6 still picks the highest-score chain — but in PoA the weight is assigned by a turn schedule, not by hashing.

`get_signer_index_distance` (`poa.py:58`) computes how far a signer is from being "in turn" for a given height — the expected signer for height `h` is `h % number_of_active_signers` (:62), rotating through the set. `calculate_weight` (`poa.py:69`) then assigns block weight from that distance:

```
BLOCK_WEIGHT_IN_TURN = 2.0
BLOCK_WEIGHT_OUT_OF_TURN = 1.0

def calculate_weight(settings: PoaSettings, block: PoaBlock, signer_index: int) -> float:
    index_distance = get_signer_index_distance(settings=settings, signer_index=signer_index, height=block.height)
    return BLOCK_WEIGHT_IN_TURN if index_distance == 0 else BLOCK_WEIGHT_OUT_OF_TURN / index_distance
```

The signer whose turn it is produces a weight-2.0 block; an out-of-turn signer produces a lighter block (weight `1.0 / distance`). Because the in-turn block is heavier, the chain built from in-turn blocks accumulates more score, and the existing heaviest-chain rule naturally prefers it. Out-of-turn production is allowed (so the chain does not stall if the scheduled signer is offline) but is "punished" by lower weight, so it only wins if the in-turn signer is absent. PoA gets its ordering for free from the PoW consensus code — it just feeds it a different weight.

Signatures instead of nonces

A PoA block (`PoaBlock`, a `Block` subclass — see Ch 25) carries a signer ID and a signature instead of a proof-of-work nonce. `PoaSigner.sign_block` (`poa_signer.py:99`) signs the block's data with the signer's private key:

```
def sign_block(self, block: PoaBlock) -> None:
    hashed_poa_data = poa.get_hashed_poa_data(block)
    signature = self._private_key.sign(hashed_poa_data, ec.ECDSA(hashes.SHA256()))
    block.signer_id = self._signer_id
    block.signature = signature
```

`verify_poa_signature` (`poa.py:86`) checks, at verification time (Ch 31), that the signature came from a currently-active signer. The `signer_id` is the first two bytes of the hash of the signer's public key (`poa_signer.py:110-114`); it is a non-unique hint that lets the verifier skip most candidates before doing the expensive signature check (`poa_signer.py:82-86`).

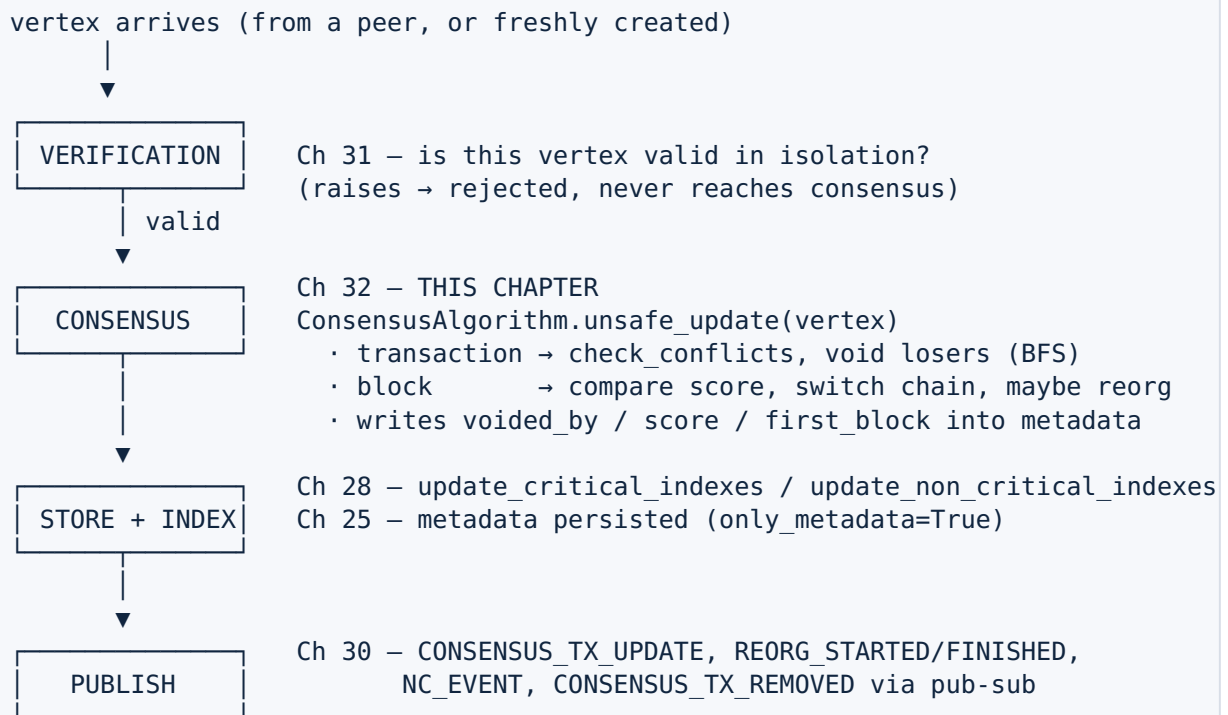
Producing blocks

The mining-equivalent is `PoaBlockProducer` (`poa_block_producer.py:43`) — the PoA node's block factory. It watches for new best blocks, works out whether it is this node's turn (`_get_signer_index`, `:95`), schedules a block at the expected timestamp (`_schedule_block`, `:137`), and produces and signs it (`_produce_block`, `:172`), setting the weight via `poa.calculate_weight` (`:184`). It is the structural analogue of the CPU miner in the PoW world (Ch 37), minus the hashing — there is no puzzle to solve, only a turn to wait for and a signature to apply.

Scope note. This section covers the consensus role of PoA — how it slots into the heaviest-chain machinery and how blocks are authorized. The deeper mechanics of block production timing, signer rotation edge cases, and how PoA interacts with the P2P layer's peer-hello hash (`consensus_settings.py:147-152`) are touched only lightly here; the production side connects forward to mining (Ch 37) and the settings side back to Ch 22.

32.8 How it plugs into the lifecycle

Consensus is not a thing the node runs on a timer. It runs exactly once per accepted vertex, sandwiched into the ingestion pipeline:



Concretely, the **vertex handler** (Ch 33) calls `self._consensus.unsafe_update(vertex)` at `vertex_handler.py:232`, immediately after a successful `validate_full` (:213). The handler treats consensus as fallible-and-fatal: any exception from `unsafe_update` triggers `crash_and_exit` (:178-183), upholding the "unsafe" contract from §32.4.

The outputs of a consensus run land in three places, all driven by the orchestrator's tail (`consensus.py:222-253`):

- **Metadata** (Ch 25): the `voided_by`, `score`, `accumulated_weight`, and `first_block` fields are persisted via `context.save(...)` (metadata only).
- **Indexes** (Ch 28): for every affected transaction, `update_critical_indexes` and `update_non_critical_indexes` are called so lookups (UTXO, addresses, mempool tips, height) reflect the new canonical view — voided vertices are pulled out of the indexes, un-voided ones put back.
- **Events** (Ch 30): a `list[ConsensusEvent]` is returned and later published through pub-sub — `CONSENSUS_TX_UPDATE` per affected tx, plus `REORG_STARTED` / `REORG_FINISHED`, `CONSENSUS_TX_REMOVED`, and nano-contract execution events when a block confirms contracts.

That is the full circuit: verification gates, consensus decides, storage and indexes record, pub-sub announces.

Recap

Question	Mechanism	Key code
Is a vertex part of the ledger?	<code>voided_by</code> empty → executed; non-empty → voided	<code>transaction_metadata.py:45</code>
Who decides?	<code>ConsensusAlgorithm.unsafe_update</code> , dispatched block vs tx	<code>consensus.py:132,157-160</code>
Which conflicting transaction wins?	Highest <code>accumulated_weight</code> ; exact tie → both stay voided	<code>transaction_consensus.py:302,327-360</code>
How does a loss spread?	BFS over funds + verification DAGs, stamping <code>voided_by</code>	<code>transaction_consensus.py:462</code>
Which chain of blocks is canonical?	Highest <code>score</code> ; tie → smaller block hash	<code>block_consensus.py:227-232</code>
What is a reorg?	Best chain switches; size = height drop to common ancestor	<code>block_consensus.py:265-282</code>
Why don't block hashes propagate to later blocks?	Side-chain blocks each carry only their own hash	<code>consensus.py:75-81</code>
What is the alternative engine?	Proof-of-Authority: signer turns set block weight	<code>poa/poa.py:58-72</code> , <code>consensus_settings.py:116</code>
When does consensus run?	Once per vertex, after verification, by the vertex handler	<code>vertex_handler.py:232</code>

Consensus is the node's answer to the one question verification cannot answer: given two valid histories, which one is real? Hathor answers it with weight — the heaviest transaction wins a conflict, the heaviest chain wins the blockchain — and records the answer as a reversible mark (`voided_by`) rather than a deletion, so the answer can change as more weight arrives. The block side and the transaction side share a metric (work) but differ in topology and in how voidance spreads, and the whole apparatus can be swapped for Proof-of-Authority on a private network by feeding the same heaviest-chain machinery a turn-based weight instead of a mined one.

The next chapter, **Ch 33 (the vertex handler)**, is the thin but pivotal pipeline that calls verification (Ch 31) and then consensus (Ch 32) in sequence and then commits the result — the place where "a vertex arrived" finally becomes "the ledger changed."

1. **Reorg** (reorganization) — when a node abandons part of its current best chain of blocks in favor of a competing chain that has accumulated more work (a higher score). Blocks on the abandoned branch become voided; their transactions return to the mempool or are re-confirmed by the new chain. The reorg size is how many blocks deep the switch goes. ↩
2. **Proof-of-Authority** (PoA) — a consensus model for permissioned networks in which a fixed, configured set of authorized signers (identified by public key) take turns producing blocks, each block carrying a valid signature instead of a proof-of-work nonce. No mining; trust is placed in a known set of authorities. ↩
3. **Weight** — a number measuring how much proof-of-work a vertex represents ($\text{weight} = \log_2$ of the expected number of hash attempts). **Accumulated weight** sums a vertex's weight with all the work piled up behind it in the DAG. Consensus prefers the history with the most accumulated weight. Full treatment in Ch. 9. ↩
4. **BFS** (breadth-first search) — a graph-traversal strategy that visits a starting node, then all of its immediate neighbors, then their neighbors, and so on, in expanding "rings." Hathor's `BFSimestampWalk` visits descendants in timestamp order and is the engine behind voiding propagation. ↩
5. **Double-spend** — two different transactions that both try to spend the same output. Each is valid alone; together they conflict, and consensus must void at least one so a coin is never spent twice. The visible "collision" is exactly what the spent-once rule and `voided_by` machinery exist to catch. ↩
6. **Side chain** — a branch of blocks whose score is lower than the current best chain's. Its head (and the blocks below it that are off the best chain) are voided. A side chain becomes the best chain — triggering a reorg — only if it later overtakes the current best chain's score. ↩
7. **Checkpoint** — a hard-coded `(height, hash)` pair in the node's settings marking a block the network agrees is final. The node refuses any reorg that would rewrite history before a checkpoint, putting a hard floor under otherwise-probabilistic finality. Full treatment in Ch. 10. ↩

Chapter 33 — Ingestion: The Vertex Handler

What you'll learn in this chapter

- What the **vertex handler** is, and why the node funnels every incoming block and transaction through this one place.
- The exact **ingestion sequence** — verify → save → consensus → re-validate → index → announce — and why that order is not negotiable.
- How the handler decides **accept** vs **reject** vs **already-known**, and what happens to a vertex whose dependencies have not all arrived yet.
- How a single failure mid-pipeline turns into a **deliberate node crash**, and why crashing is the safe choice here.
- Where this pipeline sits in the node's life: who calls it (the manager and the sync agent) and which chapters own the deep detail of each step.

This is a short chapter about a small package — two files, one of them a four-line `__init__.py`. But its size is misleading. The vertex handler is the **convergence point** of the whole node. Everything you have read about in the last several chapters — the vertex model (Ch. 25), storage (Ch. 27), indexes (Ch. 28), pub-sub (Ch. 30), verification (Ch. 31), consensus (Ch. 32) — meets here, in one method, in one fixed order. This is the box labelled "the component that runs this pipeline end-to-end" in the life-of-a-node story (Ch. 0, §0.3, Act II). We promised then that a later chapter would pay it off. This is that chapter.

33.1 Localization

The package is one of the smallest in the codebase:

```
hathor-core/
├─ hathor/
│   └─ vertex_handler/
│       ├── __init__.py
│       └─ vertex_handler.py
```

◀ YOU ARE HERE
← re-exports VertexHandler (4 lines of code)
← the whole package: class VertexHandler

`__init__.py` does nothing but lift the class up one level so the rest of the node can write `from hathor.vertex_handler import VertexHandler` instead of reaching into the inner module (`__init__.py:15`). All the substance is the single class `VertexHandler` in `vertex_handler.py:45`.

For context: the handler is built once during the builder phase (Ch. 24) and handed to the `HathorManager` (Ch. 29) as `self.vertex_handler`. It is a long-lived collaborator, created at boot and reused for the lifetime of the node.

Where it sits in the dependency graph. The vertex handler is downstream of the data model and storage, and upstream of nobody — nothing depends on it except its callers. It is glue, not foundation. It imports the verification service, the consensus algorithm, the transaction storage, the indexes, and the pub-sub bus, and it calls them in sequence. It defines no new data structures of its own. That is the shape of an **orchestrator**: a piece of code whose only job is to call other pieces in the right order.

33.2 What it does, and why it exists

The single chokepoint

Picture the two ways a new vertex¹ can appear at a running node:

1. **It was created locally.** A wallet built a transaction and submitted it; or a miner solved a block and sent it in. The manager receives it.
2. **It arrived from a peer.** The sync agent (Ch. 35) downloaded it as part of catching up, or another node relayed a brand-new transaction across the network in real time.

These are different sources, but the question they pose is identical: "Here is a candidate block or transaction. Should it become part of my ledger, and if so, what changes?" The answer requires the same work every time — check it is valid, decide what it does to canonical history, write it down, update the lookup tables, tell everyone. There is no reason to write that work twice, and several reasons not to.

So the node does it **once, in one place**: the vertex handler. Every path that wants to add a vertex to the ledger routes through this class. This is a deliberate design choice with a name — a chokepoint². Concentrating the logic here means there is exactly one definition of "what it means to ingest a vertex," exactly one place where the order of operations is decided, and exactly one place to look when something goes wrong. If verification, consensus, and storage updates were scattered across the manager, the sync agent, and the mining code, those three would inevitably drift out of sync with each other, and a vertex accepted by one path might be handled subtly differently by another. The chokepoint forbids that.

Why the order matters

The handler does not just call verification, consensus, storage, indexing, and pub-sub. It calls them **in a specific order**, and the order is load-bearing:

- **Verify before consensus.** Consensus reasons about weight, conflicts, and which history wins. It assumes the vertex is well-formed — valid signatures, sufficient proof-of-work, no internal double-spend. Running consensus on a malformed vertex would be meaningless at best and corrupting at worst. So the vertex is fully checked first.
- **Save before consensus.** Consensus walks the DAG, reading and updating the metadata of the new vertex and its neighbours. For the new vertex to participate in that walk, it must already be in storage, registered as a child of its parents. So the handler writes it to disk before asking consensus to run.
- **Consensus before announce.** The events the handler publishes — "a new transaction was accepted" — are promises to the rest of the system (the event queue, the WebSocket feed, the mining service, wallets) that this vertex is now part of the ledger and its consensus status is settled. Announcing before consensus has run would broadcast a half-truth: subscribers might react to a vertex that consensus is about to void.

So the canonical order is: **verify → save → consensus → index → announce**. The rest of this chapter is that sentence, expanded.

Recap — the ingestion pipeline (full picture: this chapter). From Chapter 0, Act II: every vertex, however it arrives, flows through **verification → consensus → store + index + announce**. The vertex handler is the code that runs exactly that pipeline. Verification answers "is this valid on its own?" (Ch. 31). Consensus answers "does this change which history is canonical?" (Ch. 32). Storage/index/pub-sub answer "record it and tell everyone" (Ch. 27, 28, 30).

33.3 The concepts it rests on

The handler is almost pure orchestration, so the heavy concepts all belong to other chapters. Here are quick recaps so you can read §33.4 without flipping back. Each ends with a pointer to its canonical treatment.

Recap — verification (full treatment in Ch. 31). Verification checks that a vertex obeys the protocol's rules: its signatures unlock the inputs it spends, its proof-of-work meets the required weight, its structure is well-formed, and it does not double-spend within itself. The `VerificationService` runs these checks. Crucially, verification comes in tiers: a basic tier that needs no dependencies, and a full tier that needs the vertex's parents and inputs to be present and themselves valid. The vertex handler calls the full tier — `validate_full ( verification_service.py:64 )`.

Recap — consensus (full treatment in Ch. 32). Consensus decides which version of history is canonical when there is any ambiguity. It does this by comparing accumulated **weight**³ and marking the losers as **voided**⁴ (recorded in each vertex's `voided_by` metadata, not deleted). Adding one vertex can ripple: a heavier block can flip the best chain, voiding some vertices and un-voiding others. The handler triggers this by calling `consensus.unsafe_update(vertex) ( consensus.py:132 )`, which returns a list of events describing what changed.

Recap — validation state and scope (full treatment in Ch. 25 & Ch. 27). Every vertex carries a **validation state** — one of `INITIAL`, `BASIC`, `FULL`, plus checkpoint variants (`validation_state.py:45`). It records how far the node has gotten in checking that vertex. `FULL` (technically `is_fully_connected()`, `validation_state.py:68`) means "fully checked, all dependencies present and valid." Separately, the storage has a **scope** that controls which validation states it will hand back. During normal operation the scope is narrowed to only fully-valid vertices — the handler asserts this with `is_only_valid_allowed() ( transaction_storage.py:402 )`.

Recap — pub-sub announce (full treatment in Ch. 30). Pub-sub is the node's internal announcement system: components publish named events to a bus without knowing who listens, and subscribers react. The handler publishes events like `NETWORK_NEW_TX_ACCEPTED ( pubsub.py:127 )` when a vertex is settled. Downstream, the event manager persists these to the durable event queue, the WebSocket feed streams them out, and the manager relays accepted vertices to peers.

33.4 The code, walked

A generic orchestrator first

Strip away Hathor and the shape is a plain pipeline orchestrator. In pseudocode, ingesting anything that must pass checks before it changes shared state looks like this:


```
def ingest(item):
    if not is_valid(item):          # 1. gate: reject bad input early
        return False
    save(item)                     # 2. persist so later steps can see it
    changes = reconcile(item)      # 3. update shared state, compute ripples
    update_lookup_tables(item)     # 4. keep derived views consistent
    announce(item, changes)        # 5. tell the rest of the system
    return True
```

Five steps: gate, persist, reconcile, re-index, announce. The vertex handler is exactly this, with `is_valid` = verification, `reconcile` = consensus, and a couple of Hathor-specific wrinkles (a re-validation pass, and a crash-on-failure policy). Keep this skeleton in mind; everything below maps onto it.

The three entry points

There is no single method called `ingest`. Instead the handler exposes **three public entry methods**, one per source, and they all funnel into one private worker. This is worth pausing on, because the naming is a little misleading — there is no method literally called `on_new_vertex`.

Entry method	Called by	Source
<code>on_new_block</code> (<code>vertex_handler.py:85</code>)	sync agent (<code>p2p/sync_v2/agent.py:617</code>)	a block from block-sync
<code>on_new_mempool_transaction</code> (<code>vertex_handler.py:117</code>)	mempool sync (<code>p2p/sync_v2/mempool.py:143</code>)	a tx from mempool-sync
<code>on_new_relayed_vertex</code> (<code>vertex_handler.py:132</code>)	manager (<code>manager.py:878</code>), real-time relay (<code>agent.py:1176</code>)	a freshly-submitted or relayed vertex

Why three? Because the three sources differ in what context they already know, not in the core work. Each entry method's real job is to assemble a `VerificationParams` object — the bundle of contextual facts verification needs, such as which block to evaluate the vertex against and which feature⁵ rules are active — and then hand off. For example, `on_new_relayed_vertex` evaluates the vertex against the current **best block** (`vertex_handler.py:140`), because a relayed real-time transaction is meant to extend the tip of the ledger. `on_new_block`, by contrast, evaluates against the block's own parent (`vertex_handler.py:87-88`), because a block arriving from sync may be deep in history, not at the tip.

All three then call the same private worker, `_old_on_new_vertex` (`vertex_handler.py:158`).⁶ That method is the real pipeline, and it is short enough to read whole:

```
def _old_on_new_vertex(self, vertex, params, *, quiet=False) -> bool:
    is_valid = self._validate_vertex(vertex, params)          # (A) verify
    if not is_valid:
        return False
    try:
        consensus_events = self._unsafe_save_and_run_consensus(vertex) # (B) save + consensus
        self._post_consensus(vertex, params, consensus_events, quiet=quiet) # (C) index + announce
    except BaseException:
        # (D) any failure here is unrecoverable: void the tx, crash the node
        self._log.error('unexpected exception in on_new_vertex()', vertex=vertex)
        meta = vertex.get_metadata()
        meta.add_voided_by(self._settings.CONSENSUS_FAIL_ID)
        self._tx_storage.save_transaction(vertex, only_metadata=True)
        self._execution_manager.crash_and_exit(...)
    return True
```

Three steps and a catch-all. Let's take them in order.

Step A — Verify (and the accept / reject / already-known decision)

`_validate_vertex` (`vertex_handler.py:187`) is the gate. It is where the handler decides whether to proceed at all, and it folds together four checks:

```
def _validate_vertex(self, vertex, params) -> bool:
    assert self._tx_storage.is_only_valid_allowed()          # scope sanity check
    already_exists = False
    if self._tx_storage.transaction_exists(vertex.hash):
        self._tx_storage.compare_bytes_with_local_tx(vertex) # same hash => must be same bytes
        already_exists = True

    if vertex.timestamp - self._reactor.seconds() > self._settings.MAX_FUTURE_TIMESTAMP_ALLOWED:
        raise InvalidNewTransaction('...in the future...') # reject: clock-skew guard

    vertex.storage = self._tx_storage
    metadata = vertex.get_metadata()

    if already_exists and metadata.validation.is_fully_connected():
        raise InvalidNewTransaction('Transaction already exists ...') # already-known
    if metadata.validation.is_invalid():
        raise InvalidNewTransaction('previously marked as invalid') # reject

    if not metadata.validation.is_fully_connected():
        self._verification_service.validate_full(vertex, params) # the real check
    return True
```

Read it as a decision tree:

- **Already-known.** If a vertex with this hash already exists and it is already fully validated (`is_fully_connected()` , `vertex_handler.py:205`), there is nothing to do — raising here is caught upstream and turns into a clean `False` return ("we already have this, didn't re-ingest"). Note the `compare_bytes_with_local_tx` call at line 191: two vertices with the same hash must have identical bytes, or the node has either a hash collision or a bug, and it wants to know immediately.

- **Reject — in the future.** A vertex whose timestamp is too far ahead of the node's clock is rejected outright (`vertex_handler.py:194`). This is an anti-abuse guard: timestamps anchor weight and difficulty calculations, so a vertex claiming to be from next year cannot be allowed to poison them.
- **Reject — previously invalid.** If the node has already seen this vertex and concluded it breaks the rules (`is_invalid()` , `vertex_handler.py:208`), it does not waste effort re-checking; it rejects again.
- **Verify — the real work.** Otherwise, if the vertex is not yet fully validated, the handler calls `validate_full` (`vertex_handler.py:213`). This is the hand-off to Chapter 31. If full validation raises a `HathorError` , the handler wraps it as `InvalidNewTransaction` (`vertex_handler.py:214-215`) and the whole ingestion ends in **reject**.

The return value is a plain `bool` : `True` means "passed the gate, proceed"; the raised exceptions are caught one level up and become `False` . So `_old_on_new_vertex` returns `True` for accepted and `False` for rejected-or-already-known — the caller cannot tell the last two apart, and mostly does not need to.

Step B — Save, then run consensus

If the gate passes, `_unsafe_save_and_run_consensus` (`vertex_handler.py:219`) does the two middle steps as one unit:

```
def _unsafe_save_and_run_consensus(self, vertex) -> list[ConsensusEvent]:
    vertex.update_initial_metadata(save=False)
    self.tx_storage.save_transaction(vertex) # (1) persist
    with non_critical_code(self._log):
        self.tx_storage.indexes.add_to_non_critical_indexes(vertex) # (2) pre-index
    return self._consensus.unsafe_update(vertex) # (3) consensus
```

The order inside is itself deliberate. `update_initial_metadata` (`vertex_handler.py:228`) registers the new vertex as a child of its parents — wiring it into the DAG. The comment in the source explains why this is done here and not earlier: the parent-child links must not be created until the vertex is known-valid, or a rejected vertex would leave a dangling child reference in storage (`vertex_handler.py:224-227`). Then `save_transaction` writes it to disk. Only now, with the vertex on disk and linked into the graph, is `consensus.unsafe_update` called (`vertex_handler.py:232`) — because consensus needs to traverse from the new vertex through its neighbours, and it can only do that if the vertex is really there.

The word **unsafe** in both method names is not decoration. It is a contract: these methods may raise, and the caller is responsible for crashing the node if they do (the docstring at `vertex_handler.py:221-222` says exactly this). We will see why at Step D.

`unsafe_update` returns a `list[ConsensusEvent]` (`consensus.py:132`). Each `ConsensusEvent` (`consensus.py:53`) is a frozen record pairing a `HathorEvents` enum value with a `kwargs` dict — a description of something consensus changed (a vertex got

voided, a block won the best chain), queued up to be published in Step C. Consensus does not publish them itself; it hands the list back so the handler can fire them in the right order, after the rest of the bookkeeping is done.

Step C — Re-validate, update indexes, announce

`_post_consensus` (`vertex_handler.py:234`) is the tail of the pipeline — the work that should happen once a vertex is fully part of the ledger:

```
def _post_consensus(self, vertex, params, consensus_events, *, quiet) -> None:
    params = replace(params, skip_block_weight_verification=True)
    assert self._verification_service.validate_full(vertex, params, init_static_metadata=False)

    self._tx_storage.indexes.update_critical_indexes(vertex) # must succeed
    with non_critical_code(self._log):
        self._tx_storage.indexes.update_non_critical_indexes(vertex) # best-effort
        self._pubsub.publish(HathorEvents.NETWORK_NEW_TX_PROCESSING, tx=vertex)
        for event in consensus_events:
            self._pubsub.publish(event.event, **event.kwargs)
        self._pubsub.publish(HathorEvents.NETWORK_NEW_TX_ACCEPTED, tx=vertex)
        self._log_new_object(vertex, 'new {}', quiet=quiet)
```

Three things happen here, and one of them may surprise you.

The surprising one: a second `validate_full`. Why validate again, having just validated in Step A? Two reasons. First, consensus may have changed the vertex's metadata (its voided status, its score), so the node re-asserts the vertex is still coherent afterwards. Second, this re-validation runs with `skip_block_weight_verification=True` (`vertex_handler.py:248`) and `init_static_metadata=False` (`vertex_handler.py:252`) — it deliberately skips the expensive weight re-check (already done) and the static-metadata initialization (already done), so it is cheaper than the first pass. It is a consistency assertion, wrapped in `assert`, not a full re-run. (The docstring at `vertex_handler.py:242-246` also notes this method can run later than ingestion for a vertex whose dependencies arrived out of order — more on that in §33.5.)

Indexes, in two tiers. `update_critical_indexes` (`vertex_handler.py:255`) updates the indexes the ledger's correctness depends on — these must succeed. Then, inside a `non_critical_code` block (`vertex_handler.py:256`), `update_non_critical_indexes` updates the convenience indexes (address history, token lists) whose failure should not crash the node — they can be rebuilt later. The `non_critical_code` context manager⁷ is the line that draws the boundary between "if this breaks, stop the world" and "if this breaks, log it and carry on."

The announce. Finally, the events fire, in order (`vertex_handler.py:259-262`):

1. `NETWORK_NEW_TX_PROCESSING` — "a new vertex is being processed, just before consensus settles" (`pubsub.py:39-41`).
2. Every `ConsensusEvent` that consensus produced — the voiding/winning ripples.

3. `NETWORK_NEW_TX_ACCEPTED` — "this vertex is now accepted into the network" (`pubsub.py:43-45`).

These are the announcements the rest of the node has been waiting for. The event manager (Ch. 30) catches them and persists them to the durable queue; the manager catches `NETWORK_NEW_TX_ACCEPTED` to relay the vertex onward to peers (`manager.py:882-883`); the mining service catches it to know a new block landed. This is the moment "data arrived" finally becomes "ledger changed, and everyone has been told."

Step D — Failure means crash, on purpose

Look again at the `except BaseException` in `_old_on_new_vertex` (`vertex_handler.py:178`). It catches everything — not just expected validation errors, but any exception at all from save, consensus, or post-consensus. And what it does is drastic: it marks the vertex voided with a special `CONSENSUS_FAIL_ID` (`vertex_handler.py:181`), saves only that metadata, and then calls `execution_manager.crash_and_exit` (`vertex_handler.py:183`) — it **takes the whole node down**.

This looks alarming. It is intentional. Recall the contract from Step B:

`_unsafe_save_and_run_consensus` may have already written the vertex to disk and begun mutating the consensus state of its neighbours before it threw. If consensus crashes halfway through a best-chain flip, the on-disk metadata is now in an unknown, possibly-inconsistent state — some vertices updated, some not. There is no safe way to continue from there. Limping on would risk acting on a corrupt ledger, which is the one thing a full node must never do (Ch. 0, §0.2: "a full node never takes another node's word for validity" — and certainly not its own corrupted state). So the handler chooses the only safe option: stop immediately, leave a marker, and let a clean restart rebuild consistent state from the storage. **Crashing is the safe failure here.** That is why the inner methods are named `unsafe`: they hand the responsibility for this crash up to `_old_on_new_vertex`, which discharges it.

The pipeline, in one diagram

Pulling Steps A–D together — and paying off the box from Chapter 0:



33.5 How it plugs into the lifecycle

Who calls it

The vertex handler is never the originator of anything — it always runs because something handed it a vertex. There are exactly two kinds of caller, both already named in earlier chapters:

- **The manager** (Ch. 29). `HathorManager.on_new_tx` (`manager.py:864`) is the front door for locally-created and submitted vertices; it calls `on_new_relayed_vertex` (`manager.py:878`) and, if the vertex is accepted and `propagate_to_peers` is set, relays it onward (`manager.py:882-883`). When a wallet submits a transaction or the local miner finds a block, this is the path.
- **The sync agent** (Ch. 35). The peer-to-peer sync code, while catching the node up to the network, calls `on_new_block` for downloaded blocks (`p2p/sync_v2/agent.py:617`, `blockchain_streaming_client.py:131`),

`on_new_mempool_transaction` for mempool transactions (`mempool.py:143`), and `on_new_relayed_vertex` for real-time relayed transactions (`agent.py:1176`).

When the node is downloading years of history from a peer, every one of those vertices passes through here.

A third, test-only caller exists — the `dag_builder` (`dag_builder/artifacts.py:88`) uses `on_new_relayed_vertex` to feed hand-built vertices into a simulated node (Ch. 43). It is the same door, used by tests.

The convergence point

Step back and notice what has assembled in this one method. Reading

`_old_on_new_vertex` top to bottom, you touch:

- the **vertex model** (Ch. 25) — `BaseTransaction`, `Block`, metadata, validation state;
- **verification** (Ch. 31) — `validate_full`;
- **storage** (Ch. 27) — `save_transaction`, `transaction_exists`, scope;
- **indexes** (Ch. 28) — critical and non-critical updates;
- **consensus** (Ch. 32) — `unsafe_update` and the events it returns;
- **pub-sub** (Ch. 30) — the `NETWORK_NEW_TX_*` announcements;
- the **execution manager** — the crash-and-exit safety valve.

The vertex handler does not reimplement any of these. It sequences them. That is the entire value of the chapter: once you understand the order and the reasons for it, you understand how a node turns a pile of incoming bytes into a coherent, growing ledger.

A note on partial vertices

One subtlety ties back to Chapter 25's validation states. A vertex can arrive before all its dependencies (its parents and the inputs it spends) have arrived — common during sync, where vertices stream in roughly but not perfectly in dependency order. Such a vertex cannot be fully validated yet, because `validate_full` needs those dependencies present. In the codebase this is the `BASIC` /partial validation state (`validation_state.py:45-48`): the vertex is structurally checked but not fully connected.

The vertex handler's gate handles this gracefully. `_validate_vertex` only calls `validate_full` when the vertex is not yet fully connected and proceeds only if it succeeds (`vertex_handler.py:211-213`); a vertex still missing dependencies will fail full validation and be held back rather than ingested. When its missing dependencies finally arrive and are themselves ingested, the machinery re-runs and the vertex completes — which is exactly why `_post_consensus`'s docstring notes it "might happen later" than first receipt (`vertex_handler.py:244-246`). The handler is one stable gate

that a vertex may approach more than once, passing only when it is finally complete. The orchestration of which vertices are ready to retry lives in the sync agent (Ch. 35); the handler's job is to be the consistent, idempotent⁸ gate they all funnel through.

Recap

Step	What it does	Method (<code>vertex_handler.py</code>)	Chapter that owns the detail
Entry	Build <code>VerificationParams</code> per source, funnel to worker	<code>on_new_block:85</code> , <code>on_new_mempool_transaction:117</code> , <code>on_new_relayed_vertex:132</code>	Ch. 29 (manager), Ch. 35 (sync)
A — Verify	Dedup / future / invalid gate, then <code>validate_full</code>	<code>_validate_vertex:187</code>	Ch. 31 (verification)
B — Save	Link into DAG, persist, pre-index	<code>_unsafe_save_and_run_consensus:219</code>	Ch. 25, Ch. 27, Ch. 28
B — Consensus	Run consensus, collect change-events	<code>_unsafe_save_and_run_consensus:219</code> → <code>consensus.unsafe_update</code>	Ch. 32 (consensus)
C — Re-validate	Cheap consistency re-check post-consensus	<code>_post_consensus:234</code>	Ch. 31
C — Index	Critical (must) + non-critical (best-effort) updates	<code>_post_consensus:234</code>	Ch. 28 (indexes)
C — Announce	Publish <code>NEW_TX_PROCESSING</code> , consensus events, <code>NEW_TX_ACCEPTED</code>	<code>_post_consensus:234</code>	Ch. 30 (pub-sub)
D — Fail	Void with <code>CONSENSUS_FAIL_ID</code> , crash the node	<code>_old_on_new_vertex:178</code>	Ch. 29 (execution manager)

The vertex handler is small because it owns no concepts of its own. Its worth is in the sequence — verify before consensus before announce, save before traverse, crash before corrupt. Hold that ordering and you hold the spine of how the node grows its ledger. The one piece left unexamined is where most vertices come from in the first

place: the network. A freshly-booted node has an almost-empty database and must download years of history from strangers it does not trust. How it finds those strangers, connects to them, and streams the ledger across — feeding the vertex handler at the far end — is the subject of the next two chapters. **Chapter 34** opens the peer-to-peer layer.

1. A vertex is any node of Hathor's ledger graph — either a `Block` or a `Transaction`. The code uses "vertex" when it does not care which of the two it holds. Full treatment in Ch. 8 and Ch. 25. ↩
2. A chokepoint (or single point of control) is a deliberate design where all instances of some operation are routed through one piece of code, so the logic exists in exactly one place. The trade-off: it can become a bottleneck or a single point of failure, but it guarantees consistency. Here, consistency is worth far more than the negligible overhead. ↩
3. Weight measures how much proof-of-work a vertex represents; accumulated weight sums the weight reachable through the graph. Consensus prefers the history with the most accumulated weight. Ch. 9 and Ch. 32. ↩
4. A vertex is voided when consensus decides it is not part of canonical history — it is marked (via a `voided_by` set in its metadata), not deleted, so the node can reverse the decision if a later vertex changes the outcome. Ch. 10 and Ch. 32. ↩
5. A feature here is a protocol upgrade gated by feature activation — a mechanism where rule changes switch on according to a schedule and miner signalling. `VerificationParams` carries which features are active so verification applies the right rules. Ch. 38. ↩
6. The `_old_` prefix and the docstring's "New method" are historical leftovers from a refactor; in the current code this private worker is the live ingestion path. The name is rot, not a clue — read it as "the core on-new-vertex worker." ↩
7. A context manager is the `with ...:` construct. `non_critical_code(self._log)` wraps a block so that an exception inside it is logged and swallowed rather than propagated — the boundary between "fatal if it fails" and "tolerable if it fails." ↩
8. Idempotent means running an operation more than once has the same effect as running it once. The handler's gate is idempotent in spirit: re-ingesting an already-accepted vertex is detected as "already-known" and does nothing, so a vertex can safely arrive at the gate repeatedly. ↩

Chapter 34 — Peer-to-Peer Networking I: Connections & Protocol

What you'll learn in this chapter

- Why a full node, which is born knowing no one, must discover, connect to, authenticate, and maintain many simultaneous peers — with no central server to coordinate any of it.
- How `ConnectionsManager` owns the whole peer population, tracking each connection through the **connecting** → **handshaking** → **ready** phases, and how it keeps a bounded, rotating set of peers it actively syncs from.
- What a **state machine** is, and how each TCP connection is driven by one `HathorProtocol` object that walks the states `HELLO` → `PEER-ID` → `READY`.
- How an incoming wire message is dispatched to the right handler for the connection's current state — the same dispatch-table idea from Chapter 4, applied to a network protocol.
- Why authentication needs a multi-step handshake: agreeing on the network, proving identity with a key, and only then opening the connection to ledger traffic.
- Where this layer plugs into the node's life: started by `HathorManager.start`, running on the Twisted reactor, and handing each `READY` peer off to a sync agent (Chapter 35).

A node is useless alone. Everything the previous chapters built — the vertex model, storage, verification, consensus — exists to maintain one shared ledger that thousands of independent machines agree on. None of that agreement can happen until the node can talk to other nodes. This chapter is the machinery of "talking": opening connections, proving who is on each end, and keeping a healthy set of peers alive. The content of those conversations once a connection is healthy — downloading the years of history a fresh node is missing — is the next chapter. Here we build the pipe; Chapter 35 fills it.

34.1 Localization

The peer-to-peer layer lives in `hathor/p2p/`. It is one of the largest packages in the codebase (around fifty files), but it splits cleanly into a **core** — connection management, the protocol, the handshake, peer identity — and a **sync** subsystem that runs on top of a healthy connection. This chapter is the core. Sync-v2 (the `sync_v2/` subdirectory and its agent) is Chapter 35.

hathor/	
├─ manager.py	← calls <code>connections.start()</code> (Ch 29)
├─ vertex_handler/	← where received vertices land (Ch 33)
└─ p2p/	◀ YOU ARE HERE (the core)
manager.py	← <code>ConnectionsManager</code> : owns ALL connections
protocol.py	← <code>HathorProtocol</code> (one per connection) + <code>HathorLineReceiver</code>
factory.py	← <code>Server/Client</code> factories: build a protocol per connection
messages.py	← <code>ProtocolMessages</code> enum: the wire vocabulary
peer.py	← <code>PeerInfo</code> / <code>UnverifiedPeer</code> / <code>PublicPeer</code> / <code>PrivatePeer</code>
peer_id.py	← <code>PeerId</code> (a 32-byte hash type)
peer_endpoint.py	← <code>PeerAddress</code> / <code>PeerEndpoint</code> : where + who to dial
peer_storage.py	← <code>Verified</code> / <code>Unverified</code> peer dictionaries
rate_limiter.py	← leaky-bucket throttling
sync_version.py	← <code>SyncVersion</code> enum (V2 is live)
├─ states/	← the per-connection state machine
base.py	← <code>BaseState</code> (<code>cmd_map</code> = message → handler)
hello.py	← <code>HelloState</code> (network/version compatibility)
peer_id.py	← <code>PeerIdState</code> (identity proof)
ready.py	← <code>ReadyState</code> (ping/peers + hands off to sync)
├─ peer_discovery/	← how new peers are found
bootstrap.py	← a static seed list
dns.py	← DNS-seed lookup
├─ netfilter/	← a small connection-firewall (accept/reject hooks)
└─ sync_v2/	← the sync protocol – DEFERRED to Ch 35

Context. `hathor/p2p/` is the node's mouth and ears. Storage and consensus decide what is true; the p2p layer is how a node learns what its peers believe and tells them what it believes. Without it the node is an island with a correct but frozen copy of the past. It is started near the end of `HathorManager.start` (`hathor/manager.py:327`, `self.connections.start()`) and runs for the node's entire life on the Twisted reactor. Everything that arrives from the network — every block, every transaction — enters the node through a connection managed here, and leaves through the vertex-handler pipeline of Chapter 33.

34.2 What it does, and why it exists

A full node is born knowing almost nothing. It has a hard-coded genesis block, perhaps a short list of seed addresses, and otherwise an empty address book. From that standing start it must solve four problems, continuously, for as long as it runs:

1. **Discovery.** Find the addresses of other nodes to try. There is no central directory — by design, since a central directory would be a single point of control and failure for a system whose whole point is decentralization. A node bootstraps from a static seed list and from DNS seeds, then learns more peers from the peers it already has (gossip).
2. **Connection.** Open a TCP connection to a discovered address, or accept one that a remote node opens to us. A node is both a client (it dials out) and a server (it listens). Hundreds of these connections may be open at once.
3. **Authentication.** A TCP connection only proves "something answered at this address." It does not prove the thing is a real Hathor node, that it is on our network (mainnet vs. testnet), that it speaks a protocol version we understand, or who it is. A node must establish all of this before trusting the connection with ledger data. This is the **handshake**.
4. **Maintenance.** Keep the healthy connections alive (detect dead ones, ping idle ones), bound the total (memory and bandwidth are finite), drop misbehaving peers, and — because a node cannot afford to actively sync from every peer at once — keep a small rotating subset of peers as its active sync partners.

Recap — distributed ledger, no central server (full treatment in Ch. 6, §6.3). A distributed ledger is one shared record of history that many computers each hold a full copy of, with no master copy and no coordinator. Every node is an equal peer. The benefit is that there is nothing to seize, bribe, or take offline; the cost is exactly the four problems above — every node must do its own discovery, connection, authentication, and upkeep, because no one is doing it for them. → full treatment in Ch. 6.

The reason this is a whole subsystem, rather than a few socket calls, is that all four problems run concurrently and indefinitely across many connections, each connection at a different stage, all on a single thread. That shape — many independent, long-lived, event-driven conversations — is precisely what Twisted exists for, and what a state machine exists to tame.

34.3 The concepts it rests on

Three ideas underpin this chapter. Two are recaps of earlier deep-dives; the third — the state machine — is introduced here, since this is the first place in the book where the node is one.

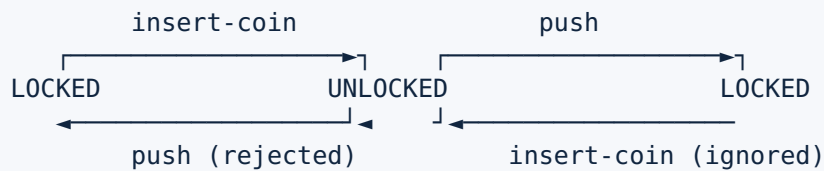
Recap — Twisted: Protocol, Factory, reactor (full treatment in Ch. 16). Twisted is the asynchronous-networking framework the node runs on. Its core, the **reactor**¹, is a single-threaded event loop that waits for network events and calls your code in response. You do not write socket loops; you write a **Protocol**² — an object with callback methods (`connectionMade`, `dataReceived` / `lineReceived`, `connectionLost`) that the reactor invokes as bytes arrive. Because there is one Protocol instance per connection, you need something to manufacture them: a **Factory**³, whose `buildProtocol` method the reactor calls once per new connection. Hathor's protocol is `HathorProtocol`; its factories are `HathorServerFactory` / `HathorClientFactory`. → full treatment in Ch. 16.

Recap — dispatch tables (full treatment in Ch. 4, §4.5). Dispatch means choosing which function to run based on a value, instead of with a long `if/elif` ladder. A **dispatch table** is a dictionary mapping the value to the handler: `handler = table[key]; handler(args)`. Adding a new case is adding a dictionary entry. The p2p layer uses this twice over: a message's type (`HELLO`, `PING`, `PEERS`, ...) is the key, and each state carries its own table of which messages it will answer. → full treatment in Ch. 4.

A state machine, from scratch

A **state machine**⁶ is a way of describing an object whose behavior depends on which phase it is currently in. The object is always in exactly one of a small, fixed set of **states**. Certain events cause **transitions** from one state to another. In each state, the object accepts only a limited set of events; an event that does not belong to the current state is an error.

The textbook example is a coin turnstile. It has two states, `LOCKED` and `UNLOCKED`, and two events, `insert-coin` and `push`:



state	event	action	next state
LOCKED	insert-coin	unlock the arm	UNLOCKED
LOCKED	push	(rejected – stays put)	LOCKED
UNLOCKED	push	let person through	LOCKED
UNLOCKED	insert-coin	(wasted coin)	UNLOCKED

Two things make this design valuable. First, **the legal events are scoped to the state**: in `LOCKED`, pushing does nothing; the turnstile does not have to defend against being pushed because that case is handled by the state, not by a tangle of flags. Second, **the transitions are explicit**: there is exactly one place that says "an inserted coin moves us from `LOCKED` to `UNLOCKED`," so the lifecycle is easy to read and hard to corrupt.

Now scale the idea up. A network connection is also an object with phases. When it first opens, the only sensible message to receive is a greeting; a transaction arriving before the greeting is nonsense (we do not even know who sent it). After the greeting, the only sensible next thing is an identity claim. Only once identity is proven is the connection allowed to carry ledger traffic. That is three states — `HELLO`, `PEER-ID`, `READY` — and the transitions between them are exactly the steps of the handshake. Modelling the connection as a state machine means each state can hold its own dispatch table of acceptable messages, and a message that does not belong to the current phase is automatically rejected. We will see Hathor's version is almost a literal transcription of the turnstile table, with three states instead of two.

34.4 The code, walked

We build the picture from the outside in: first the manager that owns every connection, then the single protocol object that drives one connection, then the state machine and handshake inside it, then the supporting cast (peer identity, endpoints, discovery, netfilter).

34.4.1 `ConnectionsManager` — owner of the peer population

`ConnectionsManager` (`hathor/p2p/manager.py:75`) is the one long-lived object that holds all connections and all knowledge of peers. There is exactly one per node; the `HathorProtocol` for each connection holds a reference back to it (`protocol.py:115`). It is built and wired by the builder (Chapter 24) and handed the reactor, the node's own identity (`my_peer`), the pub-sub bus, and a bundle of flags.

Its most informative fields are the **peer sets**, because they are the connection lifecycle made concrete (`manager.py:148–165`):

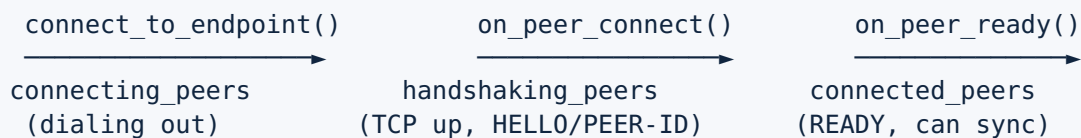
```
# All connections, in any phase.
self.connections = set()          # set[HathorProtocol]

# Outbound dials in progress (TCP not yet established).
self.connecting_peers = {}        # endpoint -> _ConnectingPeer

# TCP up, but still doing the handshake (not yet authenticated).
self.handshaking_peers = set()    # set[HathorProtocol]

# Handshake complete: authenticated and ready to communicate.
self.connected_peers = {}        # PeerId -> HathorProtocol
```

A connection moves left-to-right through these sets, and the move is the lifecycle:



- A peer enters `connecting_peers` when we start dialing it (`connect_to_endpoint` , `manager.py:743`); the value is a `_ConnectingPeer` holding the target endpoint and the Twisted `Deferred`⁴ for the in-progress connection attempt.
- When the TCP connection is established, the protocol's `on_connect` fires and calls back into `on_peer_connect` (`manager.py:413`), which adds the protocol to `connections` and `handshaking_peers` . It also enforces `max_connections` here (`manager.py:415`): if the node is already full, the new connection is dropped immediately.
- When the handshake finishes and the peer reaches READY, `on_peer_ready` (`manager.py:428`) removes it from `handshaking_peers` and records it in `connected_peers` , keyed by the now-verified `PeerId` . Note the key change: a connecting/handshaking peer is tracked by its connection object because we do not yet trust its identity; only a ready peer is keyed by its `PeerId` , because only now has that id been proven.

Each transition also publishes an event on the pub-sub bus — `NETWORK_PEER_CONNECTING` , `NETWORK_PEER_CONNECTED` , `NETWORK_PEER_READY` , `NETWORK_PEER_DISCONNECTED` (e.g. `manager.py:422` , `:439` , `:502`) — so that metrics, the status API, and the event subsystem (Chapter 30) can observe the peer population without the manager knowing who is listening.

The connection loops. `start` (`manager.py:292`) does not connect to everyone at once; that would be a thundering herd. Instead it arms several `LoopingCall`⁵ timers on the reactor:

- `lc_connect` (`manager.py:186`, every 0.2 s) pulls one new peer to dial from a queue (`connect_to_peer_from_connection_queue`, `manager.py:549`). The queue is fed by the peers our existing peers tell us about — this is the gossip mechanism in action.
- `lc_reconnect` (`manager.py:171`, every 5 s) retries known-but-disconnected peers and prunes peers we have not seen in a long time (`peers_cleanup`, `manager.py:534`).
- `lc_sync_update` (`manager.py:175`) periodically rotates which peers we actively sync from (below).
- `start` finally calls `do_discovery` (`manager.py:271`) to kick off the discovery strategies.

Sync rotation. A node may be connected to hundreds of peers, but it cannot afford to download history from all of them simultaneously. So only a bounded subset — `MAX_ENABLED_SYNC` peers — has sync enabled at any moment, and the manager periodically reshuffles that subset so no peer is starved. When a peer first becomes ready, sync is enabled for it only if we are still under the cap (`manager.py:464`). The reshuffle is `_sync_rotate_if_needed` (`manager.py:908`): it shuffles the connected peers, picks a new random `MAX_ENABLED_SYNC` set, and enables/disables sync on connections to match (`manager.py:927–931`). A small `always_enable_sync` set (`manager.py:180`) lets an operator pin specific peers as permanent sync partners.

Recap — sync agents exist, detail is Ch 35. Each READY connection owns a **sync agent**: the object that runs the actual ledger-download conversation. The manager's job is only to decide which connections have an active sync agent at any time (the rotation above) and to relay a freshly-created transaction to all ready peers (`send_tx_to_peers`, `manager.py:378`). What the sync agent does — streaming blocks and transactions in dependency order — is the canonical sync-v2 deep-dive in Chapter 35. Here, treat "enable sync on this connection" as a black box. → full treatment in Ch. 35.

34.4.2 HathorProtocol — one object per connection

Where `ConnectionsManager` is the city, `HathorProtocol` (`hathor/p2p/protocol.py:51`) is one street: a single object created for, and living exactly as long as, one connection. Its docstring states the contract plainly: "An instance of this class is created for each connection." It holds that connection's transport, its current state, its rate limiters, its idle-timeout timer, and — once the handshake succeeds — the verified `PublicPeer` on the other end (`protocol.py:86`, exposed through the `peer` property at :99 which asserts the peer is set).

`HathorProtocol` is split from the Twisted plumbing on purpose. The bytes-on-the-wire mechanics live in a subclass, `HathorLineReceiver` (`protocol.py:430`), which inherits both `HathorProtocol` and Twisted's `LineReceiver`. This is the Hathor wire format in one sentence: **one text message per line**. `LineReceiver` buffers incoming bytes and calls `lineReceived` once per complete line, so the protocol never has to deal with partial reads.

The three Twisted lifecycle callbacks (Chapter 16) map onto Hathor's own methods:

```
class HathorLineReceiver(LineReceiver, HathorProtocol):
    MAX_LENGTH = 65536

    def connectionMade(self) -> None:          # protocol.py:436
        super().connectionMade()
        self.setLineMode()
        self.on_connect()                      # -> enter HELLO state

    def connectionLost(self, reason=connectionDone) -> None: # :441
        super().connectionLost()
        self.on_disconnect(reason)            # -> tell the manager
```

`on_connect` (`protocol.py:263`) records the time, arms the idle-timeout timer, sets the initial state to `HELLO` (`protocol.py:275`), and notifies the manager via `on_peer_connect`. `on_disconnect` (`protocol.py:296`) tears the state down and notifies the manager via `on_peer_disconnect`. The `MAX_LENGTH = 65536` cap (`protocol.py:434`) is a defense: a peer cannot make us buffer an unbounded line.

Sending and receiving a message. A message is a command word, optionally followed by a space and a payload. `send_message` on the line receiver builds the line and writes it (`protocol.py:479`):

```
def send_message(self, cmd_enum, payload=None):    # protocol.py:479
    cmd = cmd_enum.value
    line = '{} {}'.format(cmd, payload).encode('utf-8') if payload else cmd.encode('utf-8')
    self.sendLine(line)
```

Receiving is the mirror image and the heart of dispatch. `lineReceived` (`protocol.py:450`) decodes the bytes, splits off the first word as the command, looks it up in the `ProtocolMessages` enum, and forwards to `recv_message`:

```
def lineReceived(self, line: bytes) -> None:      # protocol.py:450
    if self.aborting:                             # connection being torn down; ignore buffered
        return
    sline = line.decode('utf-8')                   # bad UTF-8 -> drop connection
    msgtype, _, msgdata = sline.partition(' ')
    cmd = ProtocolMessages(msgtype)                # unknown command -> drop connection
    self.recv_message(cmd, msgdata)
```

`recv_message` (`protocol.py:322`) is where the dispatch table and the state machine meet:

```

def recv_message(self, cmd: ProtocolMessages, payload: str) -> None:    # protocol.py:322
    assert self.state is not None
    ...
    if not self.ratelimit.add_hit(self.RateLimitKeys.GLOBAL):           # too many messages?
        self.state.send_throttle(...)
        return
    cmd_handler = self.state.cmd_map.get(cmd)                             # the dispatch
    if cmd_handler is None:
        self.send_error_and_close_connection('Invalid Command: ...') # not legal in this state
        return
    deferred_result = defer.maybeDeferred(cmd_handler, payload)          # handlers may be sync or
    deferred_result.addCallback(lambda _: self.reset_idle_timeout()) \
        .addErrback(self._on_cmd_handler_error, cmd)

```

Read that carefully, because three of the chapter's ideas converge in those lines. First, **the handler comes from `self.state.cmd_map`** — the current state's dispatch table — so the same command word can be legal in one state and rejected in another. Second, a command with no handler in the current state is a protocol violation and the connection is closed; this is the turnstile's "push while locked" made real. Third, `defer.maybeDeferred` lets a handler be a plain function or a coroutine returning a `Deferred` — necessary because some handlers (identity validation) must do asynchronous work like a DNS lookup, while others (ping) are instant.

34.4.3 The state machine: HELLO → PEER-ID → READY

The states are named in an inner enum on the protocol (`protocol.py:70`), and the values are the state classes:

```

class PeerState(Enum):          # protocol.py:70
    HELLO = HelloState
    PEER_ID = PeerIdState
    READY = ReadyState

```

`change_state` (`protocol.py:190`) is the single transition function. It lazily instantiates the state class the first time it is entered, calls `on_exit` on the old state and `on_enter` on the new one, and stores it as `self.state`. There is exactly one place a connection's phase changes — the turnstile virtue of "transitions are explicit."

Every state subclasses `BaseState` (`hathor/p2p/states/base.py:30`), which holds the dispatch table and a couple of always-legal handlers:

```

class BaseState:                # base.py:30
    cmd_map: dict[ProtocolMessages, Callable[...]]
    def __init__(self, protocol, settings):
        ...
        self.cmd_map = {          # base.py:41
            ProtocolMessages.ERROR: self.handle_error,
            ProtocolMessages.THROTTLE: self.handle_throttle,
        }
    def on_enter(self) -> None: raise NotImplementedError
    def on_exit(self) -> None: pass

```

`ERROR` and `THROTTLE` are legal in every state (any peer may report an error or a throttle at any time). Each concrete state then adds the messages it accepts. This is the structure of the whole machine in one diagram:

STATE	on_enter sends	accepts (cmd_map)	on success -> next
HELLO (compat)	HELLO	HELLO (+ ERROR, THROTTLE)	PEER-ID
PEER-ID (identity)	PEER-ID	PEER-ID, READY (+ ERROR, THROTTLE)	READY (when BOTH sides ready)
READY (operate)	PEERS, PING, GET-PEERS ...	PING, PONG, GET-PEERS, PEERS, + sync messages (+ ERROR, THROTTLE)	(terminal: stays READY, sync agent runs here)

Each state below is one class; each transition is one `change_state` call.

HelloState (`hathor/p2p/states/hello.py:35`) checks **compatibility**. On entry it sends a `HELLO` message describing this node (`send_hello`, `hello.py:76`): the app version, the **network name** (mainnet/testnet/...), a short hash of the genesis block, a timestamp, the node's capabilities, and the sync protocol versions it supports (`_get_hello_data`, `hello.py:46`). `handle_hello` (`hello.py:83`) validates the peer's matching message and refuses the connection on any mismatch:

- wrong network → close (`hello.py:138`); a mainnet node must never talk ledger to a testnet node.
- different genesis short hash → close (`hello.py:142`); a different genesis means a different chain entirely.
- clocks too far apart → close (`hello.py:146`); timestamps anchor proof-of-work difficulty, so wildly skewed clocks are unusable.
- no sync version in common → close (`hello.py:123`); if we cannot agree on how to sync, the connection is pointless.

Only if everything matches does it transition:

`protocol.change_state(protocol.PeerState.PEER_ID)` (`hello.py:173`). Note what is absent: nothing here proves who the peer is. `HELLO` establishes "we are the same kind of node on the same network speaking a common version." Identity is the next state's job — which is exactly why it is a separate state.

PeerIdState (`hathor/p2p/states/peer_id.py:32`) checks **identity**. On entry it sends a `PEER-ID` message containing this node's `id`, its public key, and its entrypoints (`send_peer_id`, `peer_id.py:86`). `handle_peer_id` (`peer_id.py:92`) — note it is `async`, because it may do a DNS lookup — validates the peer's claim:

```

async def handle_peer_id(self, payload: str) -> None:          # peer_id.py:92
    peer = PublicPeer.create_from_json(data)                    # parse id + public key
    if protocol.expected_peer_id and peer.id != protocol.expected_peer_id:
        protocol.send_error_and_close_connection('Peer id different from the requested one.')
    ...
    if protocol.connections.is_peer_connected(peer.id):        # already connected to this id?
        protocol.send_error_and_close_connection('We are already connected.')
    entrypoint_valid = await peer.info.validate_entrypoint(protocol)  # the address checks out?
    if protocol.use_ssl:
        certificate_valid = peer.validate_certificate(protocol)      # TLS cert key == claimed
    protocol._peer = peer                                         # accepted; remember the verified peer
    self.send_ready()

```

The identity check is cryptographic, and worth understanding precisely. A peer's `id` is not a self-chosen nickname — it is the double-SHA-256 hash of its RSA public key (`_calculate_peer_id`, `hathor/p2p/peer.py:93`). When `PublicPeer.create_from_json` runs, it calls `validate` (`peer.py:376`), which recomputes the id from the supplied public key and rejects the peer if it does not match (`peer.py:378`). So a peer cannot claim an id without also presenting the public key that hashes to it. And because the TLS connection's certificate is signed with the matching private key, `validate_certificate` (`peer.py:332`) confirms the party on the wire actually holds that private key — not merely a copy of someone's public key. Together these mean: **the id is bound to a keypair, and the connection proves possession of the private half**. That is what makes a `PeerId` a real identity rather than a label.

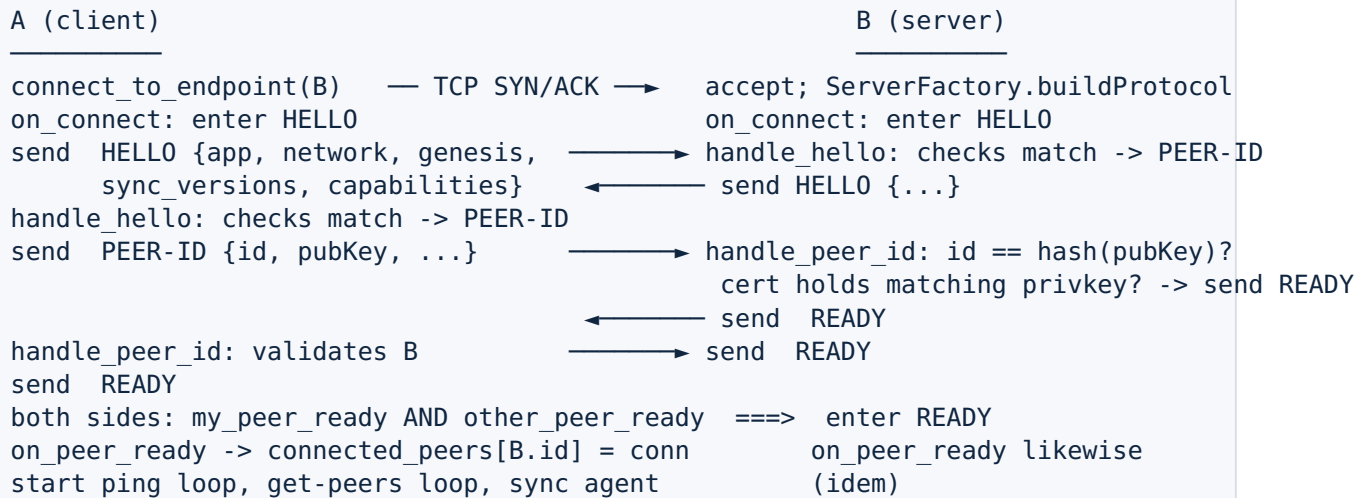
The transition to `READY` is the one genuinely subtle part of the handshake, because it is symmetric. Each side independently finishes validating the other and sends a `READY` message; a side only moves to the `READY` state once it has both sent its own `READY` (`my_peer_ready`) and received the peer's (`other_peer_ready`) (`peer_id.py:54–70`). This double-gate guarantees neither side starts sending ledger traffic before the other is prepared to receive it.

ReadyState (`hathor/p2p/states/ready.py:40`) is **operation**. The handshake is over; the connection is now a trusted, bidirectional channel. On entry (`on_enter`, `ready.py:132`) it notifies the manager (`on_peer_ready`), starts a periodic `PING / PONG` keepalive (`lc_ping`), starts a periodic `GET-PEERS` request to keep learning new peers (`lc_get_peers`, every 5 minutes), and — crucially — **creates and starts the sync agent** (`ready.py:144`, `self.sync_agent.start()`). Its `cmd_map` (`ready.py:83`) now accepts the operational messages: `PING`, `PONG`, `GET-PEERS`, `PEERS`, plus optional capability-gated commands, plus all the sync messages, which it merges in from the sync agent (`ready.py:130`, `self.cmd_map.update(self.sync_agent.get_cmd_dict())`).

This last line is where the two layers join. The sync agent contributes its own message handlers into the ready state's dispatch table. From `recv_message`'s point of view nothing special happens — it is still "look up the command in `self.state.cmd_map`" — but now a `BLOCKS` or `TRANSACTION` message resolves to a sync-agent handler. The core protocol does not know or care what those handlers do; that is Chapter 35.

34.4.4 A worked handshake, end to end

Trace a single outbound connection from nothing to READY. We are node **A**, dialing node **B**.



Every arrow is one line of UTF-8 text. Every box is a `cmd_map` lookup in `recv_message` resolving to a state handler. The whole exchange is four message types — `HELLO`, `PEER-ID`, `READY`, and then the operational set — and the connection only carries a block or a transaction after the final transition. The handshake is multi-step not by accident but because each step gates the next: you cannot meaningfully check identity before you have agreed you are the same kind of node, and you must not carry ledger data before identity is proven.

34.4.5 The factories — one protocol per connection

The reactor needs a way to manufacture a fresh `HathorProtocol` for each new connection, inbound or outbound. That is the `Factory` (`hathor/p2p/factory.py`). Both factories share a base whose `buildProtocol` constructs the line receiver and stamps it with the connection's direction:

```
class _HathorLineReceiverFactory(ABC, protocol.Factory): # factory.py:26
    inbound: bool
    def buildProtocol(self, addr) -> HathorLineReceiver: # factory.py:43
        p = HathorLineReceiver(my_peer=self.my_peer, p2p_manager=self.p2p_manager,
                               use_ssl=self.use_ssl, inbound=self.inbound, settings=self._setting
        p.factory = self
        return p

class HathorServerFactory(_HathorLineReceiverFactory, protocol.ServerFactory): # :55
    inbound = True # we accepted the connection
class HathorClientFactory(_HathorLineReceiverFactory, protocol.ClientFactory): # :61
    inbound = False # we dialed out
```

The only difference between the two is the `inbound` flag, but it matters: the direction is recorded on every protocol and is later used to break ties when two nodes accidentally connect to each other twice (`get_connection_to_drop`, `manager.py:812` — keep the

connection initiated by the peer with the larger id, deterministically, so both sides agree which to drop). The manager builds one of each factory in its constructor (`manager.py:134`); the server factory is handed to `listen` and the client factory to `connect_to_endpoint`.

34.4.6 Peer identity: the four-layer peer

A peer is modelled by four small classes that build on one another (`hathor/p2p/peer.py`). The module docstring is unusually emphatic that they **must not be mixed**, and the design is deliberate (`peer.py:14–31`):

PeerInfo	entrypoints + reconnect bookkeeping	(<code>peer.py:104</code>)
▲ held by		
UnverifiedPeer	a PeerId + PeerInfo	(<code>peer.py:207</code>)
▲ wraps		
PublicPeer	an UnverifiedPeer + a public key	(<code>peer.py:269</code>)
▲ wraps		
PrivatePeer	a PublicPeer + the private key	(<code>peer.py:393</code>)

The split encodes how much we trust this peer's identity in the type itself:

- **UnverifiedPeer** is a peer we have only heard about — typically gossiped to us by another peer via a `PEERS` message. We have an id and some addresses, but no proof either is real. We may try to connect to it; we may not trust it.
- **PublicPeer** is a peer whose public key we hold and whose id we have therefore verified (the id is the hash of the key). This is what `handle_peer_id` produces and what lands in `connected_peers`. We can verify its signatures.
- **PrivatePeer** is us. It additionally holds the private key, so it can sign messages and present a TLS certificate. Each node has exactly one `PrivatePeer` — `my_peer`.

Critically, these are **not** a subclass chain — `PublicPeer` is not a subclass of `UnverifiedPeer` (`peer.py:25`). That is intentional: it means a function typed to take an `UnverifiedPeer` will not silently accept a `PublicPeer`, so the trust level cannot be smuggled across by accident. To go "down" a level you call an explicit converter like `to_unverified_peer()`. The types are a tripwire against treating an unverified rumor as a verified identity.

`PeerId` itself (`hathor/p2p/peer_id.py:18`) is the thinnest of wrappers — a subclass of the project's `Hash` type (`hathor/types.py:30`), i.e. a 32-byte value with convenient `hex` `str/repr`. `PeerInfo` (`peer.py:104`) holds the peer's entrypoints (addresses others can reach it at) and the exponential-backoff reconnect state (`retry_interval` doubles on each failure, `increment_retry_attempt`, `peer.py:178`) so a dead peer is retried less and less often instead of being hammered.

34.4.7 Endpoints: where, and optionally who

A `PeerAddress` (`hathor/p2p/peer_endpoint.py:75`) is a frozen `(protocol, host, port)` triple — purely where to connect, e.g. `tcp://1.2.3.4:40403`. A `PeerEndpoint` pairs an address with an optional `peer_id` — where, and who we expect to find there. The distinction matters during dialing: if we connect to an endpoint that names an expected peer id, `handle_peer_id` enforces that the peer who answers actually has that id (`peer_id.py:114`), defeating an attacker who hijacks an address.

`PeerAddress.to_client_endpoint` (used at `manager.py:730`) turns the address into a Twisted client endpoint the reactor can dial. The module even forbids comparing a `PeerAddress` to a `PeerEndpoint` directly (`peer_endpoint.py:31`), again using types to prevent a where/who confusion.

34.4.8 Peer storage: two address books

The manager keeps two peer dictionaries, mirroring the trust split (`hathor/p2p/peer_storage.py`). `VerifiedPeerStorage` (`peer_storage.py:92`) holds `PublicPeer`s we have connected to and authenticated; it lives on the manager (`manager.py:165`).

`UnverifiedPeerStorage` (`peer_storage.py:100`) holds `UnverifiedPeer`s — rumors — and lives on each protocol (`protocol.py:174`), because the peers a connection tells us about are only as trustworthy as that connection. Both are `dict[PeerId, ...]` subclasses with a `max_size`; when full, they evict a random entry (`_ensure_max_size`, `peer_storage.py:44`) so a peer cannot exhaust our memory by flooding us with fake addresses. The flow of a gossiped peer is: arrives in a `PEERS` message → `on_receive_peer` adds it to the protocol's unverified storage (`protocol.py:389`) → the connect loop later picks it as a dial candidate → if the handshake succeeds, it graduates to the manager's verified storage (`on_peer_ready`, `manager.py:431`).

34.4.9 Peer discovery (brief)

Discovery answers "where do I find peers in the first place?" It is a small strategy hierarchy under `peer_discovery/`. `PeerDiscovery` (`peer_discovery/peer_discovery.py:21`) is an abstract base with one method, `discover_and_connect`, that is handed a `connect_to_endpoint` callback and is expected to call it for each peer it finds. Two implementations ship:

- `BootstrapPeerDiscovery` (`peer_discovery/bootstrap.py:27`) is the simplest possible: a static list of entrypoints from configuration; it just calls the callback for each.
- `DNSPeerDiscovery` (`peer_discovery/dns.py:35`) looks up DNS seed hostnames and turns the A/AAAA/TXT records into entrypoints — the same trick Bitcoin uses, letting the seed operator update the peer list without a software release.

The manager holds a list of these strategies (`manager.py:126`) and runs them all in `do_discovery` (`manager.py:271`), once at startup and again periodically when peers run low. After bootstrap, the dominant source of new peers is gossip (`GET-PEERS / PEERS`), so discovery seeds the network and gossip sustains it.

34.4.10 Netfilter (brief)

`netfilter/` is a small, iptables-flavored connection firewall: named chains of rules, each rule a match plus a verdict (accept/reject). Three chains are defined (`netfilter/instances.py:19–22`) and consulted at three points in the connection's life: `pre_conn` before a connection is even built (`NetfilterFactory.buildProtocol` , `netfilter/factory.py:39`), `post_hello` after the HELLO exchange (`hello.py:168`), and `post_peerid` after identity is proven (`peer_id.py:157`). By default every chain's policy is `accept` (`instances.py:20`), so the filter is inert unless an operator adds rules — but the hooks let a node reject, say, a banned address before wasting any work on it.

34.5 How it plugs into the lifecycle

The p2p core is one cog in the node's startup and steady-state machinery described in Chapter 0.

Startup. The builder (Chapter 24) constructs the `ConnectionsManager` , registers the `sync-v2` factory on it, and hands it to the `HathorManager` . Near the end of `HathorManager.start` , after storage is unlocked and metrics are up, the node calls `self.connections.start()` (`hathor/manager.py:327`). That arms the connect/reconnect/sync-update loops and runs discovery — from this instant the node is reaching out to the network.

Steady state. Everything thereafter is reactor-driven (Chapters 16 & 23). The reactor accepts inbound connections (via `listen` , building protocols from the server factory) and completes outbound ones (via `connect_to_endpoint` , the client factory); each new connection gets a fresh `HathorProtocol` that walks `HELLO → PEER-ID → READY` . Reaching `READY` enables a sync agent on a rotating subset of peers. From then on the connection carries `PING / PONG` keepalives, `GET-PEERS / PEERS` gossip, and sync traffic.

Where vertices go. When a sync agent (or a relayed transaction) pulls a new block or transaction off a `READY` connection, it does not apply it directly. It hands the vertex to the vertex handler (Chapter 33), which runs the `verify → consensus → store` pipeline; only then is the vertex part of the node's ledger. Conversely, when this node creates or accepts a new vertex, `send_tx_to_peers` (`manager.py:378`) shuffles the ready connections and pushes it out so the network converges. The p2p layer is thus the boundary: ledger data crosses into the node here and crosses out here, and the vertex handler is the gate in between.

Recap

Component	Responsibility	File (key line)
ConnectionsManager	Owns all connections; runs connect/reconnect/discovery/sync-rotation loops	p2p/manager.py:75
Peer sets	connecting → handshaking → connected — the lifecycle made concrete	p2p/manager.py:148–165
HathorProtocol	One per connection; holds state, transport, peer; dispatches messages	p2p/protocol.py:51
HathorLineReceiver	Twisted plumbing: one message per UTF-8 line	p2p/protocol.py:430
recv_message	Looks up the command in the current state's cmd_map (dispatch)	p2p/protocol.py:322
PeerState enum	The three states HELLO / PEER_ID / READY	p2p/protocol.py:70
HelloState	Compatibility: network, genesis, version, clock	p2p/states/hello.py:35
PeerIdState	Identity: id == hash(pubkey), cert holds privkey; symmetric READY	p2p/states/peer_id.py:32
ReadyState	Operation: ping/peers; creates and starts the sync agent	p2p/states/ready.py:40
HathorServerFactory / HathorClientFactory	Build one protocol per inbound / outbound connection	p2p/factory.py:55 / :61
UnverifiedPeer / PublicPeer / PrivatePeer	Trust levels encoded as distinct (non-inheriting) types	p2p/peer.py:207 / :269 / :393
PeerId	A peer's identity = double-SHA-256 of its public key	p2p/peer.py:93 , peer_id.py:18
PeerEndpoint	Where + optionally who to dial	p2p/peer_endpoint.py

Component	Responsibility	File (key line)
Verified / Unverified storage	Two address books matching the trust split	p2p/peer_storage.py: 92 / :100
ProtocolMessages	The wire vocabulary (HELLO , PEERS , PING , ... + sync)	p2p/messages.py:45
Peer discovery	Bootstrap list + DNS seeds; gossip sustains it	p2p/peer_discovery/
Netfilter	Optional accept/reject hooks at 3 connection stages	p2p/netfilter/instances.py:19

The p2p core solves a problem with no analogue in a single-machine program: how to build and keep a trustworthy, self-renewing set of relationships with strangers, on an open network, with no coordinator. It does so by owning every connection in one manager, driving each connection through an explicit `HELLO → PEER-ID → READY` state machine that gates compatibility, then identity, then trust, and dispatching every wire message through the current state's table. Reaching `READY` is not the end of the story — it is the beginning of the conversation that actually matters. Two `READY` peers must now reconcile their copies of the ledger: the joining node may be years behind, and it must download every missing block and transaction in dependency order without ever applying an invalid one. That conversation — `sync-v2` — is Chapter 35.

1. The reactor is Twisted's central event loop: a single-threaded loop that waits for events (network data, timers, connection events) and calls the right callback in response, so the node never blocks on any one connection. Full treatment in Ch. 16. ↩
2. A Twisted Protocol is an object representing one connection's behavior, with callback methods (`connectionMade` , `dataReceived` / `lineReceived` , `connectionLost`) the reactor invokes as the connection's events occur. There is one instance per connection. Full treatment in Ch. 16. ↩
3. A Twisted Factory is an object whose `buildProtocol` method manufactures a fresh Protocol instance for each new connection. Server factories build protocols for inbound connections; client factories for outbound ones. Full treatment in Ch. 16. ↩
4. A Deferred is Twisted's placeholder for a result that is not ready yet — a "future." You attach callbacks (run on success) and errbacks (run on failure); the reactor fires them when the result arrives. Full treatment in Ch. 16. ↩
5. A `LoopingCall` is a Twisted helper that calls a function repeatedly at a fixed interval on the reactor, used here for the periodic connect, reconnect, ping, and sync-rotation timers. Full treatment in Ch. 16 / Ch. 29. ↩
6. A state machine (finite-state machine) is a model in which an object is always in exactly one of a fixed set of states, certain events trigger transitions between states, and each state accepts only a limited set of events. It makes a connection's lifecycle explicit and rejects out-of-phase messages by construction. ↩

Chapter 35 — Peer-to-Peer Networking II: Sync-v2

What you'll learn in this chapter

- Why a node that has just started cannot "ask for everything" at once — and what dependency order means for a ledger.
- How two peers agree on **which** sync protocol to speak (version negotiation), and why only **sync-v2** survives in the code.
- The **backbone-first** strategy: download the chain of blocks in height order first, then fill in the transaction DAG that each block confirms.
- How sync-v2 finds the **common ancestor** — the boundary between "history I already have" and "history I am missing" — with an n-ary search over block heights.
- The two halves of the protocol: **block streaming** and **transaction streaming**, plus a separate **mempool sync** for unconfirmed transactions.
- How a received vertex re-enters the node through the verification → consensus → ingestion pipeline you met in Chapters 31-33.

This is the canonical deep-dive on synchronization. Chapter 34 covered the plumbing of the peer-to-peer layer: how a node finds peers, opens a connection, performs the handshake, and walks each connection through a state machine until it reaches the **READY** state. This chapter picks up exactly there. Once a connection is **READY**, a sync agent is attached to it, and that agent has one job: make our copy of the ledger match the peer's. Everything below is about how that agent works.

Sync is, frankly, the most stateful and intricate subsystem in the networking layer. We will build it up slowly — first the problem in the abstract, then the concepts it leans on, then the code — and we will hand-trace the message flow so the moving parts settle into place. Take it in order.

35.1 Localization

Sync-v2 lives entirely inside the peer-to-peer package, in its own sub-folder:

```

hathor/p2p/
├── sync_version.py      ← SyncVersion enum (V1_1, V2) + ordering
├── sync_agent.py        ← SyncAgent: abstract interface a sync agent implements
├── sync_factory.py      ← SyncAgentFactory: abstract factory for sync agents
├── manager.py           ← ConnectionsManager: registers/enables sync factories
├── states/
│   ├── hello.py        ← HELLO handshake: versions are negotiated here
│   └── ready.py         ← READY state: sync agent attached & started here
├── sync_v2/
│   └── factory.py       ◀ YOU ARE HERE
│       ├── agent.py     ← SyncV2Factory: a NodeBlockSync per conn
│       ├── streamers.py ← NodeBlockSync: per-connection driver (brain)
│       ├── blockchain_streaming_client.py ← StreamEnd + the two streaming servers (we→peer)
│       ├── transaction_streaming_client.py ← receives streamed blocks (peer→us)
│       ├── mempool.py   ← receives streamed transactions (peer→us)
│       ├── payloads.py  ← SyncMempoolManager: fetch unconfirmed txs
│       └── exception.py ← Pydantic models for the wire messages
│           └── streaming-specific errors

```

The folder splits cleanly into three roles. **agent.py** holds `NodeBlockSync`, the object created once per peer connection that orchestrates the whole catch-up. The **client** files (`blockchain_streaming_client.py`, `transaction_streaming_client.py`) are the receiving side — they consume data the peer pushes at us. The **server** side lives in `streamers.py` — those classes are what we run when we are the peer being asked to send data; another node's client talks to our server. Most nodes do both at once: you are catching up from a peer ahead of you while feeding a peer behind you.

Recap — the P2P connection and the READY state (full treatment in Ch. 34).

Each connection to a peer is driven by a Twisted protocol object that progresses through a state machine: HELLO → PEER-ID → READY. Only when a connection reaches **READY** is it trusted enough to exchange ledger data. The sync agent for a connection is created and started exactly when that connection enters READY — see [hathor/p2p/states/ready.py:127](#) . → full treatment in Ch. 34.

Context. Sync-v2 is what turns a freshly-installed binary into a working full node.

Without it, a node would know only its hard-coded genesis and nothing else; it could never validate a real transaction because it would be missing all the history that transaction depends on. Sync is the bridge between "I just booted with an empty database" and "I hold the same ledger as the rest of the network, and I can independently verify every byte of it."

35.2 What it does and why it exists

Start with the situation. You download `hathor-core`, point it at a data directory, and run `run_node`. Your database contains essentially one thing: the **genesis**¹ — a hard-coded first block and a couple of initial transactions that every Hathor node agrees on by definition. Meanwhile the live network has years of history: millions of blocks and transactions, all of which the peers you are about to connect to already hold.

Sync is the process of closing that gap. It must download every block and every transaction you are missing, and — this is the part that makes it interesting — it must do so in an order that lets you **verify** each one as it arrives.

Why you cannot just ask for everything

The naive idea is: "ask the peer to dump its entire database at me, I'll write it all to disk, done." This fails for a structural reason that comes straight from how the ledger is built.

A full node never trusts a peer's word that a vertex is valid (recall the full in "full node" from Chapter 0). It re-checks everything itself. But to check a transaction, you need the things it depends on to already be present and validated:

- A transaction **spends inputs** — it consumes outputs created by earlier transactions. You cannot confirm "this input is real and unspent" unless you already hold the transaction that created that output.
- A vertex **points at parents** — every vertex links back to two earlier vertices in the DAG. You cannot place a vertex in the graph until its parents are in the graph.
- A block **builds on a previous block** — it names the block before it. You cannot compute a block's height or its accumulated weight until the chain beneath it exists.

So the vertices must arrive in **dependency order**: every vertex's dependencies land before it does. This is exactly a topological order² of the DAG. Hand a node a transaction whose inputs it has never seen and it has no way to judge whether that transaction is spending real coins or inventing them out of thin air. Order is not an optimization here; it is a correctness requirement.

Recap — the DAG and the block backbone (full treatment in Ch. 8 & 9). Hathor's ledger is a DAG³, not a single chain. Every vertex (a **Block** or a **Transaction**) has two kinds of backward edge: **parents** (links into the DAG for confirmation) and, for transactions, **inputs** (the outputs being spent). Blocks form a special spine through this graph — each block names the previous block, so the blocks alone make a linear chain. That chain is the **backbone**. Ordinary transactions hang off the backbone: each transaction is eventually confirmed by a specific block (its **first_block**). → full treatment in Ch. 8 & 9.

The generic version of this problem

Before we look at Hathor's answer, meet the problem in a setting with no blockchain in it at all, because the shape is older and more general than cryptocurrencies.

Imagine a database that is replicated across several servers. One replica has been offline for a week and now rejoins. It is behind. How does it catch up? Two classic approaches:

1. **Snapshot + log.** The lagging replica grabs a checkpoint (a full copy of the state as of some recent point) and then replays the log of every change that happened since. The log must be applied **in order** — you cannot apply "update row 5" before "insert row 5" exists.
2. **Find the divergence point, then stream forward.** Both replicas agree on how far back their histories are identical (the last common entry), and the leader streams every entry after that point, in order, to the follower.

Hathor's sync is approach 2, adapted to a DAG. The "find the divergence point" step becomes **find the common ancestor block**. The "stream entries in order" step becomes **stream the block backbone in height order, then stream the transactions each block confirms**. The reason ordering matters is the same as in the database case: an entry cannot be applied before the entries it depends on.

The five concerns sync must handle, stated plainly:

1. **Agree on a protocol.** Two peers might support different sync versions; they must pick one both understand.
2. **Find the boundary.** Determine the highest point in history where our ledger and the peer's ledger agree — the common ancestor. Below it, we are identical; above it, we are missing data.
3. **Download in dependency order.** Get the missing vertices such that every dependency arrives before the thing that needs it.
4. **Verify and ingest each one.** Run every received vertex through the same validation → consensus → store pipeline as a freshly-broadcast transaction.

5. **Keep up with the present.** Once caught up to the peer's confirmed history, also fetch the unconfirmed transactions sitting in the peer's mempool, and stay subscribed so new vertices arrive in real time.

The rest of the chapter walks each of these in the code.

35.3 The concepts it rests on

Three ideas from earlier chapters carry the weight of this one. Here they are in the local context, with pointers back.

Recap — the height index (full treatment in Ch. 28). The `height index` maps a block height (an integer: 0, 1, 2, ...) to the hash of the block at that height on the best chain. It is what lets the node answer "what is my block at height 5000?" in one lookup instead of walking the chain. Sync leans on it constantly: to compare "the peer's block at height H" against "my block at height H" when hunting for the common ancestor. In the code you will see `self.tx_storage.indexes.height.get(h)` — that is this index. → full treatment in Ch. 28.

Recap — vertex ingestion (full treatment in Ch. 33). When a vertex arrives, it does not go straight to disk. It flows through the **vertex handler**, which runs three stages: **verify** (Ch. 31 — is it well-formed, signatures valid, no double-spend?), **consensus** (Ch. 32 — does it change which history is canonical? does it void anything?), and **store + index** (Ch. 27-28). Sync never bypasses this. Every block and transaction it downloads is handed to `vertex_handler.on_new_block(...)` or `vertex_handler.on_new_mempool_transaction(...)`, the same entry points a freshly-broadcast vertex uses. → full treatment in Ch. 33.

Recap — the reactor and LoopingCall (full treatment in Ch. 16 & 23). Sync is asynchronous: it sends a request, then yields control back to the Twisted reactor⁴ and waits for the reply to arrive as a network event, rather than blocking. The sync loop is driven by a `LoopingCall` — a Twisted timer that re-invokes a method every N seconds. The `@inlineCallbacks` decorator⁵ lets the agent's methods read like straight-line code (`yield self.run_sync_blocks()`) while actually pausing for I/O. → full treatment in Ch. 16 & 23.

With those in hand, we can read the code.

35.4 The code, walked

35.4.1 Negotiating a sync version

Before any ledger data moves, the two peers must agree on a protocol. Hathor models the available protocols as a small enum, `SyncVersion`, in

`hathor/p2p/sync_version.py:20`:

```
@total_ordering
class SyncVersion(Enum):
    V1_1 = 'v1.1'
    V2 = 'v2'
```

There are two named values, but do not be misled: **only V2 is wired up**. The builder registers exactly one factory and enables exactly one version (`hathor/builder/builder.py:88-90`):

```
p2p_manager.add_sync_factory(SyncVersion.V2, sync_v2_factory)
if sync_v2_support == cls.ENABLED:
    p2p_manager.enable_sync_version(SyncVersion.V2)
```

`V1_1` lingers in the enum as a historical artifact — the old **sync-v1** protocol has been removed from the codebase, and no factory backs `V1_1` any longer. Any comment that mentions sync-v1 as a live alternative is stale; treat V2 as the only protocol that exists.

Why keep an enum with two members and an ordering at all, if only one is used? Because the negotiation mechanism is built to be version-agnostic, so that adding a future protocol is a matter of registering another factory, not rewriting the handshake. The negotiation works like this. During the HELLO handshake, each peer advertises the set of sync versions it has enabled (`hathor/p2p/states/hello.py:113`). When a peer receives the other's HELLO, it intersects the two sets and picks the best common one (`hathor/p2p/states/hello.py:122-130`):

```
common_sync_versions = my_sync_versions & remote_sync_versions
if not common_sync_versions:
    # no compatible sync version to use ... just can't connect to this peer
    protocol.send_error_and_close_connection('no compatible sync version to use')
    return

# choose the best version, sorting is implemented in hathor.p2p.sync_versions.__lt__
protocol.sync_version = max(common_sync_versions)
```

`max(...)` works because `SyncVersion` defines an ordering. `__lt__` (`sync_version.py:51`) compares the values returned by `get_priority()` — `V2` returns 20, `V1_1` returns 11 — so the higher-priority version wins when both are supported. If the two peers share no version, the connection is dropped; that is a normal outcome, not an error. The chosen version is stored on the connection as `protocol.sync_version`, and that single field decides which sync agent gets built later.

35.4.2 From version to agent: the factory

Once a connection reaches the READY state, the chosen version is turned into a concrete agent. The READY state looks up the registered factory for the negotiated version and asks it to create an agent (`hathor/p2p/states/ready.py:124-129`):

```
sync_version = self.protocol.sync_version
sync_factory = connections.get_sync_factory(sync_version)
self.sync_agent: SyncAgent = sync_factory.create_sync_agent(self.protocol, reactor=self.reactor)
self.cmd_map.update(self.sync_agent.get_cmd_dict())
```

This is the **factory pattern**⁷ you met in Chapter 3: `SyncV2Factory.create_sync_agent` (`hathor/p2p/sync_v2/factory.py:44`) returns a fresh `NodeBlockSync`, wired with the settings, the connection's protocol, the reactor, the vertex parser, and the vertex handler. One agent per connection. Two lines later, `get_cmd_dict()` registers the agent's message handlers into the connection's command map — that is how an incoming `BLOCKS` or `TRANSACTION` message gets routed to the right method. The agent is then started (`ready.py:144`), which kicks off its main loop.

35.4.3 The sync agent: `NodeBlockSync`

`NodeBlockSync` (`hathor/p2p/sync_v2/agent.py:86`) is the brain. Its docstring is honest about its strategy — "An algorithm to sync two peers based on their blockchain." Blocks first; that is the central design decision, and we will return to why in §35.5.

Its constructor sets up a lot of state (`agent.py:91`), but the fields worth holding in your head are few:

```
self.state = PeerState.UNKNOWN          # where we are in the sync state machine
self.synced_block: Optional[_HeightInfo] = None  # highest block we're synced at, w/ this peer
self.peer_best_block: Optional[_HeightInfo] = None # the peer's tip block
self.blk_streaming_client = None          # receives streamed blocks
self.tx_streaming_client = None           # receives streamed transactions
self.mempool_manager = SyncMempoolManager(self) # fetches unconfirmed txs
self._lc_run = LoopingCall(self.run_sync)      # the repeating sync loop
```

`PeerState` (`agent.py:78`) is the sync state machine — distinct from the connection state machine of Chapter 34. It has five members: `UNKNOWN`, `SYNCING_BLOCKS`, `SYNCING_TRANSACTIONS`, `SYNCING_MEMPOOL`, and `ERROR`. They name the phase the agent is in, and reading them in a log tells you exactly where a slow sync is stuck.

`_HeightInfo` (`agent.py:61`) is a tiny `NamedTuple` of `(height, id)` — a block's height paired with its hash. Almost all of the block-syncing logic is expressed in terms of `_HeightInfo`, because comparing "where am I" to "where is the peer" is fundamentally a comparison of (height, hash) pairs.

The agent's main loop is a `LoopingCall` that fires `run_sync` once per second (`agent.py:166`, interval at `agent.py:58`). Each tick is a step of the algorithm — it checks where we are and does the next bit of work. The actual logic is in `_run_sync` (`agent.py:330`):

```
@inlineCallbacks
def _run_sync(self) -> Generator[Any, Any, None]:
    is_block_synced = yield self.run_sync_blocks()
    if is_block_synced:
        # our blocks are synced, so sync the mempool
        yield self.run_sync_mempool()
```

Two phases, in strict order: **first sync the blocks** (the backbone, plus the transactions those blocks confirm), and **only when the blocks are fully caught up**, sync the mempool (the unconfirmed transactions). The mempool is never touched until the confirmed history is complete, because an unconfirmed transaction depends on confirmed ones — order again.

35.4.4 Are we even behind? The fast paths

`run_sync_blocks` (`agent.py:356`) opens by figuring out whether there is anything to do at all. It compares our best block to the peer's:

```
my_best_block = self.get_my_best_block()
self.peer_best_block = yield self.get_peer_best_block()

# Are we synced?
if self.peer_best_block == my_best_block:
    # Yes, we are synced! \o/
    self.update_synced(True)
    self.send_relay(enable=True)
    self.synced_block = self.peer_best_block
    return True
```

`get_peer_best_block` (`agent.py:820`) sends a `GET-BEST-BLOCK` message and waits (via a Twisted `Deferred` ⁶) for the peer to reply with `BEST-BLOCK`, carrying its tip block's hash and height. If the peer's tip equals ours, we are done — same best block, nothing to download. The agent flips to "synced," enables **relay** (asks the peer to forward new vertices in real time from now on), and returns `True`.

There is a second fast path (`agent.py:382`): if the peer's tip is at a height **less than or equal to** ours, and our block at the peer's height is exactly the peer's tip, then the peer is behind us on the same chain — again nothing to fetch from them. This is where the height index earns its keep:

```
common_block_hash = self.tx_storage.indexes.height.get(self.peer_best_block.height)
if common_block_hash == self.peer_best_block.id:
    # nothing to sync because peer is behind me at the same best blockchain
    return True
```

Only if neither fast path fires do we have real work: the peer holds blocks we lack. Now the hard part begins.

35.4.5 Finding the common ancestor

We know the peer is ahead, but we do **not** yet know where our histories diverge. We might share the first 4,000 blocks and differ after that; or a reorg⁸ might mean we agree only up to block 3,500. Downloading must start at the highest block we both have — the **common ancestor**. Streaming from anywhere lower wastes bandwidth re-downloading what we already hold; streaming from anywhere higher leaves a gap and the new blocks won't connect.

A linear scan ("do we share block 1? block 2? block 3? ...") would cost one round-trip per block — far too slow over a network. Instead `find_best_common_block` (`agent.py:557`) runs an **n-ary search** — a generalization of binary search that probes ten heights per round instead of one:

```
hi = min(peer_best_block, my_best_block, key=lambda x: x.height)
lo = _HeightInfo(height=0, id=self._settings.GENESIS_BLOCK_HASH)

while hi.height - lo.height > 1:
    step = math.ceil((hi.height - lo.height) / 10)
    heights = list(range(lo.height, hi.height, step))
    heights.append(hi.height)
    block_info_list = yield self.get_peer_block_hashes(heights)
    ...
```

Read the invariant carefully, because it is the heart of the method:

- `lo` is a height where we are **known to be synced** (it starts at genesis, height 0 — by definition both peers share genesis).
- `hi` is a height where the sync state is **unknown** (it starts at the lower of the two tips — no point searching above where the shorter chain ends).

Each round divides the gap `[lo, hi)` into ten slices and asks the peer, in one message (`GET-PEER-BLOCK-HASHES`, `agent.py:630`), for its block hashes at all eleven boundary heights. The peer replies with `(height, hash)` pairs for the blocks on its best chain at those heights. We then walk that list from highest to lowest and ask, for each: do I have this exact block?

```
for info in block_info_list:
    try:
        blk = self.tx_storage.get_transaction(info.id)
    except TransactionDoesNotExist:
        hi = info          # I don't have it → divergence is at or below here
    else:
        ...                # I do have it → I'm synced at least this high
        lo = info
        break
```

If we have the peer's block at some probed height, then we are synced at least that high — pull `lo` up to it. If we don't, the divergence is at or below that height — pull `hi` down to it. Each round shrinks the unknown gap by a factor of ten, so the search

converges in $\log_{10}(N)$ round-trips — for a chain of a million blocks, about six messages. When `hi - lo` reaches 1, `lo` is the highest block both peers share: the common ancestor. The method returns it.

One safety check is worth noting (`agent.py:589`): the peer's reply must include `lo` itself, because we believe we are synced there. If it does not, a reorg probably happened during the search and the ground shifted under us; the method returns `None` and the loop retries on the next tick. This is the kind of defensive handling that makes sync verbose but resilient — the network is a moving target.

35.4.6 Streaming the block backbone

With the common ancestor in hand, the agent asks the peer to stream every block from the ancestor up to the peer's tip, in order (`agent.py:531`):

```
def start_blockchain_streaming(self, start_block, end_block) -> Deferred[StreamEnd]:
    self._blk_streaming_client = BlockchainStreamingClient(self, start_block, end_block)
    quantity = self._blk_streaming_client._blk_max_quantity
    self.send_get_next_blocks(start_block.id, end_block.id, quantity)
    return self._blk_streaming_client.wait()
```

`send_get_next_blocks` (`agent.py:666`) sends a single `GET-NEXT-BLOCKS` message naming the start hash, the end hash, and how many blocks to send. The peer does **not** reply with one giant message. Instead it streams: it sends a sequence of `BLOCKS` messages, one block each, followed by a final `BLOCKS-END` marker. This is the protocol's defining choice — streaming, not request-per-block — and we will weigh it in §35.5.

On the peer's side, the streaming is run by `BlockchainStreamingServer` (`streamers.py:152`). It walks the best chain forward from the start block, pushing one block per reactor tick via `send_next` (`streamers.py:165`), advancing with `get_next_block_best_chain()` until it reaches the end hash, hits the streaming limit (`DEFAULT_STREAMING_LIMIT = 1000` , `streamers.py:33`), or finds that the chain it is sending has become voided⁹ mid-stream — each outcome is a distinct `StreamEnd` code (`streamers.py:36`). The server is a Twisted push producer, which means Twisted automatically pauses it if our connection's send buffer fills up and resumes it when the buffer drains — flow control for free.

On our side, each `BLOCKS` message lands in `handle_blocks` (`agent.py:762`), which deserializes the bytes back into a `Block` and hands it to the receiving client, `BlockchainStreamingClient.handle_blocks` (`blockchain_streaming_client.py:77`). That method does three things per block:

1. **Sanity-checks the count and linearity.** If more blocks arrive than were requested, or a block does not connect to the previously received one (`blk.get_block_parent_hash() != last_block.hash`), the stream fails (`blockchain_streaming_client.py:114-122`). This catches a misbehaving or buggy peer.

2. **Detects when we've caught up.** If a streamed block is one we already have and isn't voided, we've reached known territory; after enough repeats it stops the stream early (`blockchain_streaming_client.py:100-112`). No point re-downloading.
3. **Either ingests the block or sets it aside.** Here is the subtle part:

```
if self.tx_storage.can_validate_full(blk):
    self.vertex_handler.on_new_block(blk, deps=[])
else:
    self._partial_blocks.append(blk)
```

A block can be fully validated only if every transaction it depends on is already present. During a fresh sync that is usually **not** the case — the block confirms transactions we have not downloaded yet. So the client checks `can_validate_full`: if the block's dependencies are all here, ingest it immediately (`on_new_block` with an empty `deps` list). If not, park it in `_partial_blocks` — a list of blocks that are downloaded but cannot yet be fully validated, because their transactions are still missing. Those parked blocks drive the next phase.

35.4.7 Filling in the transaction DAG

Back in `run_sync_blocks` (`agent.py:422`), once the block stream finishes, the agent checks for parked blocks:

```
partial_blocks = self._blk_streaming_client._partial_blocks
if partial_blocks:
    self.state = PeerState.SYNCING_TRANSACTIONS
    reason = yield self.start_transactions_streaming(partial_blocks)
    while reason == StreamEnd.LIMIT_EXCEEDED:
        reason = yield self.resume_transactions_streaming()
```

Now the agent streams the transactions those blocks confirm.

`start_transactions_streaming` (`agent.py:858`) sends a `GET-TRANSACTIONS-BFS` message naming the first and last parked block, and a receiving client, `TransactionStreamingClient` (`transaction_streaming_client.py:42`), is created to consume the reply. The `while` loop handles the case where there are more transactions than one stream's limit allows: if the stream ends with `LIMIT_EXCEEDED`, it resumes from where it left off (`agent.py:874`) until everything is delivered.

The peer answers from `TransactionsStreamingServer` (`streamers.py:207`). Its docstring captures the strategy: "Streams all transactions confirmed by the given block, from right to left (decreasing timestamp)." For each parked block in turn, it runs a **breadth-first walk** of the DAG (`BFSOrderWalk`, `streamers.py:235`) starting from that block and walking backward through parents and inputs, emitting every transaction the block confirms. It skips transactions that belong to an earlier block (those were confirmed

before this block and will be delivered with that earlier block instead — `streamers.py:300`), so each transaction is sent exactly once, attributed to the first block that confirms it.

Why breadth-first, and why the `start_from` mechanism? Two reasons. First, BFS over the confirmation graph naturally produces transactions in an order close to dependency order. Second, the `GET-TRANSACTIONS-BFS` message carries a `start_from` list (`payloads.py:37`) so that a resumed stream can pick up from a precise frontier rather than restarting the whole walk — that is what makes the `LIMIT_EXCEEDED` resume loop above cheap.

On our side, the receiving client is careful about ordering, because BFS does not perfectly guarantee that a transaction's dependencies arrive before it does. The client maintains a small staging area (`transaction_streaming_client.py`):

- `_waiting_for` — the set of dependency hashes it still expects to receive.
- `_db` — transactions received but held back, waiting for their dependencies.

Each incoming `TRANSACTION` lands in `handle_transaction` (`agent.py:1016` → `transaction_streaming_client.py:123`), is queued, and processed one at a time by `process_queue` (`transaction_streaming_client.py:143`). For each transaction, `_process_transaction` (`transaction_streaming_client.py:166`):

1. Runs **basic verification** (`verify_basic`, `transaction_streaming_client.py:173`) — a cheap, dependency-free sanity check (structure, weight) before doing anything expensive. (Full validation comes later, at ingestion.)
2. Updates the dependency bookkeeping: looks at everything this transaction needs (`get_all_dependencies`) and sorts each into "already have it" or "still waiting" (`_update_dependencies`, `transaction_streaming_client.py:212`).
3. When `_waiting_for` becomes empty — every dependency for the current block is now in hand — it flushes the staged transactions and the block together, in timestamp order, by calling `_execute_and_prepare_next` (`transaction_streaming_client.py:239`):

```
blk = self.partial_blocks[self._idx]
vertex_list = list(self._db.values())
vertex_list.sort(key=lambda v: v.timestamp)
yield self.sync_agent.on_block_complete(blk, vertex_list)
```

`on_block_complete` (`agent.py:612`) is the payoff. It hands the block **and** its full set of confirmed transactions to the vertex handler at once:

```
yield self.vertex_handler.on_new_block(blk, deps=vertex_list)
```

Now the block can be fully validated, because `deps=vertex_list` carries exactly the transactions it was waiting on. The vertex handler runs them through verification → consensus → store (Chapter 33). The client then advances to the next parked block and repeats. Block by block, the backbone and its transaction DAG are committed to our ledger in dependency order.

35.4.8 The mempool sync

After the confirmed history is fully caught up, `_run_sync` proceeds to the mempool (`agent.py:343`). The **mempool**¹⁰ is the set of valid transactions a peer knows about that have been seen but not yet confirmed by any block. They are not part of the backbone — no block points at them yet — so they need their own fetch.

`SyncMempoolManager` (`mempool.py:30`) handles this. It first asks the peer for its mempool **tips** (`GET-TIPS`, the transactions at the frontier of the unconfirmed DAG), keeping only those we don't already have (`mempool.py:90-93`). Then, for each missing tip, it runs a **depth-first search** down the dependency graph (`_dfs`, `mempool.py:108`):

```
def _next_missing_dep(self, tx):
    for txin in tx.inputs:
        if not self.tx_storage.transaction_exists(txin.tx_id):
            return txin.tx_id
    for parent in tx.parents:
        if not self.tx_storage.transaction_exists(parent):
            return parent
    return None
```

For a given transaction, it finds the first dependency (an input or a parent) it does not yet have, downloads that first, and only adds the transaction itself once all its dependencies are present (`_add_tx`, `mempool.py:137`). This is dependency order enforced explicitly, one transaction at a time, via the DFS stack — the unconfirmed counterpart of what block streaming did for confirmed history. Each fully-resolved transaction is handed to `vertex_handler.on_new_mempool_transaction` (`mempool.py:143`) and, if accepted, re-broadcast to our other peers.

35.4.9 The message flow, end to end

Here is one full catch-up against a peer that is ahead of us, as a sequence diagram. Time flows downward; → is a message we send, ← is one we receive.



Once the agent reaches the bottom, it is synced with this peer and has enabled relay. From here, each new block or transaction the peer learns about is pushed to us live, re-entering the same ingestion pipeline. The `LoopingCall` keeps ticking; if either side advances, the agent steps through the relevant phases again. Sync is never truly "finished" — it is a standing relationship that keeps two ledgers aligned.

35.5 The design rationale

Two decisions define sync-v2. Each is a trade-off worth understanding, because the alternatives are not obviously wrong.

Why backbone-first?

The agent downloads **blocks before transactions**, always. Why not just fetch transactions in topological order directly and let the blocks fall out?

Because the blocks give the download a **spine to hang everything on**. The block chain is linear — each block names exactly one previous block — so "download the blocks" has an unambiguous order: by height, low to high. There is no ambiguity about what comes next. The transaction DAG, by contrast, is a tangle with no single linear order; trying to stream it directly, with no anchor, would mean constantly answering "where am I, and what should come next?" with graph traversals over an enormous structure.

By fetching the backbone first, sync converts a hard problem (order a giant DAG) into two easy ones: (1) order a linear chain — trivial, it's just heights; and (2) for each block, fetch the small bag of transactions that block confirms — a bounded, local problem. Each block acts as a checkpoint that says "every transaction confirmed at or below me is now accounted for." The block backbone is, in effect, a pre-built table of contents for the DAG. That is the deep reason this chapter's subtitle calls it "streaming the block backbone in order, then filling in the transaction DAG."

The common-ancestor search exists for the same reason: it finds where on the backbone to start, so we re-download nothing and leave no gaps.

Why streaming, not request-per-vertex?

Sync could have been built as a simple loop: "request block N, wait, receive it, request block N+1...". That is easier to reason about. Sync-v2 instead **streams**: one `GET-NEXT-BLOCKS` triggers a flood of `BLOCKS` messages with a single `BLOCKS-END` at the close.

The reason is **throughput**. A request-per-vertex design pays one full network round-trip per vertex. Over a link with 50 ms latency, that caps you at ~20 vertices per second no matter how fat the pipe — you spend almost all your time waiting for the next request to travel there and the reply to travel back. Streaming decouples sending from acknowledging: the peer keeps blocks flowing continuously, limited only by bandwidth, not by round-trip time. For catching up across millions of vertices, that difference is the difference between minutes and days.

Streaming brings its own costs, and the code pays them: flow control (the Twisted push-producer machinery in `streamers.py`, so a fast sender doesn't overrun a slow receiver), bounded buffers (the `DEFAULT_STREAMING_LIMIT` and the resume loop), the staging area in the transaction client (because streamed order isn't a perfect

dependency order), and a watchdog that kills a stream that has gone stale (`agent.py:296`). This is why sync is the most complex stateful subsystem in the networking layer — and it is honest to say so. The complexity buys throughput, and throughput is what makes a fresh node's catch-up finish in a reasonable time.

35.6 How it plugs into the lifecycle

Sync does not run in isolation; it sits at the meeting point of half the node. Tracing the connections, with chapter pointers:

- **It is attached at READY (Ch. 34).** A `NodeBlockSync` is created exactly when a peer connection reaches the READY state (`ready.py:127`) and started immediately (`ready.py:144`). One agent per connection; the node runs several at once, one per peer.
- **It runs on the reactor (Ch. 16 & 23).** The whole algorithm is timer-driven (`LoopingCall` , `agent.py:166`) and non-blocking (`@inlineCallbacks` + `Deferred` throughout). It never stalls the node; between messages, the reactor is free to serve every other peer and client.
- **Every received vertex goes through ingestion (Ch. 31-33).** Sync never writes to storage directly. A downloaded block reaches the ledger through `vertex_handler.on_new_block` (`agent.py:617` , `blockchain_streaming_client.py:131`); a downloaded mempool transaction through `vertex_handler.on_new_mempool_transaction` (`mempool.py:143`). That handler runs verification (Ch. 31) and consensus (Ch. 32) on each one. Sync's job is delivery in the right order; judging validity stays with the same pipeline that judges a freshly-broadcast transaction.
- **It leans on the height index (Ch. 28).** The fast-path check and the common-ancestor search both query `tx_storage.indexes.height.get(...)` to map a height to a best-chain block hash.
- **It serializes vertices with the wire codec (Ch. 26).** Streamed blocks and transactions cross the wire as the bespoke binary format; `handle_blocks` and `handle_transaction` call `vertex_parser.deserialize(...)` (`agent.py:773` , `agent.py:1023`) to turn bytes back into `Block` and `Transaction` objects.

Put differently: sync is the input pump for the entire ingestion machine. Chapter 33 told you what happens to a vertex once it is inside the node; this chapter told you how the vertices that aren't created locally get into the node in the first place — and in an order the rest of the machine can actually use.

Recap

Concern (§35.2)	Mechanism	Where in the code
Agree on a protocol	HELLO advertises versions; <code>max(common)</code> picks the best; only V2 is registered	<code>hello.py:122-130</code> , <code>builder.py:88-90</code>
Build the agent	Factory creates one <code>NodeBlockSync</code> per READY connection	<code>factory.py:44</code> , <code>ready.py:127</code>
Drive the algorithm	A 1-second <code>LoopingCall</code> steps the state machine; blocks then mempool	<code>agent.py:166</code> , <code>_run_sync</code> <code>agent.py:330</code>
Find the boundary	n-ary search over block heights → common ancestor	<code>find_best_common_block</code> <code>agent.py:557</code>
Stream the backbone	<code>GET-NEXT-BLOCKS</code> → streamed <code>BLOCKS</code> → <code>BLOCKS-END</code> , in height order	<code>streamers.py:152</code> , <code>blockchain_streaming_client.py:77</code>
Fill in the DAG	<code>GET-TRANSACTIONS-BFS</code> ; stage txs until deps complete, then ingest block+txs	<code>transaction_streaming_client.py:166</code> , <code>agent.py:612</code>
Sync the mempool	<code>GET-TIPS</code> then DFS each tip down its missing dependencies	<code>mempool.py:30</code> , <code>_dfs</code> <code>mempool.py:108</code>
Ingest each vertex	Hand to the vertex handler: verify → consensus → store	<code>on_new_block</code> / <code>on_new_mempool_transaction</code> (Ch. 33)

Sync-v2 is the bridge from "I booted with nothing" to "I hold and independently verify the whole ledger." Its one structural idea — **backbone first, DAG second** — turns the impossible-looking task of ordering a giant graph into the tractable one of walking a linear chain and, for each link, fetching the small set of transactions it confirms. Its one performance idea — **stream, don't poll** — is what makes that catch-up finish in minutes rather than days, at the cost of the stateful machinery that fills `agent.py` . Everything it downloads re-enters the node through the same verification and consensus pipeline as any other vertex, so a synced node is no more trusting of its peers than a node that mined every block itself.

The next chapter turns to a different network surface: `hathor/websocket`. Where this chapter was about peers exchanging ledger data, Chapter 36 is about **clients** — wallets and dashboards — subscribing to a live feed of events from the node. Same reactor, same vertices, but a one-way stream out to consumers rather than a two-way sync between equals.

1. The genesis is the hard-coded starting point of the ledger — the first block and initial transactions — that every node agrees on by definition. A fresh node holds only genesis until sync fills in the rest. ↩
2. A topological order of a directed acyclic graph is a linear ordering of its vertices such that every vertex comes after all the vertices it depends on (points to). Processing a DAG in topological order guarantees you never reach an item before its prerequisites. Full treatment of the DAG in Ch. 8. ↩
3. **DAG** = Directed Acyclic Graph: items ("vertices") joined by one-way links ("edges") with no way to follow the links in a loop. Hathor's ledger is a DAG of blocks and transactions. Ch. 8. ↩
4. The reactor is Twisted's central event loop: it waits for events (network data, timers) and calls the right piece of code in response, so the program never blocks on I/O. Ch. 16 & 23. ↩
5. `@inlineCallbacks` is a Twisted decorator that lets you write asynchronous code as if it were straight-line: `yield some_deferred` pauses the function until the result arrives, instead of nesting callbacks. Ch. 16. ↩
6. A Deferred is Twisted's name for a future — a placeholder for a value that will exist later (e.g. a peer's reply). You attach code to run when it resolves, rather than blocking to wait. Ch. 16. ↩
7. The factory pattern is a design pattern where a dedicated object's job is to construct other objects, hiding the details of which concrete class is built. `SyncV2Factory` builds a `NodeBlockSync` per connection. Ch. 3. ↩
8. A reorg (reorganization) happens when the network's view of the best chain changes — a competing chain becomes heavier and replaces part of the previously-best one. During sync, a reorg can shift the common ancestor mid-search. Ch. 10 & 32. ↩
9. A vertex is voided when consensus marks it as not part of canonical history (e.g. it was on a chain that lost a reorg, or it conflicts with a heavier transaction). Voided does not mean deleted — it means "recorded but not counted." Ch. 10 & 32. ↩
10. The mempool ("memory pool") is the set of valid transactions a node knows about that have been seen but not yet confirmed by any block. Ch. 28. ↩

Chapter 36 — The Streaming Surface: WebSockets

What you'll learn in this chapter

- What a **WebSocket** is, the request-response limitation it overcomes, and why a node needs to push data to clients rather than wait to be asked.
- How the **HTTP-upgrade handshake** turns an ordinary web request into a persistent, full-duplex channel — and why the node reuses its existing HTTP server to host it.
- How **Autobahn** layers a WebSocket implementation onto Twisted, and how `hathor-core` subclasses its factory and protocol classes.
- The **two distinct WebSocket surfaces** the node exposes — the admin stream (wallet/dashboard updates) and the event-queue stream (gap-free replay) — what each carries, and who consumes it.
- How clients **subscribe** to addresses, request history streams, and receive **JSON-framed** messages with rate limiting and flow control.
- Where the WebSocket factories are **built, mounted, and started** in the node's lifecycle.

Every chapter so far has been about the node talking to itself or to its peers: storage, verification, consensus, the P2P protocol. This chapter is about the node talking to **clients** — the wallets, dashboards, and downstream integrations that are not other full nodes but humans and applications wanting to watch the ledger as it changes. That requires a different kind of channel than the ones we have seen, and a different third-party library to build it. This is the canonical treatment of **WebSockets** and the **Autobahn** library in this book.

36.1 The problem: HTTP cannot push

You almost certainly know the shape of a normal web request. A client opens a connection, sends a request (`GET /balance/abc123`), the server sends a response, and — in the classic model — the conversation is over. The client asked; the server answered. This is the **request-response**¹ model, and it is the whole of plain HTTP.

That model has one property that matters enormously here: **the server can only speak when spoken to**. It has no way to start a sentence on its own. If a new block arrives and the server would like to tell an interested wallet "your balance just

changed," it cannot. It has no open channel to do so, and HTTP gives it no way to open one. The information flows in exactly one direction at a time, and only in reply to a request.

For a great many web pages this is fine — you click a link, you get a page. But a full node is a living system: blocks land, transactions get confirmed, balances shift, peers come and go, all continuously and unpredictably. A wallet UI wants to reflect those changes the instant they happen. A monitoring dashboard wants a live readout of the node's metrics. None of these clients knows when the next interesting thing will occur — so under pure HTTP they have only one option, and it is a bad one.

Polling, and why it is wasteful

The workaround under request-response is **polling**²: the client repeatedly asks "anything new yet? anything new yet?" on a timer — say every two seconds. Each poll is a fresh HTTP request: a new connection (or at least a new round-trip), headers sent and parsed, a handler invoked, a response built. Consider what this costs:

- **Latency.** If you poll every two seconds, a change can take up to two seconds to surface. Tighten the interval to feel responsive and you multiply the next cost.
- **Wasted work.** The overwhelming majority of polls return "nothing new." You pay the full price of a request-response round-trip to learn that nothing happened. Multiply by every connected client and every poll interval, and a node spends real CPU and bandwidth answering a question whose answer is almost always "no."
- **It scales the wrong way.** The busier you want the UI to feel, the faster you must poll, the more empty requests you generate. Responsiveness and efficiency pull in opposite directions.

Polling is the client simulating a live feed by asking very often. What we actually want is for the server to **push**³: to keep a channel open and send an update the moment one exists, with no request to trigger it. That is precisely what a WebSocket provides.

Full-duplex: both sides can talk at once

A telephone call is **full-duplex**⁴: both people can speak at the same time, and either can start talking whenever they like. A walkie-talkie is **half-duplex**: only one side transmits at a time, and you must release the button before the other can reply. Plain HTTP is closer to the walkie-talkie — one side speaks, then the other, strictly in turn, always begun by the client.

A **WebSocket** is the telephone call. Once established, it is a single, long-lived, **full-duplex** connection: the client can send messages to the server, the server can send messages to the client, **independently and at any time**, with no request needed to

unlock a response. The server can stay silent for an hour and then, the instant a block arrives, push a message — and the client receives it immediately, having asked for nothing. That is the capability the node's streaming surface is built on.

Recap — request-response vs. push. Under HTTP the server answers questions; it cannot raise its hand. Polling fakes a live feed by asking constantly, paying for a round-trip each time to usually learn nothing changed. A WebSocket replaces all of that with one persistent, full-duplex pipe the server can write to whenever it has news.

36.2 What a WebSocket actually is — the upgrade handshake

Here is the part that surprises people: a WebSocket connection starts its life as an ordinary HTTP request. It does not run on some separate, exotic port with its own listener. It begins as a normal `GET`, on the same HTTP server, and is then **upgraded** into a WebSocket. This is deliberate — it lets WebSockets travel through the same ports, proxies, and firewalls that already pass web traffic, and it lets a server host both plain HTTP endpoints and WebSocket endpoints side by side.

The mechanism is the HTTP **upgrade handshake**. The client sends a `GET` carrying two special headers:

```
GET /v1a/ws HTTP/1.1
Host: node.example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13
```

`Upgrade: websocket` and `Connection: Upgrade` are the request: "I would like to stop speaking HTTP on this connection and start speaking the WebSocket protocol." The server, if it agrees, answers with a special status code — `101 Switching Protocols` — and a confirming header:

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+x0o=
```

From that `101` onward, the rules change. The same TCP⁵ connection that carried the HTTP request stays open, but the two sides stop exchanging HTTP requests and responses and instead exchange WebSocket **frames**⁶ — small, framed messages that can flow either direction at any time. The handshake is the hinge: HTTP on one side of it, full-duplex WebSocket on the other, over one unbroken socket.

The `Sec-WebSocket-Key / Sec-WebSocket-Accept` exchange is a small handshake-validation ritual (the server hashes the client's key with a fixed value and echoes it back) that proves to the client it is genuinely talking to a WebSocket-aware server and not a confused cache. You will not have to implement any of this by hand — the library does it — but it is worth knowing the 101 and the two `Upgrade` headers are the whole trick. **An HTTP request walks in; a WebSocket walks out.**

This is also why, in the code we are about to read, the WebSocket lives as a child of the HTTP resource tree. The node already runs an HTTP server (its REST API); the WebSocket endpoint is mounted on a path of that same server, and the upgrade happens when a client hits that path with the `Upgrade` headers.

36.3 Localization

The streaming surface is split across **two** packages, because the node exposes two genuinely different WebSocket services. Keep them apart in your mind from the start; §36.7 explains why they are separate.

```
hathor/
├── websocket/
│   ├── __init__.py
│   ├── factory.py
│   ├── protocol.py
│   ├── resource.py
│   ├── messages.py
│   ├── streamer.py
│   ├── iterators.py
│   └── exception.py
└── event/
    ├── websocket/
    │   ├── __init__.py
    │   ├── factory.py
    │   ├── protocol.py
    │   ├── request.py
    │   └── response.py
```

- ← surface 1: the ADMIN / general stream
- ← exports factory, protocol, stats resource
- ← HathorAdminWebsocketFactory (the server)
- ← HathorAdminWebsocketProtocol (one per client)
- ← WebsocketStatsResource (an HTTP stats endpoint)
- ← Pydantic message types (JSON frame shapes)
- ← HistoryStreamer (push-producer for big history)
- ← address-search helpers for history streaming

- ← surface 2: the EVENT-QUEUE stream
- ← EventWebsocketFactory (the server)
- ← EventWebsocketProtocol (one per client)
- ← client→server request models
- ← server→client response models

There is a third, smaller WebSocket factory worth naming so it does not confuse you later — `MiningWebsocketFactory` in `hathor/mining/ws.py`, mounted at `/mining_ws` — but it belongs to the mining surface and is covered in Chapter 37. This chapter is about the two general-purpose streams above.

Context. The `hathor/websocket/` package is the node's live link to wallets and dashboards — it pushes balance changes, new accepted transactions, and periodic metrics, and it can stream an address's whole history on demand. The `hathor/event/websocket/` package is the live link to downstream data systems — it replays the node's durable event log (Chapter 30) in strict order with no gaps. Both ride on the node's existing HTTP server via Autobahn, and both are driven by the one Twisted reactor that drives everything else.

36.4 The concepts it rests on

This chapter leans on three things taught in full elsewhere. We re-establish each just enough to read the code, then point you back.

Recap — Twisted, the reactor, factories and protocols (full treatment in Ch. 16).

Twisted is the asynchronous-networking framework the whole node runs on. Its centre is the **reactor**: one event loop that waits for things to happen (data arriving, a timer firing) and calls your code in response, never blocking. In Twisted's model, a **Protocol** object represents one connection and holds its per-connection state, while a **Factory** is the long-lived object that manufactures a fresh protocol instance for each new connection and holds state shared across all of them. You will see exactly this split below: one factory per WebSocket service, one protocol instance per connected client. `LoopingCall` (a repeating timer) and `reactor.callLater` (a one-shot delayed call) also reappear here. → Ch. 16.

Recap — the event system this streams (full treatment in Ch. 30). Chapter 30 described two layers. **PubSub** is the node's in-process announcement bus: components `publish` events like `NETWORK_NEW_TX_ACCEPTED` or `WALLET_BALANCE_UPDATED`, and any component can `subscribe` to be called back when one fires. On top sits the **EventManager**, which (when the event queue is enabled) persists a durable, gap-free, sequentially-numbered log of public events to RocksDB. The `admin` WebSocket of this chapter is a **subscriber on the PubSub bus**; the event-queue WebSocket is a **reader of the EventManager's durable log**. → Ch. 30.

Recap — publish/subscribe (full treatment in Ch. 3). Pub-sub decouples the announcer from the listeners: a publisher fires an event without knowing or caring who is listening, and subscribers register interest by event type. The `admin` factory's `subscribe()` method (below) is a textbook subscriber registration — it tells PubSub "call me whenever any of these ten event types fire." → Ch. 3.

36.5 Autobahn: a WebSocket implementation for Twisted

A WebSocket is a protocol — a precise specification of the handshake, the frame format, the masking rules, ping/pong keep-alives, and close codes. You do not want to implement that by hand. **Autobahn**⁷ is the library that implements it, and crucially it ships a Twisted-native binding. That last word is why `hathor-core` chose it: the node is already a Twisted program, so a WebSocket library that speaks Twisted's Factory/Protocol vocabulary slots straight in with no impedance mismatch.

What it is and the problem it solves

- **What it is.** Autobahn is a mature Python library implementing the WebSocket protocol (and the WAMP messaging protocol on top, which the node does not use). The relevant piece is `autobahn.twisted.websocket`.
- **The problem it solves.** It handles the entire WebSocket mechanism — the upgrade handshake, framing, masking, fragmentation, ping/pong, close handshakes — so the application only deals with whole messages. Without it the node would be parsing frame headers and computing `Sec-WebSocket-Accept` hashes by hand.
- **Its core abstractions.** Two classes mirror Twisted's own Factory/Protocol pair: `WebSocketServerFactory` and `WebSocketServerProtocol`. You subclass them. And one Twisted-web adapter: `WebSocketResource`, which makes a WebSocket factory mountable on an HTTP resource tree so the upgrade can happen on an existing HTTP path.

How you use it — a tiny echo server

Before the real code, the smallest possible Autobahn-on-Twisted server. It accepts WebSocket connections and echoes back whatever it receives — the "hello world" of WebSockets. Read it for the shape, not the details:

```

from autobahn.twisted.websocket import WebSocketServerFactory, WebSocketServerProtocol
from twisted.internet import reactor

class EchoProtocol(WebSocketServerProtocol):
    def onConnect(self, request):
        print("client connecting:", request.peer)    # handshake starting

    def onOpen(self):
        print("connection open")                    # handshake done, full-duplex now

    def onMessage(self, payload, isBinary):
        self.sendMessage(payload, isBinary)          # push it straight back

    def onClose(self, wasClean, code, reason):
        print("connection closed:", reason)

factory = WebSocketServerFactory()
factory.protocol = EchoProtocol                    # factory makes one EchoProtocol per client
reactor.listenTCP(9000, factory)
reactor.run()

```

Note the four lifecycle hooks Autobahn calls on the protocol — `onConnect` (handshake beginning), `onOpen` (handshake finished, channel live), `onMessage` (a frame arrived), `onClose` (connection ended) — and the one method you call to push data, `sendMessage`. Every real protocol in this chapter overrides exactly these hooks. The factory-makes-a-protocol-per-connection pattern is identical to Twisted's, because Autobahn's classes are Twisted Factory/Protocol subclasses. Hold this toy in mind; the real code is this skeleton with the bodies filled in.

36.6 Surface 1 — the admin WebSocket, walked

This is the wallet-and-dashboard stream. Its job: keep connected clients live-updated about wallet-relevant events and node metrics, and serve on-demand history streams.

The factory: one server, many clients

`HathorAdminWebsocketFactory` subclasses Autobahn's `WebSocketServerFactory` (`hathor/websocket/factory.py:77`). It binds the protocol class and tracks every live connection:

```

class HathorAdminWebsocketFactory(WebSocketServerFactory):
    protocol = HathorAdminWebsocketProtocol    # factory.py:81

    def buildProtocol(self, addr):              # factory.py:86
        return self.protocol(self, is_history_streaming_enabled=self.is_history_streaming_enabled

```

`buildProtocol` is the Twisted hook that mints one protocol object per incoming connection — the toy above relied on the default; here it is overridden so each protocol receives a back-reference to the factory and a flag. In its `__init__` (`factory.py:89`) the factory holds the pieces of shared state that all connections need:

```

self.connections: set[HathorAdminWebsocketProtocol] = set() # factory.py:101
self.address_connections: defaultdict[str, set[...]] = defaultdict(set) # factory.py:107
self.rate_limiter = RateLimiter(reactor=self.reactor) # factory.py:111
self._lc_send_metrics = LoopingCall(self._send_metrics) # factory.py:123

```

Why each of these exists is worth pausing on, because the factory's whole job is captured here:

- **connections** — the set of all clients that have finished handshaking. To broadcast (send the same message to everyone), the factory iterates this set. Note it holds only fully-open connections; a connection that drops mid-handshake never enters it.
- **address_connections** — a map from a wallet address to the set of clients that asked to watch that specific address. Some events are not broadcast to everyone — they are relevant only to whoever subscribed to the address involved. This map makes targeted delivery possible.
- **rate_limiter** — a guard against flooding a client. Under a burst of activity the node could try to push hundreds of messages a second; the limiter caps the rate per message type and buffers the overflow (more below).
- **_lc_send_metrics** — a `LoopingCall`, i.e. a repeating timer, that fires `_send_metrics` on a fixed interval to push a dashboard snapshot. This is server-initiated push in its purest form: no client asked, the timer fired, the data goes out.

Subscribing to the event bus

The factory is a PubSub subscriber. When the node builds it, `subscribe()` registers the factory's `handle_publish` callback for ten event types (`factory.py:166`):

```

def subscribe(self, pubsub): # factory.py:166
    events = [
        HathorEvents.NETWORK_NEW_TX_ACCEPTED,
        HathorEvents.WALLET_OUTPUT_RECEIVED,
        HathorEvents.WALLET_INPUT_SPENT,
        HathorEvents.WALLET_BALANCE_UPDATED,
        # ... ten in total
    ]
    for event in events:
        pubsub.subscribe(event, self.handle_publish)

```

From now on, whenever any of those events is published anywhere in the node, `handle_publish` runs (`factory.py:185`). It calls `serialize_message_data` to turn the event's Python arguments into a JSON-friendly `dict` (`factory.py:193` — e.g. for an accepted transaction it builds `tx.to_json_extended()`), stamps the message with a `type` field, and hands it to `send_or_enqueue`. **This is the push pathway end to end:** a block lands → consensus publishes `NETWORK_NEW_TX_ACCEPTED` on PubSub → the factory's callback fires → the data is serialized to JSON → it is sent to every connected client. No client polled for it.

Broadcast vs. targeted send

`send_message` decides the audience (`factory.py:252`):

```
def send_message(self, data):                                # factory.py:252
    if data['type'] in ADDRESS_EVENTS:                        # e.g. WALLET_ADDRESS_HISTORY
        if data['address'] in self.address_connections:
            self.execute_send(data, self.address_connections[data['address']])
        else:
            self.broadcast_message(data)
```

Address-scoped events (`ADDRESS_EVENTS` , `factory.py:70`) go only to the clients that subscribed to that address — that is the point of the `address_connections` map. Everything else is broadcast to all. The actual send is `execute_send` (`factory.py:228`): it serializes the dict to bytes with `json_dumps`, then loops the target connections calling `c.sendMessage(payload, False)` — Autobahn's push primitive, the same `sendMessage` from the toy. The `False` means "this is a text frame, not binary." It swallows `Disconnected` (the client vanished) so one dead client cannot break the broadcast to the rest.

Rate limiting: protecting the client from a firehose

`send_or_enqueue` (`factory.py:262`) sits in front of `send_message` for the four high-volume event types listed in `CONTROLLED_TYPES` (`factory.py:42`). The logic: if a buffer is already pending, or if this hit would exceed the configured rate, the message is enqueued in a bounded `deque` instead of sent immediately (`enqueue_for_later` , `factory.py:282`), and a `reactor.callLater` is scheduled to drain the buffer later (`process_deque` , `factory.py:299`). The deque is bounded (`maxlen`), so under sustained overload the oldest throttled messages are silently dropped rather than letting memory grow without limit. Each enqueued message is tagged `throttled: True` so the client can tell it was delayed. This is back-pressure: when the producer outruns what a client should reasonably receive, the node degrades gracefully rather than drowning the client or itself.

The metrics timer

`_send_metrics` (`factory.py:153`) is the `LoopingCall` target. It assembles a snapshot — transaction and block counts, best-block height, hash rate, connected peers — tags it `type: 'dashboard:metrics'`, and broadcasts it. Started in `start()` (`factory.py:126`) with `self._lc_send_metrics.start(settings.WS_SEND_METRICS_INTERVAL, now=False)`, it is the node's heartbeat to every dashboard, on a timer, with no prompting.

The protocol: one client's state machine

`HathorAdminWebsocketProtocol` subclasses `WebSocketServerProtocol` (`hathor/websocket/protocol.py:41`). One instance exists per connected client, and it implements the same four Autobahn hooks as the toy:

```

def onOpen(self):
    self.factory.on_client_open(self)
    self.send_capabilities()

def onClose(self, wasClean, code, reason):
    self.factory.on_client_close(self)

def onMessage(self, payload, isBinary):
    message = json_loadb(payload)
    _type = message.get('type')
    if _type == 'ping':
        self._handle_ping(message)
    elif _type == 'subscribe_address':
        self.factory._handle_subscribe_address(self, message)
    elif _type == 'request:history:xpub':
        self._open_history_xpub_streamer(message)
    # ... dispatch on the message's "type" field

```

`onMessage` is a **dispatch table** keyed on the incoming JSON's `type` field (`protocol.py:120`) — the client-to-server half of the conversation. A client can:

- send `{"type": "ping"}` and get `{"type": "pong"}` back (a liveness check the client initiates);
- send `{"type": "subscribe_address", "address": "..."}` to start receiving events for one address — handled by `_handle_subscribe_address` on the factory (`factory.py:317`), which enforces per-connection subscription limits (`WS_MAX_SUBS_ADDRS_CONN` , `WS_MAX_SUBS_ADDRS_EMPTY`) and replies with a success/failure frame;
- request a **history stream** of an address's whole transaction history, by xpub⁸ or by an explicit address list.

JSON message framing, typed

Every frame is JSON. The outbound message shapes are not hand-built dicts everywhere — for the streaming protocol they are Pydantic models in `hathor/websocket/messages.py` , each with a `Literal` `type` discriminator:

```

class CapabilitiesMessage(WebSocketMessage):
    type: Literal['capabilities'] = 'capabilities'
    capabilities: list[str]

class StreamVertexMessage(StreamBase):
    type: Literal['stream:history:vertex'] = 'stream:history:vertex'
    id: str
    seq: int
    data: dict[str, Any]

```

Using Pydantic models (Chapter 18) for the frames means the message shape is validated and self-documenting, and `message.json_dumpb()` produces the bytes to push (`protocol.py:346`). The `type` field is the contract: the client switches on it exactly as the server switches on the inbound `type` .

The history streamer: pushing a large result safely

Streaming an address's entire history could be huge — far more than fits in one frame, and enough to either block the reactor or overwhelm a slow client if dumped at once.

`HistoryStreamer` (`hathor/websocket/streamer.py:59`) solves both. It is a Twisted **push-producer** (`@implementer(IPushProducer)`, `streamer.py:58`) with its own small state machine (`StreamerState`, `streamer.py:37`: `NOT_STARTED` → `ACTIVE` → `PAUSED/CLOSING` → `CLOSED`) and **flow control** by sequence numbers and acknowledgements:

- It sends framed messages — `stream:history:begin`, then interleaved `stream:history:address` and `stream:history:vertex` frames, then `stream:history:end` — each carrying an incrementing `seq` number.
- The client periodically sends `{"type": "request:history:ack", "ack": N}`. The streamer tracks the last acked seq (`set_ack`, `streamer.py:131`) and a **sliding window**: it will not run more than `window_size` messages ahead of the last ack. If the client falls behind, the streamer pauses; when an ack arrives, it resumes (`resume_if_possible`, `streamer.py:164`).

This is the same windowed flow-control idea TCP uses, implemented at the application layer: the fast server is kept in step with the slow client by waiting for acknowledgements, so a client on a thin connection is never buried and the reactor is never monopolized by one greedy stream.

36.7 Surface 2 — the event-queue WebSocket

The second surface lives in `hathor/event/websocket/` and exists for a different consumer with a different need. Where the admin stream gives wallets a live, lossy-tolerant feed of current events, the event-queue stream gives downstream data systems a **durable, gap-free, exactly-ordered replay** of the node's entire public event history (Chapter 30). Think of an indexer or analytics pipeline that must process every event the node ever emitted, in order, surviving restarts and slow consumers. That demands guarantees the broadcast admin stream deliberately does not make.

`EventWebsocketFactory` (`hathor/event/websocket/factory.py:31`) and `EventWebsocketProtocol` (`hathor/event/websocket/protocol.py:34`) are again an Autobahn factory/protocol pair, but the contract is reversed: instead of the server pushing on a whim, the client drives consumption with explicit windowed requests. The client speaks three request types (`hathor/event/websocket/request.py`):

- `START_STREAM` (with `last_ack_event_id` and `window_size`) — "begin sending me events, starting after this ID, and never run more than `window_size` ahead of my acks."
- `ACK` (with `ack_event_id` and `window_size`) — "I've processed up to here; you may advance, and here's my current window."

- `STOP_STREAM`.

These are Pydantic models behind a discriminated union (`request.py:57`), parsed in `onMessage` (`protocol.py:93`) and dispatched by a `match` statement (`protocol.py:104`). The crucial guarantee is in `can_receive_event` (`protocol.py:58`): the factory will send an event to a connection **only if** the stream is active, the event is exactly the next expected one (`event_id == self.next_expected_event_id()`), and the number of unacknowledged events is below the window. Events are therefore delivered **strictly in order, one gap-free sequence, throttled by the client's acks** — no broadcast, no rate-limit-and-drop. Where the admin stream may discard throttled messages, the event stream may never skip one.

Delivery is recursive and reactor-friendly: `send_next_event_to_connection` (`factory.py:105`) sends one event, then schedules itself again via `self._reactor.callLater(0, ...)` (`factory.py:117`). The `callLater(0)` yields back to the reactor between sends so a long replay does not block the event loop — it streams the backlog one event per reactor turn until the client's window is full or the backlog is exhausted. Live events arrive via `broadcast_event` (`factory.py:84`), which the EventManager calls as new events are persisted (`hathor/event/event_manager.py:196` and `:374`).

Why two surfaces? Different guarantees for different consumers. The **admin** stream optimizes for liveness for many lightweight UI clients: broadcast, best-effort, rate-limited, drop-on-overload. The **event-queue** stream optimizes for completeness for a few heavyweight data clients: per-client cursor, in-order, gap-free, ack-driven, never-drop. Trying to serve both needs from one mechanism would force one consumer to accept the wrong trade-off, so the node runs two.

36.8 Why WebSockets, and why Autobahn

The chapter's technology choices, against their alternatives.

WebSocket vs. HTTP polling. Covered in §36.1: polling pays a full round-trip per check to usually learn nothing, trades latency against waste, and scales backwards. A WebSocket replaces N empty requests with one open pipe that carries data only when there is data. For a continuously-changing system watched by live UIs, this is the difference between a responsive feed and a wasteful approximation of one.

WebSocket vs. Server-Sent Events (SSE). SSE⁹ is a lighter standard for server-to-client push over a plain HTTP response that is held open. It is simpler and is enough when only the server needs to push and the client never sends anything back. But the node's streams are genuinely **bidirectional**: clients subscribe to addresses, request

history streams, send acks, adjust their flow-control window, ping. That two-way conversation is exactly what SSE cannot do and a WebSocket can. The interactivity is what rules SSE out.

Autobahn vs. rolling your own. The WebSocket protocol is fiddly — framing, masking, fragmentation, ping/pong, close codes. Hand-rolling it would be error-prone and pointless. The real choice is which library, and there the deciding factor is the rest of the stack: the node is a Twisted program, and Autobahn is the mature library with a first-class **Twisted-native binding** (`autobahn.twisted.websocket`). It speaks Twisted's Factory/Protocol vocabulary and plugs into Twisted's web resource tree via `WebSocketResource` (next section). An asyncio-based WebSocket library (e.g. `websockets`) would force an awkward bridge into the reactor; Autobahn needs none. The choice falls directly out of the Twisted decision made back in Chapter 16.

36.9 How it plugs into the lifecycle

The wiring lives in the composition root's resource builder (`hathor/builder/resources_builder.py`), the same place the REST API tree is assembled (Chapter 24). The connecting fact from §36.2 is visible here: the WebSocket is mounted as a child of the HTTP resource tree, using Autobahn's `WebSocketResource` adapter.

```
from autobahn.twisted.resource import WebSocketResource      # resources_builder.py:19

# admin websocket, mounted at /ws on the HTTP server
ws_factory = HathorAdminWebSocketFactory(manager=self.manager,
                                           metrics=self.manager.metrics,
                                           address_index=self.manager.tx_storage.indexes.addresses)
root.putChild(b'ws', WebSocketResource(ws_factory))          # resources_builder.py:323
ws_factory.subscribe(self.manager.pubsub)                  # :330 ← becomes a PubSub subscribe

# event-queue websocket, only if the event queue is enabled
if self._args.x_enable_event_queue or self._args.enable_event_queue: # :333
    root.putChild(b'event_ws', WebSocketResource(self.event_ws_factory)) # :334

self.manager.websocket_factory = ws_factory                # :348 ← manager will start it
```

Three things to read out of this:

1. `WebSocketResource(ws_factory)` is the adapter from §36.2 made concrete: it wraps the Autobahn factory in something Twisted-web can mount with `putChild`. A `GET /v1a/ws` carrying the `Upgrade` headers hits this child, and Autobahn performs the `101` handshake — the upgrade happens on the existing HTTP server. The admin stream is always mounted (when the status/HTTP server is enabled at all); the event-queue stream is mounted **only** when `--enable-event-queue` (or its experimental `--x-` form) is passed, because it requires the durable event log to be running.

2. `ws_factory.subscribe(self.manager.pubsub)` is where the admin factory joins the PubSub bus (§36.6) — from this line on, ledger events flow to connected clients.
3. `self.manager.websocket_factory = ws_factory` hands the factory to the manager so the manager can start it at the right moment. That moment is `HathorManager.start()` (`hathor/manager.py:321`), right after components initialize and just before metrics:

```
if self.websocket_factory:                # manager.py:321
    self.websocket_factory.start()        # starts the rate limiter + the metrics LoopingCall
```

Symmetrically, `stop()` halts it (`manager.py:362`). The event-queue factory is started instead by the EventManager (`hathor/event/event_manager.py:114`, `self._event_ws_factory.start(stream_id=...)`), since its lifecycle is bound to the durable event log it serves.

The end-to-end picture, then: at build time the factories are constructed and mounted on the HTTP tree; the admin one subscribes to PubSub; at `manager.start()` the admin factory's timers begin; a client connects to `/v1a/ws`, Autobahn upgrades the connection, and from there every relevant ledger event the node publishes is pushed to that client over the open full-duplex channel — all of it running on the one Twisted reactor (Chapter 16) that also drives storage, consensus, and the P2P layer.

Recap

Surface	Package	Server / client classes	What it streams	Delivery model	Audience
Admin / general	<code>hathor/websocket/</code>	<code>HathorAdminWebsocketFactory</code> / <code>HathorAdminWebsocketProtocol</code>	accepted txs, wallet balance/output/input events, periodic dashboard metrics, on-demand address history	broadcast or address-targeted; rate-limited, drops on overload	wallets, dashboards (many light clients)

Surface	Package	Server / client classes	What it streams	Delivery model	Audience
Event-queue	hathor/ event/ websocket/	EventWebSocketFactory / EventWebSocketProtocol	the durable, numbered public event log (Ch 30)	per-client cursor, strictly in-order, gap-free, ack-windowed, never drops	indexers, analytics (few heavy clients)
Mining	hathor/ mining/ ws.py	MiningWebSocketFactory	mining-related updates	—	miners → Ch 37

Both general surfaces are Autobahn factory/protocol pairs subclassed onto Twisted's reactor; both are mounted as children of the existing HTTP server via `WebSocketResource`, so a plain `GET` with `Upgrade` headers is promoted to a full-duplex WebSocket by the `101` handshake; both push JSON-framed messages the moment the node has something to say. The split between them is the chapter's central lesson: **liveness for UIs, completeness for data pipelines, are different problems, and the node solves each with a stream tuned to it.** Everything in this chapter has been about the node serving clients — the fifth full-node responsibility from Chapter 0. The next chapter turns to how the node produces the blocks those clients are watching: **Chapter 37 — Mining**, where we meet block templates, the CPU miner, and the canonical treatment of the Stratum protocol.

1. **Request-response** is the interaction model of plain HTTP: the client sends a request and the server sends back exactly one response, then the exchange is finished. The server cannot initiate; it can only reply. ↩
2. **Polling** is a client repeatedly asking the server "is there anything new?" on a timer, to approximate a live feed. Each poll is a full request even when the answer is "no," which makes it wasteful — most polls return nothing. ↩
3. **Server push** means the server sends data to the client without the client having requested that specific data, using a connection kept open for the purpose. It is the inverse of polling. ↩
4. **Full-duplex** describes a channel on which both ends can transmit at the same time and either can start at any moment (like a phone call). **Half-duplex** allows only one direction at a time (like a walkie-talkie). Plain HTTP is effectively half-duplex and always client-initiated; a WebSocket is full-duplex. ↩
5. **TCP** (Transmission Control Protocol) is the reliable, ordered byte-stream transport that both HTTP and WebSockets run on top of. A WebSocket reuses the very TCP connection that carried the initial HTTP request — the upgrade does not open a new socket. ↩

6. A **frame** is the unit of data in the WebSocket protocol: a small message with a tiny header indicating its type (text, binary, ping, pong, close) and length. After the handshake, the two sides exchange frames instead of HTTP requests/responses. ↩
7. **Autobahn** is a Python library implementing the WebSocket protocol (and the higher-level WAMP protocol, unused here). `hathor-core` uses its Twisted binding, `autobahn.twisted.websocket`, whose `WebSocketServerFactory` / `WebSocketServerProtocol` mirror Twisted's own Factory/Protocol pair. ↩
8. An **xpub** (extended public key) is a single public key from which a whole sequence of wallet addresses can be derived without exposing any private key. The history streamer can walk an xpub's derived addresses to stream a wallet's entire history. ↩
9. **Server-Sent Events (SSE)** is a simpler push standard where the server holds an HTTP response open and writes events to it over time. It is one-directional (server→client only), which is why it cannot serve a stream where the client must also subscribe, ack, and adjust flow control. ↩

Chapter 37 — Mining: Templates, CPU Miner & Stratum

What you'll learn in this chapter

- What a **block template** is — the well-formed candidate block the node hands a miner — and exactly what goes into one and why (parents, reward, weight).
- How the node's **CPU mining service** solves proof-of-work by grinding a nonce, and why that grinding must stay off the reactor.
- The **Stratum protocol**: the standard, long-lived connection between mining hardware and a node, built on a **job → share** model. This is Stratum's canonical treatment in the book.
- **Merged mining**: mining Hathor as a side-effect of mining Bitcoin, using an auxiliary proof-of-work.
- How a solved block re-enters the node — `submit_block` → the vertex handler (Ch. 33) → consensus (Ch. 32) — closing the loop from "found a block" to "ledger changed."

Mining is where new coins are minted and where the chain's security comes from. We have already met the theory — what proof-of-work is, what weight measures, why "most work wins" (Chapters 6 and 9). This chapter is the plumbing: the three packages that turn that theory into a running service. `hathor/mining/` builds the candidate block and can solve it on a CPU. `hathor/stratum/` is the network protocol that lets a real miner — a separate program, possibly on dedicated hardware — talk to the node. `hathor/merged_mining/` lets a Bitcoin miner mine Hathor for free on the side.

We start with the concept that everything here orbits: the template.

37.1 Localization

Three sibling packages, plus a few CLI entry points that drive them:

```

hathor-core/
├── hathor/
│   ├── mining/
│   │   ├── __init__.py           ← exports BlockTemplate, BlockTemplates
│   │   ├── block_template.py     ← BlockTemplate (NamedTuple) + BlockTemplates (list)
│   │   ├── cpu_mining_service.py ← CpuMiningService: grinds the nonce on a CPU
│   │   └── ws.py                 ← websocket helper for mining (not covered here)
│   ├── stratum/
│   │   ├── __init__.py
│   │   ├── stratum.py           ← JSONRPC, StratumProtocol, StratumFactory, StratumClient
│   │   └── resources.py         ← HTTP resource exposing miner statistics
│   └── merged_mining/
│       ├── coordinator.py       ← MergedMiningCoordinator, MAGIC_NUMBER, aux-pow build
│       ├── bitcoin.py           ← Bitcoin block/merkle primitives
│       └── bitcoin_rpc.py       ← talks to a bitcoind over JSON-RPC
└── hathor_cli/
    ├── mining.py                ← `hathor-cli mining`: standalone HTTP-polling CPU miner
    ├── stratum_mining.py        ← `hathor-cli stratum_mining`: standalone Stratum CPU mine
    └── merged_mining.py         ← `hathor-cli merged_mining`: the merge-mining coordinator

```

The node itself does not normally mine. In production a node serves work (via Stratum) and accepts solutions; the actual nonce-grinding happens in a separate miner process — often the standalone CLI tools above, or third-party mining software. The `CpuMiningService` inside the node exists mostly for tests, the simulator, and the standalone `hathor-cli mining` tool. Keep that split in mind: **the node hands out work and validates results; miners do the work.**

Context. Mining sits at the edge of the node's monetary and security responsibilities (Ch. 0 §0.2, jobs 3 and 5). It produces the blocks that mint HTR and that consensus (Ch. 32) uses to order the DAG. A found block is not special once it arrives — it flows through the same ingestion pipeline (Ch. 33) as any vertex received from a peer. Mining is therefore best understood as a producer feeding the front door we already studied.

37.2 What it does and why it exists

A miner needs two things from the node, and the node needs one thing back.

The miner needs, first, a **candidate block** to work on — a block that is already valid in every respect except that its proof-of-work is not yet solved. It must already point at the right parents, already pay the correct reward, already carry the right difficulty. The

only blank left for the miner to fill is the **nonce**¹ (and, optionally, the timestamp). If the node handed out a half-formed block, every miner would have to re-derive the chain's current state for itself — defeating the point of a full node.

The miner needs, second, to be told how hard the puzzle is — the **weight** (Hathor's difficulty unit; recap below). Without a target, "solve the proof-of-work" has no finish line.

The node needs, in return, to **accept a solved block** and fold it into the ledger: check the proof-of-work is genuine, check it still extends the current best tip, then push it through verification and consensus.

The package that bundles the first two — the candidate block plus its difficulty plus the surrounding constraints — into one object is the **block template**. Producing templates, solving them, and accepting solutions is the whole job of this chapter.

Recap — mining secures the chain and mints coins (full treatment in Ch. 6 §6.6 and Ch. 9). A blockchain stays honest because rewriting history means redoing proof-of-work, which costs real electricity (Ch. 6). Each block carries a **weight** — a number measuring how much work it represents — and consensus prefers the history with the most accumulated weight (Ch. 9, Ch. 32). Mining is simultaneously how that work gets done and how new HTR enters circulation, as the block's reward output. → full treatment in Ch. 6 and Ch. 9.

37.3 The concepts it rests on

Three ideas from earlier chapters carry the weight here. We recap each briefly; none is re-taught from scratch.

Recap — proof-of-work, weight, nonce, target (full treatment in Ch. 9). Every Hathor vertex has a **weight**: a float that is the base-2 logarithm of the expected number of hash attempts needed to solve it. A weight of w means roughly 2^w tries. From the weight the code derives a numeric **target**: a solution is valid when the block's hash, read as a big integer, is **below** the target. The miner varies one field — the **nonce**, a counter — recomputes the hash, and checks. Larger weight $\Rightarrow$ smaller target $\Rightarrow$ more tries. → full treatment in Ch. 9.

Recap — never block the reactor; use a thread or process pool (full treatment in Ch. 2 and Ch. 16). The node runs on a single-threaded event loop, the Twisted reactor. Any function that runs for a long time without returning freezes the whole node — no peers served, no timers fired. Grinding billions of hashes is exactly such a long-running CPU job. So CPU mining never happens inside the node's reactor: it runs in a separate process (the standalone CLI miners) or is offloaded to a worker. The reactor only handles the fast, I/O-bound parts: sending a job, receiving a share. → full treatment in Ch. 2 and Ch. 16.

Recap — a block is a vertex (full treatment in Ch. 8 and Ch. 25). A `Block` is one of Hathor's two vertex types. Like any vertex it has parents (graph edges) and a weight; unlike a transaction it has no inputs, and its single output mints the reward. A Hathor block points at three parents: the previous block plus, usually, two transactions it confirms. → full treatment in Ch. 8 and Ch. 25.

With those in hand, the code becomes readable.

37.4 The code, walked

We build the picture in the order data flows: first the template (what work looks like), then solving it (the CPU service), then the protocol that distributes work to real miners (Stratum), then the merged-mining variant, and finally the path a solved block takes back into the node.

37.4.1 The block template — what goes in, and why

`BlockTemplate` is a `NamedTuple`² — an immutable record with named fields — defined at `hathor/mining/block_template.py:29`. Read its fields as a contract: "here is everything you need to build a mineable block, and nothing you need to ask the chain for again."

```
class BlockTemplate(NamedTuple):
    versions: set[int]
    reward: int          # reward unit value, 64.00 HTR is 6400
    weight: float         # calculated from the DAA
    timestamp_now: int    # the reference timestamp the template was generated for
    timestamp_min: int    # min valid timestamp
    timestamp_max: int    # max valid timestamp
    parents: list[bytes]  # required parents: always a block, at most 2 txs
    parents_any: list[bytes] # extra parents to choose from when there are options
    height: int           # metadata
    score: int            # metadata
    signal_bits: int      # signal bits for blocks generated from this template
```

`hathor/mining/block_template.py:29`

Each field models something concrete:

- **reward** is how many HTR the new block mints, in the smallest unit (the comment notes `64.00 HTR is 6400` — two implied decimals). It is fixed by the block's height, because Hathor's issuance schedule (halvings) is a function of height. The miner cannot choose it; a wrong reward would make the block invalid.
- **weight** is the difficulty, computed by the **DAA**³ (difficulty-adjustment algorithm, Ch. 9). It sets the finish line. The template fixes a minimum weight so a miner cannot hand back a block that is too easy to be worth anything.
- **parents** and **parents_any** together encode the block's graph edges. A Hathor block always has exactly three parents. The first is mandatory — the previous block (the tip being extended). The remaining slots are filled with transactions the block will confirm. **parents** holds the ones that must be included; **parents_any** is a menu the miner picks the rest from when there is more than one valid choice. (Why split them? So two miners building on the same tip can still produce different blocks by choosing different transactions to confirm — useful diversity.)
- **timestamp_min** / **timestamp_max** / **timestamp_now** bound the block's timestamp. A block cannot be older than its parents or too far in the future; the template states the legal window so the miner's clock cannot drift out of bounds.
- **signal_bits** carries the miner's feature-activation vote (Ch. 38). The template pre-fills the bits the node wants signalled.
- **height** and **score** are metadata: the new block's height (parent height + 1) and its accumulated score (parent score + this block's work). The miner does not need them to mine, but the node uses them.

The template knows how to turn itself into an actual block. `generate_mining_block` (`block_template.py:53`) fills the blanks — picks random parents from the menu, attaches the reward output paying the miner's address, sets the weight and timestamp — and returns a `Block` ready to grind:

```
def generate_mining_block(self, rng, address=None, timestamp=None, ...):
    parents = list(self.get_random_parents(rng))
    block_timestamp = min(max(base_timestamp, self.timestamp_min), self.timestamp_max)
    tx_outputs = []
    if self.reward:
        output_script = create_output_script(address) if address is not None else b''
        tx_outputs = [TxOutput(self.reward, output_script)]
    block = cls(outputs=tx_outputs, parents=parents, timestamp=block_timestamp,
               weight=self.weight, signal_bits=self.signal_bits, ...)
    return block
```

`hathor/mining/block_template.py:53`

Note `get_random_parents` (`block_template.py:87`): it always returns exactly three parents — the required ones plus a random sample from `parents_any` to fill the gap — and asserts that length. The "three parents" invariant of a Hathor block is enforced right here.

`BlockTemplates` (`block_template.py:130`) is a thin `list` subclass holding one or more templates (one per competing best tip) plus the storage handle.

`choose_random_template` (`:136`) picks one. In normal operation there is a single best tip, so the list has one entry.

37.4.2 How the node builds a template

The node-side entry point is `HathorManager.make_block_template` (`hathor/manager.py:666`). It loads the parent block, gathers candidate parent transactions, then delegates to `_make_block_template` (`manager.py:696`), which is where the interesting decisions live.

The reward comes from the difficulty algorithm by height:

```
height = parent_block.get_height() + 1
...
reward=self.daa.get_tokens_issued_per_block(height),
```

`hathor/manager.py:742, :757`

The weight is the larger of "what the DAA says the next block should be" and a computed floor that guarantees the block actually moves the chain's score forward:

```
min_significant_weight = calculate_min_significant_weight(
    parent_block_metadata.score, 2 * self._settings.WEIGHT_TOL)
weight = max(
    self.daa.calculate_next_weight(parent_block, timestamp, self.tx_storage.get_parent_block),
    min_significant_weight,
)
```

`hathor/manager.py:734`

`calculate_next_weight` is the DAA from Chapter 9 — it tunes difficulty so blocks arrive at a steady rate. The `min_significant_weight` floor exists because a block whose weight is too small would not raise the accumulated score enough to count; the node refuses to hand out such trivial work.

The parents are assembled from the parent block plus the gathered transactions, then packed so the template carries either three fixed parents or a fixed set plus a menu:

```

parents = [parent_block.hash] + list(parent_txs.must_include)
parents_any = parent_txs.can_include
if len(parents) + len(parents_any) == 3:
    parents.extend(sorted(parents_any))
    parents_any = []
assert 1 <= len(parents) <= 3, 'Impossible number of parents'

```

hathor/manager.py:743

Finally the template is assembled with both block versions a miner may use (a regular block or a merge-mined block) and the feature-signal bits:

```

return BlockTemplate(
    versions={TxVersion.REGULAR_BLOCK.value, TxVersion.MERGE_MINED_BLOCK.value},
    reward=self.daa.get_tokens_issued_per_block(height),
    weight=weight,
    ...
    signal_bits=self._bit_signaling_service.generate_signal_bits(block=parent_block),
)

```

hathor/manager.py:755

`get_block_templates` (`manager.py:650`) wraps this and is the method mining APIs and the Stratum server actually call. Its docstring notes it is a "cached version... cache is invalidated when latest_timestamp changes" — templates are reused until the chain state moves, so the node is not rebuilding a template for every miner poll.

37.4.3 The CPU mining service — solving the puzzle

Once you have a block, mining it is a loop: set a nonce, hash, check against the target, increment, repeat. That loop lives in `CpuMiningService` (`hathor/mining/cpu_mining_service.py:25`). The heart is `start_mining` :

```

def start_mining(vertex, *, start=0, end=MAX_NONCE, ..., should_stop=lambda: False):
    pow_part1 = vertex.calculate_hash1()
    target = vertex.get_target()
    vertex.nonce = start
    while vertex.nonce < end:
        ...
        result = vertex.calculate_hash2(pow_part1.copy())
        if int(result.hex(), vertex.HEX_BASE) < target:
            return result
        vertex.nonce += 1
    return None

```

hathor/mining/cpu_mining_service.py:50

Three things to notice. The hash is split into two halves — `calculate_hash1` computes the part that does not depend on the nonce, once, and `calculate_hash2` finishes the part that does, every iteration. That is a real optimization: the nonce-independent prefix is hashed a single time and copied (`pow_part1.copy()`) per attempt. The win condition is the target comparison from Chapter 9: hash-as-integer below target. And

`MAX_NONCE = 2**32 ( :22 )` bounds the search; if the loop exhausts the nonce space without a hit, it bumps the timestamp (`update_time` , every 2 seconds) and restarts, because changing the timestamp changes the hash and reopens the search.

`resolve ( :26 )` wraps `start_mining` , stores the found hash on the vertex, and handles the special case of a token-creation transaction whose token id is its own hash.

This service is injected into the manager (`manager.py:112` , stored at `:199`) so collaborators have a single mining implementation to call. Block mining in production is external — real miners do that work over Stratum. But the in-node service is not idle: it solves the small proof-of-work on **user transactions**. Every Hathor transaction carries a (small, 4-byte-nonce) proof-of-work as anti-spam, so when a user submits a transaction through the wallet's REST API the node grinds that nonce for them — for example in `send_tokens ( hathor/wallet/resources/send_tokens.py:134 )` and the thin-wallet endpoint (`hathor/wallet/resources/thin_wallet/send_tokens.py:267`). That grind is offloaded to a dedicated thread pool so it never blocks the reactor:

```
self.pow_thread_pool = ThreadPool(minthreads=0, maxthreads=settings.MAX_POW_THREADS,
                                  name='Pow thread pool')
```

`hathor/manager.py:234` (started at `:310`); the thin-wallet endpoint dispatches into it via `deferToThreadPool` at `wallet/resources/thin_wallet/send_tokens.py:182`

The same service also mines the genesis vertices at startup (`hathor/transaction/genesis.py:71`). For block mining specifically, where you see the service is the standalone CLI miner. `hathor_cli/mining.py` is a small program that polls the node's HTTP mining endpoint, mines whatever block it gets back, and POSTs the solution:

```
def worker(q_in, q_out):
    from hathor.mining.cpu_mining_service import CpuMiningService
    block, start, end, sleep_seconds = q_in.get()
    CpuMiningService().start_mining(block, start=start, end=end, sleep_seconds=sleep_seconds)
    q_out.put(block)
```

`hathor_cli/mining.py:35`

It runs the grind in a separate `Process` (`mining.py:125`), never inside a reactor — exactly the "don't block the loop" rule from the recap. This HTTP-polling style is the naive approach we contrast against Stratum in §37.5.

37.4.4 Stratum — the standard miner-to-node protocol

The CLI HTTP miner works, but it is wasteful: it asks for a block, mines it, asks again. Between requests there is no live connection; if the chain tip moves, the miner keeps grinding stale work until its next poll. Real mining wants a persistent connection over which the node pushes fresh work the instant it changes, and the miner streams back partial progress. That protocol is **Stratum**.

A generic mining pool, first

Before the Hathor code, hold the generic picture — it is identical across every cryptocurrency that uses Stratum (Bitcoin included).

Imagine a pool operator coordinating many miners. The operator keeps an open TCP connection to each miner. The exchange is:

1. The miner **subscribes** — "I'm here, here's the address to pay me."
2. The operator sends a **job**: "here is the work; here is its difficulty."
3. The miner grinds. Each time it finds a hash that beats the job's (deliberately easy) difficulty, it submits a **share** — proof it is working.
4. The operator checks the share. Most shares are not full blocks; they only prove effort. But occasionally a share also beats the network's (much harder) difficulty — that share **is a block**, and the operator broadcasts it.
5. Whenever the work goes stale (a new block appears on the network), the operator pushes a **fresh job**, and the miner abandons the old one.

Two difficulties, then: the easy **share difficulty** (tuned so each miner submits a share every few seconds, giving a steady signal of its hash-rate) and the hard **network difficulty** (the real bar for a block). This job → share model is the entire conceptual core. Everything in `stratum.py` is this picture in code.

Stratum in Hathor: the layers

The protocol is JSON-RPC 2.0⁴ over a line-delimited TCP connection. The class stack reflects that:

- **JSONRPC** (`hathor/stratum/stratum.py:181`) — a Twisted `LineReceiver` that parses each newline-terminated line as a JSON-RPC message and dispatches it to `handle_request`, `handle_result`, or `handle_error`. It is the shared transport for both server and client.
- **StratumProtocol** (`stratum.py:337`) — the server side, one instance per connected miner. It holds that miner's jobs, address, and statistics. This is where subscribe and submit are handled.
- **StratumFactory** (`stratum.py:733`) — the Twisted factory⁵ that builds a `StratumProtocol` per connection and holds shared state (all miner protocols, the transaction-mining queue). It also subscribes to the node's events so it knows when to push fresh jobs.
- **StratumClient** (`stratum.py:813`) — the miner side, used by `hathor-cli` `stratum_mining`. It spawns worker processes to grind and submits shares.

`StratumProtocol.handle_request` (`stratum.py:404`) is the dispatcher. It accepts exactly two methods — `subscribe` and `submit` (with `mining.` prefixes accepted too) — and rejects anything else. If the node is still syncing it answers `NODE_SYNCING` rather than hand out work on an incomplete chain (`:418`).

Subscribe → first job

When a miner subscribes, `handle_subscribe` (`stratum.py:441`) decodes the payout address, records whether the miner wants to mine transactions or do merged mining, registers the protocol in the factory's `miner_protocols`, answers `ok`, and immediately calls `job_request` to send the first job:

```
self.factory.miner_protocols[self.miner_id] = self
self.send_result('ok', msgid)
self.subscribed = True
self.job_request()
```

`hathor/stratum/stratum.py:472`

`create_job` (`stratum.py:619`) builds a `ServerJob`: it generates the work (a block, or a queued transaction), assigns a UUID, computes the share weight for this miner, and sets the job's weight to the smaller of the share weight and the real work's weight:

```
share_weight = self.calculate_share_weight()
job.weight = min(share_weight, tx.weight)
```

`hathor/stratum/stratum.py:639`

This is the two-difficulties idea in code. `calculate_share_weight` (`stratum.py:693`) looks at how fast the miner has been solving recent jobs and picks a weight aimed at one share every `AVERAGE_JOB_TIME` (5 seconds, `:348`). A fast miner gets harder shares; a slow one gets easier shares — so every miner reports in at a steady cadence regardless of hardware. The job also gets a timeout (`:648`): if the miner does not submit within the window, the server pushes a fresh job rather than letting it grind stale work.

The actual work is built by `create_job_tx` (`stratum.py:656`). Blocks are the default; transactions are prioritized only if the miner opted into `mine_txs`:

```
block = self.manager.generate_mining_block(
    data=data, address=self.miner_address, merge_mined=self.merged_mining)
```

`hathor/stratum/stratum.py:681`

That call lands in `HathorManager.generate_mining_block` (`manager.py:769`), which uses the template machinery from §37.4.2. The loop is closed: subscribe → server asks the manager for a template → fills it into a block → ships it as a job.

The job sent to the miner carries only what it needs: the mining header without the nonce, the job id, the nonce size, and the (share) weight:

```

job_data = {
    'data': job.tx.get_mining_header_without_nonce().hex(),
    'job_id': job.id.hex,
    'nonce_size': job.tx.SERIALIZATION_NONCE_SIZE,
    'weight': float(job.weight),
}

```

hathor/stratum/stratum.py:608

`SERIALIZATION_NONCE_SIZE` differs by vertex type — a block's nonce is 16 bytes (`hathor/transaction/block.py:46`), a transaction's is 4 (`hathor/transaction/transaction.py:58`). The 4-byte tx nonce is why mining a transaction uses a short 1-second job timeout (`TX_MAXIMUM_JOB_TIME` , `stratum.py:351`): a 4-byte search space is small enough to exhaust quickly, so the timestamp must be refreshed often.

Submit → checking the share, finding a block

When the miner finds a hash that beats the job weight, it submits the nonce.

`handle_submit` (`stratum.py:479`) is the most consequential method in the file. Walk it:

1. **Validate the request** — `job_id` and `nonce` present, job id a real UUID (: 492).
2. **Find the job** — look it up; reject `JOB_NOT_FOUND` if unknown (:504).
3. **Reject stale shares** — if this is not the current job or it was already submitted, answer `STALE_JOB` (:512). This guards against a miner submitting work for a job the chain has already moved past.
4. **Reconstruct and re-hash** the block with the submitted nonce (:518 – :528).
5. **Verify the share** — re-run proof-of-work against the job's (easy) weight:

```

try:
    verifier.verify_pow(tx, override_weight=job.weight)
except PowError:
    self.log.error('bad share, discard', ...)
    return self.send_error(INVALID_SOLUTION, msgid, ...)

```

hathor/stratum/stratum.py:539

A failing share means the miner lied about doing the work — discard it. A passing share proves effort. The server marks the job submitted, answers `ok`, and **immediately schedules the next job** so the miner never idles:

```

self.send_result('ok', msgid)
self.manager.reactor.callLater(0, self.job_request)

```

hathor/stratum/stratum.py:552

1. **Is the share also a block?** Re-run proof-of-work, this time against the real network weight:


```

try:
    verifier.verify_pow(tx)          # no override: real difficulty
except PowError:
    self.log.info('high hash, keep mining', tx=tx)
    return
else:
    self.log.info('low hash, new block candidate', tx=tx)

```

hathor/stratum/stratum.py:555

Most shares fail this second check — they proved effort but are not blocks. The miner keeps going. But when a share passes, it is a genuine block, and the server submits it to the node:

```

if isinstance(tx, Block):
    try:
        self.manager.submit_block(tx)
        self.blocks_found += 1
    except (InvalidNewTransaction, TxValidationError) as e:
        self.log.warn('block propagation failed', block=tx, error=e)

```

hathor/stratum/stratum.py:564

That `submit_block` call is the bridge back into the node, covered in §37.4.6.

Pushing fresh work when the tip moves

The last piece is staying current. When a new block is accepted anywhere — found locally or arriving from a peer — every miner's current job becomes stale.

`StratumFactory.start` (`stratum.py:773`) subscribes to the node's pub-sub bus (Ch. 30) for exactly this:

```

def start(self):
    def on_new_block(event, args):
        tx = args.__dict__['tx']
        if isinstance(tx, Block):
            self.update_jobs()
    self.manager.pubsub.subscribe(HathorEvents.NETWORK_NEW_TX_ACCEPTED, on_new_block)

```

hathor/stratum/stratum.py:773

`update_jobs` (`stratum.py:765`) calls `job_request` on every subscribed miner, pushing each a fresh template built on the new tip. This event-driven push — rather than the miner polling — is the efficiency that Stratum buys, and it is why the factory must be started (subscribed) during the node's boot.

37.4.5 Merged mining — Hathor as a free side-effect of Bitcoin

Merged mining lets a miner secure two chains with one set of hashes. The idea: Bitcoin's proof-of-work hashes a Bitcoin block header; if you commit a Hathor block's hash inside that Bitcoin block (in its coinbase transaction), then a Bitcoin solution that happens to also clear Hathor's difficulty is simultaneously a valid Hathor block. Hathor rides Bitcoin's hash power for free.

The proof that "this Bitcoin work also solved this Hathor block" is an **auxiliary proof-of-work** (aux-pow). In Hathor it is `BitcoinAuxPow` (`hathor/transaction/aux_pow.py:24`), a record of the Bitcoin header pieces and the **Merkle path**⁶ needed to show the Hathor block's hash was committed in Bitcoin's coinbase:

```
class BitcoinAuxPow(NamedTuple):
    header_head: bytes      # 36 bytes
    coinbase_head: bytes    # variable
    coinbase_tail: bytes    # variable
    merkle_path: list[bytes] # each element 32 bytes
    header_tail: bytes      # 12 bytes
```

`hathor/transaction/aux_pow.py:24`

`calculate_hash` (`aux_pow.py:38`) reconstructs the Bitcoin header hash from those pieces; that hash is the merge-mined block's hash. `verify` (`:46`) checks two things: that Hathor's `MAGIC_NUMBER` (`b'Hath'`, `hathor/merged_mining/coordinator.py:55`) appears in the coinbase at the expected place — proving the commitment is to Hathor and not slipped in elsewhere — and that the Merkle path is not abusively long.

The coordinator that orchestrates this is `MergedMiningCoordinator` (`hathor/merged_mining/coordinator.py:1124`), run as the standalone `hathor-cli merged_mining` service. It is an asyncio server (not Twisted — this is a self-contained tool) that talks to a `bitcoind` over JSON-RPC and to a Hathor node, builds a combined job, and on a winning share constructs the aux-pow and forwards it to Hathor:

```
def build_aux_pow(self, work):
    bitcoin_header, coinbase_tx = self._make_bitcoin_block_and_coinbase(work)
    ...
    coinbase_head, coinbase_tail = coinbase.split(block_base_hash)
    return BitcoinAuxPow(header_head, coinbase_head, coinbase_tail,
                          list(self.merkle_path), header_tail)
```

`hathor/merged_mining/coordinator.py:230`

On the node side, the merged path rejoins the ordinary one: a `MergeMinedBlock` carrying an `aux_pow` is submitted through the same Stratum `handle_submit` (the `aux_pow` branch at `stratum.py:522`) and the same `submit_block`. Merged mining is a different way to produce the proof-of-work, not a different way to accept a block.

37.4.6 The solved block, back into the node

Whatever produced it — Stratum, the CLI miner, merged mining — a found block re-enters through `HathorManager.submit_block` (`hathor/manager.py:796`). It does two checks, then hands off:

```
def submit_block(self, blk):
    parent_hash = blk.get_block_parent_hash()
    best_block_hash = self.tx_storage.get_best_block_hash()
    if parent_hash != best_block_hash:
        self.log.warn('submit_block(): Ignoring block: parent not a tip', ...)
        return False
    ...
    return self.propagate_tx(blk)
```

`hathor/manager.py:796`

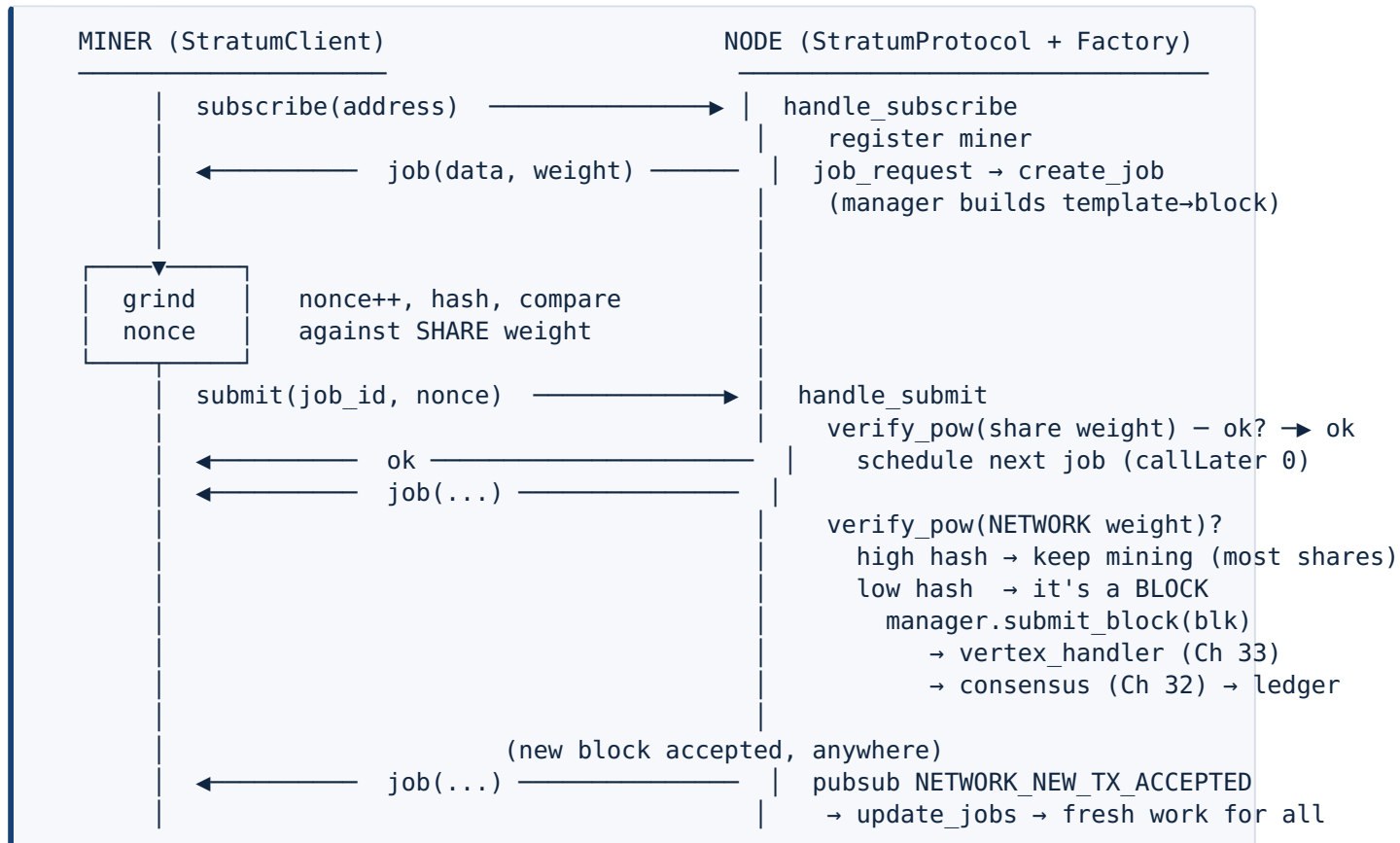
First it rejects a block whose parent is no longer the best tip — stale work that lost the race. Then it warns (but accepts) a block whose weight is below the significance floor. Finally it calls `propagate_tx` (`manager.py:851`), which routes into `on_new_tx` (`:864`) and from there to the vertex handler:

```
success = self.vertex_handler.on_new_relayed_vertex(vertex, ...)
if propagate_to_peers and success:
    self.connections.send_tx_to_peers(vertex)
```

`hathor/manager.py:878`

From here the block is just a vertex. `on_new_relayed_vertex` (Ch. 33) runs it through full verification (Ch. 31), consensus (Ch. 32), storage and indexing (Ch. 27–28), and finally re-broadcasts it to peers. The miner's block has become part of the ledger, and — through the pub-sub event in §37.4.4 — every other connected miner is immediately handed fresh work built on top of it. The loop closes.

37.4.7 The job/share lifecycle, end to end



37.5 Why Stratum, and not naive polling

The standalone `hathor-cli mining` tool (§37.4.3) shows the simple alternative: **getwork-style HTTP polling**. The miner sends an HTTP request, gets a block back, mines it to completion, then sends another request. Why does production use Stratum instead?

Staleness. With polling there is no live link. If a new block lands on the network the instant after a miner fetched work, that miner grinds doomed, already-stale work until its next poll — wasted electricity. Stratum's factory subscribes to the node's "new block accepted" event and pushes fresh jobs the moment the tip moves (§37.4.4). Wasted work shrinks to the round-trip latency.

Granularity and hash-rate measurement. Polling only reveals a miner's effort when it finds a whole block — which, for a single machine against network difficulty, may be never. Stratum's share model asks the miner to submit partial solutions against an easy, per-miner difficulty, tuned so each miner reports roughly every 5 seconds (`AVERAGE_JOB_TIME` , `stratum.py:348`). The node continuously sees each miner is alive and how fast it hashes (`estimated_hash_rate` , `stratum.py:714`) — essential for pool accounting, and impossible with bare polling.

Connection cost. Polling opens a fresh TCP connection (and re-fetches state) every cycle. Stratum holds one long-lived connection per miner, amortizing setup and letting the node push rather than the miner pull.

A standard, not a bespoke API. Stratum is the de-facto protocol the whole mining ecosystem speaks (the docstring at `stratum.py:341` cites the Bitcoin Stratum references). By implementing it, Hathor lets existing mining software — written for Bitcoin and other chains — point at a Hathor node with minimal change. A bespoke HTTP scheme would force every miner vendor to write Hathor-specific code. Interoperability is the deciding trade-off: Hathor adopts the ecosystem's protocol rather than inventing its own.

The cost of Stratum is complexity — a stateful server, per-miner job history, share-difficulty tuning, timeouts. For a protocol that must serve real hardware competing for real money, that complexity pays for itself. For a one-off test block, the polling miner is perfectly adequate, which is why both exist.

37.6 How it plugs into the lifecycle

Mining is wired in at three points along the node's life (Ch. 29):

- **Build time.** If `--stratum` was passed, the builder constructs the `StratumFactory` and attaches it to the manager (`hathor_cli/builder.py:374` , and `hathor/builder/builder.py:449` for the test-side `Builder`). The `CpuMiningService` is always injected into the manager (`hathor/manager.py:112`), whether or not Stratum is enabled.
- **Listen time.** `run_node` opens the Stratum TCP port if `--stratum` is set, pointing the listener at the factory (`hathor_cli/run_node.py:226`).
- **Start time.** `HathorManager.start` (`manager.py:336`) calls `stratum_factory.start()` , which subscribes the factory to the pub-sub bus so new blocks trigger fresh jobs. On stop (`manager.py:371`), the factory is torn down.

CPU mining, by contrast, never runs inside the node's reactor. It runs in separate processes — the CLI tools' worker `Process` es (`hathor_cli/mining.py:125` , `hathor_cli/stratum_mining.py`) — keeping the long grind off the event loop, exactly as the Chapter 2 / Chapter 16 rule demands.

And every solved block, by whatever route, funnels through `submit_block` → `on_new_relayed_vertex` → the verification → consensus → storage pipeline of Chapters 31–33. Mining produces; the rest of the node ingests.

Recap

Concept	Where it lives	Central type / method
The candidate block (work)	<code>hathor/mining/block_template.py:29</code>	<code>BlockTemplate</code> (NamedTuple)
Filling a template into a block	<code>block_template.py:53</code>	<code>BlockTemplate.generate_mining_block</code>
Node builds the template	<code>hathor/manager.py:696</code> , <code>:666</code>	<code>_make_block_template</code> , <code>make_block_template</code>
Reward & weight decisions	<code>manager.py:734</code> , <code>:742</code>	DAA: <code>calculate_next_weight</code> , <code>get_tokens_issued_per_block</code>
Solving proof-of-work on a CPU	<code>hathor/mining/cpu_mining_service.py:50</code>	<code>CpuMiningService.start_mining</code>
Standalone polling CPU miner	<code>hathor_cli/mining.py:35</code>	<code>worker</code> → <code>CpuMiningService</code>
Stratum transport (JSON-RPC)	<code>hathor/stratum/stratum.py:181</code>	<code>JSONRPC</code> (LineReceiver)
Stratum server (per miner)	<code>stratum.py:337</code>	<code>StratumProtocol</code>
Subscribe → first job	<code>stratum.py:441</code> , <code>:619</code>	<code>handle_subscribe</code> , <code>create_job</code>
Share check → block detection	<code>stratum.py:479</code>	<code>handle_submit</code> (two <code>verify_pow</code> calls)
Per-miner share difficulty	<code>stratum.py:693</code>	<code>calculate_share_weight</code>
Push fresh work on new tip	<code>stratum.py:773</code> , <code>:765</code>	<code>StratumFactory.start</code> , <code>update_jobs</code>

Concept	Where it lives	Central type / method
Standalone Stratum CPU miner	<code>hathor/stratum/stratum.py:813</code>	<code>StratumClient</code>
Merged mining coordinator	<code>hathor/merged_mining/coordinator.py:1124</code>	<code>MergedMiningCoordinator</code>
Auxiliary proof-of-work	<code>hathor/transaction/aux_pow.py:24</code>	<code>BitcoinAuxPow</code>
Solved block back into the node	<code>hathor/manager.py:796</code>	<code>submit_block</code> → <code>propagate_tx</code>

Mining is the node's monetary edge: it produces the blocks that mint HTR and that consensus uses to order the DAG. The node's role is narrow and well-bounded — build a correct **template** (parents, reward, weight; §37.4.1–2), hand it out as Stratum **jobs**, accept **shares** as proof of effort, and recognize the rare share that is also a **block** (§37.4.4). The heavy nonce-grinding is deliberately kept outside the reactor, in separate processes or external hardware (§37.3, §37.6). Whatever produces a block — local CPU, Stratum hardware, or Bitcoin merged mining (§37.4.5) — every solution re-enters through `submit_block` and travels the same ingestion pipeline as any peer-relayed vertex (§37.4.6). The next chapter, **Chapter 38 — Feature Activation**, returns to the `signal_bits` field we saw the template carry: it is the channel through which miners vote to switch on protocol upgrades — including changes to the very mining and consensus rules this chapter rests on — without a disruptive flag day.

1. A nonce ("number used once") is a counter field in a block whose only purpose is to be varied during mining. Changing the nonce changes the block's hash; the miner tries nonce after nonce until the hash falls below the target. A Hathor block's nonce is 16 bytes; a transaction's is 4. ↩
2. A `NamedTuple` is an immutable Python record type: like a tuple, but with named fields (`template.reward` instead of `template[1]`). Once created it cannot be changed, which suits a template — it describes a fixed snapshot of work. ↩
3. **DAA** = Difficulty Adjustment Algorithm. The rule that raises or lowers required block weight so blocks keep arriving at a steady target rate as total network hash power changes. Full treatment in Chapter 9. ↩
4. JSON-RPC 2.0 is a tiny standard for remote procedure calls encoded as JSON: each message is an object with a `method`, `params`, and an `id` (for requests), or a `result` / `error` (for responses). Stratum is JSON-RPC sent one message per line over a raw TCP connection. ↩

5. In Twisted, a factory is the object that creates a fresh protocol instance for each incoming connection and holds state shared across all of them. The `StratumFactory` builds one `StratumProtocol` per miner and holds the list of all miners. Full treatment of factories in Chapter 16 (and the pattern in Chapter 3). ↩
6. A Merkle path (or Merkle branch) is the short list of sibling hashes that lets you prove a single item belongs to a set summarized by one root hash, without revealing the whole set. Merged mining uses it to prove the Hathor block's hash was committed inside Bitcoin's coinbase transaction. ↩

Chapter 38 — Feature Activation: Bit Signalling

What you'll learn in this chapter

- Why a decentralized network cannot upgrade its rules by "just pushing an update," and what a fork is — the failure this whole subsystem exists to prevent.
- The idea of **activation by vote over a schedule**: miners advertise support for a proposed rule change by setting bits in the blocks they produce, and the change switches on only once enough support has accumulated.
- The **feature state machine** — `DEFINED` → `STARTED` → `(MUST_SIGNAL | LOCKED_IN)` → `ACTIVE` | `FAILED` — its six states and exactly what makes a feature move from one to the next.
- How a vote is physically carried: the `signal_bits` byte on a block, and the rolling `feature_activation_bit_counts` the node keeps so it can ask "how many of the last N blocks voted yes?".
- How the rest of the node asks **"is feature X active here?"** (`FeatureService.is_feature_active`) and changes its behaviour on the answer — with one real verification example.

A blockchain's rules are not frozen forever. New transaction types get added, a script opcode gets a tighter bound, nano-contracts get switched on. Each of these is a change to what counts as a valid block or transaction. The hard part is not writing the new rule — it is turning it on everywhere at once, on a network of thousands of independent computers that no one controls, where you cannot phone each operator and tell them to restart at noon. This chapter is about the machinery Hathi uses to coordinate that turn-on safely. It is small in code (under a thousand lines, all in `hathi/feature_activation/`) but it sits on top of a genuinely subtle idea, so most of this chapter is spent making that idea concrete.

38.1 Localization

The whole subsystem lives in one package, with one helper reaching into the vertex model.


```

hathor-core/
├── hathor/
│   ├── feature_activation/
│   │   ├── feature.py
│   │   ├── settings.py
│   │   ├── feature_service.py
│   │   ├── bit_signaling_service.py
│   │   ├── utils.py
│   │   └── model/
│   │       ├── criteria.py
│   │       ├── feature_state.py
│   │       └── feature_info.py
│   ├── storage/
│   │   └── feature_activation_storage.py
│   └── resources/feature.py
├── transaction/
│   ├── base_transaction.py
│   ├── block.py
│   └── static_metadata.py

```

◀ YOU ARE HERE

- ← the Feature enum (the catalogue of upgrades)
- ← Settings: evaluation_interval, threshold, features{}
- ← FeatureService: get_state + is_feature_active (the l
- ← BitSignalingService: produces THIS node's votes
- ← Features dataclass (a convenience snapshot)
- ← Criteria: one feature's schedule (bit, heights, thr
- ← FeatureState enum (the 6 states)
- ← FeatureInfo = (criteria, state) pair
- ← persists/validates the settings used
- ← HTTP endpoint exposing feature states
- ← signal_bits field lives here (base_transaction.py:1
- ← reads signal_bits → bit list (block.py:307)
- ← feature_activation_bit_counts (static_metadata.py:7

Where this sits in the node (recap). Feature activation is one of the higher services from the module map (Ch. 0, §0.4) — it is built on top of the base ledger, not part of it. Two of its services are wired into the `HathorManager` at build time: `FeatureService` (the read side — answers questions) and `BitSignalingService` (the write side — produces this node's votes). It plugs into **verification** (Ch. 31), which gates rules on it, and into **mining** (Ch. 37), which stamps the vote onto each new block template.

38.2 What it does and why it exists

The problem: you cannot push an update to a network you don't own

Think about how a normal app ships a new feature. The company controls the servers; it deploys the new code, and from that instant every client talks to the new version. There is one authority, and it flips the switch.

A blockchain has no such authority — that is the entire point of it (Ch. 6). The "servers" are thousands of independent full nodes, run by people who don't know each other, who upgrade their software whenever they feel like it, if at all. Some run the latest release the day it drops; some run a version from two years ago; some never upgrade. There is no one who can flip a switch for everyone.

Now suppose a new release tightens a rule — say, "from now on, a transaction may not use opcode X." Imagine the developers ship it and it takes effect immediately on upgrade. Watch what happens:

- A miner running the **new** code produces a block that obeys the new rule. Fine.

- A miner running the **old** code produces a block that uses opcode X. Under the old rules, that block is perfectly valid.
- The new node receives the old node's block and **rejects** it as invalid. The old node receives the new node's blocks and accepts them. The two nodes now disagree about which blocks belong in the chain.

From this moment the network has split into two: one set of nodes building on a history the other set refuses. This split is called a **fork**¹. A fork is not a cosmetic bug — it means two groups of users now have two different, incompatible ledgers. A coin spent on one side was never spent on the other. The shared truth that made the system worth anything is gone.

So the question feature activation answers is precise: **how do you change a consensus rule so that essentially every node enforces the new rule starting at the same block, even though the nodes upgraded their software at wildly different times?**

The shape of the answer: schedule it, and gate it on demonstrated support

You cannot make everyone upgrade at the same wall-clock moment. But you can make the new rule take effect at the same block height for everyone — height is a property of the shared ledger, so all nodes agree on it. That turns "coordinate in time" into "coordinate in the data," which is solvable.

But picking a height up front is risky on its own. If you hard-code "rule X activates at height 5,000,000" and it turns out most of the network's mining power hasn't upgraded by then, you have scheduled a fork — at height 5,000,000, the un-upgraded majority keeps producing blocks the upgraded minority rejects. So the design adds a second guard: **before the rule turns on, the network must demonstrate that enough of it is ready**. "Enough of it is ready" is measured by miners voting. Each miner that has upgraded sets a bit in the blocks it produces, signalling "I support this change and I'm ready to enforce it." The rule activates only once that vote crosses a threshold inside a defined window. If the vote never gets there, the proposal fails cleanly instead of forking the chain.

This is a vote-to-activate-over-a-window scheme. It is modelled directly on Bitcoin's **BIP9**², which solved exactly this problem first. The rest of the chapter makes each piece — the schedule, the window, the threshold, the vote, and the state machine that ties them together — concrete.

38.3 The concepts it rests on

Three ideas from earlier chapters carry this one. Quick recaps, then the new material.

Miners, blocks, and `signal_bits` (full treatment in Ch. 9 and Ch. 25). A miner is a participant that produces new **blocks** by solving a proof-of-work puzzle (Ch. 9). Every block carries a small `signal_bits` field — one byte — alongside its other contents (`base_transaction.py:168`). Feature activation borrows the low 4 bits of that byte as a ballot: each bit is one feature's "yes/no" checkbox, and a miner fills them in on every block it mines. The block is the ballot; the miner is the voter.

The settings schedule (full treatment in Ch. 22). Constants that define the network — genesis, timing, and the feature schedule — live in the settings profile selected at boot. The feature-activation settings are a sub-block of that (`settings.FEATURE_ACTIVATION`), listing every feature and its activation criteria. Because the schedule is part of settings, every node on the same network shares it exactly; that shared schedule is what lets independent nodes reach the same conclusion about when a feature turns on.

A state machine (recall Ch. 34's footnote). A state machine is a model where a thing is always in exactly one of a fixed set of named **states**, and moves between them only along defined **transitions**, each guarded by a condition. We met one for peer connections in Ch. 34 (`HELLO → PEER_ID → READY`). Feature activation has its own: each feature, at each block, is in one of six states, and the transitions are evaluated once per scheduling window. This is the core of the subsystem.

Two new concepts unique to this chapter

The evaluation interval (the window). Counting votes block-by-block would be noisy and would let a feature flicker on and off. Instead the schedule is divided into fixed-length **windows** called evaluation intervals. On Hathor's mainnet an interval is 20,160 blocks — about one week at one block every 30 seconds (`settings.py:31`). A feature's state is recomputed **only at the boundary between windows**, using the votes gathered during the window that just ended. Inside a window the state is frozen. So a feature does not change state mid-week; it changes (if at all) at the start of each new week, based on how the previous week voted. A block whose height is an exact multiple of the interval is a **boundary block** — the only kind for which a fresh state is calculated.

The threshold. A feature activates only if the number of "yes" votes in a window reaches a **threshold** — a minimum count of blocks-in-the-window that signalled the bit. Hathor's default is 18,144 out of 20,160, which is 90% (`settings.py:38`). A high threshold is deliberate: it means a feature turns on only with the support of an overwhelming majority of recent mining power, which is precisely the "is the network ready?" guard from §38.2.

38.4 The code, walked

We build the picture bottom-up: the catalogue of features, the per-feature schedule, the six states, the state machine that moves between them, how votes are physically counted, and finally the two services the rest of the node talks to.

38.4.1 The `Feature` enum — the catalogue of upgrades

Every upgrade that has ever been (or will be) put through this process has a name in one enum.

```
@unique
class Feature(StrEnum):
    # These NOP features are used in tests
    NOP_FEATURE_1 = 'NOP_FEATURE_1'
    NOP_FEATURE_2 = 'NOP_FEATURE_2'
    NOP_FEATURE_3 = 'NOP_FEATURE_3'

    INCREASE_MAX_MERKLE_PATH_LENGTH = 'INCREASE_MAX_MERKLE_PATH_LENGTH'
    COUNT_CHECKDATASIG_OP = 'COUNT_CHECKDATASIG_OP'
    NANO_CONTRACTS = 'NANO_CONTRACTS'
    FEE_TOKENS = 'FEE_TOKENS'
    OPCODES_V2 = 'OPCODES_V2'
```

`feature.py:18`

The docstring states a discipline worth pausing on (`feature.py:20`): features are never removed from this enum, and their string values are never changed. The reason is that the values are how the settings profile maps a schedule onto a feature, and they are baked into the historical record of how past activations went. Delete or rename one and you would lose the ability to reproduce history. So the enum only grows. The three `NOP_FEATURE_*` entries (`feature.py:27`) are deliberate no-ops used by the test suite to exercise the machinery without changing any real rule.

Note the base class is `StrEnum`³ — each member is its string value, which is why `feature.value` and the member compare equal to the plain string. That matters because the settings file refers to features by their string names.

38.4.2 `Criteria` — one feature's schedule

Each feature in the schedule carries a `Criteria` object: the complete recipe for when and how that feature activates. It is a Pydantic model (Ch. 18 — validated, read-only data).

```

class Criteria(BaseModel):
    bit: NonNegativeInt
    start_height: NonNegativeInt
    timeout_height: NonNegativeInt
    threshold: Optional[NonNegativeInt] = None
    minimum_activation_height: NonNegativeInt = 0
    lock_in_on_timeout: bool = False
    version: str = Field(..., pattern=version.BUILD_VERSION_REGEX)
    signal_support_by_default: bool = False

```

`criteria.py:26` (fields at `criteria.py:59 - 66`; docstring `criteria.py:32`)

Read each field as a knob on the schedule:

- **bit** — which of the (up to four) signalling bits this feature uses. Bit 0, 1, 2, or 3.
- **start_height** — the feature does nothing until the chain reaches this height; at the first boundary at-or-after it, voting opens.
- **timeout_height** — the deadline. If the feature hasn't gathered enough votes by here, it fails (unless `lock_in_on_timeout` says otherwise).
- **threshold** — how many "yes" votes per window are needed. If left `None`, the network's `default_threshold` (90%) is used (`criteria.py:83`, `get_threshold`).
- **minimum_activation_height** — even after a feature is locked in, it won't go live before this height. A floor that gives operators warning time.
- **lock_in_on_timeout** — if `True`, reaching the timeout forces activation instead of failing. This is how a change the developers consider mandatory is shipped: it still uses the schedule, but a stalling minority cannot veto it forever (see `MUST_SIGNAL` below).
- **signal_support_by_default** — whether this node, when it mines, votes yes for this feature unless told otherwise.

There is a sibling class `ValidatedCriteria` (`criteria.py:88`) whose job is to run cross-field sanity checks once the network-wide constants are known — for example, `bit` must be below `max_signal_bits` (`criteria.py:98`), the timeout must sit at least two evaluation intervals after the start (`criteria.py:105`), and `start_height`, `timeout_height`, and `minimum_activation_height` must all be exact multiples of the evaluation interval (`criteria.py:118`). That last rule is what guarantees those heights always land on window boundaries — the only places the state machine ever recomputes.

The network-wide settings that hold all this sit in `Settings` (`settings.py:25`): `evaluation_interval` (`settings.py:31`), `max_signal_bits` (`settings.py:34`, capped at 8 — only 4 used), `default_threshold` (`settings.py:38`), and `features: dict[Feature, Criteria]` (`settings.py:43`). One validator (`settings.py:66`) refuses to let two features

share a bit during overlapping height ranges — a bit can be recycled for a later feature, but only after the earlier one's window has closed, so no block's single bit is ever ambiguous about which feature it votes for.

38.4.3 The six states

A feature is never a plain "on" or "off." At every block it is in one of six states, defined in `FeatureState` (`feature_state.py:19`):

```
class FeatureState(str, Enum):
    DEFINED = 'DEFINED'
    STARTED = 'STARTED'
    MUST_SIGNAL = 'MUST_SIGNAL'
    LOCKED_IN = 'LOCKED_IN'
    ACTIVE = 'ACTIVE'
    FAILED = 'FAILED'
```

`feature_state.py:32 - 37`

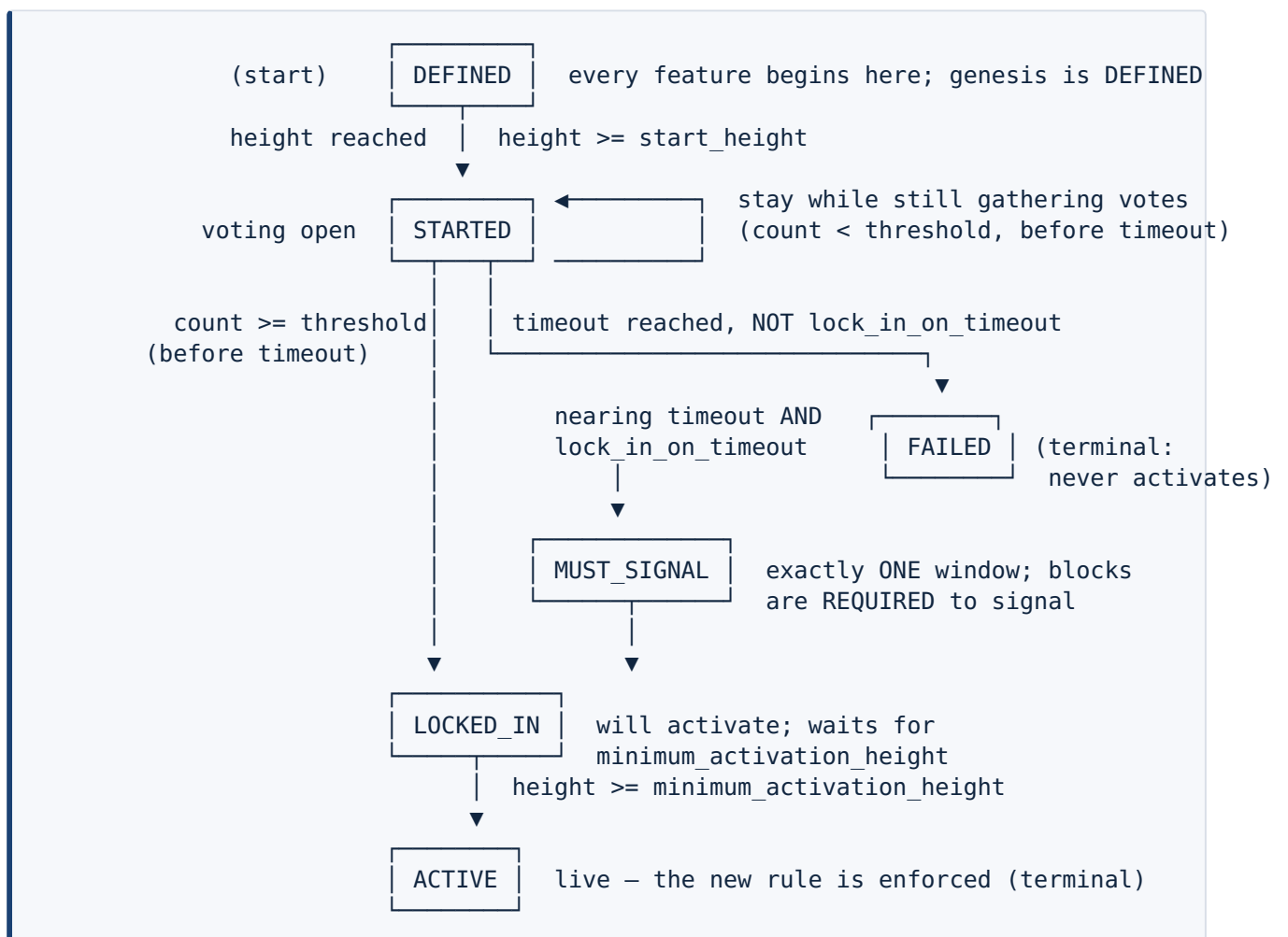
In words (docstring at `feature_state.py:20`):

- **DEFINED** — the feature exists in the catalogue but its start height hasn't been reached. Nothing is happening. Every feature starts here; genesis is **DEFINED** for everything (`feature_service.py:100`).
- **STARTED** — voting is open. Miners may signal support, and the node tallies votes each window.
- **MUST_SIGNAL** — a mandatory-feature special case: the feature is about to lock in, and the protocol now requires blocks to signal it. We cover why below.
- **LOCKED_IN** — enough support was reached. The feature will activate; this state is the grace period before it does.
- **ACTIVE** — live. The new rule is now enforced. This is the only state for which `is_active()` returns `True` (`feature_state.py:39`).
- **FAILED** — the deadline passed without enough support. The feature will never activate (on this network's history). A terminal state.

Three of these — **STARTED**, **MUST_SIGNAL**, **LOCKED_IN** — are the signalling states (`feature_state.py:43`, `get_signaling_states`): the states during which a block carrying that feature's bit is meaningful. Outside them, setting the bit has no effect.

38.4.4 The state machine

Here is the whole machine. Each arrow is evaluated **only at a window boundary**, using the criteria and the vote count from the window that just ended.



The single function that computes every one of those arrows is `_calculate_new_state` (`feature_service.py:139`). It is a tidy `if`-ladder over the previous state. The interesting transitions:

DEFINED → STARTED when the boundary height reaches `start_height` (`feature_service.py:163 – 167`). Before that, it stays **DEFINED** .

STARTED → (the fork in the road) (`feature_service.py:169 – 186`). At each boundary the node reads the vote count for the window that ended and compares it to the threshold:

```

if previous_state == FeatureState.STARTED:
    if height >= criteria.timeout_height and not criteria.lock_in_on_timeout:
        return FeatureState.FAILED

    parent_block = boundary_block.get_block_parent()
    counts = parent_block.static_metadata.feature_activation_bit_counts
    count = counts[criteria.bit]
    threshold = criteria.get_threshold(self._feature_settings)

    if height < criteria.timeout_height and count >= threshold:
        return FeatureState.LOCKED_IN

    if (height + evaluation_interval >= criteria.timeout_height) and criteria.lock_in_on_timeout:
        return FeatureState.MUST_SIGNAL

    return FeatureState.STARTED

```

feature_service.py:169

Read the four outcomes: 1. Past the timeout and not a forced feature → **FAILED**. Voting is over and it lost. 2. Votes reached the threshold while still in time → **LOCKED_IN**. It won. 3. The next window would cross the timeout and this is a forced feature (`lock_in_on_timeout`) → **MUST_SIGNAL**. Last chance to signal, and signalling is now compulsory. 4. None of the above → stay **STARTED** for another window.

MUST_SIGNAL → **LOCKED_IN**, always, after exactly one window (`feature_service.py:188 - 192`). The state exists for a single interval and then locks in unconditionally. Its purpose is to give a forced feature a final, enforced signalling window so that the historical record shows the network was told — blocks that fail to signal during this phase are rejected (we'll see the enforcement in §38.4.7).

LOCKED_IN → **ACTIVE** once the height reaches `minimum_activation_height` (`feature_service.py:194 - 198`); otherwise it waits in **LOCKED_IN**. **ACTIVE** and **FAILED** are sinks — once there, the feature stays (`feature_service.py:200 - 204`).

38.4.5 Counting the votes — `feature_activation_bit_counts`

The state machine asks "how many of the blocks in the last window set this bit?" Computing that by re-scanning thousands of blocks at every boundary would be slow. Instead the node keeps a **running tally** on each block, carried forward block by block.

That tally is `feature_activation_bit_counts` — a field on the block's static metadata⁴ (`static_metadata.py:72`). It is a small list: one integer per signalling bit, each integer being "how many blocks from the start of the current window up to and including this block have set that bit."

It is computed once, lazily, when the block's static metadata is built (`static_metadata.py:132`, `_calculate_feature_activation_bit_counts`). The logic is a one-line recurrence:


```

previous_counts = cls._get_previous_feature_activation_bit_counts(block, height, settings, vertex)
bit_list = block._get_feature_activation_bit_list()

count_and_bit_pairs = zip_longest(previous_counts, bit_list, fillvalue=0)
updated_counts = starmap(add, count_and_bit_pairs)
return list(updated_counts)

```

static_metadata.py:146 – 151

In words: take the parent's counts, add this block's own vote (a list of 0/1 per bit), element-wise. The "previous counts" come from the parent block — **unless this block is itself a boundary block**, in which case the tally resets to empty and starts fresh for the new window (`static_metadata.py:153`, `_get_previous_feature_activation_bit_counts`; the reset at `:167 – 168`). So the count climbs across a window and zeroes at each boundary. Each block's stored count is the cumulative tally for its own window so far; the count on the last block before a boundary is therefore the full window's tally — which is exactly the number `_calculate_new_state` reads from the parent block at the boundary (`feature_service.py:175`).

Where does a single block's own vote come from? From its `signal_bits` byte. `Block._get_feature_activation_bit_list` (`block.py:307`) masks off the low `max_signal_bits` bits of `signal_bits` and expands them into a list of 0s and 1s (`block.py:314 – 317`). One subtlety the code is careful about, stated in the comments: **the least-significant bit is on the left** of the list, so `bit_list[criteria.bit]` lines up with `bit` being the bit index. That is why the state machine can index the count list directly by `criteria.bit`.

38.4.6 FeatureService — the read side (the brain)

`FeatureService` (`feature_service.py:46`) is the object the rest of the node consults. It holds the feature settings and the transaction storage (`feature_service.py:49`). Its headline method is the question everyone actually wants to ask:

```

def is_feature_active(self, *, vertex: Vertex, feature: Feature) -> bool:
    """Return whether a Feature is active for a certain vertex."""
    block = self._get_feature_activation_block(vertex)
    state = self.get_state(block=block, feature=feature)
    return state.is_active()

```

feature_service.py:54

Two things to notice. First, activation is relative to a vertex. "Is feature X active?" has no absolute answer — it depends on where in the chain you ask, because the answer changes at activation height. So the query takes the vertex it is being asked about. For a block, that's the block itself; for a transaction, it resolves to the transaction's `closest_ancestor_block` (`feature_service.py:60`, `_get_feature_activation_block`) — the

deepest block the transaction depends on, recorded in its static metadata (`static_metadata.py:181`). That makes a transaction's view of "what's active" deterministic and tied to the ledger, not to wall-clock time.

Second, the real work is in `get_state` (`feature_service.py:96`), and it leans hard on the window structure to stay cheap. Its logic:

1. Genesis is `DEFINED` for everything — short-circuit (`feature_service.py:100`).
2. If this block already has the state cached in its metadata, return it (`feature_service.py:103` ; cache read via `block.get_feature_state` , `block.py:327`).
3. Otherwise, find the **previous boundary block** (`feature_service.py:109 - 114`). Every block in a window shares that window's state, so the state is really a property of the boundary. Recurse to get the boundary's state, and cache it on the boundary block (`feature_service.py:118`).
4. If this block is not itself a boundary, it inherits that state unchanged (`feature_service.py:120 - 121`).
5. If it is a boundary, run `_calculate_new_state` to compute the fresh state, and cache it on this block (`feature_service.py:123 - 135`).

The recursion walks boundary-to-boundary, not block-to-block, and caches as it goes, so after the first pass each boundary's state is a stored lookup. There is one careful detail at step 5: when a boundary block transitions a feature into `MUST_SIGNAL` , the service notifies the bit-signalling service so this node starts voting yes automatically (`feature_service.py:129 - 131`) — and it caches the new state without persisting it, because a brand-new boundary block may still be unverified and its metadata not yet safe to write (`feature_service.py:133 - 135`). The previous boundary, already verified, is the one that gets saved.

38.4.7 Gating behaviour — two real use-sites

A feature is worthless unless something changes when it's active. Two verifiers do exactly that.

Enforcing `MUST_SIGNAL` . The block verifier (Ch. 31) refuses any block that fails to signal a mandatory feature:

```
def verify_mandatory_signaling(self, block: Block) -> None:
    signaling_state = self._feature_service.is_signaling_mandatory_features(block)
    match signaling_state:
        case BlockIsSignaling():
            return
        case BlockIsMissingSignal(feature):
            raise BlockMustSignalError(
                f"Block must signal support for feature '{feature.value}' during MUST_SIGNAL phase"
            )
```

`block_verifier.py:93`

`is_signaling_mandatory_features` (`feature_service.py:69`) checks, for every feature currently in `MUST_SIGNAL` , whether enough blocks remain in the window for the threshold to still be reachable — and if a block's omission would make it unreachable, that block is rejected. This is the teeth behind a forced upgrade: during the `MUST_SIGNAL` window, you cannot mine a valid block without voting yes.

Branching on an active feature. The merge-mined block verifier picks a limit based on whether a feature is live:

```
is_feature_active = self._feature_service.is_feature_active(  
    vertex=block,  
    feature=Feature.INCREASE_MAX_MERKLE_PATH_LENGTH,  
)  
max_merkle_path_length = (  
    self._settings.NEW_MAX_MERKLE_PATH_LENGTH if is_feature_active  
    else self._settings.OLD_MAX_MERKLE_PATH_LENGTH  
)
```

`merge_mined_block_verifier.py:33`

This is the pattern every gated rule follows: ask `is_feature_active` , then apply the new rule or the old one. Before activation height the old limit holds for everyone; after it, the new one — and because every node computes activation from the same shared schedule and the same on-chain votes, every node flips at the same block. No fork.

A convenience wrapper, the `Features` dataclass (`utils.py`), bundles several of these answers into one frozen snapshot for a vertex (`utils.from_vertex`), and folds in a wrinkle: some features can be hard-forced `ENABLED` / `DISABLED` by a plain settings flag instead of going through activation (`utils.py` , `_is_feature_active` , matching on `FeatureSetting`). That is the escape hatch for features that don't need a network vote on a given profile.

38.4.8 BitSignalingService — the write side (this node's ballot)

Everything so far was the read side: given the chain, what's active? The `BitSignalingService` (`bit_signaling_service.py:29`) is the write side: when this node mines a block, which bits does it set?

It is constructed with two operator-provided sets, `support_features` and `not_support_features` (`bit_signaling_service.py:46 - 47`) — the features the operator has explicitly chosen to vote yes or no on, typically via command-line flags. It refuses a feature listed in both (`bit_signaling_service.py:184`).

Its core method assembles the ballot:

```
def generate_signal_bits(self, *, block: Block, log: bool = False) -> int:
    feature_signals = self._calculate_feature_signals(block=block, log=log)
    signal_bits = 0
    for feature, (criteria, enable_bit) in feature_signals.items():
        signal_bits |= int(enable_bit) << criteria.bit
    return signal_bits
```

bit_signaling_service.py:74

It only considers features currently in a signalling state for the given block (bit_signaling_service.py:164, _get_signaling_features, filtering on get_signaling_states()). For each, it decides yes/no (bit_signaling_service.py:106 - 110):

```
enable_bit = (default_enable_bit or support) and not not_support
```

Read it plainly: vote yes if the feature defaults to yes or the operator opted in — unless the operator explicitly opted out. Then generate_signal_bits ORs each yes into the right bit position and returns the byte. The manager stamps that byte onto every block template it produces:

```
signal_bits=self._bit_signaling_service.generate_signal_bits(block=parent_block)
```

manager.py:766

So the loop closes: this node's vote goes onto the blocks it mines → those blocks land in the chain → other nodes' feature_activation_bit_counts tally them → the state machine reads the tally at the next boundary. One more hook: when the read side detects a feature entering MUST_SIGNAL, it calls on_must_signal (bit_signaling_service.py:140), which force-adds the feature to this node's support set — so a node automatically complies with mandatory signalling without operator action.

38.4.9 A small worked example

Make it concrete. Suppose a feature `OPCODES_V2` uses **bit 1**, with `start_height = 200`, `timeout_height = 600`, `threshold = 70`, and an evaluation interval of 100 blocks (toy numbers; mainnet's interval is 20,160). Walk the boundaries:

Window	Boundary block height	Vote count for bit 1 in the window that just ended	State at boundary (after calc)	Why
... → 100	100	(irrelevant)	DEFINED	start_height 200 not reached
100 → 200	200	(irrelevant)	STARTED	height >= start_height
200 → 300	300	55 of 100 set bit 1	STARTED	55 < threshold 70 → keep
300 → 400	400	82 of 100 set bit 1	LOCKED_IN	82 >= 70, before timeout
400 → 500	500	(irrelevant)	ACTIVE	LOCKED_IN + min_activation

From height 400 onward, `is_feature_active(vertex=block_at_height>=500, feature=OPCODES_V2)` returns `True`, and the merge-of opcode rules flip. Every node on the network computes this same table from the same blocks, so they all flip at height 500.

Now the failure path. Keep everything but say the votes never climb: each window lands at 40 of 100. At the boundary of height 600 (`timeout_height`), `previous_state == STARTED`, `height >= timeout_height`, and `lock_in_on_timeout` is `False` → the feature returns **FAILED** (`feature_service.py:170 – 171`) and is never enforced. The proposal died without splitting the chain — which is the whole point.

If instead `lock_in_on_timeout` were `True` (a mandatory feature), the window ending at height 500 would see `height + interval (600) >= timeout (600)` and move to **MUST_SIGNAL** (`feature_service.py:183 – 184`); during that 500→600 window every block would be required to set bit 1 (`block_verifier.py:93`); at the 600 boundary it would move unconditionally to **LOCKED_IN** (`feature_service.py:188 – 192`), then **ACTIVE**.

38.5 How it plugs into the lifecycle

Tracing the wiring across earlier chapters:

- **Build (Ch. 24).** The builder constructs both services and injects them into the `HathorManager`: `feature_service` and `bit_signaling_service` are constructor parameters (`manager.py:110`, `:117`), stored as `self._bit_signaling_service` and `self.feature_service` (`manager.py:197`, `:207`). `BitSignalingService.__init__` also back-links itself onto the feature service (`bit_signaling_service.py:59`) so the read side can call `on_must_signal`.
- **Start (Ch. 29).** During manager start-up, `self._bit_signaling_service.start()` runs (`manager.py:451`). It validates that the persisted feature settings match the current ones (`bit_signaling_service.py:66`, via the storage's `validate_settings`), reads the best block, and logs which features this node is signalling — warning if the operator asked to signal a feature that isn't in a signalling window (`bit_signaling_service.py:190`). The storage layer (`feature_activation_storage.py`) exists so a node cannot silently change a network's feature schedule between restarts without noticing.
- **Mine (Ch. 37).** Every block template the manager produces carries this node's vote, stamped at `manager.py:766`.
- **Read blocks (Ch. 25).** Votes ride in `signal_bits` (`base_transaction.py:168`) and are tallied into `feature_activation_bit_counts` when each block's static metadata is built (`static_metadata.py:132`).

- **Gate rules (Ch. 31).** Verification consults the feature service — `verify_mandatory_signaling` (`block_verifier.py:93`) and the merge-mined merkle-path limit (`merge_mined_block_verifier.py:33`) are the two live examples; future consensus and verification rule changes follow the same `is_feature_active` pattern.
- **Expose (Ch. 36).** `resources/feature.py` serves the current per-feature state over HTTP, so an operator can see exactly where each feature stands.

38.6 Recap

State	Meaning	Leaves to...	On condition (at a window boundary)
DEFINED	In the catalogue; not yet started	STARTED	<code>height >= start_height</code>
STARTED	Voting open; tallying each window	LOCKED_IN	<code>count >= threshold</code> and before <code>timeout_height</code>
		MUST_SIGNAL	nearing timeout and <code>lock_in_on_timeout</code>
		FAILED	reached <code>timeout_height</code> and not <code>lock_in_on_timeout</code>
MUST_SIGNAL	One window; signalling compulsory	LOCKED_IN	always, after exactly one interval
LOCKED_IN	Will activate; grace period	ACTIVE	<code>height >= minimum_activation_height</code>
ACTIVE	Live — new rule enforced	—	terminal
FAILED	Never activates	—	terminal

Feature activation is how Hather changes its own consensus rules without a flag-day: a proposed change is scheduled to a block height, gated on a high threshold of miner votes gathered over fixed windows, and switched on for everyone at once only after the network has demonstrably signalled readiness — or cleanly abandoned if it hasn't. The code splits along a clean seam: `FeatureService` reads the chain to answer "is X active here?" (the question verifiers gate on), and `BitSignalingService` writes this node's votes onto the blocks it mines. The state machine in `_calculate_new_state` is the whole

protocol in one function, and the running `feature_activation_bit_counts` on each block is what makes counting votes cheap. The next chapter opens the single largest subsystem in the node — **nano-contracts** (Ch. 39) — which is itself one of the features gated behind this very mechanism (`Feature.NANO_CONTRACTS`).

1. A fork (specifically a consensus fork or chain split) is when nodes stop agreeing on which blocks are valid, so the single shared ledger splits into two incompatible histories. A coin's state can differ between the two sides. Feature activation exists to prevent forks caused by rule changes; this is distinct from a temporary fork, the brief, self-healing disagreement when two miners find a block at nearly the same time (resolved by consensus, Ch. 32). ↵
2. **BIP9** ("Bitcoin Improvement Proposal 9") is the Bitcoin scheme for "versionbits" soft-fork activation: miners signal readiness for a change by setting bits in a block's version field, and the change activates once a threshold of blocks in a retarget window signal support, within a defined timeout. Hathor's feature activation is a close adaptation of it, with the addition of the `MUST_SIGNAL` mandatory-signalling phase. ↵
3. A `StrEnum` (Python 3.11+) is an enumeration whose members are `str` values. `Feature.NANO_CONTRACTS == 'NANO_CONTRACTS'` is `True`, and `Feature.NANO_CONTRACTS.value` is the string. This lets the settings file refer to features by plain string names while the code uses the type-safe enum. ↵
4. Static metadata is per-vertex data the node computes once and then never changes — as opposed to mutable metadata that gets updated as the ledger evolves (the split is covered in Ch. 25). A block's height and its `feature_activation_bit_counts` are static: once a block is fixed in the chain, both are fixed too, so they can be computed once and cached. ↵

Chapter 39 — Nano-Contracts: The Smart-Contract Runtime

What you'll learn in this chapter

- What a smart contract is in general, and how Hathor's **nano-contract** differs from Ethereum's model — Python Blueprints run on the node's own interpreter, not a bespoke virtual machine.
- The anatomy of a **Blueprint**: the base class, the `@public` / `@view` / `@fallback` decorators, and how a plain type annotation becomes a piece of persistent state.
- How a transaction calls a contract: the **nano-header** it carries, the **NCActions** that move tokens in and out, and the **Context** the method receives.
- The **Runner** — the engine that dispatches a call to a Blueprint method — and why execution must be **metered** (the halting problem, resource exhaustion, the "never block" rule from Chapter 2).
- Where contract state lives: a **Patricia/Merkle trie**, what that data structure is, and why it gives every contract a verifiable state root.
- How execution ties into **consensus**: contracts run when a block confirms them, in a deterministic order; a contract that fails is **voided** (Chapter 32), and the whole subsystem is gated behind the `NANO_CONTRACTS` feature (Chapter 38).

This is the largest single subsystem in `hathor-core` — roughly fifteen thousand lines across eighty-odd files. It is also the one that asks the most of you conceptually, because it layers a second world on top of the ledger you have spent the whole book learning. Underneath, there is still the DAG of vertices, the UTXO accounting, the consensus that picks a canonical history. On top of it now sits a small programming language runtime: programs, state, function calls, resource limits.

We split the chapter into two parts. **Part A — the runtime** answers what runs, and how: what a Blueprint is, how a transaction invokes one of its methods, and how the Runner executes that method safely. **Part B — the state** answers where it lives, and why it is verifiable: the trie that stores every contract's data, the typed fields that serialize into it, and the way a failed execution is folded back into consensus. Read Part A first; Part B will then make sense as "and here is where all those state changes actually land."

One framing sentence to hold onto throughout: **a nano-contract is a vertex that owns a balance and runs code.** Everything else is detail on top of that.

39.A — The Runtime

39.A.1 What a smart contract is

Before any Hathor code, the general idea.

A **smart contract**¹ is a program stored on a ledger whose execution is performed and agreed upon by every node in the network, not by one trusted server. The phrase is older than blockchains, but blockchains are what made it practical, because a blockchain already gives you the two things a contract needs and a normal program cannot assume:

1. **Shared, tamper-evident state.** Every node holds the same copy of the data, and nobody can secretly alter it.
2. **Agreement on what happened.** Consensus (Chapter 32) means that when the contract's code runs, every node computes the same result and records the same state change. There is no "it worked on my machine."

The canonical mental model is a vending machine that holds money. Anyone can put coins in (call the contract). The machine's own rules — not a shopkeeper — decide what comes out. The rules are public and the same for everyone, and the money it holds is real. A token-swap, an escrow that releases funds when two of three people approve, a betting pool that pays winners — these are all "money plus rules," which is exactly what a contract is.

The hard part is that you are now running arbitrary user-supplied code on every node in the network, and you must make it:

- **Deterministic**² — every node must compute byte-identical results, or consensus breaks. No wall-clock time, no real randomness, no reading from the network.
- **Bounded** — a contract must not be able to loop forever or eat all the memory, or it could freeze every node at once (a denial-of-service against the whole network).
- **Isolated** — a contract must not be able to open files, make network calls, or reach into the node's internals.
- **Verifiable** — any node must be able to prove that the resulting state is the correct consequence of the calls, ideally with a single hash.

Every design decision in this chapter exists to satisfy one of those four constraints. Keep them in mind; they are the why behind almost everything.

How Hathor differs from Ethereum

If you have heard of smart contracts, you have heard of Ethereum, and the comparison is worth drawing because Hathor made a deliberately different choice.

Ethereum contracts are written in a language like Solidity, **compiled to bytecode**³ for a purpose-built virtual machine — the **EVM**⁴ — and that bytecode is what lives on-chain. Every node ships an EVM interpreter, executes the bytecode opcode by opcode, and charges **gas**⁵ for each opcode so that execution is bounded. The EVM is a whole separate machine with its own instruction set, its own memory model, and its own notion of "an account."

Hathor's nano-contracts take a different road. A contract template — Hathor calls it a **Blueprint**⁶ — is written in **a restricted subset of Python**, and it runs on the node's own Python interpreter, inside a sandbox, rather than on a separate VM. There is no Solidity, no bytecode compiler you have to learn, no second instruction set. You write a Python class; the node runs it. The class file is `hathor/nanocontracts/blueprint.py`; we will read it in a moment.

This buys ergonomics — Python developers can write contracts in a language they already know — at the cost of a harder sandboxing problem: Python was never designed to run untrusted code safely, so Hathor has to take things away from the language (disable `open`, `eval`, `import`, restrict the builtins) rather than start from a locked-down machine. The trade is "familiar language, careful jail" versus Ethereum's "purpose-built machine, unfamiliar language." We will see both the jail (§39.A.6) and the metering (§39.A.7) that together stand in for the EVM's guarantees.

Recap — vertex, and where contracts sit (full treatment Ch. 8 & 25). A vertex is any node in Hathor's ledger DAG — a `Block` or a `Transaction`. In Chapter 25 you met the vertex class hierarchy, including a subclass called `OnChainBlueprint` that we deferred to here. A nano-contract is **not** a fourth vertex type; it is an ordinary `Transaction` that carries an extra header instructing the node to run contract code. The Blueprint source code itself, when stored on the ledger, rides on an `OnChainBlueprint` vertex (§39.B.5).

39.A.2 Localization

The whole subsystem lives under one package, `hathor/nanocontracts/`. Here is the map, grouped by role; the rest of the chapter walks it roughly top to bottom.

hathor/nanocontracts/	◀ YOU ARE HERE
— WHAT A CONTRACT IS —	
blueprint.py	← Blueprint base class + metaclass (fields)
types.py	← @public/@view/@fallback, NCActions, ContractId...
method.py	← captures+serializes a method's signature
context.py	← Context object passed to a @public method
on_chain_blueprint.py	← OnChainBlueprint vertex: code stored on-ledger
catalog.py	← resolves a blueprint id → blueprint class
blueprints/	← built-in blueprint registry (empty by default)
— HOW A CONTRACT RUNS —	
runner/runner.py	← THE Runner: dispatch a call to a method (1467 LOC)
runner/call_info.py	← the call stack / call trace
metered_exec.py	← MeteredExecutor: fuel + memory bounds
custom_builtins.py	← the sandbox: which builtins a contract may use
blueprint_env.py	← the "syscall" surface a contract sees
contract_accessor.py	← handle for calling ANOTHER contract
proxy_accessor.py	← handle for a delegatecall-style proxy
balance_rules.py	← how deposits/withdrawals move balances
rng.py	← deterministic pseudo-random generator
— WHERE STATE LIVES —	
storage/patricia_trie.py	← the Merkle/Patricia trie (the state store)
storage/contract_storage.py	← NContractStorage: one contract's state
storage/block_storage.py	← NCBlockStorage: all contracts in a block
storage/changes_tracker.py	← stages writes so a failed call rolls back
fields/	← typed contract attributes (descriptors)
nc_types/	← serialization codecs for those types
— EXECUTION ≠ CONSENSUS —	
execution/block_executor.py	← run all NC txs in a block (pure)
execution/consensus_block_executor.py	← apply the effects, void failures
sorter/random_sorter.py	← deterministic call ordering

The on-tx instruction that triggers a contract call does not live here — it lives with the vertex model, at `hathor/transaction/headers/nano_header.py`, because it is physically part of a transaction's bytes. We meet it in §39.A.4.

Context. Nano-contracts are a higher service built on the base ledger, not part of it. The node can run with the feature switched off entirely (`ENABLE_NANO_CONTRACTS = False`), and most of mainnet's early life did. When it is on, this package is wired into three other subsystems you already know: **verification** (Chapter 31) checks that a nano-header is well-formed; **consensus** (Chapter 32) is what actually runs the contracts, when a block confirms them; and **feature activation** (Chapter 38) gates the whole thing behind the `NANO_CONTRACTS` feature so the network can switch it on by miner vote.

39.A.3 What a Blueprint is

A **Blueprint** is the template — the class — and a **contract** is a live instance of that template with its own state and balance. One Blueprint ("a betting pool") can back thousands of independent contracts (one pool per event). This is exactly the class-versus-object distinction from Chapter 1, lifted onto the ledger: the Blueprint is the class, each contract is an object, and the object's attributes are stored not in RAM but in the trie.

Let us build the intuition with a generic example before reading Hathor's real base class. Here is the shape a contract author writes — a counter that anyone can bump:

```
class Counter(Blueprint):
    count: int  # a state attribute – persisted

    @public
    def initialize(self, ctx: Context) -> None:
        self.count = 0  # set the initial state

    @public
    def increment(self, ctx: Context) -> None:
        self.count += 1  # mutate state

    @view
    def get_count(self) -> int:
        return self.count  # read state, never mutate
```

Four things are happening here, and each is enforced by the framework, not by convention:

- `count: int` is a **state attribute**. It looks like an ordinary class annotation, but the Blueprint machinery turns it into a piece of trie-backed storage. `self.count` does not read from memory; it reads from (and writes to) the contract's slice of the state trie.
- `initialize` is the **constructor-equivalent**. Every Blueprint must define exactly one method called `initialize`, and it must be `@public`. It runs once, when the contract is created.
- `increment` is a **public method** — callable by a transaction from the outside, may change state.
- `get_count` is a **view method** — read-only, callable cheaply (e.g. from an API) without producing a transaction, and forbidden from mutating state.

Now the real base class. It is shorter than you might expect, because the cleverness lives in its **metaclass**⁷.

```
# hathon/nanocontracts/blueprint.py:126
class Blueprint(metaclass=_BlueprintBase):
    __slots__ = ('__env',)

    def __init__(self, env: BlueprintEnvironment) -> None:
        self.__env = env

    @final
    @property
    def syscall(self) -> BlueprintEnvironment: # blueprint.py:142
        return self.__env

    @final
    @property
    def log(self) -> NCLogger: # blueprint.py:148
        return self.syscall.__log__
```

A Blueprint instance holds exactly one thing: `__env`, a `BlueprintEnvironment` (§39.A.6). That environment is the contract's entire connection to the outside world — its storage, its logger, its ability to call other contracts. Note `__slots__ = ('__env',)`: the instance is forbidden from storing any other attributes directly in memory. So where does `self.count` go? Into the trie — and that redirection is set up by the metaclass.

The metaclass: turning annotations into storage

A metaclass is a class whose instances are classes. When Python executes `class Counter(Blueprint): ...`, the metaclass `_BlueprintBase.__new__` runs and gets a chance to rewrite the class before it exists. Hathor uses that hook to do three jobs:

```
# hathon/nanocontracts/blueprint.py:56-73 (abridged)
cls._validate_initialize_method(attrs) # there MUST be an @public initialize()
cls._validate_fallback_method(attrs)
nc_fields = attrs.get('__annotations__', {}) # e.g. {'count': int}
# ... reject forbidden / underscore-prefixed field names ...
attrs[NC_FIELDS_ATTR] = nc_fields # remember the declared fields
attrs['__slots__'] = tuple() # forbid plain instance attributes
```

Then, for each annotated field, it replaces the annotation with a **descriptor**⁸ — a `Field` object — that knows how to read and write that attribute through the trie:

```
# hathon/nanocontracts/blueprint.py:79-92 (abridged)
for field_name, field_type in attrs[NC_FIELDS_ATTR].items():
    field = make_field_for_type(field_name, field_type) # build a Field descriptor
    setattr(new_class, field_name, field) # install it on the class
```

After this runs, `Counter.count` is not an `int` slot — it is a `Field[int]` descriptor. When contract code does `self.count += 1`, Python routes the read and the write through that descriptor's `__get__` / `__set__`, which serialize and persist the value (Part B, §39.B.4). The contract author never sees any of this; they wrote `self.count` and got persistence for free. We pick up the descriptor mechanics in Part B; for now the takeaway is: **a type annotation on a Blueprint is a declaration of persistent state.**

Two structural rules the metaclass enforces, with their reasons:

- **initialize is mandatory and must be @public** (`blueprint.py:104`). A contract with no constructor could never set up its invariants; the rule guarantees there is always one well-defined entry point that runs first.
- **Field names cannot start with `_` and cannot be `syscall` or `log`** (`blueprint.py:27` , `:65`). The underscore rule keeps the contract's persisted state separate from internal plumbing; `syscall` and `log` are reserved because they are the names of the two properties through which the contract reaches the system.

39.A.4 How a transaction calls a contract

A Blueprint is inert. Something has to call one of its methods, and in Hathor that something is a transaction. The instruction to call a method is not a separate vertex type — it is an extra **header**⁹ bolted onto an ordinary `Transaction`. That header is the `NanoHeader`, at `hathor/transaction/headers/nano_header.py`.

A header is a self-describing block of bytes appended to the transaction's serialized form (you met the idea in passing in Chapter 25; fee headers work the same way). The nano-header carries everything the node needs to perform one contract call:

```
# hathor/transaction/headers/nano_header.py:100-121 (fields, abridged)
@dataclass(slots=True, kw_only=True)
class NanoHeader(VertexBaseHeader):
    nc_seqnum: int          # caller's sequence number (replay protection)
    nc_id: VertexId         # WHICH contract (or which blueprint, if creating)
    nc_method: str          # WHICH method to call, e.g. "increment"
    nc_args_bytes: bytes    # the method's arguments, serialized
    nc_actions: list[NanoHeaderAction] # token deposits / withdrawals
    nc_address: bytes       # WHO is calling (the caller's address)
    nc_script: bytes        # the caller's signature(s) over the tx
```

Read the fields as a sentence: "caller `nc_address`, with sequence number `nc_seqnum`, calls method `nc_method` (arguments `nc_args_bytes`) on `nc_id`, attaching these token movements `nc_actions`, and proves it with signature `nc_script`." That is one contract call, fully specified.

A subtlety worth pausing on: `nc_id` means two different things depending on whether the call is creating a new contract or calling an existing one. When the method is `initialize`, the transaction is creating a contract, and `nc_id` holds the **blueprint id** (which template to instantiate). Otherwise `nc_id` holds the **contract id** (which existing instance to call). The header has a one-line helper that captures this:

```
# hathor/transaction/headers/nano_header.py:242
def is_creating_a_new_contract(self) -> bool:
    return self.nc_method == NC_INITIALIZE_METHOD    # i.e. == "initialize"
```

And the contract's own id, when it is being created, is the creating transaction's hash (`get_contract_id` , :247). So a contract's identity is the id of the transaction that gave it life — the same "the vertex is its hash" principle the whole ledger runs on.

NCActions — moving tokens into and out of a contract

A counter holds no money, but most contracts do, and the question of how tokens get in and out is central. Hathor's answer is the **NCAction**. A contract has its own balance, separate from any address's balance; to move tokens across that boundary, the calling transaction declares **actions**. There are four kinds (`hathor/nanocontracts/types.py`: 359-491):

Action	Direction	Meaning
<code>NCDepositAction</code>	into the contract	the tx pays tokens to the contract
<code>NCWithdrawalAction</code>	out of the contract	the contract pays tokens to the tx
<code>NCGrantAuthorityAction</code>	into the contract	give the contract mint/melt authority
<code>NCAcquireAuthorityAction</code>	out of the contract	take authority out of the contract

Each token-action names a token and an amount, and is a frozen dataclass:

```
# hathor/nanocontracts/types.py:463-469
@dataclass(slots=True, frozen=True, kw_only=True)
class NCDepositAction(BaseTokenAction): # token_uid + amount
    """Deposit tokens into the contract."""

@dataclass(slots=True, frozen=True, kw_only=True)
class NCWithdrawalAction(BaseTokenAction):
    """Withdraw tokens from the contract."""
```

This is where the contract world meets the UTXO world from Chapter 7. A deposit is funded by the transaction's inputs; a withdrawal becomes one of the transaction's outputs. The node enforces conservation across the boundary: the tokens a contract claims to receive must really be paid in by the tx, and the tokens it pays out must really appear as outputs. In the header the action stores a `token_index` (`nano_header.py`:50) — a pointer into the transaction's token list — rather than the token id directly, to keep the bytes small; `to_nc_action` (:54) resolves it back to a real `TokenUid` at execution time.

A contract author opts in to which actions a method will accept, on the decorator:

```
@public(allow_deposit=True, allow_withdrawal=True)
def swap(self, ctx: Context) -> None:
    ...
```


A method that does not list `allow_deposit` will reject any transaction that tries to deposit into it. This is a safety default: a method receives no tokens unless it explicitly says it can.

The Context

When a `@public` method runs, it receives a `Context` as its first argument after `self`. The Context is the call's "who, what, when" — an immutable snapshot the contract can trust:

```
# hathor/nanocontracts/context.py:95-116 (fields, abridged)
self.__caller_id = caller_id      # Address or ContractId that made the call
self.__vertex    = vertex_data   # data about the calling transaction
self.__block     = block_data    # data about the block executing it
self.__actions   = actions       # the NCActions, grouped by token
```

Note what is not there: no live transaction object, no storage handle, no clock. The Context exposes `caller_id`, the actions, and read-only views of the vertex and block — and nothing else. It is a `@final` class with private slots, and the Runner even passes the method a copy (`runner.py:675`, `ctx.copy()`) so that even a malicious contract cannot mutate the original and confuse the engine's later checks. The actions are grouped by token (`__group_actions__`, `context.py:49`) so a method can ask "what is being deposited in HTR?" with `ctx.get_single_action(token_uid)`.

View methods, by contrast, receive **no** Context (`types.py:294`, `validate_has_not_ctx_arg`). A view is a pure read — it has no caller to speak of and moves no tokens — so handing it a Context would be meaningless. That asymmetry is the cleanest way to tell the two method kinds apart: public takes a Context and may write; view takes none and may only read.

39.A.5 The decorators: `@public`, `@view`, `@fallback`

The decorators in `types.py` are how a Blueprint author marks intent, and how the framework knows which methods are callable and how. They do almost no work at call time; their job is to stamp a marker attribute onto the function so the Runner can check it later.

```
# hathor/nanocontracts/types.py:206-209
def _set_method_type(fn: Callable, method_type: NCMethodType) -> None:
    if hasattr(fn, NC_METHOD_TYPE_ATTR):
        raise BlueprintSyntaxError('method must be annotated with at most one method type')
    setattr(fn, NC_METHOD_TYPE_ATTR, method_type) # stamp PUBLIC / VIEW / FALLBACK
```

`@public` (`types.py:248`) stamps `NCMethodType.PUBLIC` and validates the method's shape at decoration time — before the node ever runs: it must have `self`, it must have a `ctx: Context` argument (`validate_has_ctx_arg`, `:270`), and its argument types must be

serializable. It also records which actions the method allows. `@view (types.py:284)` stamps `VIEW` and validates the opposite: it must **not** take a `ctx` (`validate_has_not_ctx_arg, :294`). `@fallback (types.py:312)` marks a single catch-all method, called `fallback`, that the Runner invokes when a transaction names a method the Blueprint does not have — Hathor's equivalent of a default handler.

Doing this validation at decoration time, rather than at call time, means a malformed Blueprint fails to load rather than failing mysteriously deep inside a transaction. That matters because on-chain Blueprint code is run through these same decorators when it is loaded from the ledger (§39.B.5); a Blueprint that does not obey the rules is rejected before it can ever be instantiated.

39.A.6 The Runner and the syscall surface

We now have a Blueprint (a class), a contract (an instance with state in the trie), and a transaction with a nano-header that names a method to call. The **Runner** is the engine that puts them together: it takes "call method `m` on contract `c` with these arguments and actions," fetches the right Blueprint code, instantiates it, runs the method under resource limits, and either commits the resulting state changes or throws them away.

The Runner is the heart of the subsystem — `hathor/nanocontracts/runner/runner.py`, 1467 lines. We will not read all of it; we will trace the one path that matters and name the guard-rails along the way.

The public entry point used by consensus is `execute_from_tx (runner.py:163)`. It reads the nano-header, performs the replay check, and dispatches to either "create a contract" or "call a method":

```
# hathor/nanocontracts/runner/runner.py:163-194 (heavily abridged)
def execute_from_tx(self, tx: Transaction) -> None:
    nano_header = tx.get_nano_header()
    contract_id = (ContractId(VertexId(tx.hash))          # creating: id is the tx hash
                  if nano_header.is_creating_a_new_contract()
                  else ContractId(VertexId(nano_header.nc_id))) # calling: id is nc_id

    # replay protection: the caller's seqnum must advance, by a small step
    current = self.block_storage.get_address_seqnum(Address(nano_header.nc_address))
    diff = nano_header.nc_seqnum - current
    if diff <= 0 or diff > MAX_SEQNUM_JUMP_SIZE:
        raise NCFail(f'invalid seqnum (diff={diff})')

    context = nano_header.get_context()
    nc_args = NCRawArgs(nano_header.nc_args_bytes)
    if nano_header.is_creating_a_new_contract():
        self.create_contract_with_nc_args(contract_id, BlueprintId(...), context, nc_args)
    else:
        self.call_public_method_with_nc_args(contract_id, nano_header.nc_method, context, nc_args)
```

The **seqnum**¹⁰ check deserves a word. Each caller address has a monotonically increasing sequence number stored in the block trie. A new call must use a higher seqnum than the last one seen (`get_address_seqnum`, `block_storage.py:164`), by no more than `MAX_SEQNUM_JUMP_SIZE` (10). This stops an attacker from replaying a signed transaction twice and stops calls from being reordered arbitrarily, while the small allowed jump leaves room for a few calls in flight.

Dispatching the call

Follow the "call a method" branch into `_execute_public_method_call` (`runner.py:606`), the method every public call funnels through, whether it came from a transaction or from another contract:

```
# hathon/nanocontracts/runner/runner.py:623-647 (abridged)
self._validate_context(ctx)
changes_tracker = self._create_changes_tracker(contract_id) # staging area
blueprint = self._create_blueprint_instance(blueprint_id, changes_tracker)
method = getattr(blueprint, method_name, None)

if method is None: # no such method → try the fallback
    fallback_method = getattr(blueprint, NC_FALLBACK_METHOD, None)
    if fallback_method is None:
        raise NCMethodNotFound(...)
    method = fallback_method
    ...
else:
    if not is_nc_public_method(method): # refuse to call a @view as if public
        raise NCInvalidMethodCall(...)
    parser = Method.from_callable(method) # capture the signature
    args = self._validate_nc_args_for_method(parser, nc_args) # decode + type-check args
```

Three things are worth naming:

1. **The method must be `@public`.** A transaction cannot reach a `@view` method through this path; calling a non-public method is an error (`runner.py:644`). The decorator marker from §39.A.5 is what makes this check possible.
2. **Arguments are decoded and re-encoded** (`_validate_nc_args_for_method`, `runner.py:690`). The raw bytes from the header are deserialized against the method's declared types and a fresh copy of each argument is built. This both type-checks the input and severs any shared references, so one contract cannot smuggle a mutable object into another and mutate it behind its back.
3. **The actual run is metered.** The method is not called directly. It is handed to a `MeteredExecutor`:

```
# hathon/nanocontracts/runner/runner.py:675
ret = self._metered_executor.call(method, args=(ctx.copy(), *args))
```

That single line is where contract code finally executes — under the resource bound we discuss next. After it returns, the call's balance changes are validated to be non-negative (`runner.py:678`) and the staged changes may be committed.

The syscall surface: `BlueprintEnvironment`

Recall that a `Blueprint` instance holds exactly one thing, its `__env`, exposed as `self.syscall`. That environment is the **only** door from contract code back into the node, and it is a deliberate application of the **proxy pattern**¹¹ (Chapter 3): the contract never touches storage or the Runner directly; it talks to a `BlueprintEnvironment`, which forwards each request to the Runner under controlled conditions.

```
# hathor/nanocontracts/blueprint_env.py:35-58 (abridged)
@final
class BlueprintEnvironment:
    """A class that holds all possible interactions a blueprint may have with the system."""
    __slots__ = ('__runner', '__log__', '__storage__', '__cache__')

    @property
    def rng(self) -> NanoRNG:
        return self.__runner.syscall_get_rng()          # deterministic randomness

    def get_contract_id(self) -> ContractId:
        return self.__runner.get_current_contract_id()
```

Through `self.syscall`, a contract can read its own balance, get a deterministic random number, and — the consequential one — **call another contract**. Inter-contract calls go through the Runner's `syscall_call_another_contract_public_method` (`runner.py:327`), which enforces the rules that make composition safe: a contract cannot call itself (`runner.py:342`), recursion is capped at `MAX_RECURSION_DEPTH = 100` (`runner.py:120`), the total number of calls in one transaction is capped at `MAX_CALL_COUNTER = 250` (`runner.py:121`), and re-entrancy¹² is forbidden unless the method explicitly opted into it. There is also a proxy (delegatecall-style) variant, `syscall_proxy_call_public_method` (`runner.py:391`), where another `Blueprint`'s code runs against the calling contract's storage — the same primitive Ethereum calls `DELEGATECALL`, used for upgradeable libraries.

The reason all of this is funnelled through one environment object is the **isolation** constraint from §39.A.1. If a contract could reach `tx_storage` or the reactor directly, the sandbox would be meaningless. By giving it a single, hand-built `BlueprintEnvironment` whose every method is a deliberate, checked syscall, the node decides exactly what a contract is permitted to do — and nothing leaks through.

39.A.7 Metered execution: why, and how

We have reached the single most consequential safety mechanism, and one where the code holds a surprise that you should know about.

Why bound execution at all

Recall constraint two from §39.A.1: a contract must not be able to run forever or exhaust memory. This is not a hypothetical worry; it is a direct consequence of a deep result in computer science. The **halting problem**¹³ says there is no general algorithm that can look at an arbitrary program and decide whether it will eventually stop or loop forever. So the node cannot inspect a Blueprint and prove it terminates. A contract author could write `while True: pass`, by accident or by malice.

If the node ran that code to completion, it would never finish — and because every node runs every contract, a single such transaction would freeze the entire network at once. This is the contract-world version of the "**never block the reactor**" rule from Chapter 2: the node's event loop must keep turning, and any single task must be bounded.

Since you cannot decide termination in advance, the standard answer — Ethereum's gas, every smart-contract platform's answer — is to meter execution: give each call a finite budget, charge the budget down as the code runs, and **abort** when it hits zero. You do not need to know whether the program halts; you only need to guarantee that your execution of it halts. The budget is called **fuel**¹⁴ (Hathor's word; Ethereum says gas). The same logic applies to memory: cap it, and abort if the cap is exceeded.

How Hathor sets it up

The budget lives in `MeteredExecutor` (`hathor/nanocontracts/metered_exec.py`). The Runner creates one per call, seeded from settings:

```
# hathor/nanocontracts/runner/runner.py:294
self._metered_executor = MeteredExecutor(
    fuel=self._initial_fuel,          # NC_INITIAL_FUEL_TO_CALL_METHOD
    memory_limit=self._memory_limit, # NC_MEMORY_LIMIT_TO_CALL_METHOD
)
```

The intended cost model is one fuel unit per Python bytecode opcode executed — there is a per-opcode cost table, `FUEL_COST_MAP = [1] * 256` (`metered_exec.py:33`), and the comment beside it points at Python's `sys.settrace` hook, the standard way to run a callback on every line/opcode. Two exception types stand ready: `OutOfFuelError` and `OutOfMemoryError` (`metered_exec.py:36`, `:40`).

The actual run compiles the call into a tiny code stub and `exec`s it inside a restricted environment:

```
# hathor/nanocontracts/metered_exec.py:80-111 (abridged)
def call(self, func, /, *, args):
    from hathor import NCFail
    from hathor.nanocontracts.custom_builtins import EXEC_BUILTINS
    env = {'__builtins__': EXEC_BUILTINS, '__func__': func, '__args__': args, '__result__': None}
    code = compile('__result__ = __func__(*__args__)', '<blueprint>', 'exec', ...)
    try:
        exec(code, env)
    except NCFail:
        raise
    except Exception as e:
        raise NCFail from e  # ANY error becomes a clean NC failure
    return env['__result__']
```

Two parts of this are real and load-bearing today:

- **The sandbox is real.** `EXEC_BUILTINS` (from `custom_builtins.py`) is a hand-curated replacement for Python's builtins. Dangerous functions — `eval`, `exec`, `open`, `__import__` and many more — are removed or replaced with versions that raise `NCDisabledBuiltinError` (`custom_builtins.py:262`). `import` is reduced to an allow-list, and even `range`, `all`, `any`, `enumerate`, `filter` are re-implemented in pure Python (`custom_builtins.py:91` onward) so that they execute opcode by opcode in the interpreter rather than dropping into uncountable C code. That last detail is a tell: re-implementing builtins in Python is precisely what you do so that an opcode-counting meter can see them.
- **Failures are contained.** Any exception a contract raises is converted to `NCFail` (`metered_exec.py:109`). The Runner never lets a contract's error escape as an arbitrary Python exception; it always becomes a clean, well-typed failure that consensus knows how to handle (§39.B.6).

The honest caveat (a correction to the surface story)

Here is the surprise, and this book reports the code over the tidy narrative. In this branch, `MeteredExecutor` **stores** the `fuel` and `memory_limit` but the `exec/call` methods shown above **do not actually decrement fuel or enforce the memory cap** — there is no `sys.settrace` callback installed, and the `# TODO: cost for each opcode` comment at `metered_exec.py:32` confirms the per-opcode accounting is unfinished. The `OutOfFuelError` / `OutOfMemoryError` types are defined and are caught by the on-chain-blueprint loader (`on_chain_blueprint.py:201-206`), but nothing on the normal call path currently raises them.

What this means precisely: the **architecture** for metering is in place — the budget is threaded through, the exception types exist, the builtins are deliberately written to be meterable — but the **enforcement hook itself is scaffolded, not yet wired**, in the version on this branch. The sandbox (which functions a contract may call) is fully active; the resource bound (how long it may run) is not yet enforced at this layer. Treat

the metering design as the intended and documented model, and the live enforcement as work in progress. If you are reading a later revision, check whether `call` now installs a trace function — that is where the meter will live.

39.A.8 Worked trace: deploy and call a Counter

Pull Part A together with one concrete story. We use the `Counter` Blueprint from §39.A.3. The example is realistically shaped — it uses the real API surface verified above — though the exact Counter is illustrative; `hathor-core` ships no built-in counter (the built-in registry `_blueprints_mapper` is empty by default, `blueprints/__init__.py:21`), so in practice this code would arrive as an on-chain Blueprint (§39.B.5).

Step 0 — the Blueprint exists. Either it is registered in the catalog, or — far more commonly — its source has been published in an `OnChainBlueprint` transaction whose hash is the blueprint id.

Step 1 — create a contract. Alice broadcasts a transaction whose nano-header has `nc_method = "initialize"`, `nc_id = <blueprint id>`, no actions, and her signature. When a block confirms it, consensus hands the tx to the Runner's `execute_from_tx`. Because the method is `initialize`, `is_creating_a_new_contract()` is true, so the Runner takes the **create** branch:

```
contract_id = ContractId(tx.hash)          # the new contract's identity = this tx's hash
blueprint_id = BlueprintId(nc_id)          # which template to instantiate
create_contract_with_nc_args(contract_id, blueprint_id, ctx, nc_args) # runner.py:928
```

`create_contract_with_nc_args` checks the contract does not already exist, records its blueprint id in storage, then runs `initialize` through the same `_execute_public_method_call` path as any other call. Inside, `self.count = 0` fires the `count` field's descriptor, which writes `0` into a fresh slice of the trie. After `initialize` returns, the Runner verifies every declared field was initialized (`_check_all_field_initialized`, `runner.py:314`) — a contract may not leave a field unset — and commits. The contract now exists, with `count = 0` and an empty balance.

Step 2 — call `increment`. Bob broadcasts a transaction with `nc_method = "increment"`, `nc_id = <contract id from step 1>`, a higher seqnum than his last call, and his signature. When confirmed, the Runner takes the **call** branch:


```

1. seqnum check: Bob's nc_seqnum advances by 1..10          → ok
2. changes_tracker = staging layer over the contract's storage
3. blueprint = Counter(env)                                # instance with env pointing at the tracker
4. method = blueprint.increment                             # found; it is @public          (runner.py:626)
5. args = decode+typecheck nc_args_bytes (none here)       (runner.py:647)
6. metered_executor.call(increment, args=(ctx.copy(),))    (runner.py:675)
   → self.count += 1   reads 0 from trie, writes 1 to the tracker
7. balances validated non-negative; staged change committed (runner.py:307)

```

The change is written first to a `NCChangesTracker` — a staging layer — and only flushed to the real trie once the call succeeds. We see why that staging matters in the next step.

Step 3 — a call fails. Suppose `increment` instead did `self.count += 1` and then hit an error (a bad assertion, a forbidden builtin, an explicit `raise NCFail`). The `MeteredExecutor.call` catches it and re-raises `NCFail`. Because the write went to the tracker, not the trie, the increment is discarded — the contract's `count` stays at its previous value. **A failed contract call has no effect on state.** And, as Part B shows, the failing transaction is itself voided. This atomicity — all of a call's writes land, or none do — is the same guarantee a database transaction gives you, and it is what makes contracts safe to compose.

That is the whole runtime in miniature: a transaction names a method, the Runner fetches the Blueprint, runs the method under a (future) fuel bound inside a real sandbox, stages the writes, and either commits them or throws them away. Part B is where those committed writes actually go.

39.B — The State

Part A ended with `self.count = 1` being "committed to the trie." Part B is that trie: what it is, why it is shaped the way it is, and how a contract's typed attributes turn into bytes inside it. Then we close the loop back to consensus.

39.B.1 Why a trie, and what a trie is

A contract's state is just key→value data: `count → 1`, `balances[HTR] → 500`, and so on. You could store that in a plain dictionary on disk. The reason Hathor does not — the reason it uses a **trie** with a particular hashing scheme — comes back to constraint four from §39.A.1: **verifiable state**. Every node must be able to agree, with a single short fingerprint, that its copy of all contract state is identical to everyone else's. A plain dictionary gives you the data but no cheap, tamper-evident fingerprint of the whole of it. A Merkle structure does.

Build the idea in two layers.

Layer one: a trie (prefix tree)

A **trie**¹⁵ (also "prefix tree" or, in its compressed form, "radix trie") stores keys by spelling them out along a path from the root. Instead of one big table, you have a tree where each step down consumes part of the key. Keys that share a prefix share the top of their path:

keys: "cat", "car", "dog"



The two useful properties: lookups follow the key character by character (fast), and — crucially for us — **the structure is fully determined by its contents**. Two tries holding the same set of key→value pairs are the same tree, regardless of what order the pairs were inserted. That order-independence is what lets two nodes that processed the same contract writes end up with byte-identical state.

Layer two: Merkle hashing → a state root

Now make every node carry a hash of itself and all its descendants. Each node's id is `hash(its own data + the ids of its children)`. Change any value anywhere in the tree, and that node's id changes, which changes its parent's id, all the way up — so the **root's id changes**. This is a **Merkle tree**¹⁶, and the root id is a one-hash fingerprint of the entire dataset. This single value is the **state root**¹⁷.

Two nodes can now compare all of their contract state by comparing one 32-byte number. If the roots match, the states are provably identical; if they differ, something somewhere is different. That is the verifiability we needed.

A **Patricia trie**¹⁸ is the practical, compressed version: chains of single-child nodes are collapsed into one node holding the whole shared substring, so the tree stays shallow. Hathor's combination — a Patricia (compressed) trie with Merkle (hash-linked) node ids — is the data structure at `hathor/nanocontracts/storage/patricia_trie.py`.

39.B.2 The trie in code

The implementation is faithful to the picture above and small enough to read in one sitting. A node:

```
# hathor/nanocontracts/storage/patricia_trie.py:40-65 (abridged)
@dataclass(kw_only=True, slots=True)
class Node:
    key: bytes
    length: int
    content: Optional[bytes] = None           # the stored value, if this key was set
    children: DictChildren = ...
    _id: Optional[NodeId] = None             # this node's Merkle id
```

The Merkle id is computed exactly as described — hash of key, content, and sorted child ids:

```
# hathor/nanocontracts/storage/patricia_trie.py:67-79
def calculate_id(self) -> NodeId:
    h = hashlib.sha256()
    h.update(self.key)
    if self.content is not None:
        h.update(self.content)
    for child_id in sorted(self.children.values()): # SORTED → order-independent
        h.update(child_id)
    return NodeId(h.digest())
```

`sorted(...)` is the line that guarantees order-independence: children are hashed in id order, never in insertion order, so the same set of children always yields the same parent id.

The class makes one design choice explicit in its docstring: **the nodes are immutable**.

```
# hathor/nanocontracts/storage/patricia_trie.py:94-100
class PatriciaTrie:
    """... All nodes are immutable. So every update will create a new path of nodes
    from leaves to a new root. ..."""
```

An update never edits a node in place. It copies the node it touches and rebuilds a fresh path of copies from that node up to a new root (`_update` → `_build_path`, `patricia_trie.py:281`, `:237`). The old nodes are untouched and still reachable from the old root. This is **persistence**¹⁹ in the data-structure sense: every past root id still names a complete, valid snapshot of the state at that moment. That property is exactly what lets consensus re-execute or roll back contract state during a reorg (§39.B.6) — it can pick up an earlier root and rebuild from there, because that earlier state was never overwritten.

Reading and writing go through `get / update`, and committing flushes the staged new nodes to the backing store:

```
# hathor/nanocontracts/storage/patricia_trie.py:373-381, 118
def update(self, key, content): # writes a value; may change self.root
def get(self, key, *, root_id=None): # reads a value (optionally at a past root)
def commit(self): # flush local node changes to the database
```

`root.id` is the state root. Hold that thought — it is the value that travels up into consensus.

39.B.3 Two levels of trie: contract state and block state

A single trie per contract is not the whole story, because the node must fingerprint all contracts at once. Hathor uses **two levels** of trie.

The contract trie holds one contract's state. It is wrapped by `NCContractStorage` (`hathor/nanocontracts/storage/contract_storage.py:137`), which gives the raw `key→bytes` trie a typed face: attributes, balances, and metadata, each under a distinct tag so they cannot collide:

```
# hathor/nanocontracts/storage/contract_storage.py:40-58 (abridged)
class _Tag(Enum):
    ATTR = b'\0' # a state attribute like `count`
    BALANCE = b'\1' # the contract's balance of a token
    METADATA = b'\2' # e.g. which blueprint this contract uses

class AttrKey(TrieKey):
    def __bytes__(self) -> bytes:
        return _Tag.ATTR.value + hashlib.sha256(self.key).digest()
```

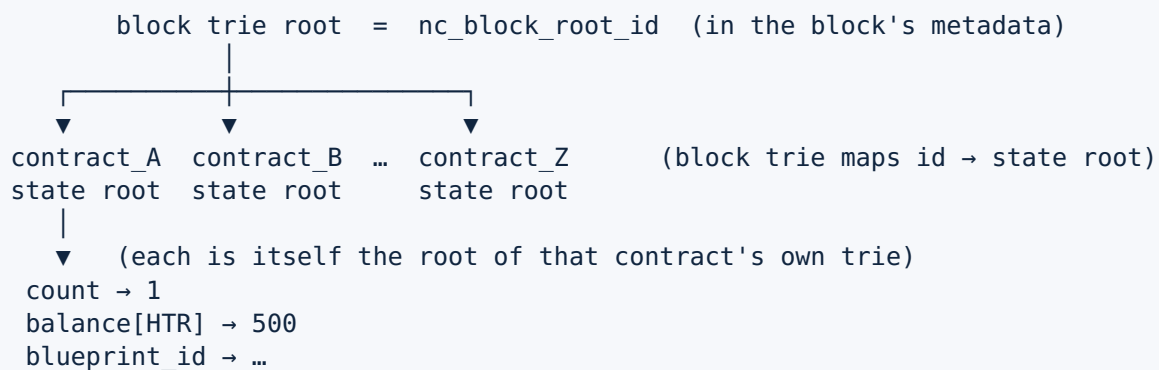
So `self.count` is stored under the trie key `b'\0' + sha256(b'count')`, its balance of HTR under `b'\1' + <htr-token-id>`, and its blueprint id under a metadata key. The contract's state root is `NCContractStorage.get_root_id()` (`contract_storage.py:384`) — its trie's root id.

The block trie sits one level up. `NCBlockStorage` (`hathor/nanocontracts/storage/block_storage.py:59`) holds a trie that maps each contract id to that contract's state root, plus per-token descriptions and the per-address seqnums:

```
# hathor/nanocontracts/storage/block_storage.py:82-89
def get_contract_root_id(self, contract_id):    # contract → its state root
    return self._block_trie.get(bytes(ContractKey(contract_id)))

def update_contract_trie(self, nc_id, root_id): # record a contract's new root
    self._block_trie.update(bytes(ContractKey(nc_id)), root_id)
```

Because the block trie is itself a Merkle trie over the contracts' roots, its own root — `NCBlockStorage.get_root_id()` (`block_storage.py:95`) — is a single fingerprint of **the entire nano-contract state of the whole network** at that block. This is the value the node stores in the block's metadata as `nc_block_root_id`, and it is how two nodes confirm they agree on all contract state with one comparison. Picture it:



Fetching a contract's storage walks down both levels: `get_contract_storage` looks up the contract's root in the block trie, then opens a `PatriciaTrie` at that root and wraps it in an `NCContractStorage` (`block_storage.py:112`).

39.B.4 Typed fields: how `self.count` becomes bytes

We left a thread dangling in Part A: the metaclass turns `count: int` into a `Field` descriptor. Now we can see where it leads.

A `Field` is a Python **descriptor** — an object that defines `__get__` / `__set__` / `__delete__` and so intercepts attribute access on the class that holds it:

```
# hathor/nanocontracts/fields/field.py:76-85 (abridged)
class Field(Generic[T]):
    def __set__(self, instance: Blueprint, value: T) -> None:
        node = self._container_node_factory.build(instance)
        node.set_value(self._prefix, value) # serialize + write to the trie

    def __get__(self, instance: Blueprint, owner=None) -> T:
        node = self._container_node_factory.build(instance)
        return node.get_value(self._prefix) # read from the trie + deserialize
```

So `self.count += 1` is really read `count` from the trie, add one, write it back to the trie — the contract author's `+= 1` hides a load and a store. The `_prefix` is the field name (`b'count'`), which becomes part of the trie key. The factory distinction between a scalar field and a container field (a `dict`, `list`, `set`, `deque` attribute) is what lets `self.balances['alice'] = 5` write only the one entry's key rather than re-serializing the whole map — containers under `hathor/nanocontracts/fields/` map each element to its own trie key.

But "write `value` to the trie" needs one more piece: the trie stores **bytes**, and `value` is a Python `int`, `str`, dataclass, whatever. The translation is the job of an **NCType** — a serialization codec keyed by the declared type:

```
# hathor/nanocontracts/nc_types/nc_type.py:28-34 (abridged)
class NCType(ABC, Generic[T]):
    """models a type with a known type signature and how it will be (de)serialized.
    Used for NC method-call args, and for the values stored in NC properties."""
```

There is one **NCType** subclass per supported value shape — `BoolNCType`, `StrNCType`, `BytesNCType`, sized-int types, `OptionalNCType`, collection types, `make_dataclass_nc_type` for dataclasses, and more, all under `hathor/nanocontracts/nc_types/`. When the metaclass builds the `Field` for `count: int`, it resolves `int` to the right **NCType**, and that codec is what turns `1` into the bytes stored under `b'\0' + sha256(b'count')`. The same **NCType** machinery serializes a method's arguments into `nc_args_bytes` for the nano-header (§39.A.4) and deserializes them back when the Runner decodes the call — one type-to-bytes system, used at both the storage boundary and the call boundary.

This typed-storage layer is why a Blueprint declares its state with ordinary annotations and gets persistence, serialization, and a verifiable hash with no extra ceremony. The author writes `count: int`; the framework supplies a descriptor, a codec, a trie key, and a Merkle root.

Staging: the changes tracker

One detail from Part A's failure case (§39.A.8, step 3) lives here. The contract a method writes to is not the live `NCContractStorage` directly — it is an `NCChangesTracker` (`hathor/nanocontracts/storage/changes_tracker.py`) layered over it. The tracker subclasses `NCContractStorage` but buffers every write — attribute changes, balance diffs, authority changes — in memory:

```
# hathor/nanocontracts/storage/changes_tracker.py (shape)
class NCChangesTracker(NCContractStorage):
    # buffers attribute writes, balance diffs, authority diffs;
    # .commit() applies them to the underlying storage; .block() discards them.
```

If the call succeeds, the tracker is committed and its buffered writes flow into the real trie. If the call raises `NCFail`, the tracker is discarded and the writes evaporate — the atomicity guarantee from §39.A.8. The tracker is also what lets the Runner compute a call's net balance change and check conservation (`_validate_balances`, `runner.py:519`) before anything is made permanent.

39.B.5 On-chain Blueprints: code stored on the ledger

Where does a Blueprint's source code come from? Two places.

A **built-in Blueprint** is one shipped with the node and registered in the catalog. The catalog maps a blueprint id to a Python class:

```
# hathor/nanocontracts/catalog.py:29-33
class NCBBlueprintCatalog:
    def get_blueprint_class(self, blueprint_id) -> Type['Blueprint'] | None:
        return self.blueprints.get(blueprint_id, None)
```

The built-in registry `_blueprints_mapper` (`blueprints/__init__.py:21`) is **empty by default** on this branch — built-ins are populated from settings (`generate_catalog_from_settings`, `catalog.py:37`), so in practice almost every Blueprint is the second kind.

An **on-chain Blueprint** is one whose source code lives on the ledger, published as a transaction. This is the `OnChainBlueprint` vertex you were promised back in Chapter 25. It is an ordinary `Transaction` subclass (version `ON_CHAIN_BLUEPRINT`) that carries a compressed blob of Python source:

```
# hathor/nanocontracts/on_chain_blueprint.py:155-185 (abridged)
class OnChainBlueprint(Transaction):
    """On-chain blueprint vertex to be placed on the DAG of transactions."""
    def __init__(self, ..., code: Optional[Code] = None, ...):
        if not self._settings.ENABLE_NANO_CONTRACTS:
            raise RuntimeError('NanoContracts are disabled')
        self.nc_pubkey: bytes = b'' # author's public key
        self.nc_signature: bytes = b'' # author's signature
        self.code = code or Code(CodeKind.PYTHON_ZLIB, b'', self._settings)
```

The `Code` object holds the source `zlib`-compressed (`Code.from_python_code`, `on_chain_blueprint.py:128`), with size caps to stop someone publishing a multi-megabyte blob. The **blueprint id is the transaction's own hash** (`blueprint_id`, `:189`) — publishing code mints its identity, exactly as creating a contract mints the contract's id from its tx hash.

When the node first needs to run an on-chain Blueprint, it must turn that stored source text into a runnable class. This is where the source is `exec`-ed — under its own metered budget, separate from the per-call budget:

```
# hathor/nanocontracts/on_chain_blueprint.py:193-208 (abridged)
def _load_blueprint_code_exec(self):
    fuel = self.settings.NC_INITIAL_FUEL_TO_LOAD_BLUEPRINT_MODULE
    memory_limit = self.settings.NC_MEMORY_LIMIT_TO_LOAD_BLUEPRINT_MODULE
    metered_executor = MeteredExecutor(fuel=fuel, memory_limit=memory_limit)
    try:
        env = metered_executor.exec(self.code.text)          # run the module's top level
    except OutOfFuelError as e:
        raise OCBOutOfFuelDuringLoading from e
    except OutOfMemoryError as e:
        raise OCBOutOfMemoryDuringLoading from e
    blueprint_class = env[BLUEPRINT_EXPORT_NAME]             # the @export-ed class
    return blueprint_class, env
```

Two safeguards stand out. First, running the module is metered too — loading code is itself a contract-like activity that an attacker could weaponize (a module whose top level loops forever), so it gets the same fuel/memory treatment (with the same caveat from §39.A.7 about enforcement being scaffolded — note the loader is one of the few places that actually catches the fuel/memory exceptions). Second, the loaded module must export exactly one Blueprint via the `@export` decorator (`types.py:299`), which stashes the class under a well-known name `__blueprint__`; the loader pulls that class out (`on_chain_blueprint.py:207`). The result is cached on the vertex (`_load_blueprint_code` , `:210`) so the source is parsed and executed at most once per node lifetime.

The sandbox from §39.A.7 is what makes running arbitrary on-chain source even thinkable: the `exec` happens with `EXEC_BUILTINS`, so the published module cannot open files, import freely, or call `eval`. A Blueprint is source code anyone can publish; the jail is the only thing standing between that and the node's machine.

39.B.6 Execution meets consensus

We can now close the largest loop in the book: how a contract call actually runs in the life of the node, and how a failure is folded back into the consensus you learned in Chapter 32.

The key fact, and it surprises people: **nano-contracts do not execute when their transaction is first received**. A nano-contract transaction sits in the mempool, verified but not run, until a **block confirms it**. Execution happens at block consensus time, for all the NC transactions a block confirms, together. The reason is determinism and ordering: two conflicting calls must be run in an order every node agrees on, and only a block gives a canonical ordering of the transactions beneath it.

Recap — voiding and `voided_by` (full treatment Ch. 10 & 32). Consensus never deletes a vertex; it voids one by adding a marker to its `voided_by` set in metadata. A voided vertex stays in storage but is excluded from the canonical ledger. In Chapter 32 you saw two markers: a tx's own hash (it lost a conflict) and `SOFT_VOIDED_ID`. Nano-contracts add a third — and this section is where the forward-reference from Chapter 32 is paid off. → full treatment of consensus in Ch. 32.

Sorting the calls

When a block is processed, the NC transactions it confirms are first put into a **deterministic order** by the sorter (`hathor/nanocontracts/sorter/random_sorter.py`). It is not a plain timestamp sort; it is a random topological sort, seeded by the block's hash:

```
# hathor/nanocontracts/sorter/random_sorter.py:29-40 (abridged)
def random_nc_calls_sorter(block, nc_calls):
    sorter = NCBlockSorter.create_from_block(block, nc_calls)
    seed = hashlib.sha256(block.hash).digest() # determinism from block hash
    order = sorter.generate_random_topological_order(seed)
    ...
```

Two properties are designed in. **Topological**: if call B depends on call A (A is an input of B, or they share a caller seqnum), A runs first — Kahn's algorithm over the dependency DAG (`NCBlockSorter` , `random_sorter.py:57`). **Random but deterministic**: among calls with no dependency between them, the order is shuffled by a pseudo-random generator seeded from the block hash, so it is unpredictable to an attacker yet identical on every node. The randomness denies anyone the ability to game execution order; the determinism keeps consensus intact.

Running the block

`NCBlockExecutor.execute_block` (`hathor/nanocontracts/execution/block_executor.py:131`) walks the sorted calls and runs each through the Runner. It is written as a pure generator: it `yields` an effect for each step rather than mutating anything itself, so the part that decides "did this succeed or fail" is cleanly separated from the part that applies the consequences. Each call produces one of three results:

```
# hathor/nanocontracts/execution/block_executor.py:38-60 (abridged)
NCTxExecutionSuccess # ran cleanly
NCTxExecutionFailure # raised NCFail
NCTxExecutionSkipped # was already voided (e.g. by an earlier failing call)
```

The per-call RNG seed is derived from the block hash, the tx hash, and the current state root (`block_executor.py:180-186`) — deterministic, but unique per call, so two contracts in the same block get different randomness.

Applying the effects, and voiding failures

The companion `NCConsensusBlockExecutor` (`hathor/nanocontracts/execution/consensus_block_executor.py`) consumes those effects and applies them inside the consensus context. The success and failure branches are where the whole chapter lands:

```
# hathor/nanocontracts/execution/consensus_block_executor.py:234-298 (heavily abridged)
case NCTxExecutionSuccess(tx=tx, runner=runner):
    tx_meta.nc_execution = NCExecutionState.SUCCESS
    runner.commit() # flush the staged state into the trie
    tx.storage.indexes.non_critical_handle_contract_execution(tx) # update indexes
    context.nc_exec_success.append(tx) # queue a pubsub event (Ch 30)

case NCTxExecutionFailure(tx=tx, runner=runner, exception=exception, traceback=tb):
    on_failure(tx) # void the tx with NC_EXECUTION_FAIL_ID
    self._nc_log_storage.save_logs(tx, runner.get_last_call_info(), (exception, tb))
```

On **success**: the Runner's staged changes are committed to the contract's trie, the indexes are updated, and a pubsub event fires so downstream listeners learn the contract executed. On **failure**: the transaction is voided via the `on_failure` callback, which adds the marker `NC_EXECUTION_FAIL_ID` (the literal bytes `b'nc-fail'`, `nanocontracts/__init__.py:25`) to the tx's `voided_by`. The executor's own invariant check spells out the resulting metadata exactly:

```
# hathor/nanocontracts/execution/consensus_block_executor.py:202-208 (abridged)
case NCExecutionState.SUCCESS: assert tx_meta.voided_by is None
case NCExecutionState.FAILURE: assert tx_meta.voided_by == {tx.hash, NC_EXECUTION_FAIL_ID}
case NCExecutionState.SKIPPED: assert NC_EXECUTION_FAIL_ID not in tx_meta.voided_by
```

This is the third voiding marker promised by Chapter 32. A transaction can be perfectly valid — correct signature, sufficient weight, no double-spend — and still be voided, because the contract method it called raised `NCFail`. Validity (Chapter 31) asks "is this transaction well-formed?"; nano-execution asks "did the program it invokes succeed?" Both must hold for the transaction to count, and a failure in the second is recorded distinctly so anyone inspecting the ledger can see why the tx was voided: not a conflict, not a structural error, but a contract that rejected the call.

At the end of the block, the executor commits the block storage and records the block-trie root in the block's metadata:

```
# hathor/nanocontracts/execution/consensus_block_executor.py:313-319 (abridged)
case NCEndBlock(block=block, block_storage=block_storage, final_root_id=final_root_id):
    block_storage.commit()
    meta = block.get_metadata()
    meta.nc_block_root_id = final_root_id # the verifiable fingerprint of ALL contract state
    context.save(block)
```

`nc_block_root_id` is the §39.B.3 block-trie root — one hash that fingerprints every contract's state after this block. Because the trie is persistent (§39.B.2), a **reorg**²⁰ is handled by `execute_chain` (`consensus_block_executor.py:101`) resetting the affected blocks' `nc_block_root_id` to `None` and re-executing from the common ancestor's root — the older states were never overwritten, so they are still there to rebuild from.

39.B.7 How it all plugs into the lifecycle

Step back and place nano-contracts on the node's spine — the life-of-a-node story from Chapter 0 — for a single contract-calling transaction:

1. A tx with a NanoHeader arrives over the network (P2P / ingestion, Ch 33–35)
2. VERIFICATION checks the nano-header is well-formed (Ch 31; gated: `ENABLE_NANO_CONTRACTS`)
3. The tx enters the mempool – NOT yet executed
4. A BLOCK confirms it
5. CONSENSUS sorts the block's NC calls deterministically (random_sorter, §39.B.6)
6. The RUNNER executes each call under a metered budget (runner.py, §39.A.6–7)
 - └ success → state committed to the contract trie (§39.B.2–4)
 - └ failure → tx voided with `NC_EXECUTION_FAIL_ID` (Ch 32, §39.B.6)
7. The block-trie root (`nc_block_root_id`) is saved into block metadata (§39.B.3)
8. Indexes updated; pubsub events fired (Ch 28, Ch 30)
9. Whole subsystem is OFF unless the `NANO_CONTRACTS` feature is active (Ch 38)

Every box has been a chapter. Nano-contracts do not replace any of them; they thread through all of them. A nano-contract transaction is still a vertex (Chapter 25), still serialized the bespoke way (Chapter 26), still stored in RocksDB (Chapter 27), still verified (Chapter 31), still ordered by consensus (Chapter 32), still ingested through the vertex handler (Chapter 33). The nano-contracts package adds exactly one new thing to that pipeline: at consensus time, when a block confirms a call, run the program and commit or void. That is the whole subsystem's contribution, and now you can defend why each of its eighty files exists to make that one step deterministic, bounded, isolated, and verifiable.

Recap

Concept	What it is	Where in code
Blueprint	the contract template (a Python class)	<code>blueprint.py:126</code> ; metaclass : 35
Contract	a live instance, with state + balance	<code>id = creating tx hash</code> (<code>runner.py:169</code>)

Concept	What it is	Where in code
<code>@public</code> / <code>@view</code> / <code>@fallback</code>	method markers; public writes + takes <code>ctx</code> , view reads only	<code>types.py:248</code> , <code>:284</code> , <code>:312</code>
State attribute	a type annotation → a trie-backed <code>Field</code> descriptor	<code>blueprint.py:79</code> ; <code>fields/field.py:76</code>
NanoHeader	the on-tx instruction: which contract, method, args, actions	<code>transaction/headers/nano_header.py:100</code>
NCAction	deposit/withdrawal/grant/acquire across the contract boundary	<code>types.py:359-491</code>
Context	immutable "who/what" passed to a public method	<code>context.py:35</code>
Runner	dispatches a call to a Blueprint method	<code>runner/runner.py:117</code> ; <code>execute_from_tx:163</code>
MeteredExecutor	fuel + memory bound (enforcement scaffolded)	<code>metered_exec.py:44</code>
Sandbox	restricted builtins; disabled <code>eval</code> / <code>open</code> / <code>import</code>	<code>custom_builtins.py</code>
Patricia/Merkle trie	order-independent, hash-linked state store	<code>storage/patricia_trie.py:94</code>
NCContractStorage	one contract's state (attrs/balances/metadata)	<code>storage/contract_storage.py:137</code>
NCBlockStorage	maps contract id → state root; its root = all NC state	<code>storage/block_storage.py:59</code>
State root	one hash fingerprinting state; per-block <code>nc_block_root_id</code>	<code>contract_storage.py:384</code> ; <code>block_storage.py:95</code>
Changes tracker	stages writes so a failed call rolls back	<code>storage/changes_tracker.py</code>
OnChainBlueprint	a tx carrying compressed Python Blueprint source	<code>on_chain_blueprint.py:155</code>

Concept	What it is	Where in code
Execution $\Rightarrow$ consensus	run at block confirm; failure $\rightarrow$ <code>NC_EXECUTION_FAIL_ID</code>	<code>execution/</code> <code>consensus_block_executor.py</code>
<code>NC_EXECUTION_FAIL_ID</code>	voiding marker <code>b'nc-fail'</code> for a failed contract call	<code>nanocontracts/__init__.py:25</code>
Feature gate	the subsystem is off unless <code>NANO_CONTRACTS</code> is active	<code>feature_activation/</code> <code>feature.py:33</code>

Nano-contracts are the node's second world: a small, sandboxed Python runtime stacked on the ledger you spent the book learning. A Blueprint is a class; a contract is an instance whose attributes live in a Merkle trie; a transaction's nano-header names a method to call; the Runner runs that method under a (designed) resource bound inside a real sandbox; and consensus — not the moment of arrival — is when the program actually executes, committing verifiable state on success and voiding the transaction with `NC_EXECUTION_FAIL_ID` on failure. The two design choices to carry forward are Python Blueprints over a bespoke VM (familiar language, careful jail) and a per-contract, per-block Merkle root (verifiable state in one hash). Note the live caveat: the metering enforcement hook is scaffolded but not yet wired on this branch — the budget is threaded through and the sandbox is active, but fuel is not yet decremented per opcode. The next chapter turns from contracts that hold money to the keys that own it: **Chapter 40 — Wallets and crypto**, where addresses, signatures, and the `nc_script` that authorizes every call you saw here finally get their full treatment.

1. A smart contract is a program stored on a ledger whose code is executed and agreed upon by every node in the network, so it can hold and move tokens according to its own public rules with no trusted operator. Hathor's variant is called a nano-contract. [↪](#)
2. Deterministic execution means the same inputs always produce exactly the same outputs on every machine. For consensus this is mandatory: if two nodes computed different results for the same contract call, they would disagree on the ledger. It rules out wall-clock time, true randomness, and reading external data. [↪](#)
3. Bytecode is a low-level, compact instruction format that a program is compiled into and that a virtual machine executes, rather than running the original source directly. Ethereum stores contract bytecode on-chain; Hathor stores Python source and runs it on the interpreter. [↪](#)
4. The EVM (Ethereum Virtual Machine) is Ethereum's purpose-built virtual machine: a stack-based interpreter with its own instruction set that every Ethereum node runs to execute contract bytecode. Hathor has no equivalent separate machine — contracts run on the node's own (sandboxed) Python interpreter. [↪](#)
5. Gas is Ethereum's unit of computational cost: each operation costs a fixed amount of gas, the transaction carries a gas budget, and execution aborts if the budget runs out. It is the mechanism that makes unbounded loops harmless. Hathor's equivalent budget is called fuel. [↪](#)

6. A Blueprint is Hathor's term for a contract template — a Python class defining the contract's state and methods. One Blueprint can back many independent contracts, just as one class can have many object instances. ↵
7. A metaclass is a class whose instances are themselves classes. Defining `class Counter(Blueprint)` runs the metaclass's code, which can inspect and rewrite the class before it exists. Hathor uses this to convert a Blueprint's type annotations into storage descriptors. ↵
8. A descriptor is a Python object that customizes what happens when an attribute is read, written, or deleted, by defining `__get__` / `__set__` / `__delete__`. Properties are descriptors. Hathor's `Field` is a descriptor that routes `self.count` to the trie instead of to memory. ↵
9. A header here is a self-describing block of extra bytes appended to a transaction's serialized form, carrying optional structured data. The nano-header carries the instructions for a contract call; a separate fee header carries fee data. ↵
10. A sequence number (seqnum) is a per-caller counter that must strictly increase with each call. It prevents replay: an attacker cannot resubmit a previously signed transaction, because its seqnum is no longer higher than the last one the node accepted. ↵
11. The proxy pattern (Chapter 3) places a stand-in object between a client and a real service so the stand-in can control, restrict, or mediate access. The `BlueprintEnvironment` is a proxy: contract code talks to it, and it forwards each request to the Runner under controlled, checked conditions. ↵
12. Re-entrancy is when a contract, mid-call, calls back into itself (often via another contract) before the first call finished. It is a classic source of exploits because the contract's state may be half-updated. Hathor forbids it by default and a method must explicitly opt in with `allow_reentrancy=True`. ↵
13. The halting problem is the proven impossibility of writing a general algorithm that decides, for any program and input, whether it will eventually stop or run forever. Its consequence here: the node cannot prove a contract terminates, so it must instead bound execution with a finite budget. ↵
14. Fuel is Hathor's name for the metered execution budget — the equivalent of Ethereum's gas. The design charges fuel per executed bytecode operation and aborts when it reaches zero; this branch threads the budget through but does not yet decrement it (see §39.A.7). ↵
15. A trie (or prefix tree) stores keys by spelling them out along a path from the root, so keys with a shared prefix share the top of their path. A radix/Patricia trie is the compressed form that collapses single-child chains into one node. ↵
16. A Merkle tree is a tree in which every node's identifier is a hash of its own data plus its children's identifiers. Any change anywhere alters the root hash, so the single root value is a tamper-evident fingerprint of the entire dataset. ↵
17. A state root is the Merkle root of a state store — one hash that uniquely fingerprints all of the state. Two parties can confirm their states are identical by comparing this one value. Hathor keeps a per-contract state root and a per-block root over all contracts. ↵
18. A Patricia trie is a compressed (radix) trie: chains of single-child nodes are merged into one node holding the whole shared substring, keeping the tree shallow and lookups efficient. Hathor combines this with Merkle hashing for verifiable state. ↵
19. A persistent data structure (in the data-structure sense, unrelated to disk persistence) is one where updates produce a new version while leaving every previous version intact and accessible. Hathor's trie copies a path on each write, so every past state root still names a complete snapshot — which is what makes reorgs cheap to undo. ↵
20. A reorg (reorganization) happens when consensus switches the canonical chain to a different branch, so some previously-canonical blocks become voided and others take their place. For nano-contracts this means re-executing the affected blocks from an earlier state root. ↵

Chapter 40 — Wallets, Keys & Crypto

What you'll learn in this chapter

- Why **owning a coin** in a blockchain means being able to prove a secret without revealing it, and how a public/private **keypair** plus a **digital signature** make that possible.
- The conceptual machinery underneath: **asymmetric cryptography**, **ECDSA** on the **secp256k1** curve, and what a signature actually proves.
- The **address pipeline** — how a public key becomes the short string you paste into a wallet (`hash160` → `base58check`), traced through `hathor/crypto/util.py`.
- The two wallet kinds in `hathor-core`: the **KeyPair** / **Wallet** (a bag of individual encrypted keys) and the **HDWallet** (one **seed** grows an entire tree of keys via **BIP32/BIP39**).
- How a wallet **tracks its UTXOs**, **selects inputs**, and **signs** a transaction so the resulting `input.data` satisfies the P2PKH script from Chapter 31.
- Why this code leans on the `cryptography` library and `pycoin` instead of hand-rolling crypto — and why that is the only defensible choice.

This chapter sits at the boundary between the node and the user. Everything before it — the vertex model, storage, verification, consensus — is about a node faithfully recording and agreeing on a ledger. But a ledger is only useful if people can move value on it, and moving value means answering one hard question: **how do I prove that a particular coin is mine, to a network of strangers, none of whom I trust and who do not trust me?** This chapter is the answer. It is also the one place in the book where cryptography stops being a black box.

We build the idea up from nothing — assume you have never met a public key — then walk the real code.

40.1 The problem: proving ownership without a trusted referee

Start from where Chapter 7 left you.

Recap — the UTXO and "ownership = a satisfiable lock" (full treatment in Ch. 7). In Hathor there are no account balances stored anywhere. Instead there are **UTXOs**: unspent transaction outputs. Each output carries a locking script — a small puzzle. To spend that output, a new transaction's **input** must supply data that satisfies the puzzle. So "owning 5 HTR" does not mean a row in a table says so; it means there exists an unspent output whose lock you, and only you, can open. **Ownership is the ability to satisfy a lock.** → full treatment in Ch. 7.

The locking script Hathor uses by default is **P2PKH** — "pay to public-key hash." Its puzzle, in words, is: "To spend me, show a public key that hashes to this fixed value, and show a valid signature over this transaction made with the matching private key." You met the machine that checks this puzzle in Chapter 31.

Recap — script evaluation and CHECKSIG (full treatment in Ch. 31). Verification runs each output's locking script together with the spending input's data on a small stack machine. For P2PKH the decisive opcode is `OP_CHECKSIG`: it pops a public key and a signature off the stack and checks the signature against the transaction's bytes. If the check fails, the whole transaction is invalid. The script evaluator never trusts the spender's word — it re-runs the cryptographic check itself. → full treatment in Ch. 31.

So the node side of ownership is settled: verification will demand a valid signature. What is missing is the other side — the side that **produces** that signature, manages the secrets it needs, and knows which outputs belong to "me." That is the wallet. And the entire scheme rests on a single piece of mathematics that lets you prove you know a secret without ever showing it. We have to understand that first.

Here is the naive approach and why it fails. Imagine ownership were proved with a password: the output says "spendable by anyone who knows the word `swordfish`," and to spend it you publish `swordfish`. This is hopeless. The moment you broadcast the spending transaction, the whole network sees `swordfish`, and any one of them can race to spend the next output you protect the same way. A shared-secret password reveals itself in the act of using it.

What we need is a secret you can **prove you possess** without ever transmitting it. That is exactly what asymmetric cryptography provides.

40.2 A primer on asymmetric cryptography and digital signatures

This section teaches the concept with neutral, generic framing. The Hathor code comes after.

40.2.1 Two keys instead of one

In ordinary ("symmetric") cryptography there is one secret key, shared by both sides — the `swordfish` problem. **Asymmetric**¹ cryptography uses two keys that are mathematically linked:

- a **private key**² — a large secret number you keep and never reveal;
- a **public key**³ — derived from the private key, which you can hand out freely.

The link is one-directional in a precise sense: it is easy to compute the public key from the private key, but — for the curve Hathor uses — infeasible to go backwards and recover the private key from the public key. The pair together is a **keypair**⁴.

A useful mental picture: the private key is a unique physical seal-stamp locked in your desk; the public key is a photograph of the seal's imprint that you publish. Anyone can look at the photograph and recognise a genuine imprint; nobody can carve a matching stamp from the photo alone.

40.2.2 What a digital signature is

A **digital signature**⁵ is a function of two things: a message (some bytes) and a private key. It produces a short blob of bytes — the signature — with two properties:

1. **Verifiable.** Anyone holding the matching public key, the message, and the signature can check that the signature was produced by the matching private key over exactly that message. This check is a `True / False` test; it needs nothing secret.
2. **Unforgeable.** Without the private key, you cannot produce a signature that passes the check — not for a new message, and not by tampering with the message of an existing valid signature (changing one byte of the message invalidates the signature).

Put those together and you have the answer to §40.1:

```
SIGNING (done by the owner, with the SECRET key)
  message + private_key  —sign→  signature

VERIFYING (done by anyone, with the PUBLIC key)
  message + public_key + signature —verify→  True / False
```

The owner signs with the private key; everyone else verifies with the public key. The secret never travels. This is the cryptographic spelling of "ownership = the ability to satisfy a lock": the lock says "produce a signature that verifies against this public key," and only the holder of the matching private key can.

There is one subtlety worth naming now because it recurs in the code. You almost never sign a raw message directly; you sign a **hash**⁶ of it. A hash function squeezes a message of any length into a fixed-size fingerprint, such that any change to the message changes the fingerprint unpredictably. Signing the hash rather than the whole message keeps signatures a fixed size and is how every real signature scheme works. Hold this thought — you will see Hathor hash the transaction before signing it.

40.2.3 ECDSA on secp256k1 — the concrete scheme, conceptually

"Asymmetric crypto" is a family. The specific scheme Hathor uses — the same one Bitcoin uses — is **ECDSA**⁷: the Elliptic Curve Digital Signature Algorithm. You do **not** need the mathematics to read this chapter; you need three facts.

1. **It is built on an elliptic curve**, a particular kind of mathematical structure on which "multiplication" is easy one way and "division" is infeasible the other way. That one-way-ness is what makes private→public easy and public→private infeasible.
2. **The specific curve is named `secp256k1`**. A "curve" here is just a fixed set of public parameters that every participant agrees on, the same way everyone agrees on the genesis block. Hathor, Bitcoin, and Ethereum all use `secp256k1`. Because it is standard, keys and signatures are interoperable across tools.
3. **The private key is a 256-bit number** (hence "256" in the name) and the public key is a point on that curve, which can be encoded compactly.

That is the whole conceptual budget. When the code says `ec.SECP256K1()` it is naming this curve; when it says `ec.ECDSA(...)` it is naming this signature scheme. Everything else is plumbing.

40.3 Localization

Three small packages collaborate, in a layered way. The lowest layer wraps a third-party crypto library; the middle layer derives keys; the top layer is the wallets.

```

hathor-core/
└─ hathor/
   └─ crypto/
      ├── __init__.py
      └── util.py
   └─ pycoin/
      ├── __init__.py
      └── htr.py
   └─ wallet/
      ├── __init__.py
      ├── base_wallet.py
      ├── keypair.py
      ├── wallet.py
      ├── hd_wallet.py
      ├── exceptions.py
      └── ... (resources/, util, etc.)

hathor_cli/
└─ generate_valid_words.py

```

← the crypto wrapper: key gen, address pipeline, hash160, base58check, pubkey (de)serialization

← registers the "HTR" network with the pycoin lib (BIP32 prefixes + version bytes) for HD derivation

← the wallet implementations

← exports Wallet, KeyPair, BaseWallet, HDWallet

← BaseWallet: UTXO tracking, input selection, signing

← KeyPair: one individual, encrypted private key

← Wallet: a file-backed bag of KeyPairs (the "keypair" wallet)

← HDWallet: one seed → a whole tree of keys (BIP32/BIP39)

← InsufficientFunds, PrivateKeyNotFound, WalletLocked, ...

← CLI tool: print a fresh BIP39 mnemonic (seed words)

The signature-producing code in P2PKH form lives one package over, in `hathor/transaction/scripts/p2pkh.py` (the P2PKH script class from Ch. 31). The wallet calls into it to assemble the `input.data` that satisfies the lock. We cite it where it is used.

Context. `hathor/crypto`, `hathor/pycoin`, and `hathor/wallet` are the node's key-management and signing surface. The verification pipeline (Ch. 31) demands valid signatures over outputs; this is the machinery that **produces** them, manages the secrets behind them, and tracks which outputs a given key controls. It is optional to a node's consensus role — a node can verify the whole ledger without holding any keys — but it is what lets a node, or a wallet built on the node, actually spend.

40.4 What it does and why it exists

A wallet, stripped to its job description, does four things:

1. **Holds secrets safely.** It stores private keys, ideally encrypted at rest, and hands out the matching public keys / addresses freely.
2. **Recognises incoming money.** When a new transaction lands, it checks each output: "is this output locked to one of my addresses?" If so, it is a UTXO the wallet now controls.

3. **Selects inputs to spend.** When you ask it to send N tokens, it picks enough of its own UTXOs to cover N (plus change), avoiding locked or already-pending ones.
4. **Signs.** It computes the bytes that must be signed (the **sighash**), signs them with the right private key for each chosen input, and packs `<signature>` `<public-key>` into the input's `data` field so the P2PKH lock will open.

Why does this live inside the full node at all, rather than only in a separate wallet app? Because `hathor-core` ships operator and developer tooling — generate a wallet, send a test transaction, run a faucet — and the node's HTTP API exposes a "send tokens" endpoint that needs to build and sign transactions server-side. A production node usually runs without an unlocked wallet; the wallet code is there for tooling, tests, and lightweight setups.

The package gives you **two flavours** of wallet, and the difference is entirely about where the keys come from:

- The **Wallet** (a.k.a. the "keypair" wallet) is a flat collection of independently-generated `KeyPair` objects, each an encrypted private key, persisted to a `keys.json` file. To back it up you must back up every key. (`hathor/wallet/wallet.py` .)
- The **HDWallet** ("hierarchical deterministic") generates every key, deterministically, from a single master **seed**, which is itself produced from a human-readable list of **mnemonic** words. To back it up you write down the words. One sheet of paper restores an unlimited key tree. (`hathor/wallet/hd_wallet.py` .)

Both share a large base class, `BaseWallet` , which holds all the UTXO-tracking, input-selection and signing logic that does not care where keys come from.

40.5 The concepts it rests on

Three earlier threads converge here. Rather than re-teach them, here are the pointers.

Recap — the cryptography dependency (foundations in Ch. 13). Hathor does not implement elliptic-curve maths itself. It depends on `cryptography` , the standard, audited Python library backed by OpenSSL, declared in `pyproject.toml` . All low-level key generation, signing, and serialization in `hathor/crypto/util.py` are thin calls into `cryptography.hazmat.primitives.asymmetric.ec` . The "hazmat" in that import path is the library authors' own warning label — "hazardous materials" — flagging that these are low-level primitives to be wrapped carefully, which is exactly what `crypto/util.py` does. → dependency manifest in Ch. 13 / Appendix B.

Recap — TxInput / TxOutput and the address (full treatment in Ch. 25).

A transaction output is a `TxOutput` carrying a `value`, a `token_data` byte, and a `script` (the lock). A transaction input is a `TxInput` naming a previous output by `(tx_id, index)` and carrying a `data` field (the lock-opening proof). The **address** you paste into a wallet is not stored in the output directly — it is encoded inside the output's `script`. The wallet's job is to translate between "address" and "script," and to fill in each input's `data`. → full treatment in Ch. 25.

Recap — the UTXO set and the index (full treatment in Ch. 7 & 28). "What can this wallet spend?" is answered by the set of unspent outputs locked to its keys. The node maintains UTXO/address indexes for fast lookup (Ch. 28), but the wallet also keeps its own in-memory bookkeeping (`unspent_txs`, `spent_txs`, ...) updated from pub-sub events, because a wallet must track its own outputs precisely, including ones in intermediate "maybe spent" states. → full treatment in Ch. 7 (the model) & Ch. 28 (the index).

40.6 The code, walked

We go bottom-up: first the address pipeline (pure crypto, no wallet), then the two key sources, then the tracking-and-signing logic that sits above both.

40.6.1 The address pipeline: public key → address

An **address** is a short, copy-pasteable, typo-resistant string that stands in for a public key. You never paste a raw public key; you paste an address derived from it. The derivation is a fixed pipeline, and it lives entirely in `hathor/crypto/util.py`.

Generically, the pipeline is:

```
public key
  | (1) serialize to compact bytes
  ▼
public key bytes
  | (2) hash160 = ripemd160(sha256(bytes)) → 20 bytes
  ▼
public-key hash
  | (3) prepend a 1-byte "version" tag
  ▼
version || hash → 21 bytes
  | (4) append a 4-byte checksum = first 4 bytes of sha256(sha256(version||hash))
  ▼
version || hash || checksum → 25 bytes (raw address)
  | (5) base58-encode the 25 bytes
  ▼
base58check address (the string you paste)
```

Step (2) is **hash160**⁹ — sha256 followed by ripemd160. Hashing the public key (rather than publishing it directly) shortens the address to 20 bytes and adds a layer of protection: the public key is only revealed when you spend, not when you receive. Step (4)'s checksum is what makes addresses typo-resistant — change one character and the recomputed checksum won't match, so a mistyped address is rejected before any coins move. The whole "version byte + payload + double-sha256 checksum, base58-encoded" envelope is called **base58check**¹⁰.

Now the real code. Hash160 is defined with a fallback for older Python builds that lack `ripemd160` in `hashlib`, in which case it borrows pycoin's pure-Python implementation:

```
# hathor/crypto/util.py:62
def get_hash160(public_key_bytes: bytes) -> bytes:
    """The input is hashed twice: first with SHA-256 and then with RIPEMD-160"""
    key_hash = hashlib.sha256(public_key_bytes)
    h = hashlib.new('ripemd160')
    h.update(key_hash.digest())
    return h.digest()
```

The public key is first serialized to compact bytes by `get_public_key_bytes_compressed` (`crypto/util.py:180`), which asks `cryptography` for the X9.62 compressed point encoding. The full address build is `get_address_from_public_key_hash`:

```
# hathor/crypto/util.py:120
def get_address_from_public_key_hash(public_key_hash: bytes, version_byte: Optional[bytes] = None) -> bytes:
    settings = get_global_settings()
    address = b''
    actual_version_byte: bytes = version_byte if version_byte is not None else settings.P2PKH_VERSION_BYTE
    address += actual_version_byte # (3) version tag
    address += public_key_hash # the 20-byte hash160
    checksum = get_checksum(address) # (4) checksum over version||hash
    address += checksum
    return address
```

The checksum is the classic double-sha256, first four bytes (`crypto/util.py:144`):

```
# hathor/crypto/util.py:144
def get_checksum(address_bytes: bytes) -> bytes:
    """Calculate double sha256 of address and gets first 4 bytes"""
    return hashlib.sha256(hashlib.sha256(address_bytes).digest()).digest()[:4]
```

The final base58 step is a one-liner wrapping the `base58` library — e.g.

`get_address_b58_from_public_key_hash` (`crypto/util.py:107`) calls `base58.b58encode(...)`. The whole chain from a public-key object to a base58 string is `get_address_b58_from_public_key` (`crypto/util.py:96`).

Going the other way, `decode_address` (`crypto/util.py:241`) base58-decodes the string, asserts it is exactly 25 bytes, **re-computes the checksum and compares** it, and raises `InvalidAddress` on mismatch. This is the typo-catching guarantee in action: a corrupted address can essentially never pass the checksum by accident.

The **version byte** (step 3) is how the same pipeline distinguishes address types.

`P2PKH_VERSION_BYTE` tags ordinary single-key addresses; `MULTISIG_VERSION_BYTE` tags multisig addresses (`crypto/util.py:202`, `get_address_b58_from_redeem_script_hash`), which hash a redeem script instead of a single public key. The version byte also differs by network (mainnet vs testnet), so a testnet address visibly differs from a mainnet one — a guard against sending real coins to a test address.

One detail to internalise: `secp256k1` is named explicitly when a public key is reconstructed from bytes:

```
# hathor/crypto/util.py:191
def get_public_key_from_bytes_compressed(public_key_bytes: bytes) -> ec.EllipticCurvePublicKey:
    return ec.EllipticCurvePublicKey.from_encoded_point(ec.SECP256K1(), public_key_bytes)
```

That `ec.SECP256K1()` is the concrete curve from §40.2.3, sitting in real code.

40.6.2 KeyPair — one individual, encrypted key

The smallest unit of key management is `KeyPair` (`hathor/wallet/keypair.py:26`). It is not a wallet; it is one private key, stored **encrypted**, plus the address it derives to:

```
# hathor/wallet/keypair.py:31
def __init__(self, private_key_bytes: Optional[bytes] = None, address: Optional[str] = None,
             used: bool = False) -> None:
    """Holds the address in base58 and the encrypted bytes of the private key"""
    self.private_key_bytes = private_key_bytes # ENCRYPTED bytes, not the raw key
    self.address = address
    self.used = used
    self._cache_priv_key_unlock = None # decrypted key, cached only after unlock
```

Creating a fresh key is where `secp256k1` and the `cryptography` library meet (`keypair.py:116`):

```
# hathor/wallet/keypair.py:123 (inside KeyPair.create)
new_key = ec.generate_private_key(ec.SECP256K1(), default_backend())
private_key_bytes = get_private_key_bytes(
    new_key, encryption_algorithm=serialization.BestAvailableEncryption(password))
address = get_address_b58_from_public_key(new_key.public_key())
```

Three things happen in those lines. A random `secp256k1` private key is generated. It is serialized to bytes encrypted under a password (`BestAvailableEncryption`) — so what `KeyPair` holds on disk is ciphertext, useless without the password. And the matching address is computed once via the §40.6.1 pipeline and stored alongside.

To use the key you must decrypt it, which requires the password (`keypair.py:61`). `get_private_key` decrypts on first call and caches the decrypted object in `_cache_priv_key_unlock`; `clear_cache` (`keypair.py:56`) wipes that cache when the wallet locks. A wrong password raises `IncorrectPassword`; no password at all raises `WalletLocked`. This is the at-rest-encryption story: the secret is only ever decrypted into memory while the wallet is explicitly unlocked.

`KeyPair` can also produce a P2PKH input proof on its own (`keypair.py:94`):

```
# hathor/wallet/keypair.py:94
def p2pkh_create_input_data(self, password: bytes, data: bytes) -> bytes:
    """Return a script input to solve the p2pkh script generated by this key pair."""
    private_key = self.get_private_key(password)
    public_key = private_key.public_key()
    public_key_bytes = get_public_key_bytes_compressed(public_key)
    signature = private_key.sign(data, ec.ECDSA(hashes.SHA256())) # ECDSA over SHA-256
    script_input = P2PKH.create_input_data(public_key_bytes, signature)
    return script_input
```

There is the full signature in miniature: decrypt the private key, derive the public key, **sign with ECDSA/SHA-256**, then hand both the public key and the signature to `P2PKH.create_input_data` to pack into the input. We will see this same shape, generalised across a whole transaction, in §40.6.5.

40.6.3 Wallet — a file-backed bag of KeyPairs

`Wallet` (`hathor/wallet/wallet.py:32`) is the "keypair" wallet: a dictionary of `KeyPair`s keyed by base58 address, persisted to `keys.json`. Its responsibilities beyond `BaseWallet` are local: generate new keys in batches (`generate_keys` , `wallet.py:173`), hand out unused addresses (`get_unused_address` , `wallet.py:148`), read/write the keys file (`read_keys_from_file` , `wallet.py:78`), and unlock/lock by holding or clearing the password (`wallet.py:116` , `wallet.py:141`).

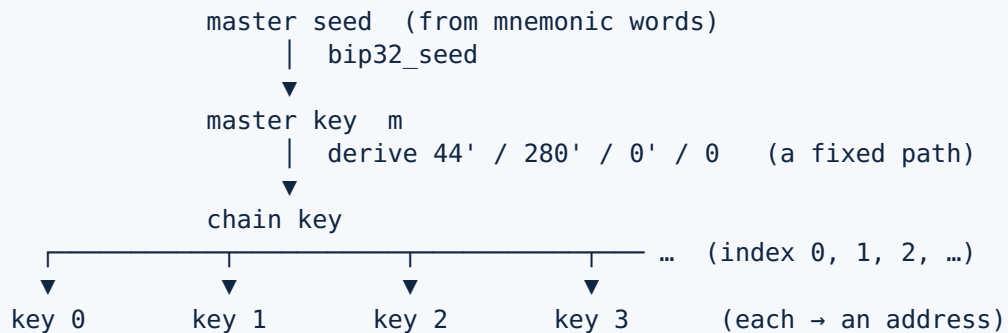
The `used` flag and the `unused_keys` set implement a privacy convention: each address is meant to be used once. When tokens arrive at an address, `tokens_received` (`wallet.py:194`) marks it used and removes it from `unused_keys` ; when you need a fresh receive address, `get_unused_address` pops an unused one (generating a new batch if the wallet is unlocked and the pool is empty). Reusing addresses links your transactions together on the public ledger, so wallets cycle through fresh addresses by default.

Its signing helper is `get_input_aux_data` (`wallet.py:208`), which we look at in §40.6.5 alongside the HD wallet's.

40.6.4 HDWallet — one seed, a whole tree

The `Wallet` has a backup problem: every key is independent random data, so a complete backup means copying every key, and a backup taken yesterday misses keys generated today. **Hierarchical-deterministic** wallets solve this. The idea, generically:

Start from one master secret (the **seed**). Define a deterministic function `derive(parent_key, index) → child_key`. Apply it repeatedly to grow a **tree** of keys, where the path from the root (e.g. account 0 / chain 0 / key 5) names each key. Because everything is derived from the one seed by a fixed function, **the seed alone regenerates the entire tree** — past, present, and future keys.



Two standards make this interoperable across every wallet vendor:

- **BIP32**¹¹ defines the derivation function and the tree (the `derive(parent, index)` above).
- **BIP39**¹² defines how a human-readable list of words — the **mnemonic**¹³ — maps to the binary seed. Twelve to twenty-four common English words are your master secret, in a form you can write on paper and read back without transcription errors.

Hathor's `HDWallet` (`hathor/wallet/hd_wallet.py:54`) implements exactly this, leaning on two libraries: the `mnemonic` package for BIP39, and `pycoin` for BIP32 derivation. The seed-to-tree construction is in `_manually_initialize` (`hd_wallet.py:122`):

```

# hathor/wallet/hd_wallet.py:126 (inside _manually_initialize)
self.mnemonic = Mnemonic(self.language)
...
seed = self.mnemonic.to_seed(self.words, self.passphrase.decode('utf-8')) # BIP39: words → seed

from pycoin.networks.registry import network_for_netcode
_register_pycoin_networks()
network = network_for_netcode('htr')
key = network.keys.bip32_seed(seed) # BIP32: seed → master key

# Chain path = 44'/280'/0'/0
self.chain_key = key.subkey_for_path('44H/280H/0H/0')
for key in self.chain_key.children(self.initial_key_generation, 0, False):
    self._key_generated(key, key.child_index())

```

Read the derivation path `44H/280H/0H/0` — the `H` means "hardened," a BIP32 detail that strengthens a derivation step. The comment in the source spells it out: `44'` is the **BIP44** purpose tag, `280'` is the **coin type** that uniquely identifies Hathor (Bitcoin is `0`, Hathor is `280`), `0'` is the account number, and the final `0` is the external chain. So

Hathor keys live at a well-defined, interoperable location in the BIP44 hierarchy — another wallet that knows "coin type 280" can restore a Hathor HDWallet from the same words.

The `htr` network code is what `hathor/pycoin/htr.py` registers — this is the entire purpose of the `pycoin` package:

```
# hathor/pycoin/htr.py:21
network = create_bitcoinish_network(
    symbol='HTR', network_name='Hathor', subnet_name='mainnet',
    wif_prefix_hex='80',
    address_prefix_hex=settings.P2PKH_VERSION_BYTE.hex(),
    pay_to_script_prefix_hex=settings.MULTISIG_VERSION_BYTE.hex(),
    bip32_prv_prefix_hex='0488ade4', bip32_pub_prefix_hex='0488B21E',
)
```

`pycoin` natively understands Bitcoin-family networks; this call teaches it Hathor's version bytes (the same `P2PKH_VERSION_BYTE` from §40.6.1) so the addresses it derives match Hathor's address format. `_register_pycoin_networks` (`hd_wallet.py:38`) wires this module into pycoin's network registry via the `PYCOIN_NETWORK_PATHS` environment variable.

Note one consequence, flagged in the source by a `TODO` (`hd_wallet.py:29`): pycoin's BIP32 keys carry their own ECDSA implementation, separate from OpenSSL-backed `cryptography`. So the **HD wallet signs through pycoin**, while the **keypair Wallet signs through cryptography**. The two wallet kinds reach the same `secp256k1`/ECDSA result by different libraries — visible in the next section.

The **gap limit** (`hd_wallet.py:103`, default 20) is the HD-wallet rule that bounds how many unused addresses to scan ahead. Because keys are generated lazily, a wallet recovered from a seed must generate addresses and check each for history; the gap limit says "if you hit 20 consecutive empty addresses, stop — assume the rest are empty too." `tokens_received` (`hd_wallet.py:205`) keeps the buffer of pre-generated keys topped up so syncing never runs out of look-ahead addresses.

40.6.5 Tracking UTXOs, selecting inputs, and signing — the shared `BaseWallet`

Everything above is about where keys come from. The logic that uses the keys — recognise money, pick inputs, sign — is in `BaseWallet` (`hathor/wallet/base_wallet.py:71`) and is shared by both wallet kinds.

Tracking. The wallet keeps several dictionaries of its outputs: `unspent_txs`, `maybe_spent_txs`, `spent_txs`, plus voided variants (`base_wallet.py:107–120`). It learns about new transactions through pub-sub.

Recap — pub-sub (full treatment in Ch. 30). Components announce events ("a new transaction is being processed," "a transaction's consensus status changed") on a `PubSubManager`; interested components subscribe to event types and get called back. The wallet is a subscriber. → full treatment in Ch. 30.

There are two subscription paths, and it pays to keep them distinct. The **manager** subscribes the wallet to `NETWORK_NEW_TX_PROCESSING`, routing each new vertex to `wallet.on_new_tx` (`hathor/manager.py:261`). Separately, `BaseWallet.start` subscribes to `CONSENSUS_TX_UPDATE` and `CONSENSUS_TX_REMOVED`, routing to `on_tx_update` (`base_wallet.py:133`, `base_wallet.py:182`). The first path notices new outputs and spends; the second reacts to consensus changes — a transaction becoming voided or winning a conflict — so the wallet can move outputs between its spent/unspent/voided buckets.

`on_new_tx` (`base_wallet.py:527`) is the recognise-money routine. For each output it parses the locking script and asks "is this address one of mine?" (`script_type_out.address not in self.keys`). If yes, it records a new `UnspentTx` and fires `WALLET_OUTPUT_RECEIVED`. For each input, it checks whether the spent output was one of ours and, if so, moves it to `spent_txs` and fires `WALLET_INPUT_SPENT`. Voided transactions are skipped (`base_wallet.py:535`).

Selecting inputs. When you want to send an amount, `get_inputs_from_amount` (`base_wallet.py:469`) walks the wallet's unspent outputs for the right token and accumulates them until they cover the requested amount, skipping outputs that are time-locked, are token authorities, or are freshly-mined rewards still within the reward-lock window (`can_spend_block`, `base_wallet.py:519`). The source is candid about the algorithm (`base_wallet.py:475`): "This is a very simple algorithm, so it does not try to find the best combination of inputs" — it is first-fit, not optimal coin-selection. If the unspent set can't cover the amount it raises `InsufficientFunds`. Chosen UTXOs are moved into `maybe_spent_txs` immediately so a second concurrent send can't pick them again. Change is handled by `handle_change_tx` (`base_wallet.py:444`): if the inputs over-cover the amount, it adds an output back to a fresh wallet address for the difference.

Signing. This is the payoff — where everything in §40.2 becomes bytes. The end-to-end builder is `prepare_transaction` (`base_wallet.py:214`):

```
# hathor/wallet/base_wallet.py:255 (inside prepare_transaction)
tx = cls(inputs=tx_inputs, outputs=tx_outputs, tokens=tokens, timestamp=timestamp)
data_to_sign = tx.get_sighash_all()

for txin, privkey in zip(tx.inputs, private_keys):
    public_key_bytes, signature = self.get_input_aux_data(data_to_sign, privkey)
    txin.data = P2PKH.create_input_data(public_key_bytes, signature)
```

Three steps, each worth naming:

1. **Compute the sighash.** `tx.get_sighash_all()` (`hathor/transaction/transaction.py:231`) serializes the transaction's inputs, outputs, and token list into bytes — the bytes that the signature commits to. The **sighash**¹⁵ ("signature hash") is what you sign: by signing it, you sign the entire shape of the transaction, so nobody can redirect an output or change an amount without invalidating your signature. (The method excludes per-input data and caches its result, since it is the same for every input — `transaction.py:237`.)
2. **Sign, per input.** `get_input_aux_data` is the abstract hook (`base_wallet.py:211`) each wallet kind implements. For the keypair `Wallet` (`hathor/wallet/wallet.py:208`):

```
# hathor/wallet/wallet.py:220
public_key_bytes = get_public_key_bytes_compressed(private_key.public_key())
hashed_data = hashlib.sha256(data_to_sign).digest()
signature = private_key.sign(hashed_data, ec.ECDSA(hashlib.SHA256()))
return public_key_bytes, signature
```

For the `HDWallet`, signing goes through `pycoin` instead (`hathor/wallet/hd_wallet.py:320`):

```
# hathor/wallet/hd_wallet.py:332
prehashed_msg = hashlib.sha256(hashlib.sha256(data_to_sign).digest()).digest()
signature = private_key.sign(prehashed_msg)
return private_key.sec(), signature
```

Both return `(public_key_bytes, signature)` — the public key in compressed form, and an ECDSA signature over the sighash. (The exact hashing differs by library: `pycoin`'s `sign` expects an already-hashed message, so the HD path hashes explicitly; `cryptography`'s `ECDSA(SHA256)` hashes internally.)

1. **Pack the input data.** `P2PKH.create_input_data(public_key_bytes, signature)` (`hathor/transaction/scripts/p2pkh.py:94`) assembles the input's `data` field by pushing the signature and then the public key onto a script:

```
# hathor/transaction/scripts/p2pkh.py:94
@classmethod
def create_input_data(cls, public_key_bytes: bytes, signature: bytes) -> bytes:
    s = HathorScript()
    s.pushData(signature)
    s.pushData(public_key_bytes)
    return s.data
```

This closes the loop with Chapter 31. The locking script that

`P2PKH.create_output_script` (`p2pkh.py:70`) put on the output was `OP_DUP OP_HASH160 <pubKeyHash> OP_EQUALVERIFY OP_CHECKSIG`. The input data we just built is `<signature> <publicKey>`. When verification (Ch. 31) runs them together on the stack machine, `OP_HASH160` plus `OP_EQUALVERIFY` confirm the public key hashes to the address baked into the output, and `OP_CHECKSIG` confirms the signature verifies against that public key over the sighash. The lock opens **only** for the holder of the matching private key — the whole point of §40.2, now mechanical.

For signing an already-built transaction (rather than building one from scratch), `sign_transaction` (`base_wallet.py:425`) does the same per-input loop, but uses `match_inputs` (`base_wallet.py:940`) to sign only those inputs that spend the wallet's own outputs — leaving other parties' inputs untouched, which is what makes collaborative/multisig transactions possible.

40.6.6 Multisig, briefly

The same address machinery extends to **multisig** (multiple signatures). A multisig output locks to the hash of a redeem script of the form `<M> <pubkey1> ... <pubkeyN> <N> OP_CHECKMULTISIG` — "any M of these N keys must sign." The address is built by `get_address_b58_from_redeem_script_hash` (`crypto/util.py:202`) using `MULTISIG_VERSION_BYTE`, and the script lives in `hathor/transaction/scripts/multi_sig.py`. Its `create_input_data` (`multi_sig.py:114`) packs the M signatures plus the redeem script, mirroring P2PKH's `<sig> <pubkey>` but with several signatures. The conceptual leap is small once P2PKH is understood: replace "one key must sign" with "M of N keys must sign," and reuse the same sighash, the same ECDSA, the same base58check envelope.

40.6.7 The CLI seed-words generator

The mnemonic that seeds an `HDWallet` has to come from somewhere safe. `hathor_cli/generate_valid_words.py` is the operator tool that prints a fresh BIP39 mnemonic:

```
# hathor_cli/generate_valid_words.py:18
def generate_words(language: str = 'english', count: int = 24) -> str:
    from mnemonic import Mnemonic
    mnemonic = Mnemonic(language)
    return mnemonic.generate(strength=int(count * 10.67))
```

The `count * 10.67` converts a word count (12–24) into the BIP39 entropy strength in bits — the same conversion `HDWallet.unlock` uses when it auto-generates words (`hd_wallet.py:295`). Run this command, write the words down, and you hold the master secret for an entire key tree.

40.7 Why these libraries, not hand-rolled crypto

This is a Track-C trade-off, and for cryptography the trade-off is unusually one-sided.

The first rule of cryptographic engineering is: do not implement your own. Not because it is hard to make code that produces a signature — that part is a few lines — but because it is brutally hard to make code that produces a signature without leaking the private key through a side channel. A naive ECDSA implementation can leak bits of the secret through how long it takes to run, through which memory it touches, or through subtle math errors that only bite on rare inputs. These bugs do not show up in tests; they show up as stolen funds. The industry's hard-won standard libraries have been audited for exactly these failure modes over years. So `hathor/crypto/util.py` is, deliberately, a thin wrapper over `cryptography` (OpenSSL-backed). It contributes Hathor-specific composition — the address pipeline, the version bytes, the encoding — and delegates every raw primitive (key generation, ECDSA sign/verify, point serialization) to the audited library.

Why `cryptography` ? It is the de-facto standard asymmetric-crypto library in the Python ecosystem, backed by OpenSSL, widely audited, and exposing exactly the `secp256k1` / ECDSA primitives Hathor needs.

Why `pycoin` on top, for the HD wallet? Because **BIP32/BIP39 are standards, and the value of a standard is interoperability.** A user must be able to take their 24 words to any compliant wallet and recover the same keys. Re-implementing BIP32 derivation by hand would risk a subtle incompatibility (a wrong hardened-derivation step, a different seed-stretching) that would silently produce different keys — a backup that restores the wrong wallet is worse than no backup. `pycoin` is a mature, Bitcoin-family library that implements these BIPs; `hathor/pycoin/htr.py` only teaches it Hathor's version bytes. The cost is the seam noted in §40.6.4: `pycoin` carries its own ECDSA (the `TODD` at `hd_wallet.py:29` flags this as worth a security review), so the HD path does not go through OpenSSL. That is the price of standards-compliant derivation, accepted knowingly and marked in the code.

Why `mnemonic` ? Same reasoning: BIP39's word list and seed-derivation are a standard, and the `mnemonic` package is its canonical Python implementation. Interoperability of the recovery words is the whole point.

The pattern across all three is identical: **own the composition, delegate the cryptography.** That is the correct posture for any application that touches keys.

40.8 How it plugs into the node's lifecycle

The wallet is an optional collaborator the builder may attach to the `HathorManager`. When present, the wiring is small and follows the lifecycle you met in Chapter 29.

Recap — manager start/stop (full treatment in Ch. 29). The `HathorManager` is the central coordinator; `start()` brings subsystems online in order and `stop()` tears them down. Collaborators like the wallet are injected at construction and started/stopped by the manager. → full treatment in Ch. 29.

- **Injection.** At construction the manager gives the wallet its `pubsub` and `reactor` and subscribes it (`hathor/manager.py:221–225`). `_subscribe_wallet` (`manager.py:255`) routes `NETWORK_NEW_TX_PROCESSING` events to `wallet.on_new_tx`, so every accepted vertex passes under the wallet's eye.
- **Start.** Inside `manager.start()`, `wallet.start()` is called (`manager.py:334`), which subscribes the wallet to the consensus-update events (Ch. 30) and schedules the periodic `maybe_spent` cleanup timer (`base_wallet.py:152`).
- **Initialize.** During initialization the manager calls `wallet._manually_initialize()` (`manager.py:448`) — for a `Wallet` this loads `keys.json`; for an `HDWallet` it derives the seed and the first batch of keys.
- **Stop.** On shutdown `wallet.stop()` unsubscribes it from pub-sub (`manager.py:369`, `base_wallet.py:161`).
- **Signing in service of clients.** When the node's HTTP "send tokens" path builds a transaction, it asks the wallet for an unused address and to sign — the same `prepare_transaction` / `sign_transaction` flow from §40.6.5. The signed transaction then re-enters the ordinary ingestion pipeline (Ch. 33): it is **verified** (Ch. 31), where the very signature the wallet produced is checked by `OP_CHECKSIG`, then run through **consensus** (Ch. 32) and stored. The wallet produces proofs; verification consumes them. Producer and consumer, finally joined.

Recap

Concern	Where it lives	Central type / function
Asymmetric keypair + ECDSA on secp256k1	<code>cryptography</code> lib, called from <code>hathor/crypto/util.py</code>	<code>ec.SECP256K1()</code> , <code>ec.ECDSA(...)</code>
Address pipeline (hash160 → base58check)	<code>hathor/crypto/util.py:62, :120, :144</code>	<code>get_hash160</code> , <code>get_address_from_public_key_hash</code> , <code>get_checksum</code>
One encrypted individual key	<code>hathor/wallet/keypair.py:26</code>	<code>KeyPair</code> (<code>create:116</code> , <code>get_private_key:61</code>)

Concern	Where it lives	Central type / function
File-backed bag of keys	<code>hathor/wallet/wallet.py:32</code>	<code>Wallet</code>
Seed → tree of keys (BIP32/BIP39)	<code>hathor/wallet/hd_wallet.py:54</code>	<code>HDWallet (_manually_initialize: 122)</code>
BIP32 network registration	<code>hathor/pycoin/htr.py:21</code>	<code>create_bitcoinish_network</code>
BIP39 mnemonic CLI	<code>hathor_cli/generate_valid_words.py:18</code>	<code>generate_words</code>
UTXO tracking via pub-sub	<code>hathor/wallet/base_wallet.py:527</code>	<code>BaseWallet.on_new_tx</code>
Input selection (first-fit)	<code>hathor/wallet/base_wallet.py:469</code>	<code>get_inputs_from_amount</code>
The sighash (bytes to sign)	<code>hathor/transaction/transaction.py:231</code>	<code>get_sighash_all</code>
Sign per input	<code>base_wallet.py:211 ; impls wallet.py:208 , hd_wallet.py:320</code>	<code>get_input_aux_data</code>
Pack <code><sig></code> <code><pubkey></code> into the input	<code>hathor/transaction/scripts/p2pkh.py:94</code>	<code>P2PKH.create_input_data</code>
Manager wiring	<code>hathor/manager.py:221 , : 255 , :334 , :448</code>	<code>_subscribe_wallet</code> , start/stop

Ownership in a blockchain is not a fact recorded in a table; it is the ability to produce a proof. This chapter showed the full circle: a private key is a secret number on the `secp256k1` curve; its public key derives, via `hash160` and `base58check` , into the address that locks an output; spending that output means signing the transaction's sighash with the matching private key and packing `<signature>` `<publicKey>` where the lock can check it. The two wallet kinds differ only in where keys come from — a flat encrypted bag (`Wallet`) or a single seed grown into a tree (`HDWallet`) — and both delegate the cryptography to audited libraries, owning only the composition. The wallet produces the proofs that verification (Ch. 31) demands.

The next chapter steps away from the ledger and into operations: **Chapter 41 — Runtime Control (`hathor/sysctl`)**, the control socket that lets an operator query and tune a running node without restarting it.

1. **Asymmetric cryptography** (also "public-key cryptography") uses a pair of mathematically linked keys — one private, one public — instead of a single shared secret. What one key does (e.g. sign), only the other can verify; you cannot derive the private key from the public key. ↩
2. A **private key** is a large secret number that you never reveal. Possessing it is what defines ownership; anyone who learns it can spend your coins. In Hathor it is a 256-bit `secp256k1` key, stored encrypted at rest. ↩
3. A **public key** is derived from the private key and may be shared freely. It is used to verify signatures and to derive your address. Computing it from the private key is easy; the reverse is infeasible. ↩
4. A **keypair** is the matched (private, public) pair generated together. The two are useless apart: the private key signs, the public key verifies, and only a matched pair agrees. ↩
5. A **digital signature** is a blob of bytes produced from a message and a private key. Anyone with the message, the signature, and the matching public key can confirm the signer held the private key and that the message is unaltered — without the signer revealing the key. ↩
6. A **hash function** maps a message of any size to a fixed-size fingerprint, such that any change to the message changes the fingerprint unpredictably and you cannot run it backwards. Hathor uses SHA-256 and RIPEMD-160. Signatures are computed over a hash of the message, not the raw message. ↩
7. **ECDSA** = Elliptic Curve Digital Signature Algorithm. The specific signature scheme Hathor (and Bitcoin) use. "Elliptic curve" is the kind of one-way maths that makes private→public easy and the reverse infeasible. You can read this chapter without the maths. ↩
8. **secp256k1** is the name of the specific elliptic curve — a fixed, public set of parameters — that Hathor, Bitcoin, and Ethereum all use. Because it is a shared standard, keys and signatures are interoperable across tools. ↩
9. **hash160** is SHA-256 followed by RIPEMD-160, producing a 20-byte fingerprint of a public key. Hashing the public key shortens the address and hides the public key until the output is spent. ↩
10. **base58check** is an encoding for addresses: a version byte, then the payload, then a 4-byte checksum (`sha256(sha256(...))` truncated), all encoded in base58 (digits/letters minus look-alikes like 0/O, I/l). The checksum makes mistyped addresses detectable. ↩
11. **BIP32** is the standard defining hierarchical-deterministic key derivation: a function that grows a tree of child keys from one master key, so a single master secret regenerates every key. ("BIP" = Bitcoin Improvement Proposal.) ↩
12. **BIP39** is the standard mapping a list of memorable words (the mnemonic) to the binary seed that BIP32 starts from. It is why a wallet backup can be 24 words on paper instead of raw bytes. ↩
13. A **mnemonic** here is the ordered list of BIP39 words (12–24 of them) that encodes a wallet's seed. Anyone with the words can regenerate the entire key tree, so the words must be kept as secret as a private key. ↩
14. A **seed** is the master secret (a block of bytes) from which an HD wallet derives all its keys. It is produced from the BIP39 mnemonic (plus an optional passphrase) and fed into BIP32 derivation. ↩
15. The **sighash** ("signature hash") is the canonical serialization of a transaction's inputs, outputs, and tokens — the exact bytes a signature commits to. Signing the sighash means signing the whole shape of the transaction, so it cannot be altered after signing without breaking the signature. ↩

Chapter 41 — Runtime Control: Sysctl

What you'll learn in this chapter

- Why a long-running daemon needs a way to inspect and tune itself while running, and why "stop, edit a config file, restart" is not good enough for a mainnet node.
- The mental model behind `hathor/sysctl/`: a **tree of named parameters**, each with an optional **getter** and **setter**, addressed by a **dotted path** like `p2p.max_enabled_sync` — directly modelled on Linux's own `sysctl`.
- How the base `Sysctl` class stores that tree, how the **runner** parses a one-line `path / path=value` command, and how `get` and `set` are dispatched down the tree.
- How the tree is **wired to the live subsystems** (the P2P manager, storage, feature activation, ...) by the `SysctlBuilder`, and how it is exposed to the outside world — over a Unix socket or TCP, or through the `SIGUSR2` signal path.
- A few **real parameters** walked end to end, so you can read and write a knob on a running node yourself.

A full node is a daemon¹: you start it once and it runs for weeks. Over those weeks the operator's needs change. A peer is misbehaving and you want to cut it off. Sync is hammering a slow disk and you want fewer peers syncing at once. You suspect a memory leak and want to start the profiler for sixty seconds. None of these are worth a restart — a mainnet node restart means dropping every connection, re-reading the whole database, and being offline for minutes while it re-initializes. You need a side door: a way to reach into the running process, read a value, and change it, then close the door and leave the node running.

`hathor/sysctl/` is that side door. This chapter is short — the package is small (about seventeen files, most of them tiny) — but the idea is worth holding clearly, because it recurs: a **uniform tree of named, addressable parameters**, decoupled from the subsystems it controls and from the wire that carries the commands.

41.1 Localization

`hathor/sysctl/` sits in the infrastructure group of the codebase (Chapter 0, §0.4) — alongside the builder, the reactor abstraction, and the observability tools. It is operator-facing plumbing, not part of the ledger's data model.

```
hathor-core/
├── hathor/
│   ├── manager.py           ← HathorManager: the subsystems sysctl reads/tunes (Ch 29)
│   ├── builder/
│   │   └── sysctl_builder.py ← assembles the tree, wiring nodes to live subsystems (Ch 29)
│   └── sysctl/              ◀ YOU ARE HERE
│       ├── sysctl.py        ← Sysctl base class: a node in the command tree
│       ├── runner.py        ← SysctlRunner: parse one line, dispatch get/set
│       ├── protocol.py      ← SysctlProtocol: the line-based socket protocol
│       ├── factory.py       ← SysctlFactory: builds a protocol per connection
│       ├── exception.py     ← the sysctl error hierarchy
│       ├── init_file_loader.py ← apply a file of commands at startup
│       ├── core/manager.py  ← HathorManagerSysctl (profiler, debugger, ipython)
│       ├── p2p/manager.py   ← ConnectionsManagerSysctl (sync knobs, kill peers)
│       ├── storage/manager.py ← StorageSysctl (rocksdb flush/stats)
│       ├── feature_activation/... ← FeatureActivationSysctl (signalling support)
│       └── websocket/manager.py ← WebsocketManagerSysctl (subscription limits)
```

Context. `hathor/sysctl/` is the runtime-control surface of the node. It does not own any state of its own; it is a thin, uniform shell wrapped around the real subsystems — the connections manager, the RocksDB storage, the bit-signalling service — letting an operator query and adjust them while the node keeps running. Everything it can read or change is a value that already lives in one of those subsystems (Chapter 29). The package's whole job is to give those values **stable names**, group them into a tree, and ferry one-line commands in and answers out.

41.2 What it does, and why it exists

The problem: a daemon you cannot pause

Most programs you have written run, do their job, and exit. You configure them by editing a file or passing flags before they start. If you got it wrong, you fix the file and run again. That loop is fine for a script that finishes in a second.

It is not fine for a node. A node is a server process that must stay up. Three things make "just restart it" a poor answer:

1. **Restarting is expensive.** On shutdown the node drops every peer connection; on startup it re-opens the database and rebuilds its in-memory

view of the ledger (Chapter 29). For a synced mainnet node that is minutes of downtime, during which it sees and serves nothing.

2. **Some values are operational, not configuration.** "How many peers may sync at once" or "is the profiler running" are decisions you make in response to what the node is doing right now. They are not the kind of thing you bake into a config file once and forget.
3. **You sometimes need to read internal state, not change it.** "Which sync versions are available?" "How big are the RocksDB memtables?" There is no config-file question for that — it is a live measurement.

So the node needs an interface that is alive while the node is alive: a way to ask "what is X right now?" and to say "set X to Y, now," without a restart and without recompiling anything.

The analogy: Linux `sysctl`

This problem is old, and the operating system you are reading this on already solved it. Linux exposes hundreds of kernel knobs through a tool called `sysctl`. Each knob has a **name** that looks like a path, with dots (or slashes) separating levels:

```
$ sysctl net.ipv4.tcp_syncookies          # read one knob
net.ipv4.tcp_syncookies = 1

$ sysctl -w net.core.somaxconn=1024      # write one knob
net.core.somaxconn = 1024
```

The kernel is running. You are changing its behaviour live. The names form a **tree**: `net` contains `ipv4` which contains `tcp_syncookies`. You address a single leaf by spelling out the full path.

Hathor copies this design wholesale — the package is even named `sysctl` — and applies it to the node instead of the kernel. The node exposes knobs like `p2p.max_enabled_sync` and `core.profiler.start`; you read one with `p2p.max_enabled_sync` and write one with `p2p.max_enabled_sync=5`. The shape is identical; only the subject changed from "the kernel" to "the running full node."

Recap — daemon (full treatment in Ch. 0). A daemon is a program that runs continuously in the background rather than finishing and exiting — a server, a database, a node. Because it is meant to stay up indefinitely, it needs ways to be observed and adjusted while running. Sysctl is one of those ways. → see Chapter 0, §0.1.

The conceptual win is **decoupling**, and it is worth naming because the same idea shows up across the codebase. The thing that holds a value (the connections manager) does not know it is reachable by `sysctl`. The thing that names the value (a small wrapper class) does not know how the command arrived — over a socket, from a file, or via a signal. And the thing that carries the command (the socket protocol) does not

know what the value means. Three layers, each ignorant of the others' internals, joined only by narrow contracts. That is what lets the same parameter tree be driven three different ways (socket, init file, signal) without any of the parameter code changing.

41.3 The concepts it rests on

Sysctl is almost pure plumbing, so it leans on machinery introduced earlier. Three recaps, then the code.

Recap — the subsystems it controls (full treatment in Ch. 29). Everything sysctl reads or writes lives in a real subsystem assembled at boot. The `HathorManager` is the central coordinator that holds the others — the P2P `ConnectionsManager`, the RocksDB storage, the `BitSignalingService`, the admin WebSocket factory. Sysctl does not duplicate any of their state; it holds a reference to each and calls methods or reads attributes on it. When you understand a sysctl parameter, you are really understanding a method on one of those subsystems. → full treatment in Chapter 29.

Recap — the reactor it runs on (full treatment in Ch. 16 & 23). The node is built on **Twisted**, whose centre is the **reactor**: a single event loop that waits for events and calls your code in response. The sysctl socket is just another thing the reactor watches: when a byte arrives on the control socket, the reactor wakes up the sysctl protocol object. The socket is opened with a Twisted `endpoint` and served by a Twisted `factory` — both standard Twisted pieces you met in Chapter 16. → full treatment in Chapters 16 and 23.

Recap — command dispatch (full treatment in Ch. 4). Dispatch means: given a name, look up and call the right piece of code. The CLI does it for subcommands; sysctl does it for parameter paths. The runner takes a string like `p2p.max_enabled_sync`, walks a dictionary-backed tree to find the getter registered under that name, and calls it. There is no giant `if/elif` chain — just table lookups, the same pattern as the CLI dispatcher. → full treatment in Chapter 4.

41.4 The code, walked

41.4.1 A tiny toy first

Before the real class, build the idea from scratch with a neutral example. We want an object that holds a few named parameters, each of which you can get and/or set by name. A parameter is really just a pair of functions: a function that returns the current value, and a function that accepts a new one. Either may be missing (a read-only sensor has no setter; a fire-once button has no getter).

```

class ParamNode:
    def __init__(self):
        self._params = {} # name -> (getter, setter)

    def register(self, name, getter, setter):
        self._params[name] = (getter, setter)

    def get(self, name):
        getter, _ = self._params[name]
        if getter is None:
            raise PermissionError(f"{name} is write-only")
        return getter()

    def set(self, name, value):
        _, setter = self._params[name]
        if setter is None:
            raise PermissionError(f"{name} is read-only")
        setter(value)

```

Now wrap something real-ish — a thermostat — and expose one read-write knob and one read-only sensor:

```

class Thermostat:
    def __init__(self):
        self.target = 20.0
        self.measured = 18.7

knobs = ParamNode()
t = Thermostat()
knobs.register("target",
               getter=lambda: t.target,
               setter=lambda v: setattr(t, "target", v))
knobs.register("measured",
               getter=lambda: t.measured,
               setter=None) # read-only

knobs.get("target") # -> 20.0
knobs.set("target", 22) # changes the live thermostat
knobs.set("measured", 5) # -> PermissionError: read-only

```

Three things to carry forward. First, a parameter is a getter/setter pair, and a `None` in either slot means "that direction is forbidden." Second, the `ParamNode` holds no thermostat state of its own — it holds references to functions that read and write the live `Thermostat`. Third, dispatch is a dictionary lookup on the name. Hathor's `Sysctl` is exactly this, plus two refinements: the parameters form a **tree** (nodes can hold child nodes), so names become dotted paths; and a separate **runner** turns a text line into a `get / set` call. We meet both next.

41.4.2 The `sysctl` base class — a node in the tree

The real base class lives in `hathor/sysctl/sysctl.py`. A getter is a zero-argument callable returning anything; a setter takes one or more arguments and returns nothing (`sysctl.py:22`):

```

Getter = Callable[[], Any]
Setter = Callable[..., None]

```

```

class SysctlCommand(NamedTuple):
    getter: Optional[Getter]
    setter: Optional[Setter]

```

`SysctlCommand` (`sysctl.py:39`) is our getter/setter pair from the toy, given a name and made into a `NamedTuple` ². Either field may be `None`, meaning that direction is not allowed.

The `Sysctl` class itself (`sysctl.py:44`) is one node in a tree. It holds two dictionaries: its child nodes, and its own leaf commands (`sysctl.py:47`):

```

class Sysctl:
    """A node in the sysctl tree."""

    def __init__(self) -> None:
        self._children: dict[str, 'Sysctl'] = {}
        self._commands: dict[str, SysctlCommand] = {}
        self.log = logger.new()

```

This is the crux of the tree shape. `_children` maps a path segment (like `'p2p'`) to another `Sysctl` node; `_commands` maps a name (like `'max_enabled_sync'`) to a getter/setter pair. A node can hold both — children and its own commands — which is how the tree gets its levels.

Two methods build the tree. `put_child` attaches a sub-node under a name (`sysctl.py:52`); `register` adds a leaf parameter (`sysctl.py:57`):

```

def register(self, path: str, getter: Optional[Getter], setter: Optional[Setter]) -> None:
    """Register a new parameter for sysctl."""
    assert path not in self._commands
    if setter is not None:
        setter = validate_call(setter)
    self._commands[path] = SysctlCommand(getter=getter, setter=setter)

```

Note the line `setter = validate_call(setter)`. `validate_call` is from **Pydantic** (Chapter 18): it wraps the setter so that the arguments are coerced and validated against the setter's type hints before the setter runs. This matters because the value is about to arrive as text from a socket. If a setter is declared `def set_max_enabled_sync(self, value: int)`, Pydantic guarantees `value` really is an `int` by the time the body runs — a malformed command is rejected at the boundary, not deep inside a subsystem. (This is the "parse, don't validate" idea from Chapter 18, applied to the control surface.)

41.4.3 Dotted-path addressing — walking down the tree

Now the lookup. Given a path like `p2p.max_enabled_sync`, how does the root node find the right leaf? The answer is a small recursion in `get_command` (`sysctl.py:67`):

```
def get_command(self, path: str) -> SysctlCommand:
    """Find and return the sysctl of the provided path."""
    if path in self._commands:
        return self._commands[path]
    for key, child in self._children.items():
        if not path.startswith(f'{key}.'):
            continue
        tail = path[len(key) + 1:]
        return child.get_command(tail)
    raise SysctlEntryNotFound(path)
```

Read it on the example `p2p.max_enabled_sync`, starting at the root node:

1. Is `'p2p.max_enabled_sync'` a leaf command directly on the root? No.
2. Walk the root's children. The child key `'p2p'` matches, because the path starts with `'p2p.'`. Strip that prefix and the dot: `tail = 'max_enabled_sync'`.
3. Recurse into the `p2p` child node with `'max_enabled_sync'`. There, step 1 finds it directly in `_commands`. Return that `SysctlCommand`.

If no leaf and no child prefix match, the node raises `SysctlEntryNotFound` (`exception.py:19`). The dot is the level separator, exactly as in Linux `sysctl`. Note a subtlety: a single node can register a dotted command name directly, like `'rate_limit.global.send_tips'` in the P2P node — that whole string is one leaf key in `_commands`, not three tree levels. So a path can have dots that are tree boundaries and dots that are just part of a leaf's name; `get_command` tries the direct-leaf match first, which makes that work.

41.4.4 Get vs. set

With the lookup in hand, reading and writing are thin. `get_getter` / `get_setter` (`sysctl.py:78`, `:85`) fetch the right half of the pair and enforce direction — asking to read a write-only entry raises `SysctlWriteOnlyEntry`, and vice versa:

```
def get_getter(self, path: str) -> Getter:
    cmd = self.get_command(path)
    if cmd.getter is None:
        raise SysctlWriteOnlyEntry(path)
    return cmd.getter
```

`get(path)` (`sysctl.py:92`) then calls the getter and returns its value. Setting goes through the runner (next), but the underlying `unsafe_set` (`sysctl.py:97`) shows the shape — and one wrinkle:


```
def unsafe_set(self, path: str, value: Any) -> None:
    setter = self.get_setter(path)
    if isinstance(value, tuple):
        setter(*value)
    else:
        setter(value)
```

The wrinkle: some setters take more than one argument. The send-tips rate limiter is set with two numbers — a count and a window in seconds. So if the parsed value is a tuple, it is unpacked into the setter's positional arguments; otherwise it is passed as a single argument. Keep that in mind when we reach the rate-limit example. (The method is called `unsafe_set` because it bypasses the runner's safety checks; the docstring says to use a runner instead — which is what every real path does.)

Two more methods, `get_all` (`sysctl.py:112`) and `get_all_paths` (`sysctl.py:122`), walk the whole tree depth-first to dump every readable parameter (used by the `!backup` command) or every path name (used by `!help`). They are the recursive cousins of `get_command`, visiting children first, then own commands.

41.4.5 The runner — from a text line to a call

The tree knows how to look up and call. What turns the operator's text — `p2p.max_enabled_sync=5` — into one of those calls? The `SysctlRunner` (`runner.py:24`). It holds the root node and has one entry method, `run` (`runner.py:31`):

```
def run(self, line: str, *, require_signal_handler_safe: bool = False) -> bytes:
    if not line:
        raise SysctlRunnerException('line cannot be empty or None')
    head, separator, tail = self.get_line_parts(line)
    if separator == '=':
        return self._set(head, tail, require_signal_handler_safe=require_signal_handler_safe)
    else:
        return self._get(head)
```

The grammar is one line, and the presence of an `=` decides the verb. `get_line_parts` (`runner.py:68`) splits on the first `=` with `str.partition`, then strips whitespace. No `=` means a **get**: `head` is the path, and `_get` calls the getter and serializes the answer. An `=` means a **set**: `head` is the path, `tail` is the value text.

The value text is **JSON**. `_set` (`runner.py:44`) hands `tail` to `deserialize` (`runner.py:83`), which parses it as JSON — so you write `p2p.max_enabled_sync=5` (the `5` is JSON for the integer five), and a string value would be `core.something="hello"` with the quotes. The clever bit handles multi-argument setters: `deserialize` wraps the text in brackets and parses it as a JSON array, and if that yields more than one element it returns a **tuple** (`runner.py:88`). So `p2p.rate_limit.global.send_tips=100, 60` deserializes to the tuple `(100, 60)`, which — per the unpacking rule from §41.4.4 —

calls the setter with two arguments. On the way back out, getter results are turned to text by `serialize (runner.py:75)`: tuples become comma-joined JSON parts, everything else is plain JSON.

There is one more parameter on `run: require_signal_handler_safe`. Ignore it for now; it belongs to the signal path and we explain it in §41.5.

41.4.6 The transport — a line-based socket protocol

The runner is transport-agnostic: it eats a string and returns bytes. Something has to carry those strings. For network access that something is the `SysctlProtocol` (`protocol.py:31`), a Twisted `LineReceiver`:

```
class SysctlProtocol(LineReceiver):
    delimiter = b'\n'

    def __init__(self, runner: SysctlRunner) -> None:
        self.runner = runner

    def lineReceived(self, raw: bytes) -> None:
        ...
        feedback = self.runner.run(line)
        if feedback:
            self.sendLine(feedback)
```

A `LineReceiver`³ is a Twisted protocol that buffers incoming bytes and calls `lineReceived` once per complete line (here, lines end in `\n`). So the wire format is dead simple: send one command per line, get one line of answer (or an error line) back. This is what makes the control socket usable with bare tools — you can `nc` (netcat) into it and type commands by hand, or pipe a file of commands at it.

`lineReceived (protocol.py:37)` also recognizes a few meta-commands before handing the line to the runner: `!help [path]` prints available paths or the docstrings of one command's getter/setter (built by reflection with `inspect`, `protocol.py:111`), and `!backup` dumps every readable parameter and its current value via `get_all`. Errors are caught per type — `SysctlEntryNotFound`, `SysctlReadOnlyEntry`, Pydantic's `ValidationError`, and so on — and turned into a human-readable `[error] ...` line rather than crashing the connection (`protocol.py:55`).

One protocol instance is created per connection by the `SysctlFactory (factory.py:21)`, a Twisted `Factory` whose `buildProtocol` returns a fresh `SysctlProtocol` wired to the shared runner. Factory and protocol are the standard Twisted server pair from Chapter 16: the factory is the long-lived listener, a protocol is one conversation.

41.4.7 Two real parameters, end to end

Abstractions land when you trace a real one. Take two from the P2P node, `hathor/sysctl/p2p/manager.py`.

Read-write integer — `p2p.max_enabled_sync`. This caps how many peers may run the sync protocol at once. The `ConnectionsManagerSysctl` registers it in its constructor, pointing the getter and setter at its own methods (`p2p/manager.py:65`):

```
self.register(
    'max_enabled_sync',
    self.get_max_enabled_sync,
    self.set_max_enabled_sync,
)
```

The getter reads a live attribute off the connections manager (`p2p/manager.py:180`); the setter validates the input and pokes the same attribute, then forces a sync re-shuffle so the new cap takes effect immediately (`p2p/manager.py:185`):

```
def get_max_enabled_sync(self) -> int:
    """Return the maximum number of peers running sync simultaneously."""
    return self.connections.MAX_ENABLED_SYNC

@signal_handler_safe
def set_max_enabled_sync(self, value: int) -> None:
    """Change the maximum number of peers running sync simultaneously."""
    if value < 0:
        raise SysctlException('value must be >= 0')
    if value == self.connections.MAX_ENABLED_SYNC:
        return
    self.connections.MAX_ENABLED_SYNC = value
    self.connections._sync_rotate_if_needed(force=True)
```

Now the full path of `p2p.max_enabled_sync=5` over the socket:

```
operator types:  p2p.max_enabled_sync=5
|
▼
SysctlProtocol.lineReceived → runner.run("p2p.max_enabled_sync=5")
|   sees '=', so this is a SET; tail "5" → deserialize → int 5
|
▼
runner._set → root.get_setter("p2p.max_enabled_sync")
|   get_command walks: root → child "p2p" → leaf "max_enabled_sync"
|   (Pydantic-wrapped) setter called with value=5
|
▼
ConnectionsManagerSysctl.set_max_enabled_sync(5)
|   validates >= 0, writes connections.MAX_ENABLED_SYNC = 5,
|   forces a sync rotate so it applies now
|
▼
runner returns b'' → protocol sends nothing back (set has no output)
```

A read, `p2p.max_enabled_sync`, takes the same path minus the `=`: the runner sees no separator, calls the getter, JSON-serializes the returned `int`, and the protocol writes it back as one line.

Read-only list — `p2p.available_sync_versions`. Registered with a getter and **no** setter (`p2p/manager.py:95`):

```
self.register(
    'available_sync_versions',
    self.get_available_sync_versions,
    None,
)
```

Because the setter slot is `None`, a `p2p.available_sync_versions=...` write fails at `get_setter` with `SysctlReadOnlyEntry`, which the protocol reports as `[error]` cannot write to `p2p.available_sync_versions`. A read returns the list of supported sync versions (currently just `v2`; `sync-v1` is removed — Chapter 35). This is the "sensor" case from the toy: observable, not adjustable.

Multi-argument setter — `p2p.rate_limit.global.send_tips`. Worth one more look because it exercises the tuple path. Its setter takes two values (`p2p/manager.py:137`):

```
def set_global_send_tips_rate_limit(self, max_hits: int, window_seconds: float) -> None:
    """Change the global rate limiter for SEND_TIPS.
    The rate limiter is disabled when `window_seconds == 0`."""
```

Sending `p2p.rate_limit.global.send_tips=100, 60` makes `deserialize` produce the tuple `(100, 60)`, which the runner unpacks into `set_global_send_tips_rate_limit(100, 60)` — a rate limit of 100 messages per 60-second window. Setting the window to `0` turns the limiter off. This is why the getter/setter contract allows `Setter = Callable[..., None]` (variadic) rather than a single-argument function.

41.4.8 The per-subsystem nodes — a tour

Each subsystem gets its own `Sysctl` subclass, holding a reference to the live object it controls and registering that object's knobs in its constructor. They are all the same shape; here is what each exposes.

- **ConnectionsManagerSysctl** (`p2p/manager.py:60`) — the richest node. Sync tuning (`max_enabled_sync`, `enabled_sync_versions`, `sync_update_interval`, `force_sync_rotate`), the `send_tips` rate limiter, peer-management actions (`kill_connection` to drop one peer or `*` for all; `always_enable_sync` to pin peers; `reload_entrypoints_and_connections`), and hostname control.
- **HathorManagerSysctl** (`core/manager.py:24`) — debugging and profiling, mounted at `core.profiler.start` / `profiler.stop` / `profiler.status` (Chapter 42), plus live-debugger doors: `pubdb.set_trace.*` drops into the `pubdb` debugger on a chosen TTY, and `ipython.run.attach_tty` opens an IPython shell with `manager` and `tx_storage` pre-bound. These last ones pause the main loop and are flagged destructive in their docstrings — they exist for incident forensics, not routine use.
- **StorageSysctl** (`storage/manager.py:23`) — RocksDB operations: `rocksdb.flush` forces memtables to disk, `rocksdb.memtable_stats` and `rocksdb.wal_stats` report live storage sizes (Chapter 27). All under the `storage` prefix.

- **FeatureActivationSysctl** (`feature_activation/manager.py:20`) — mounted at `core.features` . Reads which features this node supports / is signalling for, and `add_support` / `remove_support` flip this node's signalling bits for a named feature (Chapter 38).
- **WebsocketManagerSysctl** (`websocket/manager.py:20`) — mounted at `ws` , present only if the admin WebSocket is enabled. Caps on subscribed addresses per connection (`max_subs_addrs_conn` , `max_subs_addrs_empty`), with `-1` meaning unlimited (Chapter 36).

The pattern is uniform: a subsystem node wraps a live object, names its knobs, and translates between text-friendly values and the object's real types. None of these classes hold state; they are naming-and-validation shells over the real subsystems.

41.5 How it plugs into the lifecycle

We have the tree, the runner, and the transport. Three questions remain: who builds the tree and wires it to the live subsystems, when is it exposed, and what is the `SIGUSR2` path for.

41.5.1 Assembly — the `SysctlBuilder`

The tree is not built by hand at every call site. The `SysctlBuilder` (`hathor/builder/sysctl_builder.py:26`) does it, given the `BuildArtifacts` bundle the main `Builder` produced (Chapter 24). Its `build` reads almost as a picture of the tree (`sysctl_builder.py:32`):

```
def build(self) -> Sysctl:
    root = Sysctl()

    core = HathorManagerSysctl(self.artifacts.manager)
    core.put_child('features', FeatureActivationSysctl(self.artifacts.bit_signaling_service))

    root.put_child('core', core)
    root.put_child('p2p', ConnectionsManagerSysctl(self.artifacts.p2p_manager))
    root.put_child('storage', StorageSysctl(self.artifacts.rocksdb_storage))

    ws_factory = self.artifacts.manager.websocket_factory
    if ws_factory is not None:
        root.put_child('ws', WebsocketManagerSysctl(ws_factory))

    return root
```

This is where the **wiring** happens. Each subsystem node is constructed with the real live object pulled from `BuildArtifacts` — the actual `HathorManager` , the actual `ConnectionsManager` , the actual RocksDB storage. That is what makes a `sysctl` `set` reach the running node rather than a copy: the getter/setter closures captured these references at build time. The tree's shape — `core` , `core.features` , `p2p` , `storage` ,

optional `ws` — is fixed right here. The `ws` node only appears if the WebSocket surface was enabled, so the parameter tree's exact contents depend on how the node was started.

41.5.2 Exposure — only if configured

A sysctl tree that no one can reach is harmless. The control surface is **off by default**; the operator opts in with a CLI flag. In `run_node.py`, after the node is prepared, the start path checks for `--sysctl` and, if present, calls `init_sysctl` (`run_node.py:544`):

```
self.prepare()
self.register_signal_handlers()
if self._args.sysctl:
    self.init_sysctl(self._args.sysctl, self._args.sysctl_init_file)
```

`init_sysctl` (`run_node.py:557`) builds the runner, optionally applies an init file, then opens the listening socket from a Twisted endpoint string (`run_node.py:569`):

```
runner = self.get_sysctl_runner()
if sysctl_init_file:
    init_file_loader = SysctlInitFileLoader(runner, sysctl_init_file)
    init_file_loader.load()
factory = SysctlFactory(runner)
endpoint = serverFromString(self.reactor, description)
endpoint.listen(factory)
```

The `--sysctl` value is a Twisted endpoint description, so the same code serves either transport depending on what you pass (`run_node.py:560`):

```
--sysctl unix:/path/sysctl.sock          # a Unix-domain socket (local only)
--sysctl unix:/path/sysctl.sock:mode=660 # ... with file permissions
--sysctl tcp:5000                        # a TCP port
--sysctl tcp:5000:interface=127.0.0.1    # ... bound to localhost
```

`serverFromString` parses that string into the right kind of listening endpoint, and `endpoint.listen(factory)` registers it with the reactor (Chapter 16). From then on, a connection to that socket gets a `SysctlProtocol` and can issue commands. The choice of transport is a security posture: a Unix socket with `mode=660` is reachable only by local processes the file permissions allow; a TCP port is reachable by anything that can route to it, so it is normally bound to `127.0.0.1`. The control surface has **no authentication of its own** — whoever can open the socket can tune the node — so the access control is the socket's, which is why the Unix-socket-with-permissions form is the safe default.

The optional `--sysctl-init-file` applies a file of commands once, at startup, through `SysctlInitFileLoader` (`init_file_loader.py:4`). It reads the file line by line and feeds each line to `runner.run` (`init_file_loader.py:12`) — the same runner the socket uses.

So the init file is "a batch of sysctl commands applied at boot," handy for setting non-default operational values (a custom sync cap, a rate limit) without a permanent config change. It reuses the entire runner machinery; there is no separate parser.

41.5.3 The signal path — SIGUSR2

There is a second way in, for when there is no socket open: a Unix **signal**⁴. At startup `register_signal_handlers` (`run_node.py:296`) installs a handler for `SIGUSR2` (where the OS provides it):

```
sigusr2 = getattr(signal, 'SIGUSR2', None)
if sigusr2 is not None:
    signal.signal(sigusr2, self.signal_usr2_handler)
```

When the process receives `SIGUSR2`, the handler calls `run_sysctl_from_signal` (`run_node.py:326`). This path is deliberately different from the socket. It **pauses the main loop**, creates a temporary named pipe (FIFO) at `SIGUSR2-<pid>.pipe` in the data directory, and blocks reading commands from that pipe; an operator writes commands into the pipe, the node executes them, then resumes (`run_node.py:345`):

```
filename = os.path.join(basedir, f'SIGUSR2-{os.getpid()}.pipe')
...
with temp_fifo(filename, tempdir):
    fp = open(filename, 'r')
    lines = fp.readlines()
    ...
    for cmd in lines:
        output = runner.run(cmd, require_signal_handler_safe=True)
```

Two things make this safe. First, this is the same runner over the same tree — no separate command set. Second, note `require_signal_handler_safe=True`. Because the signal handler can interrupt the node at any instruction — possibly mid-update of some data structure — only commands explicitly marked safe to run in that frozen, mid-flight state are allowed. That mark is the `@signal_handler_safe` decorator (`sysctl.py:30`), which sets an attribute on the method; the runner refuses any setter lacking it when called from the signal path (`runner.py:52`):

```
@signal_handler_safe
def set_max_enabled_sync(self, value: int) -> None:
    ...
```

So `set_max_enabled_sync` can be driven by `SIGUSR2` (it is decorated), but a setter that touches fragile state and is not decorated cannot. The socket path imposes no such restriction, because there the reactor delivers the command at a clean moment between events, not mid-instruction.

Recap — SIGUSR2 and the signal handler (full treatment in Ch. 29). A signal is a minimal asynchronous notification the OS delivers to a process. `SIGUSR1` and `SIGUSR2` are reserved for an application to define. Hathor uses `SIGUSR1` to reload peer entryptoints and `SIGUSR2` to open the FIFO-backed sysctl path described above. The handlers are installed in `run_node.py`, not in the manager. → the surrounding lifecycle is Chapter 29.

41.5.4 Putting it together

The whole lifecycle, in one line each:

```
build time   SysctlBuilder.build() constructs the tree, wiring each node to a
              live subsystem from BuildArtifacts                               (Ch 24)
start (opt)  --sysctl opens a Unix/TCP socket serving SysctlProtocol;
              --sysctl-init-file replays a batch of commands once
run (socket) a line in → runner parses get/set → tree dispatch → live subsystem
run (signal) SIGUSR2 → pause loop → read FIFO → runner (signal-safe only) → resume
```

At every step the parameter code is untouched: the same `Sysctl` tree and the same `SysctlRunner` serve the socket, the init file, and the signal pipe. That is the payoff of the three-layer decoupling from §41.2 — the knobs were defined once, and three different mechanisms drive them.

Recap

Parameter path	Subsystem (node class)	Get	Set	Notes
<code>p2p.max_enabled_sync</code>	<code>ConnectionsManagerSysctl</code>	yes	yes	cap on peers syncing at once; signal-safe
<code>p2p.available_sync_versions</code>	<code>ConnectionsManagerSysctl</code>	yes	—	read-only sensor (currently v2)
<code>p2p.rate_limit.global.send_tips</code>	<code>ConnectionsManagerSysctl</code>	yes	yes	two-arg setter (hits, window); <code>0</code> disables

Parameter path	Subsystem (node class)	Get	Set	Notes
p2p.kill_connection	ConnectionsManagerSysctl	—	yes	drop one peer, or * for all; signal-safe
core.profiler.start	HathorManagerSysctl	—	yes	start CPU profiler (Ch 42); signal-safe
core.profiler.status	HathorManagerSysctl	yes	—	(enabled, duration)
core.features.add_support	FeatureActivationSysctl	—	yes	flip this node's signalling bit (Ch 38)
storage.rocksdb.flush	StorageSysctl	—	yes	force memtables to disk (Ch 27); signal-safe
storage.rocksdb.wal_stats	StorageSysctl	yes	—	live WAL file sizes
ws.max_subs_addrs_conn	WebsocketManagerSysctl	yes	yes	only if WebSocket enabled (Ch 36); -1 = unlimited

The whole package is one idea applied with discipline: name every operationally-interesting value, arrange the names into a dotted tree, and decouple naming from the value's home subsystem and from the wire that carries the command. The base `Sysctl` is a tree node holding child nodes and getter/setter leaves; the `SysctlRunner` turns one text line into a get or a set; a Twisted `LineReceiver` protocol carries lines over a Unix socket or TCP; the `SysctlBuilder` wires each tree node to a live subsystem at boot; and `--sysctl`, `--sysctl-init-file`, and `SIGUSR2` are three independent doors onto the very same tree. With `sysctl` you can change a running node. The next chapter, **Chapter 42 (Observability)**, covers the other half of the operator's job — watching a running node: Prometheus metrics, the CPU profiler, and the healthcheck endpoint, the surfaces through which the node reports on itself rather than being adjusted.

-
-
1. A daemon is a program that runs continuously in the background rather than finishing and exiting — a server, a database, a node. Pronounced "demon." Full treatment in Chapter 0. ↩
 2. A `NamedTuple` is a tuple whose fields also have names, so you can write `cmd.getter` instead of `cmd[0]`. It is immutable and lightweight — handy for small fixed records like a getter/setter pair. See Chapter 1. ↩
 3. A `LineReceiver` is a Twisted protocol that buffers raw incoming bytes and calls your `lineReceived(line)` method once per complete line (split on a delimiter, here `\n`). It saves you from reassembling lines out of arbitrary network chunks. Twisted protocols are covered in Chapter 16. ↩
 4. A signal is a small asynchronous notification the operating system delivers to a process (e.g. `SIGINT` from Ctrl-C). `SIGUSR1` and `SIGUSR2` are left undefined by the OS for an application to use as it likes. A signal handler can interrupt the program at almost any point, which is why `sysctl` restricts what may run from the `SIGUSR2` path. ↩

Chapter 42 — Observability: Metrics, Profiler & Health

What you'll learn in this chapter

- What **observability** is, and why a node you cannot watch is a node you cannot operate.
- The four questions an operator asks — what happened, how much, why is it slow, is it alive — and which tool answers each.
- **Prometheus and metrics**: what a counter, a gauge, and a histogram are; what pull-based scraping means; and exactly which numbers `hathor-core` exposes.
- The **CPU profiler**: why a single-threaded reactor makes a slow callback catastrophic, and how the profiler finds the offending code.
- The **health-check** endpoint: the difference between liveness and readiness, what makes a Hathor node "healthy," and how an orchestrator like Kubernetes uses it.
- How all three are wired into the node's lifecycle.

This is the second-to-last chapter of Part II. The node we have assembled across the preceding chapters now runs: it boots, syncs, verifies, reaches consensus, and serves clients. This chapter is about the windows cut into its side — the surfaces an operator uses to see what the running node is doing, so they can keep it healthy. The three packages are small. The concepts behind them are the part worth your time.

42.1 Localization

Three pieces, all in the infrastructure tier of the module map (Chapter 0, §0.4). Two are single files; one is a small package.

```

hathor/
├── metrics.py           ← the Metrics object: samples node state   ◀ YOU ARE HERE
├── prometheus.py       ← PrometheusMetricsExporter: writes the .prom file
├── profiler/
│   ├── __init__.py     ← get_cpu_profiler() singleton accessor
│   ├── cpu.py          ← SimpleCUPProfiler: the measuring engine
│   ├── site.py         ← SiteProfiler: profiles HTTP requests
│   └── resources/
│       └── cpu_profiler.py ← CUPProfilerResource: the /top REST API
└── healthcheck/
    └── resources/
        └── healthcheck.py ← HealthcheckResource: the /health endpoint
            (readiness lives at p2p/resources/healthcheck.py – /p2p/readiness)

hathor_cli/
└── top.py              ← `hathor-cli top`: the curses viewer for the profiler

```

Context. None of these packages changes the ledger. They do not verify, store, or sync a single vertex. They exist so that a human (or a machine acting for a human) can answer questions about the node from the outside. A mainnet node is a long-running server that must stay up for months; you cannot babysit it by hand. Observability is the set of surfaces that let you, or your monitoring system, watch the node's pulse, find out why it is slow, and decide whether to restart it — without ever stopping it to look inside.

42.2 What it does and why it exists

There is an old operations saying: **you cannot fix what you cannot see**. A program that runs correctly in front of you on your laptop is one thing. A program that runs for ninety days unattended on a server in another country, processing millions of transactions, is another. When that second program misbehaves — gets slow, stops keeping up with the network, leaks memory — you have no debugger attached, no `print` statements you can add without redeploying, and no way to stop it and look without taking the service down. Everything you learn about it, you must learn from surfaces it deliberately exposes while it keeps running.

That is what **observability** means: the property of a system that lets you understand its internal state purely from its external outputs. A car's dashboard is the everyday example. You do not open the engine to know your speed or your fuel level; the car exposes those numbers on a panel built for exactly that purpose. The metrics, profiler, and health-check surfaces are `hathor-core`'s dashboard.

The four questions

It helps to be precise about what you want to know, because different questions need different tools, and `hathor-core` ships a different surface for each. An operator staring at a running node asks four distinct questions:

Question	Tool	The chapter
What happened?	structured logs	Ch 17
How much, how fast?	metrics (Prometheus)	this chapter, §42.4
Why is it slow?	the CPU profiler	this chapter, §42.5
Is it alive / ready?	the health-check endpoint	this chapter, §42.6

The first question — what happened — is answered by **logs**, which we gave their full treatment in Chapter 17. A log is a record of discrete events: "block accepted," "peer disconnected," "verification failed." Logs are narrative; they tell you the story of what the node did, event by event. They are excellent for debugging a specific incident after the fact ("show me everything that happened around 14:32").

But logs are bad at the other three questions. If you want to know how many transactions the node is processing per second, reading a log line per transaction is hopeless — you would be counting by hand. If you want to know why the node is slow, a log tells you what it did, not where the time went. And if you want a yes/no answer to is this node healthy, you do not want to read a story at all; you want a single bit.

So observability is not one pillar but several. Logs are one (Chapter 17). This chapter covers the other three. The recurring slogan worth memorizing:

Logs answer WHAT. Metrics answer HOW MUCH. A log entry is one event, written once, read by a human reconstructing a timeline. A metric is one number that changes over time, sampled repeatedly, read by a machine drawing a graph. You do not log "transaction count = 4,182,991" on every transaction; you expose it as a metric and let a monitoring system chart it. The two are complementary, never substitutes. → logs are Chapter 17; metrics are §42.4 below.

42.3 The concepts it rests on

Three ideas from earlier chapters underpin this one. We recap them in their local form here; the full treatments are where the pointers say.

Recap — structured logging (full treatment in Ch. 17). `hathor-core` does not write log lines as freeform English. It writes them as key-value events: `event="new block" height=42 hash=00ab...`. This makes logs machine-queryable. Logs are the first observability pillar and answer "what happened"; this chapter covers the rest. → Chapter 17.

Recap — the single-threaded reactor (full treatment in Ch. 2 & 16). The entire node runs its logic on `one` thread, driven by the Twisted **reactor**¹ — a single event loop that waits for an event (data on a socket, a timer firing), calls the one piece of your code that handles it, and only when that code `returns` moves on to the next event. The cardinal rule (Chapter 2) is never block the reactor: while one callback runs, every other event — every other peer, every API request, every timer — waits. A callback that takes two seconds freezes the whole node for two seconds. This single fact is why a CPU profiler matters so much here (§42.5). → Chapters 2 and 16.

Recap — the manager owns observability (full treatment in Ch. 29). The `HathorManager` is the node's central coordinator. It constructs the `Metrics` object during its own `__init__` (`manager.py:212`) and calls `self.metrics.start()` inside its `start()` sequence (`manager.py:325`), stopping it on shutdown (`manager.py:360`). The metrics object is owned by the manager; the Prometheus exporter and the REST resources are wired separately by the builders (§42.7). → Chapter 29.

A note on the `LoopingCall`², which appears in all three surfaces. It is Twisted's periodic timer: "call this function every N seconds." Metrics, the Prometheus exporter, and the profiler all use one to wake up on a schedule and sample. Because it runs on the reactor, the sampling itself must be cheap — a long sample would block the node, the very thing we are trying to watch.

42.4 The code, walked — Metrics and Prometheus

This is the canonical treatment of metrics and Prometheus for the whole book. We build the concept generically first, then show Hathor's exact spelling.

42.4.1 What a metric is

A **metric** is a single named number that describes some aspect of a running system, and that you sample over time. "Number of connected peers." "Total transactions processed." "Bytes received from peer X." Each is one metric. Plot any of them against time and you get a graph; a wall of such graphs is a monitoring dashboard.

The monitoring world classifies metrics into three shapes, and the distinction is not pedantry — it tells the monitoring system how to interpret the number.

- A **counter** only ever goes up (until the process restarts, when it resets to zero). "Total transactions received since boot" is a counter. You never decrement it. What you actually chart is its rate of change — the slope — which gives you "transactions per second." Hathor's per-peer `received_messages`, `received_bytes`, and `received_txs` (`metrics.py:48–55`) are counters in spirit: monotonically rising tallies.
- A **gauge** can go up or down. It is a snapshot of a current level. "Number of connected peers right now" is a gauge — it rises when a peer connects, falls when one drops. "Best block height," "mempool size," "RocksDB column-family size in bytes": all gauges. Most of what `hathor-core` exposes is a gauge.
- A **histogram** buckets many observations of a value to show its distribution — "how long did requests take, grouped into <10ms, <100ms, <1s buckets." It answers "what does the spread look like," not just "what is the latest value." `hathor-core` does not currently expose histograms through its Prometheus surface; it leans entirely on counters and gauges. We define the shape here so the vocabulary is complete and so you recognize it in other systems.

Generic intuition before the real code — picture the simplest possible metric registry:

```
# A toy metric: a gauge you read whenever someone asks.
class Gauge:
    def __init__(self, name):
        self.name = name
        self.value = 0.0

    def set(self, v):          # a gauge can be set to any value
        self.value = v

peers = Gauge("connected_peers")
peers.set(8)                 # 8 peers connected right now
# ... later, a monitoring system reads peers.value and charts it.
```

The real Prometheus client library gives you exactly this `Gauge` (plus `Counter`, `Histogram`), with the machinery to expose every gauge's current value in a format a monitoring server can read. Hathor uses that library.

42.4.2 Pull-based scraping — the Prometheus model

Now the second concept: how does the number get from inside the node to the monitoring system? There are two philosophies.

In a **push** model, the application actively sends its metrics out to a collector — "here are my numbers" — every few seconds. In a **pull** (or scrape) model, the application merely exposes its current numbers at a known location, and a central server periodically comes and reads them. The application is passive; it does not know or care who is watching.

Prometheus⁴ is the de-facto standard monitoring system in this space, and it is built around the **pull** model. A Prometheus server is configured with a list of targets (addresses to scrape). On a fixed interval — say every 15 seconds — it visits each target, reads the current value of every metric, stamps it with the time, and stores the whole time-series in its own database. You then query and graph that database (commonly through Grafana, a dashboard tool).

We discuss why pull and not push in §42.4.5. First, Hathor's exact mechanism — which has a twist.

42.4.3 Two objects: `Metrics` and `PrometheusMetricsExporter`

`hathor-core` splits the job in two. One object collects the numbers from inside the node; a second object exports them in Prometheus format. Keeping them separate means the node always tracks its own metrics (cheap, always on) even when nobody has turned Prometheus export on.

The collector — `Metrics` (`hathor/metrics.py:58`). This is a `@dataclass` holding every number the node tracks, plus the logic to refresh them. Its fields are the metrics. A sampling of them (`metrics.py:67–118`):

```
@dataclass
class Metrics:
    pubsub: PubSubManager
    avg_time_between_blocks: int
    connections: ConnectionsManager
    tx_storage: TransactionStorage
    reactor: Reactor

    transactions: int = 0          # tx count in the network
    blocks: int = 0                # block count
    best_block_height: int = 0     # height of the best chain
    hash_rate: float = 0.0        # network hash rate
    peers: int = 0                 # peers connected
    best_block_weight: float = 0  # weight of the best block
    # ... websocket_connections, completed_jobs (stratum),
    #     rocksdb_cfs_sizes, transaction_cache_hits/misses,
    #     connected_peers, handshaking_peers, known_peers, ...
```

Notice it holds references to the live node components — the pub-sub bus, the connections manager, the transaction storage. That is how it gets fresh numbers: it asks them.

`Metrics` refreshes itself two ways, and the split between them is worth understanding because it is a recurring pattern for "how do I keep a number current."

Event-driven updates. In `subscribe()` (`metrics.py:166`) the metrics object registers with the pub-sub bus³ for a handful of events — `NETWORK_NEW_TX_ACCEPTED`, the peer connect/disconnect events. When such an event fires, `handle_publish()` (`metrics.py:181`) updates the relevant numbers immediately. A new block accepted? Refresh the block count, hash rate, best-block weight and height on the spot (`metrics.py:189–194`). A peer connected? Update the peer counts. These numbers are kept exact because the node tells the metrics object the moment they change.

Polled updates. Some numbers are too expensive or too fiddly to recompute on every event, so they are sampled on a timer. In `__post_init__` (`metrics.py:120`) a `LoopingCall` is created bound to `_collect_data` (`metrics.py:130`), and `start()` launches it (`metrics.py:159`) at an interval taken from settings (`METRICS_COLLECT_DATA_INTERVAL`). Every tick, `_collect_data` (`metrics.py:300`) refreshes the WebSocket numbers, the Stratum mining numbers, the transaction-cache hit/miss counts, the per-peer connection metrics, and — less often, gated on block height — the RocksDB column-family sizes (`set_tx_storage_data`, `metrics.py:282`).

So: cheap-and-exact numbers update on the event that changes them; expensive numbers are polled. The result is one `Metrics` object whose fields are always reasonably current. Crucially, all of this runs whether or not Prometheus is enabled. The node tracks its own vital signs by default.

The exporter — `PrometheusMetricsExporter` (`hathor/prometheus.py:75`). This is the optional second half, built only when the operator passes `--prometheus`. Its job is to take the numbers out of the `Metrics` object and publish them in Prometheus format. In `_initial_setup` (`prometheus.py:124`) it creates a Prometheus `CollectorRegistry` and one `Gauge` object per metric (`prometheus.py:134–135`), driven by the `METRIC_INFO` dictionary at the top of the file (`prometheus.py:29`) that maps each metric name to a human description. It also sets up labelled gauges for the per-peer metrics, the RocksDB sizes, and Python garbage-collection stats (`prometheus.py:137–182`).

It too runs on a `LoopingCall` (`prometheus.py:117`, started at `prometheus.py:188`). Each tick, `set_new_metrics` (`prometheus.py:191`) copies every current value out of the `Metrics` object into the matching Prometheus `Gauge`:

```
def set_new_metrics(self) -> None:
    for metric_name in METRIC_INFO.keys():
        self.metric_gauges[metric_name].set(getattr(self.metrics, metric_name))
    self._set_rocksdb_tx_storage_metrics()
    self._set_new_peer_connection_metrics()
    write_to_textfile(self.filepath, self.registry) # prometheus.py:200
```

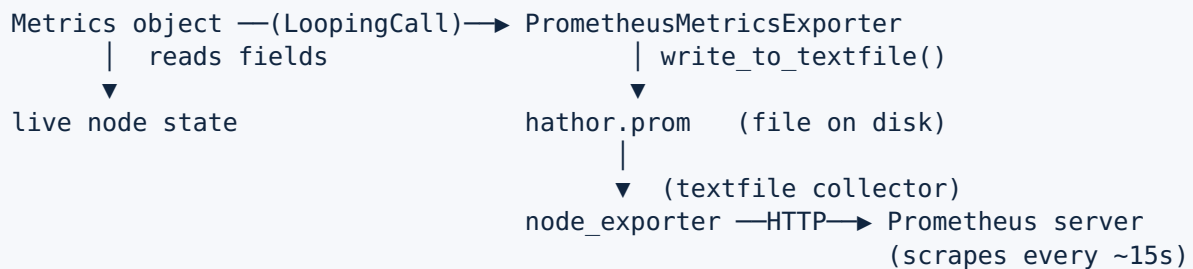
42.4.4 The twist: file-based export, not an HTTP endpoint

Here is where Hathor diverges from the textbook Prometheus setup, and it is worth stating plainly because the prompt's framing ("exposes a Prometheus endpoint") is not what the code does. Look at the last line above: `write_to_textfile`. The exporter does

not open an HTTP port that Prometheus scrapes directly. Instead, every interval it writes the current metrics to a file on disk — a `.prom` text file (`prometheus.py:79` , default filename `hathor.prom`), in a `prometheus/` directory under the node's data path (`resources_builder.py:104`).

How does Prometheus then read it? Through the **node_exporter** convention.

`node_exporter` is a separate, standard Prometheus component (a small daemon you run alongside the node) with a textfile collector feature: point it at a directory of `.prom` files and it serves their contents over HTTP for Prometheus to scrape. The class docstring says exactly this — "sends hathor metrics to a node exporter that will be read by Prometheus" (`prometheus.py:76`). So the full path is:



The end-to-end model is still pull — Prometheus scrapes `node_exporter` — but the node itself participates by writing a file, not by serving a port. This indirection keeps the node out of the business of running an HTTP metrics server and reuses a battle-tested external component for the scrape surface. (Be precise about this when you operate the node: enabling `--prometheus` alone does not give you a scrapeable URL; you must also run `node_exporter` pointed at the file.)

42.4.5 Why Prometheus, and why pull

The trade-off discussion the book promises for every technology choice.

Why metrics at all, versus just logging numbers? Covered in §42.2: a number that changes constantly is a time-series, not an event. Logging it floods the log and forces you to parse and re-aggregate; exposing it as a metric lets purpose-built tooling sample, store, and graph it.

Why pull, not push? The pull model has concrete operational advantages, which is why Prometheus chose it:

- **The monitoring system controls the load.** With pull, Prometheus decides how often to scrape; a misbehaving node cannot flood the collector by pushing too fast. Push systems need their own rate-limiting.
- **Targets are discoverable and their health is implicit.** If Prometheus tries to scrape a target and the scrape fails, that is a signal — "the node is down or unreachable." With push, a silent node is ambiguous: is it healthy-but-quiet, or dead?

- **Simplicity at the node.** The node only has to expose its current state; it does not need to know the monitoring server's address, manage retries, or buffer on network failure. (Hathor takes this even further by writing a plain file and letting `node_exporter` handle the HTTP.)

The cost is that pull does not fit every shape of system — short-lived batch jobs that finish before any scrape lands need a push gateway. A long-running daemon like a full node is the ideal pull target: it is always up and always has current numbers to read. Prometheus is the de-facto standard for exactly this kind of infrastructure monitoring, which is the practical reason to choose it: the surrounding ecosystem (Grafana dashboards, alerting rules, `node_exporter`) is mature and everyone already knows it.

42.5 The code, walked — the CPU profiler

42.5.1 The reactor-stall problem

Recall the cardinal rule from §42.3: the node runs on one reactor thread, and while any callback executes, the entire node is frozen. This makes a slow function uniquely dangerous here. In a multi-threaded server, one slow request ties up one thread; the others keep serving. In `hathor-core`, one slow callback ties up the only thread — every peer stalls, every API request hangs, every timer slips. A function that is merely "a bit slow" in isolation can, if it sits on the hot path, throttle the whole node's throughput.

So the operationally interesting question is not "is this function fast in a benchmark" but **"where is the reactor thread actually spending its time, right now, in production?"** A normal profiler (run the program under `cProfile`, read the report after it exits) does not help: you cannot stop a production node to profile it, and the interesting behavior only appears under real load. You need a profiler you can switch on inside a live node, watch, and switch off — with low enough overhead that turning it on does not itself stall the thing you are measuring.

That is what `hathor/profiler/` provides.

42.5.2 `SimpleCUPProfiler` — a sampling marker stack

The engine is `SimpleCUPProfiler` (`hathor/profiler/cpu.py:48`). It is deliberately not a full statistical profiler; it is a lightweight marker system. The idea: wrap sections of code in a `begin/end` pair, accumulate how much CPU time each named section consumed, and periodically compute each section's share of CPU.

Two methods do the measuring. `mark_begin(key)` (`cpu.py:123`) pushes a `(key, time.process_time())` pair onto a stack. `mark_end(key)` (`cpu.py:132`) pops it, computes the elapsed CPU time, and adds it to the running total for that key:

```
def mark_end(self, key: str) -> bool:
    if not self.enabled:
        return False
    # ... (stack-matching checks omitted)
    dt = time.process_time() - cur_time
    self.measures[tuple(x[0] for x in self.stack)].add_time(dt)
    self.stack.pop()
```

Two details matter. First, it uses `time.process_time()`, which measures CPU time consumed by this process, not wall-clock time — so time the node spent waiting (on the network, on disk) does not count against a section. That is exactly right for finding CPU hogs. Second, the key it records is the whole current stack (`tuple(x[0] for x in self.stack)`), so nested sections form a call path — you see not just "this function was slow" but "this function was slow when called from that one."

The accounting per section lives in `ProcItem` (`cpu.py:28`), which tracks `total_time` and, after each `update`, a `percent_cpu`. A `LoopingCall` runs `update()` (`cpu.py:156`) every few seconds (`update_interval=3.0` by default, `cpu.py:51`); it computes each section's CPU percentage over the interval, drops sections idle longer than `expiry` (15s), and sorts the list so the heaviest sections rise to the top (`cpu.py:181`). The output is `proc_list`: a CPU-sorted list of "what the reactor thread is spending its time on." Note the profiler is off by default (`enabled = False`, `cpu.py:88`); all the `mark_*` methods short-circuit immediately when disabled, so the overhead of leaving the hooks in the code is near zero.

42.5.3 Where the marks come from — the `@profiler` decorator and `SiteProfiler`

You do not litter the codebase with `mark_begin`/`mark_end` by hand. The profiler exposes a decorator⁵, `profiler(key)` (`cpu.py:190`), that wraps any function so it is automatically marked on entry and exit:

```
@cpu.profiler('http-api') # site.py:45
@cpu.profiler(key=lambda self, request: request.path.decode())
@cpu.profiler(key=lambda self, request: self._get_client_ip(request))
def getResourceFor(self, request):
    return super().getResourceFor(request)
```

This is `SiteProfiler` (`hathor/profiler/site.py:27`), a subclass of Twisted's `server.Site` (the HTTP server object). By overriding `getResourceFor` — the method Twisted calls to route every incoming HTTP request — and stacking three `@profiler` decorators on it, every API request is automatically timed three ways: under the global key `http-api`, under its URL path, and under the client IP. The result is that the profiler's output naturally groups CPU time by API endpoint and by caller — answering "which API is eating the reactor" without any manual instrumentation. `SiteProfiler` is the HTTP server the node actually installs when the status server is built (`resources_builder.py:343`).

42.5.4 The `/top` API and `hathor-cli top`

The profiler is controlled and read over a REST resource, `CPUProfilerResource` (`hathor/profiler/resources/cpu_profiler.py:31`), mounted at `/top`. A `POST` with body `start`, `stop`, or `reset` toggles the profiler (`cpu_profiler.py:43`); a `GET` returns the current `proc_list` as JSON — each section with its `percent_cpu` and `total_time` (`cpu_profiler.py:76`). So an operator can turn profiling on in a live node, sample for a while, and turn it off, all over HTTP.

Reading raw JSON is unpleasant, so there is a viewer: `hathor-cli top` (`hathor_cli/top.py`), registered as a subcommand "CPU profiler viewer" (`hathor_cli/main.py:76`). It is a **urses**⁶ terminal UI — modelled on the Unix `top` command — that polls the node's `/top` endpoint every couple of seconds via an async `ProfileAPIClient` (`top.py:658`) and redraws a live, CPU-sorted table of where the reactor thread is spending its time, with keys to start/stop/reset the profiler remotely (`top.py:489`). It is, in effect, a `top` for the inside of the node: instead of OS processes, you see the node's own code sections ranked by CPU.

Tied together: the reactor is single-threaded, so a slow callback stalls everything (§42.5.1); the profiler measures CPU time per code section with negligible overhead when off (§42.5.2); the `@profiler` decorator and `SiteProfiler` instrument the HTTP path automatically (§42.5.3); and `hathor-cli top` lets an operator watch the ranking live and pinpoint the offending section (§42.5.4).

42.6 The code, walked — the health-check endpoint

42.6.1 Liveness versus readiness

The last question — is the node OK? — is the simplest to ask and the most consequential to answer, because the answer is usually consumed not by a human but by an **orchestrator**⁷ like Kubernetes that will act on it automatically: restart the node, or stop sending it traffic.

Orchestrators distinguish two flavours of "OK," and the distinction is the heart of this section:

- **Liveness** — is the process alive at all? Is it stuck, deadlocked, crashed-but-not-exited? A failing liveness check tells the orchestrator: **kill and restart this container**. The right response to "not alive" is a restart, because a hung process will not fix itself.
- **Readiness** — is the process ready to do useful work? It may be perfectly alive yet not ready — for a node, the textbook case is still syncing: the process is healthy, but its view of the ledger is stale, so it should not be answering wallet queries with out-of-date data. A failing readiness check tells the orchestrator:

the node is fine, leave it running, but do not route traffic to it yet. The right response to "not ready" is to wait, not to restart — a restart would only make it start syncing from scratch again.

Confusing the two is a classic operational bug: wire a "still syncing" node into a liveness probe and the orchestrator will restart it forever, never letting it finish syncing. The whole point of separating the probes is that restart and withhold traffic are different remedies for different problems.

42.6.2 What "healthy" means for a Hathor node

The logic both endpoints share lives on the manager: `is_sync_healthy()` (`hathor/manager.py:923`). A node is considered healthy when **two** conditions both hold:

```
def is_sync_healthy(self) -> tuple[bool, Optional[str]]:
    if not self.has_recent_activity():
        return False, HathorManager.UnhealthinessReason.NO_RECENT_ACTIVITY
    if not self.connections.has_synced_peer():
        return False, HathorManager.UnhealthinessReason.NO_SYNCED_PEER
    return True, None
```

1. **Recent block activity.** `has_recent_activity()` (`manager.py:908`) checks that the newest block in storage has a timestamp within a tolerance of now — specifically `P2P_RECENT_ACTIVITY_THRESHOLD_MULTIPLIER × AVG_TIME_BETWEEN_BLOCKS` . If the node's latest block is too old, the node is not keeping up with the chain, so it is unhealthy.
2. **At least one synced peer.** `has_synced_peer()` (`manager.py:648` / `connections.has_synced_peer()`) checks the node has a peer it is actually in sync with. A node alone in the dark — connected to no one, or to no one current — cannot trust its own view of the ledger, so it is unhealthy.

Both must pass; the failure carries a human-readable reason from the `UnhealthinessReason` enum (`manager.py:91`): "Node doesn't have recent blocks" or "Node doesn't have a synced peer." This is a sync-health definition: a Hathor node is "healthy" precisely when it is keeping up with the network.

42.6.3 The two resources

`hathor-core` mounts two endpoints, both driven by that same `is_sync_healthy()` :

`/health` — `HealthcheckResource` (`hathor/healthcheck/resources/healthcheck.py:31`). This is the richer endpoint, built on the third-party `healthcheck` library (the `python-healthchecklib` dependency, `pyproject.toml:83` ; the library is introduced where dependencies are catalogued, Chapter 13). The library models a service as a set of named components, each with its own check; here there is one component, `sync` (`healthcheck.py:70`), whose check calls `is_sync_healthy()` (`healthcheck.py:21`). The response is structured JSON listing each component, its pass/fail status, and the

reason — the format many monitoring tools expect. By default it returns HTTP **200** when healthy and **503** when not (`healthcheck.py:53`); a `strict_status_code=1` query parameter forces 200 always, for tools that key only off the body (`healthcheck.py:48`).

/p2p/readiness — `HealthcheckReadinessResource` (`hathor/p2p/resources/healthcheck.py:8`). A leaner endpoint with the same underlying check, returning a minimal `{success: true}` / `{success: false, reason: ...}` and the same 200/503 split (`p2p/resources/healthcheck.py:14`). The path and name make its intent explicit: this is the **readiness** probe — "is the node caught up enough to serve traffic?"

The shared `is_sync_healthy()` answers both the liveness-style `/health` and the readiness-style `/p2p/readiness`. In a Kubernetes deployment you would wire `/p2p/readiness` to the readiness probe (so traffic is withheld while the node syncs) and use `/health` for broader monitoring or a liveness probe — choosing thresholds to fit, as the `/p2p/readiness` OpenAPI note about `P2P_RECENT_ACTIVITY_THRESHOLD_MULTIPLIER` spells out (`p2p/resources/healthcheck.py:61`).

42.7 How it plugs into the lifecycle

Pulling the wiring together, in lifecycle order:

1. **Construction.** When the builder assembles the manager (Chapter 24), the manager constructs its `Metrics` object in `__init__` (`manager.py:212`), handing it the live pub-sub, connections, and storage references.
2. **Start.** In `HathorManager.start()` (Chapter 29), `self.metrics.start()` runs (`manager.py:325`): the metrics object subscribes to its pub-sub events and launches its `_collect_data` `LoopingCall`. From here the node is always tracking its own numbers, Prometheus on or off.
3. **Prometheus (optional).** If `--prometheus` was passed, the `ResourcesBuilder` (Chapter 24) builds a `PrometheusMetricsExporter` (`resources_builder.py:86–108`), pointing it at the `Metrics` object and a `prometheus/` directory under the data path, and starts its file-writing `LoopingCall`. An external `node_exporter` then exposes that file for a Prometheus server to scrape.
4. **REST resources mounted.** The `ResourcesBuilder` (Chapter 24) also mounts the observability HTTP resources into the API tree: `/profiler` and `/top` for the profiler (`resources_builder.py:215–216`), `/health` for the health check (`resources_builder.py:218`), and `/p2p/readiness` for readiness (`resources_builder.py:239`). The status server it installs is the `SiteProfiler` (`resources_builder.py:343`), so the HTTP path is profiled automatically.

5. **In production.** A Prometheus server scrapes the node's metrics on its own schedule; an operator runs `hathor-cli top` to watch CPU; an orchestrator probes `/p2p/readiness` and `/health` to decide whether to route traffic or restart. None of these touch the ledger; all of them keep the node observable while it runs untouched.

Recap

Surface	Question it answers	Shape	Key code	Consumed by
structured logs (Ch 17)	what happened?	event stream	<code>hathor/logging</code>	a human / log search
Metrics + Prometheus	how much, how fast?	counters / gauges	<code>metrics.py:58</code> , <code>prometheus.py:75</code>	Prometheus (via <code>node_exporter</code>)
CPU profiler	why is it slow?	CPU-time per code section	<code>cpu.py:48</code> , <code>site.py:27</code> , <code>/top</code>	<code>hathor-cli top</code> / a human
Health check	is it alive / ready?	pass/fail + reason	<code>manager.py:923</code> , <code>/health</code> , <code>/p2p/readiness</code>	an orchestrator (Kubernetes)

The node we built across Part II now has windows. Metrics expose its vital signs as numbers a monitoring system samples and graphs; the profiler lets an operator watch where the single reactor thread spends its CPU and find the callback that is starving everything else; the health-check endpoints give an orchestrator a one-bit answer — keep running, withhold traffic, or restart — built on whether the node is keeping up with the network. Each answers a question logs cannot, and together with logs (Chapter 17) they complete the operator's view of a running node.

One package of Part II remains. Everything so far has been about the production node and the surfaces for operating it. The final chapter turns inward, to the infrastructure that lets the project test all of this deterministically: **Chapter 43 — Determinism for tests**, where `hathor/simulator/` and `hathor/dag_builder/` spin up an entire in-memory network with a controllable clock, so a four-block reorg or a sync race can be reproduced on demand rather than hunted in the wild.

-
1. The reactor is the heart of the Twisted framework: a single event loop that waits for events (data on a socket, a timer firing) and calls the one piece of your code registered to handle each. The whole node runs on it, on one thread. Full treatment in Chapters 2 and 16. ↩
 2. A `LoopingCall` is Twisted's periodic timer: you give it a function and an interval, and it calls that function every interval seconds on the reactor thread. Because it shares the reactor, the function must be quick — a slow one blocks the node. ↩
 3. Pub-sub (publish-subscribe) is a messaging pattern: publishers announce events without knowing who listens, and subscribers register interest in event types. The metrics object subscribes to events like "new tx accepted" to update its numbers on the spot. Full treatment in Chapter 30. ↩
 4. **Prometheus** is the de-facto-standard open-source monitoring system for infrastructure. It pulls (scrapes) numeric metrics from targets on a fixed interval, stores them as time-series, and lets you query and graph them (commonly via Grafana). Covered in §42.4. ↩
 5. A decorator is a function that wraps another function to add behaviour without changing its body — here, to mark the start and end of a timed section. Full treatment in Chapter 4. ↩
 6. `curses` is a standard library for building text-based, full-screen terminal user interfaces (menus, live-updating tables) instead of plain scrolling output. `hathor-cli top` uses it to draw a live `top`-style dashboard. ↩
 7. An orchestrator is a system (Kubernetes is the common one) that runs and supervises containerized services automatically — starting them, restarting failed ones, and routing traffic only to healthy ones. It learns a service's state by calling its health-check endpoints. ↩

Chapter 43 — Deterministic Testing: Simulator & DAG Builder

What you'll learn in this chapter

- Why consensus and sync bugs are timing-dependent, why that makes them rare and hard to debug against a real network, and how **determinism** turns a flaky bug into a fixed test.
- The single key trick the simulator rests on: **advance time by hand** instead of sleeping — a fake reactor whose clock you step manually.
- How `hathor/simulator/` builds several **in-memory nodes**, wires two of them together with `FakeConnection`, and drives them all from one stepped clock with `run(...)`.
- How blocks and transactions are produced without real mining — a geometric model that fires a `callLater` instead of grinding a nonce.
- How `hathor/dag_builder/` lets a test **write a DAG in a compact text language** (a DSL) and get back real, signed vertices.
- Where all of this plugs into the test suite, and how it exercises consensus, sync, and the full ingestion pipeline — deterministically.

This is the last chapter of Part II, and it is a fitting place to end. Every earlier chapter walked one moving part of the node: the vertex model, storage, consensus, the peer-to-peer layer, sync. This chapter is about the machinery the project uses to test all of those parts together — to spin up a small network of nodes inside a single test process, make them mine, talk, conflict, and sync, and check that the right thing happened. And it does so without ever touching a real clock, a real socket, or a real proof-of-work puzzle.

Two packages cooperate. `hathor/simulator/` runs time: it stands up in-memory nodes, connects them, and advances a controlled clock so that hours of network activity replay in milliseconds. `hathor/dag_builder/` runs structure: it lets a test describe a precise DAG — "blocks b1..b50, transaction tx50 spends tx10 and tx20, block b33 confirms tx50" — as a few lines of text, and turns that description into real vertices the node can ingest.

43.1 Localization

Both packages live under `hathor/`, in the infrastructure tier of the module map (Chapter 0, §0.4). Neither ships in the running node's hot path; they exist to serve tests and tools.

hathor-core/ └─ hathor/ └─ simulator/ __init__.py simulator.py clock.py fake_connection.py tx_generator.py trigger.py patches.py utils.py miner/ abstract_miner.py geometric_miner.py └─ dag_builder/ builder.py tokenizer.py types.py artifacts.py default_filler.py vertex_exporter.py cli.py utils.py / types.py	 ◀ YOU ARE HERE (time) ← re-exports Simulator, FakeConnection, ... ← the Simulator class: peers, clock, run() ← HeapClock + MemoryReactorHeapClock (fake reactor) ← FakeConnection: wire two managers in memory ← RandomTransactionGenerator (no mining) ← stop conditions for a run ← skip real PoW in the simulator ← helpers to build txs / double-spends ← AbstractMiner base ← GeometricMiner: blocks via geometric timing ◀ YOU ARE HERE (structure) ← DAGBuilder: parse + assemble the DAG ← the DSL grammar + tokenizer (DSL docstring here) ← DAGNode, DAGInput, DAGOutput ← DAGArtifacts: the built vertices + propagate_with ← fills in unspecified amounts/parents ← turns the abstract DAG into real vertices ← build a DAG from a file, print hex ← small parsing helpers / type aliases
---	--

Context. A full node is a concurrent, networked program. Most of the behaviour worth testing — does a reorg resolve correctly, do two conflicting transactions void the right one, do two peers converge on the same ledger — only appears when several nodes run over time and exchange data. You cannot reliably reproduce that against the live network: it is slow, non-repeatable, and you don't control it. The simulator package recreates the whole network inside one Python process and one controllable clock; the dag_builder package lets a test state the exact graph it wants to reason about. Together they are how the project makes the rest of the book's machinery testable.

43.2 What it does, and why it exists

The problem: timing-dependent bugs

Recall what the node actually does in steady state (Chapter 0, §0.3). It sits in an event loop and reacts: a peer connects, a block arrives, a timer fires, a transaction is broadcast. The order in which those events land is not fixed. Two peers, each mining,

each relaying — the exact interleaving of "block from peer A arrives" versus "transaction from peer B arrives" versus "my own mining timer fires" depends on network latency, CPU speed, and luck.

Most node code is correct regardless of that order. But the hardest, most consequential code is not. Consensus (Chapter 32) decides which of two conflicting transactions wins and which is voided; sync (Chapter 35) brings a lagging peer up to date over many round-trips; a reorg¹ rewrites which chain of blocks is canonical when a heavier one appears. Bugs in these subsystems are almost always **timing-dependent**: they only manifest under one particular interleaving of events. Run the same scenario again and the interleaving shifts, and the bug hides.

A timing-dependent bug that you cannot reproduce on demand is nearly impossible to fix. You see it once in a log, you change something, you can't tell whether you fixed it or merely got lucky. This is the single worst category of bug in a networked system, and consensus is exactly where it lives.

The cure: determinism

The fix is **determinism**: arrange things so that the same test always produces the same sequence of events, every single run. If a bug appears once, it appears every time; once you can reproduce it on demand, you can bisect it, fix it, and lock the fix in as a regression test. A flaky, once-in-a-thousand-runs failure becomes a permanent, always-failing test that you then make pass — and it stays passing forever after.

So where does non-determinism come from, and how does the simulator remove each source?

Source of non-determinism	In the real node	In the simulator
Wall-clock time	timers fire when real seconds pass	a fake clock you advance by hand (§43.4)
Network latency / OS scheduling	real sockets, unpredictable delays	<code>FakeConnection</code> ferries bytes in-memory, step by step (§43.5)
Proof-of-work	grinding a nonce takes random real time	PoW is skipped; block timing comes from a seeded model (§43.6)
Randomness	system entropy (addresses, choices)	one seeded random generator (§43.3)
Disk / OS state	persistent RocksDB on disk	a throwaway temporary database (§43.3)

Remove all five and the simulation is **reproducible**: a given seed plus a given scenario always replays identically. That is the whole game.

A worked analogy before any code. Imagine a board game you want to test for a rules bug. Played with real friends, real dice, and a real timer, every play-through differs — you can never re-create the exact game where the bug showed up. Now imagine a version where you roll the dice from a pre-recorded list (seeded randomness), nobody waits on a clock — you just call out "next turn" yourself (the controlled clock), and the board is a scratch copy you throw away after (temporary storage). Now every play-through with the same dice-list is identical. If a rules bug appears, it appears every time. That is precisely what the simulator does for a network of nodes.

43.3 The concepts it rests on

Four ideas from earlier chapters carry this one. None is re-taught here; each gets a short recap and a pointer.

Recap — the Twisted reactor (full treatment in Ch. 16; abstraction in Ch. 23).

The node is built on Twisted, whose centrepiece is the **reactor**: a single event loop that waits for things to happen (network data, expired timers) and calls your code in response. Code that wants to run "later" does not call `time.sleep`; it asks the reactor `callLater(delay, fn)` and returns. The reactor decides when `fn` actually runs. The simulator's entire trick is to swap the real reactor for a **fake** one whose notion of "now" is a variable you control — so "later" arrives exactly when you say it does.

Recap — sync between peers (full treatment in Ch. 35). When a node is behind, a **sync agent** on each connection downloads the missing blocks and transactions in dependency order until the two peers hold the same ledger. The protocol is sync-v2; it runs over many message round-trips. Testing it means running two nodes against each other until both report "synced" — the simulator exists in large part to make that loop deterministic.

Recap — temporary, throwaway storage (full treatment in Ch. 27). Production nodes persist the ledger to RocksDB on disk. For tests, the builder can hand a node a temporary database that is created fresh and discarded afterward, so no test sees state left by another. There is no pure in-RAM backend; "in-memory node" here means a node whose storage is a throwaway temp database and whose reactor is fake — the node object itself is a real `HathorManager`.

Recap — the test suite (full treatment in Ch. 20). The project's tests run under `pytest`. The simulator and `dag_builder` are test infrastructure: they are imported by test cases (and a couple of CLI tools), not by the running node. A simulator-based test typically inherits a base class that constructs a `Simulator`, prints its seed, and tears it down afterward.

One more building block, local to this chapter: **seeded randomness**. The simulator creates exactly one random-number generator from a seed and threads it through everything that needs a random choice — wallet words, transaction values, mining timing. The generator is `hathor.util.Random`, a subclass of Python's `random.Random` that adds a `geometric(p)` method (`util.py:264`). Because every node and generator descends from that one seed (`simulator.py:55-58`), re-running with the same seed reproduces every "random" decision exactly. When a seed is not given, one is drawn from `secrets` and printed, so a failing run can be replayed by pasting the seed back in (the base test case prints it: `hathor_tests/simulation/base.py:18-20`).

43.4 The code, walked — the key trick first

Generic warm-up: advance time by hand

Forget Hathor for a moment. Suppose you are testing a plain function that schedules work for later:

```
def remind_me(scheduler, seconds, message):
    scheduler.call_later(seconds, lambda: print(message))
```

If `scheduler` is the real system clock, a test must actually wait `seconds` to see the message — slow, and never exactly repeatable. The standard fix is a **fake scheduler** that does not wait at all. It keeps a list of `(when, callback)` pairs and a variable `now`. Calling `call_later` just appends to the list. Nothing runs until the test says so:

```
class FakeScheduler:
    def __init__(self):
        self.now = 0.0
        self.calls = []  # (fire_time, callback)

    def call_later(self, delay, fn):
        self.calls.append((self.now + delay, fn))

    def advance(self, amount):  # the test drives time itself
        self.now += amount
        for when, fn in sorted(self.calls):
            if when <= self.now:
                fn()  # "later" has arrived – run it now
```

The test is now instant and deterministic:

```

sched = FakeScheduler()
remind_me(sched, 10, "hello")
# nothing printed yet
sched.advance(10)                                     # -> prints "hello", with zero real waiting

```

That is the whole idea. **Instead of sleeping, the test advances a clock by hand**, and whatever was scheduled to run "in 10 seconds" runs the instant the clock crosses that mark. No wall-clock time passes; the order is fixed.

Hathor's spelling: HeapClock

Hathor's fake clock is exactly this idea, made to satisfy Twisted's reactor interface so it can stand in for the real reactor. It lives in `hathor/simulator/clock.py`. The class `HeapClock` (`clock.py:25`) declares itself a Twisted `IReactorTime` implementation and holds:

- `rightNow` — the current simulated time, a single float (`clock.py:32`). This is the fake "now."
- `calls` — a heap (priority queue) of pending (`fire_time`, `DelayedCall`) pairs (`clock.py:35`).

Its `seconds()` returns `rightNow` — the node, asking "what time is it?", gets the simulated answer, not the system answer (`clock.py:37-45`). Its `callLater` is our `call_later`: build a `DelayedCall`, push it on the heap keyed by fire-time, return (`clock.py:47-57`). And the engine is `advance`:

```

def advance(self, amount):
    assert amount >= 0
    self.rightNow += amount
    while self.calls:
        time, call = self.calls[0]          # smallest fire-time on the heap
        if time > self.seconds():
            break                             # nothing else is due yet
        heapq.heappop(self.calls)
        if not call.cancelled:
            call.called = 1
            call.func(*call.args, **call.kw)

```

`hathor/simulator/clock.py:65-82` — moving time forward runs everything now due, in time order.

A heap² is used so that "the next thing to fire" is always cheap to find: the earliest fire-time sits at the front. `advance` pops and runs every call whose fire-time is at or before the new `rightNow`, in chronological order, then stops. Time only moves when the simulation chooses to move it.

The class actually used is `MemoryReactorHeapClock` (`clock.py:93`), which mixes `HeapClock` together with Twisted's `MemoryReactor` — a test reactor that fakes network connections in memory rather than opening real sockets. Note the deliberate override

of `run()` (`clock.py:98-104`): the stock `MemoryReactor.run()` immediately calls `stop()`, which would be wrong here, so Hathor's version just sets `running = True` and leaves the reactor "running" for the duration of the test.

This object is the determinism. It is a real, interface-compatible reactor as far as the node knows — but its clock is a variable the test steps by hand.

43.5 The code, walked — the Simulator class

`Simulator` (`hathor/simulator/simulator.py:52`) is the conductor. Constructing one (`simulator.py:53-65`) does four things that matter:

1. Picks a **seed** — given or drawn from `secrets` — and builds the one seeded `Random` from it (`simulator.py:55-58`).
2. Copies the global settings but **shortens the average time between blocks** to 64 seconds (`simulator.py:59-61`, constant `SIMULATOR_AVG_TIME_BETWEEN_BLOCKS = 64` at `:49`), so simulated chains grow fast.
3. Creates the fake reactor: `self._clock = MemoryReactorHeapClock()` (`simulator.py:62`).
4. Prepares empty registries for peers and connections (`simulator.py:63-64`).

`start()` (`simulator.py:67-74`) must be called before anything else. It nudges the clock forward by a randomized amount past the genesis timestamp — `dt = self.rng.randint(3600, 120 * 24 * 3600)` (`simulator.py:72-73`) — so that tests don't all begin at the exact same instant. Because `dt` comes from the seeded generator, it is still reproducible.

Building an in-memory node

`create_peer(builder)` (`simulator.py:91-97`) returns a fully-wired `HathorManager` — a real node object (Chapter 29) — configured for simulation. The heavy lifting is in `create_artifacts` (`simulator.py:99-133`). It takes a `Builder` (Chapter 24, the composition root) and overrides the pieces that would otherwise reach for real time, real PoW, or real entropy:


```

artifacts = builder \
    .set_reactor(self._clock) \           # the fake reactor, not the real one
    .set_rng(Random(self.rng.getrandbits(64))) \   # a seeded child RNG
    .set_wallet(wallet) \
    .set_vertex_verifiers_builder(_build_vertex_verifiers) \ # PoW-skipping verifier
    .set_daa(daa) \
    .set_cpu_mining_service(cpu_mining_service) \   # PoW-skipping miner
    .build()

artifacts.manager.start()
self._clock.run()
self.run_to_completion()

```

hathor/simulator/simulator.py:113-124 — every non-deterministic dependency is replaced before `build()`.

Three substitutions deserve a name. `set_reactor(self._clock)` is the core swap — the node's reactor is the fake clock, so every `callLater` the node makes lands on our heap. `set_rng(...)` gives the node a child generator derived from the master seed, so even the node's internal random choices are reproducible. And the two PoW-skippers come from `hathor/simulator/patches.py: SimulatorVertexVerifier.verify_pow ( patches.py: 27-30 )` does nothing — it logs "skipping" and returns — and `SimulatorCpuMiningService.resolve ( patches.py:33-43 )` just recomputes the hash without grinding for a valid nonce. The simulation does not need real proof-of-work; it needs timing that resembles mining, which it gets elsewhere (§43.6).

After `start()`, the call `self._clock.run()` flips the reactor to "running" and `run_to_completion()` (`simulator.py:141-148`) drains any setup work the node scheduled during boot by advancing the clock to each pending call's time. The node is now alive and idle, waiting for the test to move time.

Built nodes are registered by name with `add_peer(name, manager)` (`simulator.py: 149-153`) and fetched with `get_peer(name)` (`simulator.py:158-159`).

Wiring two nodes together: FakeConnection

A real connection between two nodes is a TCP socket: bytes one node writes appear, eventually, at the other. `FakeConnection` (`hathor/simulator/fake_connection.py:50`) replaces the socket with an in-memory shuttle you step by hand.

Its constructor (`fake_connection.py:54-89`) takes the two managers, an optional `latency` in seconds, and immediately calls `reconnect()` (`fake_connection.py:268-292`), which builds a protocol object on each side from the managers' real connection factories and joins them through `HathorStringTransport` — a Twisted test transport that captures written bytes into a buffer instead of sending them to an OS socket. Crucially, this exercises the real peer-to-peer protocol code (handshake, sync agent, message dispatch from Chapters 34–35); only the wire underneath is fake.

The pump is `run_one_step` (`fake_connection.py:195-245`). Each call takes whatever bytes node 1 has buffered to send, and delivers them to node 2 — and vice versa:


```

if line1:
    if self.latency > 0:
        self.manager1.reactor.callLater(           # deliver after a fake delay
            self.latency, self._deliver_message, self._proto2, line1, debug)
    else:
        self._proto2.dataReceived(line1)          # hand node 1's bytes straight to node 2

```

hathor/simulator/fake_connection.py:222-230 — one message hop, optionally delayed by a fake latency.

When `latency > 0`, delivery is scheduled on the fake clock with `callLater(latency, ...)` instead of happening at once — so a test can model a slow link, and the message arrives only after the clock advances past that latency. This is how the simulator reproduces network-delay-dependent ordering deterministically: the "delay" is fake-clock seconds, not real ones.

Two helper predicates make step-loops readable without guessing iteration counts: `is_both_synced()` (`fake_connection.py:119-164`) returns True only when both protocols are in the `ReadyState`, neither errored, both sync agents report synced, and both peers agree on the best block and the mempool tips. `can_step()` (`fake_connection.py:166-193`) reports whether there is still useful work to do on the connection — the run loop uses it to know when to stop.

A connection is registered with the simulator via `add_connection(conn)` (`simulator.py:161-162`) so the run loop will pump it.

Running the simulation: stepping time

`run(interval, step=0.25, ...)` (`simulator.py:225-242`) is the main entry point: advance simulated time by `interval` seconds, in increments of `step`, pumping every connection at each increment. The work is in the private generator `_run` (`simulator.py:167-198`):

```

while self._clock.seconds() <= initial + interval:
    for conn in self._connections:
        conn.run_one_step()          # ferry one message hop per connection
    yield
    ...
    self._clock.advance(step)        # move the fake clock forward one step

```

hathor/simulator/simulator.py:174-198 — the heartbeat: step the connections, advance time, repeat.

Each loop iteration: deliver one round of messages across every connection, then advance the clock by `step` (default 0.25 s, `DEFAULT_STEP_INTERVAL` at `simulator.py:47`), which fires any node timers now due — mining timers, sync's periodic `LoopingCall`, transaction generation. Repeat until `interval` simulated seconds have elapsed. Real wall-clock time barely moves; simulated time marches in fixed steps. The same `interval` and `step` with the same seed produce byte-for-byte the same run.

Two siblings refine this. `run_until_complete(max_interval, ...)` (`simulator.py:200-223`) steps until every connection reports it `can_step()` no more — i.e. all peers have synced (or errored) — returning `True`, or until `max_interval` is exhausted, returning `False`. That lets a sync test say "run until they converge" without hard-coding how many steps convergence takes. And `run(..., trigger=...)` accepts a **Trigger** (`hathor/simulator/trigger.py:28`), an object with a `should_stop()` method, checked each iteration; the run ends when the condition holds. Concrete triggers include `StopAfterNMinedBlocks` (`trigger.py:36`), `StopAfterNTransactions` (`trigger.py:69`), `StopWhenSynced` (`trigger.py:94`), and `All` to combine them (`trigger.py:103`).

43.6 The code, walked — producing blocks and transactions without mining

A network is dull without traffic. The simulator can generate both blocks and transactions, and it does so without real proof-of-work — instead it models how long mining would take and schedules the result on the fake clock.

The statistical idea is the **geometric distribution**³. Mining is repeated independent trials, each with a tiny success probability `p` determined by the block's weight: `p = 2**(-weight)`. The number of trials until the first success follows a geometric distribution with parameter `p`. So instead of actually trying nonces, the simulator draws a trial count from that distribution and divides by a configured `hashpower` to get a time delay — then schedules the block to "appear" after that many fake seconds. Statistically, blocks arrive with the same timing they would under real mining, but no CPU is spent and the outcome is reproducible from the seed.

`GeometricMiner` (`hathor/simulator/miner/geometric_miner.py:30`) implements this. Its `_schedule_next_block` (`geometric_miner.py:84-121`) is the loop. When a block is "found," it sets a random nonce, updates the hash, and propagates it through the node's real ingestion path:

```
if self._block:
    self._block.nonce = self._rng.getrandbits(32)
    self._block.update_hash()
    self._manager.propagate_tx(self._block)      # enters the real pipeline (Ch 33)
    self._blocks_found += 1
    self._block = None
...
geometric_p = 2**(-block.weight)
trials = self._rng.geometric(geometric_p)      # how many "tries" mining would take
dt = 1.0 * trials / self._hashpower            # convert tries -> fake seconds
self._block = block
self._delayed_call = self._clock.callLater(dt, self._schedule_next_block)
```

`hathor/simulator/miner/geometric_miner.py:89-121` — block timing drawn from a seeded geometric model, scheduled on the fake clock.

The miner subscribes to `NETWORK_NEW_TX_ACCEPTED` so that when a competing block arrives it can abandon its current candidate and start over on the new chain head (`abstract_miner.py:43-47`, `geometric_miner.py:56-70`) — exactly the race a reorg test wants to provoke. A test creates one with `simulator.create_miner(peer, hashpower=...)` (`simulator.py:138-139`).

`RandomTransactionGenerator` (`hathor/simulator/tx_generator.py:36`) does the analogous job for transactions: on a seeded schedule it picks a wallet address and amount, builds a real transaction, sets its minimum weight, draws a geometric "mining" delay, and propagates it (`tx_generator.py:93-148`). The inter-transaction gap is `self.rng.expovariate(self.rate)` (`tx_generator.py:105`) — an exponential wait, the natural model for "events arriving at an average rate" — again drawn from the seeded generator. It can even be told to emit deliberate double-spends (`tx_generator.py:90-91, 131-137`) to test conflict resolution. A test creates one with `simulator.create_tx_generator(peer, rate=..., hashpower=...)` (`simulator.py:135-136`).

So the traffic on the simulated network — when blocks land, when transactions fire, who conflicts with whom — is entirely a function of the seed. Replay the seed, replay the traffic.

43.7 The code, walked — writing a DAG in text

The simulator runs time. The `dag_builder` builds structure. Sometimes a test does not want a random network at all; it wants a specific graph — "block b33 confirms transaction tx50, which spends outputs of tx10 and tx20" — so it can assert exactly what consensus does with it. Constructing such a graph by hand in Python is verbose and error-prone: you must create each output, wire each input to the right output index, set parents, sign everything. The `dag_builder` lets you write that graph as a few lines of a small **domain-specific language**⁴ (DSL) and get back real, signed vertices.

The DSL

The grammar is documented in the module docstring at the top of `hathor/dag_builder/tokenizer.py:20-129`. The core statements:

```
blockchain genesis b[1..50]    # create blocks b1..b50, chained, b1's parent is genesis
a <-- b <-- c                  # a is a parent of b, which is a parent of c
a --> b --> c                  # the same edges, written the other direction
a.out[0] <<< b c                # b and c each spend output 0 of a
a < b < c                      # ordering: a must be created before b before c
a.out[0] = 100 HTR [wallet1]   # output 0 of a holds 100 HTR, owned by wallet1
a.weight = 50                  # set a vertex attribute
dummy                          # auto-created tx spending genesis, to fund user txs
```

Here is a real example lifted from a consensus test (`hathor_tests/consensus/test_first_block.py:23-43`):

```

blockchain genesis b[1..50]

b30 < dummy

tx10.out[0] <<< tx50
tx20.out[0] <<< tx50
tx30 <-- tx50
tx40 <-- tx50

b31 --> tx10
b32 --> tx30
b33 --> tx50

```

Read top to bottom: make a 50-block chain off genesis; place a funding `dummy` transaction after block 30 (so its outputs clear the reward lock); transaction `tx50` spends output 0 of `tx10` and `tx20` and lists `tx30` and `tx40` as parents; and blocks `b31`, `b32`, `b33` confirm `tx10`, `tx30`, `tx50` respectively. A test can then fetch `tx50` and assert which block first confirmed it — a precise, readable consensus scenario in ten lines.

Tokenize, then assemble

`build_from_str(content)` (`hathor/dag_builder/builder.py:353-356`) is the entry point. It runs in two passes. First, `tokenize` (`tokenizer.py:164-262`) reads the text line by line, strips comments at `#`, and emits a stream of typed tokens — `BLOCKCHAIN`, `PARENT`, `SPEND`, `OUTPUT`, `ORDER_BEFORE`, `ATTRIBUTE` (the `TokenType` enum, `tokenizer.py:134-141`). For example a `<--` line is split into pairs and each becomes a `PARENT` token (`tokenizer.py:232-234`); an `a.out[i] <<< ...` line becomes one `SPEND` token per spender (`tokenizer.py:240-248`).

Second, `parse_tokens` (`builder.py:105-128`) consumes that stream with a `match` statement and mutates an in-memory graph of `DAGNode` objects (`types.py:42-57`). Each node records its parents, inputs (`DAGInput` = source node + output index, `types.py:99-101`), outputs (`DAGOutput` = amount + token + attributes, `types.py:104-107`), and dependencies. A `SPEND` token, for instance, ensures the spent output exists and adds a `DAGInput` to the spender (`builder.py:187-195`).

`build()` (`builder.py:348-351`) then finishes the job in two final steps. A `DefaultFiller` fills in anything the test left unspecified — output amounts that balance, missing parents — so the test only states what it cares about. Then a `VertexExporter` walks the graph in **topological order** (`topological_sorting`, `builder.py:310-346` — a dependency-respecting order, the same idea as a DAG toposort from Chapter 8) and turns each abstract `DAGNode` into a real `Block` or `Transaction`, allocating wallets, building outputs, wiring inputs to the right output bytes, and signing. If the description contains a cycle, the toposort detects it and raises (`builder.py:326-336`).

The result: DAGArtifacts

The output is a `DAGArtifacts` (`hathor/dag_builder/artifacts.py:33`). It holds the built vertices both as an ordered tuple and a name-keyed map (`artifacts.py:34-43`), so a test can write `artifacts.by_name['tx50']` to get the real vertex it described. Two conveniences matter:

- `get_typed_vertex(name, Block)` (`artifacts.py:51-55`) fetches a vertex and asserts its type, so the test gets a correctly-typed object.
- `propagate_with(manager, up_to=...)` (`artifacts.py:61-103`) feeds the vertices, in built order, into a real node through `vertex_handler.on_new_relayed_vertex` (`artifacts.py:88`) — the same ingestion pipeline from Chapter 33 — optionally stopping at a named vertex. This is how a test moves from "a described DAG" to "a node that has actually ingested that DAG," ready to be queried.

Tests usually construct the builder via a small wrapper,

`TestDAGBuilder.from_manager(manager)` (`hathor_tests/dag_builder/builder.py:28`), which calls the real `DAGBuilder.from_manager` (`builder.py:83-100`) with a test wallet and blueprint module. There is also a standalone CLI, `hathor/dag_builder/cli.py`, that reads a DSL file and prints each resulting vertex as hex (`cli.py:18-61`) — handy for generating fixtures.

43.8 How it plugs into the lifecycle

Neither package runs inside a production node. Both are imported by the test suite (Chapter 20) and by two small CLI tools.

A simulator-based test typically inherits `SimulatorTestCase` (`hathor_tests/simulation/base.py:8`), which builds a `Simulator`, prints its seed, and tears it down. From there a sync test reads almost like prose: create two peers, connect them with a `FakeConnection`, register it, and `run_until_complete` — then assert both are synced. The real sync test does exactly this (`hathor_tests/simulation/test_simulator.py:48-49` create and register a `FakeConnection` with 150 ms fake latency; later loops add auto-reconnecting connections to model flapping links). Because every input is seeded and time is stepped, a failure prints a seed that reproduces it forever.

Through that one mechanism the simulator exercises the heaviest machinery in the book, deterministically:

- **Consensus (Chapter 32)** — generators emitting conflicting transactions and competing blocks let tests assert exactly which vertex is voided and which chain wins after a reorg. The soft-voided-transaction tests (`hathor_tests/consensus/test_soft_voided*.py`) are built on the simulator.

- **Sync (Chapter 35)** — two or more peers run their real sync-v2 agents over `FakeConnections` until `is_both_synced()` holds; split-brain and late-joiner scenarios are modelled with latency and reconnection (`hathor_tests/p2p/test_sync.py`, `test_split_brain.py`).
- **The full ingestion pipeline (Chapter 33)** — every block and transaction the simulator or `dag_builder` produces enters the node through the same `vertex_handler` path the live node uses (verify → consensus → store), so the test exercises the real code, not a mock.
- **Feature activation (Chapter 38)** — the geometric miner accepts a list of `signal_bits` per block (`geometric_miner.py:40-51, 72-82`), so a test can drive the bit-signalling schedule deterministically (`hathor_tests/feature_activation/test_feature_simulation.py`).

The two CLI tools are thin: `hathor_cli/events_simulator/` runs a named scenario on a `Simulator` and forwards the resulting events over a WebSocket — used to develop event-queue clients against a reproducible event stream. And `hathor/dag_builder/cli.py` turns a DSL file into hex vertices for fixtures.

This is where the whole book's machinery gets tested. Every subsystem walked in Part II — the vertex model, storage, verification, consensus, the peer-to-peer layer, sync — meets here, run together, against a clock you control, with bugs that stay reproducible.

Recap

Piece	File	Role
<code>HeapClock</code> / <code>MemoryReactorHeapClock</code>	<code>simulator/clock.py:25, 93</code>	fake reactor; <code>advance()</code> steps simulated time by hand
<code>Simulator</code>	<code>simulator/simulator.py:52</code>	builds peers, holds the clock, <code>run()</code> steps the world
<code>FakeConnection</code>	<code>simulator/fake_connection.py:50</code>	wires two managers in memory; <code>run_one_step()</code> ferries bytes
PoW patches	<code>simulator/patches.py:27, 33</code>	skip real proof-of-work; hashes recomputed, not ground
<code>GeometricMiner</code>	<code>simulator/miner/geometric_miner.py:30</code>	blocks via a seeded geometric timing model

Piece	File	Role
RandomTransactionGenerator	simulator/ tx_generator.py:36	transactions on a seeded schedule, optional double- spends
Trigger family	simulator/trigger.py: 28	stop conditions: N blocks, N txs, synced, ...
The DSL	dag_builder/ tokenizer.py:20	text grammar for describing a DAG
DAGBuilder	dag_builder/ builder.py:52	tokenize → assemble nodes → export real vertices
DAGArtifacts	dag_builder/ artifacts.py:33	the built vertices; propagate_with() ingests them

Determinism is the whole point. Wall-clock time becomes a variable you step; the network becomes an in-memory shuttle you pump; proof-of-work becomes a seeded timing model; randomness and storage collapse to one seed and one throwaway database. With all five sources of non-determinism removed, a network scenario replays identically every run — and the rarest, nastiest class of bug, the timing-dependent consensus or sync failure, stops being a ghost you glimpse once in a log and becomes a test that fails every time until you fix it. The dag_builder complements this by letting a test state the exact graph it wants in a handful of readable lines instead of pages of object construction.

With that, **Part II is complete**. We have walked the node end to end: from the command line that boots it (Chapter 21) and the settings that shape it (Chapter 22), through the vertex model, serialization, storage, and indexes that hold its data (Chapters 25–28); the manager, events, verification, consensus, and ingestion that give it life and rules (Chapters 29–33); the peer-to-peer and sync layers that connect it to the world and the service surfaces it exposes (Chapters 34–37); the higher subsystems of feature activation, nano-contracts, and wallets (Chapters 38–40); the runtime control and observability that keep it operable (Chapters 41–42); and finally, here, the deterministic testing infrastructure that keeps all of it honest. What remains are the **appendices** — the glossary and jargon index (Appendix A), the annotated dependency manifest (Appendix B), and the cheat sheets (Appendix C) — reference material to keep beside you, not new ground to learn.

-
1. Reorg (reorganization) is when the node discovers a heavier chain of blocks than the one it currently treats as canonical, and switches to it — re-deciding which recent blocks and transactions count. Reorgs are the classic timing-dependent scenario, and a prime target for simulator tests. Full treatment in Chapter 32. ↩
 2. A heap is a tree-shaped data structure kept partly ordered so that the smallest (or largest) item is always at the front and cheap to remove. Python's `heapq` gives this over a plain list. The simulator uses a min-heap keyed by fire-time so "the next timer to fire" is always at hand. ↩
 3. The geometric distribution models the number of independent yes/no trials until the first success, when each trial succeeds with probability p . Mining is exactly that — repeated nonce attempts, each a long shot — so drawing one geometric sample reproduces "how long mining would have taken" without running the trials. ↩
 4. A domain-specific language (DSL) is a small, purpose-built notation for one narrow job — here, describing a DAG of blocks and transactions — as opposed to a general-purpose language like Python. The `dag_builder` DSL is parsed by a hand-written tokenizer; it is not Python, though a few attribute values are parsed as Python expressions. ↩

Chapter 44 — The Life of the Node: Everything Working Together

What this chapter does

- Takes one concrete event — **Alice sends Bob 30 HTR** — and follows it through every subsystem the book has described, so the machine is finally seen running as a whole.
- Traces four lifecycles in turn: a **connection** (the stage everything happens on), a **transaction** (the spine), a **block** (told as the differences from the transaction), and a **nano-contract call** (the one path that breaks the usual rules).
- Re-introduces nothing. Each step says what happens here and why, then points to the chapter that explains it in depth (→ Ch N). If a step is unclear, that chapter is the place to go.

Every chapter until now took one subsystem and opened it up. This one does the reverse: it keeps a single event moving and lets you watch each subsystem do its part as the event passes through. The goal is the thread, not the detail — you have the detail already. A developer who reads only this chapter should come away understanding how the pieces connect; a developer who wants to know why any single step works the way it does has forty-three chapters waiting.

We anchor everything on one scenario, fixed now and carried to the end:

The scenario. Alice's wallet holds two unspent outputs locked to her address: one worth **20 HTR** and one worth **15 HTR** (35 total). She wants to pay **Bob 30 HTR**. Her node is `N_A`; Bob runs `N_B`; between and around them is the rest of the network. We follow the payment from the moment her wallet decides to spend, to the moment a block buries it as settled history.

Alice's transaction "pay Bob 30"

INPUTS (35 consumed)

← her 20-HTR output
← her 15-HTR output

in: $20 + 15 = 35$

OUTPUTS (35 created)

30 HTR → locked to Bob	(change)
5 HTR → locked to Alice	

out: $30 + 5 = 35$ (conserved)

Part 1 — The Life of a Connection

Before any payment can move, a node must be part of the network. The connection is the stage; the transaction is the actor that walks onto it. We follow how `N_A` comes to have a live, synced link to a peer.

1.1 Boot and assembly

`N_A` starts when an operator runs `hathor-cli run_node` (→ Ch 21). The command is dispatched, arguments are parsed, and the settings profile for the network is loaded and frozen — this fixes the genesis, the timing constants, and every rule the node will enforce, so that `N_A` speaks the same dialect as every other node (→ Ch 22). The Twisted reactor — the single event loop the whole node will run on — is created but not yet started (→ Ch 16, 23).

The builder then constructs and wires every subsystem into one `HathorManager`: storage opens, indexes are prepared, the wallet loads, the pub-sub bus and the verification and consensus services come up (→ Ch 24). The manager's `start()` runs them in order — crash-check, load genesis, narrow the storage scope to valid data, bring each subsystem online — and marks the node `READY`. The very last act of boot hands the one thread to the reactor, and from here `N_A` is reactive: asleep until something happens (→ Ch 29).

1.2 Finding a peer and shaking hands

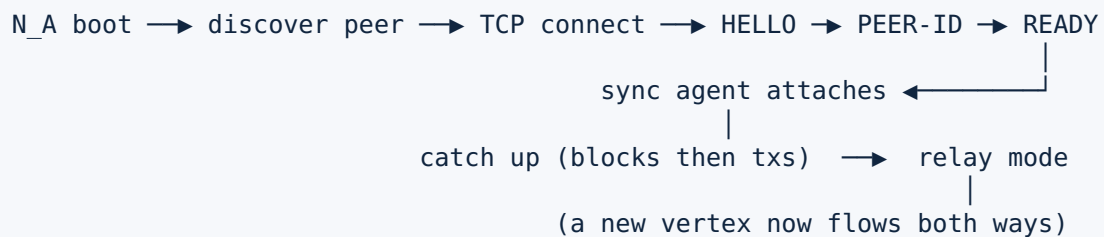
A freshly-started node knows no peers. The connections manager discovers them — from a bootstrap list and from peers that share the addresses they know — and opens an outbound TCP connection to one (→ Ch 34). Each connection is driven by its own protocol object running on the reactor, and it climbs a three-step handshake state machine. In **HELLO**, the two nodes exchange protocol and sync versions, confirm they are on the same network, and agree on the genesis. In **PEER-ID**, each proves its identity. Only when both sides are satisfied does the connection reach **READY** (→ Ch 34).

The handshake is not ceremony. The network match prevents a testnet node from polluting a mainnet peer; the identity step lets a node recognise and de-duplicate peers; the version negotiation picks the one sync protocol both speak. A connection that fails any step is dropped, which is a normal outcome rather than an error.

1.3 Catching up: sync

The moment a connection is **READY**, a **sync agent** attaches to it and the two nodes reconcile their ledgers (→ Ch 35). If **N_A** is behind, the agent finds the highest block the two share — an n-ary search over block heights — then streams the missing blocks in height order, and for each block streams the transactions it confirms, handing every received vertex into the ingestion pipeline we are about to follow in Part 2. Once caught up, the agent flips into **relay** mode: from now on, any new vertex either node learns about is pushed to the other in real time.

That relay subscription is the bridge between this part and the next. When Alice's wallet finally broadcasts her payment, it travels to **N_B** — and to the wider network — over exactly these standing, synced connections.



Part 2 — The Life of a Transaction

Now the actor walks on. We follow Alice's payment from her wallet's decision to spend, through verification and consensus, into storage, and back out as a notification she can see. This is the spine of the whole node, and the longest trace.

2.1 The wallet builds the transaction

Alice's wallet has been tracking which outputs she can spend by watching the ledger and keeping her unspent outputs in view (→ Ch 40, 28). When she asks to send 30 HTR, the wallet performs input selection: it picks a set of her unspent outputs whose values cover 30, here the 20-HTR and the 15-HTR outputs, totalling 35. Because outputs are spent whole and 35 exceeds 30, the wallet creates a second output — 5 HTR back to Alice — as change (→ Ch 7).

The transaction now has a definite shape: two **inputs**, each a pointer to one of Alice's prior outputs, and two **outputs**, one of 30 locked to Bob's address and one of 5 locked back to Alice. The wallet also chooses two **parents** for the transaction — tips of the DAG it will confirm — because in Hathor a transaction attaches itself directly to the graph rather than waiting to be packed into a block (→ Ch 8). At this point the transaction is structurally complete but unsigned and unproven.

2.2 Signing: proving ownership

Each of Alice's two inputs spends an output that was locked to her address with a script demanding a signature from her key (→ Ch 7, 31). The wallet computes the transaction's sighash — a hash over the transaction's contents that the signature will commit to — and signs it with Alice's private key, producing, for each input, an unlocking payload of `<signature> <public-key>` (→ Ch 40). This is the heart of ownership in a UTXO ledger: Alice does not assert that she owns the coins, she demonstrates it by producing data that satisfies each output's lock. No name, no account — a satisfiable lock and the key to open it.

2.3 Proof-of-work: the anti-spam cost

A Hathor transaction, not only a block, carries proof-of-work (→ Ch 9). Before it can be accepted anywhere, Alice's transaction must meet a minimum weight computed from its size and amount — a small cost that makes flooding the DAG with junk transactions expensive. The wallet grinds a nonce until the transaction's hash falls under the target implied by that weight (→ Ch 37). Because this is a brief burst of pure computation, it

runs off the reactor in a thread pool, so the node it is attached to never freezes while the work is done (→ Ch 2, 16). When a valid nonce is found, the transaction has its identity — its hash — and is ready to enter the world.

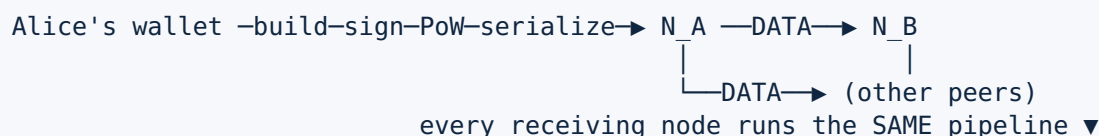
2.4 Serialization: becoming bytes

To leave Alice's wallet, the transaction must become a flat sequence of bytes — and the exact bytes matter, because the hash everyone will use to identify and verify it is computed over them (→ Ch 26). Hathor's bespoke binary format encodes the version and signal bits, then the funds section (the token list, inputs, and outputs), then the graph section (the parents), then the nonce, in a fixed, deterministic order.

Determinism is the point: every node that serializes this transaction produces byte-for-byte the same result, so every node computes the same hash and reaches the same verdict. The same bytes will be sent on the wire and written to disk.

2.5 Propagation: onto the network

Alice's node hands the serialized transaction to its peers over the standing relay connections from Part 1 (→ Ch 34). It travels as one line on the wire — a `DATA` message carrying the transaction's bytes — to each connected peer, including, after a hop or two across the network, Bob's node `N_B`. Each peer that receives it will run the very same acceptance pipeline that `N_A` ran when it first built or received the transaction; there is no privileged path. From here we follow the transaction's arrival at one node — say `N_B` — knowing every other node does the identical thing.



2.6 Ingestion: the vertex handler

When the bytes arrive at `N_B`, they are parsed back into a transaction object and handed to the **vertex handler** — the single chokepoint where "a vertex arrived" becomes "the ledger changed" (→ Ch 33). The handler first checks whether the transaction is already known, and whether its dependencies — the outputs it spends and the parents it names — are present. If everything it needs is in hand, the handler orchestrates the rest of this part as one ordered sequence: verify, then reach consensus, then save and announce. If a dependency is missing, the transaction waits until sync supplies it, so a vertex is never processed before the things it rests on.

2.7 Verification: is it valid on its own?

The verifier asks a single question: is this transaction well-formed and rule-abiding in isolation, regardless of anything else on the ledger (→ Ch 31)? It re-computes the hash and confirms the proof-of-work meets the target. It checks the structure and weight. For each input, it runs the unlocking script against the locked output's script on a small stack machine — this is where Alice's signatures are actually checked against the public keys, and a forged or mismatched signature fails here (→ Ch 31). It confirms conservation: the inputs sum to 35, the outputs sum to 35, nothing is created from nothing. A transaction that fails any check is rejected outright and never enters the ledger. Alice's is well-formed, correctly signed, and balanced, so it passes.

2.8 Consensus: which history does it belong to?

Verification proved the transaction is valid; consensus decides whether it is canonical (→ Ch 32). The key question for a payment is conflict: does any other transaction already in the DAG spend one of the same two outputs Alice is spending? If not — the normal case — Alice's transaction is accepted as executed, its metadata records it as not voided, and the outputs it consumes are now marked spent. If a conflicting transaction did exist (a double-spend), consensus would compare the accumulated weight behind each and let the heavier one win, voiding the lighter; on an exact tie, both stay voided until a later vertex breaks the deadlock (→ Ch 32). Because Alice spent her coins once, honestly, her transaction takes its place in the canonical ledger without contest.

2.9 Storage: writing it down

With the transaction verified and its consensus state decided, the node persists it (→ Ch 27). The serialized body is written under the transaction's hash in one RocksDB column family; its mutable metadata — the consensus verdict, the accumulated weight, the spent markers — goes in another; its immutable computed metadata in a third. The two outputs Alice spent are not deleted; they are recorded as spent by this transaction, which is what lets a later reorganization cleanly reverse the spend if consensus ever changes its mind. The transaction is now durable: a restart of the node will find it exactly here.

2.10 Indexes: making it findable

Storing the transaction by hash answers "give me the vertex with this hash," but not the questions wallets actually ask — "what can Bob spend now?" The node updates its derived indexes to reflect the new reality (→ Ch 28). The UTXO index removes Alice's two consumed outputs and adds the two new ones: a 30-HTR output now spendable by Bob, and a 5-HTR change output spendable by Alice. The address index records that

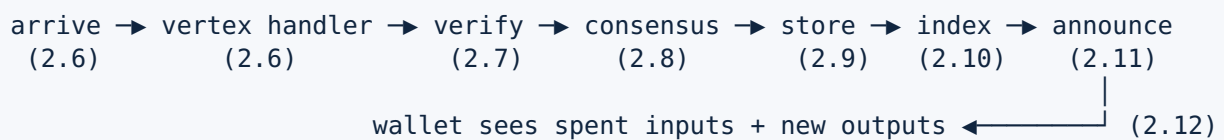
Bob's and Alice's addresses each have new history. None of this is new truth — every index entry could be rebuilt from the stored transactions — but it turns an expensive scan into an instant lookup.

2.11 Announcement: telling the interested

Finally the node announces what happened on its internal pub-sub bus (→ Ch 30). Components that subscribed to "a new transaction was accepted" react: the metrics counters tick, any connected wallet or dashboard watching over a WebSocket is notified (→ Ch 36, 42), and — if the durable event queue is enabled — the event is appended to a replayable log for downstream systems (→ Ch 30). The same acceptance also re-enters the relay path: `N_B` now forwards the transaction to its peers, which is how the payment ripples out until the whole network holds it.

2.12 The wallet sees its change

The loop closes back at Alice. Her wallet is subscribed to the same kind of notification, and when her node accepts the transaction, the wallet sees the two events that concern her: her 20-HTR and 15-HTR outputs are now spent, and a fresh 5-HTR change output is now hers (→ Ch 40). Bob's wallet, on his node, likewise sees a new 30-HTR output it controls. Neither wallet stores a balance; each re-sums its unspent outputs, and the numbers have moved by 30. The payment has happened — but, as the next part shows, it is not yet final.



Part 3 — The Life of a Block

A block travels much of the same road as a transaction — it is also a vertex, also verified, also run through consensus and stored. So rather than repeat Part 2, this part tells only the **differences**: what a block does that a transaction does not. The block we follow is the one that will eventually confirm Alice's payment and turn it from "accepted" into "settled."

3.1 The template: assembling a candidate

A transaction is built by a wallet; a block is built by the node, on request, as a template for miners (→ Ch 37). The node selects the parents the block will extend — including the previous block, which keeps the block backbone intact — sets the reward outputs that mint new HTR, and computes the minimum weight the block must meet, tuned by the difficulty algorithm toward the network's target of one block roughly every 30 seconds (→ Ch 9). The template is a fill-in-the-nonce puzzle: everything is fixed except the number a miner must find.

3.2 Mining and submission

Unlike Alice's transaction, whose small proof-of-work her wallet did itself, a block's proof-of-work is large and is done by external miners — real mining hardware speaking the Stratum protocol, or a merged-mining setup riding on Bitcoin (→ Ch 37). A miner repeatedly hashes the template with different nonces until one produces a hash under the block's hard target. When it succeeds, it submits the solved block back to the node, which feeds it into the same ingestion pipeline Alice's transaction took — vertex handler, verification, consensus, storage — with the block-specific rules layered on.

3.3 What consensus does differently for a block

Here is the real divergence. A transaction's consensus question is "does it conflict with another spend?"; a block's is "does it change which chain is the canonical one?" (→ Ch 32). Blocks compete by score — the total work of the sub-DAG behind them — and the chain with the highest score is the real history. When the new block extends the current best chain, it becomes the new tip with nothing more to decide. But if a competing branch had quietly grown heavier, accepting this block triggers a **reorganization**: the node switches to the heavier branch, voids the blocks and transactions unique to the branch it abandons, and re-applies any transactions that belong on the new one (→ Ch 32). This is the mechanism behind "finality is only probabilistic" — a recently-accepted transaction can still be undone by a reorg, which is why depth matters.

3.4 Confirming Alice's payment

The block we are following does not conflict with anything; it extends the best chain, and in doing so it reaches back through its parents to confirm the region of the DAG that contains Alice's transaction (→ Ch 8, 9). That confirmation is what changes Alice's payment from "accepted by the network" to "buried under work." Each further block built on top adds more accumulated weight that an attacker would have to out-compute to reverse her payment, so the probability of reversal falls steeply with every block (→ Ch 9). After a handful of blocks, Bob can treat the 30 HTR as his to spend without practical risk.

3.5 Two side-effects only blocks have

Accepting a block also advances two machineries a transaction never touches. First, **feature activation**: the block's signal bits are counted into the rolling tally that decides whether a proposed protocol upgrade has gathered enough miner support to lock in and activate (→ Ch 38). Second, **nano-contract execution**: contracts do not run when their calling transaction arrives — they run now, at block consensus, in a deterministic order, with their results committed to contract state (→ Ch 39, and Part 4 below). A block, then, is not only a confirmation of past transactions but the heartbeat that drives the network's slow-moving machinery forward.

Transaction trace (Part 2):	build – verify – consensus(conflict?) – store – announce
Block trace adds/changes:	template – EXTERNAL mining – consensus(SCORE, reorg?) – reward minting – confirms txs beneath it – feature signalling – runs nano-contracts

Part 4 — The Life of a Nano-Contract Call

One last trace, because it breaks an assumption that held through Parts 2 and 3: that a vertex's effect is settled the moment consensus accepts it. A nano-contract call is a transaction, and it travels the entire Part 2 pipeline — built, signed, serialized, propagated, verified, consensus-checked, stored. But what the contract does happens later, and elsewhere.

4.1 A call arrives, but does not run

When a nano-contract transaction is accepted, its method does not execute on arrival (→ Ch 39). The transaction carries, in a header, the identity of the contract and the method to call, plus any token deposits or withdrawals the call makes. Acceptance records the intent to run, but execution is deferred — because running a state-changing program the instant it arrives, before the network has agreed on its place in history, would let conflicting or soon-to-be-voided calls corrupt contract state.

4.2 Execution at block consensus

Instead, contracts run when a **block** confirms them, as part of that block's consensus (→ Ch 39, 32). The block fixes an order, and the calls it confirms are executed in a deterministic, seeded sequence so that every node, replaying the same block, produces the same result. Each call is handed to the runner, which executes the blueprint's method against the contract's current state. A resource-metering layer is intended to bound how much work a call may do — though on the current branch that bound is scaffolded rather than enforced, while the sandbox that restricts what a contract may touch is active (→ Ch 39). This is the one place in the node where the "execute on arrival" model of Part 2 is deliberately set aside.

4.3 State, and the cost of failure

A contract's state lives not in outputs but in a verifiable Merkle/Patricia trie, one per contract, whose root is anchored into the confirming block (→ Ch 39). A successful call commits its state changes into the trie and moves the contract forward. A call that fails — runs out of its (intended) budget, violates a rule, raises — is marked with a dedicated voiding identifier, its state changes are rolled back, and the failure is recorded rather than allowed to corrupt anything (→ Ch 32, 39). Because state is trie-

backed and block-anchored, two nodes can prove to each other that they hold the same contract state by comparing roots — the same verifiability that the rest of the ledger gets from its hashes.

Recap

Lifecycle	Spans	The one thing to remember
Connection (Part 1)	Ch 21–24, 29, 34, 35	a synced, relayed link is the stage every vertex travels on
Transaction (Part 2)	Ch 7, 8, 9, 26, 28, 30–34, 37, 40	arrive → verify → consensus → store → index → announce, at every node identically
Block (Part 3)	Ch 9, 32, 37, 38, 39	same pipeline, but consensus is by score, and a block confirms and triggers
Nano-contract (Part 4)	Ch 32, 39	accepted on arrival, but executed later, at block consensus, against trie state

Trace one payment all the way through and the node stops being a collection of packages and becomes a single machine: a wallet that proves ownership with a signature, a binary format that makes the proof reproducible, a network that carries it, a pipeline that checks and records it, a block that buries it, and a chain of further blocks that makes burying it irreversible. Every subsystem in this book exists to play one part in that sequence, and the sequence is the node. With the four lifecycles in hand — and the chapters behind each step when a detail is wanted — you can now open `hathor-core` anywhere and know not just what a piece does, but where it sits in the life of the thing passing through it.

Chapter 44 (Extended) — The Life of the Node: Everything Working Together

What this chapter does

- Takes one concrete event — **Alice sends Bob 30 HTR** — and follows it through every subsystem the book has described, so the machine is finally seen running as a whole rather than as parts.
- Traces four lifecycles in turn: a **connection** (the stage everything happens on), a **transaction** (the spine), a **block** (told as the differences from the transaction), and a **nano-contract call** (the one path that breaks the usual rules).
- Re-derives nothing from first principles. Each step gives enough framing to follow it standing alone, describes what happens to our transaction and why, then points to the chapter that proves it (→ Ch N). Read this for the thread; read the cited chapter for the depth.

Every chapter until now took one subsystem and opened it up on a workbench. This one does the reverse: it keeps a single event moving and lets you watch each subsystem do its part as the event passes through. The value here is the thread — the way the pieces hand off to one another — not the internals, which you already have. A developer who reads only this chapter should finish able to narrate how a payment lives and dies inside the node; a developer who wants to know why any single step behaves as it does has forty-three chapters waiting behind the pointers.

We anchor everything on one scenario, fixed now and carried to the end. It is deliberately small — two inputs, two outputs — because a small trace you can hold in your head teaches more than a large one you cannot.

The scenario. Alice's wallet holds two unspent outputs locked to her address: one worth **20 HTR** and one worth **15 HTR** (35 total). She wants to pay **Bob 30 HTR**. Her node is `N_A`; Bob runs `N_B`; around them is the rest of the network. We follow the payment from the moment her wallet decides to spend, to the moment a block buries it as settled history — and then, in Part 4, we follow a different kind of transaction that does not finish when it is accepted.

Alice's transaction "pay Bob 30"

INPUTS (35 consumed)

← her 20-HTR output
← her 15-HTR output

in: $20 + 15 = 35$

OUTPUTS (35 created)

30 HTR → locked to Bob	(change)
5 HTR → locked to Alice	

out: $30 + 5 = 35$ (conserved)

A note on the order of the four parts. We start with the connection because nothing else can happen until a node is woven into the network; then the transaction, which is the richest and most representative trace; then the block, which we tell only as its differences from the transaction so as not to repeat ourselves; and finally the nano-contract call, which earns its own part precisely because it violates an assumption the first three parts quietly rely on.

Part 1 — The Life of a Connection

A transaction needs somewhere to travel. Before Alice's payment can reach Bob or anyone else, her node must already be a living member of the network: booted, connected to peers, and caught up with the shared ledger. This part follows `N_A` from a cold start to a standing, synced link — the stage on which the rest of the chapter is performed.

1.1 Boot and assembly

`N_A` comes to life when an operator runs `hathor-cli run_node`. That single command is routed through a dictionary of subcommands to the `run-node` path, its flags are parsed, and — the first consequential decision — the **settings profile** for the chosen network is loaded and frozen (→ Ch 21, 22). This profile fixes the genesis vertices, the timing constants, the consensus rules: everything that makes "mainnet" a different network from "testnet." Get it wrong and `N_A` would not be a broken node, it would be a node speaking a different language, unable to agree with anyone. The Twisted **reactor** — the one event loop the whole node will run on — is created at this point but left idle, like starting a car's electrical system before putting it in gear (→ Ch 16, 23).

The node then assembles itself. A builder constructs each subsystem in dependency order and wires them into a single coordinating object, the `HathorManager`: the RocksDB storage opens, the indexes are prepared, the wallet loads, the pub-sub bus is created, and the verification and consensus services are built (→ Ch 24). Construction is separate from operation on purpose — the builder makes the parts, the manager runs them.

Starting is its own careful sequence. The manager first checks a flag in storage: if the last shutdown was unclean, the node refuses to start rather than risk acting on metadata that may be corrupt. It then loads the genesis, rebuilds its in-memory view of the ledger from disk, brings each subsystem online in order, and marks itself `READY` (→ Ch 29). The final act of boot is to hand the one thread to the reactor and let it run. From here `N_A` is reactive: it sleeps until something happens — a peer connects, a timer fires, bytes arrive — and responds.

1.2 Finding a peer

A node just born knows no one. The connections manager solves this two ways at once: it reads a small bootstrap list of well-known peers, and it asks the peers it does reach for the addresses they know, so the node's view of the network grows by word of mouth (→ Ch 34). For each candidate it opens an outbound TCP connection, and each

connection becomes its own protocol object living on the reactor — one of potentially thousands, all multiplexed onto the single thread because the node never blocks waiting on any one of them (→ Ch 2, 16).

1.3 The handshake

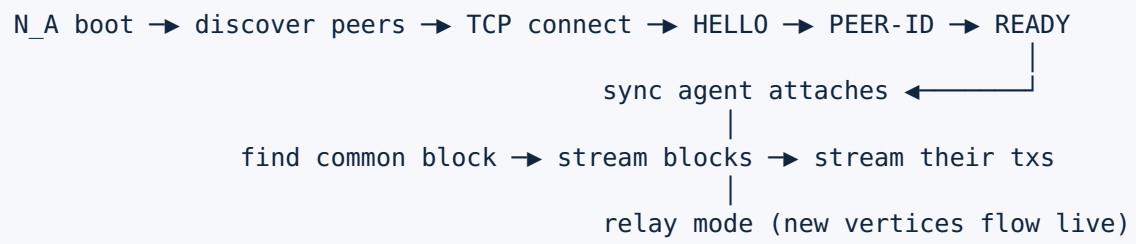
A raw TCP connection is not yet a peer. Each connection climbs a three-step state machine before it is trusted to carry ledger data (→ Ch 34). In **HELLO**, the two nodes exchange protocol and sync versions, confirm they are on the same network, and check that they share the same genesis — a mismatch here means the two are not even trying to maintain the same ledger, and the connection is dropped. In **PEER-ID**, each side proves its identity, which lets the node recognise peers it has seen before and avoid connecting to the same one twice. Only when both sides have completed both steps does the connection transition to **READY**.

It is worth pausing on why a payment system bothers with this much ceremony before exchanging a single transaction. The handshake is the network's immune system: the network check keeps incompatible chains apart, the identity check resists a peer pretending to be many, and the version negotiation guarantees both sides speak one agreed protocol. A connection that cannot satisfy all of it is not an error to be logged and retried in panic — it is only a door that did not open, and the node moves on to the next.

1.4 Catching up, then relaying

The instant a connection reaches **READY**, a **sync agent** attaches to it and the two nodes reconcile their ledgers (→ Ch 35). If **N_A** is behind — a fresh node may have almost nothing while the network has years of history — the agent first finds the most recent block the two peers share, using an n-ary search over block heights that converges in a handful of round-trips rather than walking the whole chain. It then streams the missing blocks in height order, the block backbone first, and for each block streams the transactions that block confirms. Every vertex received this way is fed into the ingestion pipeline that Part 2 is about to follow in detail; sync is not a special path, it is a firehose feeding the ordinary one.

Once **N_A** has caught up, the agent shifts the connection into **relay** mode. From now on, any new vertex either node learns about is pushed across the link in real time. This is the hinge between Part 1 and Part 2: the standing, synced, relaying connections built here are the very wires along which Alice's payment will travel the moment her wallet lets it go.



Part 2 — The Life of a Transaction

Now the actor walks onto the stage. We follow Alice's payment from the instant her wallet decides to spend, through the proofs that make it acceptable, across the network, and through the pipeline every node runs to absorb it — ending where it began, at a wallet whose balance has changed. This is the spine of the entire node and the longest trace in the chapter, so we take it one subsystem at a time.

2.1 The wallet builds the transaction

A wallet's quiet, constant job is to know which coins it can spend. Alice's wallet watches the ledger and keeps her unspent outputs in view, so when she asks to send 30 HTR it already has the raw materials at hand (→ Ch 40, 28). Its first task is **input selection**: choose a set of her unspent outputs whose values cover the payment. It picks the 20-HTR and the 15-HTR outputs — 35 total, enough to cover 30. Because outputs in a UTXO ledger are spent whole and cannot be split, the 5 HTR of surplus does not vanish; the wallet creates a second output of 5 HTR locked back to Alice's own address, the **change** (→ Ch 7).

The transaction now has a definite shape: two **inputs**, each a pointer of the form (prior transaction id, output index) identifying exactly one of Alice's coins, and two **outputs**, one of 30 HTR carrying a lock that only Bob's key can open and one of 5 HTR carrying a lock only Alice's key can open. The wallet also selects two **parents** for the transaction — current tips of the DAG — because in Hathor a transaction does not wait to be packed into a block; it attaches itself directly to the graph by confirming earlier vertices (→ Ch 8). Keep the two kinds of edge distinct: the inputs say which coins this spends, the parents say which earlier vertices this builds on. They are wholly separate, and conflating them is the classic way to misread the codebase.

At this moment the transaction is structurally finished but worthless to anyone else: it makes a claim on Alice's coins without yet proving she has the right to make it, and it carries no work to deter spam. The next three steps supply the proofs.

2.2 Signing — proving ownership

Each of Alice's inputs spends an output that was locked, when it was created, with a script demanding a valid signature from the key behind her address (→ Ch 7, 31). Ownership in this model is not a name in a ledger row; it is the ability to satisfy that lock. To do so the wallet computes the transaction's **sighash** — a hash taken over the transaction's contents, so that the signature commits to this exact transaction and

cannot be lifted onto another — and signs it with Alice's private key. For each input it produces an unlocking payload of `<signature> <public-key>`, which it attaches to that input (→ Ch 40).

The asymmetry here is the whole foundation of the system. Anyone can verify Alice's signature using her public key, which is published openly; only Alice, holding the matching private key, could have produced it. So the transaction now carries, for each coin it spends, cryptographic proof that its rightful owner authorised the spend — without that owner ever revealing the secret that proves it.

2.3 Proof-of-work — the anti-spam toll

In most blockchains only blocks carry proof-of-work; in Hathor, every transaction does too, because a transaction attaches itself directly to the ledger and so must pay a small toll for the privilege (→ Ch 9). The wallet computes the minimum **weight** the transaction must meet — a value derived from its size and amount — and then grinds a **nonce**, hashing the transaction over and over with different nonce values until its hash falls below the target that weight implies (→ Ch 37). The cost is deliberately modest for an honest single payment and ruinous for anyone trying to flood the DAG with millions of junk transactions.

Because this grinding is a burst of pure computation rather than waiting on the network, doing it on the node's single reactor thread would freeze every other connection while it ran — so it is pushed onto a thread pool and done off to the side (→ Ch 2, 16). When a satisfying nonce is found, the transaction's hash is now fixed, and that hash is its identity for the rest of its life: the thing every node will use to name it, look it up, and refer to it.

2.4 Serialization — becoming bytes

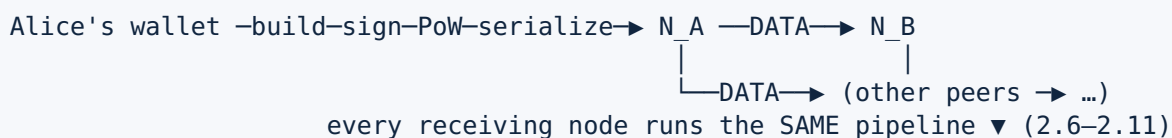
A transaction held as Python objects in Alice's wallet cannot travel a wire or rest on a disk; it must become a flat sequence of **bytes**, and not just any encoding will do (→ Ch 26). The bytes are load-bearing, because the hash that identifies the transaction and the signatures that authorise it are both computed over exactly this byte sequence. Hathor uses a bespoke binary format that lays the fields down in a fixed, deterministic order: the version and signal bits, then the funds section (the token list, the inputs, the outputs), then the graph section (the parents), then the nonce.

Determinism is the reason the format is hand-rolled rather than borrowed from JSON or a general scheme. Every node that serializes this transaction must produce byte-for-byte the same result, because every node must compute the same hash from it and reach the same verdict on it — consensus would be impossible if two honest nodes could encode the same transaction two different ways. The very bytes produced here are what will be handed to the network and, later, written to storage unchanged.

2.5 Propagation — onto the network

Alice's node releases the serialized transaction to its peers over the standing relay connections established in Part 1 (→ Ch 34). On the wire it is a single framed message — a **DATA** message carrying the transaction's bytes — sent to each connected peer. Those peers forward it to their peers, and within moments the transaction has rippled across the network, reaching Bob's node **N_B** after a hop or two.

There is no privileged route and no special recipient. Every node that receives the transaction — Bob's, and every other — runs the identical acceptance pipeline that we follow next; the network has no centre, so "being accepted" means each node independently arrives at the same conclusion. From here we watch the transaction arrive at one node, **N_B**, with the understanding that the same thing is happening everywhere at once.



2.6 Ingestion — the vertex handler

When the bytes arrive at **N_B**, they are parsed back into a transaction object and handed to the **vertex handler**: the single, narrow chokepoint where "a vertex arrived" turns into "the ledger changed" (→ Ch 33). Funnelling every new vertex — whether freshly received or downloaded during sync — through one place is what keeps the node's behaviour consistent and its bugs findable.

The handler's first job is triage. Has it seen this transaction before? If so, it stops; there is nothing to do. Are the things the transaction depends on — the outputs its inputs spend, the parents it names — all present on this node? If a dependency is missing, the transaction cannot be judged yet, and it waits until sync delivers what it lacks, so that a vertex is never evaluated before the things it stands on. For Alice's transaction, assume **N_B** already holds her two spent outputs and the parents she chose; the handler proceeds, orchestrating the next three steps as one ordered unit — verify, then run consensus, then save and announce.

2.7 Verification — is it valid on its own?

Verification asks one bounded question: taken entirely by itself, ignoring everything else on the ledger, does this transaction obey the protocol's rules (→ Ch 31)? The node re-computes the transaction's hash and confirms its proof-of-work clears the target. It checks the structural rules and the minimum weight. For each input, it runs the input's unlocking script together with the spent output's locking script on a small **stack machine** — and this is the precise moment Alice's signatures are tested against the

public keys they claim to match; a forged signature, a tampered output, a spend of someone else's coin, all fail right here (→ Ch 31). It confirms **conservation**: the inputs total 35, the outputs total 35, and no value has been conjured from nothing.

A transaction that fails any of these checks is rejected outright and never enters the ledger — there is no appeal, because the failure is in the transaction itself. Note the careful scope of this stage: it can prove a transaction internally valid without yet knowing whether it belongs in the canonical history, because that second question depends on everything else and is consensus's job, not verification's. Alice's transaction is well-formed, correctly signed, and balanced, so it clears verification and moves on.

2.8 Consensus — which history does it belong to?

Verification proved the transaction is valid; consensus decides whether it is canonical — part of the one history the network agrees on (→ Ch 32). For a plain payment the decisive question is conflict: does any other transaction already in the DAG spend either of the two specific outputs Alice is spending? In the normal case the answer is no. Alice's transaction is accepted as **executed**, its metadata is written to record that it is not voided, and the two outputs it consumes are marked as spent by it.

The interesting case is the one Alice is not in: a **double-spend**, where two transactions each try to consume the same output. The DAG holds both at once — its structure does not forbid the conflict — and consensus resolves it by comparing the **accumulated weight** behind each, letting the heavier history win and marking the lighter transaction voided; on an exact tie, both stay voided until some later vertex tips the balance (→ Ch 32). Voiding is a mark, not a deletion, which is what allows the decision to be reversed later if the weights shift. Because Alice spent her coins exactly once, in good faith, her transaction faces no rival and takes its place in the canonical ledger uncontested.

2.9 Storage — writing it down

With the transaction verified and its consensus state decided, the node makes it durable (→ Ch 27). Hathor's storage is an embedded key-value store split into separate **column families**, and the transaction is written across three of them: its serialized body (the very bytes from §2.4) under its hash in one family, its mutable metadata — the consensus verdict, the accumulated weight, the spent markers — in a second, and its immutable computed metadata in a third. Keeping the unchanging body apart from the evolving metadata lets each be read and rewritten without disturbing the other.

One detail repays attention. The two outputs Alice spent are not erased; they are recorded as spent by this transaction. Marking rather than deleting is what makes a future reorganization survivable: if consensus ever changes its mind about this region

of history, the spend can be cleanly undone and the coins returned to the unspent set, because the record of what they were never went away. Once written, the transaction is permanent — a restart of the node will find it exactly here, byte for byte.

2.10 Indexes — making it findable

Storing the transaction under its hash answers the question "give me the vertex with this hash," but that is not a question wallets ask. Bob's wallet wants to know "what can I spend now?" — a question the raw store cannot answer without scanning everything. So the node maintains **derived indexes** that precompute the answers, and it updates them to reflect the new reality (→ Ch 28). The **UTXO index**, keyed by address, removes Alice's two now-spent outputs and adds the two new ones: a 30-HTR output recorded against Bob's address and a 5-HTR change output recorded against Alice's. The **address index** notes that both addresses have new history.

Nothing in the indexes is independent truth — every entry could be rebuilt from scratch by replaying the stored transactions, which is exactly what the node does when it initialises after a restart. Their entire purpose is speed: they convert a question that would otherwise require walking the whole ledger into a direct lookup, at the cost of the small bookkeeping we just watched.

2.11 Announcement — telling the interested

The transaction is now verified, settled, stored, and indexed; the last step is to tell the parts of the system that care. The node publishes the event on its internal **pub-sub bus**, and whoever subscribed to "a new transaction was accepted" reacts (→ Ch 30). The metrics counters advance (→ Ch 42); any wallet or dashboard watching over a **WebSocket** is pushed a live notification (→ Ch 36); and, if the durable event queue is enabled, the event is appended to a replayable log so that downstream systems can later reconstruct exactly what the node saw and when (→ Ch 30). The publisher knows none of these subscribers individually — it announces into the room, and the interested parties act, which is what lets new subsystems hook into the flow without the core being rewired.

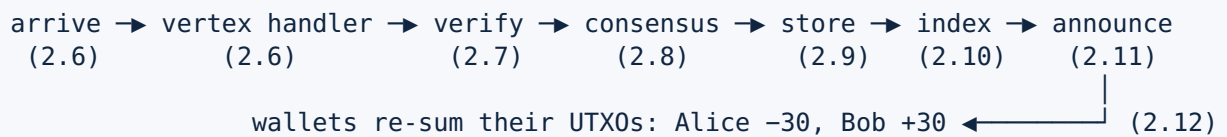
This same acceptance also re-enters the relay path from Part 1: **N_B**, having accepted the transaction, now forwards it to its peers, which accept and forward in turn. That outward ripple is how a payment that began in one wallet comes, within seconds, to be held by the entire network.

2.12 The loop closes — the wallet sees its change

The journey ends where it started, back at a wallet. Alice's wallet subscribes to the same notifications, so when her own node accepts the transaction it sees the two facts that concern her: her 20-HTR and 15-HTR outputs are now spent and gone, and a fresh

5-HTR change output is now hers (→ Ch 40). Bob's wallet, on his node, sees a new 30-HTR output it controls appear. Crucially, neither wallet stores a number called "balance" — each re-sums the unspent outputs it can spend, and after this transaction those sums have moved by exactly 30 HTR. The payment has, to all appearances, happened.

And yet it is not final. A transaction accepted into the DAG is canonical for now, but its place rests only on the work accumulated around it, which is still thin. To turn "accepted" into "settled beyond practical doubt" takes a block — and that is the next trace.



Part 3 — The Life of a Block

A block walks much of the same road as a transaction: it too is a vertex, verified, run through consensus, and stored. Re-telling that shared road would only repeat Part 2, so this part follows the **differences** — the handful of things a block does that a transaction does not. The block we follow is the one whose arrival will finally confirm Alice's payment and begin turning it from "accepted" into "irreversible."

3.1 The template — a node-built candidate

The first difference is in who builds it. Alice's transaction was assembled by her wallet; a block is assembled by the node, on demand, as a **template** for miners to solve (→ Ch 37). To build one, the node selects the block's parents — including, always, the previous block, which keeps the block backbone an unbroken chain — adds the **reward** outputs that mint brand-new HTR into existence, and computes the minimum **weight** the block must meet. That weight is set by the difficulty-adjustment algorithm, which nudges it up or down to keep blocks arriving at the network's target rhythm of roughly one every 30 seconds regardless of how much mining power is at work (→ Ch 9). The finished template is a fill-in-the-blank puzzle: everything is fixed except the nonce.

3.2 Mining and submission — work done elsewhere

The second difference is scale and location. Alice's transaction carried a small proof-of-work her own wallet could grind in a moment; a block's proof-of-work is enormous and is done by dedicated **miners** outside the node — specialised hardware speaking the Stratum protocol, or a merged-mining arrangement that earns Hathor blocks as a by-product of mining Bitcoin (→ Ch 37). A miner takes the template and hashes it with nonce after nonce, billions of times, until one produces a hash beneath the block's hard target. The moment it succeeds, it submits the solved block back to the node — which then feeds it into the same ingestion pipeline Alice's transaction travelled: the vertex handler receives it, verification checks it, consensus weighs it, storage records it. The road is shared; only the rules layered on top differ.

3.3 Consensus by score — and the possibility of a reorg

Here lies the deepest difference. A transaction's consensus question was local — "does this conflict with another spend?" A block's question is global — "does this change which chain is the true one?" (→ Ch 32). Blocks compete by **score**, the total accumulated work of the sub-DAG behind them, and the chain whose tip has the highest score is the canonical history by definition. When our block extends the current best chain, it becomes the new tip and little drama follows.

But if, while our block was being mined, a competing branch had quietly accumulated more work, then accepting our block forces a **reorganization**: the node abandons the lighter branch for the heavier one, marks the blocks and transactions unique to the abandoned branch as voided, and re-applies any transactions that still belong on the new branch (→ Ch 32). This is the concrete machinery behind a phrase from early in the book — that finality is only ever probabilistic. A transaction one or two blocks deep could, in principle, still be undone by a large enough reorg; what protects it is not a guarantee but accumulating cost.

3.4 Confirming Alice's payment

The block we are following conflicts with nothing and extends the best chain, and in doing so it reaches back through its parents into the region of the DAG that holds Alice's transaction, **confirming** it (→ Ch 8, 9). This is the event that changes the character of her payment. Before the block, Alice's transaction was accepted but lightly anchored; now a block's full weight sits on top of it, and every further block mined onto this chain piles more accumulated work above it. To reverse Alice's payment, an attacker would have to privately out-mine the entire honest network from before her transaction forward — a cost that climbs so steeply with each new block that after a handful of confirmations Bob can treat the 30 HTR as unconditionally his. "Probabilistic finality" is, in practice, certainty that deepens with every block.

3.5 Two machineries only blocks drive

A block does two further things no transaction ever does, and both are easy to miss because they happen quietly at acceptance. The first is **feature activation**: each block carries signal bits, and accepting the block folds those bits into a rolling tally that decides whether a proposed protocol upgrade has gathered enough sustained miner support to lock in and, eventually, switch on — the mechanism by which the network changes its own rules without a disruptive flag-day (→ Ch 38). The second is **nano-contract execution**: any contract calls confirmed by this block are run now, as part of the block's consensus, in a deterministic order, with their results committed to contract state (→ Ch 39, and Part 4). A block, then, is more than a stack of confirmed payments — it is the heartbeat that advances the network's slow, deliberate machinery one step with every tick.

Transaction trace (Part 2): build – verify – consensus(conflict?) – store – announce
The block trace adds/changes:

- template built by the NODE, with reward minting (3.1)
- EXTERNAL mining (Stratum / merged), then submission (3.2)
- consensus by SCORE – may trigger a REORG (3.3)
- CONFIRMS the transactions beneath it (Alice's payment) (3.4)
- advances feature signalling + runs nano-contracts (3.5)

Part 4 — The Life of a Nano-Contract Call

The fourth trace earns its own part because it breaks a rule the first three quietly assumed: that a vertex's effect is settled the instant consensus accepts it. A nano-contract call is, on the surface, just another transaction — it is built, signed, serialized, propagated, verified, consensus-checked, and stored exactly as Alice's payment was, travelling the whole of Part 2. But what the contract actually does happens later, and somewhere else, and that gap is the point of this part.

4.1 Accepted on arrival, but not run

A nano-contract transaction carries, in a dedicated header, the identity of the contract it calls and the method to invoke, along with any tokens it deposits into or withdraws from the contract. When such a transaction is accepted, the node records the intention to make that call — but it does **not** execute the method then and there (→ Ch 39). The reason is the same caution that runs through the whole consensus design: running a state-changing program the moment its transaction arrives, before the network has agreed on that transaction's place in history, would let a call that is about to lose a conflict — or be voided in a reorg — leave its fingerprints on shared contract state. Acceptance, here, means "this call is queued and ordered," not "this call has happened."

4.2 Executed at block consensus

Contracts run when a **block** confirms them, as part of that block's consensus (→ Ch 39, 32). A block imposes a definite order, and the calls it confirms are executed in a deterministic, seeded sequence, so that every node replaying the same block performs the same calls in the same order and arrives at exactly the same result — without which, two honest nodes could compute different contract state and the network would fork. Each call is handed to the **runner**, which executes the blueprint's method against the contract's current state. A resource-metering layer is meant to bound how much computation a single call may consume — a defence against a contract that loops forever or tries to exhaust the node — though on the current branch that bound is scaffolded but not yet enforced, even as the sandbox that restricts what a contract is allowed to touch is fully active (→ Ch 39). This is the one place in the entire node where the "execute on arrival" model that governed Alice's payment is deliberately set aside in favour of "execute on confirmation."

4.3 State, verifiability, and the cost of failure

A contract's state does not live in outputs the way Alice's coins do; it lives in a verifiable **Merkle/Patricia trie**, one per contract, whose root is anchored into the confirming block (→ Ch 39). A call that succeeds commits its changes into that trie and advances the contract. A call that fails — exhausts its intended budget, breaks a rule, raises an error mid-execution — is marked with a dedicated voiding identifier, its partial state changes are rolled back as if they never happened, and the failure itself is recorded rather than allowed to corrupt the contract (→ Ch 32, 39). Because the state is trie-backed and its root sits inside the block, two nodes can prove they hold identical contract state by comparing roots alone — the very same verifiability the rest of the ledger earns from its hashes, now extended to running programs.

Recap

Lifecycle	Spans	The one thing to remember
Connection (Part 1)	Ch 21–24, 29, 34, 35	a synced, relaying link is the stage every vertex travels on
Transaction (Part 2)	Ch 7–9, 26, 28, 30–34, 37, 40	arrive → verify → consensus → store → index → announce, at every node identically
Block (Part 3)	Ch 9, 32, 37, 38, 39	same pipeline, but consensus is by score, and a block confirms and triggers
Nano-contract (Part 4)	Ch 32, 39	accepted on arrival, but executed later, at block consensus, against trie state

Follow one small payment all the way through and the node stops being a catalogue of packages and resolves into a single machine. A wallet proves ownership with a signature; a binary format makes that proof reproducible on every node; a network of handshaken, synced connections carries it; a pipeline verifies, agrees on, records, indexes, and announces it; a block buries it; and the chain of blocks that follows makes burying it irreversible. Every subsystem in this book exists to play one part in that sequence, and the sequence, taken whole, is the node. With these four lifecycles in hand — and the chapter behind each step waiting whenever a detail is wanted — you can open `hathor-core` at any file and know not only what the code in front of you does, but where it sits in the life of the thing passing through it.

Appendix A — Glossary & Jargon Index

This appendix collects every term the book defines in a footnote — 298 of them — into one alphabetical reference. Each entry gives a one- or two-sentence definition and a pointer to the chapter (or chapters) where the term is introduced and used in context. Where a term recurs across several chapters, all are listed; the chapter in **bold-ish first position** is usually where it is first defined.

Use this the way a junior reader should: when a later chapter uses a word you have forgotten, look it up here for the quick definition, then follow the chapter pointer if you need the full treatment. Terms are sorted ignoring case and punctuation, so `base58check`, `BFS`, and `BIP32` sit where you would expect them.

How to read an entry. **Headword** · Ch N — definition. The chapter number is a clickable pointer in the source and a lookup target in print.

A

Abstract base class · Ch 1, Ch 25, Ch 26, Ch 27 — An abstract base class (ABC) is a class you cannot instantiate directly; it declares methods (often `@abstractmethod`) that concrete subclasses must implement. `Serializer` and `Deserializer` are ABCs, with `BytesSerializer` / `BytesDeserializer` as the concrete in-memory implementations. Full treatment in Chapter 1.

Abstraction · Ch 1 — Abstraction = exposing what an operation does while hiding how it is done, so callers depend on a simple contract rather than complex internals.

Account model · Ch 7 — The account model (or account/balance model) records ownership as a mutable balance per account; a transaction debits one balance and credits another. Used by banks and by Ethereum. Contrast the UTXO model.

Acyclic · Ch 8 — Acyclic means containing no cycles — no path that follows edges and returns to its starting node. A directed acyclic graph (DAG) combines directedness with acyclicity.

Adapter · Ch 3 — An adapter wraps an object to present a different interface than it natively offers, so it can be used where that other interface is expected. Also called a wrapper.

Any · Ch 5 — `Any` is the type that is compatible with everything and disables checking for the values it touches. It is the deliberate escape hatch from the type system; overusing it defeats the purpose of typing.

Argparse · Ch 19 — `argparse` is Python's standard-library module for parsing command-line arguments: you declare the options a program accepts, and it parses `sys.argv` into a structured object and generates help text. It handles only the command line.

Args in a function signature collects any extra** · *Ch 4* — args in a function signature collects any extra **positional** arguments into a tuple. In a call, `seq` unpacks a sequence into separate positional arguments. The name `args` is convention; the `*` is what matters.

Assertnever · Ch 31 — `assert_never(x)` is a typing helper that the static type-checker treats as "this line must be unreachable." If the checker can prove some case reaches it (e.g. an unhandled enum value), the build fails. At runtime it raises if ever actually hit. It is how a `match` over a fixed set of cases is made provably exhaustive.

Asymmetric cryptography · Ch 40 — **Asymmetric cryptography** (also "public-key cryptography") uses a pair of mathematically linked keys — one private, one public — instead of a single shared secret. What one key does (e.g. sign), only the other can verify; you cannot derive the private key from the public key.

Attribute · Ch 1 — An attribute is a named piece of data attached to an object (e.g. `account.balance`) or to a class. Reached with the dot operator.

Authority output · Ch 25 — An authority output carries no spendable value; instead its `value` field is reinterpreted as a permissions bitmask (mint and/or melt rights for a token). It is flagged by the high bit of the output's `token_data` byte. Authorities are how a token issuer retains control over a token's supply — see Chapter 7 §7.6.

Autobahn · Ch 36 — **Autobahn** is a Python library implementing the WebSocket protocol (and the higher-level WAMP protocol, unused here). `hathor-core` uses its Twisted binding, `autobahn.twisted.websocket`, whose `WebSocketServerFactory` / `WebSocketServerProtocol` mirror Twisted's own `Factory/Protocol` pair.

Autocomplete · Ch 5 — Autocomplete is the editor feature that suggests valid attributes/methods as you type. Type information makes these suggestions accurate, because the editor knows each variable's type.

B

Back-pressure · Ch 30 — Back-pressure is a flow-control technique where a fast producer is held back so a slower consumer is not overwhelmed. Here the client declares a window (how many unacknowledged events it can hold), and the node refuses to send beyond it until the client acknowledges, keeping memory bounded on both sides.

Base58check · Ch 40 — **base58check** is an encoding for addresses: a version byte, then the payload, then a 4-byte checksum (`sha256(sha256(...))` truncated), all encoded in base58 (digits/letters minus look-alikes like 0/O, I/l). The checksum makes mistyped addresses detectable.

Base image · Ch 15 — A base image is the existing image a Dockerfile starts `FROM` (here an official Python-on-Debian image). You build on top of it rather than from an empty filesystem.

BFS · Ch 32 — **BFS** (breadth-first search) — a graph-traversal strategy that visits a starting node, then all of its immediate neighbors, then their neighbors, and so on, in expanding "rings." Hathor's `BFSTimestampWalk` visits descendants in timestamp order and is the engine behind voiding propagation.

BIP32 · Ch 40 — **BIP32** is the standard defining hierarchical-deterministic key derivation: a function that grows a tree of child keys from one master key, so a single master secret regenerates every key. ("BIP" = Bitcoin Improvement Proposal.)

BIP39 · Ch 40 — **BIP39** is the standard mapping a list of memorable words (the mnemonic) to the binary seed that BIP32 starts from. It is why a wallet backup can be 24 words on paper instead of raw bytes.

BIP9 · Ch 38 — **BIP9** ("Bitcoin Improvement Proposal 9") is the Bitcoin scheme for "versionbits" soft-fork activation: miners signal readiness for a change by setting bits in a block's version field, and the change activates once a threshold of blocks in a retarget window signal support, within a defined timeout. Hathor's feature activation is a close adaptation of it, with the addition of the `MUST_SIGNAL` mandatory-signalling phase.

Block · Ch 6 — A block is a batch of transactions grouped and added to the ledger together. In classic blockchains, blocks are the units that form the chain; in Hathor, blocks coexist with free-standing transactions in a graph (Ch 8).

Blocking · Ch 2, Ch 16 — A blocking call does not return control to your program until it has finished; while it waits, the calling thread can do nothing else. Its opposite is non-blocking (returns immediately, result delivered later). Full treatment in Ch. 2.

Bloom filter · Ch 27 — A **Bloom filter** is a compact, probabilistic membership test: it can answer "definitely not present" with certainty, or "possibly present" (with a small false-positive rate). RocksDB keeps one per data file so a lookup can skip files that certainly do not hold the key. `key_may_exist` exposes this — a fast "no" without touching the data, which is why `transaction_exists` probes it before doing a real read.

Blueprint · Ch 39 — A Blueprint is Hathor's term for a contract template — a Python class defining the contract's state and methods. One Blueprint can back many independent contracts, just as one class can have many object instances.

Boundary · Ch 18 — A boundary is any point where data enters a program from outside (files, network, user input, other systems). Data at the boundary is untyped and untrusted and must be validated before the program relies on it.

Build backend · Ch 13 — A build backend is the component that turns a project's source into installable distributions (a wheel and/or sdist). `pyproject.toml`'s `[build-system]` table names it; Poetry provides its own.

Builder · Ch 0, Ch 3 — A builder is a design pattern: an object whose job is to construct another, complicated object step by step. Hathor's builder constructs the fully-wired node. The pattern is explained in Chapter 3; Hathor's builders in Chapter 24.

Build system · Ch 14 — A build system turns source into artifacts, tracking dependencies and timestamps to rebuild only what changed. Make is one; for compiled languages this incremental rebuild is its main value, mostly unused in Python projects.

Byte · Ch 26 — A byte is eight bits, holding an integer from 0 to 255. All files and network data are sequences of bytes; everything else — text, numbers, objects — is an interpretation layered on top of bytes by agreement.

Bytecode · Ch 11, Ch 39 — Bytecode is a low-level, compact instruction format that a program is compiled into and that a virtual machine executes, rather than running the original source directly. Ethereum stores contract bytecode on-chain; Hathor stores Python source and runs it on the interpreter.

C

Callback · Ch 2 — A callback is a function passed to other code to be called ("called back") when a specific event happens or a result becomes available.

Cast · Ch 23 — `typing.cast(T, x)` tells the type checker "treat `x` as type `T`." It performs no runtime check or conversion; it only changes how the checker reasons about the value — which is why Hathor pairs it with a runtime `verifyObject` check.

Change · Ch 7 — Change is an output a transaction sends back to the payer, equal to the consumed inputs minus the amount paid (minus any fee). It exists because outputs must be spent whole, like receiving change after paying cash with a larger note.

Checkpoint · Ch 6, Ch 10, Ch 29, Ch 32 — **Checkpoint** — a hard-coded `(height, hash)` pair in the node's settings marking a block the network agrees is final. The node refuses any reorg that would rewrite history before a checkpoint, putting a hard floor under otherwise-probabilistic finality. Full treatment in Ch. 10.

Chokepoint · Ch 33 — A chokepoint (or single point of control) is a deliberate design where all instances of some operation are routed through one piece of code, so the logic exists in exactly one place. The trade-off: it can become a bottleneck or a single point of failure, but it guarantees consistency. Here, consistency is worth far more than the negligible overhead.

Classmethod · Ch 4 — `@classmethod` marks a method whose first argument is the class itself (`cls`) rather than an instance. Common for alternative constructors (e.g. `Vertex.from_bytes(...)`).

Closure · Ch 4 — A closure is a function that retains access to variables from the scope in which it was defined, even after that scope has finished executing. The inner function "closes over" those variables.

Code coverage · Ch 20 — Code coverage measures which lines (or branches) of the code are executed by the test suite, reported as a percentage. High coverage doesn't guarantee good tests, but low coverage reliably reveals untested code.

Column family · Ch 27 — A **column family** is a named, independent key-space inside a single RocksDB database — like a separate dictionary that shares the same database file and write-ahead log but keeps its own keys and can be tuned and compacted on its own. Hathor uses five: `tx`, `meta`, `static-meta`, `attr`, `migrations`.

Compaction · Ch 27 — **Compaction** is the LSM-tree's background housekeeping: it merges several sorted on-disk files into fewer, dropping values that have been overwritten or deleted, so reads stay fast and disk space is reclaimed. It runs even when the node is otherwise idle.

Composition · Ch 1 — Composition builds an object by giving it other objects as attributes and delegating work to them — a "has-a" relationship, as opposed to inheritance's "is-a."

Composition root · Ch 24 — A **composition root** is the single place in an application where the

Concurrency · Ch 2 — Concurrency is making progress on multiple tasks over the same period by interleaving them. It is about structure (dealing with many things at once), not necessarily simultaneity.

Configargparse · Ch 19 — `configargparse` is a Python library that subclasses the standard `argparse`, letting each declared option be set from a command-line flag, an environment variable, or a config file, with a defined precedence — a drop-in upgrade for `argparse`-based programs.

Confirmations · Ch 10 — Confirmations are the confirming vertices (especially blocks) accumulated on top of a transaction; more confirmations mean more work an attacker must overcome, so recipients wait for a threshold before trusting a payment.

Confirmed · Ch 8 — A vertex is confirmed by every later vertex that can reach it via parent edges. More confirmations (and accumulated work behind them) make a vertex harder to reverse — the DAG analogue of chain depth.

Conflict · Ch 10 — A conflict is a pair (or set) of individually-valid transactions that cannot all be part of the ledger — classically, two transactions spending the same output (a double-spend). Distinct from an invalid transaction, which breaks a rule on its own.

Console-script entry point · Ch 21 — A console-script entry point is a line in packaging metadata of the form `name = "package.module:function"`. At install time the packaging tool generates an executable called `name` on your `PATH` that imports and calls that function. It is how a `pip install` makes a shell command appear.

Constructor · Ch 1 — A constructor is the routine that creates and sets up a new object. In Python the setup step is `__init__`; technically object creation is handled by `__new__`, which you almost never need to touch.

Container · Ch 15 — A container is a running instance of an image: an isolated process (or process tree) with its own filesystem view, using the image's contents. Many containers can run from one image.

Containerization · Ch 15 — Containerization is packaging an application with everything it needs to run into a container image, so it runs consistently across different machines regardless of their host configuration.

Content-addressed · Ch 29 — Content-addressed means a piece of data is identified by the hash of its own bytes. Change one byte and the hash changes, so a vertex cannot be silently altered without its id changing — which is why the blocks and transactions themselves cannot be quietly corrupted, only the separately-stored metadata about them.

Context manager · Ch 33 — A context manager is the `with ...:` construct. `non_critical_code(self._log)` wraps a block so that an exception inside it is logged and swallowed rather than propagated — the boundary between "fatal if it fails" and "tolerable if it fails."

Continuous integration · Ch 5, Ch 20 — Continuous integration (CI) is an automated system that runs checks (linters, type checker, tests) on every proposed change and blocks merging unless they pass — the gate that makes the toolchain mandatory.

Cooperative · Ch 2 — Cooperative multitasking relies on each task voluntarily yielding control (e.g. at an `await`) so others can run. No task is forcibly interrupted, so a task that never yields blocks all others.

Coroutine · Ch 2 — A coroutine is a function that can suspend its execution (at an `await`) and later resume from exactly that point, preserving its local state. Defined with `async def` in Python.

CPU-bound · Ch 2 — CPU-bound describes work whose speed is limited by raw computation — the processor is the bottleneck. Examples: hashing, encryption, large numeric loops.

CPython · Ch 11 — CPython is the reference implementation of the Python interpreter, written in C. It is what you get from python.org and what runs by default when you type `python`. Alternatives (PyPy, Jython, GraalPy) exist but are not used by `hathor-core`.

Cryptographic hash function · Ch 6 — A cryptographic hash function maps input of any size to a fixed-size output (the hash), with properties — determinism, one-wayness, avalanche, collision-resistance — that make it usable as a tamper-evident fingerprint. Examples: SHA-256, SHA-3.

Curses · Ch 42 — curses is a standard library for building text-based, full-screen terminal user interfaces (menus, live-updating tables) instead of plain scrolling output. `hathor-cli top` uses it to draw a live `top`-style dashboard.

D

DAA · Ch 9, Ch 31, Ch 37 — **DAA** = Difficulty Adjustment Algorithm. The code that decides how hard mining should be right now, so blocks keep arriving at a steady average rate as total network hashing power rises and falls. Full treatment in Ch 32 / see Ch 9.

Daemon · Ch 0, Ch 41 — A daemon is a program that runs continuously in the background rather than finishing and exiting. Web servers and databases are daemons. The word is pronounced "demon" and comes from an old Unix tradition.

DAG · Ch 0, Ch 35 — **DAG** = Directed Acyclic Graph. A graph is a set of items ("nodes") joined by links ("edges"). Directed means each edge points one way. Acyclic means you can never follow the edges in a loop back to where you started. Full treatment in Chapter 8.

Deadlock · Ch 2 — A deadlock is a standstill where two or more threads each wait for a resource the other holds, so none can ever proceed.

Decorator · Ch 4, Ch 42 — A decorator is a callable that takes a function (or class) and returns a replacement, applied with `@name` above a definition. It is the standard Python way to wrap behavior around a function.

Decoupling · Ch 3, Ch 30 — Decoupling means reducing how much two pieces of code need to know about each other. Tightly coupled code shares references and assumptions, so a change to one forces changes to the other. Loosely coupled code communicates through a narrow, stable interface — here, "publish an event" — so each side can change freely.

Deferred · Ch 29, Ch 34, Ch 35 — A Deferred is Twisted's placeholder for a result that is not ready yet — a "future." You attach callbacks (run on success) and errbacks (run on failure); the reactor fires them when the result arrives. Full treatment in Ch. 16.

Dependency hell · Ch 12 — Dependency hell is the situation where multiple projects (or libraries) require conflicting versions of the same package, making it impossible to satisfy them all from a single shared installation.

Dependency injection · Ch 3 — Dependency injection is the practice of passing an object its collaborators from outside (e.g. via its constructor) rather than having it create or fetch them itself. It makes dependencies explicit and substitutable (e.g. for testing).

Dependency resolution · Ch 13 — Dependency resolution is computing a single set of package versions that simultaneously satisfies every direct and transitive constraint. It can be computationally hard and is the core job of a dependency manager.

Derived state · Ch 28 — Derived state (also secondary state) is data computed from some primary source of truth, holding nothing the primary does not already imply. Its defining property is that it can be discarded and recomputed. Hathor's indexes are derived from the vertices in storage; storage is primary. This is what makes index corruption recoverable by rebuild rather than fatal.

Descriptor · Ch 39 — A descriptor is a Python object that customizes what happens when an attribute is read, written, or deleted, by defining `__get__` / `__set__` / `__delete__`. Properties are descriptors. Hathor's `Field` is a descriptor that routes `self.count` to the trie instead of to memory.

Design pattern · Ch 3 — A design pattern is a named, reusable solution to a commonly occurring problem in software design — a description of a shape to apply, not a finished piece of code.

Deterministic · Ch 39 — Deterministic execution means the same inputs always produce exactly the same outputs on every machine. For consensus this is mandatory: if two nodes computed different results for the same contract call, they would disagree on the ledger. It rules out wall-clock time, true randomness, and reading external data.

Digest · Ch 6 — A digest (or hash) is the fixed-size output of a hash function — here, 256 bits, usually written as 64 hexadecimal characters. It acts as a compact fingerprint of the input.

Digital signature · Ch 7, Ch 40 — A digital signature is data produced with a private key that anyone can verify with the matching public key, proving the signer authorized a specific message without revealing the private key. It is how a locking script confirms a spender's authority. (Full treatment with the crypto libraries in Ch 40.)

Directed graph · Ch 8 — A directed graph has edges with a direction — each edge goes from one node to another, not symmetrically. An edge $A \rightarrow B$ does not imply $B \rightarrow A$.

Dispatch · Ch 4, Ch 26 — Dispatch means selecting which piece of code to run based on a value at runtime — here, choosing a vertex class based on the version byte. Full treatment of dispatch as a pattern is in Chapter 4.

Dispatch table · Ch 4 — A dispatch table is a data structure (usually a dict) mapping keys — message types, command names, event names — to the functions that handle them, replacing a long `if/elif` chain.

Distributed ledger · Ch 6 — A distributed ledger is a transaction record replicated across many independent computers, each holding a full copy, with no central master. Agreement among the copies replaces trust in a single operator.

Docker · Ch 15 — Docker is a platform for building and running containers — isolated, portable packages of an application together with its entire runtime environment (libraries, interpreter, OS userland).

Dockerfile · Ch 15 — A Dockerfile is a text file of instructions (`FROM`, `RUN`, `COPY`, `ENTRYPOINT`, ...) describing how to build an image. Each instruction produces a cached layer.

Domain-specific language · Ch 43 — A domain-specific language (DSL) is a small, purpose-built notation for one narrow job — here, describing a DAG of blocks and transactions — as opposed to a general-purpose language like Python. The `dag_builder` DSL is parsed by a hand-written tokenizer; it is not Python, though a few attribute values are parsed as Python expressions.

Double-spend · Ch 6, Ch 32 — **Double-spend** — two different transactions that both try to spend the same output. Each is valid alone; together they conflict, and consensus must void at least one so a coin is never spent twice. The visible "collision" is exactly what the spent-once rule and `voided_by` machinery exist to catch.

Duck typing · Ch 1 — Duck typing is Python's style of polymorphism: an object's suitability is determined by whether it has the needed methods/attributes, not by what class it inherits from. "If it quacks, it's a duck."

Dunder · Ch 1 — Dunder = "double underscore." Dunder methods like `__init__`, `__repr__`, `__eq__`, `__len__` let your objects integrate with Python's built-in syntax and functions. Also called magic or special methods.

Dynamic dispatch · Ch 4 — Dynamic dispatch selects the implementation at run time based on an object's actual type — the mechanism behind method calls and polymorphism. Contrast static dispatch, decided at compile time by the declared type.

Dynamic typing · Ch 5 — Dynamic typing means variable types are not fixed and are checked while the program runs. A name can be bound to a value of any type, and type errors surface at execution time. Python, Ruby, and JavaScript are dynamically typed.

E

ECDSA · Ch 40 — **ECDSA** = Elliptic Curve Digital Signature Algorithm. The specific signature scheme Hathor (and Bitcoin) use. "Elliptic curve" is the kind of one-way maths that makes private→public easy and the reverse infeasible. You can read this chapter without the maths.

Encapsulation · Ch 1 — Encapsulation = bundling data with the methods that operate on it, and restricting outside access to that data so it can only change through approved methods.

Endianness · Ch 26 — Endianness is the order in which the bytes of a multi-byte number are laid down. Big-endian puts the most-significant byte first (the way we write decimal); little-endian puts the least-significant byte first. Neither is more correct; writer and reader need only agree. Hathor uses big-endian everywhere.

Environment variable · Ch 19, Ch 22 — An environment variable is a named value the operating system makes available to a running process (read in Python via `os.environ`). It is a process-global channel: any part of the program can read it without being passed it explicitly, which is why it suits a setting that must be reachable from hundreds of call sites.

Event loop · Ch 0 — An event loop is a program structure that repeatedly waits for "something to happen" and then dispatches a handler for it, instead of running top-to-bottom and blocking. It is the core idea behind asynchronous programming — Chapter 2.

Event queue · Ch 30 — The event queue is Hathor's name for the durable, numbered log of events on disk. "Queue" here means an ordered sequence consumers read in order, not a transient in-memory buffer. It is the persistent half of the event system, off by default and enabled per node.

EVM · Ch 39 — The EVM (Ethereum Virtual Machine) is Ethereum's purpose-built virtual machine: a stack-based interpreter with its own instruction set that every Ethereum node runs to execute contract bytecode. Hathor has no equivalent separate machine — contracts run on the node's own (sandboxed) Python interpreter.

Exit code · Ch 21 — An exit code (or status code) is the small integer a process returns to the shell when it ends. `0` means success; any non-zero value signals an error. Shells and deployment tools branch on it (`if program; then ...`). Hathor uses `0` for success, `1` for a `Ctrl-C` abort, `2` for an uncaught crash.

F

Facade · Ch 3 — A facade is an object that provides a simplified, high-level interface to a larger, more complex subsystem, hiding its coordination behind a few easy methods.

Factory · Ch 3, Ch 34, Ch 35, Ch 37 — In Twisted, a factory is the object that creates a fresh protocol instance for each incoming connection and holds state shared across all of them. The `StratumFactory` builds one `StratumProtocol` per miner and holds the list of all miners. Full treatment of factories in Chapter 16 (and the pattern in Chapter 3).

Fail-fast · Ch 18 — Fail-fast is the principle of detecting and reporting an error as early and as close to its source as possible — here, validating input at the boundary so a bad value raises immediately with a clear message rather than causing an obscure failure later.

Failure · Ch 16 — A Twisted Failure is an object that captures an exception together with its traceback, so an error can be carried along a Deferred's errback chain (and inspected later) rather than being raised and lost. Think of it as "an exception, boxed for asynchronous travel."

Fallback · Ch 4 — A fallback is an alternative path taken when the preferred one is unavailable or fails — a default value, a backup service, or a simpler implementation. It makes code degrade gracefully instead of breaking.

Feature · Ch 33 — A feature here is a protocol upgrade gated by feature activation — a mechanism where rule changes switch on according to a schedule and miner signalling. `VerificationParams` carries which features are active so verification applies the right rules. Ch. 38.

Feature activation · Ch 22 — Feature activation is Hathor's mechanism for switching protocol upgrades on over a schedule, by miner signalling. The schedule for a network is part of its settings profile (the `FEATURE_ACTIVATION` field). Full treatment in Ch. 38.

Finality · Ch 6, Ch 10 — Finality is the assurance that a transaction cannot be reversed. In proof-of-work systems it is probabilistic: never absolute, but exponentially more certain as more blocks are built on top (greater depth).

First-class · Ch 2 — A value is first-class if it can be stored in variables, passed as an argument, and returned from functions. In Python, functions are first-class values.

Flake8 · Ch 20 — flake8 is a Python linter that combines several checkers (pyflakes for likely bugs, pycodestyle for style) to report problems by static inspection. Configurable line length and per-file ignores.

Fluent interface · Ch 3 — A fluent interface is an API style where methods return the object itself so calls can be chained (`obj.a().b().c()`), reading almost like a sentence.

Fork · Ch 6, Ch 38 — A fork (specifically a consensus fork or chain split) is when nodes stop agreeing on which blocks are valid, so the single shared ledger splits into two incompatible histories. A coin's state can differ between the two sides. Feature activation exists to prevent forks caused by rule changes; this is distinct from a temporary fork, the brief, self-healing disagreement when two miners find a block at nearly the same time (resolved by consensus, Ch. 32).

Formatter · Ch 20 — A formatter automatically rewrites code into a canonical style (spacing, import order) so formatting is consistent and never argued over. `isort` formats imports; it changes files rather than only reporting.

Frame · Ch 36 — A **frame** is the unit of data in the WebSocket protocol: a small message with a tiny header indicating its type (text, binary, ping, pong, close) and length. After the handshake, the two sides exchange frames instead of HTTP requests/responses.

Fuel · Ch 39 — Fuel is Hathor's name for the metered execution budget — the equivalent of Ethereum's gas. The design charges fuel per executed bytecode operation and aborts when it reaches zero; this branch threads the budget through but does not yet decrement it (see §39.A.7).

Full-duplex · Ch 36 — **Full-duplex** describes a channel on which both ends can transmit at the same time and either can start at any moment (like a phone call). **Half-duplex** allows only one direction at a time (like a walkie-talkie). Plain HTTP is effectively half-duplex and always client-initiated; a WebSocket is full-duplex.

Future · Ch 2 — A future (a.k.a. promise) is an object representing a result that will be available later. You attach callbacks to it or `await` it. Twisted's version is the Deferred; JavaScript's is the Promise.

G

"Gang of Four" · Ch 3 — The "Gang of Four" (GoF) are the four authors of Design Patterns: Elements of Reusable Object-Oriented Software (1994), the book that catalogued and named 23 classic patterns and made the vocabulary standard.

Gas · Ch 39 — Gas is Ethereum's unit of computational cost: each operation costs a fixed amount of gas, the transaction carries a gas budget, and execution aborts if the budget runs out. It is the mechanism that makes unbounded loops harmless. Hathor's equivalent budget is called fuel.

Generic · Ch 5, Ch 25 — A generic class is parameterized by a type variable, written `Class[T]`. `GenericVertex[StaticMetadataT]` is generic over its static-metadata type, so `Block = GenericVertex[BlockStaticMetadata]` and `Transaction = GenericVertex[TransactionStaticMetadata]` stay distinct to the type-checker. Full treatment in Chapter 5.

Genesis · Ch 0, Ch 22, Ch 29, Ch 35 — The genesis is the hard-coded first block and initial transactions that every node on a network agrees on by definition — the shared root of history. Different networks have different genesis data; that difference is part of what makes them distinct networks. Full treatment in Ch. 25 & 32.

Geometric distribution · Ch 43 — The geometric distribution models the number of independent yes/no trials until the first success, when each trial succeeds with probability p . Mining is exactly that — repeated nonce attempts, each a long shot — so drawing one geometric sample reproduces "how long mining would have taken" without running the trials.

Global Interpreter Lock · Ch 2, Ch 16 — The Global Interpreter Lock is a mutex in standard CPython that allows only one thread to execute Python bytecode at a time. It simplifies the interpreter but prevents threads from running Python computation in parallel — a reason the ecosystem leans on the single-threaded event-loop model. Full treatment in Ch. 2.

Global state · Ch 3 — Global state is data accessible from anywhere in the program. It makes code harder to reason about and test, because any function might read or change it invisibly.

Gradual typing · Ch 5 — Gradual typing allows mixing typed and untyped code in the same program, so annotations can be added incrementally rather than all at once. Python's type system is gradual.

Graph · Ch 8 — A graph is a set of nodes connected by edges. It is an abstract structure for representing relationships; many systems (networks, maps, dependencies, ledgers) are naturally graphs.

H

Halting problem · Ch 39 — The halting problem is the proven impossibility of writing a general algorithm that decides, for any program and input, whether it will eventually stop or run forever. Its consequence here: the node cannot prove a contract terminates, so it must instead bound execution with a finite budget.

Hash160 · Ch 40 — **hash160** is SHA-256 followed by RIPEMD-160, producing a 20-byte fingerprint of a public key. Hashing the public key shortens the address and hides the public key until the output is spent.

Hash function · Ch 40 — A **hash function** maps a message of any size to a fixed-size fingerprint, such that any change to the message changes the fingerprint unpredictably and you cannot run it backwards. Hathor uses SHA-256 and RIPEMD-160. Signatures are computed over a hash of the message, not the raw message.

Hash pointer · Ch 6 — A hash pointer is a reference to a piece of data that includes that data's hash, so the reference both locates the data and certifies it is unchanged. Chaining blocks by hash pointer is what makes the history tamper-evident.

Header · Ch 39 — A header here is a self-describing block of extra bytes appended to a transaction's serialized form, carrying optional structured data. The nano-header carries the instructions for a contract call; a separate fee header carries fee data.

Heap · Ch 43 — A heap is a tree-shaped data structure kept partly ordered so that the smallest (or largest) item is always at the front and cheap to remove. Python's `heapq` gives this over a plain list. The simulator uses a min-heap keyed by fire-time so "the next timer to fire" is always at hand.

Higher-order function · Ch 2 — A higher-order function is one that takes other functions as arguments and/or returns a function. `map` , `sorted(key=...)` , and decorators are examples.

I

Idempotent · Ch 33 — Idempotent means running an operation more than once has the same effect as running it once. The handler's gate is idempotent in spirit: re-ingesting an already-accepted vertex is detected as "already-known" and does nothing, so a vertex can safely arrive at the gate repeatedly.

Image · Ch 15 — An image is a read-only, layered package of a filesystem (OS userland, libraries, app) — the blueprint from which containers are created. Built from a Dockerfile, stored in registries.

Immutable · Ch 1 — Immutable data cannot be changed after creation; you make a modified copy instead. Python strings and tuples are immutable; lists and dicts are mutable (changeable in place).

Index · Ch 28 — An **index**, in databases, is a derived (secondary) data structure that precomputes the answer to a class of lookup queries, keyed the way the query is phrased, so the answer can be found without scanning all the primary data. It costs extra storage and extra work on every write, in exchange for fast reads. The index at the back of a book — topic → page numbers — is the same idea on paper. Because it is computed from the primary data, it can always be rebuilt and contains no information that is not already implied by that data.

Inheritance · Ch 1 — Inheritance lets a subclass acquire the attributes and methods of a superclass, then add or override behavior. Models an "is-a" relationship.

Inlinecallbacks · Ch 35 — `@inlineCallbacks` is a Twisted decorator that lets you write asynchronous code as if it were straight-line: `yield some_deferred` pauses the function until the result arrives, instead of nesting callbacks. Ch. 16.

Input · Ch 7 — An input of a transaction is a reference to a specific earlier output that this transaction consumes (spends). In Hathor it is the `(tx_id, index)` pair plus an unlocking script. An input does not hold value itself; it points at the output that does.

Instance · Ch 1 — An instance is one concrete object built from a class. "Instantiate" means "create an instance." `Account("Alice")` instantiates the `Account` class.

Interface · Ch 23 — An interface is a named list of methods an object promises to provide — a contract. It says nothing about implementation, only about the shape of the object.

Interfaces · Ch 16, Ch 23 — `zope.interface` is a library that lets Python code declare interfaces — named sets of methods a class promises to provide — and verify at runtime that an object implements one. Twisted uses it to describe the reactor's capabilities (`IReactorCore`, `IReactorTime`, `IReactorTCP`). It is an older, runtime-checked cousin of `typing.Protocol`.

Intersection type · Ch 23 — An intersection type describes a value that satisfies several contracts at once. Python has no dedicated syntax for it, so the idiom is to define a type that inherits from each contract, as `ReactorProtocol` does with its three slice-protocols.

Invariant · Ch 1 — An invariant is a condition that must always hold true for an object to be valid — e.g. "an account's balance is never negative." Much of good design is about identifying invariants and ensuring no code can violate them.

I/O-bound · Ch 2, Ch 16 — I/O-bound describes work whose speed is limited by input/output — network, disk, waiting on other systems — rather than by the processor. A node is overwhelmingly I/O-bound. Full treatment in Ch. 2.

Isort · Ch 20 — isort is a tool that automatically sorts and groups Python import statements into a consistent order (standard library, third-party, first-party). Run in "fix" mode it edits files; in "check" mode it only verifies.

J

JSON · Ch 17 — JSON (JavaScript Object Notation) is a text format for structured data as key-value objects and arrays. Emitting logs as one JSON object per line makes them directly ingestible by log-analysis tools.

JSON-RPC 2.0 · Ch 37 — JSON-RPC 2.0 is a tiny standard for remote procedure calls encoded as JSON: each message is an object with a `method`, `params`, and an `id` (for requests), or a `result` / `error` (for responses). Stratum is JSON-RPC sent one message per line over a raw TCP connection.

K

Kernel · Ch 15 — The kernel is the core of an operating system, managing hardware, processes, and memory. Containers share the host's kernel (unlike VMs, which each run their own), which is why they are lightweight.

Keypair · Ch 40 — A **keypair** is the matched (private, public) pair generated together. The two are useless apart: the private key signs, the public key verifies, and only a matched pair agrees.

Keyword · Ch 4 — `**kwargs` collects any extra keyword arguments into a dict (`kw` = keyword). In a call, `**d` unpacks a dict into keyword arguments. Together `*args`, `**kwargs` forward an arbitrary argument list unchanged.

L

Layer · Ch 15 — A layer is the filesystem change produced by one Dockerfile instruction. Layers are cached and reused, so rebuilds skip unchanged steps; images are stacks of layers.

Lazy loading · Ch 3 — Lazy loading (lazy initialization) means deferring the creation of an expensive object until the first moment it is actually needed, rather than up front.

LEB128 · Ch 26 — LEB128 ("Little Endian Base 128") is the standard varint scheme Hatos's framework uses: the number is split into 7-bit groups, each stored in the low 7 bits of a byte, with the high bit of each byte acting as a "more bytes follow" flag. The reader stops at the first byte whose high bit is 0. It is the same scheme used by WebAssembly and the DWARF debug format.

Ledger · Ch 0 — A ledger is just a record of transactions and balances — historically an accountant's book. A distributed ledger is one copy of which is held and maintained by many computers at once, with no single authoritative master copy.

Linereceiver · Ch 41 — A `LineReceiver` is a Twisted protocol that buffers raw incoming bytes and calls your `lineReceived(line)` method once per complete line (split on a delimiter, here `\n`). It saves you from reassembling lines out of arbitrary network chunks. Twisted protocols are covered in Chapter 16.

Linter · Ch 5, Ch 20 — A linter is a tool that flags suspicious or non-conforming code (style violations, likely bugs) by static analysis. `flake8` is the linter in `hathor-core`'s toolchain; a type checker is a stricter cousin.

Lock · Ch 2 — A lock (mutex) is a mechanism ensuring only one thread enters a critical section of code at a time, used to prevent race conditions. Misused locks cause deadlocks and slowdowns.

Locking script · Ch 7 — A locking script (Bitcoin: `scriptPubKey`) is the spending condition attached to an output — a small program that must be satisfied to spend it, typically "provide a valid signature for this public key." It encodes ownership without naming an owner.

Logarithm · Ch 9 — A logarithm `logb(x)` is the exponent to which the base `b` must be raised to obtain `x` (e.g. `log2(1024) = 10`). Logarithms compress large numbers and convert multiplication into addition. Hathor's weight is a base-2 logarithm of work.

Logging · Ch 17 — Logging is the practice of emitting timestamped records of a program's activity to a destination (console, file, service), typically with severity levels and configurable routing, so behavior can be understood after the fact.

Log level · Ch 17 — A log level is a severity tag on a log entry — commonly DEBUG, INFO, WARNING, ERROR, CRITICAL — used to filter which entries are recorded or shown, so routine detail can be separated from serious problems.

Loopingcall · Ch 34, Ch 42 — A `LoopingCall` is Twisted's periodic timer: you give it a function and an interval, and it calls that function every interval seconds on the reactor thread. Because it shares the reactor, the function must be quick — a slow one blocks the node.

LSM-tree · Ch 27 — An **LSM-tree** (log-structured merge-tree) is a storage design that never overwrites data in place. Writes are appended to an in-memory buffer (plus an on-disk log for safety); when the buffer fills it is flushed as a new immutable sorted file; background compaction later merges those files and discards superseded values. It makes writes fast and crash-safe at the cost of occasional extra read work and background CPU.

M

Make · Ch 14 — Make is a build-automation tool (originally 1976) that runs commands defined in a `Makefile`. Built to compile programs by rebuilding only out-of-date files, it is widely repurposed as a task runner for named commands.

Mempool · Ch 0, Ch 35 — The mempool ("memory pool") is the set of valid transactions a node knows about that have been seen but not yet confirmed/ordered by a block. Chapter 28.

Merkle path · Ch 31, Ch 37, Ch 39 — A Merkle path (or Merkle branch) is the short list of sibling hashes that lets you prove a single item belongs to a set summarized by one root hash, without revealing the whole set. Merged mining uses it to prove the Hathor block's hash was committed inside Bitcoin's coinbase transaction.

Metaclass · Ch 39 — A metaclass is a class whose instances are themselves classes. Defining `class Counter(Blueprint)` runs the metaclass's code, which can inspect and rewrite the class before it exists. Hathor uses this to convert a Blueprint's type annotations into storage descriptors.

Metadata · Ch 0 — Metadata here means data the node computes and stores about a vertex (its height, accumulated weight, voided status, ...) as opposed to the vertex's own fixed contents. It is recomputed and updated as the ledger evolves. Chapter 25.

Metered execution · Ch 0 — Metered execution means the contract runner counts the resources a contract uses (a kind of "gas") and stops it if it exceeds a limit, so a buggy or malicious contract cannot run forever. Chapter 39.

Miner · Ch 6 — A miner is a participant that performs proof-of-work — repeatedly hashing candidate blocks to find a valid nonce — in exchange for a reward (newly minted coins and/or fees) when it succeeds.

Miner signalling · Ch 0 — Miner signalling is a voting mechanism: miners set specific bits in the blocks they produce to indicate support for a proposed upgrade, and the rule change activates once enough support accumulates over a defined window. Chapter 38.

Mnemonic · Ch 40 — A **mnemonic** here is the ordered list of BIP39 words (12-24 of them) that encodes a wallet's seed. Anyone with the words can regenerate the entire key tree, so the words must be kept as secret as a private key.

Model · Ch 18 — A Pydantic model is a class subclassing `BaseModel` whose attributes are declared with type hints. It validates and coerces data on construction and provides serialization to/from dicts and JSON.

Module · Ch 11 — A module is a single `.py` file, importable as a namespace whose attributes are the names defined at its top level. The module's name is the filename without `.py`.

Multi-paradigm · Ch 1 — A multi-paradigm language supports several programming styles (procedural, functional, object-oriented) rather than forcing one. Python, JavaScript, and Scala are multi-paradigm; older languages often were not.

Multi-stage build · Ch 15 — A multi-stage build uses multiple `FROM` stages in one Dockerfile, doing the heavy building in early stages and copying only the needed results into a final, smaller image — leaving build tools behind.

Mypy · Ch 5, Ch 23 — `mypy` is a static type checker for Python: it reads type annotations and flags mismatches before the program runs. `hathor-core` runs it in CI (Ch 20), so untyped objects weaken its coverage.

N

Namedtuple · Ch 37, Ch 41 — A `NamedTuple` is an immutable Python record type: like a tuple, but with named fields (`template.reward` instead of `template[1]`). Once created it cannot be changed, which suits a template — it describes a fixed snapshot of work.

Namespace · Ch 11 — A namespace is a mapping from names to objects — a "where names live" container. Modules, packages, classes, and function locals are all namespaces; the dot operator reaches into them.

Node · Ch 6, Ch 8 — A node is one computer participating in the network, running the protocol software and (for a full node) keeping and verifying a complete copy of the ledger. `hathor-core` is a full-node implementation.

Nominal typing · Ch 5 — Nominal typing decides compatibility by declared name/inheritance — an object fits only if its class explicitly is (or subclasses) the required type. ABCs (§1.7) are nominal.

Nonce · Ch 6, Ch 9, Ch 37 — A nonce ("number used once") is a counter field in a block whose only purpose is to be varied during mining. Changing the nonce changes the block's hash; the miner tries nonce after nonce until the hash falls below the target. A Hathor block's nonce is 16 bytes; a transaction's is 4.

O

Observability · Ch 17 — Observability is the degree to which a system's internal state can be understood from its external outputs (logs, metrics, traces). Structured logs are a pillar of it, especially for systems no one watches live.

Observer · Ch 3 — The observer pattern lets an object (the subject) maintain a list of dependents (observers) and notify them automatically when its state changes, typically by calling a registered callback.

Oldname · Ch 33 — The `_old_` prefix and the docstring's "New method" are historical leftovers from a refactor; in the current code this private worker is the live ingestion path. The name is rot, not a clue — read it as "the core on-new-vertex worker."

Orchestrator · Ch 42 — An orchestrator is a system (Kubernetes is the common one) that runs and supervises containerized services automatically — starting them, restarting failed ones, and routing traffic only to healthy ones. It learns a service's state by calling its health-check endpoints.

Overriding · Ch 1 — Overriding is redefining, in a subclass, a method that already exists in the superclass, so instances of the subclass use the new version. Distinct from overloading (multiple methods with the same name but different parameters), which Python does not do in the classic sense.

P

Package · Ch 11 — A package is a directory of modules (and sub-packages) treated as one importable unit, traditionally marked by an `__init__.py` file. Its dotted import name mirrors its directory path.

Parallelism · Ch 2 — Parallelism is executing multiple tasks at literally the same instant, which requires multiple processors/cores. All parallel programs are concurrent; not all concurrent programs are parallel.

Parents · Ch 8 — Parents are the earlier vertices a vertex attaches to and confirms — the DAG's structural (topology) edges. Stored as a list of vertex hashes. The generalization of a blockchain's single `prev` pointer.

"Parse, don't validate" · Ch 18 — "Parse, don't validate" is a design maxim: instead of repeatedly checking untrusted data throughout the code, parse it once at the boundary into a type that cannot hold invalid data, so downstream code can trust it by construction.

Path · Ch 12 — `PATH` is an environment variable holding an ordered list of directories the shell searches to resolve a typed command name. Activating a virtual environment prepends its `bin/` so the environment's `python / pip` are found first.

Patricia trie · Ch 39 — A Patricia trie is a compressed (radix) trie: chains of single-child nodes are merged into one node holding the whole shared substring, keeping the tree shallow and lookups efficient. Hathor combines this with Merkle hashing for verifiable state.

Peer-to-peer · Ch 0 — Peer-to-peer (P2P) means every participant is an equal "peer" that connects directly to other peers, with no central server in the middle. Contrast a client-server model, where everyone talks through one company's servers.

PEP · Ch 13 — A PEP (Python Enhancement Proposal) is a design document defining a Python standard or feature. PEP 518 and PEP 517 standardized `pyproject.toml` and the build-backend interface, so packaging isn't tied to one tool.

Persistent · Ch 39 — A persistent data structure (in the data-structure sense, unrelated to disk persistence) is one where updates produce a new version while leaving every previous version intact and accessible. Hathor's trie copies a path on each write, so every past state root still names a complete snapshot — which is what makes reorgs cheap to undo.

Phony target · Ch 14 — A phony target (declared with `.PHONY`) is a Make target that does not correspond to a file, so Make runs its recipe every time rather than skipping it based on file timestamps. Essential when using Make as a task runner.

Pip · Ch 12 — pip is Python's standard package installer. It downloads packages (by default from PyPI) and installs them into the active environment's site-packages. Poetry uses the same underlying mechanisms but adds dependency resolution and locking.

Poetry · Ch 13 — Poetry is a Python dependency manager and packaging tool. It declares dependencies in `pyproject.toml`, resolves and pins them in `poetry.lock`, manages a project's virtual environment, and builds distributable packages.

Polling · Ch 36 — **Polling** is a client repeatedly asking the server "is there anything new?" on a timer, to approximate a live feed. Each poll is a full request even when the answer is "no," which makes it wasteful — most polls return nothing.

Polyfill · Ch 4 — A polyfill is code (the term comes from web development) that implements a feature the current environment lacks, so older runtimes behave like newer ones. A kind of shim for missing features.

Polymorphism · Ch 1 — Polymorphism lets a single piece of code operate on objects of different types, each responding to the same method name in its own way.

Precedence · Ch 19 — Precedence is the rule deciding which source wins when the same option is set in more than one place. `configargparse`'s order, highest first, is: command-line flag, environment variable, config file, default.

Preemptive · Ch 2 — Preemptive multitasking lets the operating system suspend a thread at any moment to run another. The opposite is cooperative, where a task keeps running until it voluntarily yields.

Privacy · Ch 1 — Python has no enforced `private`. Convention: one leading underscore (`_x`) means "internal, please don't touch"; two leading underscores (`__x`) trigger name mangling, a stronger but still bypassable form of hiding. The community relies on discipline over locks.

Private key · Ch 40 — A **private key** is a large secret number that you never reveal. Possessing it is what defines ownership; anyone who learns it can spend your coins. In Hathor it is a 256-bit `secp256k1` key, stored encrypted at rest.

Processor · Ch 17 — A processor in `structlog` is a function in the pipeline that receives and transforms a log event dict (adding a timestamp, level, bound context, etc.) before the final renderer turns it into output.

Prometheus · Ch 42 — **Prometheus** is the de-facto-standard open-source monitoring system for infrastructure. It pulls (scrapes) numeric metrics from targets on a fixed interval, stores them as time-series, and lets you query and graph them (commonly via Grafana). Covered in §42.4.

Proof-of-Authority · Ch 22, Ch 31, Ch 32 — **Proof-of-Authority** (PoA) — a consensus model for permissioned networks in which a fixed, configured set of authorized signers (identified by public key) take turns producing blocks, each block carrying a valid signature instead of a proof-of-work nonce. No mining; trust is placed in a known set of authorities.

Proof-of-work · Ch 0, Ch 6, Ch 16 — Proof-of-work (PoW) is a scheme where producing a valid block (or transaction) requires finding a number that makes its hash fall below a target — hard to do, trivial to check. The hashing is CPU-bound, which is why Hathor runs it on a thread pool. Full treatment in Ch. 9.

Protobuf · Ch 26 — Protobuf (Protocol Buffers) is Google's schema-driven binary serialization tool: you write a `.proto` schema and it generates reader/writer code. It is compact and convenient, but its output is not guaranteed to be byte-for-byte canonical across implementations, which is why it is unsuitable for data whose hash must match on every node.

Protocol · Ch 5, Ch 27, Ch 34 — A **Protocol** (Python `typing.Protocol`) is a structural interface: any object that happens to have the right methods satisfies it, with no explicit inheritance required ("if it walks like a duck..."). It lets a function accept any object shaped correctly, not just one named subclass. Full treatment in Chapter 5.

Proxy pattern · Ch 3, Ch 39 — The proxy pattern (Chapter 3) places a stand-in object between a client and a real service so the stand-in can control, restrict, or mediate access. The `BlueprintEnvironment` is a proxy: contract code talks to it, and it forwards each request to the Runner under controlled, checked conditions.

Public key · Ch 40 — A **public key** is derived from the private key and may be shared freely. It is used to verify signatures and to derive your address. Computing it from the private key is easy; the reverse is infeasible.

Pub-sub · Ch 0, Ch 3, Ch 42 — Pub-sub (publish-subscribe) is a messaging pattern: publishers announce events without knowing who listens, and subscribers register interest in event types. The metrics object subscribes to events like "new tx accepted" to update its numbers on the spot. Full treatment in Chapter 30.

Pure function · Ch 1 — A pure function always returns the same output for the same input and has no observable effect beyond that return value. `len(x)` is pure; a function that prints, or edits a global, is not.

Pydantic · Ch 18, Ch 30 — Pydantic is a Python library for declaring data models as classes with typed fields; it validates and coerces data at runtime and can serialize to/from JSON. Full treatment in Ch. 18. Here it gives `BaseEvent` its field types and the validator that keeps each event's data shape matching its type.

Pydantic v2 · Ch 18 — Pydantic v2 is the current major version, with a rewritten core and APIs including `ConfigDict`, `field_validator`, and `model_validator`. It is faster than v1 and is what `hathor-core` uses.

PyPI · Ch 12 — PyPI (the Python Package Index, at pypi.org) is the central public repository of third-party Python packages that pip downloads from by default.

Pyproject · Ch 21 — `pyproject.toml` is the standard configuration file for a modern Python project — dependencies, build settings, and tool config in one place. Hathor uses Poetry to read it. Full treatment in Chapter 13.

Pytest · Ch 20 — pytest is the most widely used Python testing framework. It discovers test functions, runs them, provides rich assertions and fixtures, and reports results. Extensible via plugins like `pytest-xdist` and `pytest-cov`.

Pytest-xdist · Ch 20 — pytest-xdist is a pytest plugin that distributes tests across multiple CPU cores (or machines), running them in parallel to shorten total test time. Enabled here with `-n auto`.

Python Virtual Machine · Ch 11 — The Python Virtual Machine (PVM) is the part of CPython that executes bytecode — a loop that reads and performs bytecode instructions one at a time. "Virtual machine" here means a software CPU, not a whole virtualized computer.

R

Race condition · Ch 2 — A race condition is a bug where the result depends on the unpredictable timing/order in which concurrent tasks run — e.g. two threads incrementing the same counter and losing an update.

Reactor · Ch 0, Ch 34, Ch 35, Ch 42 — The reactor is the heart of the Twisted framework: a single event loop that waits for events (data on a socket, a timer firing) and calls the one piece of your code registered to handle each. The whole node runs on it, on one thread. Full treatment in Chapters 2 and 16.

Reactortype · Ch 23 — Twisted predates Python's modern type system, so its reactor is exported as a plain object satisfying many interfaces rather than as one statically-typed class. Without help, a checker treats most accesses on it as untyped.

Read-only · Ch 22 — A read-only object is one that nothing is supposed to mutate after it is created; configuration is treated this way so no stray code can change a network constant mid-run. (Pydantic models are mutable by default unless explicitly frozen; in Hathor the read-only discipline is enforced partly by convention and partly by the load-time `extra='forbid'` validation that rejects malformed input in the first place.)

Re-entrancy · Ch 39 — Re-entrancy is when a contract, mid-call, calls back into itself (often via another contract) before the first call finished. It is a classic source of exploits because the contract's state may be half-updated. Hathor forbids it by default and a method must explicitly opt in with `allow_reentrancy=True`.

Reorg · Ch 6, Ch 10, Ch 25, Ch 32, Ch 35, Ch 39, Ch 43 — **Reorg** (reorganization) — when a node abandons part of its current best chain of blocks in favor of a competing chain that has accumulated more work (a higher score). Blocks on the abandoned branch become voided; their transactions return to the mempool or are re-confirmed by the new chain. The reorg size is how many blocks deep the switch goes.

REPL · Ch 11 — The REPL (Read-Eval-Print Loop) is the interactive Python prompt: it reads a line, evaluates it, prints the result, and loops. Started by running `python` with no arguments.

Request-response · Ch 36 — **Request-response** is the interaction model of plain HTTP: the client sends a request and the server sends back exactly one response, then the exchange is finished. The server cannot initiate; it can only reply.

RocksDB · Ch 0 — **RocksDB** is an embedded key-value database: a fast on-disk store of `key → value` byte pairs that runs inside the node's own process rather than as a separate server. Full treatment, and the comparison against alternatives like MongoDB, in Chapter 27.

S

Secp256k1 · Ch 40 — **secp256k1** is the name of the specific elliptic curve — a fixed, public set of parameters — that Hathor, Bitcoin, and Ethereum all use. Because it is a shared standard, keys and signatures are interoperable across tools.

Seed · Ch 40 — A **seed** is the master secret (a block of bytes) from which an HD wallet derives all its keys. It is produced from the BIP39 mnemonic (plus an optional passphrase) and fed into BIP32 derivation.

Semantic versioning · Ch 13 — Semantic versioning (semver) is a convention where a version `MAJOR.MINOR.PATCH` signals the kind of change: PATCH = bug fixes, MINOR = backward-compatible features, MAJOR = breaking changes. Constraint operators like `^` rely on this promise.

Sentry · Ch 17 — Sentry is an error-tracking service that aggregates and alerts on exceptions from running software. structlog can forward error events to it via a processor; it is an optional dependency in `hathor-core`.

Sequence number · Ch 39 — A sequence number (seqnum) is a per-caller counter that must strictly increase with each call. It prevents replay: an attacker cannot resubmit a previously signed transaction, because its seqnum is no longer higher than the last one the node accepted.

Server push · Ch 36 — **Server push** means the server sends data to the client without the client having requested that specific data, using a connection kept open for the purpose. It is the inverse of polling.

Server-Sent Events (SSE) · Ch 36 — **Server-Sent Events (SSE)** is a simpler push standard where the server holds an HTTP response open and writes events to it over time. It is one-directional (server→client only), which is why it cannot serve a stream where the client must also subscribe, ack, and adjust flow control.

Shim · Ch 4 — A shim is a small compatibility layer that transparently intercepts calls and bridges a difference (between API versions, platforms, or libraries), letting surrounding code stay unaware of the difference.

Side chain · Ch 32 — **Side chain** — a branch of blocks whose score is lower than the current best chain's. Its head (and the blocks below it that are off the best chain) are voided. A side chain becomes the best chain — triggering a reorg — only if it later overtakes the current best chain's score.

Side effect · Ch 1 — A side effect is any change a function makes to the world outside itself — modifying a global variable, writing a file, printing, mutating an argument. Pure functional code minimizes side effects.

Sighash · Ch 25, Ch 40 — The sighash ("signature hash") is the hash of a transaction's body that a signer actually signs. Each input's `data` (unlocking script) is blanked while computing it, because you cannot sign over the signature you are in the middle of producing. `Transaction` caches it (`transaction.py:93`) because signing many inputs re-serializes the same body repeatedly.

Signal · Ch 41 — A signal is a small asynchronous notification the operating system delivers to a process (e.g. `SIGINT` from Ctrl-C). `SIGUSR1` and `SIGUSR2` are left undefined by the OS for an application to use as it likes. A signal handler can interrupt the program at almost any point, which is why `sysctl` restricts what may run from the `SIGUSR2` path.

Singledispatch · Ch 4 — `functools.singledispatch` is a Python standard-library decorator that turns a function into one whose implementation is chosen by the type of its first argument, with implementations registered via `@fn.register`.

Singleton · Ch 3, Ch 23 — A singleton is a resource of which exactly one instance exists per process. An event loop is inherently one-per-process, which makes the reactor a natural singleton. Full treatment in Ch. 3.

Site-packages · Ch 12 — `site-packages` is the directory where third-party (pip-installed) packages are placed. There is a global one for the system interpreter and a private one inside each virtual environment; which is used depends on which `python` runs.

Slots · Ch 25 — `__slots__` declares the fixed set of attributes instances may have, replacing each object's per-instance dictionary with a compact fixed layout. It saves memory (decisive when millions of vertices are in RAM) and prevents accidental attribute typos. It is a Python optimization, not a domain concept.

Smart contract · Ch 0, Ch 39 — A smart contract is a program stored on a ledger whose code is executed and agreed upon by every node in the network, so it can hold and move tokens according to its own public rules with no trusted operator. Hather's variant is called a nano-contract.

Stack · Ch 31 — A stack is a last-in-first-out collection: you can only add (push) to the top and remove (pop) from the top. Think of a stack of plates. A stack machine evaluates a program using only one such stack as its working memory.

State · Ch 1 — State is the data an object currently holds — its attribute values at a moment in time. An object with state "remembers" things between method calls; that memory is what makes it stateful.

State machine · Ch 34 — A state machine (finite-state machine) is a model in which an object is always in exactly one of a fixed set of states, certain events trigger transitions between states, and each state accepts only a limited set of events. It makes a connection's lifecycle explicit and rejects out-of-phase messages by construction.

State root · Ch 39 — A state root is the Merkle root of a state store — one hash that uniquely fingerprints all of the state. Two parties can confirm their states are identical by comparing this one value. Hathor keeps a per-contract state root and a per-block root over all contracts.

Static · Ch 5 — Static (in "static typing"/"static analysis") means "determined from the source ahead of run time," without executing the program. Its opposite is dynamic — determined while running.

Static analysis · Ch 5 — Static analysis is examining a program's source to learn facts about it (type errors, unused variables, security issues) without running it. Type-checking and linting are forms of static analysis.

Static metadata · Ch 38 — Static metadata is per-vertex data the node computes once and then never changes — as opposed to mutable metadata that gets updated as the ledger evolves (the split is covered in Ch. 25). A block's height and its `feature_activation_bit_counts` are static: once a block is fixed in the chain, both are fixed too, so they can be computed once and cached.

Staticmethod · Ch 4 — `@staticmethod` marks a method that receives neither `self` nor the class — a plain function namespaced inside a class. Use it for helpers that logically belong to the class but need no instance.

Strenum · Ch 38 — A `StrEnum` (Python 3.11+) is an enumeration whose members are `str` values. `Feature.NANO_CONTRACTS == 'NANO_CONTRACTS'` is `True`, and `Feature.NANO_CONTRACTS.value` is the string. This lets the settings file refer to features by plain string names while the code uses the type-safe enum.

Structural typing · Ch 5, Ch 16, Ch 23 — Python's `typing.Protocol` enables structural typing: any object that has the required methods/attributes satisfies the protocol, with no need to inherit from it. It is a static-typing tool, checked by `mypy`, and is unrelated to Twisted's connection-handler `Protocol` class. Full treatment in Ch. 5.

Structured logging · Ch 0, Ch 17, Ch 21 — Structured logging records log entries as machine-readable key-value data (e.g. `event="block accepted" height=42`) rather than as freeform sentences, so the logs can be searched and analyzed programmatically. Hathor uses a library called `structlog`; full treatment in Chapter 17.

Subsystem · Ch 3 — A subsystem is a cohesive group of classes/modules that together provide some capability (e.g. "storage" or "networking"). A facade fronts a subsystem.

Sybil attack · Ch 6 — A Sybil attack is the creation of many fake identities by one actor to gain disproportionate influence in a system that counts participants. Open, permissionless networks must make influence cost a scarce resource to resist it. (Named after a case study of a person with many identities.)

Symbolic link · Ch 12 — A symbolic link (symlink) is a filesystem entry that points to another file or directory — like a shortcut. A virtual environment's `python` is typically a symlink to a real interpreter, so the environment needn't copy all of Python.

Syncing · Ch 0 — Syncing (synchronizing) is the process by which a node that is behind downloads the blocks and transactions it is missing from its peers until its copy of the ledger matches the network's.

Sync-v2 · Ch 0 — Sync-v2 is the current peer-to-peer synchronization protocol. An earlier sync-v1 existed but has been removed; only sync-v2 remains. Chapter 35.

Syntactic sugar · Ch 4 — Syntactic sugar is syntax that makes something easier to write or read but adds no new capability — it could always be expressed the longer way. `@decorator` is sugar for `f = decorator(f)`.

Sysmodules · Ch 11 — `sys.modules` is a dictionary CPython keeps of every module already imported in the current process, keyed by dotted name. Imports check it first, which is why a module's top-level code runs only once.

Syspath · Ch 11 — `sys.path` is the ordered list of directories Python searches to find a module on import. It includes the standard library and the active environment's installed-packages directory; the first match wins.

T

Target · Ch 6, Ch 9 — A target is the numeric threshold a vertex's hash must fall below to be valid proof-of-work. In Hather it is derived from the vertex's weight: roughly $2^{(256 - \text{weight})}$, so higher weight means a smaller target and more work.

Task runner · Ch 14 — A task runner is a tool that defines and runs a project's common named commands (test, lint, build) from one place, so contributors and CI invoke identical commands without memorizing them.

TCP · Ch 36 — **TCP** (Transmission Control Protocol) is the reliable, ordered byte-stream transport that both HTTP and WebSockets run on top of. A WebSocket reuses the very TCP connection that carried the initial HTTP request — the upgrade does not open a new socket.

Test runner · Ch 20 — A test runner discovers and executes a project's automated tests and reports results. `pytest` is the runner here; it finds test functions, runs them, and summarizes passes and failures.

Thread · Ch 2 — A thread is an independent sequence of execution within a process. Multiple threads in one process share the same memory, which makes communication easy and corruption easy.

Thread pool · Ch 2, Ch 16 — A thread pool is a managed set of reusable worker threads. Jobs are submitted to the pool rather than spawning a new thread each time; used to run blocking or CPU-bound work off the main event loop. Full treatment in Ch. 2.

Tip · Ch 28 — A **tip** of the ledger DAG is a vertex that no other vertex confirms yet — a leaf at the growing frontier of the graph. A **mempool tip** is such a tip that is also still unconfirmed by any block. New transactions attach to (confirm) existing tips, so the node must always know what its current tips are. Full treatment of the DAG and tips in Chapter 8.

Tips · Ch 8 — Tips are the vertices at the frontier of the DAG that no other vertex has yet confirmed (named as a parent). New vertices select their parents from among the tips.

Topological order · Ch 8, Ch 35 — A topological order of a directed acyclic graph is a linear ordering of its vertices such that every vertex comes after all the vertices it depends on (points to). Processing a DAG in topological order guarantees you never reach an item before its prerequisites. Full treatment of the DAG in Ch. 8.

Transitive · Ch 13 — A transitive dependency is a dependency of a dependency — a package you don't ask for directly but that something you depend on needs. The full set forms the transitive dependency tree.

Trie · Ch 39 — A trie (or prefix tree) stores keys by spelling them out along a path from the root, so keys with a shared prefix share the top of their path. A radix/Patricia trie is the compressed form that collapses single-child chains into one node.

Twelve-factor · Ch 19 — Twelve-factor refers to a set of widely-cited principles for building deployable web/network software. One factor is that configuration should come from the environment, keeping config separate from code.

Twistederror · Ch 23 — Twisted's own `ReactorAlreadyInstalledError` is raised by `asyncioreactor.install` when a reactor is already in place. Hathor catches it only inside the `asyncio` branch, to replace the bare error with a message explaining the likely cause (an indirect default-reactor install at import time).

Type annotation · Ch 5 — A type annotation (type hint) is syntax attaching an expected type to a variable, parameter, or return value (`x: int`, `def f() -> str`). In Python it is informational by default — read by tools and humans, not enforced by the interpreter.

Type variable · Ch 5 — A type variable (`TypeVar`) is a placeholder standing for "some specific type, the same throughout this signature," used to write generics.

`def first(x: list[T]) -> T` returns the list's element type.

U

Unlocking script · Ch 7 — An unlocking script (Bitcoin: `scriptSig`) is the data an input supplies to satisfy the locking script of the output it spends — usually a signature and public key. In Hathor it is the input's `data` field.

UTXO model · Ch 7 — The UTXO model (Unspent Transaction Output) records ownership as a set of discrete outputs created by past transactions and not yet spent. A balance is the sum of one's unspent outputs. Used by Bitcoin and Hathor.

V

Validator · Ch 22 — A validator (in Pydantic) is a method that runs while the model is being built, to coerce raw input into the declared type (`mode='before'`) or to check relationships between fields once they are all set (`mode='after'`). It is how a human-friendly YAML shape becomes a strongly-typed, internally-consistent object — or is rejected.

Varint · Ch 26 — A varint (variable-length integer) is an integer encoding that uses fewer bytes for smaller numbers — one byte for small values, more only as needed — instead of a fixed width. It saves space when most values are small.

Verifyobject · Ch 23 — `zope.interface.verify.verifyObject(Interface, obj)` checks at runtime that `obj` actually provides every method the interface declares, raising if not. Hathor uses it to confirm the real reactor matches `ReactorProtocol` before casting to that type.

Vertex · Ch 0, Ch 8, Ch 33 — A vertex in `hathor-core` is any node of the ledger DAG — a block or a transaction. The umbrella term used when the kind does not matter; implemented as `GenericVertex` (aliased `BaseTransaction`).

Vertexid · Ch 25 — `VertexId` is a type alias for `bytes` (`hathor/types.py:26`) — specifically the 32-byte double-SHA256 hash that identifies a vertex. Storing parents and inputs as `VertexId` rather than as object references keeps a vertex small and lets the storage layer load the graph lazily.

Virtual environment · Ch 12 — A virtual environment is an isolated, self-contained Python setup for one project — a directory with its own interpreter link and its own package directory — so each project's dependencies stay separate from other projects' and from the system Python.

Virtual machine · Ch 15 — A virtual machine emulates a complete computer, running a full guest operating system on a hypervisor. Stronger isolation than a container but far heavier, because each VM ships and boots an entire OS.

Voided · Ch 33, Ch 35 — A vertex is voided when consensus marks it as not part of canonical history (e.g. it was on a chain that lost a reorg, or it conflicts with a heavier transaction). Voided does not mean deleted — it means "recorded but not counted." Ch. 10 & 32.

Voiding · Ch 10 — Voiding marks a vertex as not counted toward the real ledger (its outputs unspendable, its effects undone) without deleting it, via the `voided_by` metadata field. Reversible if consensus later changes.

W

WebSocket · Ch 30 — A WebSocket is a persistent, two-way connection between a client and a server over a single long-lived TCP link, unlike HTTP's request-then-close model. It lets the node push events to a client the moment they happen, rather than the client polling. Full treatment in Ch. 36.

Weight · Ch 0, Ch 22, Ch 32, Ch 33 — **Weight** — a number measuring how much proof-of-work a vertex represents ($\text{weight} = \log_2$ of the expected number of hash attempts). **Accumulated weight** sums a vertex's weight with all the work piled up behind it in the DAG. Consensus prefers the history with the most accumulated weight. Full treatment in Ch. 9.

Wheel · Ch 13 — A wheel (`.whl`) is Python's standard pre-built package format — an archive that installs without a compile/build step, making installation fast and deterministic. An sdist is the source-archive counterpart.

Wrapper · Ch 3 — A wrapper is any object that encloses another and forwards calls to it, usually adding or altering behavior. Adapters, proxies, and decorators are all kinds of wrapper (decorators are Chapter 4).

X

Xpub · Ch 36 — An **xpub** (extended public key) is a single public key from which a whole sequence of wallet addresses can be derived without exposing any private key. The history streamer can walk an xpub's derived addresses to stream a wallet's entire history.

Appendix B — The Annotated Dependency Manifest

Chapter 13 explained how Poetry declares dependencies — the split between the flexible ranges in `pyproject.toml` and the exact pins in `poetry.lock`, the `~` and `^` operators, the runtime-vs-development groups. This appendix is the payoff: a line-by-line annotation of the actual manifest, so that every package name you have seen flash past in an import or a stack trace has a one-paragraph answer to "what is this, and why is it here?"

Every entry below is read directly from `pyproject.toml` at the repository root. The **version** column is the constraint as declared (not the resolved pin — that lives in `poetry.lock`). The **chapter** column points to where the book treats the package in depth; a dash means the package is a supporting actor with no dedicated treatment. A reminder of the constraint syntax from Chapter 13: `~X.Y.Z` accepts patch updates within `X.Y`; `^X.Y` accepts any release below the next major; `=X` pins exactly; `{git = ...}` pulls from a repository rather than PyPI.

Why pin versions at all? A node must behave identically across every operator's machine, or consensus fractures (Chapter 22). Loose, unpinned dependencies would let two nodes resolve different library versions and, in the worst case, disagree on a validation result. The manifest's discipline — declare ranges, lock exact versions, commit the lock — is part of how the network stays in agreement. → full treatment in **Chapter 13**.

B.1 The Python platform

Package	Version	What it is / why Hathor uses it	Chapter
python	<code>>=3.11,<4</code>	The interpreter constraint itself (<code>pyproject.toml: 56</code>). Poetry refuses to install into an incompatible interpreter, turning "needs Python 3.11+" from a README note into a machine-checked guarantee. The <code><4</code> upper bound guards against an unknown future Python 4 breaking the world.	11

B.2 Runtime core frameworks

The packages the node is built on — the ones whose abstractions shape the architecture.

Package	Version	What it is / why Hathor uses it	Chapter
<code>twisted</code>	<code>~24.7.0</code>	The asynchronous networking engine (:57). Its reactor is the single event loop the entire node runs on; its Deferred , Protocol , and Factory abstractions underpin every connection. The load-bearing dependency of the whole codebase.	16
<code>autobahn</code>	<code>~24.4.2</code>	A WebSocket implementation that runs on Twisted (:58). Provides the <code>WebSocketServerFactory / Protocol</code> classes behind both the admin streaming surface and the event-queue stream.	36
<code>zope-interface</code>	<code>=8.2</code>	A formal interface system (:87) that Twisted relies on to declare contracts (e.g. the reactor interfaces). Pinned exactly because interface changes ripple widely. The reactor abstraction intersects three of its interfaces into one typed <code>ReactorProtocol</code> .	16, 23
<code>pydantic</code>	<code>^2.0</code>	Run-time data validation using type hints (:80). Validates everything crossing the node's boundary — settings, CLI args, API bodies — turning untyped external input into trusted typed objects. The run-time complement to mypy's static checks.	18
<code>typing-extensions</code>	<code>~4.12.2</code>	Back-ports of newer <code>typing</code> features to the supported Python versions (:82). Lets the code use modern type-system constructs uniformly.	5

B.3 Storage & serialization

Package	Version	What it is / why Hathon uses it	Chapter
rocksdb	{git = ...}	The embedded key-value store that persists the entire ledger (:74). Pulled from Hathon's own fork of python-rocksdb (github.com/hathornetwork/python-rocksdb) rather than PyPI, because the node needs a specific build of the C++ bindings. Chosen over a database server (MongoDB/SQL) precisely because it is embedded, ordered, and accessed by key.	27
cffi	=1.17.1	The C Foreign Function Interface (:86) — the bridge that lets Python call the compiled C/C++ code inside RocksDB and the crypto libraries. The mechanism behind "push the slow parts into C" from Chapter 11.	11, 27
pyyaml	^6.0.1	A YAML parser (:81). Reads the network settings profiles (mainnet / testnet) that define which network a node joins.	22
sortedcontainers	~2.4.0	Pure-Python sorted list/dict/set types (:72). Used where the node needs collections kept in order efficiently — e.g. timestamp-ordered structures in indexes and the mempool.	28
base58	~2.1.1	Base58 encoding (:59) — the alphabet (no 0, 0, I, l) used to render addresses as human-safe strings. The final step of the address pipeline.	40

B.4 Networking & APIs

Package	Version	What it is / why Hathon uses it	Chapter
aiohttp	~3.10.3	An async HTTP client (:75). Used for outbound HTTP — notably peer discovery (fetching bootstrap peer lists).	34

Package	Version	What it is / why Hathor uses it	Chapter
<code>requests</code>	<code>=2.32.3</code>	The classic synchronous HTTP client (:69), pinned exactly. Used by tooling and tests for straightforward request/response calls where the async machinery is unnecessary.	—
<code>idna</code>	<code>~3.4</code>	Internationalized Domain Names handling (:76) — correct encoding of non-ASCII hostnames, needed for TLS and DNS-based peer discovery.	34
<code>service_identity</code>	<code>~21.1.0</code>	Verifies that a TLS certificate actually matches the host you connected to (:70). Closes a class of man-in-the-middle attacks on encrypted peer connections.	34
<code>multidict</code>	<code>=6.7.0</code>	A dictionary allowing repeated keys (:84), pinned exactly. A dependency of the HTTP stack (headers can repeat); used where multi-valued mappings are needed.	—

B.5 Cryptography

The "never roll your own crypto" stack — standard, audited libraries wrapped thinly.

Package	Version	What it is / why Hathor uses it	Chapter
<code>cryptography</code>	<code>~42.0.5</code>	The primary cryptography library (:62) — ECDSA signing/verifying on the secp256k1 curve, key generation, hashing. The <code>Wallet</code> signs transactions through it. Wrapped by <code>hathor/crypto/</code> .	40
<code>pyopenssl</code>	<code>=24.2.1</code>	A Python wrapper over OpenSSL (:67), pinned exactly. Provides the TLS layer that encrypts peer-to-peer connections (Twisted uses it for transport security).	34, 40

Package	Version	What it is / why Hathor uses it	Chapter
<code>pycoin</code>	<code>~0.92.20230326</code>	A Bitcoin-style key utility library (:68). Provides BIP32 hierarchical-deterministic key derivation for the <code>HDWallet</code> ; the <code>hathor/pycoin/</code> wrapper registers Hathor's address version bytes so derived addresses are correct. (Note: the HD wallet signs via pycoin's own ECDSA — flagged in Chapter 40.)	40
<code>mnemonic</code>	<code>~0.20</code>	BIP39 mnemonic phrases (:65) — the word lists that turn a random seed into the 12/24 human-writable recovery words and back. Behind the <code>gen_hd_words</code> CLI.	40

B.6 Observability & logging

Package	Version	What it is / why Hathor uses it	Chapter
<code>structlog</code>	<code>~22.3.0</code>	Structured logging (:73) — records log entries as machine-readable key-value events rather than prose, so a long-running node's logs can be queried and aggregated. The same calls render human-readable in dev and JSON in production.	17
<code>prometheus_client</code>	<code>~0.15.0</code>	The Prometheus metrics library (:66). The node maps its counters/gauges (peer counts, tx/block rates, mempool size) through it; the exporter writes them to a <code>.prom</code> file scraped by an external collector.	42
<code>python-healthchecklib</code>	<code>^0.1.0</code>	A small health-check framework (:83). Powers the <code>/health</code> endpoint that tells an orchestrator (e.g. Kubernetes) whether the node is alive and ready.	42

Package	Version	What it is / why Hathor uses it	Chapter
sentry-sdk	^1.5.11 (optional)	Error-tracking service client (:78). When enabled (the <code>sentry</code> extra), unhandled exceptions are forwarded to Sentry for alerting. Optional so the node runs fine without it — an example of the fallback pattern from Chapter 4.	17, 42
structlog-sentry	^1.4.0 (optional)	The glue (:79) that pipes structlog events into Sentry. Part of the same optional <code>sentry</code> extra.	17

B.7 CLI, process & utilities

Package	Version	What it is / why Hathor uses it	Chapter
configargparse	~1.7.1	An <code>argparse</code> superclass (:61) letting one declared option be set from a command-line flag, an environment variable, or a config file. Gives the whole CLI an <code>HATHOR_*</code> env-var interface with one setting.	19
colorama	~0.4.6	Cross-platform coloured terminal output (:60). Makes the CLI's help text and the structured-log console renderer legible (and correct on Windows).	19, 21
ipython	~8.7.0 (+kernel)	An enhanced interactive Python shell (:64). Powers <code>hathor-cli shell</code> — a REPL with the running node's objects loaded, invaluable for live exploration.	11, 21
setproctitle	^1.3.3	Sets the process's name as it appears in <code>ps / top</code> (:77), so operators can identify a running node at a glance.	—
pexpect	~4.8.0	Drives interactive subprocesses by expecting and responding to their output (:71). Used in tooling/tests that automate command-line interactions.	—
graphviz	~0.20.1	Python bindings to the Graphviz graph-drawing tool (:63). Renders the DAG to a visual diagram — a debugging/inspection aid for a graph-shaped ledger.	8

Package	Version	What it is / why Hathor uses it	Chapter
packaging	=26.0	Version-parsing and comparison utilities (: 88), pinned exactly. Used where the code reasons about version strings (e.g. migrations, compatibility checks).	—
hathorlib	{path, develop}	Not third-party — a sibling library developed in-tree (: 85 , develop = true means edits are picked up live). Holds shared settings, genesis data, and transaction utilities used by both the full node and lighter clients. Most settings fields actually live here (Chapter 22).	22

B.8 Development & quality toolchain

These live in the `[tool.poetry.group.dev.dependencies]` group (`pyproject.toml:40`) — needed to develop and test the node, deliberately excluded from the production Docker image via `--only=main` (Chapter 15).

Package	Version	What it is / why Hathor uses it	Chapter
mypy	^1.19.1	The static type checker (: 43) — reads the type hints across the codebase and reports inconsistencies before the code runs. A required merge gate. Restricted to CPython via a marker.	5, 20
mypy-zope	^1.0.14	A mypy plugin (: 44) teaching the checker to understand <code>zope.interface</code> contracts (which Twisted uses) — without it, type-checking the networking code would be impossible.	5, 20
flake8	~7.1.1	The linter (: 41) — flags stylistic problems and likely bugs (unused imports, undefined names, over-long lines) by static inspection.	20
isort	~5.13.2 (+colors)	The import formatter (: 42) — sorts and groups imports into a canonical order, ending all debate over import ordering. Its line length is kept equal to flake8's.	20

Package	Version	What it is / why Hathor uses it	Chapter
<code>pytest</code>	<code>~8.3.2</code>	The test runner (:45) — discovers and executes the test suite, reports pass/fail.	20, 43
<code>pytest-cov</code>	<code>~5.0.0</code>	A pytest plugin (:46) measuring code coverage — which lines the tests actually execute. Some test targets enforce a coverage floor.	20
<code>pytest-xdist</code>	<code>~3.6.1</code>	A pytest plugin (:48) distributing tests across CPU cores (<code>-n auto</code>), keeping a large suite's feedback loop short.	20
<code>flaky</code>	<code>~3.8.1</code>	A pytest plugin (:47) that re-runs tests which fail intermittently, distinguishing genuine failures from timing-flakes — useful for a system with concurrency.	20, 43
<code>yamllint</code>	<code>~1.35.1</code>	A linter for YAML files (:49) — catches malformed settings profiles and config before they reach the node. Part of <code>make check</code> .	20, 22
<code>types-requests</code> , <code>types-pyopenssl</code> , <code>types-pyyaml</code>	pinned	Type-stub packages (:52 - :54) — they add type hints for third-party libraries that ship none, so mypy can check code that calls into <code>requests</code> , <code>pyOpenSSL</code> , and <code>PyYAML</code> .	5, 20

B.9 Build & packaging

Not in the dependency lists proper, but part of the manifest — the machinery that turns the source tree into an installable package.

Item	Value	What it is / why Hathor uses it	Chapter
build backend	<code>poetry.core.masonry.api</code>	The code that packages the project into a wheel/sdist (<code>pyproject.toml: 191</code>). Declared per the PEP 517/518 standard, so the project can be built by tooling other than Poetry. The Docker build runs <code>poetry build -f wheel</code> against exactly this.	13, 15
<code>[tool.poetry.scripts]</code>	<code>hathor-cli = 'hathor_cli.main:main'</code>	The console-script entry (<code>:38</code>) that creates the <code>hathor-cli</code> terminal command, wiring it to the <code>main()</code> function. The book's front door.	21
<code>[tool.poetry.extras]</code>	<code>sentry = [...]</code>	The optional-feature bundle (<code>:90</code>) — installing the <code>sentry</code> extra pulls in <code>sentry-sdk</code> + <code>structlog-sentry</code> together.	17, 42

Recap

Group	Count	The load-bearing ones
Runtime core	5	<code>twisted</code> , <code>pydantic</code>
Storage & serialization	5	<code>rocksdb</code> (custom fork), <code>pyyaml</code>
Networking & APIs	5	<code>aiohttp</code> , <code>service_identity</code>
Cryptography	4	<code>cryptography</code> , <code>pycoin</code>

Group	Count	The load-bearing ones
Observability	5	<code>structlog</code> , <code>prometheus_client</code>
CLI & utilities	8	<code>configargparse</code> , <code>ipython</code> , <code>hathorlib</code>
Dev & quality	12	<code>mypy</code> , <code>pytest</code>
Build & packaging	3	the Poetry backend

Every package in the node's `pyproject.toml` earns its place against the same test: it solves a problem the node would otherwise have to solve itself, badly. Twisted gives it concurrency, RocksDB gives it durable ordered storage, Pydantic and the type stubs give it boundary safety, the crypto libraries give it audited primitives no one should hand-roll, and the dev group gives it the discipline to stay correct as it grows. The manifest is short by the standards of a modern application — a deliberate choice, since every dependency is also a thing that can break, be compromised, or drift. With this annotated, the last appendix collects the quick-reference tables — column-family layouts, message types, command lists — that you reach for once the concepts are already in hand.

-
1. A manifest is the file listing a project's dependencies and metadata. For a Poetry project that is `pyproject.toml` (declared ranges) paired with `poetry.lock` (resolved exact versions). ↩
 2. An extra (optional dependency group) is a named bundle of packages a user can choose to install — here, `sentry`. Code guards their use so the node runs whether or not they are present. ↩
 3. A type stub is a file (or package, named `types-*`) providing type hints for a library that ships none, so a static checker like mypy can verify code that uses that library. ↩

Appendix C — Cheat Sheets

The chapters explain why; these tables are the what, collected for fast lookup. Nothing here is new — every table condenses material from a chapter named in its heading, verified against the source as of the branch on the cover. Use this appendix the way you would a wall chart: when you are reading the code and need the exact column-family name, the precise message string, or the value of a network constant, find it here without re-reading the chapter.

C.1 RocksDB column families

The on-disk store is split into named **column families**¹ — independent key spaces inside one database. Vertex bodies, mutable metadata, and immutable static metadata are kept apart so each can be read, written, and (in principle) tuned independently. All vertex-keyed families use the 32-byte vertex hash as the key. (Chapter 27; `transaction/storage/rocksdb_storage.py:46 – 50`.)

Column family	Constant	Key	Value	Holds
tx	<code>_CF_NAME_TX</code>	vertex hash (32 B)	binary <code>bytes(vertex)</code>	the serialized vertex body (Ch 26)
meta	<code>_CF_NAME_META</code>	vertex hash (32 B)	binary metadata	mutable <code>TransactionMetadata</code> — <code>voided_by</code> , <code>score</code> , ... (Ch 25, 32)
static-meta	<code>_CF_NAME_STATIC_META</code>	vertex hash (32 B)	JSON	immutable <code>VertexStaticMetadata</code> — <code>height</code> , <code>min-height</code> (Ch 25)
attr	<code>_CF_NAME_ATTR</code>	attribute name	flag/value	node flags incl. the crash markers <code>manager_running</code> / <code>full_node_crashed</code> (Ch 29)
migrations	<code>_CF_NAME_MIGRATIONS</code>	migration name	state	which schema migrations have run

Note. There is no in-RAM storage backend; ephemeral runs use a throwaway temporary RocksDB (`create_temp()` , `--temp-data`), not a different class. (Chapter 27.)

C.2 P2P protocol messages

Every peer-to-peer message is one line of text: `COMMAND payload` . The commands are the `ProtocolMessages` enum (Chapter 34; `p2p/messages.py:45`). Grouped by purpose:

Handshake & connection management

Message	Meaning
HELLO	open the handshake; advertise protocol/sync versions
PEER-ID	exchange and verify peer identity
READY	both sides verified; connection is live
ERROR	protocol error
THROTTLE	rate-limit notice
PING / PONG	liveness check / reply
GET-PEERS / PEERS	request / send known peer addresses

Best-chain & generic data

Message	Meaning
GET-BEST-BLOCKCHAIN / BEST-BLOCKCHAIN	query / report the peer's best chain
GET-DATA / DATA	request / send a specific vertex
NOT-FOUND	requested vertex is unknown
RELAY	enable/disable live push of new vertices

Sync-v2 — block backbone (Chapter 35)

Message	Meaning
GET-BEST-BLOCK / BEST-BLOCK	am I behind?
GET-PEER-BLOCK-HASHES / PEER-BLOCK-HASHES	n-ary common-ancestor search

Message	Meaning
GET-NEXT-BLOCKS / GET-PREV-BLOCKS	request a run of blocks
BLOCKS / BLOCKS-END	streamed blocks / stream end
STOP-BLOCK-STREAMING	abort a block stream

Sync-v2 — transactions & mempool (Chapter 35)

Message	Meaning
GET-TRANSACTIONS-BFS	request the txs a block confirms (BFS)
TRANSACTION / TRANSACTIONS-END	streamed tx / stream end
STOP-TRANSACTIONS-STREAMING	abort a tx stream
GET-TIPS / TIPS / TIPS-END	mempool tip discovery
GET-MEMPOOL / MEMPOOL-END	mempool sync

Nano-contract state sync (Chapter 39)

Message	Meaning
GET-BLOCK-NC-ROOT-ID / BLOCK-NC-ROOT-ID	query a block's nano-contract state root
GET-NC-DB-NODE / NC-DB-NODE	fetch a node of the contract-state trie

Rot warning. The enum still contains sync-v1-era messages (GET-NEXT , NEXT) — sync-v1 has been removed and these are not used. (Chapter 34/35.)

C.3 CLI command groups

`hathor-cli <command>` dispatches through a name→module dictionary (Chapter 21; `hathor_cli/main.py`). The ~30 commands by group:

Group	Commands	Purpose
<code>hathor</code>	<code>run_node</code> , <code>gen_peer_id</code> , <code>top</code>	run a node; generate a peer id; CPU-profiler viewer

Group	Commands	Purpose
mining	<code>run_miner</code> , <code>run_stratum_miner</code> , <code>run_merged_mining</code>	external miners (Ch 37)
wallet	<code>gen_kp_wallet</code> , <code>gen_hd_words</code>	create a keypair wallet; generate HD seed words (Ch 40)
multisig	<code>gen_multisig_address</code> , <code>spend_multisig_output</code> , <code>tx_signature</code>	multisig tooling
oracle	<code>oracle-create-key</code> , <code>oracle-get-pubkey</code> , <code>oracle-encode-data</code>	oracle data signing
side-dag	<code>run_node_with_side_dag</code> , <code>gen_poa_keys</code> , <code>gen_genesis</code>	side-DAG / proof-of- authority (Ch 32)
events	<code>reset-event-queue</code>	clear the event queue (Ch 30)
features	<code>reset-feature-settings</code>	clear feature-activation state (Ch 38)
docs	<code>generate_openapi_json</code>	API docs
tests	<code>gen_rand_tx</code> , <code>gen_twin_tx</code>	test-data generators
dev	<code>shell</code> , <code>quick_test</code> , <code>db_export/import</code> , <code>nc_dump</code> , <code>check_blueprint</code> , ... (11 total)	developer/debug tools

C.4 Sysctl parameter paths

Runtime control exposes a tree of dotted-path parameters over a socket (Chapter 41; `hathor/sysctl/`). Address a parameter as `path` (get) or `path=value` (set, JSON value). Representative paths:

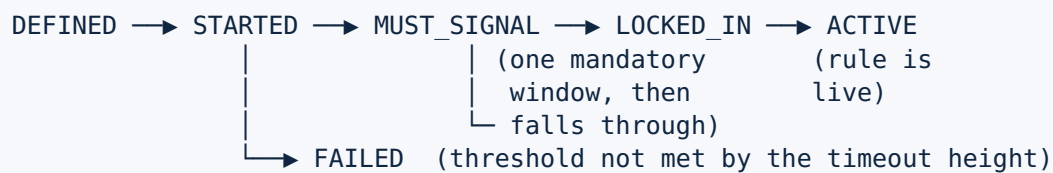
Path	Get / Set	Effect
<code>core</code>	—	the <code>HathorManagerSysctl</code> root for node-wide knobs (debugging, profiling)
<code>core.features</code>	get	feature-activation state
<code>p2p.max_enabled_sync</code>	get / set	cap on simultaneously-syncing peers

Path	Get / Set	Effect
p2p.available_sync_versions	get	sync versions on offer (v2 in practice)
p2p.kill_connection	set	drop a specific peer connection
storage	—	the StorageSysctl subtree
ws	—	websocket controls (if enabled)

Three entry points feed the same command tree: the socket (--sysctl), a batch init file (--sysctl-init-file), and the SIGUSR2 signal (reads a per-pid FIFO; only signal-safe setters allowed). No authentication — access is governed by socket permissions. (Chapter 41.)

C.5 Feature-activation state machine

A feature moves through six states as evaluation windows pass, gated on the fraction of blocks signalling its bit (Chapter 38; `feature_activation/model/feature_state.py`).



State	Meaning
DEFINED	known but not yet started (before its start height)
STARTED	the signalling window is open; blocks may vote for the bit
MUST_SIGNAL	a single window where signalling is mandatory before lock-in
LOCKED_IN	enough support gathered; activation is now scheduled
ACTIVE	the feature's rule is in force
FAILED	the timeout height passed without enough support

Key tuning constants (`feature_activation/settings.py`): `evaluation_interval = 20_160` blocks per window, `default_threshold = 18_144` (90%), `max_signal_bits = 8`.

C.6 P2P handshake states

Each connection is driven by a three-state machine in `HathorProtocol` (Chapter 34; `p2p/states/`). The transition to `READY` is symmetric — it fires only when both peers are satisfied.



State	Class	Handles
HELLO	<code>HelloState</code>	version negotiation, network match, genesis check
PEER_ID	<code>PeerIdState</code>	identity exchange & verification
READY	<code>ReadyState</code>	live connection; attaches the sync agent

C.7 Network constants

The values that define the network — every node must agree on them or it is on a different network (Chapter 22; the genesis/timing constants live in `hathorlib/conf/settings.py`). Mainnet values:

Constant	Value	What it controls
<code>AVG_TIME_BETWEEN_BLOCKS</code>	30 s	target block interval; the DAA tunes weight toward it (Ch 9)
<code>MIN_BLOCK_WEIGHT</code>	21	floor on block proof-of-work weight
<code>MIN_TX_WEIGHT</code>	14	floor on transaction (anti-spam) weight
<code>MIN_TX_WEIGHT_COEFFICIENT</code>	1.6	size term in the min-tx-weight formula
<code>MIN_TX_WEIGHT_K</code>	100	amount term in the min-tx-weight formula
<code>DECIMAL_PLACES</code>	2	smallest unit = 1/100 of a token
<code>BLOCKS_PER_HALVING</code>	1,051,200	≈ every 365 days; the reward-halving schedule
<code>HATHOR_TOKEN_UID</code>	<code>b'\x00'</code>	the native-token marker — a single null byte, not a 32-byte hash (Ch 7)
<code>MAX_DISTANCE_BETWEEN_BLOCKS</code>	4,500 s	150×30 ; weight-decay trigger when blocks stall (Ch 9)

These are read everywhere through `get_global_settings()` and never change for the life of the process — the single immutable source of truth for "what network am I on." (Chapter 22.)

Recap

Cheat sheet	Chapter	Source of truth
RocksDB column families	27	<code>transaction/storage/rocksdb_storage.py:46</code>
P2P messages	34, 35, 39	<code>p2p/messages.py:45</code>
CLI commands	21	<code>hathor_cli/main.py</code>
Sysctl paths	41	<code>hathor/sysctl/</code>
Feature states	38	<code>feature_activation/model/feature_state.py</code>
Handshake states	34	<code>p2p/states/</code>
Network constants	22, 9	<code>hathorlib/conf/settings.py</code>

These tables are the residue of the whole book — the facts that remain once the explanations have done their work and become second nature. With the three appendices in place — the **glossary** for every term, the **dependency manifest** for every package, and these **cheat sheets** for every recurring table — the master-book is complete: a junior reader can now go from the first "what is Hathor?" in Chapter 0 to reading any corner of the running node, with a reference for anything they forget along the way.

1. A column family in RocksDB is a named, independent key-value namespace within a single database — like separate tables sharing one store. Each can be configured and accessed on its own, while still committing atomically with the others. ↩